SAVED ON LINKEDIN

LinkedIn Saved Posts — last 6 months

Generated Oct 22, 2026

673
posts, newest first

Topics (colour key)

- AI / LLM (178)
- Java / Spring (138)
- Kafka / Messaging (30)
- Cloud / K8s / DevOps (27)
- System Design (234)
- Other (66)

Ordered newest first by LinkedIn's relative date (as of collection). 18 duplicate entries merged from 691 collected. Emoji and other unrenderable symbols were removed or replaced for clean rendering; stylized Unicode bold/italic text was converted to plain letters. Topic tags are assigned automatically from keywords.

Index

AI / LLM (178) Java / Spring (138) Kafka / Messaging (30) Cloud / K8s / DevOps (27) System Design (234) Other (66)

1 **Shaurya Pratap Singh** · 5h — Want to get better at System Design? ... 15
2 **Design Gurus** · 1d — Your application is growing. Your database is the bottleneck. You have... ... 16
3 **Ankur Dhawan** · 1d — Stripe offers ₹60–80 LPA for SDE-1 and ₹1 Cr+ for SDE-2, ... 17
4 **Joel Teodoro Gómez** · 1d — 10 GB file. 512 MB of RAM. Load it into a database. ... 18
5 **Suresh G.** · 1d — Top 12 GitHub repos every software engineer should bookmark to learn AI e... ... 19
6 **Arslan Ahmad** · 2d — Same feature. Two engineers. Very different solutions. ... 20
7 **Neo Kim** · 2d — If you want to become god-level at system design ... 21
8 **Pragya Verma** · 3d — ## Payment System Design | High-Level Architecture ... 22
9 **Mahmood Salah** · 3d — AI engineer technical interview in the next 24 hours, practice these... ... 23
10 **Swadesh Kumar** · 3d — Most people learn System Design by memorising architectures. ... 24
11 **Sampath Kasoji** · 4d — System Design interviews become much easier when you understand how... ... 26
12 **Neo Kim** · 4d — If you want to get dangerously good at generative AI, ... 27
13 **Ranjan Kumar** · 4d — Netflix Interview Questions That Will Make Your Concepts Strong! ... 28
14 **Raul Junco** · 4d — I thought Cache was only about storing my data for quick access, and I w... ... 29
15 **Puneet Patwari** · 5d — I'm a Principal Engineer at Atlassian with 13 years of experience. I... ... 30
16 **Chandrasekar Srinivasan** · 6d — The best AI engineering material I read this year was free.... ... 31
17 **Arslan Ahmad** · 1w — Most teams scale their database in the wrong order. ... 32
18 **Mudassir Maredia** · 1w — Caching improves performance. Until it becomes your consistency pr... ... 33
19 **Ankur Dhawan** · 1w — I cracked Expedia Group in my first attempt! ... 34
20 **Adnan Khan** · 1w — 4 Rounds. ₹95 LPA–₹1.3 Crore. The Atlassian Interview Breakdown! ... 35
21 **Nelson Djalo** · 1w — A URL shortener looks easy ... 36
22 **Aashish Gupta** · 1w — Microservices aren't just about breaking a monolith into smaller serv... ... 36
23 **Crispus Roshan** · 1w — I interviewed at Airbnb in 2025 for a Data Engineering role. ... 37
24 **Naushad A.** · 1w — #Barclays Interview Experience | Senior Java Developer↓ ... 38
25 **Ravi Kant** · 1w — What happens when the bank processes a payment, but our service times out... ... 39
26 **Neo Kim** · 1w — If you want to become S-tier with AI agents, ... 39
27 **Ranjan Kumar** · 1w — JPMorgan Chase Interview Questions | DSA + SQL + Web/Frameworks + Syst... ... 40
28 **Sumit Goyal** · 1w — Sapient Interview: 52% hike ... 41
29 **Dinesh Solanki** · 1w — "Sorry, we made you grind LeetCode." ... 42
30 **Alamin Hossain** · 1w — NEVER UPLOAD YOUR CV INTO CHATGPT AND ASK, "CAN YOU FIX MY CV?" ... 43
31 **Harish kumar** · 1w — Stanford's full LLM lecture series is free. ... 44
32 **Sampath Kasoji** · 1w — I came across a Uber L5A Senior Software Engineer interview experien... ... 45
33 **Hina Arora** · 1w — I run a team of 12. None of them are human. ... 47
34 **Shubham Kamboj** · 1w — #Barclays Asked Interview Questions in Technical Round (October, 202... ... 48
35 **Shubham Kamboj** · 1w — #MasterCard Asked Interview Questions in Technical Round (September,... ... 49
36 **Neo Kim** · 1w — If you want to get GOD-LEVEL with AI agents, ... 50
37 **Alok Sharan** · 1w — System Design Mastery: 6 Core Pillars Explained! ... 51
38 **Arslan Ahmad** · 1w — A slow query does not always need a bigger database. ... 52
39 **Ankur Dhawan** · 1w — Atlassian pays around 50-70LPA for SDE-1 and 60-90LPA for SDE-2 ... 53
40 **Akshat Jain** · 1w — Resources I'm Following for System Design (Both LLD and HLD) ↓ ... 54
41 **Rohit Turambekar** · 2w — Most engineers learn system design by memorizing architectures. ... 55
42 **Yash Verma** · 2w — Spotify System Design Explained ... 55
43 **Saurabh Jadhav** · 2w — System Design: How do you scale from 0 → 1 Million Users? ... 56
44 **Anurag Rana** · 2w — Shopify recently replaced Redis with MySQL for their inventory reservat... ... 58
45 **Jonish Prajapati** · 2w — JUnit vs Mockito — same goal, different responsibilities. ... 58
46 **Shubham Shete** · 2w — Apache Kafka Design Patterns — Interview Preparation Guide. ... 59
47 **Harish kumar** · 2w — NVIDIA just released 10 AI courses for FREE — with certificates. ... 60
48 **Raul Junco** · 2w — Push-based systems come up in 90% of system design interviews. ... 61

49 **Naushad A.** · 2w — #PWC Interview Experience | Java Consultant ↓ ... 61
50 **Neo Kim** · 2w — If I had to start with SYSTEM DESIGN,... 62
51 **Vinay Tiwari** · 2w — HASHMAP vs LINKEDHASHMAP vs TREEMAP... 63
52 **Sathish Kumar Subramani** · 2w — You can draw a perfect architecture diagram and still fail... ... 64
53 **Harshit Kumar** · 2w — I've created a System Design preparation roadmap covering core concep... ... 66
54 **Neo Kim** · 2w — If you want to get GOAT-level with system design,... 66
55 **Serge Elman** · 2w — Payments Terminology of the Day: Token Provisioning ... 67
56 **Anup Panwar** · 2w — Tokenized deposits and stablecoins may look similar, but the underlying... ... 68
57 **Basia Kubicka** · 2w — Google's 31-day AI Agents Challenge starts today.... 69
58 **Adnan Khan** · 2w — Google Interview Experience – SDE-3 Backend... 70
59 **Umesh Palai** · 2w — REST API vs WebSocket in Algorithmic Trading — What Actually Happens Un... ... 71
60 **Sandhya Sahani** · 3w — System Design Interview Preparation ... 72
61 **Saiful Khan** · 3w — Same cloud. Different names.... 73
62 **Sajeev KR** · 3w — Java & Spring Boot–Real-World Scenario-Based Interview Questions... 73
63 **Petar Ivanov** · 3w — 8 load balancing algorithms you should know: ... 74
64 **Amit Mishra** · 3w — System Design | Real-Time Trading Platform... 75
65 **Dhiral Kaniya** · 3w — Your database probably isn't struggling because of writes.... 76
66 **Tauqeer Ahmed** · 3w — A correlation ID can tell you which request failed. It cannot tell yo... ... 76
67 **Raul Junco** · 3w — Every architecture diagram makes mathematical promises.... 77
68 **Raul Junco** · 1mo — 80% of System Design Interviews Test the Same 20% of the concepts. ... 77
69 **Alex Xu** · 3w — Watch an ex-YouTube engineer rebuild YouTube in 45 minutes. ... 79
70 **Java Developers Community (moderated)** · 3w — Best thing that ever happened to java, the on... ... 79
71 **Swapnil Dhamal** · 3w — JAVA | COLLECTIONS / SCALABILITY | QUICK 2-MIN READ... 80
72 **Lokanadham Ganta** · 3w — System Design Fridays | Episode 1... 81
73 **Basia Kubicka** · 3w — Most people dismiss a harness as just scaffolding... 83
74 **Ankur Dhawan** · 3w — I cracked Atlassian in my first attempt!... 83
75 **Serge Elman** · 3w — Payments Terminology of the Day: TLCM — Token Life Cycle Management ... 84
76 **Naushad A.** · 3w — #HSBC Interview Experience | Senior Consultant↓... 85
77 **Sameer Bhardwaj** · 3w — I spent 8+ hours collecting resources so you can become an AI agent... ... 86
78 **Akash Namdev** · 4w — 11 Essential System Design Patterns... 87
79 **Hisham Mosajo** · 4w — A common Java interview question is: What changed from Java 7 to Java... ... 88
80 **Alex Li** · 4w — The Java 8 Stream API provides a powerful, functional way to process collec... ... 89
81 **Naushad A.** · 4w — #Virtusa Interview Experience for #Citi client | Java | Senior Consultan... ... 89
82 **Mahesh Mallikarjunaiah ↗** · 4w — What actually happens inside an LLM when you send a prompt. ... 90
83 **Prem N.** · 4w — Not all AI processors are built for the same job.... 91
84 **Yogesh Kshirsagar** · 4w — Spring Boot Under the Hood — Vol. 21 of 30... 92
85 **Alex Xu** · 4w — Jev is TypeSafe AI's first System One Model. It is 100x faster and cheaper... ... 93
86 **Vishal Kumar** · 4w — Recently, I was learning about Microservices and API Architecture and... ... 94
87 **Naushad A.** · 4w — #Fulcrum Interview Experience for #Mastercard client for my Friend | Jav... ... 94
88 **Neo Kim** · 4w — How to design APIs like a Senior Engineer... 95
89 **Nageswararao Dongari** · 1mo — Functional vs Non-Functional Requirements in System Design ... 95
90 **Sairam Sundaresan** · 1mo — You can cut token use by 49% and API costs by 33% without changin... ... 96
91 **Yogesh Kshirsagar** · 1mo — Spring Boot Under The Hood - 20 of 30... 96
92 **Rocky Bhatia** · 1mo — Your database is not slow.... 97
93 **NIPIN G.** · 1mo — Java's ReentrantLock is like a car key?... 98
94 **Ankit Pangasa** · 1mo — How Databases Actually Work ... 99
95 **Sathish Kumar Subramani** · 1mo — Your AI app is only as good as the database beneath it.... 100
96 **Ankit Pangasa** · 1mo — Monitoring vs Observability: They Are NOT the Same ... 101
97 **Neo Kim** · 1mo — If you want to become a 10x software engineer,... 103
98 **Rocard Fonji F.** · 1mo — FINTECH & REMITTANCE SYSTEMS (3/3) ... 104
99 **Suraj Mishra** · 1mo — Spring Boot Interview Question – Improve API Latency ... 104
100 **Sameer Bhardwaj** · 1mo — If I travel back to my Interview Days, I will study these 35 Engine... ... 105
101 **Java Developers Community (moderated)** · 1mo — Most engineers leave load balancers on round... ... 107
102 **Manochitra Selvam** · 1mo — Java Interview Series #3 — Idempotency in Payment Processing... 108
103 **Rocky Bhatia** · 1mo — What really happens when you hit Send on WhatsApp? ... 108

104 **Manu Agrawal** · 1mo — Give me 2 minutes, and I'll show you how to use ChatGPT for Google-lev... 109
105 **Raul Junco** · 1mo — 99.9% of backend interviews will ask you about APIs. 110
106 **Samson O. Sanyaolu** · 1mo — 9 Distributed Systems Patterns You Should Know 111
107 **Sajeev KR** · 1mo — Spring Security, OAuth2 & JWT Interview Questions – Top 25 112
108 **Sunke Nagarani** · 1mo — 25 Kubernetes Components Every DevOps Engineer Should Know 113
109 **Shalini Goyal** · 1mo — The System Design Iceberg: What You See vs. What You Miss 113
110 **Manu Agrawal** · 1mo — Give me 2 minutes, and I'll show you how to use Claude to crack Google... 114
111 **Raul Junco** · 1mo — Improving the Cache Hit Ratio is the way to cut latency. 116
112 **Ajeet Singh** · 1mo — We're building end-to-end tech teams for top BFSI clients across the UA... 116
113 **MD SHABAB ALI** · 1mo — Microservices don't automatically make your system scalable. 117
114 **Yogesh Kumbhare** · 1mo — The API response time suddenly increases from 200ms to 8 seconds in... 117
115 **Anton Martyniuk** · 1mo — I paid $1000 to learn microservices. 118
116 **Alok Sharan** · 1mo — A system that works for 100 users can fall apart at 100,000. 119
117 **Harshad Bhatt** · 1mo — Most engineers preparing for Visa make the same mistake: they grind 4... 120
118 **Muhammad Usman Arif** · 1mo — Software Architecture vs System Architecture, What's the Real D... 121
119 **Pitchayut Roejanapreecha** · 1mo — Enterprise Architect vs Solution Architect vs System Archi... 122
120 **Mansi Kaushik** · 1mo — After working on microservices for several years, one thing I've real... 123
121 **Rocky Bhatia** · 1mo — Choosing the right database isn't about the tool, 123
122 **Aiswarya Venkitesh** · 1mo — The software lifecycle is learning to think. 124
123 **Eslam Aly** · 1mo — I have created This visual outlines how modern microservice systems work:... 125
124 **Neo Kim** · 1mo — If I had to start with system design, 125
125 **Palash Bharadwaj** · 1mo — Most teams don't need more agents. 127
126 **Harun Sulaiman** · 1mo — If I had 6 months to become an AI Infrastructure Engineer. 127
127 **RAMESHKUMAR S** · 1mo — Asynchronous Alert Processing: Incoming notification requests are ins... 128
128 **KHYATI PATEL** · 1mo — Java Multithreading — Day 10: volatile vs synchronized 128
129 **Alex Xu** · 1mo — Agent Harness, Explained Visually 131
130 **Akash Namdev** · 1mo — Deep Diving into Apache Kafka — Part 2 131
131 **Dhivya Sri Lingala** · 1mo — Recently, I had the opportunity to interview for a Software Engi... 132
132 **Nikki Siapno** · 3mo — 8 Deployment Patterns Developers Should Know 133
133 **Durgesh Choubey** · 1mo — Want to become a Claude Certified Architect? 134
134 **Ratnakar Sahoo** · 1mo — Most people are still using Classic RAG for AI applications. 135
135 **Big Data Data Science | Machine Lea...** · 1mo — Learning faster is about using the right ment... 136
136 **Clouding Academy** · 1mo — THE MOST RELEVANT AI CERTIFICATIONS IN 2026 136
137 **Anton Martyniuk** · 1mo — You learn Design Patterns the wrong way 137
138 **Rocky Bhatia** · 1mo — Your API works perfectly - until someone hammers it with 10,000 reques... 139
139 **Nikki Siapno** · 4mo — Postgres vs MySQL 139
140 **Satyanarayan Maheshwari** · 1mo — The API already told you what's wrong. 140
141 **DevOps Professionals CI/CD Docker K...** · 1mo — If your Pods can't find each other, your appl... 142
142 **Charly Wargnier** · 1mo — SPOTIFY JUST OPEN-SOURCED A CLAUDE CODE SETUP THAT CUTS TOKEN USAGE... 143
143 **Snahasis Barat** · 1mo — "@Transactional" in Spring Boot looks simple. 143
144 **Amar Bhise** · 1mo — If you're learning Spring Boot, don't just memorize annotations. Underst... 144
145 **Ganesh P** · 1mo — After ~3 years of working with Spring Boot, I realized something. 145
146 **Tejal Rathod** · 1mo — Kafka acks=0 vs acks=1 vs acks=all — When Should You Use Which? 147
147 **Sireesha D** · 1mo — A Spring Boot application has a cron/@Scheduled job and is deployed on t... 148
148 **DevOps Professionals CI/CD Docker K...** · 1mo — CrashLoopBackOff is NOT a Kubernetes Pod phase. 149
149 **Pujitha Bandla** · 1mo — Java development in 2026 is about much more than just writing Java c... 150
150 **DevOps Professionals CI/CD Docker K...** · 1mo — Kubernetes Namespaces: The Secret to Keeping... 151
151 **Shreekrushanaprasad Mane** · 1mo — Everyone adds Redis for caching. 151
152 **Rahuul Gaarg** · 1mo — Why Real-World Architectures Are the Best Way to Learn System Design 152
153 **Kubernetes** · 1mo — Cloud-Native MLOps with Kubernetes 154
154 **Bhanu Prasad T.** · 1mo — Java Interview Topic: Why Java Records Matter 155
155 **Java Developers Community (moderated)** · 1mo — Hibernate & JPA can be confusing... until you... 156
156 **Sweety Sharma** · 1mo — Java Concurrency & Multithreading - Top 40 Interview Questions 156
157 **Srinjoy Ghosh** · 1mo — 3 RAG Architectures. One word separates them: 158
158 **Subhashree Mohan Swain** · 1mo — One caching question in an interview made me think beyond ju... 158

159 **Anshika Sinha** · 1mo — If you're targeting Senior SDE roles, be ready for this question: ... 160
160 **Mohammad F.** · 1mo — Google is making their whole interview process AI based? ... 161
161 **Sajeev KR** · 1mo — Spring Boot Production Issues – 30 Real Scenarios ... 161
162 **Code Vimaan** · 1mo — 30 DAYS OF JAVA — DAY 20 ... 163
163 **Code Vimaan** · 1mo — 30 DAYS OF JAVA — DAY 21 ... 164
164 **Alok Sharan** · 1mo — The tools change every few years. ... 165
165 **Spring Users** · 1mo — Senior Spring Boot Interview Question: Debug a Concurrent Subscription... ... 166
166 **Samson O. Sanyaolu** · 1mo — Latency vs Throughput vs Bandwidth ... 168
167 **Java Developers Community (moderated)** · 1mo — Do you think you're ready for a Java 8 Stream... ... 168
168 **Yann Pascal DEUNGOUE** · 1mo — Calling an LLM from Java is easy. Making AI a real part of you... ... 169
169 **Kanishk Raj** · 1mo — While going through some basics of designing scalable applications, I c... ... 170
170 **Shankar Thakur** · 1mo — Java 8 → 17 → 21 → 25 ... 171
171 **Vinay Tiwari** · 1mo — JAVA 8 → 11 → 17 → 21 → 25 ... 171
172 **Raul Junco** · 1mo — Building distributed systems taught me one thing the hard way: ... 172
173 **Pradeep Pai** · 1mo — Shopify replaced Redis with MySQL for one of their most critical checko... ... 173
174 **Manu Agrawal** · 1mo — Give me 2 minutes and I'll show you how to use Claude to master System... ... 174
175 **Gaurav Gupta** · 1mo — Java 8 vs Java 17 vs Java 21 vs Java 25: The Journey of Modern Java ... 175
176 **Dnyaneshwar Ghodake** · 1mo — AtomicInteger vs synchronized — What should you use? ... 176
177 **Java Developers Community (moderated)** · 1mo — Thanks for Java ... 177
178 **Shalini Goyal** · 1mo — Your app works. ... 177
179 **Mithra Gopal** · 1mo — PAYMENTS ENGINEERING, EXPLAINED · #02 ... 178
180 **Muhammad Mubbashir** · 1mo — In fintech, processing a payment is easy. Processing it exactly. ... 179
181 **Spring Users** · 1mo — (no text) ... 179
182 **Yogesh Kumbhare** · 1mo — Payment Gateway Design ... 180
183 **Hassan Zaib Hayat** · 1mo — A 10 GB upload should not be one HTTP request. ... 180
184 **Java Developers Community (moderated)** · 1mo — You think your Kafka publish is safe inside @... ... 182
185 **Manoj Vurukonda** · 1mo — Why 'synchronized' keyword cannot prevent duplicate payments? How p... ... 182
186 **Dr. Sandeep Srivastava** · 1mo — Still confused about how to validate your startup idea? ... 183
187 **Shubham Sharma** · 1mo — If you want to Become Good at System Design? Start by understanding... ... 183
188 **Deepika Sidhanandan** · 1mo — Your "@Transactional" is NOT broken. Your proxy is. ... 185
189 **Neo Kim** · 1mo — If I had to design a system, ... 187
190 **shujat khan** · 1mo — Yesterday I appeared for an interview with Accenture for a Java Develop... ... 188
191 **Aashish Gupta** · 1mo — Everyone treats URL Shortener as a warm-up interview question. ... 189
192 **Mohammad Iliyas Bagali** · 1mo — A method can have @Transactional and still run without the t... ... 190
193 **Yogesh Kshirsagar** · 1mo — Spring Boot Under the Hood- Vol. 16 of 30 ... 191
194 **Yogesh Kshirsagar** · 1mo — Spring Boot Under the Hood- Vol. 15 of 30 ... 193
195 **Big Data Data Science | Machine Lea...** · 1mo — the discipline point is the real story, most... ... 194
196 **Rocky Bhatia** · 1mo — You might think "caching" = Redis. ... 195
197 **Alex Xu** · 6mo — How the JVM Works ... 196
198 **Sai Ram Somanaboina** · 1mo — If you want to land a software engineering job in 2026, bookmar... ... 196
199 **Raja Anand** · 1mo — Are you a Java developer? ... 198
200 **Sudhir Dhangar** · 1mo — Some engineers prepare for System Design interviews by memorizing ar... ... 198
201 **Kumari Mahima** · 1mo — Most system designs don't start with Kafka, Redis, or 10 microservices. ... 200
202 **Neo Kim** · 1mo — If you want to become good at system design, ... 202
203 **Pradeep Kumar Saini** · 1mo — Some engineers prepare for System Design interviews by memorizi... ... 203
204 **Vibhav Gupta** · 1mo — "@Transactional" in Spring Boot: The Annotation Is Easy. Understanding... ... 204
205 **Jainath Jaiswal** · 1mo — Idempotency vs Deduplication — Two Concepts Every Microservices Eng... ... 205
206 **Glynes D'Souza** · 1mo — Spring Boot quietly eliminated an entire category of boilerplate cod... ... 206
207 **Mukesh Dhale** · 1mo — I've been working with Java and Spring Boot, and one topic that keeps... ... 206
208 **Shameer Ahamad** · 1mo — How HashMap Works Internally in Java ... 207
209 **Parikh Jain** · 1mo — Trust me, cracking offers is not hard if you follow this ↓ ... 207
210 **Ujjwal Sontakke** · 1mo — System Design becomes much easier when you stop memorizing componen... ... 208
211 **Sudhir Dhangar** · 1mo — Which System Design resource should I follow? ... 209
212 **Vishal Singh** · 1mo — If I had 6 months to master System Design. ... 212
213 **Java Developers Community (moderated)** · 1mo — (!) This Java statement looks thread-safe: ... 213

214 **nishant lakhara** · 1mo — Java Concurrency — CAS (Compare-And-Swap) 214
215 **nishant lakhara** · 1mo — Java Concurrency | Why does HashMap allow null, but ConcurrentHashM... 216
216 **Anuj Sharma** · 1mo — Apache Kafka — The Backbone of Real-Time Data Streaming 217
217 **Neo Kim** · 1mo — If you want to get god-level with coding, ... 218
218 **Virendra Beniwal** · 1mo — Load Balancer ≠ NGINX. Here's the difference. 219
219 **Puneet Patwari** · 1mo — I've cleared system design rounds at Amazon, Atlassian, Salesforce,... 219
220 **Rocky Bhatia** · 1mo — If your system keeps breaking as you scale... it's not your code, it's... 221
221 **Sahilkumar Prasad** · 1mo — More application threads can improve throughput. 222
222 **Parikh Jain** · 1mo — If you are targeting Google or Uber: 223
223 **Java Developers Community (moderated)** · 1mo — Java Streams — The Mental Model 224
224 **Kubernetes** · 1mo — "Autoscaling Cannot Fix Inefficient Architecture." 225
225 **Rakesh Gohel** · 1mo — Anthropic just launched Claude Academy, a free platform for using and... 226
226 **Deepak Kumar Kesharwani** · 2mo — Just came out of my Karat interview — and honestly, it was... 227
227 **Kubernetes** · 2mo — "AI Cost Is Becoming a Platform Engineering Problem." 228
228 **Neo Kim** · 2mo — If you want to become GOAT-level with system design, ... 230
229 **Yash Vardhaan** · 2mo — 10:00 AM. Tatkal opens. Millions of users hit "Book Now." 231
230 **DevOps Professionals CI/CD Docker K...** · 2mo — DNS is invisible... until it breaks. 232
231 **Sahilkumar Prasad** · 2mo — More threads don't always mean more performance. 232
232 **Nafiul Islam** · 2mo — What Actually Happens When 10,000 Users Hit Your Spring Boot API? 233
233 **Dnyaneshwar Ghodake** · 2mo — How do you make a Java API handle 10K+ requests per second? 234
234 **Himanshu Sharma** · 2mo — How Spring Automatically Finds Implementations 235
235 **Nishant Ojha** · 2mo — Spring Boot Day 6 – Key Learnings 236
236 **Waseem Shaikh** · 2mo — I've put together a collection of Java & SQL interview coding problem... 237
237 **Yogesh Kshirsagar** · 2mo — Spring Boot Under the Hood - Vol. 11 of 30 238
238 **Java Developers Community (moderated)** · 2mo — Honestly one of the best resource , thanks an... 239
239 **Maxim Konstantinov** · 2mo — Spring Boot 4 quietly deleted a whole category of code most of u... 240
240 **Saleamlak Asrat Shiferie** · 2mo — One of the best System Design writers on LinkedIn is Neo K... 240
241 **Aman Mishra** · 2mo — "Design a URL Shortener." 241
242 **Woongsik Dr. Su, MBA** · 2mo — Preparing for a System Design Interview? 243
243 **Prachi Jain** · 2mo — System Design is not about drawing boxes. 244
244 **Achal Soni** · 2mo — Learn Easy System Design on One Page 245
245 **Petar Ivanov** · 2mo — 7 message queue patterns every backend engineer should know: 245
246 **Alex Xu** · 2mo — Top 12 Agent Skills You Should Know 246
247 **Parikh Jain** · 2mo — Zero-to-hired DSA curriculum (All free resources) 247
248 **Ali Raza** · 2mo — Can you answer these 10 practical backend questions? 248
249 **Satendra Rajput** · 2mo — Mastercard – 60 Senior Software Engineer Interview Questions 249
250 **Jeet Chatterjee** · 2mo — Kafka Duplicate Events in Production 250
251 **Puja Agrawalla** · 2mo — Kafka interviews aren't just about Kafka-they're about what happens. 251
252 **Aman Mishra** · 2mo — "We use Redis for caching." 251
253 **Sonam Yadav** · 2mo — Microservices Interview Preparation – Part 1 253
254 **Neha Jain** · 2mo — "DSA isn't about solving 500+ problems — it's about solving the right 25." 253
255 **Ankita Sinha** · 2mo — Apache Kafka Architecture — Simplified 255
256 **Deshdeepak Rajpoot** · 2mo — Choosing the right system design pattern isn't about knowing eve... 256
257 **DevOps Professionals CI/CD Docker K...** · 2mo — Most Kubernetes YAML looks complicated. 256
258 **Java Developers Community (moderated)** · 2mo — Building Enterprise AI Applications with Spri... 257
259 **Ashwani Gupta** · 2mo — Most system design diagrams look complicated. 257
260 **AYAN HASNAIN** · 2mo — I used to think System Design was about memorizing big architecture di... 258
261 **Aman Mishra** · 2mo — 5,000 people are trying to book the same Avengers seat on BookMyShow. 259
262 **Swetha Telu** · 2mo — 30 Days of System Design for Java Developers – Day 4/30 260
263 **Anand Gunasekaran** · 2mo — We broke our Java monolith into 47 microservices. 262
264 **Chandrakant Kumar** · 2mo — Recently faced one of those incidents that reminds you why distri... 263
265 **Pradeep Singh** · 2mo — Redis Cache with Spring Boot Microservices — From Basics to Production 264
266 **Aman Mishra** · 2mo — You have been doing System Design in Spring Boot every day. 264
267 **Imran Nur** · 2mo — Building resilient backend architectures requires mastering core API prin... 266
268 **Javali Bathula** · 2mo — I asked a Spring Boot developer what @SpringBootApplication actually... 266

269 **Ankit Pangasa** · 2mo — 20 API Concepts Every Software Engineer Should Know ... 267
270 **Vibhu Agarwal** · 2mo — Microservices Design Patterns Cheat Sheet ... 269
271 **Lokesh Alwani** · 2mo — MCP vs API – Understanding the Difference ... 269
272 **Nuzaim Noushad Thappi** · 2mo — "Exactly-once" message processing in distributed systems does... ... 270
273 **Raul Junco** · 2mo — A distributed system can be perfectly consistent... and still completely... ... 271
274 **Nick Gejadze** · 2mo — Microservices sound great in theory. ... 271
275 **Rani Dhage** · 2mo — I thought I understood HashMap... ... 272
276 **Neo Kim** · 2mo — If you want to get started with system design, ... 273
277 **Mansi Kaushik** · 2mo — Inside Paytm's System Design: A Public Architecture Deep Dive ... 275
278 **Siddharth Kanojia** · 2mo — Load Balancer vs NGINX (Reverse Proxy) | What's the Difference? ... 276
279 **Raul Junco** · 2mo — I'm always learning from history, and one of my favorite teachers is Fey... ... 277
280 **Nikki Siapno** · 2mo — 8 RAG Architectures Developers Should Know ... 278
281 **Neo Kim** · 2mo — I built a GitHub repo to help you learn system design. ... 279
282 **Shalini Goyal** · 2mo — 12 System Design Patterns Simplified ... 279
283 **Aritri Mukherjee** · 2mo — 5 Spring Boot Mistakes I Stopped Making After 4 Years as a Java De... ... 280
284 **Neo Kim** · 2mo — Most people think Redis is just a cache. ... 281
285 **Harjeet Singh** · 2mo — Building a Microservices Architecture is Easy. Building a Team That C... ... 282
286 **Niladri Chatterjee** · 2mo — We spent years making payments faster. Now we need to make them... ... 283
287 **Saptarshi Dey** · 2mo — Every payment looks simple to the customer. Behind the scenes, it's a... ... 284
288 **Deepak Kumar** · 2mo — Java + AI: The Next Evolution of Enterprise Banking ... 284
289 **Asha Shanmugam** · 2mo — What is Transaction Monitoring? ... 285
290 **Rishi Jha** · 2mo — How Banks Detect Card Fraud in Under 100 ms ... 286
291 **Adarsh C.** · 2mo — I spent the last few days turning Networking into a visual cheat sheet. ... 288
292 **Muhammad Nouman** · 2mo — 15 Must-Know Networking Concepts Every Developer Should Know ... 290
293 **Ashish Pratap Singh** · 2mo — If you want to get good at computer networks, learn these 15 co... ... 290
294 **Pravin Chalak** · 2mo — From Monoliths to Microservices to AI-Governed Architectures: The Nex... ... 291
295 **Arpit Aggarwal** · 2mo — Tokenization: The Invisible Technology That Makes Digital Payments S... ... 292
296 **Sivasankar Natarajan** · 2mo — Everyone is talking about how powerful AI is. But very few are... ... 292
297 **B Rajasekhar Reddy** · 2mo — Spring Boot Explained | Part 1 ... 293
298 **Aashi Gangrade** · 2mo — If someone asked you to explain the difference between Kafka and Rab... ... 294
299 **Imran Shaikh** · 2mo — How Spring Boot Works Internally – Complete Architecture Explained! ... 295
300 **Spring Users** · 2mo — Spring Boot Interview Quiz : Why Is CPU Low but Latency High? ... 296
301 **Sathish Kumar Subramani** · 2mo — Your app isn't slow because of your code. It's slow because... ... 296
302 **Neo Kim** · 2mo — If you want to become top-tier with AI engineering, ... 297
303 **Java Developers Community (moderated)** · 2mo — Low Latency Practices are essential ... 299
304 **Java Developers Community (moderated)** · 2mo — Circuit Breaker in Spring Boot ... 300
305 **Arun Goel** · 2mo — I've prepared for LLD interviews across Uber, Facebook, Google, Amazon, M... ... 301
306 **Neo Kim** · 2mo — If you want to become god-level with system design, ... 302
307 **Java Developers Community (moderated)** · 2mo — If you want to become top-tier at API design, ... 303
308 **Arvind Kumar** · 2mo — If you're preparing for System Design interviews, focus less on memori... ... 305
309 **Arslan Ahmad** · 2mo — Most system design interviews reward you for explaining how a system w... ... 306
310 **Chandra Sekhar** · 2mo — Accenture asked this in L3 round of Senior AI Engineer: ... 307
311 **Parikh Jain** · 2mo — Your first CTC is not a paycheck. It is the number every future recruit... ... 307
312 **Gaurav Nikumbh** · 2mo — Tokenization Explained - why replacing a 16-digit number changes eve... ... 308
313 **Neo Kim** · 2mo — If you want to become world-class with AI agents, ... 309
314 **Ashish Pratap Singh** · 2mo — Kafka is NOT just a messaging system. Here are the top 10 use c... ... 310
315 **Manan Solanki** · 3mo — While interviewing across top PBC like Uber , Flipkart , Rippling 311
316 **Kriti Rohilla** · 3mo — I interviewed at Goldman Sachs, Coinbase, and now Microsoft. ... 312
317 **Ashish Pratap Singh** · 3mo — If you want to get better at designing large scale systems, und... ... 312
318 **Krutam Hathi** · 3mo — After switching from Microsoft → Cohesity and now Weave, one interview... ... 313
319 **Aashish Gupta** · 3mo — One thing that significantly improves your chances of getting selecte... ... 314
320 **Neo Kim** · 3mo — If you want to become S-tier with API design, ... 315
321 **Sivasankar Natarajan** · 3mo — When Did "AI Engineer" Go From Calling One API to Orchestratin... ... 317
322 **Spring Users** · 3mo — thank you for sharing! ... 318
323 **Parikh Jain** · 3mo — Amazon loves: Heaps, Graph BFS, Intervals ... 318

324 **Ankit Pangasa** · 3mo — "The customer clicked 'Pay' three times. How do you ensure they're ch... 319
325 **Venkatesh Patnala** · 3mo — We added one annotation. 320
326 **Raja Anand** · 3mo — Spring Boot - @Cacheable & @CacheEvict 321
327 **Niladri Chatterjee** · 3mo — Designing Low-Latency Payment Systems: It's More Than Just Speed 321
328 **Rakesh kumar** · 3mo — Every expert was once a beginner. 322
329 **Big Data Data Science | Machine Lea...** · 3mo — Everyone is building RAG systems. 323
330 **Priyam Jain** · 3mo — One PDF That Made DSA Revision Much Easier 323
331 **Suchita Chaturvedi** · 3mo — Ever wondered what happens behind the scenes when millions of In... 324
332 **Antrixsh Gupta** · 3mo — Most teams pick a vector DB in the first hour of a RAG project. 325
333 **AlgoMaster.io** · 3mo — If you want to master Array problems for interviews, learn these 10 p... 326
334 **Navya Shrri Bommu** · 3mo — Kafka looks complicated until the core pieces click. 326
335 **Shubham Srivastava** · 3mo — Imagine running a library where 50 people are reading the exact... 327
336 **Aashish Gupta** · 3mo — Distributed Job Scheduler is one of the most common interview questio... 328
337 **Gourav A.** · 3mo — Optimistic Locking vs. Pessimistic Locking — How SQL Databases Handle Con... 329
338 **Puneet Patwari** · 3mo — It's a Wrap! Here are all the links to 25 important system design co... 330
339 **Marcos Vinicius Marcon** · 3mo — Java 26 introduces a cleaner and safer way to propagate cont... 331
340 **Jay kumar** · 3mo — 3 Powerful Problem-Solving Patterns Every Programmer Should Know 332
341 **Thiago Fernandes** · 3mo — Java Streams are a powerful way to write cleaner, more readable, a... 332
342 **AlgoMaster.io** · 3mo — 10 Must-Know Time Complexity Patterns for Interviews: 332
343 **Shubham Adik** · 3mo — ISO 8583 Transaction Lifecycle – What Really Happens After You Tap You... 333
344 **Suneel Kumar Manepalli** · 3mo — How Modern AI Systems Actually Work 334
345 **RAMAKRISHNA NOOLU** · 3mo — Understanding Payment Switch Architecture — Simplified for Everyone 335
346 **Gaurav Nikumbh** · 3mo — Payment Gateway vs Payment Switch - most people in payments use thes... 336
347 **Alok Sharan** · 3mo — RAG looks simple when shown as one flow: 337
348 **Chirag Jain** · 3mo — Most Java developers use HashMap every day, but how many actually know... 337
349 **Apoorv Kapsime** · 3mo — 10 Java Stream API Methods Every Java Developer Should Know (and Whe... 338
350 **Akhila G** · 3mo — RAG Interview Question — PolicyBazaar (AI Engineer) 340
351 **Java Developers Community (moderated)** · 3mo — Our Java service was running inside a 2-core... 340
352 **Alexandre Reis** · 3mo — Most Java devs know Stream.collect(). Far fewer use Gatherer. 342
353 **Chandrasekhar Y.** · 3mo — Two Microservices Update the Same Record at the Same Time... What... 342
354 **Mayur Gaikwad** · 3mo — Offset pagination doesn't become a scalability problem because it's s... 343
355 **Alex Xu** · 3mo — Pagination Strategies for Large Systems 344
356 **Rishi Jha** · 3mo — Australia to India in 37 Seconds: Is SWIFT Finally Becoming Real-Time? 345
357 **Java Developers Community (moderated)** · 3mo — I once spent two hours debugging a `NullPoint... 346
358 **Sathish Kumar Subramani** · 3mo — Everyone's building on top of LLMs. 348
359 **Bijaya Bhaskar Swain** · 3mo — Ever Wondered How Instagram Instantly Says "Username Already T... 349
360 **Amit Raj** · 3mo — Most developers know how to code. Very few know these 15 design patterns,... 351
361 **Namra Patel** · 3mo — Idempotency: The Unsung Hero of Reliable APIs 352
362 **Ashish Pratap Singh** · 3mo — 15 Must-Know Design Patterns: 353
363 **Manu Agrawal** · 3mo — Amazon & Airbnb Rejected Me 3 Times in System Design. These Free Artic... 354
364 **Srinath Vasudevan** · 3mo — Most people think payment success depends on the payment gateway. 355
365 **Bhavesh Arora** · 3mo — Most developers know how to code. Very few know these 15 design patte... 355
366 **NeetCode** · 3mo — 24 Problems To Practice 24 Coding Interview Patterns 356
367 **AlgoMaster.io** · 3mo — Quick and useful 358
368 **Ashish Pratap Singh** · 3mo — DSA was HARD until I Learned these 15 Patterns: 358
369 **Sathish Kumar Subramani** · 3mo — Can You Draw Kafka's Architecture on a Whiteboard From Memo... 359
370 **Java Developers Community (moderated)** · 3mo — Spring Boot 4.1 just made Redis Pub/Sub a lot... 360
371 **Java Developers Community (moderated)** · 3mo — You did not need Kafka. You needed a job queue. 360
372 **Rajnikant Pandey** · 3mo — Harness Engineering: The Missing Layer Between LLMs and Production... 362
373 **Rakesh Gohel** · 3mo — Prompt engineering gets the headlines. 362
374 **Sumit Srivastava** · 3mo — Most people learn DSA by solving random coding questions. 363
375 **AlgoMaster.io** · 3mo — 13 Must-Know Data Structures for Coding Interviews: 365
376 **AKASH BHATNAGAR** · 3mo — MCP vs RAG vs AI Agents 366
377 **Ashish Pratap Singh** · 3mo — Top 6 Load Balancing Algorithms You Should Know: 366
378 **Ashish Pratap Singh** · 3mo — If you want to master concurrency and multi-threading interview... 367

379 **Ganesh Kumbhar** · 3mo — AI Architect Interview Patterns Repo ... 368
380 **Neo Kim** · 3mo — Arguably the 20 most influential books for software engineers. ... 369
381 **Rakesh Gohel** · 3mo — Most enterprises talk about responsible AI. Santander shipped it. ... 370
382 **Bapurao Patil** · 3mo — Interview Insight: Beyond RAG – Understanding KAG & CAG ... 371
383 **Sherry Jasal** · 3mo — Top 6 RAG architectures every AI engineer should know ... 371
384 **Alok Sharan** · 3mo — After building and reviewing multiple AI systems, one pattern shows up... ... 372
385 **Java Developers Community (moderated)** · 3mo — Java 21 shipped with 15 JEPs. ... 373
386 **Prasad Rao** · 3mo — I shared a 6-week Claude Architect certification roadmap I created for m... ... 375
387 **Alex Xu** · 3mo — RAG vs Graph RAG vs Agentic RAG ... 376
388 **Java Developers Community (moderated)** · 3mo — How we removed a synchronous dependency in an... ... 377
389 **Anshika Sinha** · 4mo — I lost 3 senior-level offers from Amazon, DE Shaw, & Salesforce betwe... ... 377
390 **Shivam Garg** · 4mo — In most SDE-3 interviews I've seen at Google, one thing becomes very ob... ... 378
391 **Julia Danyal** · 4mo — Most people are using Claude like Google. ... 379
392 **Parmeshwar Pardeshi** · 4mo — Complete OOP in Java — Concepts, Rules & Corner Cases ... 381
393 **Pradeep Kumar Tyagi** · 4mo — Most viral AI architecture at Internet ... 381
394 **Anmol S** · 4mo — AI Agents are only as powerful as the architecture behind them. ... 382
395 **Balaji Mani** · 4mo — Dear LinkedIn family, ... 383
396 **Maryam Miradi, PhD** · 4mo — How to Use Claude Code Like a Senior AI Engineer. → Even If You... ... 384
397 **Rakesh Gohel** · 4mo — AI agents are only as good as the engineers who build them. ... 386
398 **Anusha Sundaramurthi** · 4mo — Stop calling everything an AI Agent ... 387
399 **Aman Mishra** · 4mo — How do you route a million food orders a day without your servers catch... ... 388
400 **Rocky Bhatia** · 4mo — Behind the Scenes of UPI: How It Handles ₹18.41 Trillion Transactions! ... 389
401 **Abdul Rehman** · 4mo — LLMs → RAG → Agents → Agentic AI ... 390
402 **Smarter with AI** · 4mo — Free credentials won't make you AI-ready. A learning path will. ... 391
403 **Rishi Jha** · 4mo — Tokenization in Cards — The Invisible Security Layer Behind Modern Payments ... 392
404 **Rajesh Kumar** · 4mo — A Production-Ready Agentic AI Architecture Pattern on AWS ... 395
405 **AI Architecture Insights** · 4mo — Today's topic: Enterprise AI Architecture — From Demo to P... ... 396
406 **Daniel Moka** · 4mo — Most developers don't use agentic loops. ... 397
407 **Startup Adviser** · 4mo — The smartest job seekers aren't applying randomly anymore.They're↓ ... 397
408 **Antrixsh Gupta** · 4mo — Most people think LLMs have memory. ... 400
409 **Sagar Agarwal** · 4mo — Most Claude users are stuck at 10% capacity. ... 401
410 **Viktoriia Horokh** · 4mo — Reversal, refund and chargeback are often confused. ... 403
411 **Sivasankar Natarajan** · 4mo — Traditional AI vs Agentic AI vs Agentic RAG ... 403
412 **Java Developers Community (moderated)** · 4mo — Many developers think GC solves all memory pr... ... 404
413 **Aravind Kumar bysani** · 4mo — Java Evolution: Key Features from Java 8 to Java 22 ... 404
414 **Santosh Patidar** · 4mo — Day 11 of #30DaysOfJavaMastery | HashMap Internals Explained ... 406
415 **Java Developers Community (moderated)** · 4mo — Performance problems in Java Spring Boot appl... ... 407
416 **Big Data Data Science | Machine Lea...** · 4mo — <> 7000+ Course Free Access: https://lnkd.in/... ... 408
417 **Maryam Miradi, PhD** · 4mo — How to Build a Supply Chain AI Agent System → Even If Everything... ... 410
418 **Imran Shaikh** · 4mo — Java Virtual Threads: The Future of Concurrency in Java ... 411
419 **Ankit Pangasa** · 4mo — 8 Redis Use Cases Every Backend Engineer Should Know ... 412
420 **Ravena O** · 4mo — Why do so many AI agents become expensive at scale? ... 414
421 **Neo Kim** · 4mo — If you want to elevate your AI engineering career ... 414
422 **Anchal Sharma** · 4mo — Everyone wants the "perfect roadmap" for DSA. ... 416
423 **Sanjay M** · 4mo — Learning Networking Fundamentals ... 416
424 **Kamlesh Kawadkar** · 4mo — The Workflow Doesn't Exist Until The User Asks ... 417
425 **Kubernetes** · 4mo — Most repeated Kubernetes interview question: ... 418
426 **Big Data Data Science | Machine Lea...** · 4mo — Want to make Claude Code understand your comp... ... 418
427 **Swetha Telu** · 4mo — For a Java 8 interview, these 30+ programs test your understanding of: ... 419
428 **Sivasankar Natarajan** · 4mo — What is MCP (Model Context Protocol)? ... 419
429 **Gabriel Millien** · 4mo — Everyone is rushing to deploy AI agents. ... 420
430 **Vikram Kumar** · 4mo — Tokenization in Modern Payments ... 421
431 **Madhur Shyam Maurya** · 4mo — 15 Architecture Types ... 423
432 **Alex Xu** · 4mo — SLMs vs. LLMs, Clearly Explained ... 423
433 **Venkata Yugandhar Babu Teki** · 4mo — The most expensive architecture mistake isn't choosing... ... 424

434 **Utsav Bhanderi** · 4mo — Modern applications are not powered by APIs + databases alone anymore. ... 425
435 **Abdulla Basha Shaik** · 4mo — Spring AI Ecosystem: Building Production-Ready AI Applications... ... 427
436 **Karthikeyan Yuvaraj** · 4mo — 15 Practical LLM Cost Optimization Techniques Every AI Engineer... ... 428
437 **Mithun Venkatesan** · 4mo — Everyone is talking about smarter AI models. ... 429
438 **Ravena O** · 4mo — AI agents don’t fail loudly. ... 429
439 **Surajit P.** · 4mo — Which LLM should you actually use? ChatGPT vs. Grok vs. Gemini vs. Claud... ... 430
440 **Chris Donnelly** · 4mo — All people want to know is which LLM is "the best." ... 431
441 **Rajesh Ravindran** · 4mo — 7 AI agent concepts that will define how you build in 2026: ... 432
442 **Janjirala Anusha** · 4mo — Top 8 AI Models That Can Help You Work Smarter in 2026 ... 433
443 **Ajit Kumar** · 4mo — A standard, local in-memory lock (like Java's synchronized block or Reen... ... 434
444 **Abhishek Pandey** · 4mo — Not All Payments Are the Same... And That's Exactly What Makes Paym... ... 434
445 **Hina Arora** · 4mo — Are You Still Writing Custom Integrations for Every Tool Your AI Agent N... ... 435
446 **Saju Thomas** · 4mo — Enterprise AI Architecture — From Prompt to Production ... 435
447 **Kubernetes** · 4mo — Kubernetes ConfigMap vs Secret: A Common DevOps Interview Question ... 436
448 **Tarun Vegi** · 4mo — Continuing my journey of building a Wallet & Payment system from scratch. ... 438
449 **Neo Kim** · 4mo — If you want to get dangerously good at system design, ... 439
450 **Tarun Khandagare** · 4mo — After 2+ yrs at Visa as a Senior SDE, I decided to switch again so... ... 440
451 **Java Developers Community (moderated)** · 4mo — Every backend engineer should know how the Tr... ... 441
452 **Divyang Panchasara** · 4mo — As a Solution Architect, one of the biggest challenges is not de... ... 443
453 **Gabriel Millien** · 4mo — The best AI courses are free. ... 443
454 **SOORAJ VIDYASAGAR** · 4mo — Spring Boot + Spring AI MCP Server Architecture: Building AI-Nati... ... 444
455 **Rignor Labs** · 4mo — AI Engineer is no longer a single role. ... 446
456 **Rakesh Gohel** · 4mo — Vector databases just became optional for AI agents. ... 446
457 **Asim Kumar** · 4mo — I built a RAG pipeline from scratch — zero hallucination, full auditabil... ... 447
458 **ASHISH KUMAR** · 4mo — Pure Coding Vibe Coding ... 450
459 **Aiswarya Venkitesh** · 4mo — Master RAG with Azure – from documents to answers ... 450
460 **Vinod Thube** · 4mo — Building Resilient Microservices with Resilience4j & Circuit Breaker in... ... 451
461 **Rajendra M G** · 4mo — Still confused between LLMs, RAG, AI Agents, and MCP? ... 452
462 **Kashish Kathuria** · 4mo — Everyone is talking about Agentic AI... but most are still buildin... ... 454
463 **Aymane Mrini** · 4mo — This CLAUDE.md file will make you 10x engineer ↓ ... 454
464 **Srinivas Mahakud** · 4mo — Most AI agents fail not because they lack intelligence, ... 455
465 **Avinash Jha** · 4mo — We all use ChatGPT/Copilot/Claude every day. But did we ever wondered w... ... 455
466 **Shivani Virdi** · 4mo — Everyone talks about context engineering. No one shows you the memory... ... 456
467 **Julia Danyal** · 4mo — Everyone wants to start with Agentic AI, but the first layer is still... ... 457
468 **SOORAJ VIDYASAGAR** · 4mo — Spring Boot + Spring AI: Building Enterprise-Grade RAG Applications ... 458
469 **Brij Kishore Pandey** · 4mo — MCP is powerful. ... 459
470 **Nikki Siapno** · 4mo — Things Every Engineer Should Know: SQL Execution Order. ... 460
471 **Rakesh Gohel** · 4mo — Agentic Engineering in 2026 is not about mastering the latest model re... ... 461
472 **Yash Wadpalliwar** · 4mo — BEFORE vs AFTER: How AI Tools Are Evolving ... 462
473 **Nagarjuna Reddy** · 4mo — Most people use AI every day. ... 463
474 **Julia Danyal** · 4mo — Your AI stack is probably heavier than it needs to be. ... 464
475 **Ankit Pangasa** · 4mo — Bookmark This: 12 Agentic AI Terms Every Engineer Should Know in 2026 ... 465
476 **Java Developers Community (moderated)** · 4mo — We had a thread leak in production for months. ... 467
477 **Srinivas Mahakud** · 4mo — Building AI Agents with MCP? Don’t just connect... design smart. ... 469
478 **Ashish Sahu** · 4mo — Most organisations are focused on building more powerful AI agents. ... 469
479 **Brij Kishore Pandey** · 4mo — AI in 2026 is no longer just about prompts. ... 470
480 **Vrushabh Gawale** · 4mo — Every one of us must be using chatbots such as GPT/Co-pilot/Claude. ... 471
481 **Maryam Miradi, PhD** · 4mo — How to Use Knowledge Graphs for AI Coding Agents → Even If Your... ... 472
482 **Prasanth N.** · 4mo — AI engineering is rapidly becoming an optimization problem—not just a m... ... 473
483 **Satyam Jyottsana Gargee** · 4mo — My team reduced their LLM bill from $5,000 to $1,000 withou... ... 474
484 **Java Developers Community (moderated)** · 4mo — Strangler Pattern in Microservices ... 475
485 **SOORAJ VIDYASAGAR** · 4mo — Spring AI for Java Developers: The Missing Guide ... 476
486 **Java Developers Community (moderated)** · 4mo — Ever wondered why arrays start at 0 and not 1? ... 477
487 **Janjirala Anusha** · 4mo — Unlocking the Power of AI with Claude Commands! ... 477
488 **Mohd Salim** · 4mo — How a Modern AI Agent Works ... 478

489 **Big Data Data Science | Machine Lea...** · 4mo — Welcome to the next level of RAG Optimization! 479
490 **Vanshika Agarwal** · 4mo — I've sat in interviews at 3 top product giants like Google , Atlas... 480
491 **Balaji Mani** · 4mo — Agentic AI Engineering Stack – The Future of Enterprise AI Systems 480
492 **Akhila G** · 4mo — Asked by Deloiite for an AI Engineer role Round-2 482
493 **saisuresh yerragogu** · 4mo — Why IRCTC Faces Challenges During Tatkal Booking While Amazon H... 483
494 **Julia Danyal** · 4mo — Most AI teams are losing time because every task goes into the same box. 484
495 **Gabriel Millien** · 4mo — Prompt engineering gets the headlines. 485
496 **Julia Danyal** · 4mo — ChatGPT is not average. Most prompts are. 486
497 **vaibhav sable** · 4mo — Most Java Backend interviews don't fail candidates on coding. 488
498 **Harsh Arora** · 4mo — Apache Kafka Interview Preparation Sheet 489
499 **Julia Danyal** · 4mo — Claude certification is not a prompt test. 490
500 **Srinath P.** · 4mo — Did You Know? You Can Build Machine Learning Models in Pure Java Without... 491
501 **SOORAJ VIDYASAGAR** · 4mo — Stop Memorizing. Start Thinking Like an Architect. 492
502 **Sai Ram Somanaboina** · 4mo — JWTs are used by 67% of modern web apps for authentication. But... 493
503 **Big Data Data Science | Machine Lea...** · 4mo — FREE AI Courses! 493
504 **AI For SEO** · 4mo — RAG vs AI Agents 494
505 **Mansi Kaushik** · 4mo — Understanding the Saga Pattern in Microservices 495
506 **Java Developers Community (moderated)** · 4mo — Based on which measurements are method-handle... 495
507 **Priyanka Logani** · 4mo — Every Database Transaction Relies On These 4 Principles 496
508 **vaibhav sable** · 4mo — Java Streams Internals — How filter(), map(), and collect() Actually... 497
509 **Ashok Bollepalli** · 4mo — Java Stream API makes data processing simple, clean, and powerful. 498
510 **Devarasetty Badarinath** · 4mo — AI Explained Like a Human Body — The Simplest Way to Underst... 498
511 **Janjirala Anusha** · 4mo — AI Engineer vs Generative AI Engineer — Do you know the difference? 499
512 **Vishal Sharma** · 4mo — The Future of Java Enterprise Architecture: Agentic, Intelligent & Ev... 501
513 **Nagoor Meeran S.** · 4mo — Java + Microservices + Kafka (Part 5 — Scaling Event-Driven System... 501
514 **Sunil Reddy** · 4mo — One of the most common questions in system design: 502
515 **Parikh Jain** · 4mo — 2 patterns will save you from 200 Sliding Window rejections. 503
516 **Umashanker Pandey** · 4mo — I lost data in production because of 2 lines of code. 504
517 **Varun Sharma** · 4mo — Understanding Circuit Breaker: 505
518 **Md Ashikur Rahman** · 4mo — E-Commerce Transaction Flow Through ACS (3D Secure) 506
519 **Patrick Al Kreidy** · 4mo — Understanding the End-to-End Card Payment Lifecycle 507
520 **Prakash Sethia** · 4mo — A ₹500 Coffee Purchase: The Hidden Journey Through the Global Paymen... 508
521 **Srinivas Mahakud** · 4mo — Your architect keeps saying things like Idempotency, Immutability,... 509
522 **Alok Sharan** · 4mo — Your architecture isn't tested by traffic. 509
523 **Spring Users** · 4mo — So I've been experimenting with AI-assisted development on a Spring Bo... 510
524 **Shalini Goyal** · 4mo — GenAI vs AI Agents vs Agentic AI vs ML vs Data Science vs LLM 511
525 **Hina Arora** · 4mo — Everyone is talking about AI Agents. 511
526 **Java Developers Community (moderated)** · 4mo — Dumb move.... Say this to Claude, explain thi... 513
527 **Konica Dhingra** · 4mo — Java Garbage Collection Explained 513
528 **Anubhav Sethi** · 4mo — You know your system design answer felt incomplete. 514
529 **Revanth Boggarapu** · 4mo — Everyone wants to build scalable systems. 514
530 **Big Data Data Science | Machine Lea...** · 4mo — AI - aitagem.com/login.php - Join 10 million+... 517
531 **Java Developers Community (moderated)** · 4mo — Monolithic, Microservices, Serverless, or Eve... 518
532 **Mahesh Mallikarjunaiah ↗** · 4mo — Do you practice system design by memorizing architectures ? 518
533 **Prasanth V** · 4mo — One of the BIGGEST challenges for Kubernetes beginners is 519
534 **Ashish Pratap Singh** · 4mo — How to Design URL Shortener in a System Design Interview: 520
535 **Pramod Ahirwar** · 4mo — Exploring Java + AI Integration 522
536 **Chandrakumar R Pillai** · 4mo — Most people are still paying ₹20,000–₹2,00,000 for AI courses... 522
537 **Kubernetes** · 4mo — K8s Services and types ? #kubernetes. 523
538 **Zoran Milosevic** · 4mo — MCP vs RAG vs AI Agents 523
539 **Sivasankar Natarajan** · 4mo — Is Your AI Agent Architecture Just a Box Labeled "LLM"? 524
540 **Shalini Goyal** · 4mo — An Agentic AI system is not just a chatbot connected to tools. 525
541 **Gabriel Millien** · 4mo — 9 steps to ship an AI agent. 525
542 **Brij Kishore Pandey** · 4mo — Everyone's throwing around the same 12 words right now. 527
543 **Lakshmi Venkatesan** · 4mo — Problem solving in software engineering is rarely about memorisi... 528

544 **SOORAJ VIDYASAGAR** · 4mo — Caching improves performance and scalability by storing frequentl... 529
545 **Ganesh Bankar** · 4mo — A well-designed caching strategy can reduce database load, improve re... 529
546 **Sudeep Srivastava** · 4mo — Pub/Sub Pattern in Distributed Systems Explained Simply 530
547 **Chinmaya Lenka** · 4mo — Improving API Performance in Spring Boot (Real Optimization Approach) 531
548 **Anis Rahman** · 4mo — Most backend devs think they're interview-ready... until the first "exp... 532
549 **Nitesh Kumar Saw** · 4mo — Most candidates do not fail coding interviews because they lack ta... 534
550 **Adarsh C.** · 4mo — 10 Golden Rules for Solving a Coding Question in an Interview 535
551 **Sahil Kumar** · 4mo — Sorting algorithms are one of those DSA topics everyone learns... 535
552 **Sai Roshan Neelam** · 4mo — If you're preparing for coding interviews... 536
553 **Sai Krishna Chivukula** · 4mo — Most people fail DSA not because they can't code... 537
554 **Java Developers Community (moderated)** · 4mo — Great breakdown of the Prototype pattern. In... 539
555 **Parikh Jain** · 4mo — Solve this self-testing DSA cheatsheet with 13 groups of patterns & 65... 539
556 **Marcel Dybalski** · 4mo — Your architect keeps using words like Idempotency or Immutability? 540
557 **Ashutosh Bhondele** · 4mo — Most Java developers know how to sort. 541
558 **Java Developers Community (moderated)** · 4mo — New article: RAG Can't Cancel Orders — Buildi... 542
559 **Raghavendra Bagalkoti** · 4mo — Nobody teaches you AWS this way. 542
560 **Sai Kiran** · 4mo — HOW MUCH YOU SHOULD ASK 543
561 **Java Developers Community (moderated)** · 4mo — Helpful.. 544
562 **Rakesh Gohel** · 4mo — The real risk in AI Agents is not capability. It is control. 544
563 **Vinod Alabana** · 4mo — In today's real-time systems, APIs and Apache Kafka play a major role... 545
564 **Ambadas Garje** · 4mo — Spring Framework | Day 27 545
565 **Java Developers Community (moderated)** · 4mo — 100+ AI tools. One screen. Organized by what... 547
566 **Parikh Jain** · 4mo — DSA Preparation Roadmap (From Zero to OA Ready) 548
567 **Ankur Kesharwani** · 5mo — In 2025, I made my 2nd switch with a 70% hike to Senior MTS at Ora... 549
568 **Ayushmaan Chandra** · 5mo — Why Most Payment Failures Are Actually False Failures in Digital... 550
569 **AlgoMaster.io** · 5mo — 10 Must-Know Graph Algorithms for Coding Interviews: 551
570 **Java Developers Community (moderated)** · 5mo — Blocking vs Non-Blocking in Java — 552
571 **SHIVAM SOMANI** · 5mo — Before I made my 2nd switch to Microsoft , I gave an interview at Wal... 553
572 **Shivam Garg** · 5mo — What does it take to go from SDE-1 → 40-50+ LPA Senior SDE at Google, A... 554
573 **Narendra Sahoo** · 5mo — Most developers say they know microservices. 555
574 **Ramya A K** · 5mo — The Spring Framework is a powerful Java framework used for building enter... 556
575 **Ashish Pratap Singh** · 5mo — Redis is NOT just a cache. Here are 10 use cases you should know: 557
576 **Ashish Pratap Singh** · 5mo — SOLID Principles Explained with Clear Examples: 557
577 **AlgoMaster.io** · 5mo — 20 must-know DSA patterns for coding interviews: 558
578 **Java Developers Community (moderated)** · 5mo — Concurrency vs Parallelism in Java 559
579 **Julia Danyal** · 5mo — AI isn't magic. It's a stack. 560
580 **Spring Users** · 5mo — Java/Spring Boot Scheduler Cheatsheet 562
581 **Sagar Palai** · 5mo — One thing that completely changed how I approach DSA interviews ↓ 562
582 **Ankit Kumar** · 5mo — SAGA PATTERN in Spring Boot — The Backbone of Distributed Transactions... 563
583 **Anis Rahman** · 5mo — HashMap vs ConcurrentHashMap in Java 564
584 **Sangram Sekhar Panda** · 5mo — Recently explored Apache kafka with SpringBoot and learned how... 564
585 **Omkar S.** · 5mo — Most "AI agent failures" are not AI failures. They are org failures wearin... 564
586 **Himanshu Sharma** · 5mo — 7 Architecture Decisions That Quietly Kill Scalability 565
587 **Srinath P.** · 5mo — AI is no longer limited to Python ecosystems. 567
588 **Rajesh Kumar** · 5mo — Microservices Design Patterns: Building Scalable & Resilient Systems 568
589 **Manish Kala Jain** · 5mo — Visa vs Mastercard vs American Express ↓ 569
590 **Ravindra Shinde** · 5mo — Apache Kafka Architecture — Explained for Engineers 570
591 **Anshul Agarwal** · 5mo — Most engineers prepare for System Design interviews by memorizing di... 571
592 **Arslan Ahmad** · 5mo — Most candidates draw the same vague boxes-and-arrows for every system... 574
593 **Aravind Kumar bysani** · 5mo — Java Interview Programs — Basic to Advanced With Code Examples 574
594 **Parth Sarthi** · 5mo — Sharing My Recent Technical Interview Experience at Visa! 575
595 **Neo Kim** · 5mo — How to get ahead of 99% of people at deployments 576
596 **Sami Najim** · 5mo — BREAKING: Everyone wants to learn AI Agents in 2026... 577
597 **DataLinkk** · 5mo — Choosing the right workflow orchestration framework is becoming one of th... 578
598 **Docker Professionals Network Contai...** · 5mo — https://lnkd.in/dxQSkx5a 579

599 **Alex Xu** · 5mo — Forward Proxy, Reverse Proxy, and API Gateway Explained 579
600 **Ravena O** · 5mo — Integrating Agentic RAG Systems with MCP ↓ 580
601 **Shameer Ahamad** · 5mo — Deadlock in Java — One of the Silent Killers in Multithreading 580
602 **Tanmay Khilari** · 5mo — Heap vs Stack in Java Memory Management 581
603 **AlgoMaster.io** · 5mo — 14 must-know Data Structures for coding interviews: 582
604 **Subhash Kumar** · 5mo — The best things in life are free. 583
605 **Ashish Pratap Singh** · 5mo — If you want to level up as a backend engineer, consider reading... 583
606 **Java Developers Community (moderated)** · 5mo — Most explanations of Java Virtual Threads sto... 584
607 **Arslan Ahmad** · 5mo — The "Sidecar" Pattern. 585
608 **Alok Kumar** · 5mo — Designing a Modern AI-Powered Daily Activity Tracker using Java, Spring... 586
609 **Narendra Sahoo** · 5mo — Most developers learn Java syntax. 587
610 **Java Developers Community (moderated)** · 5mo — Ok, bust the myth right away - virtual thread... 588
611 **Java Developers Community (moderated)** · 5mo — Spring Boot Developers: Stop writing response... 588
612 **Java Developers Community (moderated)** · 5mo — Solace PubSub+ vs Kafka: Different Approaches... 589
613 **Reshma Sriraman** · 5mo — The Complete Agentic AI Stack Every Developer Must Know in 2026 589
614 **Ashish Pratap Singh** · 5mo — 10 Golden Rules to Write Clean Code: 592
615 **Priyanka Logani** · 5mo — Optimistic locking vs pessimistic locking 593
616 **Ved Vekhande** · 5mo — Someone built an AI from scratch. 594
617 **Technical Spark** · 5mo — Apple Pay vs Google Pay: Which is More Secure? 595
618 **Ana Pedra** · 5mo — Most people still think AI = ChatGPT answering questions 595
619 **Ashish Pratap Singh** · 5mo — Most people think money moves instantly when they swipe a card.... 596
620 **Java Developers Community (moderated)** · 5mo — Java serialization broke our rolling deployme... 597
621 **Java Developers Community (moderated)** · 5mo — Every backend engineer should know how paymen... 598
622 **Ankit Pangasa** · 5mo — Most database interviews focus on SQL syntax. 599
623 **Arslan Ahmad** · 5mo — 10 Must Read Papers for Senior Software Engineers 602
624 **Feezan Khattak** · 5mo — Multi-Region Payment Systems 603
625 **Feezan Khattak** · 5mo — Observability in Payments 603
626 **SOORAJ VIDYASAGAR** · 5mo — Large switch statements are one of the most common maintainabilit... 604
627 **Saumya Makker** · 5mo — A junior came to me with a problem: 605
628 **Kanika Gosain** · 5mo — Microservices Design Patterns Cheat Sheet (Save this) 606
629 **Srikant Mahanty** · 5mo — You're Using This Pattern Daily in Spring... But Do You Really Unde... 607
630 **Neo Kim** · 5mo — 6 algorithms that'll help you load balance traffic: 608
631 **Vinay Kumar** · 5mo — Two developers. Same row. Same millisecond. 609
632 **Sameer Bhardwaj** · 5mo — If I travel back to my Interview Days, I will study these 40 Articl... 610
633 **Asmita Pednekar** · 5mo — Most Java developers say: 610
634 **Sai Ram Somanaboina** · 5mo — Cloudflare handles over 45 million HTTP requests every single s... 612
635 **Prashant Varshney** · 5mo — The AI Learning Gap is Real. You Need a Map, Not Noise. 613
636 **Java Developers Community (moderated)** · 5mo — Garbage Collection in Java — The Silent Hero... 614
637 **Divya Pratap Singh** · 5mo — Concurrency looks simple until your backend starts behaving unpr... 615
638 **Ganesh Bankar** · 5mo — Modern Java Microservices Architecture The Backbone of Scalable Enter... 616
639 **Sohel khan** · 5mo — Spring AI Cheat Sheet for Java Developers! 616
640 **Suraj Mishra** · 5mo — Spring Boot Interview Question: Selective Retry on Network Errors 617
641 **Dipu Kumar** · 5mo — *Design Patterns in Java: The Complete Cheat Sheet * 617
642 **Java Developers Community (moderated)** · 5mo — Lazy vs. Eager Loading in Hibernate with Spri... 618
643 **Blesson V** · 5mo — Java Spring Boot Microservices Learning Roadmap (2026) 620
644 **Sunil Reddy** · 5mo — Most people add cache like this: 620
645 **HarshaVardhan Sakhamuri** · 5mo — Your microservices aren't slow because of traffic... 621
646 **Kanika Gosain** · 5mo — Most developers use Spring annotations daily... 622
647 **Java Developers Community (moderated)** · 5mo — Very informative. 623
648 **Ambadas Garje** · 5mo — Spring Framework | Day 15 623
649 **Anis Rahman** · 5mo — 8 Java Maps. 8 Different Problems. Which One Do You Actually Need? 624
650 **Sarvesh Kumar Singh** · 5mo — JAR vs WAR vs EAR — Java Packaging Demystified! 625
651 **Java Developers Community (moderated)** · 5mo — The one thing that cost me FAANG interview (a... 625
652 **Mandar Pandit** · 5mo — "Architecting Knowledge" - Java Wisdom Series 627
653 **Panagiotis Kriaris** · 5mo — Card payments are a complex business. A layered stack of provide... 627

654 **Priya Jain** · 6mo — I walked into an HSBC Karat interview completely unprepared for System D... 628
655 **Pramod Dhembare** · 6mo — Mastering @Transactional in Spring — One of the MOST asked intervie... 629
656 **Java Developers Community (moderated)** · 6mo — Most developers think Spring Boot = APIs... 630
657 **Eid Badr** · 6mo — Java interview classic: static vs non-static 630
658 **AlgoMaster.io** · 6mo — 10 must-know Array patterns for coding interviews: 631
659 **Parikh Jain** · 6mo — Solve these 11 Groups of 50+ DP problems to crack & solve any DP proble... 631
660 **SERKUT YILDIRIM** · 6mo — Java/Spring Boot Tip - Missing Values 633
661 **Manish Kala Jain** · 6mo — **Payment System Architecture 633
662 **Manish Kala Jain** · 6mo — Payment Wallet Architecture — What Really Happens Behind “Pay” in... 635
663 **Dinesh Anbumani** · 6mo — → The Netflix Architecture Mystery 638
664 **Mandar Pandit** · 6mo — "Architecting Knowledge" - Java Wisdom Series 638
665 **Alex Xu** · 6mo — Optimistic locking vs pessimistic locking 639
666 **Kubernetes** · 6mo — If you’re still using single-stage Dockerfiles, you’re probably shipping... 639
667 **Java Developers Community (moderated)** · 6mo — HashMap internal working 640
668 **Pedro Ostanik** · 6mo — Thread Pools to Virtual Threads. 640
669 **Rocky Bhatia** · 6mo — Top 6 API architecture Styles 641
670 **Aman Mishra** · 6mo — Java Collections interview questions that keep showing up — with real a... 641
671 **Kanak Agrawal** · 6mo — Came across one of the most structured and practical preparation guid... 642
672 **Suraj Mishra** · 6mo — Spring Boot Interview Question: Connection Pool Exhaustion 643
673 **Big Data Data Science | Machine Lea...** · 6mo — LLM API Request in 400ms 643

#1 Shaurya Pratap Singh

System Design

SDE-2 at Microsoft | Ex-Salesforce, Ex-Amazon | ICPC Regionalist

5h · View on LinkedIn

Want to get better at System Design?

After working as a Software Engineer at Amazon, Salesforce, and Microsoft, one thing I've realized is this:

The best engineers don't just know system design concepts, they understand how real systems actually work.

If you're preparing for product companies, these are some of the best topics to study, along with free YouTube resources ↓

1. How Stock Exchange Works

Low latency, matching engines, fault tolerance

https://lnkd.in/gCXk-Pti

2. How Payment Systems Work

Transactions, idempotency, consistency, fraud handling

https://lnkd.in/gxmrEKS4

3. How YouTube Works

Video storage, CDN, encoding pipelines, scaling

https://lnkd.in/gWFXwe67

4. How Google Docs Works

Real-time collaboration, CRDTs, sync conflicts

https://lnkd.in/g-iF2XVm

5. How Kafka Works

Event streaming, partitions, brokers, replication

https://lnkd.in/gBiu9ePG

6. How URL Shortener Works

Database design, hashing, scaling

https://lnkd.in/gHmAvbHY

7. How WhatsApp Works

Message queues, delivery guarantees, presence systems

https://lnkd.in/g-iF2XVm

8. How Airbnb Works

Marketplace scaling, booking workflows

Search in the above playlist

9. How Spotify Works

Music streaming architecture, caching

Search "Spotify System Design" in the same playlist

How Slack Works

Chat architecture, WebSockets, event-driven systems

Search "Slack System Design" in the same playlist

11. How Reddit Works

Feed generation, ranking, caching

Search "Reddit System Design" in the same playlist

12. How Uber Finds Nearby Drivers

Geospatial indexing, proximity search

https://lnkd.in/gGcy4s3V

13. How Amazon S3 Works

Distributed storage, replication, durability

https://lnkd.in/gZbQ5Q4M

14. How LLM Systems (like ChatGPT) Work

Inference scaling, model serving, pipelines

https://lnkd.in/gZbQ5Q4M

Don’t just watch the videos.

✓ Pause frequently

✓ Draw the architecture from memory

✓ Identify bottlenecks

✓ Think about trade-offs and alternative designs

That’s where real learning happens.

In my experience interviewing and working at Amazon, Salesforce, and Microsoft, strong system design isn’t about throwing around buzzwords, it’s about making the right engineering trade-offs.

Go deep into even 6–7 of these systems, and you’ll be ahead of most candidates walking into system design interviews.

Save this post and share it with someone preparing for top product companies.

Follow Shaurya Pratap Singh for more!

LINKS

- https://lnkd.in/gCXk-Pti
- https://lnkd.in/gxmrEKS4
- https://lnkd.in/gWFXwe67
- https://lnkd.in/g-iF2XVm
- https://lnkd.in/gBiu9ePG
- https://lnkd.in/gHmAvbHY
- https://lnkd.in/gGcy4s3V
- https://lnkd.in/gZbQ5Q4M

#2 Design Gurus

System Design

1d · **View on LinkedIn**

Your application is growing. Your database is the bottleneck. You have 6 options.

Most engineers know 2. Here is the full map.

1. Vertical Scaling

Get a bigger machine. More CPU, more RAM, faster SSD.

The simplest option. No application changes required.

Limit: there is a ceiling. The biggest available machine still has a maximum. And it is a single point of failure.

When to use: early stage, when the bottleneck is straightforward and the scale is manageable.

2. Read Replicas

Add read-only copies of the primary database. Direct all read traffic to replicas.

Works when reads vastly outnumber writes, which is most applications.

The primary handles writes. Replicas handle reads. Both stay in sync via replication.

Limit: replication lag means replicas may return slightly stale data. Not suitable when reads must always see the latest write.

3. Caching

Store frequently read data in Redis or Memcached. Serve it without hitting the database at all.

A cache hit is 10-100x faster than a database query.

Strategies: cache-aside (application checks cache first), write-through (write to cache and DB together), read-through (cache fetches from DB on miss).

Limit: cache invalidation. Stale data if the cache is not updated when the database changes.

4. Connection Pooling

Maintain a pool of open database connections shared across application threads.

Opening a new database connection is expensive. Reusing existing ones removes that overhead.

PgBouncer for PostgreSQL. RDS Proxy for AWS Lambda and serverless.

Limit: does not increase the database's own capacity. Only reduces the overhead of connection management.

5. Horizontal Sharding

Split the data across multiple database instances. Each shard holds a subset of the rows.

Common strategies: shard by user_id range, by geographic region, or by a hash of the key.

Reads and writes for a given record go to one specific shard.

Limit: cross-shard queries become complex or impossible. Joins across shards do not work cleanly.

6. Denormalization

Intentionally duplicate data to reduce the need for joins.

Store the user's name inside the order record instead of joining the users table on every order query.

Fewer joins means faster reads at scale.

Limit: writes become more expensive. Keeping duplicated data consistent requires care.

Quick map:

Read-heavy traffic: Read Replicas

Repeated queries for the same data: Caching

Too many connections from app servers: Connection Pooling

Single machine hitting its ceiling: Vertical Scaling

Writes exceeding what one machine handles: Sharding

Slow joins on large tables: Denormalization

Real systems use several of these together. The interview question is which ones, in which order, and why.

Save this. Follow DesignGurus.io for more.

LINKS

- http://DesignGurus.io

#3 Ankur Dhawan System Design

MTS-2 @Adobe | 112K+ @LinkedIn | 50+ Brand Collabs | AI • Tech • Marketing

1d · **View on LinkedIn**

Stripe offers ₹60–80 LPA for SDE-1 and ₹1 Cr+ for SDE-2,

These are the classic questions they ask

1. Design a Payment Processing System
2. Design Stripe Connect for Marketplace Payments
3. Design a Payment Gateway
4. Design a Subscription Billing System
5. Design a Fraud Detection System for Payments
6. Design an Idempotent Payment API
7. Design a Payment Retry and Recovery System
8. Design a Double-Entry Ledger System

9. Design a Real-Time Payment Reconciliation System
10. Design a Refund Processing System
11. Design a Webhook Delivery System
12. Design a Rate Limiter for Payment APIs
13. Design a Distributed Transaction System for Payments
14. Design a Global Payment Routing System
15. Design a Credit Card Tokenization Service
16. Design a Dispute and Chargeback Management System
17. Design a Financial Reporting and Analytics Platform
18. Design a Multi-Currency Payment Processing System
19. Design a Real-Time Risk Scoring Engine
20. Design a High-Availability Payment Infrastructure

Recently i gave interview and if you need resources + Questions

Checkout: https://lnkd.in/gPjeRmDY

LinkedIn

LinkedIn Guide to Creating

Follow Ankur Dhawan for more!

LINKS

- https://lnkd.in/gPjeRmDY

#4 Joel Teodoro Gómez

System Design

Backend Engineer · Go (Golang) | Kubernetes · Docker · gRPC · GraphQL · PostgreSQL · MySQL · Redis · Kafka · Elasticsearch · AWS · GCP | Platform & Infrastructure · Distributed Systems

1d · **View on LinkedIn**

10 GB file. 512 MB of RAM. Load it into a database.

The file is 20x bigger than your memory, and the first instinct is Kafka.

Even the author of a popular Medium post on this question admits it: under pressure, he answered Kafka or Flink.

On one machine with one core, that just moves the problem.

The interviewer isn't asking about tools.

They're asking: what does your memory depend on?

The answer: the batch. Never the file.

1. Read: stream it with a small buffer. Never load it whole.
2. Batch: group a few thousand rows.
3. Process: workers transform each batch.
4. Write: one bulk insert per batch. COPY in Postgres, LOAD DATA in MySQL.
5. Checkpoint: save the byte offset after each commit. Crash at 7 GB? Resume at 7 GB.

Put a bounded queue between every stage.

Slow DB → queue fills → workers wait → reader stops.

That's backpressure. Memory ≈ queue capacity × batch size, whether the file is 10 GB or 10 TB.

And Kafka? It's the right second answer, not the first.

When one machine isn't fast enough, the same stages become pods and the in-memory queues become a broker.

Same design, more pieces to run.

Your memory should depend on your batch size, not your file size.

The question → https://lnkd.in/eryiCN94

#Backend #SystemDesign #DistributedSystems #DataEngineering

LINKS

- https://lnkd.in/eryiCN94

#5 Suresh G.

AI / LLM

SSE @Oracle | ex Amazon | ex Microsoft | Best Selling Udemy Instructor | IIT KGP || Heartfulness Meditation Trainer

1d · **View on LinkedIn**

Top 12 GitHub repos every software engineer should bookmark to learn AI engineering ↓

Between them, they cover the whole path: how an LLM works inside, how to build agents and RAG systems, and how to ship real AI apps.

If you're a backend or full-stack engineer wondering where to start, start here. All 12 are free:

1/ AI Engineering from Scratch by Rohit Ghumare (★ 71.6k)

https://lnkd.in/gGebwCMC

A full path from the basics to shipping AI products, built around projects.

2/ LLMs from Scratch by Sebastian Raschka (★ 106.9k)

https://lnkd.in/gAf9WRq6

Build a ChatGPT-style model in PyTorch, one chapter at a time.

3/ nanochat by Andrej Karpathy (★ 58.8k)

https://lnkd.in/g4ScwdzM

The full pipeline of a small ChatGPT, from training to a chat UI, in one readable codebase.

4/ LLM Course by Maxime Labonne (★ 84.5k)

https://lnkd.in/g3exPMYq

Roadmaps and Colab notebooks for fine-tuning, quantisation and evaluation.

5/ Generative AI for Beginners by Microsoft (★ 121.7k)

https://lnkd.in/gdH_hzba

21 lessons on building apps with LLMs.

6/ AI Agents for Beginners by Microsoft (★ 77.5k)

https://lnkd.in/g_upcDYh

18 lessons on tools, planning, memory and multi-agent design.

7/ RAG Techniques by Nir Diamant (★ 29.9k)

https://lnkd.in/gu8KnPXT

Every major retrieval technique, each with runnable code.

8/ Awesome LLM Apps by Shubham Saboo (★ 142.4k)

https://lnkd.in/gzHJ52J6

100+ working agent and RAG apps you can clone and take apart.

9/ AI Engineering Hub (★ 38.4k)

https://lnkd.in/gRMjZFjt

In-depth tutorials on real-world agents, RAG and LLM apps.

10/ Claude Cookbooks by Anthropic (★ 53.6k)

https://lnkd.in/ghQjRFDT

Production recipes for tool use, RAG, evals and agents.

11/ MCP Servers (★ 91.4k)

https://lnkd.in/gU38xifE

Reference servers for connecting agents to files, databases and APIs.

12/ Agent Skills by Anthropic (★ 182.4k)

https://lnkd.in/gbhj8fa9

How to package instructions and scripts so an agent can reuse them.

My suggestion: start with #2 or #3 on a weekend. Then pick one app from #8 and rebuild it from memory. You learn more from that one rebuild than from a month of tutorials.

LINKS

- https://lnkd.in/gGebwCMC
- https://lnkd.in/gAf9WRq6
- https://lnkd.in/g4ScwdzM
- https://lnkd.in/g3exPMYq
- https://lnkd.in/gdH_hzba
- https://lnkd.in/g_upcDYh
- https://lnkd.in/gu8KnPXT
- https://lnkd.in/gzHJ52J6
- https://lnkd.in/gRMjZFjt
- https://lnkd.in/ghQjRFDT
- https://lnkd.in/gU38xifE
- https://lnkd.in/gbhj8fa9

#6 Arslan Ahmad

Other

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

2d · **View on LinkedIn**

Same feature. Two engineers. Very different solutions.

The task: send a welcome email when a user signs up.

A mid-level engineer and a senior engineer will both ship something that works. The difference is what happens next week, when real traffic and real failures show up.

Here's how the two solutions differ:

1. Understanding the problem

Mid-level: calls the email API right inside the signup handler.

Senior: first asks, "Must it arrive? How fast? What if it fails?"

2. Design

Mid-level: a synchronous call. Signup waits for the email to send.

Senior: drops a job on a queue. Signup returns in milliseconds.

3. Failure

Mid-level: the email provider goes down, so signups start failing.

Senior: retries with backoff, then moves the job to a dead-letter queue.

4. Duplicates

Mid-level: a retry fires twice. The user gets two welcome emails.

Senior: uses an idempotency key. One email per user, guaranteed.

5. Scale

Mid-level: fine at 100 signups a day. Launch day blows through the provider's quota.

Senior: a rate-limited worker drains the queue. The spike just waits its turn.

6. Visibility

Mid-level: finds out from a user. "I never got the email."

Senior: tracks sent, failed, and queue depth, with an alert.

7. Rollout

Mid-level: ships to 100% of users at once.

Senior: puts it behind a feature flag. 5% first, instant rollback.

8. Handoff

Mid-level: closes the ticket and moves on.

Senior: writes down the trade-offs, adds a runbook, and tells support what changed.

The pattern:

Mid-level engineers make it work. Senior engineers make it keep working.

None of this is extra code for its own sake. Each step answers one question: what happens after it ships?

In a system design interview:

This is exactly what the "senior" bar looks like. Don't stop at the happy path. Walk through failure, duplicates, scale, and how you'd know something broke.

Which of these 8 do you see skipped most often?

#softwareengineering #systemdesign #careergrowth #seniorengineer #backend

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview

#7 Neo Kim System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2d · **View on LinkedIn**

If you want to become god-level at system design

(in 3 weeks), then learn these 15 case studies:

1. How Apache Kafka Works

https://lnkd.in/eTtVAjTg

2. How Google Search Works

https://lnkd.in/exsvNqFn

3. How YouTube Works

https://lnkd.in/e7q9F4Sg

4. How Google Docs Works

https://lnkd.in/ehPNA7Az

5. How WhatsApp Works

https://lnkd.in/eU2fswMi

6. How Airbnb Works

https://lnkd.in/dGVfstQM

7. How Spotify Works

https://lnkd.in/eGbWVeNW

8. How Slack Works

https://lnkd.in/eATMDjrK

9. How Reddit Works

https://lnkd.in/egmm_P7a

10. How Bluesky Works

https://lnkd.in/eEhB8V_k

11. How Twitter Works

https://lnkd.in/eniXMPfU

12. How Uber Computes ETA

https://lnkd.in/eVKV2ePC

13. How Bitly Shortener Works

https://lnkd.in/evFTZVQq

14. How Stock Exchange Works

https://lnkd.in/eNf2QxVZ

15. How Payment System Works

https://lnkd.in/ecVw7jfi

Anything else you'd add to this list?

===

PS - Want my AI Engineering Playbook for FREE?

Join my newsletter with 250K+ software engineers:

→ https://lnkd.in/eRCMs_JP

===

Repost to help others master system design.

Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eTtVAjTg
- https://lnkd.in/exsvNqFn
- https://lnkd.in/e7q9F4Sg
- https://lnkd.in/ehPNA7Az
- https://lnkd.in/eU2fswMi
- https://lnkd.in/dGVfstQM
- https://lnkd.in/eGbWVeNW
- https://lnkd.in/eATMDjrK
- https://lnkd.in/egmm_P7a
- https://lnkd.in/eEhB8V_k
- https://lnkd.in/eniXMPfU
- https://lnkd.in/eVKV2ePC
- https://lnkd.in/evFTZVQq
- https://lnkd.in/eNf2QxVZ
- https://lnkd.in/ecVw7jfi
- https://lnkd.in/eRCMs_JP

#8 Pragya Verma System Design

Senior Software Engineer @Coforge |Core JAVA|DSA|Spring boot|Spring Hibernate|Microservices|Docker|Kafka|MongoDB

3d · **View on LinkedIn**

Payment System Design | High-Level Architecture

Recently, I explored designing a **scalable, reliable, and fault-tolerant Payment System**, focusing on handling concurrent requests, preventing duplicate payments, and ensuring reliable event processing.

Here are the key components and design decisions:

1. API Gateway

- Entry point for payment requests.
- Handles authentication, rate limiting, and request routing.

2. Payment Service & Idempotency

- Creates and manages payment transactions.
- Uses Redis for atomic idempotency checks to prevent concurrent duplicate processing.
- Maintains payment details in SQL DB for durable duplicate protection.

3. Payment Orchestrator

- Coordinates payment processing across different payment methods.
- Communicates with external payment gateways.
- Handles payment authorization and status retrieval.

4. Transactional Outbox Pattern

- Updates the payment record and outbox table within a single database transaction.
- Ensures payment events are not lost if the application crashes before publishing to Kafka.

5. Debezium CDC + Kafka

- Debezium captures committed outbox changes from database transaction logs.
- Publishes events to Kafka asynchronously.
- Enables downstream services like Inventory and Analytics to process payment events independently.

6. Webhook Handler

- Receives asynchronous payment status updates from the gateway.
- Handles duplicate notifications and updates payment status safely.

7. Reconciliation Workers

- Identify pending or uncertain transactions.
- Verify actual payment status with the gateway.
- Prevent unnecessary retries and potential double charges.

Key Design Takeaways

✓ Idempotency prevents duplicate payment processing.
✓ Database transactions ensure atomic business and outbox updates.
✓ Outbox + CDC enables reliable asynchronous event publishing.
✓ Idempotent Kafka consumers handle duplicate event delivery.
✓ Reconciliation helps recover from crashes, timeouts, and uncertain payment outcomes.

The biggest learning: Designing a payment system isn't just about processing transactions. It's about ensuring correctness when requests are retried, services crash, or external systems fail.

#SystemDesign #HighLevelDesign #Java #SpringBoot #Microservices #Kafka #Redis #Debezium #PaymentSystem #SoftwareEngineering

#9 Mahmood Salah

AI / LLM

Founder @ BootcampAI | AI, Data Science

3d · **View on LinkedIn**

AI engineer technical interview in the next 24 hours, practice these 25 problems.

They cover 95% of the concepts.

1) Design a RAG system for 10M documents
2) Your RAG gives wrong answers. Debug it
3) Pick a chunking strategy for legal contracts
4) BM25 vs dense vs hybrid search: when to use each
5) Add a reranker and measure if it helped
6) Reduce LLM cost by 70% without hurting quality
7) Cut first-token latency for a chat app
6) Design a semantic cache with hit-rate tracking
9) Route requests between a cheap and a smart model
10) Build an agent that uses tools reliably
11) Your agent gets stuck in a loop, Fix it
12) Design memory for a long-running assistant
13) Make LLM output always valid JSON
14) Detect and reduce hallucinations
15) Build an eval suite for a customer support bot
16) LLM-as-a-judge: how to make it trustworthy

17) Block prompt injection in a tool-using agent
18) Redact PII before it hits the model
19) Fine-tune vs RAG vs prompting: pick one and defend it
20) Fine-tune a small model with LoRA on 5K examples
21) Serve an open model at scale (batching, KV cache, quantization)
22) Stream responses over SSE and handle disconnects
23) Trace every step of a multi-agent workflow
24) Roll back a bad prompt change in production
25) Design ChatGPT-style chat for 1M daily users
Don't memorize answers.
Practice explaining trade-offs out loud.
That's what interviewers actually grade.

MEDIA document/carousel (possible)

#10 Swadesh Kumar

System Design

Software Engineer | Co-founder @CodenexAI | 150k+ Followers | RAG, Generative & Agentic AI | AI, Tech & Marketing Content | Brand Partnership | Campaign execution

3d · **View on LinkedIn**

Most people learn System Design by memorising architectures.
That’s why they struggle when the interviewer changes the problem.
Instead, learn these 12 algorithms/concepts and understand where each one fits ↓
1. Consistent Hashing
→ Distribute data across servers
→ Useful for distributed caches & databases
2. Rendezvous Hashing
→ Select the best server/node for a key
→ Useful for load distribution
3. Rate Limiting Algorithms
→ Token Bucket
→ Leaky Bucket
→ Fixed Window
→ Sliding Window
4. LRU Cache
→ Evict least recently used data
→ Important for caching systems
5. Bloom Filter
→ Quickly check whether an item may exist
→ Useful for reducing unnecessary database lookups
6. Count-Min Sketch
→ Approximate frequency/count of events
→ Useful for high-volume analytics
7. Leader Election
→ Choose one node as the coordinator
→ Important in distributed systems
8. Consensus Algorithms

→ Raft
→ Paxos
→ Help distributed nodes agree on a state
9. Gossip Protocol
→ Nodes exchange information with each other
→ Useful for service discovery and cluster membership
10. Merkle Tree
→ Efficiently compare large datasets
→ Used in distributed storage and replication
11. MapReduce
→ Split a large computation into smaller tasks
→ Process them in parallel
→ Combine the results
12. Top-K / Heavy Hitters
→ Find the most frequent items from huge datasets
→ Useful for trending searches, recommendations and analytics
A simple System Design learning roadmap:
Algorithms
↓
Data Structures
↓
Caching
↓
Load Balancing
↓
Databases
↓
Distributed Systems
↓
Queues & Streaming
↓
Scalability
↓
Fault Tolerance
↓
Complete System Design
Don't just learn what each algorithm does.
Learn:
→ What problem does it solve?
→ Why is it needed?
→ What are its trade-offs?
→ Where would you use it in a real system?
Follow Swadesh Kumar for more
Do checkout w3schools.com articles for system design & backend understanding.
Save this roadmap if you're preparing for System Design interviews.
#SystemDesign

#11 Sampath Kasoji

System Design

Machine Learning Engineer @ ServiceNow | Gen AI & Agentic Workflows | DSA & System Design | Ex-Experian

4d · View on LinkedIn

System Design interviews become much easier when you understand how distributed systems behave under real-world conditions.

Before your next System Design interview, make sure you understand these 10 concepts:

1. Replication

→ Why replicate data?

→ Leader vs follower

→ Synchronous vs asynchronous replication

→ Handling replica failures

2. Partitioning

→ Splitting data across multiple nodes

→ Horizontal partitioning

→ Choosing a partition key

→ Handling uneven distribution

3. Consistency

→ Strong consistency

→ Eventual consistency

→ Read-after-write consistency

→ Consistency trade-offs

4. Idempotency

The same request may arrive multiple times.

You should understand how to make operations safe to retry.

→ Idempotency keys

→ Duplicate requests

→ Retry handling

5. Distributed Locks

When multiple instances access the same resource:

→ Lock acquisition

→ Lock expiration

→ Deadlocks

→ Failure scenarios

6. Rate Limiting

Protect services from excessive traffic.

Common approaches:

→ Token Bucket

→ Leaky Bucket

→ Fixed Window

→ Sliding Window

7. Message Delivery

When using queues, understand:

→ At-most-once

→ At-least-once

→ Exactly-once

→ Duplicate messages
→ Ordering
8. Fault Tolerance
Services will fail.
Be prepared to discuss:
→ Timeouts
→ Retries
→ Circuit Breakers
→ Failover
→ Graceful degradation
9. Hotspots
A system can be scalable overall and still have one overloaded component.
Think about:
→ Hot keys
→ Uneven partitions
→ Popular users
→ Traffic spikes
10. Observability
A distributed system is difficult to debug without visibility.
Understand:
→ Logs
→ Metrics
→ Traces
→ Alerts
→ Health checks
What stood out:
System Design isn't only about drawing:
Client → API → Database
The interviewer wants to know what happens when the system is distributed, traffic increases, data is replicated, and components start failing.
A strong System Design answer should connect:
Requirements → Architecture → Data → Scale → Failure → Recovery
Learn these concepts well and you'll have a much stronger foundation for SDE-2 and senior-level System Design interviews.
#SystemDesign #SystemDesignInterview #DistributedSystems #BackendEngineering #SoftwareEngineering #SDE2 #Scalability #Microservices #SystemArchitecture #InterviewPreparation

#12 Neo Kim

AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter
4d · **View on LinkedIn**

If you want to get dangerously good at generative AI,
then save these 10 GitHub repos:
1 Generative AI for Beginners
→ https://lnkd.in/eATfVXBF
2 LLM Course
→ https://lnkd.in/eHDx-_dN

3 Build a Large Language Model From Scratch
→ https://lnkd.in/e5_Cg9C2
4 Prompt Engineering Guide
→ https://lnkd.in/eExpkGvE
5 Awesome LLM Apps
→ https://lnkd.in/epNdTzHm
6 OpenAI Cookbook
→ https://lnkd.in/ebqEf4uX
7 Hands-On Large Language Models
→ https://lnkd.in/ew32KNNj
8 GenAI Agents
→ https://lnkd.in/eXvcYBzi
9 Awesome Generative AI Guide
→ https://lnkd.in/eyzFW7ke
10 Anthropic Courses
→ https://lnkd.in/eYs--pH8
Anything else you'd add?
===
PS - Want my AI Engineering Playbook for FREE?
Try this:
→ https://lnkd.in/eXVZ5Zg6
===
Repost to help others learn generative AI.
Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eATfVXBF
- https://lnkd.in/eHDx-_dN
- https://lnkd.in/e5_Cg9C2
- https://lnkd.in/eExpkGvE
- https://lnkd.in/epNdTzHm
- https://lnkd.in/ebqEf4uX
- https://lnkd.in/ew32KNNj
- https://lnkd.in/eXvcYBzi
- https://lnkd.in/eyzFW7ke
- https://lnkd.in/eYs--pH8
- https://lnkd.in/eXVZ5Zg6

#13 Ranjan Kumar

System Design

Software Engineer at Olyv(Smartcoin)|| ex - maq software ll Java || C++ || System design || 1550+ (codeforces) || IIT Goa(CSE)

4d · **View on LinkedIn**

Netflix Interview Questions That Will Make Your Concepts Strong!
Want to crack top product-based companies? Start by mastering the fundamentals.
Here are 20 DSA and System Design questions to strengthen your problem-solving and system design skills.
SECTION 1: DSA — 10 QUESTIONS
1. Find a Triplet That Sums to a Given Value
2. Boolean Matrix Problem

3. Rearrange a Given Linked List In-Place
4. Floor in a Sorted Array
5. Second Most Repeated Word in a Sequence
6. Merge Intervals
7. Number of Islands
8. LRU Cache
9. Longest Increasing Subsequence
10. Trapping Rain Water
SECTION 2: SYSTEM DESIGN — 10 QUESTIONS
1. Design a Movie Recommendation System
2. Design a URL Shortener
3. Design a Payment Processing System
4. Design a Service for Validating and Sorting TV Series
5. Design Netflix — A Video Streaming Platform
6. Design a Distributed Cache
7. Design a Watch History and Continue Watching Service
8. Design a Content Search Service
9. Design a Distributed Rate Limiter
10. Design a High-Availability Microservices Platform
Why practice these questions?
✓ Strengthen your DSA fundamentals.
✓ Understand real-world system design.
✓ Learn scalability, caching, concurrency, and fault tolerance.
✓ Improve your problem-solving and interview approach.
Save this post and start preparing!
#Netflix #DSA #SystemDesign #LeetCode #SoftwareEngineering #CodingInterview #InterviewPreparation

#14 Raul Junco

System Design

Simplifying System Design

4d · **View on LinkedIn**

I thought Cache was only about storing my data for quick access, and I was wrong.
7 concepts you need to master cache.
Cache Hit Ratios
This is the answer to how effective your cache strategy is.
A high hit ratio indicates that the cache frequently contains the requested data.
A high hit ratio minimizes fetching it from slower layers.
Eviction Policies
They determine which data stays in the cache and which gets removed; it's about efficiency.
Common policies include:
- Least Recently Used (LRU)
- Most Recently Used (MRU)
- Least Frequently Used (LFU)
Cache Coherence
Of course, it gets more complex In distributed systems.
Maintaining data consistency across these caches is critical.

Cache coherence ensures that all copies of a data item reflect the most recent version.
Cache Granularity
This is about the size of the data units stored in the cache.
Granularity decisions impact the cache's efficiency and hit ratio.
How much data do you need to store in each operation?
Cache Warm-up
Preloading the cache with data expected to be in high demand will improve performance.
Especially in systems where cache hits are critical for speed.
Cache Partitioning
In a well-organized toolbox, you'd have separate compartments or trays for different tools.
Cache partitioning is the same. It's about dividing the cache into different sections, each tailored for a specific data type or request.
This organization improves the cache since retrieving the data is easier and faster.
Cache Compression:
When space is at a premium, compressing cached data allows you to store more information.
But, this comes with the trade-off of processing overhead during compression and decompression.
Caching isn't just dumping data somewhere and forgetting about it.
It's also about what, how, and when you store data to make your system faster.

#15 Puneet Patwari

System Design

Principal Software Engineer @Atlassian| Ex-Sr. Engineer @Microsoft | Sharing insights on SW Engineering, Career Growth & Interview Preparation

5d · **View on LinkedIn**

I’m a Principal Engineer at Atlassian with 13 years of experience. I wish I knew these in the first or second year of my career instead of learning about them years later.....
through production bugs, bad designs, and painful debugging sessions.
None of these are “advanced” in isolation. But together, they completely change how you think about system design:
→ p95 / p99 latency
→ Tail latency amplification
→ Connection pooling
→ Backpressure
→ Rate limiting
→ Idempotency keys
→ Retry with backoff + jitter
→ Circuit breakers
→ Bulkheads
→ Timeouts
→ Reading query plans
→ Index selectivity
→ Write amplification
→ Read amplification
→ Consistent hashing
→ Sharding keys
→ Hot partitions
→ Replication lag

→ Strong vs eventual consistency
→ Optimistic locking
→ Pessimistic locking
→ Change Data Capture
→ Outbox pattern
→ Dead-letter queues
→ At-least-once delivery
→ Cache invalidation
→ Cache stampede protection
→ Graceful degradation
→ SLOs / error budgets
→ Correlation IDs + tracing

Btw, if you're preparing for Senior to Principal-level system design interviews, I've put together 90+ fundamentals like these into a guide.

You can check it out here: puneetpatwari.in

The reason I'm sharing this is because a lot of backend engineering becomes much easier once you recognize the pattern underneath the incident.

For example:

API suddenly slow?

Maybe it is not CPU. Maybe every request is waiting for a DB connection.

Duplicate payment?

Maybe the retry worked exactly as designed, but the endpoint was not idempotent.

One Redis node melting?

Maybe your hashing strategy created a hot partition.

Hope this helps :)

LINKS

- http://puneetpatwari.in

#16 Chandrasekar Srinivasan

AI / LLM

Engineering and AI Leader at Microsoft

6d · **View on LinkedIn**

The best AI engineering material I read this year was free. Engineers at Anthropic, OpenAI and Google wrote down what they learned building systems, and gave it away. I read most of it on weekends.

If you're a software engineer moving into AI, these 8 reads will teach you more than most paid courses:

1. Building Effective Agents (Anthropic)

The best starting point. When you need an agent, when a simple workflow is enough, and which patterns hold up in production.

→ https://lnkd.in/gCpFkCuS

2. Effective Context Engineering for AI Agents (Anthropic)

How to decide what an agent sees, and why more context often makes results worse.

→ https://lnkd.in/gpr4yQE6

3. A Practical Guide to Building Agents (OpenAI)

A 34-page PDF on tools, instructions, guardrails and when to hand off to a human.

→ https://lnkd.in/gbyTsK9y

4. Agents whitepaper (Google)

A clear walkthrough of the model, the tools and the orchestration layer, with working examples.

→ https://lnkd.in/g3FDNUE4

5. LLM Powered Autonomous Agents (Lilian Weng)

The post that gave many of us our first mental model of planning, memory and tool use.

→ https://lnkd.in/gNiEBFiF

6. Patterns for Building LLM-based Systems & Products (Eugene Yan)

Evals, RAG, caching, guardrails and feedback loops, written by someone who shipped them.

→ https://lnkd.in/g9xgdCq2

7. Your AI Product Needs Evals (Hamel Husain)

If you read one thing about testing AI systems, make it this.

→ https://lnkd.in/g_6sKkct

8. Neural Networks: Zero to Hero (Andrej Karpathy)

For when you want to understand what's happening underneath. You build a GPT from scratch, one step at a time.

→ https://lnkd.in/gjpsBeM9

LINKS

- https://open.substack.com/pub/chandrasdoodle
- https://lnkd.in/gCpFkCuS
- https://lnkd.in/gpr4yQE6
- https://lnkd.in/gbyTsK9y
- https://lnkd.in/g3FDNUE4
- https://lnkd.in/gNiEBFiF
- https://lnkd.in/g9xgdCq2
- https://lnkd.in/g_6sKkct
- https://lnkd.in/gjpsBeM9

#17 Arslan Ahmad System Design

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

1w · **View on LinkedIn**

Most teams scale their database in the wrong order.

They hear "sharding" in a design review and jump straight to it.

Then they spend months untangling cross-shard queries.

Here are 6 ways to scale a database. The order matters as much as the technique.

1. Vertical scaling

Give the same server more CPU, RAM, and faster disks.

Zero code changes. One modern box handles far more than most people think.

↳ Limit: a hard hardware ceiling, and still a single point of failure.

2. Caching layer

Put Redis or Memcached in front of the database.

Hot reads come from memory and never touch disk.

↳ Limit: stale data. Cache invalidation is the hard part.

3. Read replicas

One primary takes all the writes. Copies serve the reads.

Perfect when your app reads far more than it writes.

↳ Limit: replication lag. A user may not see their own write right away.

4. Partitioning

Split one huge table into smaller pieces inside the same database, usually by date or range.

Queries scan less data. Old partitions are easy to archive.

↳ Limit: still one server. It fixes table size, not total load.

5. Federation

Split by feature. Users in one database, orders in another, payments in a third.

Each database gets its own hardware and its own load.

↳ Limit: no easy joins across databases.

6. Sharding

Spread rows across many servers using a key like user_id.

Each server holds only a slice of the data. This is how you scale writes past one machine.

↳ Limit: resharding, hot keys, and cross-shard queries. It's a one-way door.

The rule:

Climb a step only when the one below is maxed out.

In an interview:

Don't open with sharding. Walk up the ladder and explain why each step stops being enough. That's what senior judgment looks like.

Which step did your team skip?

#systemdesign #databases #softwareengineering #backend

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview

#18 Mudassir Maredia

System Design

Staff Software Engineer | Distributed Systems | SaaS | Microservices | AI Agents

1w · **View on LinkedIn**

Caching improves performance. Until it becomes your consistency problem.

Caching is often one of the first solutions we consider when an application experiences latency or database pressure. And for good reason: why repeatedly query a database for information we could serve from memory?

But a cache introduces another copy of our data, with its own lifecycle and consistency challenges.

Consider a typical cache-aside architecture.

An application checks Redis before querying the database. On a miss, it reads from the database, populates the cache, and returns the result.

Now imagine an e-commerce application caching product availability.

Request A misses the cache and reads the current stock count from the database.

Before A populates the cache, Request B purchases an item, updates the database, and invalidates the cache key.

Request A then writes the older stock count into Redis.

The database is correct. The cache is stale. And subsequent requests may keep receiving outdated availability information.

That's the subtle part: invalidating a cache after a database update doesn't necessarily prevent an earlier in-flight read from repopulating it with stale data.

So what can we do?

TTL policies bound how long stale entries survive, but don't guarantee freshness.

Versioned entries and conditional writes can help reject older values, provided versions are checked against an authoritative source.

Coordinated invalidation and read/write protocols can offer stronger guarantees, but add complexity. And for critical operations, the database should remain the authority rather than trusting cached availability.

Consistency isn't the only concern.

When a popular key expires, thousands of requests can miss simultaneously and overwhelm the database. That's a cache stampede. Request coalescing, expiration jitter, and stale-while-revalidate can help, depending on the workload.

A product description may tolerate a few seconds of staleness. An inventory reservation or financial balance may not.

Both can use Redis. They shouldn't necessarily use the same caching strategy.

To me, the architectural question isn't simply whether caching improves latency. It's what freshness guarantees the business needs, how the system behaves during concurrent updates, and what happens when the cache fails.

Caching is a performance optimization. Keeping it correct is a systems-design decision.

P.S. Diagram created with ChatGPT to illustrate this race condition. Do you use AI for architecture diagrams, or prefer Mermaid, PlantUML, or draw.io? How do you check their technical accuracy?

#SoftwareArchitecture #DistributedSystems #SystemDesign #BackendEngineering

LINKS

- http://draw.io

#19 Ankur Dhawan

System Design

MTS-2 @Adobe | 112K+ @LinkedIn | 50+ Brand Collabs | AI • Tech • Marketing

1w · **View on LinkedIn**

I cracked Expedia Group in my first attempt!

Here are 20 System Design problems I repeatedly practised before the interviews:

1. Design Expedia
2. Design a Hotel Booking System
3. Design a Flight Search Engine
4. Design a Reservation System
5. Design a Payment System
6. Design a Hotel Availability Service
7. Design a Flight Price Tracking System
8. Design a Search Autocomplete System
9. Design a Hotel Recommendation System
10. Design a Pricing and Discount Engine
11. Design a Cancellation and Refund System
12. Design a Travel Notification System
13. Design an Itinerary Management System
14. Design a Review and Rating System
15. Design a Distributed Rate Limiter
16. Design a URL Shortener
17. Design a Booking History Service
18. Design a Distributed Caching System
19. Design an Event-Driven Booking Architecture
20. Design a Travel Deals Aggregator

Checkout all questions + experience:

https://lnkd.in/gjsn957f

Atlassian Resources: https://lnkd.in/gfxB9sXh
4500+ Recruiter Contacts: https://lnkd.in/ghHxbXsf
DSA Roadmap: https://lnkd.in/g2wrixSY
LinkedIn
LinkedIn Guide to Creating
#sde1 #sde2 #sde3 #hiring #layoff #explore #jobHunt

LINKS
- https://lnkd.in/gjsn957f
- https://lnkd.in/gfxB9sXh
- https://lnkd.in/ghHxbXsf
- https://lnkd.in/g2wrixSY

#20 Adnan Khan

System Design

Java Backend Developer | Spring Boot | Microservices | Kafka | REST APIs | Hibernate | SQL | AWS | Redis | CI/CD
1w · **View on LinkedIn**

4 Rounds. ₹95 LPA–₹1.3 Crore. The Atlassian Interview Breakdown!
Role: Senior Software Engineer (P50)
Round 1: Karat Screen
Identify architectural flaws in a package delivery system.
Analyze load balancers, authentication, servers, and database availability.
Coding: Word formation using letters and grid-based word search.
Round 2: Code Design
Design a cinema scheduling system operating from 10 AM to 11 PM.
Implement canSchedule(movie, schedule) without overlapping bookings.
Follow-ups: Replace movies and calculate revenue.
Round 3: Data Structures
Implement upsertCommodityPrice(timestamp, price) and getMaxCommodityPrice().
Handle out-of-order timestamps and updates correctly.
Example: After the updates, prices are 20, 26, and 25. What is the maximum? Answer: 26.
Round 4: System Design
Design a popular feed for Confluence.
Track views, likes, and shares.
Calculate the top N pages.
Support real-time updates and high traffic.
Result
According to the shared interview account, the candidate was down leveled from P50 to P40 and did not receive an offer.
Key takeaway: Strong DSA, clean code, and scalable system design are essential for senior engineering interviews.
Prepare with my Java Backend Interview Success Kit covering Java, Spring Boot, Microservices, Kafka, DSA, SQL, and System Design.
→ https://lnkd.in/dst_Ggbs
Keep Learning. Keep Growing. Keep Improving!
#Atlassian #JavaDeveloper #SystemDesign #CodingInterview #BackendDeveloper #InterviewPreparation

LINKS
- https://lnkd.in/dst_Ggbs

#21 Nelson Djalo

System Design

I help developers get hired, from first job to senior engineer | Founder of Amigoscode, 1.1M on YouTube | Java, Spring Boot, System Design, DevOps, AI Engineering | Team training

1w · View on LinkedIn

A URL shortener looks easy

Until you ask where the short IDs come from

Every box on the board has to answer a number

So start with the numbers, not the boxes

Here is the mental model you should have

→ 100M new links a month is about 40 writes a second

→ 100 reads for every write means about 4,000 redirects a second

→ Hot codes live in a cache, so most redirects never touch the database

→ IDs come from pre-allocated counter ranges in base62: 7 characters give about 3.5 trillion codes

→ Stateless app servers scale out behind a load balancer

→ Redirect with 302, not 301, or browsers cache it and your analytics go quiet

→ Clicks go to a queue and get counted later, so analytics never slow a redirect

The mistake many candidates make is drawing boxes first

Load balancer, database, cache, all before a single number

Senior engineers size it before they draw it

40 writes and 4,000 reads a second says it is a read problem, so the cache comes first

Hashing or ID ranges: which would you pick, and why?

#systemdesign #softwareengineering #backend

MEDIA design bitly whiteboard

LINKS

- http://skool.com/amigoscode-academy

#22 Aashish Gupta

System Design

Sr Software Engineer@Uber | Mentoring Engineers in the AI Era | Backend & System Design | Building Scalable Systems | Brand Collabs

1w · View on LinkedIn

Microservices aren't just about breaking a monolith into smaller services.

They're about managing the complexity that comes with distributing a system.

Here are 12 microservices patterns worth understanding:

→ API Gateway: One entry point for routing requests, authentication, and rate limiting.

→ Saga: Coordinates distributed transactions through local transactions and compensating actions.

→ Event Sourcing: Stores changes as events instead of only maintaining the latest state.

→ CQRS: Separates read and write models when their requirements differ.

→ Strangler Fig: Replaces legacy functionality incrementally instead of rewriting everything at once.

→ Service Discovery: Helps services locate each other in dynamic environments.

→ Circuit Breaker: Stops repeated calls to a failing dependency from making an outage worse.

→ Bulkhead: Isolates resources so one overloaded component doesn't exhaust resources needed by others.

→ Database per Service: Gives each service ownership of its data, reducing coupling between services.

→ Sidecar: Moves supporting capabilities, such as logging or networking, into a separate component.

→ Retry: Handles transient failures with bounded retries, backoff, and jitter.

→ API Composition: Combines data from multiple services to produce a single response.

But here's the important part:

Knowing these patterns is one thing. Knowing when to use them is another.

The real engineering skill is understanding the problem first, then choosing the simplest pattern that solves it.

Good architecture isn't about using more patterns. It's about making the right trade-offs.

Resources:

Microsoft - Microservices Design Patterns

https://lnkd.in/guztYvnR

AWS - Cloud Design Patterns

https://lnkd.in/gNWxfs64

#Microservices #SystemDesign #SoftwareArchitecture #BackendEngineering #DistributedSystems

LINKS

- https://lnkd.in/guztYvnR
- https://lnkd.in/gNWxfs64

#23 Crispus Roshan

Kafka / Messaging

AI Data Engineer @ Meta | Educating the Next Gen of Data & AI Engineers | Founder @ Stackle | Send your resume at cris@stackle.io for ATS approved Resume | US Citizen | Book an appointment

1w · **View on LinkedIn**

I interviewed at Airbnb in 2025 for a Data Engineering role.

Here’s the exact system design question they asked, and what they wanted to see ↓

“Design a data warehouse for Airbnb.”

Most people start drawing Kafka in the first minute.

That’s the wrong start.

The wrong way:

→ Name every tool you know

→ Draw a pipeline nobody asked for

→ Never say what the business is trying to measure

The right way:

Start with the metric.

Bookings, cancellations, host earnings, review scores. Ask which one matters before you draw anything.

Model it as a star schema.

booking_fact and review_fact in the middle. listing, user, date and location as dimensions.

Partition by how people query.

Most Airbnb questions are about a city or a region. Partition by location and watch for hot partitions.

Don’t skip the reviews.

Reviews are free text. Run sentiment on them and store it as something you can filter on.

Only now do you talk about tools.

Kafka for ingest.

Flink for real time

Spark for batch.

Bronze, silver and gold on Iceberg.

Airflow to run it all.

But the tools were the last 20% of the round.

The first 80% was the questions.

What's the first question you ask in a system design round before you draw anything?

LINKS

- https://calendar.app.google/2HjLnF1HSov9m5Gk9

#24 Naushad A.

Java / Spring

Senior Specialist | 509k Engagement | Java 8 & 21 | Spring Boot | Microservices | AWS | Ex-@Capgemini

1w · **View on LinkedIn**

#Barclays Interview Experience | Senior Java Developer↓

✓ All the Answer for the interview Questions including All Java concepts, are available in Single Kit - https://lnkd.in/dW_5jZie

1. What happens internally when you execute HashMap.put(key, value)?

2. Why is String immutable in Java? How does String Pool work, and what are the security/ performance benefits of immutability?

3. What is the difference between synchronized, volatile, AtomicInteger, and Lock? Give a real-world example where each would be appropriate.

4. Consider a class used as a key in a HashMap. What happens if you modify one of its fields after inserting it into the map? Explain the consequences.

5. Explain Java's equals() and hashCode() contract.

7. Given a list of employees, find the second-highest salary in each department using Java Streams, while handling duplicate salaries.

8. How would you find duplicate elements in a large list using Streams?

9. What is the difference between map() and flatMap()? Give a practical example involving List<List<Employee>>.

10. When should you avoid parallelStream() in a production application? Explain issues related to thread pools, shared state, ordering, and I/O operations.

11. How would you process millions of transactions using Java Streams without causing excessive memory consumption?

12. Two threads update the same bank account simultaneously. How would you design the solution to prevent race conditions?

13. Explain the difference between ExecutorService, CompletableFuture, and virtual threads. When would you choose each for a backend application?

14. What is a deadlock? Given four threads and multiple locks, how would you identify, prevent, and troubleshoot a deadlock in production?

15. Explain the difference between Future.get() and CompletableFuture.

16. Suppose an API receives 10,000 requests per second and each request performs CPU-intensive processing.

17. Explain Spring Boot's dependency injection lifecycle.

18. What is the difference between @Component, @Service, @Repository, and @Controller? Is there any functional difference between them?

19. You have a Spring Boot application where two beans implement the same interface. How does Spring decide which bean to inject?

20. How would you implement global exception handling for a REST API?

21. Design a payment processing microservice for a banking application. Explain API design, database transactions, idempotency, retries?

22. A downstream service is taking 10 seconds to respond, causing your API's response time to increase. How would you solve this using timeouts, retries?

23. Kafka consumer processes the same payment event twice. How would you make the consumer idempotent?

24. How would you use ElastiCache to improve microservice performance?

25. Design a highly available Banking Transaction API using Java, Spring Boot, Kafka, Redis, SQL, and AWS. Explain:

✓Get the Java Developer Kit here - https://lnkd.in/dW_5jZie

LINKS

- https://lnkd.in/dW_5jZie

#25 Ravi Kant

Kafka / Messaging

Lead Backend Engineer | Distributed Systems | Java | AWS | Event-Driven Architecture | Payments | Cloud-Native Microservices | System Design | AWS Certified Solutions Architect

1w · **View on LinkedIn**

What happens when the bank processes a payment, but our service times out before receiving the response?

This is where the design matters.

- Idempotency to avoid duplicate payments.

- Payment state management to track pending, successful and failed transactions.

- Webhooks and reconciliation to resolve uncertain payment outcomes.

- Transactional outbox + Kafka to reliably notify downstream services.

A timeout doesn't always mean a failed payment.

Before retrying, we need to know what actually happened.

#SystemDesign #Payments #DistributedSystems #BackendEngineering

MEDIA high level design payment gateway

#26 Neo Kim

AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1w · **View on LinkedIn**

If you want to become S-tier with AI agents,

read these 10 guides (save now):

1. Effective Context Engineering for AI Agents

↳ Teaches you what enters the context window

https://lnkd.in/eTnbAJpF

2. AI Agents for Beginners

↳ Teaches agent design with Python code samples

https://lnkd.in/eJw8TkSi

3. How We Built Our Multi-Agent Research System

↳ Teaches the lead agent + sub-agent architecture

https://lnkd.in/eEpZC2Fv

4. Agents SDK Documentation

↳ How to build agents: handoffs, guardrails & tracing

https://lnkd.in/eqbrBUrc

5. Writing Effective Tools for AI Agents

↳ How to design & evaluate tools that agents actually use correctly

https://lnkd.in/eKcR3cgx

6. AGENTS-md
↳ Standardizes a "README for agents"
https://agents.md/
7. Startup Technical Guide: AI Agents
↳ Takes you from prototype to production with ADK, MCP & A2A
https://lnkd.in/eDPiCHTK
8. Agent Development Kit Docs
↳ Open-source framework for building & deploying multi-agent systems
https://adk.dev/
9. AI Agent Orchestration Patterns
↳ Compares sequential, concurrent, group chat, handoff & magentic patterns
https://lnkd.in/eaUtvdW7
10. MCP for Beginners
↳ Explains MCP with examples
https://lnkd.in/enkDTY8A
Anything else you'd add to this list?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 250K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others ace AI agent engineering.
Follow Neo Kim + turn on notifications.

MEDIA document/carousel (possible)

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eTnbAJpF
- https://lnkd.in/eJw8TkSi
- https://lnkd.in/eEpZC2Fv
- https://lnkd.in/eqbrBUrc
- https://lnkd.in/eKcR3cgx
- https://agents.md/
- https://lnkd.in/eDPiCHTK
- https://adk.dev/
- https://lnkd.in/eaUtvdW7
- https://lnkd.in/enkDTY8A
- https://lnkd.in/ehTrcyak

#27 Ranjan Kumar

System Design

Software Engineer at Olyv(Smartcoin)|| ex - maq software ll Java || C++ || System design || 1550+ (codeforces) || IIT Goa(CSE)

1w · **View on LinkedIn**

JPMorgan Chase Interview Questions | DSA + SQL + Web/Frameworks + System Design

JPMorgan Chase (JPMC) is a popular target for software engineers, but how well are you prepared for its technical interviews?

Here are the key interview questions to practice, along with the reported compensation ranges for different engineering levels.

CTC (Approximate)

SDE-2: ₹22–35 LPA

SDE-3: ₹32–42 LPA

Note: Compensation varies by experience, location, role, and bonus structure. These figures are indicative, not guaranteed offers.

DSA/Coding

1. Find the middle node of a linked list in one pass.
2. Print the left view of a binary tree.
3. Detect a cycle in a graph.
4. Find the Nth Fibonacci number.
5. Merge overlapping intervals.
6. Reverse a linked list in groups of K.
7. Find the Longest Common Subsequence between two strings.
8. Check whether a binary tree is height-balanced.
9. Find the Kth largest element in an unsorted array.
10. Use multithreading to compute two sums and multiply their results.

Database (SQL)

1. Difference between clustered and non-clustered indexes.
2. Find duplicate records without using GROUP BY.
3. Find the second-highest salary without using LIMIT or TOP.
4. Design a database schema for a trade settlement system.

Web/Frameworks

1. How does @Transactional work internally in Spring, and why can it fail on private or protected methods?
2. Difference between ExecutorService, ForkJoinPool, and a plain Thread.
3. How do you make a HashMap thread-safe without using ConcurrentHashMap?
4. Difference between @Async and a manually managed thread pool in Spring.
5. How does Spring resolve autowiring when multiple beans of the same type exist?
6. What is a circuit breaker, and how does it differ from a bulkhead?
7. Difference between Kafka and a traditional message queue like ActiveMQ.
8. How would you design an idempotent Kafka consumer?

System Design

1. Explain the architecture of one of your recent projects.
2. Design a real-time fraud detection system for card transactions.
3. Design a trade settlement system that reconciles trades across multiple exchanges.
4. Design an auditable payment ledger that retains records for years.
5. Design a system to process millions of transactions within an overnight batch window.

Server/System

1. How do you capture and analyze a JVM heap dump to find a production memory leak?
2. How would you debug a thread stuck while holding a lock under high load?
3. How would you diagnose a sudden CPU spike without recent code changes?
4. How would you investigate a JVM crash that occurred overnight?

Save this list for your JPMC interview preparation!

Have you interviewed with JPMorgan Chase recently? Share your interview experience and the questions you faced.

#JPMorganChase #JPMC #SoftwareEngineer #DSA #Java #SQL #SystemDesign #InterviewPreparation #CTC

#28 Sumit Goyal

Java / Spring

SDET | 30+ Brand Collabs | Marketer | Making You Interview Ready | Udemy Instructor | 70K+ Family | Follow For Latest Interview Questions | Selenium | Playwright | Performance | API | AI

1w · **View on LinkedIn**

Sapient Interview: 52% hike

Round 1 (virtual) : 1 hour

Questions asked from API , SELENIUM

Q1) WAP to find last non repeating character from a string

Q2) WAP to find the count of each character of a string.

Q3) Selenium 3 vs Selenium 4

Q4) Difference between page object vs Page factory

Q5) Collection vs Collections difference

Q6) How long is your sprint ?

Q7) How you will handle elements which are inside shadow dom

Q8) Tell me how HashMap works internally.

Round 2 : (1 hour) From Sapient Office

I was expecting it will be face to face , but again it was virtual interviewer was sitting in different city , only thing is HR will provide his laptop and from there you need to join on teams. 1 hour round , asked from api , java , selenium ✓

Q1 : Suppose a website is not ready then how you will do testing , how you will collaborate with team.

Q2 : How you will resolve conflict in your team (can anyone answer what is best way in comments)

Q3 : How you will put validation in rest assured write code

Q4 : Bug life cycle explain

Q5 : Explain difference between severity and priority for a defect

Q6 : What is most challenging thing you faced while designing framework ?

Q7) What is functional interface ?

Q8) What is Comparable and Comparator in Java ?

Q9) What is multi threading in Java ?

Q10) How will you throw custom exception in java ?

Q11) Difference between Request Specification and Response Specification.

(Interviewer was a good person he was trying to add on to the answers which I gave , learned a lot from him)

Round 3:

Behavioural Round (virtual)

A lady took my interview , here you just need to be confident like she asked me why you want to join sapient ? what are its core values ?

Round 4 : HR

(salary discussion and notice)

HR was supportive

can anyone answer Q11 ? RequestSpecification vs ResponseSpecification in comments ? Let see this question very frequently asked

#29 Dinesh Solanki Other

Senior Consultant @ EY | Java Microservices & Cloud-Native Systems | Cross-Border Team Collaboration | Security & IAM (Keycloak, OAuth2) | DevOps | OSS contributor | Expertise in AI-assisted development

1w · View on LinkedIn

“Sorry, we made you grind LeetCode.”

HackerRank, I appreciate the apology. But what about the interviews I already lost to such things?

I've never liked LeetCode-style interview questions.

Not because I don't want to learn algorithms. Not because I don't believe in problem-solving. But because I've always questioned whether solving a puzzle under pressure is a reliable measure of how good someone is at building software.

I've lost interviews because of these rounds.

And like many developers, I've seen how much time and energy goes into preparing for problems that have very little connection to the work we actually do every day.

As a Java developer, my actual work involves designing APIs, debugging production issues, understanding distributed systems, handling failures, writing maintainable code, and making trade-offs that affect real users.

Yet, in an interview, months of practical experience can sometimes take a back seat to whether you remember the right trick for a problem you've never seen before.

The irony? You can become very good at passing coding interviews without necessarily becoming a better software engineer.

And you can be a capable engineer who struggles to demonstrate their abilities in a timed algorithmic round.

Those two things shouldn't be confused.

I'm not arguing that data structures, algorithms, or coding assessments have no place in hiring. They absolutely do.

I'm arguing that the interview should reflect the job we're hiring someone to do.

With AI changing how developers write, test, review, and ship code, this conversation is even more important. We need engineers who can reason about systems, evaluate solutions, identify mistakes, and take responsibility for what reaches production.

Maybe it's time to stop optimizing candidates for interviews and start designing interviews that actually evaluate engineering.

To everyone who ever walked out of a coding interview thinking, “I'm better at my job than I was able to show in that room” — I get it.

Have we spent too many years measuring interview preparation instead of engineering ability?

#SoftwareEngineering #TechHiring #LeetCode #DeveloperExperience #HackerRank

#30 Alamin Hossain

AI / LLM

AI & Tech Creator with 213k followers | Helping you achieve 10x growth at work and beyond through Artificial Intelligence. Want to feature on my profile? → iamchonchol94@gmail.com | View my newsletter

1w · View on LinkedIn

NEVER UPLOAD YOUR CV INTO CHATGPT AND ASK, "CAN YOU FIX MY CV?"

Better prompts = Better results.

Try these 6 prompts instead and see the difference ↓

1. The CV X-Ray Analyzer

Paste into ChatGPT: "You are a senior HR professional with 15 years of recruiting experience in [industry]. Analyze my CV like you're screening a real candidate. Tell me: (1) which sections make you skip immediately, (2) achievements that are too weak & need strengthening, (3) the one thing that would get me rejected in the first 10 seconds. Be brutal. Don't be nice."

2. The Job Description Decoder

Paste into ChatGPT: "Here is the job description: [paste JD]. Break it down section by section & give me: (1) hidden skills not written but definitely expected, (2) red flags if a candidate is missing them,

(3) the 3 must-have keywords that must appear in my CV, (4) what the hiring manager actually cares about that the JD never says directly."

3. The ATS Bypass Optimizer

Paste into ChatGPT: "My CV keeps failing initial screening. Act as an ATS specialist. Here is my CV: [paste]. Here is the job description: [paste]. Identify every gap between the two. Then rewrite my Experience and Skills sections using the exact keywords from the JD naturally, not stuffed. Target score: 90+ on ATS. Show me before and after for every change."

4. The Achievement Transformer

Paste into ChatGPT: "Here are my current job descriptions on my CV: [paste bullet points]. Every single one describes a duty not a result. Rewrite each one using this formula: Strong action verb + what I did + measurable result + business impact. If I haven't given you numbers, suggest realistic placeholders I can verify. Make a recruiter stop scrolling."

5. The Recruiter Mind Reader

Paste into ChatGPT: "You are a recruiter who just received 200 CVs for this role: [paste JD]. You have 6 seconds per CV. Here is mine: [paste CV]. Tell me: (1) do you shortlist me or skip me in those 6 seconds and why, (2) what is the first thing that catches your eye, (3) what is the first thing that kills my chances, (4) what would make you call me immediately."

6. The Final CV Audit

Paste into ChatGPT: "Perform a complete audit of my final CV: [paste]. Check every section: ATS compatibility, keyword strength, achievement quality, formatting clarity, summary impact, skills relevance, length appropriateness. Score each section out of 10. Tell me exactly what is still weak & the precise fix for each. I want this CV to score 90+ before I send it anywhere."

Want to master ChatGPT?

Don't know where to start?

Get my Bestseller Prompts Book and ChatGPT Bible.

You can use it for your job, daily life, and in your business.

→ Get here: https://lnkd.in/eQrDuXtv

Follow Alamin Hossain for more AI content

Bonus Alert

Learn AI in 3 minutes a day for FREE!

Subscribe now and receive:

* 50+ AI courses

* 1000+ Advanced Prompts

* 30+ ChatGPT Cheatsheets and more

100% FREE

https://lnkd.in/ezuqAaFH

LINKS

- https://lnkd.in/ezuqAaFH
- https://lnkd.in/eQrDuXtv
- https://www.linkedin.com/feed/update/urn:li:activity:7505634345194242049

ALSO SAVED AS Alamin Hossain (1mo) link

#31 Harish kumar

AI / LLM

Helping 625K+ Professionals Navigate AI & Tech | AI Content Creator |AI & Marketing Consultant| ATS Resume Expert | Weekly Insights That Actually Move Careers Forward | Open to collaborations| harishkumar.sh100@gmail.com

1w · View on LinkedIn

Stanford's full LLM lecture series is free.

All 9 lectures.

Bookmark this for your next AI study sprint.

CME 295 (Autumn 2025) takes you from Transformers to Agentic LLMs in order:

Lecture 1: Transformers

https://lnkd.in/dDNUB5Wr

Lecture 2: Transformer-Based Models

https://lnkd.in/d49PUJ-h

Lecture 3: Large Language Models

https://lnkd.in/dARWCHdf

Lecture 4: LLM Training

https://lnkd.in/dyeXKRNx

Lecture 5: LLM Tuning

https://lnkd.in/dKNYB558

Lecture 6: LLM Reasoning

https://lnkd.in/du9YXSR2

Lecture 7: Agentic LLMs

https://lnkd.in/dUUwGF-T

Lecture 8: LLM Evaluation

https://lnkd.in/df-yudsJ

Lecture 9: Recap & Current Trends

https://lnkd.in/d44qJ6C8

Save this for your next learning sprint.

Repost it for someone learning LLMs.

Follow Harish Kumar for more AI & Career resources

LINKS

- https://lnkd.in/dDNUB5Wr
- https://lnkd.in/d49PUJ-h
- https://lnkd.in/dARWCHdf
- https://lnkd.in/dyeXKRNx
- https://lnkd.in/dKNYB558
- https://lnkd.in/du9YXSR2
- https://lnkd.in/dUUwGF-T
- https://lnkd.in/df-yudsJ
- https://lnkd.in/d44qJ6C8
- https://www.linkedin.com/feed/update/urn:li:activity:7509197483193880576

ALSO SAVED AS Harish kumar (3w) link

#32 Sampath Kasoji

System Design

Machine Learning Engineer @ ServiceNow | Gen AI & Agentic Workflows | DSA & System Design | Ex-Experian

1w · View on LinkedIn

I came across a Uber L5A Senior Software Engineer interview experience on LeetCode Discuss.

The candidate had 7 years of experience and was interviewing for a Senior Software Engineer role in Bangalore.

The process had 5 stages:

Screen — Coding

The screening round focused on implementing the core of a rate limiter in C++.

→ Implemented using a queue
→ Handled incoming requests
→ Discussed rate limits
→ Considered edge cases
Round 1 — Coding
The round included:
→ Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
The problem focused on maintaining a valid sliding window while tracking the minimum and maximum values.
Round 2 — LLD
The candidate was asked to design a running-code version of a Twitter-like news feed.
The system included:
→ Users can follow and unfollow other users
→ Users can post
→ Personalized news feeds
→ Feed customization
→ Post ranking
Round 3 — HLD
The problem was to design a stock alerts system where users receive notifications when stock prices move above or below a specified threshold.
The discussion covered:
→ Stock price updates
→ Alert processing
→ Notifications
→ Scalability
→ Data flow
→ Real-time processing
Round 4 — Hiring Manager
The discussion focused on projects, ownership, and leadership.
→ Most complex project
→ Handling conflict
→ Critical feedback from a manager
→ On-call incidents
→ Mentoring others
Final Outcome — Positive
All rounds were positive.
What stood out:
This interview loop covered almost the complete spectrum expected from a senior software engineer:
Coding → LLD → HLD → Projects → Leadership
The LLD round tested the ability to design and implement a real-world system.
The HLD round focused on scalability and real-time processing.
And the Hiring Manager round went beyond technical skills into ownership, conflict handling, mentoring, and production experience.
For senior-level interviews, preparation needs to go beyond DSA.
Be ready to demonstrate:
Coding → Design → Scalability → Ownership → Leadership

#InterviewExperience #Uber #SeniorSoftwareEngineer #SSE #SDE2 #DSA #LLD #SystemDesign #HLD #BackendEngineering #CodingInterview #InterviewPreparation

#33 Hina Arora

AI / LLM

Helping Tech Leaders Build Their Personal Brand and Industry Authority | Founder @BrandBuilders | Tech / AI / Career Content Creator (500K+) | 10+ YEO in Tech & AI | Ex-Engineering Manager @Jio

1w · View on LinkedIn

I run a team of 12. None of them are human.

No payroll. No meetings. No handover delays.

Just me as CEO, and a very specific tool doing a very specific job. Here's my org chart:

1. The Chief of Staff: Claude

• Everything flows through here first

• Thinking, structuring, long form writing, decisions

• Insight: You need one tool that holds context. Without it, you're briefing from scratch all day.

2. The Leadership Table

• Chief Research Officer: Perplexity, for sourced answers

• Chief Strategy Officer: Gemini, for a second perspective

• Insight: Never let one model be your only opinion. Different models catch different blind spots.

3. The Creative Floor

• Chief Content Officer: ChatGPT

• Creative Director: Midjourney

• Head of Design: Canva AI

• Insight: Ideas and execution are different jobs. Stop asking one tool to do both.

4. The Media Team

• Video Director: Google Flow

• Voice Director: ElevenLabs

• Video Editor: Descript

• Insight: Video used to need a crew and a budget. Now it needs three tools and an afternoon.

5. The Operations Layer

• Head of Engineering: Cursor

• Head of Automation: n8n Agents

• Head of Knowledge: Notion Agents

• Insight: This is the layer most people skip, and it's the one that turns tools into a system that runs without you.

What most people get wrong

They collect tools instead of assigning roles.

Twelve subscriptions, no clarity on which does what, so they default to one for everything and get mediocre output across the board.

The actual shift

The skill is no longer using AI. It's delegation.

Knowing which task goes to which tool, in what order, and where you personally still need to sit in the chair.

You don't need more tools. You need clearer roles.

Which seat is empty on your team right now?

I (Hina Arora) help tech executives position themselves online so they grow professionally.

DM me "Branding" if you want to position yourself on LinkedIn.

#AI #PersonalBranding #Productivity #AITools

#34 Shubham Kamboj

Java / Spring

Lead Software Engineer | Java | Spring Boot | Microservices | Kafka | AWS | System Design | Distributed Systems

1w · View on LinkedIn

#Barclays Asked Interview Questions in Technical Round (October, 2026) ↓

✓ All The Answer of below Questions is available in this Kit - https://lnkd.in/dNSPjZg8

Interview Handbook : https://lnkd.in/gzPNAC63

✓ Core Java & Concurrency

1. Race condition with a banking example (two concurrent debits)?

2. Methods are synchronized but balances are inconsistent. Why? How do you investigate?

3. putIfAbsent vs computeIfAbsent in ConcurrentHashMap?

4. Why no null keys/values in ConcurrentHashMap?

5. Plain HashMap shared across threads: can it cause hangs or timeouts?

6. Design thread-safe, idempotent payment processing in Java 21. Where do virtual threads help, and where not?

✓ Performance & Troubleshooting

7. API went from 100 ms to 4-5 s. How do you investigate (traces, thread dumps, GC, pools, DB)?

8. Requests time out but the service looks “up”. What do you check, and how do you mitigate first?

✓ Databases

9. PostgreSQL vs MySQL MVCC? Which isolation level for a ledger?

10. Optimistic vs pessimistic locking: how do you prevent double-spend and lost updates?

11. JPA CascadeType/orphanRemoval risks with financial records? Audit and soft deletes?

✓ Microservices & Kafka

12. How do you avoid lost or duplicate transactions (outbox, Saga, idempotency keys, Kafka EOS)?

13. Design a funds-transfer flow with retries, DLQ, and reconciliation.

14. HA and DR design with RTO/RPO for a tier-1 banking service?

✓ Security & Compliance

15. Securing APIs: OAuth2/OIDC, JWT pitfalls, mTLS, secrets?

16. PII, audit trails, and data retention in a regulated setup?

✓ Delivery & Leadership

17. Trunk-based vs GitFlow? Branch protection (reviews, status checks, CODEOWNERS)?

18. Safe releases in a regulated bank (gates, canary/blue-green, flags, rollback)?

19. A production incident you led: mitigation, comms, postmortem?

20. Mentoring, code quality (TDD, pairing, reviews), and safe Copilot use under compliance?

Like and reshare if this helps

✓ Get the Guide here - https://lnkd.in/dNSPjZg8

Interview Handbook : https://lnkd.in/gzPNAC63

If you’ve read so far, do LIKE and RESHARE the post

#Java #JavaDeveloper #SpringBoot #Microservices #BackendDevelopment #SystemDesign #SoftwareEngineering #CodingInterview #TechInterview #InterviewPreparation #Barclays #BarclaysInterviewQuestion

MEDIA advertisement

LINKS

- https://lnkd.in/dNSPjZg8
- https://lnkd.in/gzPNAC63

#35 Shubham Kamboj

Java / Spring

Lead Software Engineer | Java | Spring Boot | Microservices | Kafka | AWS | System Design | Distributed Systems

1w · View on LinkedIn

#MasterCard Asked Interview Questions in Technical Round (September, 2026) ↓

✓ All The Answer of below Questions is available in this Kit - https://lnkd.in/dNSPjZg8

Interview Handbook : https://lnkd.in/gzPNAC63

1. What is a race condition and when does it occur?
2. What is the solution to prevent race conditions?
3. What is the difference between putIfAbsent and computeIfAbsent in ConcurrentHashMap?
4. If two methods are synchronized but we still get a race condition, why does that happen?
5. Why can a race condition still occur if two methods are synchronized, and how would I investigate that?
6. In which scenario would I use a big HashMap?
7. Is there any possibility of getting a deadlock situation in HashMap?
8. Is there any possibility of getting a deadlock situation in HashMap?
9. Does ConcurrentHashMap allow null values?
10. Why does ConcurrentHashMap not allow null keys or null values?
11. How to investigate an application that normally responds in 100 ms but is now taking 4-5 seconds?
12. How to investigate an API that is responding slowly, including checking the database side and indexes?
13. What issues arise when a HashMap is shared by multiple threads in production, even if it works fine in testing?
14. Why would requests time out in an application using a shared HashMap across multiple threads, even though the application itself is not crashing?
15. Why are all requests in the application timing out with exceptions, even though the application is running on a server?
16. In which scenarios will you get concurrency issues?
17. What branching strategies are available in GitHub and how can you configure branch protection rules?
18. What databases have you worked with? How does PostgreSQL handle MVCC differently from MySQL?
19. Do you know what is cascade type as well as orphan removal?
20. Have you used any locking mechanism in your code? Like database level locking?

I have prepared Complete Interview Preparation Guide for Java Engineers.

✓ Get the Guide here - https://lnkd.in/dNSPjZg8

Interview Handbook : https://lnkd.in/gzPNAC63

If you've read so far, do LIKE and RESHARE the post

#Java #JavaDeveloper #SpringBoot #Microservices #BackendDevelopment #SystemDesign #SoftwareEngineering #CodingInterview #TechInterview #InterviewPreparation #MasterCard #MastercardInterviewQuestion

LINKS

- https://lnkd.in/dNSPjZg8
- https://lnkd.in/gzPNAC63
- https://www.linkedin.com/feed/update/urn:li:activity:7507451485253857280
- https://www.linkedin.com/feed/update/urn:li:activity:7506743948657184768
- https://lnkd.in/dW_5jZie

ALSO SAVED AS Shubham Kamboj (1mo) link; Naushad A. (1mo) link

#36 Neo Kim

AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1w · View on LinkedIn

If you want to get GOD-LEVEL with AI agents,
bookmark these 10 GitHub repos:
1 Awesome LLM Apps
↳ Working AI agent & RAG projects with full source code you can run
https://lnkd.in/epNdTzHm
2 Generative AI for Beginners
↳ Teaches GenAI fundamentals in structured lessons
https://lnkd.in/eATfVXBF
3 LLM Course
↳ Walks you through the full LLM stack with roadmaps & Colab notebooks
https://lnkd.in/eHDx-_dN
4 Prompt Engineering Guide
↳ Prompting techniques behind every agent with papers, examples & lectures
https://lnkd.in/eExpkGvE
5 AI Agents for Beginners
↳ Teaches agent design with code samples & video walkthroughs
https://lnkd.in/eJw8TkSi
6 AI Agents Projects
↳ Agent use cases & links each one to an open-source implementation
https://lnkd.in/eafDd7u4
7 Hugging Face Agents Course
↳ Takes you to a certified project using smolagents, LangGraph & LlamaIndex
https://lnkd.in/einQQDTF
8 Awesome AI Agents
↳ Maps entire agent ecosystem into a list of frameworks, tools & platforms
https://lnkd.in/eX8tt2P9
9 GenAI Agents
↳ Builds agents step by step in Jupyter notebooks
https://lnkd.in/eXvcYBzi
10 Agents Towards Production
↳ How to ship agents with tutorials on memory, deployment, monitoring
https://lnkd.in/ekqbkBW5
Remember to bookmark these repos.
Anything else you'd add?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 250K+ tech professionals:
→ https://lnkd.in/ehTrcyak
===
Repost to help others ace AI agent engineering.
Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/epNdTzHm
- https://lnkd.in/eATfVXBF
- https://lnkd.in/eHDx-_dN
- https://lnkd.in/eExpkGvE
- https://lnkd.in/eJw8TkSi
- https://lnkd.in/eafDd7u4
- https://lnkd.in/einQQDTF
- https://lnkd.in/eX8tt2P9
- https://lnkd.in/eXvcYBzi
- https://lnkd.in/ekqbkBW5
- https://lnkd.in/ehTrcyak

#37 Alok Sharan

System Design

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author

1w · **View on LinkedIn**

System Design Mastery: 6 Core Pillars Explained!

After 20+ years working in enterprise technology and architecture, I've learned that great system design isn't about choosing the latest technologies.

It's about understanding how every component works together under real-world pressure.

A system that handles 100 users might struggle at 100,000. The difference often comes down to architectural decisions made early.

Here are 6 pillars every engineer and architect should understand:

→ 1. Core Fundamentals

Scalability, CAP theorem, consistency models, latency, throughput, and availability form the foundation of distributed systems.

→ 2. Networking & APIs

DNS, load balancers, API gateways, CDNs, and WebSockets determine how systems communicate efficiently.

→ 3. Data Storage

SQL, NoSQL, indexing, replication, and sharding help manage growing data and workloads.

→ 4. Caching & Messaging

Redis, Kafka, message queues, and Pub/Sub improve responsiveness and decouple services.

→ 5. Reliability & Resilience

Rate limiting, circuit breakers, retries, idempotency, and failover keep systems operational during failures.

→ 6. Architecture Patterns

Microservices, event-driven systems, CQRS, Saga, and observability support maintainable architectures.

In my experience, strong architecture isn't measured by complexity. It's measured by how well a system performs, scales, and recovers when things go wrong.

Which system design concept has challenged you the most in real-world projects?

P.S. Preparing for a software engineering role? Check out my book, System Design and Behavioral Intelligence.

It will help you:

• Build strong System Design foundations and approach design problems with confidence

• Develop practical frameworks to navigate complex behavioral and leadership challenges

Get Your Copy anywhere internationally:

US: https://lnkd.in/eVy2ACxQ

UK: https://lnkd.in/evzmHKm4

India : https://lnkd.in/e25PthqU

#SystemDesign #SoftwareArchitecture #DistributedSystems #CloudArchitecture #SoftwareEngineering

LINKS

- https://lnkd.in/eVy2ACxQ
- https://lnkd.in/evzmHKm4
- https://lnkd.in/e25PthqU

#38 Arslan Ahmad

System Design

Author of Bestselling 'Grokking' Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

1w · **View on LinkedIn**

A slow query does not always need a bigger database.

Sometimes, it needs the right index.

But "add an index" is only half the answer.

Every index takes storage and adds work when indexed data changes.

Here are 8 index choices every backend engineer should know:

1. B-tree

For equality lookups, date ranges, and sorted results.

Example: orders placed in the last 30 days.

Trade-off: writes must maintain the tree.

2. Hash

For exact-match lookups.

Example: finding a session by its token.

Trade-off: no range queries or sorting.

3. Composite

For queries that use several columns together.

Example: a user's orders, sorted by date.

Trade-off: column order shapes which queries benefit.

4. Partial

For a small subset of rows you query often.

Example: pending jobs in a table full of completed jobs.

Trade-off: the query must match or imply the index filter.

5. Covering

For queries whose required columns fit inside the index.

Example: reading order status and total from the index.

Trade-off: a larger index and more work on writes. Some databases may still need table checks.

6. Full-text

For searching words across text.

Example: matching keywords in article titles and content.

Trade-off: one document can create many index entries.

7. Spatial

For distance, location, and area queries.

Example: restaurants within 5 km.

Trade-off: candidate locations may still need exact distance checks.

8. Bitmap

For combining filters in analytics workloads.

Example: sales filtered by region, tier, and status.

Trade-off: frequent concurrent writes can cause contention.

These choices can overlap. A composite B-tree can also be partial and cover a query.

In a system design interview, go beyond naming the index:

“This query filters by user_id and sorts by created_at, so I would test an index on (user_id, created_at), then check the query plan and write cost.”

Start with the query. Choose the index. Measure the result.

Save this guide for your next design review. Repost it to help another engineer.

Full guide in the first comment.

MEDIA document/carousel (possible)

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview

#39 Ankur Dhawan

System Design

MTS-2 @Adobe | 112K+ @LinkedIn | 50+ Brand Collabs | AI • Tech • Marketing

1w · **View on LinkedIn**

Atlassian pays around 50-70LPA for SDE-1 and 60-90LPA for SDE-2

This is the classic question they generally ask

1. Design a Project Management Tool Like Jira
2. Design a Real-Time Collaboration Tool.
3. Design a Scalable Notification System
4. Design a Search System for Knowledge Base Articles
5. Design an API Gateway for Atlassian Services
6. Design a Version Control System for Documentation
7. Design a Real-Time Analytics Platform
8. Design a Scalable User Authentication and Authorization System
9. Design a Workflow Automation System
10. Design a Logging and Monitoring System
11. Design a Rate Limiter
12. Design a parking
13. Database Design
14. Design Snake Game
15. Design a Ticketing System like Jira
16. Design a URL Shortening Service
17. Design a Notification System
18. Design a Distributed Messaging System
19. Design a Scalable Chat Application
20. Design a Job Scheduler

I gave a interview, cracked them, and made some notes on how to crack interview.

Checkout: https://lnkd.in/dp6YPxZH

Atlassian Resources: https://lnkd.in/gfxB9sXh

4500+ Recruiter Contacts: https://lnkd.in/ghHxbXsf

DSA Roadmap: https://lnkd.in/g2wrixSY

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/dp6YPxZH
- https://lnkd.in/gfxB9sXh
- https://lnkd.in/ghHxbXsf
- https://lnkd.in/g2wrixSY
- https://www.linkedin.com/feed/update/urn:li:activity:7503323911594442752

ALSO SAVED AS Ankur Dhawan (1mo) link

#40 Akshat Jain

System Design

SWE @ Microsoft | 85K+ Linkedin | Tech, AI and Marketing | Codeforces Expert | LeetCode Guardian | IIT BHU'24 | Brand Collaborations

1w · **View on LinkedIn**

Resources I'm Following for System Design (Both LLD and HLD) ↓

I've started focusing more seriously on System Design, and instead of jumping between random resources, I've been following a structured set of resources.

Sharing them here in case you're also looking for System Design resources. I believe these resources are more than enough to prepare for System Design interviews for SDE-1 and SDE-2 roles.

For LLD:

CoderArmy - LLD Playlist

A completely free YouTube playlist that I've been following to understand LLD concepts and design patterns.

https://lnkd.in/eMWNb68e

Questions practice on Hello Interview: https://lnkd.in/emAiZzHh

and Ashish Pratap's LLD resource: https://lnkd.in/eiqx8zsb

For HLD:

Before jumping directly into designing large systems, I wanted to first understand the core concepts behind HLD.

Things like:

- Caching
- Load Balancers
- Networking basics
- OSI Layers
- TCP vs UDP
- CAP Theorem
- Sharding
- Replication
- Databases
- Message Queues

For this part, I followed Hello Interview and ChatGPT. The explanations are simple and help build the foundation before getting into actual system design problems.

Hello Interview: https://lnkd.in/eJims3xm

I also bought Alex Xu's System Design Interview Book: https://lnkd.in/eJk2p_6P

I'll keep adding more resources as I progress through System Design.

If you're also preparing for HLD/LLD, which resources are you following?

LINKS

- https://lnkd.in/eMWNb68e
- https://lnkd.in/emAiZzHh

- https://lnkd.in/eiqx8zsb
- https://lnkd.in/eJims3xm
- https://lnkd.in/eJk2p_6P
- https://www.linkedin.com/feed/update/urn:li:activity:7497850253971202048

ALSO SAVED AS Akshat Jain (1mo) link

#41 Rohit Turambekar

System Design

AI Solution engineering @ Microsoft | Ex-AWS

2w · **View on LinkedIn**

Most engineers learn system design by memorizing architectures.

But the real skill is understanding why a system is designed a certain way.

Found a useful GitHub repo with 28 chapters of system design notes, based on Alex Xu's popular books.

It covers:

• Fundamentals: Scaling, rate limiting, consistent hashing, capacity estimation

• Distributed systems: Key-value stores, message queues, monitoring

• Real-world architectures: YouTube, WhatsApp-like chat, Google Drive, payment systems, and more

★ 24K GitHub stars.

Useful for interview preparation, architecture discussions, or simply refreshing your fundamentals.

https://lnkd.in/eBNu75fs

Worth bookmarking. And credit to the maintainer for making these notes publicly available.

LINKS

- https://lnkd.in/eBNu75fs

#42 Yash Verma

System Design

Serving Notice Period | Software Engineer II, Backend | Java, Kafka, Redis | 8.5M DAU @ 99.97% Uptime

2w · **View on LinkedIn**

Spotify System Design Explained

Spotify isn't just about playing songs. Behind every tap on Play, a distributed system ensures your music starts in milliseconds, even with millions of users streaming simultaneously.

Here's how the request flows:

1. Mobile App

The user logs in, searches for music, or plays a song.

2. Authentication Service

Verifies the user's identity and permissions before allowing access.

3. Load Balancer

Distributes incoming traffic across multiple web servers to avoid overload.

4. Web Servers

Handle API requests such as playlists, recommendations, song metadata, and user profiles.

5. Cache (Redis)

Frequently accessed data is served directly from cache instead of querying databases, dramatically reducing latency.

6. Metadata Database (~70 GB)

Stores song information, playlists, artists, albums, and user data.

* Leader handles writes.

* Followers (Read Replicas) serve read requests, allowing horizontal scaling.

7. Blob Storage (~600 TB)

Stores the actual audio files. Metadata lives in SQL databases, while large binary objects (songs) are stored separately in object/blob storage.

8. Search Service

Indexes songs, artists, albums, and playlists so searches return almost instantly.

9. CDN (Content Delivery Network)

Music files are cached closer to users worldwide, minimizing latency and buffering.

Message Queue (Kafka/RabbitMQ)

Processes asynchronous events like:

* Song plays
* Likes
* Playlist updates
* Notifications
* Recommendation events

This prevents user requests from waiting on background work.

11. Analytics Pipeline

Collects streaming events for:

* Personalized recommendations
* Trending charts
* Business analytics
* Monitoring and observability

Key Design Principles

* ✓ Stateless web servers for easy scaling
* ✓ Cache-first architecture to reduce database load
* ✓ Read replicas for high read throughput
* ✓ Blob storage for massive audio files
* ✓ CDN for low-latency global streaming
* ✓ Message queues for asynchronous processing
* ✓ Analytics pipeline for recommendations and insights

At scale, Spotify is less about streaming MP3 files and more about coordinating distributed systems that deliver content reliably to millions of users with minimal latency.

#SystemDesign #Spotify #BackendEngineering #Java #Microservices #DistributedSystems #Kafka #Redis #CDN #Scalability #SoftwareEngineering #TechArchitecture #Engineering

#43 Saurabh Jadhav

System Design

Senior Software Engineer @ Capital One | Java | Cloud & Microservices | AWS Certified | Kubernetes / OpenShift | FinTech & Payments | Building High-Scale Payments & Cloud-Native Systems

2w · View on LinkedIn

System Design: How do you scale from 0 → 1 Million Users?

One of the biggest mistakes in system design is starting with:

“Let’s use microservices, Kafka, Kubernetes, Redis, and multiple databases.”

The better question is:

What breaks first as the system grows?

Here’s how I think about scaling a system from 0 → 1 million users:

0 → 1K users

Start simple.

Client → Backend → Database

A modular monolith with a relational database is often more than enough.

1K → 10K users

Introduce redundancy.

Client → Load Balancer → Multiple App Servers → DB

Make application servers stateless so traffic can be distributed across instances.

10K → 100K users

The database often becomes the bottleneck.

Add:

• Redis for frequently accessed data
• Proper database indexes
• Read replicas for read-heavy workloads
• Connection pooling

100K → 500K users

Some operations shouldn't block the user request.

Introduce asynchronous processing:

Service → Kafka/SQS → Consumers

Now notifications, analytics, fraud checks, and other background workloads can happen independently.

500K → 1M users

Now reliability and operational complexity become increasingly important.

Introduce where justified:

• Docker + Kubernetes
• Horizontal autoscaling
• API Gateway
• CDN
• Rate limiting
• Multi-AZ deployment
• Observability and distributed tracing
• Database partitioning when necessary

For critical operations such as payments, also think about:

Idempotency — prevent duplicate transactions

Retries + exponential backoff — handle transient failures

Circuit breakers — prevent cascading failures

Dead-letter queues — isolate failed messages

Metrics, logs & traces — understand what's happening in production

The key lesson?

1 million users doesn't automatically mean you need a complicated architecture.

Good system design is about evolving the architecture based on bottlenecks, traffic patterns, reliability requirements, and business needs.

Don't add technology because it's popular.

Add it because you have a problem it solves.

That's the difference between:

✗ “I know Kafka, Redis and Kubernetes.”

and

✓ “I know when and why to use Kafka, Redis and Kubernetes.”

Scale the architecture with the problem - not with the hype.

What would you add to this 0 → 1M scaling journey?

#SystemDesign #SoftwareEngineering #DistributedSystems #Scalability #BackendEngineering #Java #SpringBoot #AWS #Kafka #Kubernetes #Redis #Microservices

#44 Anurag Rana

System Design

System Design & Application Security | AVP Technical Architect @ Info Edge (Naukri.com)

2w · View on LinkedIn

Shopify recently replaced Redis with MySQL for their inventory reservation system, and it scaled perfectly.

Here is how they did it:

The Old Way: Most reservation systems use Redis to quickly count and lock items in a cart. MySQL acts as the final source of truth. At millions of transactions per minute, keeping these two systems in sync created lag and consistency issues.

The New Way: Shopify moved everything into MySQL using a feature called SKIP LOCKED (available since MySQL 8.0.1).

The Schema Change: Instead of having one row per item with a "quantity" column (e.g., item_id: 1, quantity: 10), they changed the schema to have one row per individual unit (e.g., 10 rows for 10 units).

How it works: When a user tries to reserve an item, the database locks that specific row. Because of SKIP LOCKED, other incoming requests simply skip the locked row and move to the next available one. This prevents the "queue" effect where transactions wait on each other.

By simplifying the architecture to a single source of truth, they removed the need for Redis entirely for this use case.

Shopify blog link in comments.

#redis #cache #mysql #systemdesign

#45 Jonish Prajapati

Java / Spring

Senior Software Engineer | Java 17/21 | Spring Boot | Microservices | Distributed Systems | AWS | Kafka | React | Angular | Cloud-Native Applications

2w · View on LinkedIn

JUnit vs Mockito — same goal, different responsibilities.

While building my Java Engineering Labs – Order Management Project, I've been spending more time on something that becomes increasingly important as applications grow: testing behavior, not just making the code work.

A simple way I've learned to think about it:

JUnit helps me verify that my application behaves as expected.

Mockito helps me isolate dependencies so I can test one piece of logic without relying on everything around it.

For example, in my order-management project, I tested business scenarios such as:

→ A new order should start with the correct status.

→ An order shouldn't be shipped before it's processed.

→ A shipped order shouldn't be cancelled.

→ The service should handle an order that doesn't exist.

→ The repository should be called correctly when an order is created.

The biggest lesson for me is that good testing isn't about writing more tests—it's about building confidence that the system behaves correctly when things go right and when they go wrong.

Simple takeaway:

JUnit → Did my code produce the right result?

Mockito → Did my code interact with its dependencies correctly?

I’m continuing to build this project step by step and document what I learn along the way.

For developers: What testing practice has saved you the most trouble in production?

For everyone else: When you use an app, what kind of bug frustrates you the most?

#Java #SoftwareEngineering #JUnit #Mockito #UnitTesting #BackendDevelopment #LearningInPublic #JavaEngineeringLabs

MEDIA JUnit vs Mockito infographic from Java Engineering Labs Order Management Project; document/carousel (possible)

#46 Shubham Shete

Kafka / Messaging

Consultant @Deloitte USI

2w · View on LinkedIn

Apache Kafka Design Patterns — Interview Preparation Guide.

Kafka is much more than just a messaging system. Understanding the right Kafka design patterns is important when building scalable, reliable, and loosely coupled microservices and event-driven applications.

I’ve created this quick reference guide covering some of the important Kafka patterns:

Producer–Consumer Pattern

Publish/Subscribe Pattern

Consumer Groups

Fan-out Pattern

Competing Consumers

Event Sourcing

CQRS

Retry Pattern

Dead Letter Topic (DLT/DLQ)

Transactional / Exactly-Once Processing

Why are these patterns important?

They help us understand how to:

✓ Scale message processing

✓ Build loosely coupled microservices

✓ Handle failures and retries

✓ Process events asynchronously

✓ Support multiple consumers

✓ Improve reliability and fault tolerance

✓ Design better event-driven architectures

Interview Tip:

Don't just memorize Kafka concepts. Try to understand when and why each pattern should be used. Interviewers often give a real-world scenario and ask you to design the Kafka flow.

I hope this cheat sheet helps developers preparing for Java, Spring Boot, Microservices, Kafka, and System Design interviews.

Save this post for your Kafka interview preparation!

Which Kafka topic would you like me to cover next — Partitions & Offsets, Consumer Groups, Retry & DLT, or Kafka Architecture?

#ApacheKafka #Kafka #Java #SpringBoot #Microservices #EventDrivenArchitecture #SystemDesign #SoftwareEngineering #InterviewPreparation #BackendDevelopment #CodingInterview #TechInterview

#47 Harish kumar

AI / LLM

Helping 625K+ Professionals Navigate AI & Tech | AI Content Creator |AI & Marketing Consultant| ATS Resume Expert | Weekly Insights That Actually Move Careers Forward | Open to collaborations|
harishkumar.sh100@gmail.com

2w · **View on LinkedIn**

NVIDIA just released 10 AI courses for FREE — with certificates.

No payment.

No “AI guru.”

No expensive bootcamp.

Just practical AI skills — straight from NVIDIA. ↓

Here’s the full list:

1. Generative AI Explained
https://lnkd.in/gBb3peXi

2. Agentic AI Explained
https://lnkd.in/guBtU3-k

3. Getting Started with AI on Jetson Nano
https://lnkd.in/gnmrhBJm

4. Building A Brain in 10 Minutes
https://lnkd.in/gCaA-XKp

5. A Beginner's Guide to Autonomous Robots
https://lnkd.in/g5sVKrWu

6. Building RAG Agents with LLMs
https://lnkd.in/gcK2ZJ4a

7. Data Science Workflows with Zero Code Changes
https://lnkd.in/gF7eVk2V

8. AI Infrastructure and Operations Fundamentals
https://lnkd.in/gKTS6uMS

9. NVIDIA Tools for Generative AI in Digital Health
https://lnkd.in/gaMUhSVR

10. An Introduction to Developing With NVIDIA Omniverse
https://lnkd.in/gmS-kc3p

You don't need to finish all 10.

Pick ONE.

Spend 45 minutes this week learning something you didn't know yesterday.

Instead of another hour of scrolling Netflix, YouTube, or LinkedIn...

Build a skill that can actually compound.

Save this post. Pick your course. Start today.

Which one are you starting with? ↓

Repost it to help someone else discover these free courses.

#AI #ArtificialIntelligence #GenerativeAI #AgenticAI #NVIDIA

LINKS

- https://lnkd.in/gBb3peXi
- https://lnkd.in/guBtU3-k
- https://lnkd.in/gnmrhBJm

- https://lnkd.in/gCaA-XKp
- https://lnkd.in/g5sVKrWu
- https://lnkd.in/gcK2ZJ4a
- https://lnkd.in/gF7eVk2V
- https://lnkd.in/gKTS6uMS
- https://lnkd.in/gaMUhSVR
- https://lnkd.in/gmS-kc3p

#48 Raul Junco

System Design

Simplifying System Design

2w · View on LinkedIn

Push-based systems come up in 90% of system design interviews.

Here's the exercise you should be able to solve:

Design a real-time feed for 100M users. Some have 50 followers. Some have 10M.

I've watched a lot of engineers answer this, and almost all of them start the same way:

"I'll keep a WebSocket open for every active user and push updates as they arrive."

The problem with this approach is celebrities.

Once one of them posts, 10M deliveries hit your queue at once. Your broker backs up, your WebSocket servers choke, and the system dies at the exact moment everyone is watching.

This is the fan-out problem.

Here is a solution:

-> Small accounts use push. When they post, every follower gets the update immediately.

-> Large accounts use pull. Their posts merge into the feed when a follower opens the app. Nobody gets a push.

Basically, follower count decides which path a post takes.

This is how Twitter's home timeline worked.

Most people stop here; please don't.

You still have three problems:

1. Routing. Connections are stateful. Which server holds which user? Most teams use Redis pub/sub: each WebSocket server subscribes to a channel for the users it holds, and the backend publishes to it.

2. Delivery. Redis pub/sub is fire-and-forget. If a node blinks, the message is gone.

3. Recovery. A client drops for 3 seconds. What did it miss? You need sequence IDs, a buffer, and reconnect logic that replays events.

Your "simple" push system is now push, a pull fallback, a broker, a connection registry, and a replay buffer.

Keep pulling the thread until the system breaks. That's what gets you the offer.

#49 Naushad A.

Java / Spring

Senior Specialist | 509k Engagement | Java 8 & 21 | Spring Boot | Microservices | AWS | Ex-@Capgemini

2w · View on LinkedIn

#PWC Interview Experience | Java Consultant ↓

✓ All the Answer for the interview Questions including concepts are available in Single Kit - https://lnkd.in/dW_5jZie

1. What is a Circuit Breaker in Microservices?

2. What happens when two transactions update the same bank account simultaneously?

3. Can volatile make a bank balance thread-safe?

4. REQUIRED vs REQUIRES_NEW vs NESTED?
5. What if money is debited but the application crashes before credit?
6. How do you prevent duplicate payment processing?
7. Why can HashMap be dangerous in concurrent applications?
8. Different between HashMap vs ConcurrentHashMap?
9. What happens when two threads execute balance = balance - amount?
10. synchronized vs ReentrantLock?
11. How can you prevent deadlocks between two account transfers?
12. Explain transaction isolation levels with banking examples.
13. Why is BigDecimal preferred over double for money?
14. Why avoid double even when the database uses DECIMAL?
15. What happens if a Kafka payment event is delivered twice?
16. How do you make a payment API idempotent?
17. What happens if an exception occurs after a DB update?
18. Why might @Transactional not roll back?
19. How would you handle a debit-success/credit-failure scenario?
20. How do you guarantee an account doesn’t go negative under 100 concurrent withdrawals?
21. Heap Dump vs Thread Dump?
22. What tools do you use to investigate Java memory issues?
23. How does Java Flight Recorder help with memory analysis?
24. How can an incorrectly configured cache cause OutOfMemoryError?

✓ Are you seriously looking for Job Offer Letter? Check out the below Kit which provides Complete interview Prep Question with Answer which interviewer 100% will ask and make Base stronger, Our motive is to help other developer, many Developer cracked interview easily.

✓Check the Kit here - https://lnkd.in/dW_5jZie

Follow Naushad A. and stay tune for next Interview Questions and do LIKE and Comment, share with others

#PWC #Java #JavaDeveloper #JavaBackend #SpringBoot #BackendDeveloper #CodingInterview #JavaInterview #SoftwareEngineer

#DSA #SystemDesign #TechInterview #InterviewPreparation #Programming #Developers

LINKS

- https://lnkd.in/dW_5jZie

#50 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2w · **View on LinkedIn**

If I had to start with SYSTEM DESIGN,

I'd learn these 25 concepts (save now):

1 Scalability

2 High Availability

3 Fault Tolerance

4 Latency

5 Throughput

6 Load Balancing

7 Caching

8 CDN

9 DNS
10 API Design
11 REST
12 SQL vs NoSQL
13 Indexing
14 Replication
15 Sharding
16 CAP Theorem
17 Consistency Models
18 ACID
19 Message Queue
20 Pub/Sub
21 Microservices
22 API Gateway
23 Rate Limiting
24 Authentication
25 Authorization
(... and much more!)
Anything else you'd add to this list?
===
PS - Want a clear breakdown of all key concepts?
Read my letters right now:
→ Part 1: https://lnkd.in/eK87NkE5
→ Part 2: https://lnkd.in/dicBp9N4
→ Part 3: https://lnkd.in/dDKk3-Xw
===
Repost to help others learn system design.
Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eK87NkE5
- https://lnkd.in/dicBp9N4
- https://lnkd.in/dDKk3-Xw

#51 Vinay Tiwari Java / Spring

Brand partnership • Software Engineer at Wipro | Java | Spring Boot | Microservices | REST APIs | Kafka | MySQL | AWS | Docker | Kubernetes | DevOps | GenAI

2w · **View on LinkedIn**

HASHMAP vs LINKEDHASHMAP vs TREEMAP
Which Java Map should you use?
We use Map almost every day in backend development.
But choosing the right implementation can make a difference.
HashMap
Fast lookup
get() → O(1) average
put() → O(1) average

Order → Not guaranteed
Null key → One allowed
Best for: Fast key-value lookups
Example:
customerId → Customer
LinkedHashMap
Lookup + predictable order
get() → O(1) average
put() → O(1) average
Order → Insertion order by default
Null key → One allowed
Best for: Ordered data and LRU-style caching
Example:
requestId → Request
TreeMap
Sorted keys
get() → O(log n)
put() → O(log n)
Order → Sorted by key
Null key → Not supported with natural ordering
Best for: Sorted data and range-based operations
Example:
price → Product
Quick Decision
Need fast lookup?
→ HashMap
Need lookup + insertion order?
→ LinkedHashMap
Need sorted keys?
→ TreeMap
Remember:
HashMap → SPEED
LinkedHashMap → ORDER
TreeMap → SORTING
The best Java developer doesn't just know the API.
They know which data structure fits the requirement.
Which one do you use most in your backend projects?

#Java #JavaDeveloper #BackendDevelopment #SpringBoot #DataStructures #JavaInterview #SoftwareEngineering #Programming #Coding #BackendDeveloper

#Java #JavaDeveloper #HashMap #JavaProgramming #BackendDevelopment #SpringBoot #DataStructures #JavaInterview

#SoftwareEngineering #Coding #Programming #Developers #TechCommunity #LearnJava #Java8 #Java11 #Java17 #Java21 #Java25 #SpringBoot #BackendDevelopment #SoftwareEngineering #Programming

#JavaProgramming #SpringBoot #LTS

#TechCommunity #Technology #Trending

#52 Sathish Kumar Subramani

System Design

Senior Engineering Manager | Generative AI | Azure Cloud Solutions | Leadership Excellence | Driving Operational Efficiency

2w · **View on LinkedIn**

You can draw a perfect architecture diagram and still fail the interview.

Because the follow-up is always the same: "Okay, but how does that actually work?"

These 12 algorithms are the real answer behind that question:

1. Bloom Filter:

• Checks if an item might exist in a huge set.

• It can say "maybe," but never a false "no." Databases use it to skip disk reads.

2. Geohash:

• Turns lat/long into a short string like "ezs42."

• Shared prefix = nearby location. "Drivers near me" starts here.

3. HyperLogLog:

• Estimates unique counts in massive datasets.

• Redis counts millions of uniques in ~12 KB with under 1% error.

4. Consistent Hashing:

• Spreads keys across nodes on a ring.

• Add or remove a server and only a small slice of keys moves.

5. Merkle Tree:

• Verifies data integrity with a tree of hashes.

• Compare one root hash to know if two replicas match. Cassandra and Git rely on it.

6. Raft:

• Elects a leader and replicates logs to followers.

• Built to be understandable. etcd and Consul run on it.

7. Lossy Count:

• Finds frequent items in a stream with little memory.

• Trending hashtags without storing every event.

8. QuadTree:

• Splits 2D space into four quadrants, recursively.

• Dense areas get finer cells, so map searches stay fast.

9. Operational Transformation:

• Merges concurrent edits from multiple users.

• The idea behind real-time collaborative editors.

10. Leaky Bucket:

• Processes requests at a fixed outflow rate.

• Traffic spikes get smoothed instead of crushing your backend.

11. Rsync:

• Syncs only the changed parts of files.

• Rolling checksums find the diff and save massive bandwidth.

12. Ray Casting:

• Checks if a point lies inside a polygon.

• Odd intersections = inside. Even = outside. Core to geofencing and GIS.

You don't need to memorize all 12.

You need to know which one fits the problem.

Which one has saved you in an interview?

Repost to help someone prepping for system design

+ Follow Sathish for more

#SystemDesign #SoftwareEngineering #TechInterviews

#53 Harshit Kumar

System Design

Building agentic AI on Java + Spring Boot | Breaking down how enterprise backends actually adopt LLMs | GenAI Engineer

2w · **View on LinkedIn**

I've created a System Design preparation roadmap covering core concepts, HLD, LLD, distributed systems, Kafka, Redis, CQRS, Saga, caching, scalability, and real-world design problems.

From CAP theorem, APIs, consistent hashing amp; microservices to YouTube, Uber, WhatsApp, URL Shortener amp; Google Docs — this checklist is built for interview prep. SD_TOPICS

Useful for developers preparing for Senior / Staff / Lead Software Engineer and product-company interviews.

✓ Save it, follow it topic by topic, and share it with someone preparing for System Design interviews.

#SystemDesign #SoftwareEngineering #SoftwareDeveloper #Java #BackendDevelopment #DistributedSystems #Microservices #HighLevelDesign #LowLevelDesign #HLD #LLD #CodingInterview #TechInterview #InterviewPreparation #Engineering #Scalability #Kafka #Redis #Architecture #Developers

#54 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2w · **View on LinkedIn**

If you want to get GOAT-level with system design,

then learn these 15 case studies (save now):

1 How LLMs Like ChatGPT Work:

↳ https://lnkd.in/eSd6fS7n

2 How WhatsApp Works:

↳ https://lnkd.in/eU2fswMi

3 How YouTube Works:

↳ https://lnkd.in/e7q9F4Sg

4 How Apache Kafka Works:

↳ https://lnkd.in/eTtVAjTg

5 How URL Shortener Works:

↳ https://lnkd.in/evFTZVQq

6 How Uber Finds Nearby Drivers:

↳ https://lnkd.in/eeqH9Hjh

7 How Amazon S3 Works:

↳ https://lnkd.in/e2p7qXri

8 How Amazon Lambda Works:

↳ https://lnkd.in/eNd3Z5Yn

9 How Spotify Works:

↳ https://lnkd.in/eGbWVeNW

10 How Twitter Timeline Works:

↳ https://lnkd.in/eniXMPfU

11 How Google Docs Works:

↳ https://lnkd.in/ehPNA7Az

12 How Airbnb Works:

↳ https://lnkd.in/dGVfstQM

13 How Slack Works:

↳ https://lnkd.in/eATMDjrK

14 How Stock Exchange Works:

↳ https://lnkd.in/eNf2QxVZ

15 How Do AirTags Work:

↳ https://lnkd.in/ec75eiCH

These are the case studies I keep coming back to time after time.

Which case studies do you still periodically revisit?

===

PS - Want my System Design Playbook for FREE?

Join my newsletter with 250K+ software engineers:

→ https://lnkd.in/ehTrcyak

===

Repost to help others master system design.

Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eSd6fS7n
- https://lnkd.in/eU2fswMi
- https://lnkd.in/e7q9F4Sg
- https://lnkd.in/eTtVAjTg
- https://lnkd.in/evFTZVQq
- https://lnkd.in/eeqH9Hjh
- https://lnkd.in/e2p7qXri
- https://lnkd.in/eNd3Z5Yn
- https://lnkd.in/eGbWVeNW
- https://lnkd.in/eniXMPfU
- https://lnkd.in/ehPNA7Az
- https://lnkd.in/dGVfstQM
- https://lnkd.in/eATMDjrK
- https://lnkd.in/eNf2QxVZ
- https://lnkd.in/ec75eiCH
- https://lnkd.in/ehTrcyak

#55 Serge Elman

Other

Head of ApplePay Client Services | Payment Solutions Integration Expert | Technical Project Management.

2w · **View on LinkedIn**

Payments Terminology of the Day: Token Provisioning

When you select “Add Card” in a digital wallet, you initiate a process that establishes a payment token, its permitted usage and its readiness for payment.

Token provisioning delivers a payment token and related data to a wallet or another tokenized payment environment. The broader onboarding flow can include eligibility checks, risk evaluation, identification and verification (ID&V), token creation and activation.

Who participates?

• Cardholder: requests enrollment and completes verification when required.

• Token Requestor (TR): requests the token; its Token Requestor ID (TRID) identifies the requesting participant.

• Token Service Provider (TSP): manages token issuance, the token-to-PAN relationship and token controls.

• Issuer: participates in eligibility, risk and provisioning decisions.

A typical “Add Card” journey:

1. INITIATE

The customer enters card details in the wallet or selects “Add to Wallet” from the bank’s app. The latter is commonly called issuer push provisioning.

2. CHECK ELIGIBILITY

Determine whether the card and issuer support the requested wallet and tokenization use case. An eligible card still needs to pass the applicable approval checks.

3. EVALUATE RISK

Assess the account, device and provisioning context. The decision may be:

APPROVE → continue

STEP-UP → require additional verification

DECLINE → stop

4. COMPLETE ID&V

Where required, the cardholder completes an issuer-supported verification method, such as an OTP sent to an established contact channel.

5. CREATE AND PROVISION

The TSP issues a payment token associated with the underlying PAN and applicable usage controls, then securely delivers the token and related data to the destination.

NOTE: A token can be restricted to a device, merchant or transaction context. It is a substitute credential—not simply the PAN with some digits hidden.

6. ACTIVATE AND CONFIRM

Complete any remaining activation requirements and synchronize status across the participating systems. Exact sequencing varies by implementation.

Illustrative decision payload—not a network API schema:

{
"requestId": "prov-demo-001",
"tokenRequestorId": "<registered-TRID>",
"cardReference": "<opaque-card-reference>",
"decision": "STEP_UP",
"verificationMethod": "ISSUER_OTP",
"activationPending": true
}

Here, the TRID identifies WHO requests the token. The card reference identifies WHICH credential is being enrolled. STEP_UP means more verification is required before the flow can complete.

Operationally, track eligibility failures, issuer declines, verification completion, provisioning latency and activation success separately. They represent different failure points.

Provisioning approval is separate from purchase authorization. An active token enables payment attempts; each transaction still faces its applicable security, risk and issuer authorization checks.

#Payments #Tokenization #DigitalWallets #PaymentTechnology

#56 Anup Panwar Other

Payments & Blockchain Architecture | Cross-Border Payments at Wells Fargo | Backend & Distributed Systems | Research: DLT, Tokenization & Digital Money | Advisory & Collaboration

2w · **View on LinkedIn**

Tokenized deposits and stablecoins may look similar, but the underlying structure is very different.

Both can represent digital value and support faster movement of money, yet they differ in who issues them, what backs them, where they operate, and how settlement happens.

Tokenized deposits are typically issued by commercial banks as digital representations of customer deposits.

The flow can look like:

Customer funding → Bank issuance → Deposit tokenization → Payment routing → Ledger settlement → Recipient credit

The important point: the token generally remains a liability of the issuing bank, much like a traditional deposit.

Stablecoins follow a different model.

Fiat funding → Mint request → Token issuance → Wallet routing → Network confirmation → Recipient receipt

Here, the stablecoin is usually issued against cash, cash equivalents, or other reserves. The liability sits with the issuer rather than automatically becoming a bank deposit.

The biggest differences appear across six areas:

→ Issuer: Bank vs stablecoin provider

→ Liability: Bank obligation vs issuer obligation

→ Backing: Bank balance sheet vs reserve assets

→ Ledger: Bank or permissioned network vs blockchain

→ Redemption: Through the bank vs through the issuer

→ Settlement: Banking rails vs blockchain confirmation rules

This distinction becomes increasingly important as banks, payment companies, and enterprises explore programmable money and always-on settlement.

The question is no longer just, “Is the money tokenized?”

It is also: who issued it, what supports it, where is it recorded, and what gives the transaction finality?

Those details define the real payment architecture.

#57 **Basia Kubicka** AI / LLM

AI Product Manager · Agentic AI · Vibe Coding | I build with Claude & teach 90K+ to do the same | ex-Techstars founder (0→$7M), ex-AI PM (Sequoia-backed)

2w · **View on LinkedIn**

Google's 31-day AI Agents Challenge starts today.

180k+ AI builders already joined. For FREE.

Built by Google Cloud AI team.

One project a day you can run in 5 minutes.

It’s called the Advent of Agents.

It’s the 3rd time they run it.

Seasons 1 and 2 taught you to build agents.

(56 tutorials, still archived, still free)

This season is about governing them.

What's actually in those 31 days:

→ Foundations: the agent lifecycle and the real threat surface

→ Identity: a cryptographic ID per agent, acting for a signed-in user

→ Guardrails: prompt injection, data exfiltration, memory isolation

→ Fleet ops: Agent Registry, tool governance, kill switches

→ Cost controls: before one runaway loop eats your entire budget

Daily drills.

Copy-paste code if you want to try.
Official docs and codelabs if you want to go deeper.
Link: https://lnkd.in/gTgar4Gt
I'm in. Are you?
Repost so others don't miss the deadline
+ Follow Basia Kubicka for frameworks that ship

MEDIA Google 31-day AI Agent Challenge

LINKS
- https://www.basiakubicka.com/
- https://lnkd.in/gTgar4Gt

#58 Adnan Khan

Java / Spring

Java Backend Developer | Spring Boot | Microservices | Kafka | REST APIs | Hibernate | SQL | AWS | Redis | CI/CD
2w · **View on LinkedIn**

Google Interview Experience – SDE-3 Backend
Compensation: ₹80 LPA
Role: SDE-3 (L4)
Experience: ~4 Years
Here’s what the interview process looked like ↓
Round 1 – Phone Screen (DSA)
Problem: Given a stream of float values, return the mean of the last N values at any point.
Follow-up:
Now remove the K highest values from those N values before calculating the mean.
Focus Areas:
• Running sum
• Sliding window
• Efficient data structures
• Optimizing beyond a basic heap approach
Round 2 – Onsite 1 (Graphs)
Problem: Given a graph of movies where each movie has a rating, return the top N similar movies by rating.
Follow-up:
Movies can be added dynamically while the system is running. How would you modify the solution?
Concepts:
• BFS
• Min Heap of size N
• DSU / Union-Find
• Handling dynamic updates
Round 3 – Onsite 2 (Strings)
Problem: Minimum Window Substring – LeetCode 76.
The interviewer used the standard problem and test cases.
Concepts:
• Sliding Window
• Character Frequency Map
• Two Pointers

Round 4 – Onsite 3 (Trees + Recursion)
Problem: Merge two N-ary trees where every node contains a key and value.
Rules:
• If the same key exists in both trees, Tree B's value wins
• Matching nodes should merge their children recursively
• Tree A's children come first
• Remaining children from Tree B are added afterward
Concepts:
• Recursion
• HashMap
• Tree traversal
• N-ary tree manipulation
Round 5 – Googleyness (Behavioral)
Questions included:
• Deep discussion around resume projects
• Ownership-related questions
• Difficult decisions you have taken
• Standard behavioral questions
• STAR-based answers
Key Takeaway:
For a Java Backend Developer interview, don't prepare only DSA.
You should also be ready for:
✓ DSA & Problem Solving
✓ Java & Spring Boot
✓ Machine Coding
✓ Multithreading & Concurrency
✓ SQL
✓ System Design
✓ Trees & Graphs
✓ Strings & Sliding Window
✓ Project-based questions
✓ Behavioral rounds
I've covered these interview-focused topics in my Java Backend Interview Kit to help with structured interview preparation.
Get the Kit: https://lnkd.in/dst_Ggbs
If you have any doubts, feel free to DM me. I also conduct mock interviews.
Save this post | Share with a Java Developer
ava #JavaDeveloper #JavaInterview #JavaBackend #SpringBoot #Microservices #Kafka #Java8 #CodingInterview #Capgemini #InterviewPreparation

MEDIA advertisement

LINKS
- https://lnkd.in/dst_Ggbs

#59 Umesh Palai

System Design

Founder – Quant Alpha, Technology Partner for Hedge Funds, Prop Firms & Professional Traders | Quant Trading Systems- HFT & MFT | AI / ML | Equities, Crypto & FX Markets | 100+ Successful Projects across Globe

2w · View on LinkedIn

REST API vs WebSocket in Algorithmic Trading — What Actually Happens Under the Hood?

1. REST is a stateless HTTP request/response model where the client explicitly initiates each operation, making it well suited for commands and queries such as order submission, cancellation, account information, and position snapshots.

2. WebSocket begins with an HTTP Upgrade handshake and then establishes a persistent TCP connection over which both client and server can continuously exchange messages without creating a new HTTP request for every event.

3. In an algorithmic trading system, REST can act as the command/query interface while WebSocket can act as the event-driven interface for streaming market data, order acknowledgements, fills, cancellations, and other asynchronous events.

4. REST-based polling repeatedly asks the server for state, whereas WebSocket allows the server to push state changes as events, which fundamentally changes the architecture from polling-driven logic to event-driven processing.

5. WebSocket does not inherently mean lower end-to-end latency than REST; the actual latency depends on TCP/TLS, serialization, network propagation, exchange gateways, buffering, scheduling, and application-level processing.

6. A production WebSocket market-data handler must handle sequence numbers, message gaps, dropped packets, reconnects, snapshots, and incremental updates to maintain a consistent local order book.

7. A persistent WebSocket connection also introduces state-management responsibilities such as heartbeat detection, reconnection, re-authentication, subscription recovery, and reconciliation of missed events after a connection failure.

8. REST and WebSocket have different rate-limit and message semantics, so an execution engine must separately manage order-rate limits, connection limits, subscription limits, request throttling, and exchange-specific constraints.

9. A robust trading engine should therefore decouple the strategy, risk engine, order manager, REST adapter, WebSocket adapter, and state manager rather than allowing trading logic to depend directly on a broker or exchange protocol.

10. The practical engineering pattern is not REST versus WebSocket, but REST for deterministic commands and queries combined with WebSocket for continuous event streams, with both feeding a single source of truth for order, position, and portfolio state.

#60 Sandhya Sahani

System Design

Data Analyst & Design Solutions @ Delhivery | AI/ML | Model Deployment | Building Real-world AI Solutions

3w · View on LinkedIn

System Design Interview Preparation

Sharing a PDF with important System Design Interview Questions covering:

HLD & LLD

Microservices Architecture

Fund Transfer System

Kafka & Message Queues

Database & Caching

Scalability & High Availability

Distributed Systems

API Gateway & REST

Fault Tolerance & Circuit Breaker

AWS & Cloud Deployment

Design Patterns & SOLID

Real-world System Design Scenarios

Save this post for your interview preparation.

Sharing the PDF for anyone preparing for Software Engineer / Backend / AI-ML / Data Engineering interviews.

#SystemDesign #HLD #LLD #Microservices #Kafka #AWS #SoftwareEngineering #BackendDevelopment #InterviewPreparation #TechInterview

System Design Interview Questi

.

23 pages

MEDIA document/carousel (possible)

#61 Saiful Khan

Cloud / K8s / DevOps

Senior DevOps Engineer | Solutions Architecture | AWS | Gen AI | Agentic AI |

3w · **View on LinkedIn**

Same cloud. Different names.

Here is the cheat sheet (AWS / Azure / GCP):

Compute: EC2 / Virtual Machines / Compute Engine

Serverless: Lambda / Azure Functions / Cloud Functions

Containers: ECS, EKS / AKS / GKE

Object storage: S3 / Blob Storage / Cloud Storage

Block storage: EBS / Managed Disks / Persistent Disks

Relational DB: RDS / Azure SQL Database / Cloud SQL

NoSQL: DynamoDB / Cosmos DB / Firestore

Data warehouse: Redshift / Synapse Analytics / BigQuery

Networking: VPC / VNet / VPC

CDN: CloudFront / Azure CDN / Cloud CDN

Load balancing: ELB / Azure Load Balancer / Cloud Load Balancing

Identity: IAM / Microsoft Entra ID / Cloud IAM

Monitoring: CloudWatch / Azure Monitor / Cloud Monitoring

Learn the building blocks once. Then moving between clouds becomes much easier.

Which cloud do you use the most at work? ↓

#DevOps #CloudComputing #AWS #Azure #GCP #Kubernetes #CloudArchitecture

#62 Sajeev KR

Java / Spring

Senior Software Engineer @ IBM Software Labs| 11+ Yrs Java/J2EE | Spring Boot & Microservices | AWS | Full-Stack Developer | AI Enthusiast | Jenkins | Shell & Python | Go | Writing Deep Dive Java & System Design Content

3w · **View on LinkedIn**

Java & Spring Boot–Real-World Scenario-Based Interview Questions

Answer is shared in this link - https://lnkd.in/dryt84Gn

1. Two threads update the same account balance at the same time. How would you prevent lost updates in a Java application?

2. A shared HashMap is accessed by multiple threads and starts producing inconsistent results. How would you fix the design?

3. A Java application has a deadlock in production. How would you identify which threads are involved and resolve it?

4. You need to process 100,000 independent tasks concurrently. How would you choose between ExecutorService, CompletableFuture, and virtual threads?

5. A Spring Boot application needs to support different business rules for different customers. How would you design the code without creating a large number of if-else statements?

6. A REST API must support backward-compatible changes while thousands of existing clients are still using the old contract. How would you design the API evolution?

7. A client sends a very large JSON payload to your Spring Boot API. How would you protect the application from excessive payloads and invalid input?

8. Your API must accept requests from multiple frontend applications hosted on different domains. How would you configure CORS securely?

9. A user can access an API only when they own the requested resource. How would you implement this authorization rule in Spring Security?

10. Your JWT contains a user's old role after the role has been changed in the database. How would you handle authorization when token information becomes stale?

11. A JPA application loads thousands of entities into the persistence context during a batch operation. How would you prevent excessive memory usage and maintain good performance?

12. Two users edit the same customer record at nearly the same time. How would you prevent one user's changes from silently overwriting the other's changes?

13. You need to update millions of database records using Spring Data JPA. How would you design the operation instead of loading every entity into memory?

14. Your application frequently reads product information that changes only once a day. How would you design the caching strategy and handle cache invalidation?

15. A cache contains stale data after a database update. How would you keep cached and persistent data reasonably consistent?

16. A Spring Boot application needs different configuration and secrets for development, staging, and production. How would you manage them without putting secrets in source control?

17. A service must call several independent external APIs and combine their responses into one result. How would you design the Java code to avoid unnecessary sequential waiting?

Java Spring Boot & Microservices Interview Q&A (200 Qs) - https://lnkd.in/gqhJhmcZ

Full interview kit (600 Qs: Java, Springboot, database, devops, cloud, system design, microservice) - https://lnkd.in/gPi2W587

Java & Spring Boot – 20 Real-World Scenario-Based Interview Questions with Deep Answers

Java & Spring Boot – 20 Real-World Scenario-Based Interview Questions with Deep Answers

logicbrace.com

LINKS

- https://lnkd.in/dryt84Gn
- https://lnkd.in/gqhJhmcZ
- https://lnkd.in/gPi2W587
- https://www.logicbrace.com/2026/09/java-spring-boot-20-real-world-scenario.html

#63 Petar Ivanov

System Design

I help JavaScript devs think like architects in the AI era • React · Node · Architecture · System Design · Production AI • Senior AI Engineer • 100K+ Audience • Founder of petarivanov.tech

3w · **View on LinkedIn**

8 load balancing algorithms you should know:

1. Round Robin:

It sends each new request to the next server in a rotating order.

↳ Useful when all servers have similar capabilities and you want to spread the load evenly.
2. Least Connections:
It directs traffic to the server with the fewest active connections.
↳ Useful when servers have different workloads, balancing them more efficiently.
3. Weighted Round Robin:
It gives more requests to servers with higher weights or capacities.
↳ Useful when some servers are more powerful and can handle more traffic.
4. Weighted Least Connections:
It considers both server capacity and current connections to assign requests.
↳ Useful when servers differ in performance, ensuring a fair distribution.
5. IP Hash:
It uses the client's IP address to decide which server will handle the request.
↳ Useful to keep a client connected to the same server, which is important for session consistency.
6. Least Response Time:
It sends requests to the server with the quickest response and fewest connections.
↳ Useful to reduce delays and improve the user experience.
7. Random:
It picks a server at random for each new request.
↳ Useful when you don't need to consider server load or differences in server capacity.
8. Least Bandwidth:
It directs traffic to the server using the least network bandwidth at the moment.
↳ Useful when managing network usage is important to prevent congestion.
Image Credits: DesignGurus
——
Save this for later.
Repost to help others find it.
+ Follow Petar Ivanov + turn on notifications.

LINKS

- https://www.linkedin.com/feed/update/urn:li:activity:7485665965410566144

ALSO SAVED AS Petar Ivanov (2mo) link

#64 Amit Mishra

System Design

Technology Leader | Product & Software Design Development & Integration Specialist

3w · **View on LinkedIn**

System Design | Real-Time Trading Platform

Exploring the architecture of a high-scale trading system designed for real-time market data and reliable order processing.

The design brings together API Gateway, Price Services, SSE, Redis Pub/Sub, Kafka, OMS, Order DB, Time-Series Storage and Exchange Gateway.

The key architectural challenges are around:

- High throughput & low latency
- Real-time data distribution
- Asynchronous processing with Kafka
- Order consistency & idempotency
- Fault tolerance and retries

• Independent horizontal scaling
• Observability and failure recovery

A great exercise in understanding how individual microservices come together to build a scalable and resilient distributed system.

#65 Dhiral Kaniya

System Design

Engineering @Wise | Ex JPMC & Flipkart

3w · View on LinkedIn

Your database probably isn't struggling because of writes.

It's struggling because thousands of requests per second are asking it the same question: "What's this user's profile?"

That's the problem CQRS solves: separate the path that changes data from the path that reads it, and scale each on its own.

But before reaching for it, I'd ask four questions:

- Are you actually operating at high scale?

- Do reads massively outnumber writes?

- Do reads and writes need different shapes?

- Can users tolerate slightly stale reads?

That last one decides everything. CQRS is excellent for user profiles, catalogues and search. It's a poor fit for account balances, payments or inventory at checkout.

I also cover the modern managed alternative, DynamoDB, including what it solves out of the box and what it still leaves to you.

Full breakdown with diagrams in the comments ↓

Where has your team drawn the line on eventual consistency?

#SystemDesign #SoftwareArchitecture #CQRS #BackendDevelopment #DynamoDB

CQRS in Practice: How to Serve Massive Read Traffic Without Melting Your Database

CQRS in Practice: How to Serve Massive Read Traffic Without Melting Your Database

medium.com

LINKS

- https://medium.com/@dhiralkaniya3146/cqrs-in-practice-how-to-serve-massive-read-traffic-without-melting-your-database-fe37cf79700c

#66 Tauqeer Ahmed

Java / Spring

Staff Engineer | Lead Backend Engineer | Java, Spring Boot, Microservices | Distributed Systems | Kafka | Kubernetes

3w · View on LinkedIn

A correlation ID can tell you which request failed. It cannot tell you why the request was slow.

That distinction matters once a call crosses a gateway, several services, a queue, and a database.

Imagine a checkout request with `request_id=abc-123`. If every log line includes it, you can search the journey. But you still may not know whether 900 ms was spent in inventory, payment, message publishing, or waiting for a connection pool.

Distributed tracing adds the missing structure:

One trace groups the end-to-end request.

Spans show each operation and its parent-child relationship.

Span timing makes serial work, parallel work, and waiting visible.

A practical rule: keep a stable request/correlation ID for log search, and propagate trace context for causality and timing. Do not make one field carry both jobs.

In a Spring Boot service, this also means checking the boundaries: HTTP clients, messaging producers/consumers, async executors, and scheduled work. Context that disappears at one boundary creates a misleading “trace” that looks complete until the most important hop is missing.

Observability is strongest when each identifier answers one question clearly: “Which request?” and “What happened along its path?”

#Java #SpringBoot #Observability #DistributedSystems #Microservices

#67 Raul Junco

System Design

Simplifying System Design

3w · View on LinkedIn

Every architecture diagram makes mathematical promises.

These 7 formulas help you test them.

1. Peak throughput

Peak RPS = (daily users × requests per user ÷ 86,400) × peak factor

Average traffic hides the busiest hour. Peak RPS gives you a better starting point for sizing APIs, load balancers, and rate limits.

2. Network bandwidth

Bandwidth = RPS × (request bytes + response bytes)

This estimates traffic at each network hop. Internal fan-out, replication, retries, and protocol overhead all increase the real number.

3. Concurrency

Requests in flight = RPS × latency

This is Little’s Law. At 10,000 RPS with 200 ms latency, the system carries roughly 2,000 concurrent requests.

4. Resource utilization

$\rho = \lambda \div (c \times \mu)$

λ is the arrival rate, c is the number of workers, and μ is the service rate per worker. As utilization approaches 100%, queues grow and latency becomes unpredictable.

5. Storage

Storage = write rate × record size × retention × replicas ÷ compression ratio

This gives you the base data footprint. Indexes, backups, tombstones, and compaction headroom still need separate capacity.

6. Replica quorum

$R + W > N$

$W > N \div 2$

For quorum-based replication, read and write sets overlap, while two writes cannot each win with separate majorities. The exact consistency guarantee still depends on the database protocol.

7. End-to-end availability

$A_{system} = A1 \times A2 \times \ldots \times An$

A request that requires three independent services at 99.9% availability gets about 99.7% end-to-end availability. Every required dependency spends part of your reliability budget.

I think the diagram shows how the system connects, but you need the math; yes, the math tells you where it breaks.

#68 Raul Junco

System Design

Simplifying System Design

1mo · View on LinkedIn

80% of System Design Interviews Test the Same 20% of the concepts.

Here is what you need to master.

1. Scalable Data Storage

• Relational vs. NoSQL: Know when to use SQL vs. NoSQL databases.
• Partitioning: Vertical and horizontal partitioning (sharding). Understand trade-offs.
• Indexing: Covering indexes, primary vs. secondary indexes.
• Consistency Models: Strong, eventual, causal.

2. Caching

• Client-side vs. Server-side Cache: Understand where caching should happen.
• Caching Strategies: Write-through, write-back, write-around.
• Distributed Cache: Redis, Memcached.
• Cache Eviction Policies: LRU, LFU, etc.

3. Load Balancing

• Horizontal Scaling: Why and how to horizontally scale services.
• Load Balancing Techniques: Round-robin, consistent hashing.
• Reverse Proxy: Understand how to use Nginx, HAProxy.

4. Asynchronous Processing

• Message Brokers: Kafka, RabbitMQ. When to use queues vs. streams.
• Event-Driven Architecture: Benefits of decoupling and event sourcing.
• Task Queues: For delayed jobs or retries.

5. Database Read and Write Scaling

• Read Scaling: Master replication, read replicas.
• Write Scaling: Challenges with partitioning for writes, leader-election.
• CAP Theorem: Consistency, Availability, or Partition tolerance may be compromised.

6. Distributed Systems Concepts

• Consensus Algorithms: Paxos, Raft.
• Conflict Resolution: Last Write Wins, CRDTs, vector clocks for data reconciliation.

7. Reliability and Failover

• Redundancy: Active-passive vs. active-active configurations.
• Health Checks.
• Retries and Circuit Breakers: How to protect systems from cascading failures.

8. CDNs (Content Delivery Networks)

• Static Content Delivery: Why use a CDN, how does it work?
• Caching at the Edge: How CDNs improve latency for end users.

9. API Design and Rate Management

• REST vs. GraphQL: Difference and practical use-cases for each.
• Pagination and Filtering: Strategies for efficiently fetching data.
• API Versioning: Best practices for evolving APIs.
• Throttle Requests: Why rate limiting is essential, algorithms like token bucket, leaky bucket.

10. Search Systems

• Indexing: Building and maintaining indexes for fast search.
• Full-Text Search Engines: ElasticSearch, Azure AI Search.
• Ranking and Relevance: Basic understanding of how scoring works.

11. Monitoring, Observability and Security

• Metrics Collection: Prometheus, Grafana.
• Distributed Tracing: OpenTelemetry, Sentry.
• Centralized Logging.
• Authentication and Authorization: OAuth, JWT.
• Encryption: Data in transit vs. data at rest.

If you master these 11 areas, you'll be ready for most system design interviews thrown at you.

Add more if you can ↓

LINKS

• https://www.linkedin.com/feed/update/urn:li:activity:7510009873284206592

ALSO SAVED AS Pradeep Kumar Saini (3w) link

#69 Alex Xu

Cloud / K8s / DevOps

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

3w · **View on LinkedIn**

Watch an ex-YouTube engineer rebuild YouTube in 45 minutes.

By the end it's running:

- Home feed
- Video upload to the cloud
- Video playback from the cloud
- Data in a real database
- Search, using multimodal embeddings
- Deployed to a real URL

All of those explained in under an hour. Thanks Mikhail Sychev for making the video with us.

Watch the full video here: https://lnkd.in/g-ASxqWB

LINKS

• https://bit.ly/bbg-a-li-profile
• https://lnkd.in/g-ASxqWB

#70 Java Developers Community (moderated)

Java / Spring

Vahid Hasani • 3rd+ | Project Loom is not just about Virtual Threads. | Virtual Threads are the feature most Java developers see first — but the bigger idea is a change in how we model concurrency in Java. | A blocked task shouldn’t have to keep an expensive platform thread occupied. | That simple idea leads us much deeper: | → Virtual Threads make concurrent tasks cheap | → Continuations make execution suspendable and resumable | → The JVM scheduler maps many Virtual Threads onto a smaller set of carrier threads | → Mounting, unmounting, parking, stack chunks, and pinning explain what actually happens underneath | → Structured Concurrency gives related tasks a clear lifetime and owner | → Scoped Values give contextual data a clear scope | And this also changes an old architectural trade-off: | Do we still need Reactive just to avoid blocking expensive threads? | Often, no. | But that does not mean Reactive is obsolete. Virtual Threads solve an execution/resource problem; Reactive remains valuable when data flow, streaming composition, and backpressure are the actual problem. | The mental model I wanted to build with this carousel is: | Virtual Threads → Execution | Structured Concurrency → Lifecycle | Scoped Values → Context | And underneath them: Continuations, scheduling, stack chunks, mount/unmount, park/unpark, carrier threads, and the JVM runtime. | Loom doesn’t make CPU, database connections, memory, or downstream services unlimited. | It changes the economics of waiting. | That distinction is where the interesting engineering decisions begin. | What part of Project Loom would you like to go deeper into next — Virtual Thread internals, Structured Concurrency, Scoped Values, or production performance? | #Java #ProjectLoom #VirtualThreads #JVM #Java21 #Concurrency #OpenJDK #JavaPerformance #SoftwareArchitecture #BackendEngineering | 1/15 | 40 | 1 | 3 | | Kasra Shamsaei 3rd+ | Kasra Shamsaei • 3rd+ | Technical Lead @ SeaMasterTech | Software Architect

3w · **View on LinkedIn**

Best thing that ever happened to java, the only problem is that it just took too long!

MEDIA document/carousel (possible)

#71 Swapnil Dhamal

Java / Spring

Advanced Software Engineer | Certified Tridium Niagara Developer | Security Advocate | Spring Boot | Microservices | REST | JAVA | React.js | Next.js | MERN | IOT | Fullstack Development

3w · View on LinkedIn

JAVA | COLLECTIONS / SCALABILITY | QUICK 2-MIN READ

ONE CODE. ONE CONCEPT. #47

Which Java Collection would you choose for 10 million records—and why?

There is no “10 million records = X” answer.

The right collection depends on what your application does with those records.

REAL PRODUCTION PROBLEM

Imagine an API loads 10 million customer IDs.

You need to:

• iterate through them

• check whether an ID exists

• occasionally add/remove records

Choosing the collection based only on size can create a serious performance problem.

1. Need fast lookup by ID?

Use:

Map<Long, Customer> customers = new HashMap<>();

customers.get(customerId);

Average lookup → O(1)

But 10 million entries means memory matters: bucket arrays, entry/node objects, references, and resizing overhead.

If you don’t need ordering, HashMap is a strong candidate—but measure the actual memory footprint.

2. Need sorted keys?

Use:

TreeMap<Long, Customer> customers;

Operations are typically O(log n), but you get sorted keys and range operations:

customers.subMap(1000L, 5000L);

3. Need sequential processing?

Don’t automatically use ArrayList just because it’s “fast.”

If you genuinely need all 10 million objects in memory:

List<Customer> customers =

new ArrayList<>(10_000_000);

Pre-sizing avoids repeated growth/copying.

But ask the bigger question:

Do you need all 10 million objects in memory at once?

Often, the better solution is pagination, streaming, batching, or database-side processing.

THE IMPORTANT PRODUCTION LESSON

10 million records changes the question from:

“Which collection is fastest?”

to:

“What operations do I need, and can I avoid keeping 10 million objects in the JVM?”

For example:
Database
↓
Fetch 1,000 records
↓
Process
↓
Discard
↓
Fetch next 1,000
This can be far more memory-friendly than loading everything into a collection.
(!) ONE MORE THING
A thread-safe collection isn’t automatically right just because your app is concurrent.
If multiple threads share it, evaluate the access pattern:
ConcurrentHashMap
CopyOnWriteArrayList
BlockingQueue
Don’t pay concurrency costs you don’t need.
ONE-LINE TAKEAWAY
For 10 million records, don’t choose a collection from the number alone. Choose based on lookup, ordering, mutation, concurrency, memory footprint—and whether you should load all 10 million records into memory at all.
Interview question:
You have 10 million records and only need to check whether an ID exists. Which collection would you choose, and what memory concerns would you investigate?
#Java #JavaCollections #CoreJava #HashMap #Performance #Scalability #BackendDevelopment #SystemDesign #JavaInterview

#72 Lokanadham Ganta

System Design

Banking Technology Architect | Principal Engineer | Solution Architect | 22+ Years | Java | Spring Boot | ReactJS | Payments | Fraud Prevention | AI in Banking

3w · **View on LinkedIn**

System Design Fridays | Episode 1
API Gateway in Banking Systems: Authentication, Rate Limiting & Circuit Breakers
Most developers think an API Gateway simply forwards requests.
In reality, it's the security and resilience layer that protects banking microservices before a single request reaches your backend.
Why Every Banking System Needs an API Gateway
Imagine Internet Banking, Mobile Banking, and Third-Party FinTech Partners calling your APIs simultaneously.
Without an API Gateway, every microservice would have to handle authentication, routing, rate limiting, and failure handling individually.
Instead, banks place an API Gateway in front of their services.
Think of it as the airport security checkpoint for APIs.
The End-to-End Flow
ReactJS or Mobile App sends a request.
API Gateway validates the JWT.

Rate Limiter prevents abuse.
Circuit Breaker protects downstream services during failures.
Request is routed to the correct Spring Boot microservice.
The response is returned securely to the client.
Spring Cloud Gateway Example
spring.cloud.gateway.routes[0].id=account-service
spring.cloud.gateway.routes[0].uri=http://account-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/accounts/**
spring.cloud.gateway.routes[0].filters[0]=StripPrefix=1
spring.cloud.gateway.routes[0].filters[1]=JwtAuthenticationFilter
spring.cloud.gateway.routes[0].filters[2]=RequestRateLimiter
spring.cloud.gateway.redis-rate-limiter.replenishRate=100
spring.cloud.gateway.redis-rate-limiter.burstCapacity=200
Circuit Breaker Example
@Bean
public CircuitBreakerConfig circuitBreakerConfig() {
return CircuitBreakerConfig.custom()
.failureRateThreshold(50)
.waitDurationInOpenState(Duration.ofSeconds(30))
.build();
}
Real Banking Use Case
Imagine a customer initiates a fund transfer.
At the same time:
Thousands of balance checks arrive.
Fraud detection is running.
KYC services are processing requests.
If one downstream service slows down, the API Gateway can trigger a Circuit Breaker, return a fallback response, and prevent a system-wide cascade failure.
This is why API Gateways are critical for high-volume banking platforms.
60-Second Architect
API Gateway is the single entry point.
JWT validation happens before reaching microservices.
Rate limiting protects against abuse.
Circuit Breakers improve resilience.
Routing keeps services independent.
A must-know topic for Senior Java, Spring Boot, and Full Stack interviews.
Interview Challenge
You're designing a banking platform where 10,000 users hit the Login API within one minute.
How would you configure the API Gateway to prevent abuse while ensuring genuine customers can still log in?
Share your approach in the comments.
#SystemDesign #APIGateway #SpringBoot #Java #ReactJS #Microservices #BankingTechnology #FinTech #CodeWithLoka
Better Engineers Build Better Banking.
See you next Monday for Banking Architecture Masterclass | Part 6.

LINKS
- http://account-service

#73 Basia Kubicka

AI / LLM

AI Product Manager · Agentic AI · Vibe Coding | I build with Claude & teach 90K+ to do the same | ex-Techstars founder (0→$7M), ex-AI PM (Sequoia-backed)

3w · **View on LinkedIn**

Most people dismiss a harness as just scaffolding

Until it makes a shitty model run like frontier.

When the model flops, most people's first reaction is "it isn't good enough, let me pay for a bigger one."

This open-source course blowing up on GitHub right now proves the fix isn't a pricier model. It's a better harness. I built my own research and content harness from scratch, and it's been one of the better resources.

Here is what's inside:

1/ The 5 parts of a harness

Instructions, state, verification, scope, lifecycle. The model decides what code to write. The scaffolding decides when, where, and how.

2/ Loop engineering

One agent writes, another checks its work, and you design that loop from outside it. How to stop hand-holding your agent through every step.

3/ Graph engineering

A loop is just a graph with one node. The moment you need parallel work, specialists, and recovery, it becomes a graph. What to build when one agent isn't enough.

4/ Frontier teardowns

Claude Code's layered memory, Codex treating the repo as the source of truth, DeepSeek's "everything is a plugin." How the real products build their harness under the hood.

5/ A skill that scaffolds one for you

Generates the instruction files, feature list, and setup script for you. Drops a working harness into your own repo in minutes.

6/ An audit script

Point it at any repo and it scores all 5 parts, then shows you exactly what's missing. The fastest way to see how weak your current setup really is.

14 lectures. 8 hands-on projects.

Free, MIT-licensed, translated into 15 languages.

Bookmark it before your next agent session: https://lnkd.in/grk6CNnT

What's the first thing you'd add to your agent's setup?

Repost so your team stops paying for a bigger model when the setup is the problem.

+ Follow me (Basia Kubicka) for more on building with AI.

LINKS
- https://www.basiakubicka.com/
- https://lnkd.in/grk6CNnT

#74 Ankur Dhawan

System Design

MTS-2 @Adobe | 112K+ @LinkedIn | 50+ Brand Collabs | AI • Tech • Marketing

3w · View on LinkedIn

I cracked Atlassian in my first attempt!

Here are System Design problems I repeatedly practised before the interviews:

1. Design a Project Management Tool Like Jira
2. Design a Real-Time Collaboration Tool.
3. Design a Scalable Notification System
4. Design a Search System for Knowledge Base Articles
5. Design an API Gateway for Atlassian Services
6. Design a Version Control System for Documentation
7. Design a Real-Time Analytics Platform
8. Design a Scalable User Authentication and Authorization System
9. Design a Workflow Automation System
10. Design a Logging and Monitoring System
11. Design a Rate Limiter
12. Design a parking
13. Database Design
14. Design Snake Game
15. Design a Ticketing System like Jira
16. Design a URL Shortening Service
17. Design a Notification System
18. Design a Distributed Messaging System
19. Design a Scalable Chat Application
20. Design a Job Scheduler

Atlassian Resources: https://lnkd.in/gfxB9sXh

4500+ Recruiter Contacts: https://lnkd.in/ghHxbXsf

DSA Roadmap: https://lnkd.in/g2wrixSY

Form: https://lnkd.in/gW6_Ffrs

One of my friend is providing referrals, drop comment 'Refer' and i will share it with you↓

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/gfxB9sXh
- https://lnkd.in/ghHxbXsf
- https://lnkd.in/g2wrixSY
- https://lnkd.in/gW6_Ffrs

#75 Serge Elman

Other

Head of ApplePay Client Services | Payment Solutions Integration Expert | Technical Project Management.

3w · View on LinkedIn

Payments Terminology of the Day: TLCM — Token Life Cycle Management

A payment token has a life beyond the moment it is created.

Imagine that a customer loses a phone containing a digital wallet. The physical card may still be safe, but the token associated with that device needs attention. The issuer can suspend that token while the situation is investigated, resume it if appropriate, or delete it when the device is permanently lost.

A different event calls for a different action: when a card is replaced, lifecycle management can update the relationship between the account and its tokens so eligible wallet or card-on-file payments continue without the customer adding the card again.

The common actions are:

• Suspend: Temporarily stop use of a token.

• Resume: Restore a suspended token.

• Delete: End that token’s use.

• Update: Keep token-related account information current.

The network names differ. Visa offers token and credential management through Visa Token Service (VTS). Mastercard documents token lifecycle operations through the MDES Customer Service API. American Express describes Life Cycle Management within American Express Token Service for Issuers (AETS).

The practical question for an issuer is: Which token is affected, on which device or payment channel, and what action protects the customer while preserving their other ways to pay?

That is why TLCM matters. One card can have multiple tokens, and a change to one token does not always mean changing them all.

#Payments #Tokenization #DigitalWallets #Visa #Mastercard #AmericanExpress

MEDIA document/carousel (possible)

#76 Naushad A.

Kafka / Messaging

Senior Specialist | 509k Engagement | Java 8 & 21 | Spring Boot | Microservices | AWS | Ex-@Capgemini

3w · **View on LinkedIn**

#HSBC Interview Experience | Senior Consultant↓

✓ All the Answer for the interview Questions including concepts, Coding, AWS, Kafka, HR FAQ, Non-Tech QnA are available in Single Kit - https://lnkd.in/dW_5jZie

• Introduce yourself with skill Set ?

• What is a Circuit Breaker in Microservices?

• What is Service Discovery in Microservices?

• Your thread pool gets exhausted under load. What's your approach?

• How do you register a microservice with a Service Discovery server?

• If Service A wants to call Service B, how does the complete Service Discovery flow work?

• Have you worked with Kafka?

• What is Spring Kafka and why do we use Kafka in an application?

• What is a Kafka Consumer?

• A HashMap is causing performance issues under heavy load. Why?

• Multiple threads are updating shared data incorrectly. How will you fix it?

• What is a Consumer Group in Kafka?

• How are Kafka partitions assigned to consumers within a Consumer Group?

• Can the same Kafka partition be consumed by consumers from different Consumer Groups?

• Have you worked with SonarQube and code coverage?

• How do you implement code coverage in an application?

• What is the purpose of code coverage?

• Why do we use SonarQube?

• Why are applications moving from Java 8 to Java 17 from a performance and security perspective?

• What is WebClient? and WebSocket

• What is the difference between RestTemplate and WebClient?

• Kafka Consumer Lag: Your Kafka consumer is slower than the producer and the lag keeps increasing. How would you troubleshoot it?

• What is CloudFront and Lambda function?

Congratulation Ankit, He easily cracked the interview because he prepared from Below Kit which provides Complete Interview Preparation with all java concepts in Single Kit -

✓Check the Kit here - https://lnkd.in/dW_5jZie

Candidate Selected✓

Follow Naushad A. for for such content and do LIKE and Comment, share with others

#HSBC #HSBCInterview #Java #JavaDeveloper #JavaBackend #SpringBoot #BackendDeveloper #CodingInterview #JavaInterview #NewInterviewQuestion

LINKS

- https://lnkd.in/dW_5jZie

#77 Sameer Bhardwaj AI / LLM

AI Engineer | Founder @Layrs (Voice AI) | Ex-Google, Flipkart | LLMs · Real-time Voice · Distributed Systems

3w · **View on LinkedIn**

I spent 8+ hours collecting resources so you can become an AI agent engineer without drowning in random tutorials.

If you are starting from zero, follow this order.

Phase 1: Understand the fundamentals

1. Learn AI Harness Engineering in 14 Minutes https://lnkd.in/gQASEFDm

2. Real AI Agent Stack: Harness → Loop → Graph

https://lnkd.in/gYdS-57T

3. AI Agent Graph Engineering in 21 Minutes

https://lnkd.in/gd-sbyQX

4. Loop Engineering + Graph Engineering

https://lnkd.in/geUiEJAD

Phase 2: actually build an agent harness

5. Claude Agent SDK Full Workshop, Anthropic

https://lnkd.in/gaP7JenF

6. Anthropic Workshop: Build Agents That Run for Hours

https://lnkd.in/gu4q-ZZt

7. Harness Engineering: Building the Production Cage, AWS

https://lnkd.in/gYj2m4WS

Phase 3: Learn context, memory, and orchestration

8. When to Build Your Own Agent Harness, Harrison Chase

https://lnkd.in/g_AYWWJj

9. Context Engineering Our Way to Long-Horizon Agents

https://lnkd.in/gzbjgGxD

10. Building an AI Agent From Scratch in Python

https://lnkd.in/gzAg64wR

11. What if the Harness Mattered More Than the Model?

https://lnkd.in/gFPDNgcx

Phase 4: production thinking

12. One Million Lines of Code. Zero Keystrokes.

https://lnkd.in/gXgYMQFm

13. Inside the Stack Behind Antigravity, Claude Code & Cursor
https://lnkd.in/gDj23Yzf
14. How AWS Thinks About Agent Harnesses
https://lnkd.in/gTT4wfUb
15. SE Radio: Harness Engineering for AI Agents
https://lnkd.in/gwH8QYx3
If you only have time for two long sessions:
→ Watch #5 to learn how to build
→ Watch #8 to understand when the harness itself becomes the product

LINKS

- http://layrs.me
- https://lnkd.in/gQASEFDm
- https://lnkd.in/gYdS-57T
- https://lnkd.in/gd-sbyQX
- https://lnkd.in/geUiEJAD
- https://lnkd.in/gaP7JenF
- https://lnkd.in/gu4q-ZZt
- https://lnkd.in/gYj2m4WS
- https://lnkd.in/g_AYWWJj
- https://lnkd.in/gzbjgGxD
- https://lnkd.in/gzAg64wR
- https://lnkd.in/gFPDNgcx
- https://lnkd.in/gXgYMQFm
- https://lnkd.in/gDj23Yzf
- https://lnkd.in/gTT4wfUb
- https://lnkd.in/gwH8QYx3

#78 Akash Namdev System Design

Senior Software Developer @ Citi | Backend & Distributed Systems | Java | Kafka | MongoDB | Payment Processing | Performance Engineering | 11+ Years Experience

4w · **View on LinkedIn**

11 Essential System Design Patterns

Architecture isn't about picking the "perfect" pattern — it's about picking the right one for the right problem. Here's a practical breakdown:

1. Microservices ★ Most Used
✓ Independent deployment, technology flexibility, fault isolation
✗ Complex to operate, network overhead, distributed debugging
Use case: E-commerce, fintech, large-scale apps
2. Event-Driven (EDA) ★ Most Used
✓ Loose coupling, highly scalable, real-time processing
✗ Harder to trace flow, eventual consistency challenges
Use case: Order processing, IoT, payments, notifications
3. Layered (N-Tier) ★ Most Used
✓ Simple, structured, well understood
✗ Can become rigid, changes ripple across layers
Use case: Web apps, enterprise systems
4. Monolithic
✓ Simple to develop, low operational overhead
✗ Hard to scale parts independently, slower for large teams
Use case: MVPs, small apps, internal tools

5. CQRS
✓ Optimized read/write paths, better scalability
✗ Added complexity, data sync overhead
Use case: High read/write load, complex domains
6. Event Sourcing
✓ Complete history, easy state rebuild
✗ Steep learning curve, storage grows fast
Use case: Audit trails, financial systems
7. Microkernel (Plugin)
✓ Easily extensible, supports plugins
✗ Core can become a bottleneck if overloaded
Use case: IDEs, CMS, extensible tools
8. Hexagonal (Ports & Adapters)
✓ Technology independence, easier testing
✗ More upfront design effort, added abstraction
Use case: Enterprise apps, domain-driven design
9. Serverless
✓ No server management, auto-scaling, cost-effective
✗ Cold starts, vendor lock-in, limited execution time
Use case: Event-driven workloads, APIs
10. Space-Based
✓ Extreme scalability, high availability
✗ Complex to design and manage state
Use case: High-traffic, low-latency systems
11. Peer-to-Peer (P2P)
✓ No central point of failure, high availability
✗ Harder to secure and coordinate, inconsistent performance
Use case: Decentralized systems, file sharing
Key Takeaway: There's no one-size-fits-all. Most modern systems blend multiple patterns (e.g., Microservices + Event-Driven + CQRS). Understand the trade-offs, start simple, and evolve as real needs emerge.
Which pattern are you working with right now? ↓
#SoftwareArchitecture #SystemDesign #Microservices #EventDrivenArchitecture #TechLeadership #Engineering #ScalableSystems #CQRS #Serverless

#79 Hisham Mosajo Java / Spring
Computer engineer
4w · **View on LinkedIn**

A common Java interview question is: What changed from Java 7 to Java 27?
The best answer is not to memorize every release, but to remember the major milestones.
Java 7 → try-with-resources, multi-catch, diamond operator, NIO.2.
Java 8 → lambdas, Streams, functional interfaces, Optional, Date/Time API.
Java 9 → modules, JShell, collection factory methods.
Java 10 → var for local type inference.
Java 11 (LTS) → HTTP Client, new String APIs, single-file source execution.
Java 14–16 → switch expressions, text blocks, records, pattern matching for instanceof.

Java 17 (LTS) → sealed classes.

Java 21 (LTS) → virtual threads, record patterns, pattern matching for switch, sequenced collections.

Java 22 → unnamed variables/patterns and the Foreign Function & Memory API.

Java 25 (LTS) → module imports, compact source files, instance main methods, flexible constructor bodies.

Java 27 → further runtime, performance, and security improvements.

In interviews, the important part is not listing every feature.

Be ready to explain: why a feature matters, when you would use it, and at least one practical example.

That shows real Java experience, not just memorization.

#Java #JavaInterview #SoftwareEngineering #BackendDevelopment #SpringBoot

#80 Alex Li

Java / Spring

Senior JavaEE Developer at Giesecke+Devrient

4w · **View on LinkedIn**

The Java 8 Stream API provides a powerful, functional way to process collections of data. While the core java.util.stream.Stream interface itself contains fewer than 50 methods, a complete toolkit for working with streams includes factory methods for initialization, intermediate operations, terminal operations, specialized primitive streams (IntStream, LongStream, DoubleStream), and the highly essential static methods found in java.util.stream.Collectors.

Here are the 50 most frequently used Java 8 stream methods and operations, grouped by their function in a pipeline:

> **MEDIA** advertisement

LINKS

- http://java.util.stream.Stream

#81 Naushad A.

Java / Spring

Senior Specialist | 509k Engagement | Java 8 & 21 | Spring Boot | Microservices | AWS | Ex-@Capgemini

4w · **View on LinkedIn**

#Virtusa Interview Experience for #Citi client | Java | Senior Consultant ↓

✓ All the Answer for the interview Questions including concepts are available in Single Kit - https://lnkd.in/dW_5jZie

1. Spring Boot Features: What are the main features of Spring Boot that you use in your day-to-day work?

2. What is a memory leak in Java if Garbage Collection is automatic?

3. How does the JVM determine whether an object is eligible for Garbage Collection?

4. What are GC Roots?

5. Optimistic vs. Pessimistic Locking: Explain the difference between optimistic and pessimistic locking.

6. What are the most common causes of memory leaks in Java?

7. How can static collections cause memory leaks?

8. How can ThreadLocal cause a memory leak?

9. How can unbounded caches cause memory problems?

10. How can listeners and callbacks cause memory leaks?

11. What is the difference between a memory leak and high memory usage?

12. OutOfMemoryError vs StackOverflowError?

13. What is a Heap Dump?
14. How do you capture a Heap Dump from a production JVM?
15. How do you analyze a Heap Dump?
16. What is a Dominator Tree?
17. How do you identify objects consuming most of the heap?
18. What is a Retained Heap?
19. Heap Dump vs Thread Dump?
20. Complex Production Scenario: What is a complex scenario or problem you have faced recently, and what techniques did you use to troubleshoot it?
21. How would you investigate memory issues in a Java application running inside Kubernetes?

✓ Candidate Selected !!

Rohit easily cracked the interview because he prepared from Below Kit which provides Complete Interview Preparation with all java concepts in Single Kit -

✓Check the Kit here - https://lnkd.in/dW_5jZie

✓1: 1 Guide from Top IT Professional - https://lnkd.in/dnbMDfRj

Follow Naushad A. and stay tune for next Interview Questions and do LIKE and Comment, share with others

#Virtusa #Java #JavaDeveloper #JavaBackend #SpringBoot #BackendDeveloper #CodingInterview #JavaInterview #SoftwareEngineer #DSA #SystemDesign #TechInterview #InterviewPreparation #Programming #Developers

LINKS

- https://lnkd.in/dW_5jZie
- https://lnkd.in/dnbMDfRj

#82 Mahesh Mallikarjunaiah ↗ AI / LLM

AI Executive & Generative AI Transformation Leader | Driving Enterprise Innovation & AI Community Growth | From Idea to Intelligent Product | Driving Technology Transformation | AI community Builder

4w · **View on LinkedIn**

What actually happens inside an LLM when you send a prompt.

Here is an explanation what actually happens under the hood

User → Client → SDK

You type in the web or mobile UI, and the client wraps your message into a streaming-ready API call—often via an SDK that keeps the connection open so tokens stream back as soon as they’re ready.

API Gateway / Provider

Your request hits an entry gate: auth with an API key, rate limiting, request validation, and logging. If anything fails here, the LLM never sees your prompt.

Multi-Layer Caching (Primary)

- Exact-match cache: hash(prompt) → if the same exact prompt + context exists, return the stored response.

- Semantic cache: embed the prompt, search; if it’s close enough, reuse the prior answer.

- Prompt cache (prefix): reuse the tokenized system + context + history prefix so the LLM only recomputes the suffix.

A green “Cache HIT → Instant Response” shortcut bypasses the transformer entirely; that’s the fast path you want to hit as often as possible.

Retrieval Augmentation (RAG, optional)

If enabled, your query is embedded and top-k docs are pulled from a vector DB. Those context snippets are injected into the prompt so the LLM reasons over them instead of only its weights.

Prompt Construction (Primary)

The system assembles four pieces: system instructions, retrieved context, conversation history, and your latest user prompt. The ordering is designed for prefix stability—exactly what makes prompt caching viable.

Tokenization & Context Window

Text goes into a BPE-style tokenizer, and all pieces stack into a single token budget. Exceed the model's limit and you're truncating or chunking.

Transformer Inference Engine (Primary)

Tokens → embedding layer → positional encoding → stacked attention and feed-forward layers. The KV cache tracks past keys/values so each new token doesn't re-compute everything. This is where most latency and cost live.

Decoding Strategy

Right after the transformer, decoding shapes the output: greedy, top-k, top-p, temperature—the knob between "safe" and "creative."

Post-Processing

Tokens are detokenized into text, then run through safety filters, moderation, and structural formatting (e.g., JSON for tool calls). This is where you scrub content that passes the model but not your policy.

Response Streaming & Delivery

Tokens stream token-by-token back to the client, making the response feel faster than the total wall-clock time.

Latency & Edge Cases

Cache hits run in milliseconds; retrieval adds 10–100 ms; LLM inference adds 100–2000+ ms. The "Failure / Edge Cases" panel reminds you of risks: cache miss, token limit exceeded, hallucination, retrieval mismatch, rate limits.

Bottom lifecycle strip

The band compresses everything into: User → API → Cache → Retrieval → Prompt → Tokens → LLM → Decode → Response.

#GenAI #SystemsThinking #EngineeringLife

#83 Prem N.

AI / LLM

AI Transformation Leader | AI Adoption & Enablement | Evangelist | Perplexity Fellow | 28k Community Builder

4w · View on LinkedIn

Not all AI processors are built for the same job.

We often talk about "AI compute" as if it is one thing.

But under the hood, modern AI systems depend on very different types of processors, each optimized for a different part of the workload.

CPU, GPU, TPU, NPU, LPU, and DPU all play very different roles.

A simple way to think about it:

CPU → General orchestration, preprocessing, control flow

GPU → Massive parallelism for training and deep learning

TPU → Large-scale tensor workloads, especially in Google Cloud

NPU → Low-power AI inference on phones, laptops, and edge devices

LPU → Real-time, low-latency LLM inference

DPU → Data-center infrastructure, networking, security, and offload

The important point:

There is no "best" processor for AI.

There is only the right processor for the right workload.

For enterprise AI systems, the decision usually comes down to:

Latency

Parallelism

Power efficiency

Cost

Scale

Deployment environment

A model running on a phone has very different constraints than a model training on thousands of accelerators.

A real-time LLM serving system has different needs than a data-center networking layer.

And a production AI platform rarely depends on just one type of processor.

It is usually a carefully designed combination of compute, memory, networking, orchestration, and infrastructure.

That is why understanding the AI hardware stack is becoming just as important as understanding the model stack.

I created this visual breakdown to simplify the differences.

Which one do you think will become more important in the next wave of AI infrastructure: GPU, NPU, LPU, or DPU?

Credit: Brij Kishore Pandey

Repost this to help your network get started

+ Follow Prem for more

#84 Yogesh Kshirsagar Java / Spring

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

4w · **View on LinkedIn**

Spring Boot Under the Hood — Vol. 21 of 30

You wrote one line.

Spring started a web server.

SpringApplication.run(MyApplication.class, args);

But who actually starts Tomcat?

Let’s follow the request ↓

You start your Spring Boot application...

1. Spring creates the ApplicationContext

↓

2. It discovers a WebServerFactory

For a servlet-based application, Spring Boot typically finds:

TomcatServletWebServerFactory

↓

3. The factory creates the embedded Tomcat

No external Tomcat installation.

No WAR deployment.

Tomcat becomes part of your application.

↓

4. The Servlet container is configured

Spring Boot sets up things like:

Servlet context
Connectors
Filters
Servlet mappings
Error handling
↓
5. Port binding happens
The connector binds Tomcat to:
server.port → 8080 by default
And suddenly...
Your application is listening for HTTP requests.
The interesting part?
SpringApplication.run() doesn't directly say:
"Start Tomcat!"
Instead, Spring Boot uses a WebServerFactory abstraction.
That means the same startup mechanism can work with different embedded servers.
Spring Boot → WebServerFactory → Embedded Server → Port
That's one of the reasons Spring Boot feels so simple on the surface while doing so much underneath.
Follow along as we go deeper into Spring Boot internals
#SpringBoot #SpringFramework #Java #SpringInternals #BackendEngineering #JavaDeveloper #SoftwareEngineering #UnderTheHood

MEDIA advertisement

#85 Alex Xu

AI / LLM

Author of 4 Bestselling Books | Co-Founder of ByteByteGo
4w · View on LinkedIn

Jev is TypeSafe AI's first System One Model. It is 100x faster and cheaper than frontier LLMs.
That opens up a lot of use cases people usually skip because the big models are too slow or too expensive for them.
Here are the top 9 places to use Jev instead of an LLM:
1-Model routing. Jev takes the prompt as input and routes it to a proper LLM.
2-Guardrails. Detect security risks in the prompt first. Then pass it to the LLM.
3-Gating tool-calls. Classify agent tool calls into one of their permission categories.
4-Triage inbox. Classify lots of emails into categories (e.g., spam, urgent, archive).
5-Reranking. Score passages against a query (prompt) so we can rank them by relevance.
6-LLM evals. Evaluate an LLM's output and return a score within a range.
7-Bulk labeling. Label lots of rows from a huge table fast and cheap (map-reduce style).
8-Real-time decisions. Use cases where a fast decision is needed in a loop (e.g., trading).
9-Confidence gate. Classify anything based on confidence and act properly.
The bottomline is to use the LLM for generations and use Jev on the decisions around it.
What are other places to use Jev instead of an LLM?
--
Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages):
https://lnkd.in/gF7Vsw6X
#systemdesign #coding #interviewtips

LINKS
- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gF7Vsw6X

#86 Vishal Kumar

System Design

Senior Software Engineer | ASP.NET Core | C# | Azure | AI/ML | Generative AI

4w · **View on LinkedIn**

Recently, I was learning about Microservices and API Architecture and came across two commonly confused concepts:

Load Balancer → Distributes traffic across healthy instances.

API Gateway → Manages API traffic, routing, authentication, rate limiting, etc.

Sharing a simple architecture diagram to visualize the difference and how they can work together.

Learning and improving every day!

#DotNet #Microservices #ASPNetCore #APIGateway #LoadBalancer #Azure #SoftwareArchitecture

#87 Naushad A.

Java / Spring

Senior Specialist | 509k Engagement | Java 8 & 21 | Spring Boot | Microservices | AWS | Ex-@Capgemini

4w · **View on LinkedIn**

#Fulcrum Interview Experience for #Mastercard client for my Friend | Java + AWS | Senior Consultant ↓

Round 1 – Technical Interview(internal)

1. Future vs. CompletableFuture: Explain the difference between Future and CompletableFuture and provide a real-life example where this concept is used.

2. Fail-Fast vs. Fail-Safe Iterators: Explain each one with an example.

3. HashMap vs. ConcurrentHashMap: Explain the differences. Also, explain one major production defect you faced and how you fixed it.

4. Thread-Safe HashMap: How would you make a HashMap thread-safe?

5. Polymorphism: Explain the different types of polymorphism, particularly runtime polymorphism, with an example.

6. SOLID Principles: Explain the SOLID principles, with a focus on the Liskov Substitution Principle (LSP) and provide a code example.

7. @SpringBootApplication: Explain what this annotation does and which annotations it combines.

8. PUT vs. PATCH: Explain the difference between PUT and PATCH APIs.

9. @Controller vs. @RestController: Explain the difference between @Controller and @RestController in Spring Boot.

10. HTTP 429 Status Code: When does a server return a 429 Too Many Requests status code, and how would you handle it?

11. Factory Pattern: Explain the Factory Design Pattern and provide a code example.

12. Microservices Communication: How do your microservices communicate with each other?

13. Authentication and Authorization: How are authentication and authorization implemented in your application?

14. AWS Services: Which AWS services have you used in your application, and how does your Java application integrate with AWS services?

✓ Result: Selected for Round 2(client round)

He easily crack the interview because he prepared from Below Kit which provides Complete Interview Preparation with all java concepts in Single Kit -

✓Check the Kit here - https://lnkd.in/dW_5jZie

Follow Naushad A. and stay tune for 2nd round result and do LIKE and Comment, share with others

#Fulcrum #Java #JavaDeveloper #JavaBackend #SpringBoot #BackendDeveloper #CodingInterview #JavaInterview #SoftwareEngineer #DSA #SystemDesign #TechInterview #InterviewPreparation #Programming #Developers

LINKS

- https://lnkd.in/dW_5jZie

#88 Neo Kim

Other

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

4w · **View on LinkedIn**

How to design APIs like a Senior Engineer

LINKS

- https://newsletter.systemdesign.one/join

#89 Nageswararao Dongari

System Design

software developer

1mo · **View on LinkedIn**

Functional vs Non-Functional Requirements in System Design

Before designing APIs, databases, microservices, or distributed systems, there is one important step:

→ Understand the requirements first.

Functional Requirements — WHAT the system should do

These describe the features and actions the system must provide.

Examples:

- User registration & login
- Search
- Upload/download
- Send messages
- Make payments
- Book tickets
- Track orders

Non-Functional Requirements — HOW WELL the system should work

These describe the quality, performance, and constraints of the system.

Examples:

Performance — Response within 2 seconds

Scalability — Support millions of users

Availability — 99.99% uptime

Security — Protect user and payment data

Reliability — Prevent data loss or duplicate transactions

Maintainability — Easy to modify and deploy

Throughput — Handle thousands of requests per second

Easy way to remember:

Functional = WHAT?

→ What should the system do?

Non-Functional = HOW WELL?

→ How fast, secure, scalable, reliable, and available should it be?

Why does this matter?

Requirements directly influence the architecture.

For example:

1,000 users → A simple architecture may be enough.

10 million users → You may need:

Load Balancer + Caching + Database Replication + Message Queue + CDN + Horizontal Scaling.

So, don't start system design by drawing boxes.

Start with requirements → then design the architecture.

Key takeaway:

«“Functional requirements tell you WHAT to build; non-functional requirements tell you HOW WELL it must work.”»

#90 Sairam Sundaresan

AI / LLM

AI Engineering Leader | Author of AI for the Rest of Us | I help engineers land AI roles and companies build valuable products

1mo · View on LinkedIn

You can cut token use by 49% and API costs by 33% without changing the model.

All you need to change is the harness.

Each step a coding agent takes adds more context to the next model call: conversation history, file changes, test results, logs and tool outputs.

Much of that context gets sent repeatedly. The model rereads old output, and simple follow-up actions trigger more calls.

The longer the task runs, the more expensive this becomes.

Researchers from NVIDIA, MIT and NTU built a harness called SoL-Pi to reduce that waste. They tested roughly 150 ideas across more than 3,000 runs and kept four changes.

1. The harness combines steps that usually happen one after another, such as editing a file and running the test.

2. It shortens the context when doing so will save money.

3. It stores large outputs instead of sending them repeatedly.

4. It also gives routine log-reading to a cheaper model and passes the useful evidence to the main model.

The same harness worked with both GPT-5.6 Sol and Claude Opus 5. In both cases, it cut token use by about half and API costs by about a third, with a small drop in task performance.

If you run agents at scale, every repeated tool output, unnecessary model call and oversized context adds to your bill.

Before switching models or cutting prompts, check how much you’re spending to make the current model read the same context again.

Agent Harness Design

.

15 pages

MEDIA document/carousel (possible)

LINKS

- https://www.amazon.com/AI-Rest-Us-Illustrated-Introduction/dp/B0F29THNLT

#91 Yogesh Kshirsagar

Java / Spring

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

1mo · View on LinkedIn

Spring Boot Under The Hood - 20 of 30

You call a @Bean method twice.

So why don’t you get two objects?

That question leads to one of Spring's cleverest tricks.

Imagine:

```
@Configuration
class AppConfig {
@Bean
OrderService orderService() {
return new OrderService(repository());
}
@Bean
OrderRepository repository() {
return new OrderRepository();
}
}
```

At first glance, repository() looks like an ordinary Java method.

Call it twice →

Shouldn't it create two objects?

But Spring has a different plan.

When Spring processes @Configuration, it can enhance the configuration class using CGLIB.

Instead of letting every @Bean call simply execute the method, the enhanced class intercepts the call and checks the Spring container.

Already created? → Return the existing bean.

Not created yet? → Create it, register it, then return it.

So this:

repository()

can effectively become:

“Spring, give me the managed repository bean.”

That's the real magic

You write a normal-looking Java method.

Spring turns the configuration class into something smarter.

Your code defines the recipe.

Spring controls the objects.

And that's why @Configuration enhancement matters.

→ Follow along as we continue uncovering what Spring is actually doing behind the scene.

#SpringBoot #Java #SpringFramework #AOP #JDKProxy #CGLIB #BackendEngineering #Microservices #SoftwareEngineering #UnderTheHood

MEDIA advertisement

#92 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · View on LinkedIn

Your database is not slow.

Your indexes are wrong.
Every engineer reaches a point where adding RAM stops fixing things.
That's when they finally learn indexing properly.
Here are the 8 indexes that actually move the needle - and when to use each ↓
B-Tree (the default)
For equality + range queries. ORDER BY, BETWEEN.
The one you already use, whether you know it or not.
Hash
For pure equality lookups. O(1) speed. No ranges, no sorts.
Composite
For multi-column WHERE filters. Get the column order wrong, and it's useless.
Bitmap
For low-cardinality columns. Brilliant on reads, painful on writes.
Covering
Includes every column the query needs. The DB never touches the table.
Partial
Indexes a subset of rows. Smaller. Faster. Cheaper.
Full-Text
For keyword search and search-as-you-type. Goodbye, brute-force LIKE.
GIN / GiST
For JSON columns, arrays, and geospatial data. Postgres magic.
Good engineers know SQL.
Great engineers know which index to reach for.
The difference is a query that takes 3 seconds vs 3 milliseconds.
Save this. Show it to anyone who says "we need a bigger database."
Follow for more such insights!!

LINKS

- https://www.youtube.com/@learnwithrockybhatia
- https://www.linkedin.com/feed/update/urn:li:activity:7474784127033057281

ALSO SAVED AS Rocky Bhatia (3mo) link

#93 NIPIN G.

Java / Spring

Java Developer | Backend Engineering | Exploring Machine Learning & AI | ISTQB CTFL
1mo · **View on LinkedIn**

Java’s ReentrantLock is like a car key?
Imagine you have a physical car key in your hand. You use this car key to
1. Unlock and enter the car (lock acquired, count = 1).
2. Start the car engine using the same key (lock acquired, count = 2). Since you hold a valid key, you can start the engine without getting locked out.
3. Switch off the engine (lock released, count remaining = 1).
4. Exit the car and lock it (lock released, count remaining = 0).
This is exactly how ReentrantLock works in Java.
It allows the same thread to acquire the lock multiple times without getting blocked.
The ReentrantLock must be unlocked by that thread the exact same number of times before any other thread can acquire it.

#94 Ankit Pangasa

System Design

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

1mo · View on LinkedIn

How Databases Actually Work

We interact with databases every day.

You log in → user data is fetched.

You search → matching results appear.

You make a payment → a transaction is recorded.

But what happens between sending a SQL query and getting the result?

Let’s break it down ↓

1. Send a Query

Your application sends a SQL query through a database driver.

SELECT * FROM users WHERE id = 101;

The database receives the request and starts processing it.

2. Parsing & Validation

The database checks:

✓ Is the SQL syntax valid?

✓ Do the table and columns exist?

✓ Does the user have permission?

The query is then converted into an internal representation.

3. Query Optimization

Imagine the users table has 100 million rows.

The database could scan every row to find id = 101.

But if an index exists on id, it can find the data much faster.

The query optimizer evaluates possible execution plans and considers things like:

Indexes

Table statistics

Join strategies

Estimated cost

It then chooses an execution plan.

4. Query Execution

The database executes the selected plan.

A simplified path could be:

Index → Memory/Cache → Disk

If the required data is already in memory, a disk read may not be necessary.

5. Data Storage

Databases organize persistent data using structures such as:

Pages/Blocks

B-Trees

Indexes

Data Files

Buffer Pools

These structures help databases efficiently read and write large amounts of data.

6. Results Are Returned

Once the required data is found, the database sends the results back to your application.

And all of this can happen in milliseconds.

What Makes Databases Fast?

Indexes → Find data efficiently

Caching → Keep frequently used data in memory

Query Optimization → Select efficient execution plans

Connection Pooling → Reuse connections

Partitioning → Split large datasets

Replication → Maintain copies of data

Sharding → Distribute data across servers

What About Reliability?

Databases also need to handle:

Transactions & ACID — Reliable data changes

Concurrency — Multiple users accessing data simultaneously

Isolation — Control what transactions can see

Replication — Maintain multiple copies

Backup & Recovery — Recover from failures

The Big Picture

A database isn't just a place to store data.

It's a system that:

Parses → Validates → Optimizes → Executes → Reads/Writes → Manages concurrency → Returns results

Once you understand this flow, concepts like indexes, B-Trees, transactions, locks, caching, replication and sharding become much easier to understand.

Save this for your Database + System Design interview preparation.

#Database #SQL #SystemDesign #BackendDevelopment #SoftwareEngineering #TechInterview #DistributedSystems

#95 Sathish Kumar Subramani

System Design

Senior Engineering Manager | Generative AI | Azure Cloud Solutions | Leadership Excellence | Driving Operational Efficiency

1mo · **View on LinkedIn**

Your AI app is only as good as the database beneath it.

Slow queries, duplicate records, lost updates, and outages rarely start in the code. They start with a database decision nobody fully understood.

Here are 15 concepts every engineer should know:

1. Entity Relationship

• How entities relate, like one user with many orders

• Insight: Get relationships wrong early, and every feature fights the schema.

2. Primary and Foreign Keys

• Link rows across tables

• Insight: Keys are how your data stays connected and trustworthy.

3. Normalization

• Split tables to remove duplication

• Insight: Great for data integrity, but more joins.
4. Denormalization
• Merge tables for faster reads
• Insight: You trade storage and consistency for speed.
5. Data Types
• int, text, date, bool, and more
• Insight: The wrong type quietly wastes space and breaks comparisons.
6. Constraints
• NOT NULL and UNIQUE rules every row must pass
• Insight: Let the database reject bad data before your app has to.
7. SQL vs NoSQL
• Fixed schema versus flexible JSON
• Insight: Choose based on access patterns, not trends.
8. Query Execution Plans
• How the database actually runs your query
• Insight: Before adding hardware, read the plan.
9. Indexing
• Jump straight to matching rows
• Insight: Indexes speed up reads but slow down writes.
10. ACID Properties
• Atomicity, Consistency, Isolation, Durability
• Insight: All or nothing is what keeps payments trustworthy.
11. Isolation Levels
• What running transactions can see from each other
• Insight: Higher isolation means safer data, but less throughput.
12. Concurrency Control
• Locks keep writers in line and prevent lost updates
• Insight: Two users buying the last item is a concurrency problem, not a UI bug.
13. Replication
• Copies data for faster reads and failover
• Insight: Replicas protect uptime, but can lag behind the primary.
14. Sharding
• Split data across multiple databases
• Insight: Powerful at scale, painful to undo. Delay it until you truly need it.
15. Consistency Models
• Strong now or eventual later
• Insight: Banking needs strong. Social feeds can live with eventual.
The pattern:
Design it well. Query it smart. Protect every transaction. Scale only when you must.
Which concept caused your toughest production issue?
Repost to help an engineer strengthen their fundamentals
+ Follow Sathish for more practical tech breakdowns
#Databases #SQL #SystemDesign #DataEngineering

#96 Ankit Pangasa

Cloud / K8s / DevOps

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

1mo · **View on LinkedIn**

Monitoring vs Observability: They Are NOT the Same

One of the most common misconceptions in system design and production engineering is treating monitoring and observability as synonyms.

They aren't.

Think of it this way:

Monitoring = “Is something wrong?”

Observability = “Why is it wrong?”

Monitoring

Monitoring focuses on predefined signals and known failure conditions.

You typically monitor:

• CPU & memory usage

• Error rates

• Request latency

• Traffic

• Disk usage

• Service availability

• Queue depth

For example:

«“API error rate crossed 5%.”»

Monitoring can alert you that something is going wrong.

But it may not tell you the underlying cause.

Observability

Observability is about being able to understand the internal state of a system from the data it produces.

The three commonly discussed pillars are:

Metrics - What is happening?

Logs - What happened?

Traces - Where did it happen across the request path?

Imagine a payment request:

User → API Gateway → Payment Service → Database → External Payment Provider

Monitoring might tell you:

✗ “Payment API latency increased.”

Observability can help you discover:

→ API Gateway is healthy

→ Payment Service is healthy

→ Database queries are normal

→ External payment provider is taking 4 seconds

Now you know WHERE the problem is.

The key difference

- Monitoring is generally based on questions you already know to ask.

- Observability helps you investigate questions you didn't anticipate.

That distinction becomes extremely important as systems become:

→ Distributed

→ Microservice-based
→ Event-driven
→ Cloud-native
→ Dependent on third-party services
A simple application might survive with basic monitoring.
A large distributed system needs strong observability.
Interview tip:
If asked:
“Why do we need observability when we already have monitoring?”
A simple answer is:
«Monitoring detects known problems. Observability gives you the context to investigate unknown problems.»
That's the difference between:
“Something is broken.”
and
“I can understand what broke, where it broke, and why.”
What does your production stack rely on more today, traditional monitoring or full observability?
#SystemDesign #SoftwareEngineering #Observability #Monitoring #DevOps #SRE #DistributedSystems #CloudComputing

#97 Neo Kim Other

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1mo · **View on LinkedIn**

If you want to become a 10x software engineer,
read these 12 blogs (save now):
1 Google Research
2 Meta Engineering
3 AWS Blog
4 Microsoft Engineering
5 Netflix Tech Blog
6 Figma Engineering
7 Reddit Engineering
8 Spotify Engineering
9 Slack Engineering
10 Uber Engineering
11 OpenAI Engineering
12 Cloudflare Tech Blog
Anything else you'd add?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 250K+ software engineers:
→ https://lnkd.in/ehTrcyak
===
Repost to help others learn system design.
Follow Neo Kim + turn on notifications.

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#98 Rocard Fonji F.

Other

Senior Software Engineer | Fintech, Payments & Digital Banking | Remittance & Payment Gateways | Stablecoin On-Ramp/Off-Ramp | Cross-Border B2B Invoice Payments

1mo · **View on LinkedIn**

FINTECH & REMITTANCE SYSTEMS (3/3)

The real goal of a fintech or remittance system isn’t speed.

It’s correctness under failure.

Fast transactions are useful.

But when you are moving money across banks, mobile wallets and payment providers, being wrong quickly is worse than being right a few seconds later.

A few principles matter a lot:

• The ledger should be the financial source of truth

External providers tell you what happened on their side.

Your ledger tells you what happened inside your platform.

• Every transaction should be idempotent

Retries will happen.

They should never create duplicate payments.

• Never trust one API response as final truth

A timeout does not always mean failure.

A successful response does not always mean settlement.

• Reconciliation should be part of the architecture

Not something you add later when mismatches start appearing.

In financial systems, reliability is not just uptime.

It is knowing that every unit of money can be accounted for, even when something fails halfway through.

That is the standard payment infrastructure should be designed around.

#Fintech #Remittance #Payments #Reconciliation #Ledger #PaymentEngineering

#99 Suraj Mishra

Java / Spring

Technical Lead & Staff Software Engineer @ PayPal

1mo · **View on LinkedIn**

Spring Boot Interview Question – Improve API Latency

Scenario

You are working on a Spring Boot API that returns large JSON responses (100 KB – 2 MB). Your Spring Boot API serves large JSON responses (100 KB – 2 MB).

Recently, partners have reported that API response times have increased.

Seems like there was a new change made to the endpoint that shot up the latency. The changes are following:

Some facts about the system:

- Redis is distributed / clustered

- CPU on app servers is high

- API latency at P95/P99 is increasing

- Cache reads happen very frequently

Solution with detailed explanation

https://lnkd.in/gbhAiq7v

Subscribe for more such content:

https://lnkd.in/gwiRqWBV

#java #spring #springboot #interview

LINKS

- https://lnkd.in/gbhAiq7v
- https://lnkd.in/gwiRqWBV

#100 Sameer Bhardwaj System Design

AI Engineer | Founder @Layrs (Voice AI) | Ex-Google, Flipkart | LLMs · Real-time Voice · Distributed Systems

1mo · **View on LinkedIn**

If I travel back to my Interview Days, I will study these 35 Engineering blogs to learn system design lessons.

This list includes blogs from top engineering teams from all over the world and how they think about architecture.

If you're preparing for system design interviews or learning for the first time check out our AI Tutor: https://lnkd.in/gcWfR7jW

You can voice chat, get real-time feedback, learn concepts, and practice HLD questions even if you're starting from scratch.

1. Google System Design Principles

https://lnkd.in/gPKFwSYD

2. Meta Distributed Priority Queue

https://lnkd.in/gBBr4Vjq

3. Microsoft Platform Engineering Paths

https://lnkd.in/gymvyfbd

4. Adobe Unified Search Architecture

https://lnkd.in/g_SNVuzE

5. Salesforce Guaranteed Data Delivery

https://lnkd.in/gxCVtXZu

6. AWS Event Driven Architecture

https://lnkd.in/gT463eWY

7. Netflix Distributed Tracing Infrastructure

https://lnkd.in/gESgRhWU

8. Uber Rider App Architecture

https://lnkd.in/gGbxqbpU

9. Airbnb Key Value Architecture

https://lnkd.in/gqTSagR4

10. Dropbox Magic Pocket Architecture

https://lnkd.in/gfdfk7FV

11. Pinterest Cache Infrastructure Scaling

https://lnkd.in/g3NZnxAm

12. Slack Cellular Architecture Migration

https://lnkd.in/gtnkNEzF

13. Spotify Backend Infrastructure Architecture

https://lnkd.in/gCebV4sR
14. Cloudflare Global Edge Architecture
https://lnkd.in/gfAq7JTH
15. Stripe Database Migration Platform
https://lnkd.in/gVvP7VWQ
16. Shopify Sharded Monolith Changes
https://lnkd.in/g_KZ-BA4
17. DoorDash Microservices Migration Journey
https://lnkd.in/gBC5E3g2
18. Discord Trillion Message Indexing
https://lnkd.in/gRkk_d9G
19. Twilio Exactly Once Delivery
https://lnkd.in/ga7Zfank
20. Datadog Husky Event Store
https://lnkd.in/gWcZgw3S
21. Atlassian Cloud Engineering Architecture
https://lnkd.in/gyaGJxHY
22. PayPal Kafka Scaling Architecture
https://lnkd.in/gBRT8R-P
23. eBay Distributed Listing Architecture
https://lnkd.in/gqUVQRiW
24. Walmart Autocomplete Backend Rebuild
https://lnkd.in/gbVKZT2p
25. Capital One Resilient Systems
https://lnkd.in/gSv385XX
26. Canva Analytics Event Pipeline
https://lnkd.in/gReCsAkW
27. Figma Multi Database Scaling
https://lnkd.in/gQqayzyk
28. Razorpay Reliable Dual Writes
https://lnkd.in/gRiV9ypn
29. PhonePe Distributed Job Scheduler
https://lnkd.in/gZ4ZVDkN
30. Zomato Billing Platform Scaling
https://lnkd.in/g9kcikQy
31. Coinbase Solana Processing Architecture
https://lnkd.in/gyYwXm8g
32. Etsy Kafka Zonal Resiliency
https://lnkd.in/gJcnfTer
33. Expedia Configuration Management Platform
https://lnkd.in/gkj5GerG
34. Instacart Multi Database Scaling
https://lnkd.in/gUfawb2B
35. GitHub Distributed Storage System
https://lnkd.in/gDAAq6RP

LINKS

- http://layrs.me
- https://lnkd.in/gcWfR7jW
- https://lnkd.in/gPKFwSYD
- https://lnkd.in/gBBr4Vjq
- https://lnkd.in/gymvyfbd
- https://lnkd.in/g_SNVuzE
- https://lnkd.in/gxCVtXZu
- https://lnkd.in/gT463eWY
- https://lnkd.in/gESgRhWU
- https://lnkd.in/gGbxqbpU
- https://lnkd.in/gqTSagR4
- https://lnkd.in/gfdfk7FV
- https://lnkd.in/g3NZnxAm
- https://lnkd.in/gtnkNEzF
- https://lnkd.in/gCebV4sR
- https://lnkd.in/gfAq7JTH
- https://lnkd.in/gVvP7VWQ
- https://lnkd.in/g_KZ-BA4
- https://lnkd.in/gBC5E3g2
- https://lnkd.in/gRkk_d9G
- https://lnkd.in/ga7Zfank
- https://lnkd.in/gWcZgw3S
- https://lnkd.in/gyaGJxHY
- https://lnkd.in/gBRT8R-P
- https://lnkd.in/gqUVQRiW
- https://lnkd.in/gbVKZT2p
- https://lnkd.in/gSv385XX
- https://lnkd.in/gReCsAkW
- https://lnkd.in/gQqayzyk
- https://lnkd.in/gRiV9ypn
- https://lnkd.in/gZ4ZVDkN
- https://lnkd.in/g9kcikQy
- https://lnkd.in/gyYwXm8g
- https://lnkd.in/gJcnfTer
- https://lnkd.in/gkj5GerG
- https://lnkd.in/gUfawb2B
- https://lnkd.in/gDAAq6RP

#101 Java Developers Community (moderated)

System Design

Sameer Paradkar • 3rd+

1mo · **View on LinkedIn**

Most engineers leave load balancers on round robin and never touch the setting again. That default is quietly overloading your biggest server and underusing your smallest one, and almost nobody notices because the traffic "looks" evenly split.

Round robin assigns requests sequentially, one to each server in strict order, regardless of what each server can actually handle or how busy it currently is. That's fine when every server is identical and every request costs the same. It falls apart the moment either assumption breaks. Weighted round robin fixes the capacity problem, distributing requests based on predefined server weights so a bigger box gets proportionally more traffic. But it's still static. It doesn't know that Server 2 is drowning in slow requests right now while Server 3 sits nearly idle.

That's what least connections and least response time actually solve, and they solve different problems than people assume. Least connections routes to whichever server has the fewest active connections in real time, catching the case where one server is quietly backed up even though it received the same request count as everyone else. Least response time goes further, factoring in both current latency and connection count, so it can route around a server that's technically available but measurably slower right now. Neither of these is "smarter round robin." They're solving for real-time load, not request count.

The other two algorithms aren't about performance at all, and conflating them with the first four is where teams get really confused. Source IP hash exists purely for session persistence, routing the same user to the same server every time so session state doesn't break. URL or URI hash routes based on the actual request path, sending /api traffic to one server pool and /images traffic to another, content-aware routing, not load distribution.

The uncomfortable truth: picking an algorithm because it's the default, not because it matches your traffic pattern, is how "load balanced" systems end up with one server running hot while three others coast.

Which algorithm is your load balancer actually running, and did anyone choose it deliberately?

I share practical insights on system design & architecture, AI, platform engineering, and interview preparations. Join here: https://lnkd.in/egW679gc

LINKS

- https://lnkd.in/egW679gc

#102 Manochitra Selvam

System Design

Principal Application Architect | Lead Software Engineer | Principal Engineer | Product and Engineering Leadership | Payment Network Domain Specialist @ Discover Financial Services | AWS Certified Cloud Practitioner

1mo · **View on LinkedIn**

Java Interview Series #3 — Idempotency in Payment Processing

What happens when a payment succeeds, but the response never reaches the customer?

The client sees a timeout and retries.

Should we process the payment again? No.

Should we reject every retry? Also no.

This is where idempotency becomes an important part of payment-system design.

In distributed systems, retries are inevitable. Network failures, timeouts, message redelivery, and service failures can all cause the same operation to reach our system more than once.

The goal isn't to eliminate retries — it's to make retries safe.

For payment processing, this raises some interesting design questions:

How should an idempotency key identify a payment attempt?

Where should we store the key and result?

How do we handle two identical requests arriving concurrently?

What if the same key is reused with a different payload?

How long should an idempotency record be retained?

How does the approach change in event-driven processing?

I’ve captured the high-level flow, Java example, design considerations, and trade-offs in the infographic below. ↓

Key takeaway:

Resilient systems handle failures. Correct systems handle duplicates.

#Java #SystemDesign #Payments #Idempotency #DistributedSystems #BackendEngineering #Microservices #SoftwareArchitecture #InterviewPreparation

#103 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · **View on LinkedIn**

What really happens when you hit Send on WhatsApp?

It feels instant but behind the scenes, there's a beautifully orchestrated system at work.

Here’s a breakdown of the WhatsApp Message Flow:

Your message is encrypted instantly and sent to WhatsApp’s servers.

The server checks if the recipient is online or offline.

If online → message is delivered and synced across devices.

If offline → message is queued and delivered once they’re back online.

Read receipts are sent back when the user reads your message.

And yes - all this happens within seconds, securely and efficiently.

I visualized the entire architecture in this diagram to simplify how it works.

Whether you're into system design, distributed systems, or just curious about real-time messaging, this is a great example to learn from.

*There is a typo in step 4 it should be online

Want me to break down more real-world systems like this?

LINKS

- https://www.youtube.com/@learnwithrockybhatia
- https://www.linkedin.com/feed/update/urn:li:activity:7453405826364649472

ALSO SAVED AS Rocky Bhatia (5mo) link

#104 Manu Agrawal

System Design

AI & Tech Creator | Ex-Google, Microsoft | AI Engineer | 143K+ LinkedIn | Tech Speaker | 50 Millions+ Impressions | View my portfolio

1mo · **View on LinkedIn**

Give me 2 minutes, and I’ll show you how to use ChatGPT for Google-level system design ₹70 LPA+ roles.

Here’s the framework:

1. Build Your “Interviewer Simulator”

Give ChatGPT:

“Act like a senior system design interviewer. Give me one unfamiliar problem. Don’t suggest components, patterns or technologies. Start with an ambiguous requirement. Ask questions one at a time.”

You’re not memorizing “Design YouTube.”

You’re learning to turn ambiguity into requirements.

2. Run the “Capacity Before Components” Drill

Before drawing anything:

“Give me the business requirements and traffic assumptions. Make me estimate QPS, peak QPS, storage, bandwidth, read/write ratio and latency. Challenge my assumptions.”

No Kafka. No Redis. No sharding.

First prove you understand the scale.

3. Create “Trade-Off Traps”

After proposing your architecture:

“Don’t improve my design. Attack it. Find 5 situations where it fails. Identify the trade-off I ignored.”

Then answer:

Why SQL? NoSQL? Cache? Async? Partitioning? Replication? Consistency?

4. Run the “One Requirement Changed” Attack

Tell ChatGPT:

“Change exactly ONE requirement. Don’t redesign it for me. Make my architecture partially or completely invalid.”

Examples:

10K → 100M users

99% → 99.99% availability
Single region → multi-region
Read-heavy → write-heavy
Seconds → milliseconds latency
Then defend it again.
Same system.
New pressure.
5. Build a “Failure Injection” Interview
Tell ChatGPT:
“Inject one production failure at a time. Don’t give me the solution. Ask what happens and how I recover.”
Attack it with:
Database down.
Cache unavailable.
Queue delayed.
Region disappears.
Network partition.
20× traffic.
Slow dependency.
Retry storm.
6. Use the “Architecture Courtroom”
Tell ChatGPT:
“Cross-examine every major architectural decision. Ask one question at a time. Don’t accept ‘because it scales’ without numbers.”
Defend:
Capacity → Latency → Consistency → Availability → Cost → Recovery → Security
7. Build Your “Design Debt Ledger”
After every mock:
Requirements → Estimates → Architecture → Trade-offs → Failures → Objections → Mistakes
Ask:
“Find my earliest reasoning failure. Classify it and give me 3 system-design problems targeting that weakness.”
Your mistakes decide what you study next.
Not a random list of 50 questions.
#chatgpt #systemdesign #google #jobs
#techjobs #googlejobs

LINKS
- https://x.com/themanuagrawal

#105 Raul Junco

Other

Simplifying System Design
1mo · **View on LinkedIn**

99.9% of backend interviews will ask you about APIs.
Not just how to build them, but how to design them well.
• CRUD vs HTTP methods
• PUT vs PATCH

• Rate-limiting algorithms
• Idempotency in retries
• Caching for performance
• Versioning strategies
• Pagination techniques
• Real-world HTTP status codes
• Security best practices

I wrote a survival kit with the 11 API questions you need to know: clear, concise, and interview-tested.

https://lnkd.in/e7YJkBTv

Save it. Study it. Add more. Share it.

LINKS
• https://lnkd.in/e7YJkBTv

#106 Samson O. Sanyaolu

System Design

Business Systems Manager | Software Engineer | Driving Digital Transformation & Business Growth Through Enterprise Technology & AI

1mo · **View on LinkedIn**

9 Distributed Systems Patterns You Should Know

1. In replication, you make exact copies of your data and store them on different servers.

2. In sharding, you break a large database horizontally, so different rows live on separate machines.

3. In consistent hashing, you distribute data across different machines or servers using a virtual circular ring.

4. PubSub is an asynchronous messaging pattern that decouples service creators from service consumers.

5. In circuit breaker distributed pattern, it stops an application from repeatedly executing an operation that is likely to fail.

6. In Retry with a backoff pattern, you handle temporary network glitches or brief service timeouts. It is a resiliency pattern.

7. In the leader election pattern, you designate a single master node to manage actions and maintain cluster state. It’s what stops two nodes from thinking they’re in charge.

8. In quorum read/write pattern, you ensure enough replicas agree on each read and write so that the set overlaps. It is a data consistency pattern that is used in distributed databases to guarantee up-to-date information.

9. Saga is a design pattern that manages distributed transactions through a sequence of local transactions across multiple microservices. If one step fails, each earlier step is undone by its own compensating action.

✓ Learn programming at w3schools.com and JavaScript Mastery

✓ Follow Samson O. Sanyaolu for more daily insights

✓ And Join this coding community https://lnkd.in/eNmtPNQV

Image Cc: bytebytego

#programming #softwareengineering #WebDevelopment #AI #FrontendDevelopment #sosamsony #BackendDevelopment #FullStackDeveloper #WebAppDevelopment #systemdesign #coding #interviewtips

MEDIA Distributed Systems Patterns

LINKS
• https://lnkd.in/eNmtPNQV
• https://www.linkedin.com/feed/update/urn:li:activity:7500941931372294144
• https://bit.ly/bbg-a-li-profile

- https://lnkd.in/gF7Vsw6X

ALSO SAVED AS Alex Xu (1mo) link

#107 Sajeev KR

Java / Spring

Senior Software Engineer @ IBM Software Labs| 11+ Yrs Java/J2EE | Spring Boot & Microservices | AWS | Full-Stack Developer | AI Enthusiast | Jenkins | Shell & Python | Go | Writing Deep Dive Java & System Design Content

1mo · **View on LinkedIn**

Spring Security, OAuth2 & JWT Interview Questions – Top 25

Answer is shared in this link - https://lnkd.in/gy6edCCK

1. What is Spring Security, and how does it work internally?
2. What is the difference between authentication and authorization?
3. What is the difference between authentication and authorization in Spring Security?
4. How does Spring Security's filter chain work?
5. What is SecurityFilterChain in Spring Security?
6. What is the difference between hasRole() and hasAuthority()?
7. What is the difference between HTTP Basic Authentication and JWT authentication?
8. What is JWT, and how does it work?
9. What are the three parts of a JWT?
10. How does JWT-based authentication work in a Spring Boot application?
11. What is the difference between access tokens and refresh tokens?
12. Where should JWT access tokens and refresh tokens be stored?
13. How does OAuth2 work?
14. What are the different OAuth2 grant types?
15. What is the difference between OAuth2 and JWT?
16. What is the difference between OAuth2 and OpenID Connect?
17. What is an OAuth2 Authorization Server?
18. What is an OAuth2 Resource Server?
19. How does Spring Boot validate a JWT issued by an Authorization Server?
20. What is the difference between JWT authentication and session-based authentication?
21. How do you implement role-based and permission-based authorization in Spring Security?
22. How do you secure REST APIs using Spring Security and OAuth2?
23. How do you handle expired access tokens using refresh tokens?
24. How would you secure a microservices architecture using OAuth2 and JWT?
25. A production API suddenly starts returning 401/403 errors. How would you troubleshoot the Spring Security, OAuth2, or JWT configuration?

Java Spring Boot & Microservices Interview Q&A (200 Qs) - https://lnkd.in/gqhJhmcZ

Full interview kit (Java, Springboot, database, devops, cloud, system design, microservice) - https://lnkd.in/gPi2W587

Spring Security, OAuth2 & JWT Interview Questions – Top 25

Spring Security, OAuth2 & JWT Interview Questions – Top 25

logicbrace.com

MEDIA advertisement

LINKS

- https://lnkd.in/gy6edCCK
- https://lnkd.in/gqhJhmcZ
- https://lnkd.in/gPi2W587

- https://www.logicbrace.com/2026/09/spring-security-oauth2-jwt-interview.html

#108 Sunke Nagarani

Cloud / K8s / DevOps

DevOps Engineer @ NTT Data | immediate joiner | Mercedes-Benz | CKA | AWS | Kubernetes | GitOps | Cloud Migration | DevOps Content Creator | 500K+ Weekly Impressions

1mo · **View on LinkedIn**

25 Kubernetes Components Every DevOps Engineer Should Know

Kubernetes becomes much easier once you understand what each component is responsible for.

Instead of memorizing definitions, think of Kubernetes as a collection of building blocks working together:

Workloads: Pod, Deployment, ReplicaSet, StatefulSet, DaemonSet, Job & CronJob

Networking: Service, ClusterIP, NodePort, LoadBalancer & Ingress

Configuration: ConfigMap & Secret

Storage: Volume, PersistentVolume (PV) & PersistentVolumeClaim (PVC)

Organization: Namespace

Node Components: Kubelet & Kube-Proxy

Control Plane: API Server, Controller Manager & Scheduler

A simple way to remember the flow:

Developer → API Server → Scheduler → Node → Kubelet → Pod → Service → Users

Interview Tip: Knowing definitions isn't enough. Be prepared to explain differences such as Deployment vs StatefulSet, ClusterIP vs NodePort vs LoadBalancer, PV vs PVC, ConfigMap vs Secret, and Kubelet vs Kube-Proxy.

Master these building blocks and Kubernetes architecture starts making much more sense.

Save this Kubernetes cheat sheet for interview preparation and share it with someone learning Kubernetes.

— Nagarani Sunke

#Kubernetes #DevOps #CloudNative #Containers #DevOpsEngineer

#109 Shalini Goyal

System Design

Executive Director, AI & Engineering @ JPMorgan | Amazon Alum | Author · Speaker · Professor | Helping Engineers Break into AI & High-Impact Careers

1mo · **View on LinkedIn**

The System Design Iceberg: What You See vs. What You Miss

Most people think system design is just about picking a tech stack, drawing some boxes, and scaling with load balancers. But that’s just the tip of the iceberg.

This visual cracks open the myths vs. reality of system design - showing what truly lies beneath the surface.

What Everyone Sees (Expectations):

1. Just add more servers or services = better performance

2. Caching is optional

3. Cloud handles everything

4. Downtime is rare and tolerable

5. “It’s just diagrams and arrows”

What You Actually Face (Reality):

1. Every design involves trade-offs (latency, consistency, availability — CAP Theorem)

2. Failure is inevitable — retries, timeouts, and failovers are essential

3. Security must be built-in — not bolted on

4. Legacy systems, versioning, and backward compatibility will test your patience

5. Microservices increase complexity, not reduce it

6. Logs aren’t just for debugging — observability pipelines are critical

7. Scaling demands statelessness + horizontal scale + smart caching + resilience

Great system design isn’t about making things work, it’s about making them fail gracefully.

P.S. Preparing for a software engineering role? Check out my book, System Design and Behavioral Intelligence.

It will help you:

• Build strong System Design foundations and approach design problems with confidence

• Develop practical frameworks to navigate complex behavioral and leadership challenges

Paperback available on Amazon internationally:

US: https://lnkd.in/eVy2ACxQ

UK: https://lnkd.in/eBC_SNdb

India : https://lnkd.in/e25PthqU

LINKS

- https://lnkd.in/eVy2ACxQ
- https://lnkd.in/eBC_SNdb
- https://lnkd.in/e25PthqU

#110 Manu Agrawal

AI / LLM

AI & Tech Creator | Ex-Google, Microsoft | AI Engineer | 143K+ LinkedIn | Tech Speaker | 50 Millions+ Impressions | View my portfolio

1mo · **View on LinkedIn**

Give me 2 minutes, and I’ll show you how to use Claude to crack Google-level interview questions.

Here’s the framework:

1. Build Your “Interview Twin”

Give Claude:

«“Act like a Google interviewer. Give me one unfamiliar problem. Don’t reveal the pattern, data structure or expected complexity. Ask only questions an interviewer would realistically ask.”»

Now you’re not solving another LeetCode problem.

You’re training for the conversation around it.

2. Run the “Wrong Turn” Drill

After you propose an approach, tell Claude:

«“Do not tell me if I’m right. Find the strongest reason my approach could fail. Give me one counterexample.”»

If your approach survives, continue.

If it breaks, fix your reasoning.

You’re training yourself to debug ideas before debugging code.

3. Create “Constraint Ambushes”

Once you solve a problem, ask:

«“Change one constraint that makes my current solution invalid. Don’t tell me what algorithm to use.”»

Examples:

O(n) → O(log n)

105 → 107 inputs

One query → 106 queries

Static data → streaming data

Enough memory → O(1) extra space

Same problem.

Completely different thinking.

4. Use the “Three Solutions, One Survivor” Test

Tell Claude:

«“Generate 3 plausible approaches to this problem. For each, identify the exact constraint, invariant or complexity requirement that could eliminate it. Do not reveal the final answer yet.”»

Now you learn something most interview prep misses:

Why good-looking solutions die.

That is often where the real interview difficulty lives.

5. Turn Claude Into a Code Reviewer From Hell

After coding, give Claude your solution and say:

«“Review this like a senior interviewer. Find the smallest input that breaks my solution. Do not rewrite the code.”»

Then ask:

«“What assumption caused the failure?”»

You’re building a database of your own mistakes.

Not someone else’s.

6. Run the “No-Hint Until Earned” Mode

Tell Claude:

«“I get a maximum of 3 hints. Each hint must be smaller than the previous one. Never reveal the pattern or solution. Track how many hints I use.”»

Now every hint has a cost.

You’ll quickly discover whether you actually know the problem or simply know how to ask AI for the answer.

7. Build Your “Weakness Map”

After every session, give Claude:

Problem → Approach → Mistake → Code → Failed Test → Final Solution

Ask:

«“Identify my earliest incorrect assumption. Classify it as pattern recognition, invariant, data structure, complexity, edge case, proof or implementation. Give me 3 new problems targeting ONLY that weakness.”»

Your next 3 problems should come from your mistakes.

Not from a random LeetCode list.

8. Run the 20 → 5 → 1 Interview Loop

20 problems → Discover weaknesses

5 problems → Attack those weaknesses

1 problem → Full interview simulation

For every problem:

Understand → Hypothesize → Defend → Attack → Code → Test → Explain

Then repeat.

#claude #google #techjobs #jobs #interview #coding #dsa #systemdesign #hiring #job

LINKS

- https://x.com/themanuagrawal

#111 Raul Junco

System Design

Simplifying System Design

1mo · View on LinkedIn

Improving the Cache Hit Ratio is the way to cut latency.

Measure this if you are using Cache.

The Cache Hit Ratio measures how often requests are served from the cache rather than the primary store the primary store. (e.g., database, API, etc.).

Simply put, it tells you how well the cache performs.

The ratio is calculated by dividing the number of cache hits by the total number of cache lookups (hits and misses) and multiplying by 100 to get a percentage.

The formula is:

Cache Hit Ratio (%) = (Cache Hits / Total Lookups) x 100

For context, let’s consider a simple example:

• Your service receives 10 requests.

• The first request queries the cache, finds nothing (cache miss), queries the database, and then stores the result in the cache.

• The following 9 requests hit the cache and get served immediately (cache hits).

In this scenario, you have 9 cache hits out of 10 total lookups, resulting in a 90% cache hit ratio.

While there's no one-size-fits-all number, here’s a general guideline:

• For small to medium-scale services: Aim for a cache hit ratio between 90% and 95%.

• For large-scale systems handling millions of queries per second (QPS): Aim for 99% or higher.

3 ways to Boost Your Cache Hit Ratio

• Pre-load popular cache entries during deployments or cold starts to avoid an initial spike in cache misses.

• Tune your time-to-live (TTL) value. Too short, and items expire quickly, causing misses. Too long, and you risk serving stale data.

• Make sure keys are unique but predictable to prevent missed cache entries.

One final thought:

A high cache hit ratio is great, but not at the expense of serving outdated information.

You need a balance between cache effectiveness and data freshness.

You can't improve what you don't measure.

Do you know how good your Cache is?

#112 Ajeet Singh

AI / LLM

AI & Data Hiring Alerts | AI-Powered Talent Sourcing | IIT Kanpur | Book an appointment

1mo · View on LinkedIn

We're building end-to-end tech teams for top BFSI clients across the UAE, UK, and Switzerland — powering AI infrastructure and global banking applications (including SWIFT payment frameworks).

Open roles:

AI Engineers (BLR/HYD/Pune) ..... Up to ₹60 LPA

Full Stack Engineers (Pan India) ..... Up to ₹80 LPA

Data Engineers/Architects (BLR/HYD) ..... Competitive

Business Analyst (HYD) ..... Up to ₹50 LPA

Comment "Interested" and we'll reach out with details. Apply links are in the first comment below ↓

LINKS
- https://topmate.io/ajeetiitk

#113 MD SHABAB ALI

System Design

Senior Backend Engineer | Go (Golang) | Microservices & Distributed Systems | gRPC, MySQL, Redis | Backend Architecture | Technical Leadership

1mo · **View on LinkedIn**

Microservices don't automatically make your system scalable.

Sometimes, they just make your problems distributed.

When I first started working with microservices, it was easy to think:

Monolith → limited scalability

Microservices → scalable architecture

But in production, it's not that simple.

Breaking an application into 20 services doesn't magically improve scalability.

You can still have:

→ A database that becomes the bottleneck

→ Services making too many synchronous calls

→ APIs overfetching data they don't need

→ Cascading failures when one dependency slows down

→ Network latency that didn't exist in the monolith

→ Distributed transactions that become difficult to reason about

The real advantage of microservices is independent boundaries.

A well-designed system lets you scale the parts that actually need scaling.

For example:

If your Reporting Service gets 10× more traffic than your User Service, you can scale reporting independently.

That's useful.

But if one request travels through:

API → Service A → Service B → Service C → Service D → Database

...you may have created a distributed bottleneck instead of a scalable architecture.

Microservices are an architectural tool — not a scalability button.

Sometimes the most scalable architecture for your requirements is still a well-designed modular monolith.

The important question isn't:

“Should we use microservices?”

It's:

“What problem are we trying to solve by using them?”

#BackendEngineering #Microservices #SystemDesign #DistributedSystems #SoftwareArchitecture

#114 Yogesh Kumbhare

System Design

Software Engineer III @J.P. Morgan Chase & Co. | Java Full Stack Developer | Generative AI & AI Agent | 2xAWS Certified | Scalable Cloud Systems

1mo · **View on LinkedIn**

The API response time suddenly increases from 200ms to 8 seconds in production.

An API that normally responds in 200ms can become an 8 second bottleneck before anyone realizes the real problem is downstream.

In enterprise systems, this kind of latency affects user experience, thread utilization, service capacity, and eventually the stability of dependent applications.

The difficult part is that the API itself may still be healthy while database queries, external services, connection pools, or infrastructure become the actual source of delay.

In one production scenario, an order API suddenly moved from 200ms to several seconds after traffic increased, and tracing showed that a downstream customer service call was waiting for database connections.

A common mistake is to increase server resources or API timeouts without identifying which dependency introduced the latency.

The better approach is to use distributed tracing and latency metrics to break the request into database, network, external service, application processing, and queueing time.

Set explicit timeouts, connection pool limits, circuit breakers, and alerts around each critical dependency so one slow component cannot consume resources across the entire service.

The practical lesson is to treat API latency as an end to end system problem rather than an isolated application metric.

This approach improves scalability and maintainability because performance bottlenecks become visible before they turn into cascading failures.

Measure every critical dependency, trace slow requests in production, and fix the actual bottleneck first; how does your team identify the root cause when a normally fast API suddenly becomes slow?

#Java #APIPerformance #Microservices #SystemDesign #Scalability #PerformanceEngineering #BackendDevelopment

MEDIA advertisement

LINKS

- https://

#115 Anton Martyniuk

System Design

I help developers master .NET, so AI can’t replace them | Founder of AntonDevTips - platform that helps .NET developers reach Senior & Architect level | Software Architect & Microsoft MVP | View my newsletter

1mo · **View on LinkedIn**

I paid $1000 to learn microservices.

This is everything I learned:

1. A message queue does not remove coupling

↳ It moves it from a URL to an event schema.

↳ Add a required field to an event and old consumers break.

↳ Plan event versioning on day one.

2. Be sync at the boundary, async behind it

↳ The user is waiting, so availability checks and payment stay synchronous.

↳ Emails, analytics and stock updates go on a message queue.

3. Idempotency is a database problem

↳ Every broker delivers at least once. Your consumer will see the same message twice.

↳ You don't need a distributed lock. You need a unique index on the idempotency key.

↳ The second insert throws, you catch it, ack the message, move on.

4. Async failures are invisible

↳ A failed HTTP call gives you a 500 in the same second.

↳ A failed event sits in a dead letter queue and waits for someone to care.

↳ Alert on dead letter depth, or your customer reports the bug first.

5. Tracing is not a later task

↳ In a monolith, one stack trace tells the whole story.
↳ Across 5 services, logs without a trace ID are 5 separate mysteries.
↳ Add OpenTelemetry and correlation IDs day 1 when creating a new service.
6. Scaling the app tier can kill the database
↳ 50 instances with 100 pooled connections each is 5,000 connections.
↳ Postgres defaults to 100.
↳ Keep instances x pool size under 80% of max connections, or put a pooler in front.
7. Autoscaling always arrives late
↳ Metrics, decision, provisioning and warmup stack up into minutes.
↳ That is the exact window when your sale happens.
↳ Pre-scale before a known spike and let autoscaling add headroom on top.
8. Duplicate your contracts in tests
↳ If tests reference the API models, a rename compiles everywhere and doesn't break the test.
↳ Copy the request and response records into the test project instead.
↳ Now a breaking contract change fails a test.
And the lesson that cost the most:
Microservices solve an organizational problem before a technical one.
Different teams, different release schedules, different scaling needs.
If that is not your problem yet, a modular monolith gives you the same boundaries, one deployment, and one stack trace when things break.
What was your most expensive problem with microservices?
Download the free microservices roadmap here: https://lnkd.in/dSZMFtVX

LINKS

- https://antondevtips.com?utm_source=linkedin&utm_medium=social&utm_campaign=linkedin-profile
- https://lnkd.in/dSZMFtVX

#116 Alok Sharan System Design

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author

1mo · **View on LinkedIn**

A system that works for 100 users can fall apart at 100,000.
And the architecture decisions you make along the way determine what happens next.
After 20+ years across enterprise architecture, cloud, APIs, and distributed systems, I have learned that system design maturity comes from understanding increasingly complex problems.
A working application is the starting point.
A reliable, scalable, secure, and cost-efficient platform is the real engineering challenge.
Here is how I would break down the 7 levels of system design maturity:
→ Level 1: System Design Fundamentals
Understand client-server architecture, HTTP, APIs, DNS, networking, and the request-response lifecycle.
→ Level 2: Core Building Blocks
Learn how load balancers, databases, caches, message queues, CDNs, and proxies work together.
→ Level 3: Scalability & Data Architecture
Explore replication, sharding, indexing, consistent hashing, stateless services, and scaling strategies.
→ Level 4: Distributed System Design
Understand CAP theorem, consistency models, consensus, leader election, distributed transactions, and idempotency.

→ Level 5: Reliability & Resilience

Design for failures using retries, circuit breakers, redundancy, failover, and disaster recovery.

→ Level 6: Production & Optimization

Master observability, SLOs, capacity planning, autoscaling, performance profiling, and cost optimization.

→ Level 7: Architecture, Governance & Scale

Connect security, compliance, multi-region architecture, platform engineering, data governance, and service ownership.

Here is what experience teaches you:

Every architectural decision creates a trade-off.

Replication improves availability but introduces consistency challenges.

Microservices enable independent scaling but increase operational complexity.

Multi-region deployment improves resilience but adds cost and coordination requirements.

The real skill is understanding these consequences before they become production problems.

This is a high-level maturity stack, but it captures how system design thinking evolves from building applications to architecting enterprise-scale platforms.

P.S. Preparing for a software engineering role? Check out my book, System Design and Behavioral Intelligence.

It will help you:

• Build strong System Design foundations and approach design problems with confidence

• Develop practical frameworks to navigate complex behavioral and leadership challenges

Get Your Copy anywhere internationally:

US: https://lnkd.in/eVy2ACxQ

UK: https://lnkd.in/evzmHKm4

India : https://lnkd.in/e25PthqU

LINKS

- https://lnkd.in/eVy2ACxQ
- https://lnkd.in/evzmHKm4
- https://lnkd.in/e25PthqU

#117 Harshad Bhatt

System Design

Senior AI Engineer @Visa | Nvidia | Ex-Opentext | 39K+ Developer Community | Sharing System Design & Interview Prep | Partnering with AI, DevTools & Premium Brands | DM for Brand Collaborations & Speaking

1mo · **View on LinkedIn**

Most engineers preparing for Visa make the same mistake: they grind 400+ LeetCode problems and assume that’s the entire game.

I used to think the same. But once you sit on the other side of the table, you realize where candidates actually get filtered out.

Everyone in the loop can solve a binary search or two-pointer problem. Clearing the bar doesn't come down to optimal time complexity alone, it comes down to engineering maturity.

Here is what actually moves the needle when interviewing for Visa:

1. Don’t just write code, defend the runtime internals

In most rounds, writing a working solution is just step one. The real evaluation starts right after:

- How does your chosen collection handle rehashing under heavy loads?

- How does memory allocation work behind the scenes?

- What happens to concurrency, thread safety, and race conditions?

At financial scale, a solution that works on paper but leaks memory or locks threads in production is useless.

2. Think in failure modes, not just happy paths

When designing systems or discussing your past architecture, standard textbook answers fall flat. Visa runs high-throughput, mission-critical infrastructure where 99.999% availability is a reality, not a slide deck bullet point.

- What happens when your downstream message queue lags?

- How do you guarantee idempotency across duplicate payment requests?

- How does your service recover during a cold restart?

Mentioning failure scenarios before the interviewer asks is the quickest signal that you've built real systems.

3. The "Why" behind every line on your resume

Expect deep dives into your previous production work. If you mention Kafka, Redis, or Docker, you need to defend why you chose them over the alternatives. Surface-level buzzwords get caught within two minutes of probing questions.

The candidates who stand out aren't the ones racing to finish coding in 10 minutes. They are the ones who treat the interview like an engineering discussion between two colleagues solving a production outage.

Master the fundamentals, know your stack inside out, and design for scale and reliability.

If you’re currently preparing for upcoming backend/systems engineering rounds, drop your biggest roadblock in the comments, happy to share perspective.

#SoftwareEngineering #SystemDesign #TechInterviews #BackendEngineering #CareerGrowth

#118 Muhammad Usman Arif

System Design

Associate Software Engineer @ InvoByte | ASP.NET Core | Clean Architecture | CQRS | MediatR | REST APIs

1mo · **View on LinkedIn**

Software Architecture vs System Architecture, What's the Real Difference?

These two terms are often used interchangeably, but they operate at completely different scopes.

Software Architecture

This focuses on the internal structure of a single application, how its components are organized and interact with each other. It typically follows a layered approach:

Presentation Layer (UI / Web / API)

Application Layer (Business Logic)

Domain Layer (Entities / Rules)

Infrastructure Layer (Database / External Services)

It's guided by principles like SOLID, design patterns, and architectural styles such as Layered, Clean, or Hexagonal Architecture. In short, it's about how one application is structured internally.

System Architecture

This zooms out to the bigger picture, how multiple systems, services, and infrastructure components work together as a whole. It covers things like:

Clients (Web, Mobile, Users)

Load Balancers distributing traffic

Multiple microservices (User, Order, Payment, Analytics, etc.)

Shared infrastructure (Database, Message Broker, Cloud Servers, File Storage)

It's about how the entire ecosystem is designed to communicate, scale, and stay reliable.

The Core Difference:

Software Architecture = structure within one application

System Architecture = structure across multiple applications and services

Understanding both is essential, a well-designed application still needs a well-designed system around it to truly scale.

#SoftwareEngineering #SystemDesign #SoftwareArchitecture #DotNet #BackendDevelopment #TechCommunity #CleanArchitecture

MEDIA advertisement

#119 Pitchayut Roejanapreecha

System Design

Delivery Excellence Director | Driving Faster Delivery, Higher Quality & Enterprise Agility

1mo · View on LinkedIn

Enterprise Architect vs Solution Architect vs System Architect vs Software Architect

They are not interchangeable roles.

They operate at different levels of abstraction, answer different questions, and own different architectural decisions.

The easiest way to see the distinction is to move from enterprise strategy → solution design → system structure → software implementation.

Enterprise Architect

Focus: enterprise-wide direction

Core question: Are we building the right enterprise landscape?

Typical ownership: capability maps, target architecture, principles, standards, technology roadmap, portfolio alignment, governance.

Solution Architect

Focus: end-to-end solution for a business problem

Core question: What is the right solution for this need?

Typical ownership: solution blueprint, integration design, application/data/security views, architecture decisions, NFR alignment.

System Architect

Focus: structure and behavior of a specific system or platform

Core question: How should this system be structured to work reliably at scale?

Typical ownership: subsystems, interfaces, deployment architecture, scalability, resilience, performance, technical consistency.

Software Architect

Focus: internal software and code-level design

Core question: How should we build this software well?

Typical ownership: components, APIs, coding patterns, modularity, quality attributes, technical standards, developer guidance.

A useful rule of thumb:

Enterprise Architect = enterprise direction

Solution Architect = business solution design

System Architect = platform/system structure

Software Architect = application/code architecture

The scope gets narrower as you move toward implementation, while the level of technical detail increases.

But these roles should not work in silos.

Strong architecture requires traceability across all four levels:

Business strategy → Capability → Solution → System → Software

When that chain breaks, common symptoms appear:

• duplicate platforms

• inconsistent technology choices

• integration complexity

• architecture debt

• local optimization

• higher delivery cost

• slower change

The goal is therefore not to argue about titles.

It is to make sure the right architectural decisions are made at the right level, by the right role, with clear governance and traceability.

#EnterpriseArchitecture #SolutionArchitecture #SystemArchitecture #SoftwareArchitecture #TOGAF #TechnologyStrategy #Architecture #DigitalTransformation #ITStrategy #EnterpriseIT

MEDIA advertisement

#120 Mansi Kaushik

Java / Spring

Java Technical Lead | 10+ Years Experience | Java, Spring Boot & Microservices | CI/CD | Banking & Financial Services | Scaling Systems for 1M+ Daily Transactions

1mo · View on LinkedIn

After working on microservices for several years, one thing I've realized is that writing the Java code is only one part of the job.

When I started working more deeply with Java, Spring Boot, Kafka, AWS and Kubernetes, I gradually started looking beyond just the API and business logic.

I started asking questions like:

→ What should this service actually own?

→ How will it handle failures and retries?

→ What happens when another service is down?

→ Do we really need synchronous communication here, or would an event work better?

→ How will we secure, monitor and troubleshoot it in production?

→ Will the database design and APIs still work when traffic increases?

→ How do we deploy it safely and roll back if something goes wrong?

These are the things that make a microservice production-ready, not just a working application.

I put together this infographic based on the way I approach building a microservice — from understanding the requirement and defining boundaries to development, database, Kafka, security, observability, testing, system design and deployment.

It's also a checklist I find useful while preparing for Java/Spring Boot and Technical Lead interviews.

Still learning and refining these patterns with every project. And honestly, that's one of the things I enjoy most about backend engineering — there's always another "what if?" to think about.

What's one thing you always consider before calling a microservice production-ready?

#Java #SpringBoot #Microservices #SystemDesign #Kafka #AWS #Kubernetes #SoftwareArchitecture #BackendDevelopment #TechnicalLead

#121 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · View on LinkedIn

Choosing the right database isn't about the tool,

it’s about the type of data you're storing.

Most teams struggle with databases because they start with the technology first.

The correct approach? Start with your data model, then map it to the right engine.

This graphic breaks it down into 3 simple paths:

1. Structured Data → Relational or Columnar DBs

Best when your data is clean, tabular, and requires transactions.

Think: finance apps, inventory systems, billing, CRM.

Examples:

RDS, Aurora, SQL Server, PostgreSQL, BigQuery, Redshift, Snowflake.

2. Semi-Structured Data → Key-Value, Wide-Column, Graph, or Ledger

Perfect for flexible schemas, large-scale lookups, caching, relationships, or immutability.

Examples:

DynamoDB, BigTable, Cassandra, Redis, CosmosDB, Neo4j, QLDB, Hyperledger.

3. Unstructured Data → Blob Storage & Search Engines

When dealing with files, media, logs, documents, or text-heavy content.

Examples:

S3, Blob Storage, HDFS, ElasticSearch, OpenSearch, Solr.

Don’t pick a database because it’s “popular.”

Pick the one that matches your data type, performance needs, and access patterns.

MEDIA document/carousel (possible)

LINKS

- https://www.youtube.com/@learnwithrockybhatia

#122 Aiswarya Venkitesh

AI / LLM

Principal Cloud Solution AI Architect @Microsoft | AI, Data and Tech Content Creator | Global Speaker | Worldwide Ranked 4th in the World’s Top Female Tech Creators | ★ Top AI Voice | Opinions are my own!

1mo · **View on LinkedIn**

The software lifecycle is learning to think.

For years, development followed a familiar sequence: requirements, design, coding, testing, review, deployment and production feedback.

Agentic SDLC turns that sequence into a faster, continuous loop.

Humans define the intent, outcomes and boundaries. AI agents support every stage:

→ Goals replace lengthy requirement handoffs.

→ AI-assisted PRDs create structured solution plans.

→ Agent architecture connects roles, tools, APIs and workflows.

→ Autonomous implementation generates and refactors code.

→ Human review protects quality and critical decisions.

→ Continuous testing runs evaluations and security checks.

→ Agent-driven deployment executes CI/CD changes.

→ Production monitoring detects drift and learns from feedback.

Value moves from manually completing every task to designing a system that completes those tasks safely. Engineers define goals, permissions, guardrails, evaluation criteria and escalation paths.

This approach can compress cycles from weeks to days, but speed needs control.

An Agentic SDLC combines autonomous execution with human judgment, continuous evaluation, observability and governance.

We are moving from managing development tasks to designing software systems that can execute, learn and improve.

How much of your SDLC is agent-assisted?

Become better at AI in just 1 minute a day. Join my weekly newsletter where I document the real-world journey of AI transformation.

→ Sign up free now → https://avsl.beehiiv.com/

Follow Aiswarya Venkitesh for more such insights!!

#AgenticAI #SDLC #AIAgents #AIEngineering

LINKS

- https://www.instagram.com/aiswaryavenkitesh?utm_source=ig_web_button_share_sheet&igsh=ZDNlZDc0MzIxNw==
- https://avsl.beehiiv.com/

#123 Eslam Aly

System Design

Senior Software Architect & Technical Product Manager – Expert in Micro-Services, Domain-Driven Design & Event-Driven ASP.NET Core Solutions (Kafka / RabbitMQ), CI/CD & Cloud (Docker/Kubernetes)

1mo · **View on LinkedIn**

I have created This visual outlines how modern microservice systems work: from client requests through Nginx & Ocelot API Gateway, to secure authentication/authorization, event-driven communication via RabbitMQ/Kafka, and distributed data consistency using CQRS, read models, and multi-cluster failover.

I'm currently writing a comprehensive book on Domain-Driven Design (DDD) and Microservices, where I dive deep into these patterns — from strategic design to implementation best practices in .NET Core.

This diagram is just one piece of the puzzle: how we break down complex systems into bounded contexts, ensure loose coupling, and build scalable, maintainable architectures.

Stay tuned for more insights! If you're working with microservices or DDD, I'd love to hear your thoughts — what challenges are you facing in your architecture journey?

MEDIA advertisement

#124 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1mo · **View on LinkedIn**

If I had to start with system design,

I'd learn these concepts (save now):

1 Database

2 SQL vs NoSQL

3 Availability

4 Reliability

5 Latency

6 API Design

7 REST

8 Scalability

9 Load Balancing

10 Caching

11 Fault Tolerance

12 High Availability

13 CAP Theorem
14 GraphQL
15 gRPC
16 Sharding
17 Indexing
18 Leader Election
19 Consensus
20 Consistency Models
21 Event-Driven
22 Message Queue
23 Pub/Sub
24 Denormalization
25 Secrets Management
26 Bulkhead
27 Retry Logic
28 Timeout
29 Feature Flags
30 ACID
31 Blue-Green Deployment
32 Canary Release
33 Erasure Coding
34 Service Discovery
35 Sync vs Async
36 Correlation ID
37 BFF
38 Connection Pooling
39 Monitoring
40 Alerting
41 Query Optimization
42 WebSockets
43 Cache Stampede
44 MapReduce
45 WebRTC
46 Deserialization
47 Strangler Pattern
48 LSM Trees
49 Batch Processing
50 Stream Processing
(... and much more.)
What else should make this list?
===
PS - Want a detailed breakdown of each concept?
Read right now:
→ Part 1: https://lnkd.in/eK87NkE5
→ Part 2: https://lnkd.in/dicBp9N4
→ Part 3: https://lnkd.in/dDKk3-Xw

===

Repost to help others learn system design.

Follow Neo Kim + turn on notifications.

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eK87NkE5
- https://lnkd.in/dicBp9N4
- https://lnkd.in/dDKk3-Xw

#125 Palash Bharadwaj

AI / LLM

VP Technology | AI, Data, BI & Cloud Transformation | $100M+ P&L | Healthcare IoT | Remote Patient Monitoring | Ex-Sun Microsystems

1mo · **View on LinkedIn**

Most teams don’t need more agents.

They need the right agentic design pattern for the problem.

A simple retrieval request should not become a multi-agent system.

A complex enterprise workflow should not depend on one oversized agent doing everything.

The real architecture decision is:

How should the AI reason, act, coordinate, review, and recover?

This infographic maps nine practical patterns:

Tool Use — direct execution for focused tasks

ReAct — iterative thinking, acting, and observing

Reflection — self-review before finalizing

Planning — breaking goals into executable steps

Orchestrator — coordinating specialized agents

Sequential Chain — structured stage-by-stage processing

Parallel Fan-Out/Fan-In — concurrent work with aggregation

Hierarchical — manager and specialist agent layers

P2P Mesh — decentralized agent collaboration

The important insight:

These patterns are not mutually exclusive.

A production-grade agentic system may use:

Planning → Orchestration → Parallel Execution → Reflection → Human Approval

The best architecture is rarely the one with the most agents.

It is the one that delivers the required outcome with the least unnecessary autonomy and complexity.

Before selecting a pattern, ask:

What requires reasoning? What can run in parallel? What must be verified? Where should a human remain in control?

Which pattern are you using most today—and which one has been the hardest to operationalize?

#AgenticAI #ArtificialIntelligence #AIAgents #MultiAgentSystems #EnterpriseAI #AIArchitecture #GenerativeAI #ResponsibleAI #SoftwareArchitecture #DigitalTransformation #TechLeadership #Innovation #PalashBharadwaj

#126 Harun Sulaiman

AI / LLM

Software Engineering | AI Engineering | Machine Learning | AI Research

1mo · **View on LinkedIn**

If I had 6 months to become an AI Infrastructure Engineer.

I'd do this.
Stage 1: Linux, Networking, and Python Foundations
Linux, bash, SSH, TCP/HTTP, DNS, Python, async programming, systemd.
Stage 2: Containers and Orchestration
Docker, Kubernetes, Helm, GPU scheduling, resource limits, multi-node clusters.
Stage 3: GPU Fundamentals
CUDA basics, nvidia-smi, VRAM math, NVLink and InfiniBand, MIG partitioning.
Stage 4: Model Serving
vLLM, SGLang, TGI, continuous batching, KV cache management, quantization (AWQ, FP8).
Stage 5: Inference Optimization
Speculative decoding, prefix caching, chunked prefill, TensorRT-LLM, Triton kernels.
Stage 6: Model Distribution and Storage
Weight sharding, safetensors, model registries, CDN for weights, lazy loading.
Stage 7: GPU Autoscaling
KEDA, queue-based scaling, cold start mitigation, spot instances, multi-region failover.
Stage 8: Training Infrastructure
Ray and Slurm, FSDP and tensor parallelism, checkpointing, fault tolerance, gradient accumulation.
Stage 9: AI Gateway Layer
Routing, load balancing, fallback chains, rate limiting, per-tenant token budgets.
Stage 10: Observability and FinOps
GPU utilization (DCGM), TTFT and ITL metrics, cost per token, tenant dashboards, alerting.
Stage 11: Security and Multi-Tenancy
Tenant isolation, secrets management, sandboxed execution, audit logs, compliance controls.
Stage 12: Portfolio and Public Benchmarks
Ship a self-hosted inference cluster, publish latency and cost benchmarks, write architecture teardowns.
Most people stay stuck watching tutorials.
Builders get hired.
(Bookmark it)

#127 RAMESHKUMAR S

Kafka / Messaging

Data & AI Engineer| Certified in Databricks, Fabrics, Cloudera, Claude AI, AWS and Azure AI Engineer, Dremio|ML | RAG | LLM AI Workflow | Agent automation | USA

1mo · View on LinkedIn

Asynchronous Alert Processing: Incoming notification requests are instantly buffered into message queues so system traffic spikes never crash the core application.
Smart Multi-Channel Delivery: Specialized worker nodes pull messages from the queue to route them concurrently across iOS/Android Push Services, SMS Gateways, and Email Providers.
Fault-Tolerant Reliability: Built-in deduplication blocks duplicate alerts, while exponential backoff retries and Dead Letter Queues catch and save failed deliveries.
SoftwareEngineering SystemDesign Database MongoDB Elasticsearch Backend Architecture Vinayak Pai Ramesh Vanteru Vimalan Sadhasivam

MEDIA advertisement

#128 KHYATI PATEL

Java / Spring

Senior Software Engineer | CKA | Azure Solutions Architect Expert | AWS Certified | GCP Certified | Java | Spring Boot | Kubernetes | Cloud-Native Architecture | AI Enthusiast

1mo · View on LinkedIn

Java Multithreading — Day 10: volatile vs synchronized

Both volatile and synchronized are used in Java multithreading.

But they solve different problems.

The easiest way to remember:

volatile = SEE the latest value

synchronized = PROTECT the operation

Let’s understand with two simple examples.

Example 1: Stop a Worker

Suppose one thread is doing some work:

```
private volatile boolean running = true;
while (running) {
// keep working
}
```

Another thread says:

```
running = false;
```

What do we need?

We need the worker thread to see the updated value.

Thread A Thread B

running = true running = false

↓ ↓

working updates flag

↓

sees false

↓

STOP

This is a good use case for:

volatile

Because we’re mainly concerned with visibility.

Example 2: Withdraw Money

Now consider:

```
balance = balance - amount;
```

This operation involves:

READ balance

↓

CALCULATE

↓

WRITE balance

Two threads could interfere with each other.

So simply writing:

```
volatile int balance;
```

doesn’t solve the whole problem.

Instead, we could protect the operation:

```
public synchronized void withdraw(int amount) {
```

balance = balance - amount;
}
Now:
Thread A → ENTER
Thread B → WAIT
Thread A → UPDATE
Thread A → LEAVE
Thread B → ENTER
This is where:
synchronized
makes sense.

Simple analogy
Imagine a whiteboard in an office.
volatile
Everyone needs to see the latest message on the board.
Think:
“Make the update visible.”
synchronized
Only one person is allowed to edit an important calculation at a time.
Think:
“Protect this operation.”

(!) Important difference
This:
volatile int count = 0;
count++;
is still not thread-safe.
Because count++ is not one indivisible operation.
But:
synchronized void increment() {
count++;
}
protects the complete read-modify-write operation from concurrent execution using the same monitor.
Easy rule to remember
Need threads to observe a simple state change?
→ Consider volatile
Example:
volatile boolean shutdown;
Need to protect a multi-step update to shared mutable state?
→ Consider synchronized
Example:
synchronized void withdraw() {
// read + modify + write
}
So remember:

volatile → Visibility

synchronized → Mutual exclusion + Visibility

They aren’t competitors.

They’re different tools for different concurrency problems.

#Java #Multithreading #Concurrency #Volatile #Synchronization #JavaDeveloper #BackendDevelopment #SoftwareEngineering #JavaInterview

#129 Alex Xu

AI / LLM

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

1mo · View on LinkedIn

Agent Harness, Explained Visually

An LLM can reason and generate text, but that alone can be unreliable. The reliability usually comes from the harness, the layer around the LLM shown in the diagram below.

1. The user sends a query into the harness and the context builder converts the query into a prompt using its knowledge base.

2. The LLM reasons over the provided prompt and proposes some actions (functions to call)

3. The policy gate checks actions and determines whether they should be allowed or blocked.

4. The runtime first executes the tools and sends the results back to the LLM.

5. Finally, the verify step runs tests on the result and includes the feedback in the context for future iterations.

6. Once the verify step passes, the harness shows the result to the user.

Aside from these, two other layers exist and are shared between all stages. The observability layer tracks things like metrics, and errors. The constraints layer sets the rules and permissions so the system can enforce them.

Over to you: Where do you think most agent failures happen, in context, tools, constraints, or verification?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gF7Vsw6X

#systemdesign #coding #interviewtips

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gF7Vsw6X

#130 Akash Namdev

Kafka / Messaging

Senior Software Developer @ Citi | Backend & Distributed Systems | Java | Kafka | MongoDB | Payment Processing | Performance Engineering | 11+ Years Experience

1mo · View on LinkedIn

Deep Diving into Apache Kafka — Part 2

Continuing my Kafka learning journey with Producer Idempotence.

Kafka Idempotence helps prevent duplicate messages during producer retries by using Producer ID (PID) + Sequence Numbers.

Prevents duplicate records

Maintains ordering

Improves reliability

Important for high-throughput systems

#ApacheKafka #Kafka #DistributedSystems #Microservices #Java #SystemDesign

#131 Dhivya Sri Lingala

AI / LLM

Software Development Engineer at Amazon

1mo · View on LinkedIn

Recently, I had the opportunity to interview for a Software Engineer role at JPMorganChase , focused on AI and enterprise payment technology.

This was one of the most in-depth technical interviews I've experienced so far.

What stood out to me wasn't just the number of questions, but the depth of the questions.

The discussion went deeply into my projects and experiences. Instead of simply asking "What technology did you use?", the conversation often went further:

Why did you choose it?

Why not another approach?

How exactly did you implement it?

What happens if it fails?

How would you scale it?

How would you secure it?

How would you test it?

What would you change in a production environment?

We discussed areas including:

AI/ML applications in enterprise systems

Backend engineering & REST APIs

Java / Spring

Authentication & security

AWS & cloud deployment

Docker & CI/CD

Microservices & scalability

Load balancing, health checks & high availability

Automated testing and deployment

OCR pipelines & EasyOCR

Messaging and enterprise systems

One thing I really appreciated was that the conversation wasn't focused only on knowing concepts.

It was about understanding the engineering decisions behind them.

A project isn't just:

"I used X technology."

It's also:

Why X? Why not Y? What trade-offs did you consider? What happens in failure scenarios? How do you validate it? How does it behave at scale?

That made me reflect on how important it is to truly understand the projects we put on our resumes, not just the technologies listed on them.

The interview also reinforced something I've been learning throughout my journey:

Building AI solutions is only one part of the problem. Building AI solutions that are reliable, secure, testable, scalable, and useful in real-world systems is where software engineering really matters.

And one thing I'm especially grateful for is that building my projects from scratch gave me the confidence to answer many of these questions.

When you build something yourself, from defining the problem and choosing the technology to implementing, debugging, deploying, and improving it, you develop an understanding that goes beyond simply learning a technology.

That hands-on experience helped me reason through questions even when I didn’t know the exact answer.

Build from scratch. Understand what you build. Learn from every challenge.

Every challenging interview teaches me something new, and this one definitely did.

#JPMorgan #InterviewExperience #SoftwareEngineer #SoftwareEngineering #AIEngineer #ArtificialIntelligence #MachineLearning #BackendEngineering #AWS #CloudComputing #CareerGrowth #TechCareers

#132 Nikki Siapno Cloud / K8s / DevOps

Eng Manager & Founder | Ex-Canva | 450k+ Community | Helping Senior Engineers Master System Design and AI Engineering | View my newsletter

3mo · **View on LinkedIn**

8 Deployment Patterns Developers Should Know

Deployment patterns all ship code, but what actually happens in each strategy is very different.

The goal isn't just to ship code. It's to control risk and reduce the impact of failures when they inevitably happen. Each pattern optimizes for different risks and tradeoffs.

1) Rolling

↳ Gradually replaces old instances with new ones. Useful for reducing downtime, but old and new versions may run at the same time.

2) Blue-Green

↳ Keeps one live version and one idle candidate version. Once the new version is validated, traffic switches from blue to green.

3) Canary

↳ Releases the new version to a small percentage of users first. If metrics look healthy, the rollout expands.

4) Feature Flag

↳ Deploys code while keeping the feature hidden or disabled. This separates deployment from user exposure.

5) Progressive

↳ Combines gradual rollout, metrics, feature flags, and automated decisions to increase exposure only when the release looks safe.

6) Shadow

↳ Sends real production requests to the current version, while copying those requests to the new version for observation.

7) A/B

↳ Sends different user groups to different versions so teams can compare behavior, performance, or product outcomes.

8) Immutable

↳ Deploys the new version to fresh infrastructure instead of modifying existing instances in place.

Each pattern moves risk to a different place. There's no single best one. The choice depends on what you are trying to protect: availability, user experience, data compatibility, rollback speed, or operational simplicity.

But how do these patterns actually work under the hood? To see how, subscribe to receive our Deployment Patterns Deep Dive when it's released in a few weeks → https://lnkd.in/gguD5-54

What else would you add?

——

PS: Want to improve at system design? Download my free System Design Handbook and join 34,000+ engineers who get my free weekly newsletter → https://lnkd.in/gueNmru7

——

Repost to help others learn deployment patterns.

+ Follow me (Nikki Siapno) + turn on notfications.

LINKS

- https://lucode.co/luc-system-design-newsletter-pl1
- https://lnkd.in/gguD5-54
- https://lnkd.in/gueNmru7
- https://www.linkedin.com/feed/update/urn:li:activity:7505152354267127808
- https://lnkd.in/gNbucjiU

ALSO SAVED AS Nikki Siapno (1mo) link

#133 Durgesh Choubey

AI / LLM

AI @ Accenture | Data Analysis & GenAI Practitioner| Building AI Systems That Actually Work | BHU Gold Medalist | Claude Certified Architect | Gemini Enterprise CPS | OpenAI Certified Technical Practitioner

1mo · **View on LinkedIn**

Want to become a Claude Certified Architect?

If you’re learning Generative AI, LLMs, Claude, MCP, APIs, or AI Agents, there’s a much better approach than jumping straight into certification prep.

Build the skills first. Then prepare for the exam.

6-step Claude Architect roadmap

1. Start with Anthropic Academy

Build your foundation with courses covering:

• AI Fluency
• Claude 101
• Claude Code
• Claude API
• MCP
• AI Agents

Free resources: https://lnkd.in/dVWyBUrB

2. Learn Claude Code by building

Don’t just watch tutorials.

Use Claude Code to build real projects, experiment with agentic workflows, work with repositories, and understand how AI-assisted development is configured and verified.

3. Get comfortable with Claude API and MCP

Two areas worth going deep on:

Claude API: prompting, tool use, structured outputs, RAG, agents, evaluations, and production patterns.

MCP: how AI applications connect with external tools, APIs, databases, and other sources of context.

Docs:

Claude API: https://lnkd.in/dhYDcfG5

MCP: https://lnkd.in/dbXqST3J

4. Start thinking like an architect

Certification prep should go beyond knowing Claude features.

Learn how to reason about:

• Agentic architectures
• Tool design and MCP integrations
• Context management
• Prompt engineering

• Structured outputs
• Reliability and evaluation
• Cost optimization
• Safety and responsible AI

The goal is to understand why you would choose one architecture over another.

5. Build 3–5 real projects

For example:

• Customer-support agent
• RAG knowledge assistant
• MCP-powered business agent
• Data-analysis agent
• Multi-step automation workflow

Projects force you to deal with the parts tutorials often skip: failures, permissions, context limits, tool errors, evaluation, and cost.

6. Then prepare for the certification

The Claude Certified Architect – Foundations exam is currently listed as:

• 60 questions
• 120 minutes
• $125 USD
• Online proctored or Pearson test center
• 12-month certification validity

The listed exam domains include:

• Agentic Architecture & Orchestration
• Tool Design & MCP Integration
• Claude Code Configuration & Workflows
• Prompt Engineering & Structured Output
• Context Management & Reliability

One important detail: Anthropic currently lists the certification as available to people at organizations in the Claude Partner Network, so check your eligibility before planning around the exam.

Certification can validate your knowledge.

But the more valuable skill is being able to design, build, integrate, evaluate, secure, and deploy AI-powered systems.

That’s the skill I’d optimize for.

Save this roadmap if you’re learning Claude.

#claude #anthropic #generativeai #mcp #claudecode #llm #aiagents #aiarchitecture :::

LINKS

- https://lnkd.in/dVWyBUrB
- https://lnkd.in/dhYDcfG5
- https://lnkd.in/dbXqST3J

#134 Ratnakar Sahoo

AI / LLM

Information Technology Analyst at Tata Consultancy Services

1mo · **View on LinkedIn**

Most people are still using Classic RAG for AI applications.

Meanwhile, top engineering teams are quietly moving to Graph RAG and Agentic RAG.

If your AI system is missing context, hallucinating, or failing at complex tasks, you’re likely using the wrong retrieval architecture.

Here is the exact visual breakdown of how RAG is evolving:

Classic RAG (The Retriever)

* How it works: Query \rightarrow Embedding \rightarrow Vector Database \rightarrow Top Chunks \rightarrow LLM.

* Best for: Fast, simple, single-hop factual lookups.

* Limitation: Fails when answers require connecting dots across multiple documents.

Graph RAG (The Connector)

* How it works: Query \rightarrow Entity Extraction \rightarrow Knowledge Graph \rightarrow Connected Context \rightarrow LLM.

* Best for: Relational data, entity-rich queries, and multi-source context.

* Advantage: Maps relationships ("A creates B", "X relates to Y") so the LLM actually understands the big picture.

Agentic RAG (The Reasoner)

* How it works: Query \rightarrow Reasoning Agent \rightarrow Multi-Tool Retrieval (Vector DB + Knowledge Graph + Web Search) \rightarrow Self-Evaluation Loop \rightarrow Final Answer.

* Best for: Complex, multi-step problem solving and dynamic workflows.

* Advantage: Adaptive, tool-aware, and self-correcting before delivering the final output.

The Key Takeaway:

* Classic retrieves facts.

* Graph connects concepts.

* Agentic reasons and solves problems.

Which RAG architecture are you currently building with in 2026? Let’s discuss in the comments! ↓

#AI #MachineLearning #GenerativeAI #RAG #SystemDesign #SoftwareEngineering #AgenticAI

MEDIA document/carousel (possible)

#135 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

Other

Manish Rana magar • 3rd+

1mo · **View on LinkedIn**

Learning faster is about using the right mental leverage.

Learn AI in 3 Minutes a Day → https://lnkd.in/duwhtNtW

Learning better, not learning more.

You should Save and Share it.

Vel accusamus quidem sequi.

#AIforLearning

#LearningHacks

#SkillMastery

#Productivity

#LearningFlywheel

LINKS

- https://lnkd.in/duwhtNtW

#136 Clouding Academy

AI / LLM

1mo · **View on LinkedIn**

THE MOST RELEVANT AI CERTIFICATIONS IN 2026

• AWS GenAI Developer Professional (AIP-C01) · US$ 300 · new in 2026

• Google Professional ML Engineer (PMLE) · US$ 200
• Claude Certified Architect Pro (CCAP) · US$ 175
• Claude Certified Architect (CCAA) · US$ 125
• AWS ML Engineer Associate · MLA-C02 is in beta for US$ 75
• Azure AI Apps and Agents Developer (AI-103) · US$ 165 · new, replaced AI-102
• Claude Certified Developer (CCDF) · US$ 125
• Claude Certified Associate (CCAF) · US$ 99
• AWS AI Practitioner (AIF-C01) · US$ 50 with the AIF2CLOUD coupon until 09/30
• AWS AI Business Strategist (AIB-C01) · US$ 50 in beta
• Azure AI Fundamentals (AI-900 / AI-901) · US$ 99
• Google Cloud GenAI Leader · US$ 99
• NVIDIA Certified Associate GenAI · ~US$ 135
• Databricks GenAI Fundamentals · free

Number 1 on the list is AWS's most advanced AI certification.

• AIP-C01 left beta in March and became AWS's first Professional-level GenAI certification. The exam covers the complete pipeline of taking FMs and agents into production, with long-form scenarios similar to the Professional-level exams. For anyone working with GenAI on AWS, this is the top of the line.

CCAP is the newest bet on the list.

• It's the highest level in Anthropic's certification lineup, focused on applying solutions in production using Claude. Anthropic is the undisputed leader when it comes to AI innovation from all perspectives, so their most difficult certification is consequently one of the most important in the world. The program has existed for just over a year, and there are still very few people in Brazil with this certification.

AIB established a new way of evaluating candidates.

• The other exams mainly measure technical knowledge. AIB-C01 evaluates something different: the ability to apply AI to business, choose use cases, evaluate ROI, and decide where AI makes sense and where it doesn't. It's AWS's first certification focused on this side of the table, and the beta costs US$ 50.

If you're studying for AWS, Azure or Claude certifications, I invite you to check out Clouding Academy.

The platform offers practice exams with scaled scoring, following the same model as official AWS exams, as well as hands-on labs, memorization tools, and architecture scenarios designed to accelerate learning.

Platform: https://clouding.academy/

LINKS

- https://clouding.academy/

#137 **Anton Martyniuk** System Design

I help developers master .NET, so AI can't replace them | Founder of AntonDevTips - platform that helps .NET developers reach Senior & Architect level | Software Architect & Microsoft MVP | View my newsletter

1mo · **View on LinkedIn**

You learn Design Patterns the wrong way

Here is a correct approach (27,000 devs learned from me)

You don't know the order in which to use design patterns based on your experience.

And please don't trust AI because it can't replace the judgment of someone who has mastered them.

Not all patterns have the same value at every stage of your career.

There are even some design patterns that are easy on paper, but in reality they require the skills of a mid- or senior-level developer to get them right.

I recommend learning design patterns in the following way ↓

Start with Core Patterns

They are your foundation at the beginner level. You will recognize and use them in almost every project.

1. Builder
2. Factory Method
3. Abstract Factory
4. Strategy
5. Adapter
6. Decorator
7. Facade

→ You use them to create and organize objects to keep your architecture modular and clean.

If you're a junior developer, mastering these patterns will immediately improve your design thinking.

If you are a middle developer, it's time for Intermediate Patterns

Once you gain more experience, you will be assigned to more complex tasks and to solve them, you need better tools:

1. Chain of Responsibility
2. State
3. Proxy
4. Template Method
5. Bridge
6. Command

→ With these patterns, you will learn how to manage behavior flow, abstract implementations, and manage different responsibilities.

You will use these for building scalable and extensible systems.

If you are a senior or your architecture gets complex, you will need Advanced Patterns:

1. Singleton
2. Mediator
3. Flyweight
4. Interpreter
5. Composite
6. Visitor
7. Prototype

→ These patterns will help you manage object hierarchies, optimize performance, and coordinate multiple class hierarchies.

If you are a middle or a senior developer, remember this golden rule:

Don't use patterns blindly.

Use patterns ONLY when they simplify your design or solve a problem.

And never use because your code will feel "more cool" or "more Enterpisy".

Overusage of design patterns and SOLID principles leads to overcomplicated code and multiple layers of unnecessary abstraction.

Do you agree with my priority list? Leave a comment down below ↓

+ Follow me (Anton Martyniuk) to improve your .NET and Architecture Skills

LINKS

- https://antondevtips.com?utm_source=linkedin&utm_medium=social&utm_campaign=linkedin-profile

#138 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · View on LinkedIn

Your API works perfectly - until someone hammers it with 10,000 requests in a second.

Rate limiting is what stands between a stable system and a full outage. But not all rate limiting algorithms are equal ↓

1. Fixed Window Counter

Counts requests in a fixed time window and resets after each interval. Simple to implement but burst-prone at window boundaries.

2. Sliding Window Log

Stores each request timestamp and removes expired entries. Accurate limiting but memory-heavy at scale.

4. Sliding Window Counter

Combines current and previous window counts to smooth traffic. Lower memory usage, better burst protection than fixed windows.

4. Token Bucket

Adds tokens at a fixed rate. Requests consume tokens. Supports controlled bursts while maintaining average rate limits. Most widely used.

5. Leaky Bucket

Processes requests at a fixed outflow rate. Smooths bursts by queuing or dropping excess traffic. Predictable but less flexible.

6. Concurrency Limiter

Limits how many requests run simultaneously - not per time window. Essential for protecting downstream services from overload.

How to choose:

→ Need simplicity? Fixed Window

→ Need accuracy? Sliding Window Log

→ Need balance? Sliding Window Counter

→ Need burst tolerance? Token Bucket

→ Need smooth throughput? Leaky Bucket

→ Protecting a slow backend? Concurrency Limiter

Most production systems combine 2–3 of these at different layers - gateway, service, and database.

One algorithm rarely covers all your attack surfaces.

Which one does your system rely on?

LINKS

- https://www.youtube.com/@learnwithrockybhatia

#139 Nikki Siapno

System Design

Eng Manager & Founder | Ex-Canva | 450k+ Community | Helping Senior Engineers Master System Design and AI Engineering | View my newsletter

4mo · View on LinkedIn

Postgres vs MySQL

Postgres is built as a single, integrated system, and that design choice shapes how everything else works.

Data is stored in heap tables, where multiple row versions coexist. Updates create new versions, while old ones remain until cleaned up (VACUUM). MVCC is built into the data model, enabling advanced indexing and extensibility.

MySQL is built around a pluggable storage engine architecture.

The SQL layer sits on top, while InnoDB handles storage. Old row versions are kept in undo logs and reconstructed when needed. Data is organised as a clustered index, with the primary key defining physical layout.

Which database do you prefer?

——

PS: Want to improve at system design? Download my free System Design Handbook and join 33,000+ engineers who get my free weekly newsletter → https://lnkd.in/gueNmru7

——

Repost to help others learn databases.

+ Follow me (Nikki Siapno) + turn on notifications.

LINKS

- https://lucode.co/luc-system-design-newsletter-pl1
- https://lnkd.in/gueNmru7
- https://www.linkedin.com/feed/update/urn:li:activity:7505202595574136832

ALSO SAVED AS Java Developers Community (moderated) (1mo) link

#140 Satyanarayan Maheshwari

System Design

DevOps Engineer | AWS Solutions Architect | Kubernetes, OpenShift & CI/CD | Practical Cloud, Linux & Automation Content for Aspiring DevOps Engineers

1mo · **View on LinkedIn**

The API already told you what's wrong.

Most developers just don't know how to read it.

.

.

That's why they spend:

✗ 2 hours checking the backend

✗ 1 hour reviewing logs

✗ 30 minutes asking AI

When the answer was already in the response.

Here's the simplest API debugging framework you'll ever learn:

2xx = Everything worked

200 → Request succeeded

201 → Resource created

204 → Success, nothing to return

Translation:

"Your API is doing exactly what you asked."

Stop debugging.

3xx = Look at the route

301 → Moved permanently

302 → Temporary redirect

304 → Cached version available

Translation:

"You're probably looking in the wrong place."

- Check URLs.
- Check redirects.
- Check caching.

4xx = The request is wrong
400 → Bad request
401 → Missing authentication
403 → Permission denied
404 → Resource not found
409 → State conflict
422 → Validation failed
429 → Rate limited

Translation:
"The server understood you. It just doesn't like what you sent."

Check:
✓ Headers
✓ Authentication
✓ Parameters
✓ Request body
✓ Rate limits

5xx = The server is broken
500 → Internal server error
502 → Bad gateway
503 → Service unavailable
504 → Gateway timeout

Translation:
"The problem is not on your side."

Check:
✓ Server logs
✓ Infrastructure
✓ Third-party services
✓ Database connections
✓ Network issues

The fastest debugging rule I know:
2xx → Celebrate
3xx → Follow the redirect
4xx → Fix your request
5xx → Check the server

That's it.
No fancy framework.
No 500-page documentation.
Just four categories.
And suddenly API debugging becomes predictable.
Learn the fundamentals:
• HTTP methods • Request/response cycle • Status codes • REST basics
→ MDN Web Docs :
Understand how the web really works:

• HTTP headers • Authentication • Caching • Error handling
→ Postman Learning Center :
Learn by doing:
• API testing • Collections • Environment variables • Automated requests
Follow Satyanarayan Maheshwari for more such content
Save this before your next production incident.
You'll need it sooner than you think.
Repost if you know a developer who still treats every error the same way.

MEDIA document/carousel (possible)

#141 DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef

Cloud / K8s / DevOps

Neel Shah • 3rd+
1mo · **View on LinkedIn**

If your Pods can’t find each other, your application isn’t broken. Your DNS probably is.
One of the most frustrating Kubernetes incidents is when:
✗ Pods are running.
✗ Services are healthy.
✗ Network looks fine.
...but applications still can't communicate.
The culprit?
DNS resolution inside the cluster.
That's why I created this Kubernetes DNS Troubleshooting Cheat Sheet—a practical guide to diagnose DNS failures step by step.
It covers everything you need:
✓ Check CoreDNS health
✓ Analyze CoreDNS logs
✓ Validate CoreDNS configuration
✓ Test DNS resolution with nslookup & dig
✓ Verify kube-dns Service
✓ Inspect Network Policies
✓ Validate your CNI plugin (Calico, Cilium, Flannel, Weave)
A simple troubleshooting flow:
1. Check CoreDNS Pods
2. Review CoreDNS Logs
3. Validate ConfigMap
4. Test DNS from inside a Pod
5. Verify Network Connectivity
6. Inspect Network Policies
7.Check the CNI Plugin
One lesson every Kubernetes engineer eventually learns:
If Pods can communicate using IPs but not Service names, stop looking at your application. Start looking at DNS.
That single observation can save hours of debugging.

Save this cheat sheet- you'll thank yourself during your next production incident.

Question: What's the most difficult Kubernetes issue you've ever debugged—DNS, networking, storage, or scheduling?

Let's learn from each other's experiences. ↓

Repost if this helps someone become a better Kubernetes engineer.

MEDIA advertisement

LINKS
- https://sessionize.com/neel-shah/

#142 Charly Wargnier

AI / LLM

Deep in Agents, LLMs & automation · Sharing AI gems daily to help you work smarter · Open source at · Ex-data scientist Snowflake | Streamlit · 181K followers on X · DM for collab ·

1mo · View on LinkedIn

SPOTIFY JUST OPEN-SOURCED A CLAUDE CODE SETUP THAT CUTS TOKEN USAGE BY ~90%

Most coding-agent spend isn’t reasoning.

It’s I/O: reading huge files, finding patterns, and generating repetitive code.

Their solution:

→ Gemini Flash handles bulk reads + boilerplate

→ Claude handles the actual reasoning

→ hooks automatically block expensive large-file reads and redirect them

They first tried CLAUDE.md rules.

Claude could ignore them.

Hooks fixed that.

Result: ~90% fewer tokens on bulk reads in a Java monorepo.

100% free and open-source.

Plugin link → https://lnkd.in/dRdsvQkN

Huge props to Spotify for sharing it with the community - Don’t forget to ★ it.

Full breakdown on their blog → https://lnkd.in/dqsevwFZ

LINKS
- http://CLAUDE.md
- https://lnkd.in/dRdsvQkN
- https://lnkd.in/dqsevwFZ

#143 Snahasis Barat

Java / Spring

Java Backend Developer | 4 Years Experience | Java | Spring Boot | Microservices | REST APIs | Kafka | Banking/ BFSI | Cloud | Ex-Virtusa | Ex-GlobalLogic | Immediate Joiner

1mo · View on LinkedIn

"@Transactional" in Spring Boot looks simple.

Add the annotation and Spring handles the transaction. But in real projects, understanding where the transaction starts and when it rolls back is what really matters.

Here are 4 common mistakes I’ve seen ↓

1. Multiple DB operations without a transaction

```
@Transactional
public void processTransfer() {
saveTransaction();
updateBalance();
```

updateStatus();

}

If one operation fails, we usually want all related DB changes rolled back to maintain consistency.

2. Misunderstanding propagation

"REQUIRED" joins the existing transaction (or creates one), while "REQUIRES_NEW" suspends it and starts a separate transaction.

Choosing the wrong propagation can completely change rollback behavior.

3. Internal method calls

public void process() {

saveTransaction();

}

@Transactional

public void saveTransaction() { ... }

Calling a "@Transactional" method from another method in the same class normally bypasses Spring's transactional proxy.

4. Assuming every exception triggers rollback

By default, checked exceptions don't normally trigger rollback.

@Transactional(rollbackFor = Exception.class)

Example from my current banking project

A transaction flow can involve multiple related operations:

Create transaction → Update account data → Update transaction status

These should often behave as a single unit of work. If one DB operation fails, we don't want partially updated financial data.

That's where defining the correct transaction boundary at the service layer becomes critical.

My biggest takeaway:

"@Transactional" isn't just about wrapping database calls. It's about defining the atomic boundary of a business operation.

What "@Transactional" issues have you faced in production?

#Java #SpringBoot #Microservices #JavaDeveloper #BackendDevelopment #SoftwareEngineering #BankingTechnology

MEDIA advertisement

#144 Amar Bhise

Java / Spring

Full Stack Engineer focused on Java Development and collaboration

1mo · **View on LinkedIn**

If you're learning Spring Boot, don't just memorize annotations. Understand what they actually do.

Spring Boot annotations reduce configuration, simplify development, and help us build clean, maintainable applications.

Here are some important annotations every Java developer should know:

"@SpringBootApplication"

Main entry point. Combines "@Configuration", "@EnableAutoConfiguration", and "@ComponentScan".

"@RestController"

Used to build REST APIs and return data directly in the response.

"@RequestMapping"

Maps HTTP requests to controller methods.

"@Service"

Marks the business logic layer.

"@Repository"

Used for the data access layer.

"@Autowired"

Provides dependency injection. Constructor injection is generally preferred in modern applications.

"@Configuration" + "@Bean"

Used to define and configure Spring-managed beans.

"@Value"

Injects configuration values from properties or environment variables.

"@Profile"

Supports environment-specific configurations like "dev", "test", and "prod".

"@Transactional"

Manages database transactions and helps maintain data consistency.

"@ExceptionHandler"

Handles specific exceptions and is commonly used with "@ControllerAdvice".

Developer Tip:

Don't learn annotations as definitions. Learn when to use them, why they are needed, and how Spring processes them.

Save this for your Spring Boot learning journey.

#Java #SpringBoot #JavaDeveloper #BackendDeveloper #RESTAPI #SpringFramework #SpringDataJPA #Hibernate #Coding #Programming

#145 Ganesh P Java / Spring

Immediate joiner | Full-Stack Engineer | React.js, Next.js, Node.js | AI & Backend Engineering | System Design | Scalable Production Systems

1mo · **View on LinkedIn**

After ~3 years of working with Spring Boot, I realized something...

When I first started working with Spring Boot, I thought being good at it meant knowing a lot of annotations.

@RestController

@Autowired

@Service

@Repository

@Transactional

@Entity

...and many more.

I could tell you what each one does.

But over time, I realized that knowing an annotation and understanding why it exists are two completely different things.

For example:

→ @Transactional isn't just "for transactions."

You need to understand what happens when a transaction starts, how Spring manages it, what happens when an exception occurs, rollback behavior, propagation, isolation, and even how Spring proxies can affect it.

→ @Async isn't simply "make this method asynchronous."

You need to understand thread pools, task executors, proxy-based execution, and why calling an @Async method from the same class can behave differently than expected.

→ @Cacheable isn't just "add caching."

You need to think about cache keys, stale data, eviction, consistency, and whether caching actually makes sense for that particular use case.

And this is something I've gradually learned through working with backend systems.

So I put together this Spring Boot Annotations Cheat Sheet

It covers annotations across:

Core Spring & Dependency Injection

REST APIs

JPA & Hibernate

Spring Data JPA

Transactions

Validation

Exception Handling

Configuration & Profiles

Spring Security

Caching

Async Processing

Scheduling

AOP

Testing

Kafka & Messaging

Feign Clients

MongoDB

Lombok

Some annotations I believe every Spring Boot backend developer should be comfortable with:

@SpringBootApplication

@Service / @Repository

@RestController

@RequestBody / @PathVariable / @RequestParam

@Entity / @Id / @GeneratedValue

@OneToMany / @ManyToOne

@Transactional

@Valid

@RestControllerAdvice

@ConfigurationProperties

@PreAuthorize

@Cacheable

@Async

@Scheduled

@KafkaListener

@FeignClient

My biggest takeaway after 3 years:

Don't learn Spring Boot by memorizing annotations.

Learn the problem first, understand how Spring solves it, and then understand what the annotation is actually doing behind the scenes.

That's when Spring Boot starts feeling much less like "magic" and much more like a system you actually understand.

I made this cheat sheet as a quick revision guide for myself and thought it might help other Java/ Spring Boot developers too.

Save it for your next Spring Boot revision.

What Spring Boot concept or annotation took you the longest to understand?

#Java #SpringBoot #Spring #BackendDevelopment #JavaDeveloper #BackendEngineer #SoftwareEngineering #Programming #TechInterview #SystemDesign

MEDIA advertisement

#146 Tejal Rathod

Kafka / Messaging

Cloud-native microservices developer specializing in RESTful API design.

1mo · **View on LinkedIn**

Kafka acks=0 vs acks=1 vs acks=all — When Should You Use Which?

Here’s the simple way to remember it ↓

acks=0 — No ACK

Producer sends the message and doesn't wait for Kafka's acknowledgment.

✓ Fastest

✗ Lowest durability

Use case:

High-volume, non-critical data like metrics, telemetry, or some logging events.

→ Speed > Durability

acks=1 — Leader ACK

Producer waits for the leader broker to acknowledge the message.

Producer → Leader → ACK

↓

Followers

✓ Good performance

(!) Message can potentially be lost if the leader fails before replication.

Use case:

Analytics or user activity events where occasional loss is acceptable.

→ Balance between speed & durability

acks=all — ISR ACK

Producer waits for the message to be acknowledged by the required in-sync replicas (ISR).

Producer → Leader

↓

ISR Replicas

↓

ACK

✓ Strongest durability

(!) More latency than acks=0/1

Use case:

Payments

Orders

Financial transactions

Audit events

→ Durability > Speed

Important interview point

acks=all does NOT mean "all replicas".

It means acknowledgment from the required in-sync replicas (ISR).

For example:

replication.factor=3 min.insync.replicas=2 acks=all

For critical events, you may also use:

acks=all enable.idempotence=true

Easy way to remember:

acks=0 → Don't wait

acks=1 → Leader received it

acks=all → Required ISR received it

For critical business events like payments and orders, I would generally prefer acks=all with appropriate replication and min.insync.replicas. For non-critical, high-volume data, acks=0 may be suitable. acks=1 is a middle ground.

#Kafka #ApacheKafka #Java #SpringBoot #Microservices #SystemDesign #DistributedSystems #BackendDevelopment #JavaDeveloper #TechInterview

#147 Sireesha D Java / Spring

Lead Software Engineer | Java, Spring Boot, Microservices | Event-Driven Architecture, AWS | System Design | Software Architecture| AI-Driven Development | Ex-JPMorgan Chase

1mo · **View on LinkedIn**

A Spring Boot application has a cron/@Scheduled job and is deployed on three instances. How do you ensure that the job runs on only one instance?

Imagine a Spring Boot application with a scheduled job:

@Scheduled

Everything works fine when we have one instance.

But what happens when the same application is deployed on 3 instances?

Each instance can trigger the same scheduled job.

I may not want the job to run three times because it can lead to:

Duplicate processing

Unnecessary resource usage

Inconsistent data or duplicate transactions

How do we solve it?

→ Distributed Lock.

A distributed lock allows multiple instances to coordinate through a shared store such as a database or Redis.

Only the instance that acquires the lock executes the job.

The other instances skip that execution.

When the scheduled time arrives:

✓ Only one instance acquires the lock and executes the job.

The other instances skip that execution.

In Spring Boot, ShedLock is one practical way to implement this for scheduled jobs.

But now, here is an interesting question...

If the job should run only once, why do we need three instances?
What do you think? Let me know in the comments ↓

#148 DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef

Cloud / K8s / DevOps

Neel Shah • 3rd+
1mo · View on LinkedIn

CrashLoopBackOff is NOT a Kubernetes Pod phase.
This is a small distinction, but an important one when troubleshooting Kubernetes.
A Pod moves through 5 official phases:
1. Pending
The Pod is accepted by Kubernetes, but one or more containers are not ready to run yet.
Possible reasons:
→ Waiting for scheduling
→ Image pulling
→ Volume mounting
→ Init container execution
2. Running
The Pod is assigned to a node and at least one container is running, starting, or restarting.
(!) Running does NOT always mean the application is healthy.
3. Succeeded
All containers terminated successfully and won't restart.
Common example: a completed Kubernetes Job.
4. Failed
All containers have terminated, and at least one terminated unsuccessfully.
5. Unknown
Kubernetes cannot determine the Pod state, often because communication with the node is lost.
But what about:
ContainerCreating
CrashLoopBackOff
ImagePullBackOff
Terminating
These are not official Pod phases.
For example, CrashLoopBackOff usually means a container repeatedly starts → crashes → restarts, while Kubernetes applies increasing backoff between restart attempts.
How I troubleshoot
I don't stop at:
kubectl get pods
I check:
kubectl describe pod <pod-name>
and look at:
→ Container State / Last State
→ Restart Count
→ Exit Code

→ Pod Conditions

→ Events

To check the actual Pod phase:

kubectl get pod <pod-name> -o jsonpath='{.status.phase}'

For termination, Kubernetes normally follows:

Delete Pod → SIGTERM → Grace Period → SIGKILL

The default terminationGracePeriodSeconds is typically 30 seconds.

Force deletion:

kubectl delete pod <name> --grace-period=0 --force

should be a last resort, not the first troubleshooting step.

Interview Question

What's the difference between Pod phase and CrashLoopBackOff?

Answer:

Pod phase is the high-level lifecycle summary of a Pod.

CrashLoopBackOff is a container-level waiting reason indicating repeated container failures and restart backoff.

Key takeaway:

Don't just memorize Pod states.

Understand the difference between Pod-level status and container-level status — that's where real Kubernetes troubleshooting starts.

What do you check first when you see CrashLoopBackOff?

#Kubernetes #DevOps #KubernetesTroubleshooting #CloudNative #SRE

MEDIA advertisement

LINKS

- https://sessionize.com/neel-shah/

#149 Pujitha Bandla

Cloud / K8s / DevOps

Sr. Java Full Stack Developer @ Best Buy | Java 17 | Spring Boot | Microservices | AWS | GCP | Azure | Angular | React | Kafka | Kubernetes | Docker | REST APIs | GraphQL | PostgreSQL | Redis | CI/CD Automation | Agile.

1mo · **View on LinkedIn**

Java development in 2026 is about much more than just writing Java code.

To stand out in today’s market, developers need to understand how modern enterprise applications are built, deployed, scaled, and improved.

The skills I believe are especially valuable right now are:

Spring Boot & Microservices

AWS & Cloud

Kafka & Event-Driven Architecture

Docker, Kubernetes & CI/CD

GenAI-assisted development

Strong fundamentals still matter, but combining them with cloud, distributed systems, DevOps, and AI can make a huge difference in how effectively we build modern applications.

I’m continuing to learn and strengthen these areas as technology evolves.

Which of these skills are you focusing on right now?

#Java #SpringBoot #Microservices #AWS #Kafka #Kubernetes #Docker #DevOps #GenerativeAI #SoftwareEngineering #FullStackDeveloper #JavaDeveloper #CloudComputing

MEDIA advertisement

#150 DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef

Cloud / K8s / DevOps

Neel Shah • 3rd+

1mo · View on LinkedIn

Kubernetes Namespaces: The Secret to Keeping Your Cluster Organized & Scalable

As Kubernetes environments grow, managing resources efficiently becomes critical. One of the most fundamental concepts every Kubernetes practitioner should master is Namespaces.

What are Namespaces? Namespaces provide logical isolation within a Kubernetes cluster, allowing teams, applications, and environments to coexist without interfering with each other.

Why Use Namespaces? ✓ Organize resources effectively

✓ Isolate teams, projects, and environments

✓ Simplify access control with RBAC

✓ Apply resource quotas and limits

✓ Avoid naming conflicts across workloads

Common Default Namespaces • default – For general workloads

• kube-system – Kubernetes system components

• kube-public – Publicly readable resources

• kube-node-lease – Node heartbeat information

Namespace vs Cluster-Scoped Resources Namespaced Resources: Pods, Services, Deployments, ConfigMaps, Secrets, Jobs, PVCs

Cluster-Scoped Resources: Nodes, Persistent Volumes, Namespaces, ClusterRoles, StorageClasses

Useful kubectl Commands

1.kubectl get ns

2.kubectl create ns dev

3.kubectl get pods -n dev

4.kubectl get all -n prod

5.kubectl delete ns staging

Best Practices ✓ Use meaningful namespace names (dev, test, prod)

✓ Always specify namespaces in manifests and commands

✓ Apply RBAC policies per namespace

✓ Configure ResourceQuotas to prevent resource overconsumption

(!) Remember: Namespaces provide logical separation, not security by themselves. Combine them with RBAC, Network Policies, and Resource Quotas for stronger multi-tenant environments.

Mastering namespaces is a small step that makes a huge difference in managing Kubernetes at scale.

#kubernetes #namespaces #devops #container #cloud

MEDIA advertisement

LINKS

- https://sessionize.com/neel-shah/

#151 Shreekrushanaprasad Mane

System Design

SDE - Backend @GrayQuest Finance | Software Development | Backend Developer

1mo · View on LinkedIn

Everyone adds Redis for caching.

In backend it does five jobs — and caching isn't the only thing.

1. A lock so cron jobs run once

Kafka consumers split work through the consumer group. In-process schedulers don't. So the moment we autoscaled, every pod fired the same report job: duplicate emails, duplicate retry runs.

SET scheduler_lock:<job> 1 NX EX <ttl> → one pod wins, the rest skip.

For interval jobs we never release the lock. The TTL covers most of the interval, so a pod whose timer drifted still finds it locked.

2. A 5-second dedup window for Kafka events

SET dedup:<key> 1 NX EX 5. Key already there → duplicate → skip.

This one bit us. Two KYC documents uploaded in the same second collapsed into one key, and the second was dropped silently — no error, no trace. A later code review found the same trap waiting for two payments in one checkout.

Lesson: the key must carry everything that makes two events genuinely different. Document type. Transaction id.

3. Delayed Kafka messages

Kafka still has no "deliver this at 10 AM tomorrow" (KIP-1277 wants to add it). A small scheduler service parks the message in asynq — a Redis-backed queue — and publishes it when due, with retries and timeouts.

4. Rate limiting

INCR on a per-second key for the rules engine. Per-mobile-number limits for login and OTP, shared across every server.

One catch: set the TTL only when INCR returns 1, inside a Lua script. Do it as two commands and a crash in between leaves a key with no expiry — that user is throttled forever.

5. Progress for background jobs

A bulk Excel upload returns a job_id immediately. The worker does HINCRBY processed / failed, a status endpoint reads it, a TTL cleans it up. No extra table.

SET NX with a TTL solves more distributed-systems problems than I expected.

What's the strangest thing you've used Redis for?

#Redis #BackendDevelopment #DistributedSystems #Kafka #SystemDesign

MEDIA advertisement; document/carousel (possible)

#152 Rahuul Gaarg

System Design

Enterprise & Solution Architect | 16+ Years Designing & Delivering Scalable Enterprise Solutions | Cloud-Native, Microservices & Event-Driven Architecture | Azure & .NET Expert | Technology Leader | Problem Solver

1mo · **View on LinkedIn**

Why Real-World Architectures Are the Best Way to Learn System Design

After working across enterprise architecture, cloud platforms, APIs, and distributed systems, event driven, payment gateway etc... I've learned one important lesson:

System design concepts make sense only when you see them implemented in real-world products.

Scalability, resilience, caching, fault tolerance, consistency, and distributed computing are easy to understand in theory. The real learning happens when you analyze how global-scale platforms solve these challenges in production.

That's why architecture case studies are invaluable for architects, engineers, and technology leaders.

What These Platforms Teach Us

WhatsApp:

-Real-time messaging

-WebSockets & persistent connections

-Distributed message storage
-Event-driven architecture
YouTube:
-Media ingestion & transcoding
-Metadata management
-Object storage
-CDN-based content delivery
Uber:
-Real-time GPS processing
-Geospatial indexing
-Driver-rider matching
-Dispatch and routing systems
Netflix:
-Microservices architecture
-Global streaming infrastructure
-Recommendation engines
-Resilience and chaos engineering
Amazon:
-Order orchestration
-Payment processing
-Inventory management
-Distributed transactions
Instagram:
-Feed generation
-Media storage
-Distributed caching
-Content delivery optimization
X (Twitter):
-Timeline architecture
-Fanout strategies
-Search indexing
-High-throughput data processing
Zoom:
-WebRTC architecture
-Signaling services
-Real-time media transport
-Collaboration platforms
-The Real Value Lies in Understanding Trade-offs
Every large-scale system is built around architectural decisions involving:
✓ Scalability
✓ Sharding & Partitioning
✓ Replication & Consistency
✓ Caching Strategies
✓ Load Balancing
✓ Event-Driven Architecture
✓ Microservices

✓ Fault Tolerance
✓ Observability
✓ Cloud-Native Design

The goal isn't to memorize architecture diagrams, the goal is to understand:

• Why a design was chosen
• Which trade-offs were accepted
• How bottlenecks are addressed
• How failures are handled
• How systems scale reliably

Great system design is the art of breaking complex business problems into loosely coupled, resilient, and independently scalable services.

If you're preparing for Architect, Staff Engineer, Principal Engineer, Engineering Manager, or System Design interviews, studying these real-world architectures is one of the highest ROI investments you can make.

Note: The diagrams represent high-level conceptual views and are intended as learning aids, not production-grade architectures.

#SystemDesign #SoftwareArchitecture #DistributedSystems #CloudArchitecture #Microservices #PlatformEngineering #TechLeadership #SolutionArchitecture #EngineeringExcellence #Scalability

#153 Kubernetes

Cloud / K8s / DevOps

Muhammad Ramzan • 3rd+
1mo · View on LinkedIn

Cloud-Native MLOps with Kubernetes

Your ML model works in a notebook.

But can you reliably deploy, scale, monitor, and retrain it in production?

That’s where Kubernetes + MLOps comes in.

A modern cloud-native MLOps stack can connect the complete ML lifecycle:

Feast — Feature Store

Consistent, reusable features for training and inference.

Kubeflow / Argo — ML Pipelines

Build reproducible, version-controlled training and validation workflows.

Kubernetes + KEDA — Elastic Compute

Dynamically scale workloads and GPU resources based on demand.

KServe / Seldon — Model Serving

Support autoscaling, canary deployments, A/B testing, and zero-downtime releases.

Prometheus + Grafana — Observability

Monitor latency, throughput, resource utilization, and infrastructure health.

Evidently — Drift Monitoring

Detect data drift and model-performance degradation.

The real power comes from connecting everything into an automated feedback loop:

Data → Training → Validation → Deployment → Monitoring → Drift Detection → Retraining

This approach brings familiar Kubernetes principles into the ML world:

✓ Declarative infrastructure
✓ Automation
✓ Scalability
✓ Self-healing workloads

✓ Version-controlled pipelines
✓ Continuous monitoring
MLOps is essentially about taking ML from experimentation to reliable production.
What does your MLOps stack look like today?
Are you using Kubernetes, Kubeflow, KServe, Argo, SageMaker, Azure ML, Vertex AI, or something custom?
What has been the hardest part: GPU management, deployment, pipelines, monitoring, or model drift?
#MLOps #Kubernetes #CloudNative #MachineLearning #AI #DevOps #K8s #PlatformEngineering #DataEngineering

MEDIA advertisement

#154 Bhanu Prasad T.

Java / Spring

Senior Software Engineer @ HCLTech | Java | Spring Boot | Microservices | MBA in IT | PM Fellow @ NextLeap
1mo · **View on LinkedIn**

Java Interview Topic: Why Java Records Matter
Have you ever written a DTO with:
✗ Private fields
✗ Constructor
✗ Getters
✗ equals()
✗ hashCode()
✗ toString()
Java Records can dramatically reduce this boilerplate.
```java
public record Employee(
String name,
int age,
String department
) {}
```
The compiler automatically provides the canonical constructor, accessors, equals(), hashCode(), and toString().
Why are Records important?
✓ Concise data-carrier classes
✓ Immutable state by design
✓ Less boilerplate
✓ Cleaner REST API DTOs
✓ Useful in microservices
✓ Easier to read and maintain
Interview Question
Q: Are Java Records completely immutable?
→ The record components cannot be reassigned after construction, but the objects referenced by those components may still be mutable.
Example:
```java
```

record Employee(String name, List<String> skills) {}
```

The `Employee` reference is immutable, but the `skills` list itself can still be modified if it is mutable.

interview takeaway:

> Record ≠ magically deep immutable.

Records were introduced as a preview in Java 14 and became a permanent feature in Java 16.

Write less. Build more.

#Java #Java17 #JavaRecords #SpringBoot #Microservices #JavaInterview #BackendDevelopment #SoftwareEngineering

## #155 Java Developers Community (moderated)

Java / Spring

Mohamed EL Laithy • 3rd+

1mo · View on LinkedIn

Hibernate & JPA can be confusing... until you see the big picture.

I've created a collection of 12 visual Hibernate & JPA infographics to make some of the most important concepts easier to understand, revise, and remember.

One of them is this Hibernate & JPA Annotations Map ↓

It brings together 32 essential annotations across 4 core areas:

Entity & Identity

@Entity • @Id • @Column • @Embedded • @GeneratedValue

Relationships & Mapping

@OneToMany • @ManyToOne • @ManyToMany • @JoinColumn • @JoinTable • @Cascade

Querying & Performance

@Query • @NamedQuery • @BatchSize • @Fetch • @Cacheable • @EntityGraph

Modern Hibernate & Lifecycle

@Version • @SoftDelete • @DynamicUpdate • @CreationTimestamp • @UpdateTimestamp • @Repository

And the best part?

I'm giving away all 12 infographics for FREE.

They're designed as quick visual references for:

✓ Java Backend Developers

✓ Spring Boot Developers

✓ JPA/Hibernate learners

✓ Interview preparation

✓ Developers who want a quick reference while coding

→ Get all 12 infographics for free here: https://lnkd.in/ejmZj_YT

If you find them useful, save this post and share it with another Java developer who might benefit from them.

#Java #Hibernate #JPA #SpringBoot #JavaDeveloper #BackendDevelopment #SoftwareEngineering #Programming

**LINKS**

- https://lnkd.in/ejmZj_YT

## #156 Sweety Sharma

Java / Spring

Top 1% | SDE-III | JAVA | Springboot | System Design | Distributed Systems | Technical Architect @ Qwik Labs

1mo · View on LinkedIn

Java Concurrency & Multithreading - Top 40 Interview Questions
```

1. What is the difference between a Process and a Thread?
2. Platform Thread vs Virtual Thread?
3. How does the Java Memory Model work?
4. What is the happens-before relationship?
5. synchronized vs ReentrantLock?
6. volatile vs Atomic variables?
7. What is CAS and how does it work?
8. How does ConcurrentHashMap achieve thread safety?
9. What is a Race Condition?
10. How do you prevent Race Conditions?
11. What is Deadlock?
12. How do you detect and prevent Deadlocks?
13. Deadlock vs Livelock vs Starvation?
14. What is Thread Starvation?
15. ExecutorService vs ForkJoinPool?
16. How does ThreadPoolExecutor work?
17. How do you choose Core Pool Size and Maximum Pool Size?
18. What happens when a ThreadPoolExecutor queue becomes full?
19. CompletableFuture vs Future?
20. How does CompletableFuture work internally?
21. thenApply vs thenCompose vs thenCombine?
22. What is CountDownLatch?
23. CountDownLatch vs CyclicBarrier?
24. What is Semaphore and where would you use it?
25. What is Phaser?
26. What is ThreadLocal?
27. What problems can ThreadLocal cause?
28. What is False Sharing?
29. What is ForkJoinPool?
30. How do parallel streams use threads?
31. Why can parallel streams hurt performance?
32. How do Virtual Threads work internally?
33. When should you NOT use Virtual Threads?
34. Virtual Threads vs CompletableFuture?
35. How do you handle blocking operations with Virtual Threads?
36. How would you debug a production thread issue?
37. How would you investigate high CPU caused by Java threads?
38. How would you identify thread pool exhaustion?
39. How would you design a thread-safe cache?
40. Describe a real-world concurrency problem you solved.

Keeping this in mind, I went deep and documented everything into a Java Backend Developer Guide.
Get the Guide here:

https://lnkd.in/dpGUaWZT

#Java #Concurrency #Multithreading #VirtualThreads #JavaInterview #SpringBoot #BackendEngineering

Complete Interview Prep Kit- Java Backend Engineer

Complete Interview Prep Kit- Java Backend Engineer

prepflixnet.graphy.com

LINKS

- https://lnkd.in/dpGUaWZT
- https://prepflixnet.graphy.com/telegram/6a64d12b1a6e56209575c031

#157 Srinjoy Ghosh

AI / LLM

Senior Java Backend Developer | Spring Boot, WebFlux, Kafka, Microservices, DSA | Open to SDE-2/SDE-3 Product Roles | Oracle Certified: Agentic AI Foundation

1mo · **View on LinkedIn**

3 RAG Architectures. One word separates them:

Retrieves. Connects. Reasons.

Each designed for a different class of problem.

The biggest mistake? Not “building RAG wrong” but building the right RAG for the wrong problem... then wondering why answers miss the mark.

Classic RAG — Retrieves

- Flow: Query → Embed → Vector DB → Top-K Chunks → LLM → Answer

- Fast. Simple. Single-hop.

- Best for: straightforward Q&A where similarity search is enough.

- Limitation: one-pass only, no relationships, no adaptation.

Graph RAG — Connects

- Flow: Query → Entity Extraction → Knowledge Graph → Connected Context → LLM → Answer

- Relational. Entity-rich. Multi-source.

- Best for: queries where relationships matter more than isolated chunks.

- Instead of “nearest neighbors,” Graph RAG maps entities, traverses the graph, and assembles connected context that classic retrieval misses.

Agentic RAG — Reasons

- Flow: Query → Reasoning Agent → Vector DB + Knowledge Graph + Web Search + Tools → Self-Evaluation → Final Answer

- Adaptive. Step-by-step. Self-correcting.

- Best for: complex, open-ended queries needing multiple retrieval passes, dynamic source selection, and quality checks.

- The agent decides what to retrieve, when to search, and whether results are good enough looping until they are.

Decision Framework

- Simple document Q&A → Classic RAG

- Entity-rich, relationship-heavy queries → Graph RAG

- Complex, adaptive, multi-step reasoning → Agentic RAG

The architecture you choose sets the ceiling of your system.

Choose based on your data reality not what’s easiest to build first.

Save this breakdown and share it with anyone still treating RAG architectures as interchangeable.

#RAG #GenerativeAI #LLM #AIArchitecture #KnowledgeGraph #AgenticAI #VectorDB #AIEngineering #MachineLearning #AIForBusiness #TechLeadership #AIInnovation #DataDrivenAI #AICommunity

MEDIA document/carousel (possible)

#158 Subhashree Mohan Swain

System Design

Full-Stack Software Engineer (2+ years) [Serving Notice Period] | Payment Orchestration & Backend Systems | Node.js • React • GraphQL • PostgreSQL • GCP • Kubernetes

1mo · View on LinkedIn

One caching question in an interview made me think beyond just “use Redis.”

Recently, I was asked a question around cache penetration and cache stampede.

The interviewer’s question was essentially:

«“What happens if your cache starts causing more load on the database instead of reducing it?”»

My approach was to first understand where the cache failure could happen.

1. Cache Penetration

I considered the case where clients repeatedly request data that doesn't exist.

For example:

"GET /users/999999999"

The flow becomes:

→ Cache miss

→ Database query

→ No data found

→ Nothing gets cached

→ Same request comes again

→ Database gets hit again

If this happens at scale, the cache provides almost no protection.

My possible solutions:

• Cache negative results with a short TTL

• Use a Bloom Filter for existence checks

• Validate request parameters

• Apply rate limiting

2. Cache Stampede

Then I looked at a different scenario.

Suppose:

"product:123 → TTL = 60 seconds"

The cache expires and suddenly thousands of requests arrive.

If every request sees a cache miss and independently queries the database:

→ Thousands of DB requests

→ Thousands of cache rebuild attempts

→ Increased latency

→ Potential cascading failure

Here, my approach would be:

• Distributed locking

• Request coalescing / Single-flight

• TTL jitter

• Background refresh

• Stale-while-revalidate

The key distinction I keep in mind:

Cache Penetration → The requested data doesn't exist.

Cache Stampede → The data exists, but many requests try to rebuild an expired cache simultaneously.

What I liked about this question is that it wasn't really about Redis.

It was about thinking through failure scenarios in a distributed system.

That's the mindset I try to bring into system design:

Don't just ask, “How do I make it fast?”

Also ask:

→ “How does this system behave when things go wrong?”

#SoftwareEngineering #BackendDevelopment #SystemDesign #Redis #Caching #DistributedSystems #NodeJS #Scalability #Architecture #TechInterview

#159 Anshika Sinha System Design

Engineering Leader@NetApp | Ex- Google | Gen & Agentic AI Platforms | Multi-Cloud Infrastructure | Organizational Scaling | Product & Platform Strategy

1mo · **View on LinkedIn**

If you're targeting Senior SDE roles, be ready for this question:

“Your team shipped a feature that caused a production incident. What do you do?”

This is where Senior SDE interviews go beyond technical skills.

Having led engineering teams at Google and now NetApp, I've seen why this question matters.

Ownership isn't about finding someone to blame.

First, stabilize production.

Then understand what happened.

I'd look at logs, metrics and distributed traces.

But the interviewer may push further:

“What if your team made the wrong architectural decision?”

I'd own the outcome.

Then I'd ask why the decision made sense at the time.

Were the requirements unclear?

Did we miss an edge case?

Was testing strong enough?

Did we have the right observability?

And then comes the most important part:

“What changes so this doesn't happen again?”

Maybe it's better deployment guardrails.

Maybe automated rollback.

Maybe stronger monitoring.

Or maybe the architecture itself needs to change.

That's what senior-level ownership looks like.

You don't just fix the incident.

You improve the system so the same failure is less likely to happen again.

That's why Senior SDE interviews evaluate more than System Design knowledge.

They're looking for ownership, judgment and accountability.

Bosscoder Academy's EVOLVE helps SDEs with 6+ YOE prepare for these deeper conversations through System Design, mentorship and mock interviews.

If you're preparing for your next PBC switch, EVOLVE is built for this level of preparation.

Explore EVOLVE for SDEs with 6+ YOE here → https://lnkd.in/g-TUhJbd

They’ve helped 2200+ engineers crack top PBC roles through:

Structured DSA, System Design & AI curriculum

Real-world, AI-powered projects & 24/7 doubt-solving

1:1 mentorship from industry experts

Resume reviews, mock interviews & job-switch support

#SeniorSDE #Leadership #EngineeringLeadership #InterviewPreparation #SoftwareEngineering

LINKS

- https://lnkd.in/g-TUhJbd

#160 Mohammad F.

AI / LLM

Founder @LearnYard | Ex-Google | 500K+ YouTube learners | TEDx Speaker

1mo · **View on LinkedIn**

Google is making their whole interview process AI based?

I have seen this Reddit post going around lately, and I think people are reading way too much into it.

The post claims Google is adding two new "AI Fluency" rounds:

→ AI Debugging

→ AI System Design

The direction makes sense.

Google has already been testing AI-assisted code comprehension interviews on some teams, where candidates get a messy codebase and use Gemini to understand, debug, and improve it.

That is much closer to how engineers actually work now.

But this does not mean DSA is suddenly dead.

One person I know recently interviewed for an L4 role and still got the standard loop with coding rounds, Googleyness, and the usual technical evaluation.

From what I am seeing, the change is going slowly.

Some teams may test AI-assisted debugging.

Some may keep the classic loop.

Some may mix both.

And that is probably how this evolves for a while.

So I would not completely redo my interview preparation because of one Reddit post.

But I would definitely take the hint.

If Google is experimenting with interviews that test how you work with AI, start building these skills now:

→ Read unfamiliar code quickly

→ Debug existing systems instead of only writing from scratch

→ Use AI with targeted prompts

→ Verify every suggestion instead of accepting it blindly

→ Spot hallucinations and bad assumptions

→ Explain why you accepted or rejected an AI answer

→ Understand logs, tests, edge cases, and failure paths

→ Practice system design with AI components like RAG, inference, vector search, and evals

→ Keep DSA strong

→ Keep CS fundamentals strong

The interview may change.

The thing being tested will be the same which is:

Can you understand a problem, use the tools available, make good decisions, and explain your reasoning when something goes wrong?

AI is becoming part of the workflow but fundamentals still decide whether you know what to do with it.

#161 Sajeev KR

Java / Spring

Senior Software Engineer @ IBM Software Labs| 11+ Yrs Java/J2EE | Spring Boot & Microservices | AWS | Full-Stack Developer | AI Enthusiast | Jenkins | Shell & Python | Go | Writing Deep Dive Java & System Design Content

1mo · **View on LinkedIn**

Spring Boot Production Issues – 30 Real Scenarios

Answer is shared in this link - https://lnkd.in/gKj8SSqW

1. Your API latency suddenly increases from 200ms to 5 seconds. How would you investigate?

2. CPU usage reaches 100% after a new deployment. What would you check first?

3. Application memory keeps increasing even after Garbage Collection. What could be causing it?

4. Your application is throwing OutOfMemoryError. How would you identify the root cause?

5. Database connections are exhausted. How would you troubleshoot the issue?

6. HikariCP connection pool is completely occupied. What could cause this?

7. One API is making hundreds of database queries for a single request. How would you identify and fix it?

8. A Spring Boot application suddenly starts experiencing frequent Full GC. What would you investigate?

9. Requests are getting stuck and thread count keeps increasing. How would you troubleshoot it?

10. A downstream microservice is slow and your service is also becoming unresponsive. How would you prevent this?

11. Your application receives duplicate requests for the same transaction. How would you make the API idempotent?

12. Kafka consumer lag suddenly increases. How would you investigate it?

13. Messages are being processed twice. How would you handle duplicate processing?

14. Your Spring Boot application works locally but fails after deployment to Kubernetes. How would you troubleshoot it?

15. Kubernetes keeps restarting your application Pod. What would you check?

16. A Pod is getting OOMKilled even though the Java heap appears normal. What could be happening?

17. A readiness probe starts failing intermittently. How would you investigate it?

18. Your application becomes slow only during peak traffic. How would you identify the bottleneck?

19. A database query is fast locally but extremely slow in production. What would you check?

20. Redis cache hit rate suddenly drops. How would you investigate it?

21. Your cache contains stale data after a database update. How would you solve it?

22. An external API occasionally takes 30 seconds to respond. How should your Spring Boot service handle it?

23. A retry mechanism causes traffic to increase dramatically during an outage. Why?

24. Logs show errors, but you cannot determine which microservice caused the failure. How would you troubleshoot it?

25. How would you use Micrometer, Prometheus, and Grafana to identify a production bottleneck?

26. How would distributed tracing help you debug a slow request across multiple microservices?

27. A new release causes API performance to drop by 40%. How would you identify whether the deployment caused it?

28. How would you perform a zero-downtime deployment for a Spring Boot application?

29. Your application is healthy, but users are receiving intermittent 500 errors. How would you investigate?

Spring Boot Microservices: 200 Q&A with Code - https://lnkd.in/gqhJhmcZ

Spring Boot Production Issues – 30 Real Scenarios and Solutions

Spring Boot Production Issues – 30 Real Scenarios and Solutions

logicbrace.com

LINKS

- https://lnkd.in/gKj8SSqW
- https://lnkd.in/gqhJhmcZ
- https://www.logicbrace.com/2026/09/spring-boot-production-issues-30-real.html

#162 Code Vimaan

Java / Spring

1mo · View on LinkedIn

30 DAYS OF JAVA — DAY 20

List, Set or Queue?

If you're learning Java Collections, this is one decision you need to get right.

Suppose you have:

10, 20, 10, 30, 20

Do you want:

→ The elements in order?

→ Only unique elements?

→ Elements processed one by one?

Your answer determines which collection makes sense.

List → ordered collection, duplicates allowed

Set → unique elements

Queue → elements processed according to the queue's ordering rules

But choosing a collection isn't just about remembering definitions.

For example:

ArrayList vs LinkedList

HashSet vs TreeSet vs LinkedHashSet

PriorityQueue vs ArrayDeque

Each has different behavior and different use cases.

In today's carousel, you'll learn:

How List works

ArrayList vs LinkedList

How Set handles unique elements

HashSet vs TreeSet vs LinkedHashSet

How Queue processing works

PriorityQueue

ArrayDeque

Common collection interview traps

A Java output-based interview question

Remember this:

List → Order matters.

Set → Uniqueness matters.

Queue → Processing order matters.

But don't assume every Set is unordered.

For example:

HashSet → no guaranteed iteration order

LinkedHashSet → maintains insertion order

TreeSet → maintains sorted order

The collection doesn't just hold your data.
It also determines how that data can be accessed, inserted, removed and processed.
Save this carousel for your Java interview revision.
Share it with someone preparing for Java interviews.
Next:
DAY 21 — HashMap & HashSet Internals
We'll go under the hood and understand how hashing, buckets, keys and collisions actually work.
Follow Code Vimaan for the complete Java journey.
LEARN • BUILD • FLY
#Java #CoreJava #JavaProgramming #JavaCollections #CollectionsFramework #List #Set #Queue #ArrayList #LinkedList #HashSet #TreeSet #JavaDeveloper #LearnJava #JavaInterview #JavaInterviewQuestions #CodingInterview #Programming #Coding #SoftwareDevelopment #SoftwareEngineer #BackendDevelopment #TechEducation #Developer #ProgrammingTips #CodeVimaan #30DaysOfJava
30 DAYS OF JAVA —
.
9 pages

MEDIA advertisement; document/carousel (possible)

#163 Code Vimaan

Java / Spring

1mo · View on LinkedIn

30 DAYS OF JAVA — DAY 21
You use HashMap all the time.
But do you actually know what happens after you write:
Map<String, Integer> map = new HashMap<>();
map.put("Java", 100);
Where does that entry go?
How does Java find it later?
And what happens when two keys end up in the same bucket?
That's where the real understanding of HashMap begins.
The simplified flow is:
Key
↓
hashCode()
↓
Bucket Index
↓
Store / Find Entry
But hashing alone isn't enough.
Java also needs equals() to determine whether two keys should be treated as equal.
And when multiple keys map to the same bucket?
Collision.
Modern HashMap implementations can use linked structures and, under the appropriate conditions, tree bins to improve collision handling.
Now here's the important connection:
HashSet is built on HashMap.
Conceptually, HashSet stores its elements as keys in an internal HashMap.

That's why understanding HashMap makes HashSet much easier to understand.
In today's carousel, you'll learn:
What hashing actually means
How HashMap locates buckets
What happens during a collision
Linked list vs tree bins
How HashSet uses HashMap
Why hashCode() and equals() must be consistent
Custom classes as HashMap keys
Common HashMap interview traps
A Java output-based interview question
The most important rule:
If you override equals(), you should generally override hashCode() consistently.
Otherwise, hash-based collections can behave in ways you don't expect.
For example, two logically equal objects should produce the same hash code.
This is not just about memorizing HashMap interview answers.
It's about understanding the journey:
Key → Hash → Bucket → Collision → equals()
Once this clicks, HashMap and HashSet become much less mysterious.
Next:
DAY 22 — Comparable vs Comparator
We'll understand how Java decides the ordering of objects — and when to use Comparable vs Comparator.
Follow Code Vimaan for the complete Java journey.
LEARN • BUILD • FLY
#Java #CoreJava #JavaProgramming #HashMap #HashSet #JavaCollections #CollectionsFramework #Hashing #DataStructures #JavaDeveloper #LearnJava #JavaInterview #JavaInterviewQuestions #CodingInterview #Programming #Coding #SoftwareDevelopment #SoftwareEngineer #BackendDevelopment #TechEducation #Developer #ProgrammingTips #CodeVimaan #30DaysOfJava
30 DAYS OF JAVA —
.
9 pages

MEDIA advertisement; document/carousel (possible)

#164 Alok Sharan

System Design

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author
1mo · **View on LinkedIn**

The tools change every few years.
The architecture questions barely do.
After 20+ years across enterprise architecture, cloud, APIs, data, and distributed systems, I have seen the same system design fundamentals show up again and again, just at a different scale.
Before I think about Kafka, Redis, Kubernetes, or a database, I want clarity on the basics:
What are we building?
Who is using it?
How much traffic are we expecting?
Where does the data live?

What happens when a dependency fails?

What latency can the business tolerate?

That is why I like looking at system design through these 10 pillars:

→ Features: requirements, constraints, consistency, trade-offs

→ Users: traffic patterns, growth, devices, peak usage

→ Data Model: SQL, NoSQL, indexing, replication, sharding

→ Geography & Latency: regions, CDNs, RTT, network distance

→ Server Capacity: CPU, memory, storage, concurrency, QPS

→ Availability: redundancy, failover, retries, circuit breakers

→ APIs & Security: protocols, identity, encryption, rate limits

→ Caching: hit rates, eviction policies, hashing, sessions

→ Proxies: routing, load balancing, Layer 4 vs Layer 7

→ Messaging: queues, pub/sub, ordering, polling, streaming

The interesting part is how connected these decisions are.

A caching decision changes database load.

A consistency decision affects latency.

A multi-region design changes availability, cost, and data architecture.

A queue can improve resilience while introducing new operational complexity.

That is where architecture experience starts to matter.

System design is less about finding the “perfect” component and more about understanding the consequences of every decision around it.

P.S. Preparing for a software engineering role? Check out my book, System Design and Behavioral Intelligence.

It will help you:

• Build strong System Design foundations and approach design problems with confidence

• Develop practical frameworks to navigate complex behavioral and leadership challenges

Get Your Copy anywhere internationally:

US: https://lnkd.in/eVy2ACxQ

UK: https://lnkd.in/evzmHKm4

India : https://lnkd.in/e25PthqU

This is only a high-level working diagram, but it is a useful mental checklist whenever you start designing a new system.

LINKS

- https://lnkd.in/eVy2ACxQ
- https://lnkd.in/evzmHKm4
- https://lnkd.in/e25PthqU

#165 Spring Users

Java / Spring

Suraj Mishra • 2nd

1mo · **View on LinkedIn**

Senior Spring Boot Interview Question: Debug a Concurrent Subscription API

Context

You are building a Spring Boot service that manages user subscriptions.

A user can subscribe to a plan, change their plan, pause their subscription, resume it, or cancel it.

The API exposes:

“””

POST /api/v1/users/{userId}/subscriptions
PATCH /api/v1/subscriptions/{subscriptionId}
POST /api/v1/subscriptions/{subscriptionId}/pause
POST /api/v1/subscriptions/{subscriptionId}/resume
POST /api/v1/subscriptions/{subscriptionId}/cancel
“””
A subscription looks like this:
“””
@Entity
public class Subscription {
@Id
@GeneratedValue
private Long id;
private Long userId;
private Long planId;
@Enumerated(EnumType.STRING)
private SubscriptionStatus status;
private Instant startDate;
private Instant endDate;
// getters/setters
}
“””
The possible states are:
“””
public enum SubscriptionStatus {
ACTIVE,
PAUSED,
CANCELLED
}
“””
The service was implemented as attached.
The application works correctly during normal testing.
However, production has started showing inconsistent subscription states.
Your Task
You are reviewing this code as the senior engineer responsible for the service.
Identify the problems and redesign the implementation.
Identify all correctness problems you can find.
Consider:
- HTTP status codes
- exception handling
- validation
- transaction boundaries
- duplicate subscriptions
- concurrent requests
- state transitions
- persistence behavior

- database constraints
- multi-instance deployment

Detailed solution with an explanation:

https://lnkd.in/gVuvkfGj

#java #spring #springboot #interview

LINKS

- https://lnkd.in/gVuvkfGj

#166 Samson O. Sanyaolu

System Design

Business Systems Manager | Software Engineer | Driving Digital Transformation & Business Growth Through Enterprise Technology & AI

1mo · **View on LinkedIn**

Latency vs Throughput vs Bandwidth

Ever wondered why your app feels slow even when the bandwidth looks fine? Latency, throughput, and bandwidth often get used interchangeably, but each one tells a different story about performance.

✓ Latency is the delay. How long it takes for a single packet to travel from sender to receiver. If your ping shows 40 ms round-trip, that's latency.

Throughput is the actual delivery rate. How much data is successfully transferred per second. If your download shows 62 Mbps, that’s throughput.

✓ Bandwidth is the maximum capacity of the link. For example, a 100 Mbps connection is the upper limit under ideal conditions.

✓ Throughput is always less than bandwidth. Network congestion, packet loss, and protocol overhead all affect throughput, which is why you never actually hit the maximum bandwidth capacity in practice.

Similarly, low latency doesn't always mean high throughput. Small payloads, single connections, and tight window sizes can all keep throughput low, which is why fast responses don't guarantee you're sending a lot of data.

Another way to understand these three concepts: Bandwidth is the highway width. Throughput is the traffic flow. Latency is how long it takes a car to go from A to B.

All three matter, but they solve different problems.

Over to you: How do you measure these metrics in a way that actually predicts when things will break?

✓ Learn programming at w3schools.com and JavaScript Mastery

✓ Follow Samson O. Sanyaolu for more daily insights

✓ And Join this coding community https://lnkd.in/eNmtPNQV

Image Cc: bytebytego

#programming #softwareengineering #WebDevelopment #AI #FrontendDevelopment #sosamsony #BackendDevelopment #FullStackDeveloper #WebAppDevelopment #systemdesign #coding #interviewtips

MEDIA Latency Vs Throughput vs Bandwidth

LINKS

- https://lnkd.in/eNmtPNQV

#167 Java Developers Community (moderated)

Java / Spring

Vasant Bhosale • 3rd+

1mo · **View on LinkedIn**

Do you think you’re ready for a Java 8 Streams interview? Let’s start with these 5 questions.

From my experience, these are the kinds of Java 8 Streams coding questions you should be prepared for.

Interviewers may ask different problems, but the core concepts are often the same.

Top 5 to practice:

1. Find duplicate elements using Streams
2. Find the top 3 highest-paid employees
3. Group employees by department
4. Flatten nested lists using flatMap()
5. Find the longest palindromic substring

I compiled 100+ Java coding questions from Easy → Medium → Hard covering Streams, Collectors, Optional, Concurrency & CompletableFuture.

PDF includes solutions.

Save it for your next interview.

Share it with other Java developers preparing for interviews.

Which one would you struggle with?

#Java #JavaDeveloper #Java8 #JavaStreams #CodingInterview #InterviewPreparation

45 pages

MEDIA document/carousel (possible)

#168 Yann Pascal DEUNGOUE

AI / LLM

Tech Lead-Front Architect for Generali

1mo · **View on LinkedIn**

Calling an LLM from Java is easy. Making AI a real part of your enterprise architecture is the interesting part.

For a long time, adding AI to a Java platform often meant adding another ecosystem beside it:

Python service

Additional APIs

Another deployment

Another runtime to monitor

Another stack to maintain

Spring AI changes that equation.

And no — Spring AI isn’t just ChatClient.

It gives Spring Boot applications a coherent way to integrate several AI capabilities:

ChatClient — interact with different LLM providers through a consistent API.

Tool Calling — let models execute controlled application capabilities instead of only generating text.

MCP — consume or expose tools and resources through the Model Context Protocol, creating a standardized interface between agents and enterprise systems.

RAG — retrieve enterprise knowledge from vector stores and inject relevant context into prompts.

Advisors & Memory — enrich interactions with contextual behavior and conversational state.

Structured Output — map model responses back into typed Java objects.

And this is where things become architecturally interesting.

Your AI layer can now sit naturally on top of capabilities you already have:

LLM / Agent

↓

Spring AI

↓
ChatClient · MCP · Tools · RAG
↓
Domain Services
↓
REST · Kafka · Databases · Enterprise APIs
All while keeping the engineering ecosystem your teams already understand:
Spring Boot
Spring Security
Micrometer / OpenTelemetry
Testing
CI/CD
Kubernetes
And what about LangChain4j?
It remains a strong Java AI framework, particularly when you want more explicit building blocks for sophisticated AI-centric applications.
But for an existing Spring Boot platform, Spring AI has one enormous architectural advantage: Spring itself.
Your AI capabilities become part of your existing application model rather than another architecture beside it.
That distinction matters.
Because the hard problem in enterprise AI isn't:
✗ "How do I send a prompt to GPT?"
It's:
Which tools may an agent execute?
On behalf of which user?
How much does each execution cost?
Which enterprise data can enter the context?
How do we trace Prompt → Model → MCP → Tool → Domain → Response?
How do we stop an agent from performing an unauthorized action?
The future of enterprise AI isn't another chatbot.
It's existing enterprise capabilities becoming safely accessible to models and agents.
And for Java teams, Spring AI + MCP makes that architecture particularly interesting.
↓ Spring AI or LangChain4j: what are you using for your Java AI projects — and why?
#Java #SpringAI #SpringBoot #MCP #LangChain4j #AgenticAI #GenerativeAI #LLM #EnterpriseAI #SoftwareArchitecture

#169 Kanishk Raj System Design

Senior Software Engineer @ Principal | Java/Spring Boot | Distributed Systems & Microservices Architecture | Scaling High-Throughput Backend Systems

1mo · **View on LinkedIn**

While going through some basics of designing scalable applications, I came across something pretty interesting.
You solve one problem... and suddenly another one shows up.
A single database starts struggling → introduce caching.
Read traffic keeps growing → add read replicas.

Then data size and write traffic grow → sharding enters the picture.

And then even something as simple as choosing the right database index can make a big difference.

The more I go through these concepts, the more I realize that System Design is less about knowing a list of technologies and more about understanding where the bottleneck is, what trade-off you're making, and what happens when the system grows 10x.

I ended up putting my thoughts and the whole progression into a story-driven write-up on Medium ↓

https://lnkd.in/dpDm35nc

#SystemDesign #SoftwareEngineering #BackendEngineering #DistributedSystems #Scalability #Database

MEDIA advertisement

LINKS

- https://lnkd.in/dpDm35nc

#170 Shankar Thakur

Java / Spring

Java Full Stack Developer | Java | Spring Boot | React.js | REST APIs | SQL | DSA

1mo · **View on LinkedIn**

Java 8 → 17 → 21 → 25

How Java evolved — and why it matters for backend developers

Java has evolved significantly over the years, but one thing has remained constant: its importance in building reliable, scalable, and maintainable applications.

A quick evolution:

Java 8 — Try-with-resources, Multi-catch, String in switch, improved I/O

Java 17 — Records, Sealed Classes, Pattern Matching, modern Java features

Java 21 — Virtual Threads, Record Patterns, Pattern Matching

Java 25 — The latest LTS generation with continued improvements in language, performance, security, and developer productivity

For backend developers, learning Java isn't simply about moving to a newer version.

It's about understanding how the language is evolving, writing cleaner code, improving scalability, modernizing applications, and building software that is easier to maintain.

As Java continues to evolve, the real question is:

Are we just using Java, or are we keeping up with how Java is evolving?

Which Java version are you currently learning or working with — 8, 11, 17, 21, or 25?

#Java #JavaDeveloper #BackendDevelopment #SpringBoot #JavaProgramming #SoftwareDevelopment #Programming #EnterpriseJava #Microservices #TechCareers

#171 Vinay Tiwari

Java / Spring

Software Engineer at Wipro | Java | Spring Boot | Microservices | REST APIs | Kafka | MySQL | AWS | Docker | Kubernetes | DevOps | GenAI

1mo · **View on LinkedIn**

JAVA 8 → 11 → 17 → 21 → 25

How Java Evolved Across Its Major LTS Releases

If you’re a Java developer, knowing how Java evolved is just as important as knowing Java syntax.

Here’s the journey

Java 8 — The Functional Revolution

Java 8 changed everyday coding with Lambda Expressions, Stream API, Functional Interfaces, Optional, Method References, Default/Static Interface Methods, and the Date & Time API.

Result: More concise, expressive and functional code.

Java 11 — Cleaner APIs & Better Productivity

Java 11 brought the standard HTTP Client, useful String methods like isBlank(), strip(), lines() and repeat(), plus var in Lambda parameters and JVM improvements.

Result: Cleaner APIs and less boilerplate.

Java 17 — Modern Language Design

Java 17 made Sealed Classes permanent and continued improvements around encapsulation, pattern matching, random number generation, security and JVM performance.

Result: Better type design and maintainable code.

Java 21 — A Major Concurrency Upgrade

Java 21 made Virtual Threads, Record Patterns and Pattern Matching for switch permanent and introduced Sequenced Collections and Generational ZGC.

Result: Simpler concurrency and highly scalable backend applications.

Java 25 — The Latest LTS

Java 25 brings Module Import Declarations, Compact Source Files & Instance main Methods, Flexible Constructor Bodies, along with continued improvements in Scoped Values, JVM, GC and developer tooling.

Result: Simpler code and continued platform evolution.

The Java Evolution

Java 8 → Functional Programming

Java 11 → Modern APIs

Java 17 → Modern Language Design

Java 21 → Scalable Concurrency

Java 25 → Simpler & Modern Java

The real lesson:

Java didn’t become “better” through one big release.

It evolved step by step — making code cleaner, safer, more scalable and easier to maintain.

Which Java LTS version are you working with today?

#Java #JavaDeveloper #Java8 #Java11 #Java17 #Java21 #Java25 #SpringBoot #BackendDevelopment #SoftwareEngineering #Programming

#JavaProgramming #SpringBoot #BackendDevelopment #SoftwareEngineering #Programming #Coding #Developers #TechCommunity #Technology #LTS #Trending

#Java #JavaDeveloper #SpringBoot #BackendDevelopment #SoftwareEngineering

#172 Raul Junco System Design

Simplifying System Design

1mo · **View on LinkedIn**

Building distributed systems taught me one thing the hard way:

Every step forward needs a way back.

In a monolith, a failed transaction rolls back with a single command. In a distributed system, five services may have already committed before you realize something went wrong.

That's why you need compensating transactions.

Here's the idea: for every forward step in your workflow, you design a step that can undo it.

Take a simple order flow:

Forward Transactions:

• Create an order.

• Process the payment.

• Update inventory.

• Ship the order.

Compensating Transactions:

• Cancel the order (undo order creation).

• Reverse the payment (refund the customer).

• Reverse the inventory update (restore stock).

If step 3 fails, you don't just stop there. You trigger compensations in reverse order until the system reaches a consistent state.

One thing most engineers miss: compensating transactions aren't rollbacks.

They're new forward transactions that reverse a previous effect. That means they must be idempotent, safe to run more than once if retries happen.

Next hard question: have you ever had a compensating transaction fail mid-way?

#173 Pradeep Pai System Design

SDE-3 | Writing about software engineering and career growth

1mo · **View on LinkedIn**

Shopify replaced Redis with MySQL for one of their most critical checkout workloads. The trick was surprisingly simple.

Every time a buyer clicks "Complete purchase," Shopify has milliseconds to answer one question: is that last unit still available?

If Shopify says yes when the item is already gone, two people could buy the same item, leading to a cancelled order, an apology email, and a loss of trust.

If they say no when the item is actually available, they lose a genuine sale.

For years, Shopify used Redis to handle inventory reservations during checkout. When a customer started buying an item, Shopify would reserve that item in Redis so another customer couldn't buy the same unit at the same time.

But the actual inventory lived in MySQL, so reservations and inventory updates happened in two different systems with no shared transaction. A payment could succeed while the inventory update failed, causing an oversell, or the inventory could be deducted while the Redis reservation remained, causing an undersell.

The obvious fix was to move reservations into MySQL too, so everything could happen in one atomic transaction.

But MySQL had already failed at this once. Their first attempt used a single row with a quantity column, and at checkout scale, every buyer fought over that one row. Locking it, waiting, retrying. It buckled.

This is where the magic comes in.

Instead of using one row per item with its available quantity, they used one row for each individual unit. So an item with 10 units gets 10 rows. To reserve three units, they simply grab any three available rows.

And then the key: MySQL 8's SKIP LOCKED.

Normally, if a query wants rows another transaction has locked, it waits, everyone queues behind the same rows, and throughput collapses. SKIP LOCKED flips that: if some rows are locked, MySQL simply skips them and hands back other free ones. No waiting. No queue on a hot row.

So when 500 buyers check out the same product at once, they're not fighting over one row, they each quietly grab a different free unit and move on. Contention, which killed the naive design, basically disappears.

To keep it fast at huge scale, they cap the pool at ~1,000 rows per item and refill it from the ledger as it drains. Same idea, bounded.

That's the whole trick: turn "everyone fights over one number" into "everyone grabs a different row." A single database feature made MySQL viable for a workload people assumed needed specialized infrastructure.

The boring database you already run is often more capable than you think. MySQL didn’t just handle Shopify’s scale, it also made their system much simpler.

Found this on NayaRead (my feed for engineering deep dives), link in the first comment.

#174 Manu Agrawal System Design

AI & Tech Creator | Ex-Google, Microsoft | AI Engineer | 143K+ LinkedIn | Tech Speaker | 50 Millions+ Impressions | View my portfolio

1mo · **View on LinkedIn**

Give me 2 minutes and I’ll show you how to use Claude to master System Design for a ₹70 LPA+ FAANG role.

Here’s the framework:

1. Build your “Architecture DNA”

Take 20 well-known systems:

Instagram

Uber

Netflix

WhatsApp

YouTube

Google Search

Dropbox

Payment systems

Notification systems

RAG systems

Don’t ask Claude for their solutions.

Ask it to extract:

Problem → Constraint → Decision → Alternative → Trade-off → Failure Mode

You stop memorizing:

“Instagram uses X.”

You start recognizing:

“When this constraint appears, this architecture emerges.”

That’s pattern recognition.

2. Generate problems you’ve NEVER seen

Give Claude your system list and ask:

“Generate 50 new system-design problems by combining constraints from these systems. Avoid obvious clones. Every problem must contain at least 2 conflicting requirements.”

Now you get problems like:

“Design a food-delivery platform where driver locations update every 2 seconds, but drivers can lose connectivity for 5 minutes.”

Or:

“Design a social platform where a celebrity can generate 10M feed events while maintaining sub-200ms feed latency.”

You're no longer preparing for questions.

You're preparing for unknown questions.

3. Play the “Constraint Mutation” game

Take ONE design.

Then change one constraint at a time:

10M → 100M users

Normal → 10× traffic spike

99.9% → 99.99% availability

Single-region → Multi-region

Eventual → Strong consistency

Normal budget → 50% lower budget

Same product.

Completely different architecture.

This teaches the most important lesson:

Architecture is a response to constraints.

4. Build your “Trade-off Library”

After every design, ask Claude:

“Extract every architectural decision I made. For each, tell me what I chose, what I rejected, why, what I gained, what I sacrificed, and when this decision would stop working.”

Do this for:

Kafka vs SQS

SQL vs NoSQL

Redis vs DB

Polling vs WebSockets

Fan-out on write vs read

Strong vs eventual consistency

Sync vs async

Active-active vs active-passive

Eventually, you stop memorizing what to use.

You understand why to use it.

5. Make Claude destroy your architecture

After finishing a design, don't ask:

“Is this correct?”

Ask:

“Destroy my architecture. Find the first bottleneck, consistency bug, expensive component, cascading failure, operational nightmare and scaling wall. Don’t fix anything.”

Then fix those problems yourself.

That's architecture adversarial training.

6. Run the Staff Engineer Test

Finally ask:

“Evaluate my reasoning, not my diagram. Identify assumptions I didn't validate, trade-offs I failed to explain, unnecessary components, ignored failure modes and premature optimizations.”

Now you're training decision quality, not answer memorization.

#claude #systemdesign #interview #hiring #jobs #techjobs #job #faangjobs

LINKS

- https://x.com/themanuagrawal

#175 Gaurav Gupta

Java / Spring

Senior Software Engineer @ Atlassian | Xoogler
1mo · View on LinkedIn

Java 8 vs Java 17 vs Java 21 vs Java 25: The Journey of Modern Java

Java has evolved significantly over the last decade — transforming from a traditional enterprise programming language into a modern platform for building scalable, cloud-native, high-performance, and developer-friendly applications.

#Java #Java8 #Java17 #Java21 #Java25 #Programming #SpringBoot #BackendDevelopment #SoftwareDevelopment #Coding #Developers #JVM #CloudNative #AI

#176 Dnyaneshwar Ghodake

Java / Spring

Senior Java Backend Developer | 9+ Years | Java, Spring Boot, Spring Batch | Core Banking & Financial Systems | High-Volume Data Processing | REST APIs | Kafka | Docker | Kubernetes | AWS | Immediate Joiner
1mo · View on LinkedIn

AtomicInteger vs synchronized — What should you use?

In the previous post, we saw that "volatile" provides visibility — but does not make "count++" atomic.

So what do we use instead?

AtomicInteger

AtomicInteger count = new AtomicInteger();

count.incrementAndGet();

Uses atomic operations (CAS internally) without explicit locking.

Best for:

• Counters and sequences
• Simple atomic updates
• High-throughput shared state

Synchronized

synchronized (lock) {

balance -= amount;

}

Uses a monitor lock. Only one thread can execute the protected critical section at a time.

Best for:

• Multiple operations that must execute together
• Complex business logic
• Protecting invariants across multiple variables/objects

The key difference

AtomicInteger → “Make this operation atomic.”

synchronized → “Make this entire critical section atomic.”

For example, a withdrawal involves:

"check balance → update balance"

Making only the update atomic isn't enough. The check + update must be protected together.

Bottom line

Use AtomicInteger for simple atomic state updates.

Use synchronized when multiple operations must behave as one indivisible unit.

The right concurrency primitive depends on the problem — not simply on which one is “faster”.

#Java #CoreJava #Concurrency #Multithreading #JVM #SpringBoot #BackendEngineering #JavaDeveloper #Performance #SystemDesign

MEDIA advertisement

#177 Java Developers Community (moderated)

Java / Spring

Lucas Amorim • 3rd+ | RAG with Java: how to give your LLM "external memory" | One of the most common problems when using LLMs in production is hallucination: the model "makes up" answers when it doesn't have enough context about your company's data. | That's where RAG (Retrieval-Augmented Generation) comes in: instead of relying only on the model's pre-trained knowledge, you search for relevant information in your own knowledge base (embeddings + vector database) and inject that context into the prompt before generating the response. | In practice, the flow is simple: | 1. Documents are split into pieces (chunks) and turned into embeddings | 2. The embeddings are stored in a vector database (e.g. PGVector, Redis, Chroma) | 3. When the user asks a question, we retrieve the most relevant chunks by similarity | 4. Those chunks become extra context in the prompt sent to the LLM | AiServices handles the "tedious" part: retrieving the relevant chunks, building the augmented prompt, and calling the model. You just define the interface and inject the retriever. | #Java #RAG #LangChain4j #AI #SpringBoot #LLM #SoftwareEngineering | 63 | 1 | 1 | | Dorian ROSSE, Open to work 3rd+ | Dorian ROSSE • 3rd+ | A étudié à : Suzanne Masson

1mo · View on LinkedIn

Thanks for Java

MEDIA advertisement; document/carousel (possible)

#178 Shalini Goyal

System Design

Executive Director, AI & Engineering @ JPMorgan | Amazon Alum | Author · Speaker · Professor | Helping Engineers Break into AI & High-Impact Careers

1mo · View on LinkedIn

Your app works.

Then real traffic arrives.

Pages slow down. Requests fail. Databases struggle. Services stop communicating. And suddenly, choosing a modern framework does not feel like system design anymore.

Because a reliable application is built across multiple connected layers.

The frontend delivers the user experience.

API gateways and reverse proxies control how traffic enters, where requests go, and which services users can access.

Load balancers distribute demand so one server does not become the breaking point.

Backend services run the core business logic.

Then comes the data foundation:

→ Relational databases process structured transactions

→ NoSQL databases handle flexible, distributed workloads

→ Caches reduce latency and repeated database queries

→ Message brokers keep services loosely connected

→ Search engines make large datasets discoverable

Storage platforms manage files, media, and backups, while content delivery networks serve content closer to users.

Containers make applications consistent across environments.

Orchestration platforms deploy them, scale them, restart failed instances, and manage computing resources.

But here is the layer teams appreciate only after something breaks: observability.

Metrics reveal unusual behavior.

Logs explain what happened.

Traces follow a request across multiple services and expose exactly where it failed.

A strong system is not the one using the highest number of technologies.

It is the one that stays fast, available, secure, and cost-efficient when real users, failures, and traffic spikes arrive together.

Learn why every layer exists before choosing the tools inside it.

Your architecture will be better for it.

BTW, pre-order for the book is now open!

Get your copy of System Design and Behavioral Intelligence here:

US: https://lnkd.in/eVy2ACxQ

UK: https://lnkd.in/eBC_SNdb

India : https://lnkd.in/e25PthqU

Which layer would fail first in the system you are building today?

LINKS

- https://lnkd.in/eVy2ACxQ
- https://lnkd.in/eBC_SNdb
- https://lnkd.in/e25PthqU

#179 Mithra Gopal

System Design

Senior Lead Software Engineer at Chargebee

1mo · **View on LinkedIn**

PAYMENTS ENGINEERING, EXPLAINED · #02

Same webhook twice ≠ process twice.

Payment providers can retry webhook delivery.

So this assumption is dangerous:

1 payment event

→ 1 webhook delivery

→ 1 execution

Imagine a payment-success webhook triggers:

• Order fulfilment

• Subscription activation

• Customer notification

• Ledger updates

If the same event arrives again, those business actions should not happen again.

That is why webhook handlers need to be idempotent.

A common approach is to identify the provider event and ensure that only one processing path can claim it.

But there is an important concurrency trap:

Check event

→ Not found

→ Process business logic

→ Save event ID

Two workers can perform that check at the same time, both see “not processed”, and both execute the business logic.

The deduplication itself needs to be atomic.

There is another useful distinction:

API idempotency protects

Application → Payment Provider

Webhook idempotency protects

Payment Provider → Application

And idempotency solves only one part of webhook reliability.

Events can also arrive out of order.

Reliable payment systems therefore need to design for retries, duplicates, concurrency and ordering — instead of assuming every event will arrive exactly once and in sequence.

The takeaway:

Design webhook consumers so repeated delivery does not repeat the business outcome.

#PaymentsEngineering #Payments #Fintech #SoftwareEngineering

MEDIA Diagram showing the same payment webhook being delivered more than once, passing through an atomic idempotency gate so the associated business action is processed only once.; advertisement

#180 Muhammad Mubbashir

System Design

Software Engineer at Paysyslabs | Ex-iPath | Java 8 | JEE | Spring | Spring Boot | Hibernate | Microservices | Spring Security | Spring Cloud | Docker | FinTech | Angular | Jenkins | CICD

1mo · **View on LinkedIn**

In fintech, processing a payment is easy. Processing it exactly once is the real challenge.

Duplicate transactions can occur for many reasons:

User retries the payment

Network or gateway timeouts

Multiple webhook callbacks

Message queue redelivery

High-concurrency requests

Without proper controls, a single payment request can result in multiple debits, reconciliation issues, customer complaints, and financial losses.

The key to preventing duplicate transactions is Idempotency.

A robust payment system should:

✓ Generate a unique Idempotency Key for every request

✓ Enforce database-level uniqueness constraints

✓ Validate transaction status before processing

✓ Handle retries safely without creating new transactions

✓ Deduplicate messages and webhooks

✓ Continuously monitor and reconcile transactions

In payment systems, success isn't measured by how many transactions you process per second—it's measured by how reliably you ensure that every transaction is processed once and only once.

Building trust starts with preventing duplicates before they happen.

#Fintech #Payments #Java #SpringBoot #SystemDesign #DistributedSystems #BackendEngineering #PaymentGateway #Idempotency #SoftwareEngineering

MEDIA advertisement

#181 Spring Users

Other

Sibin M. • 3rd+

1mo · **View on LinkedIn**

(no post text captured)

MEDIA advertisement

#182 Yogesh Kumbhare

System Design

Software Engineer III @J.P. Morgan Chase & Co. | Java Full Stack Developer | Generative AI & AI Agent | 2xAWS Certified | Scalable Cloud Systems

1mo · View on LinkedIn

Payment Gateway Design

A payment gateway is not just an API integration; it is a critical distributed system where failures can directly create financial and customer trust problems.

In enterprise software, the design must handle secure transaction processing, unreliable external providers, duplicate requests, asynchronous confirmations, and strict audit requirements.

The real challenge is ensuring that every payment reaches one clear business state even when networks, clients, or gateway services fail.

In production, I have seen timeout scenarios where a customer retried a payment while the gateway had already processed the original request, creating a serious risk of duplicate charges.

A common developer mistake is treating a timeout as a failed payment instead of moving the transaction into a pending state and verifying its final status through provider callbacks or reconciliation.

The recommended approach is to use idempotency keys, persist payment state transitions, process webhooks asynchronously, and separate the payment orchestration layer from individual gateway adapters.

Security also requires tokenization, strong access controls, signed callbacks, and complete audit trails without storing sensitive card data unnecessarily.

The practical lesson is to design payment processing around state consistency and recovery rather than assuming every request receives an immediate answer.

This approach improves maintainability, supports multiple providers, and makes the system easier to scale and operate over time.

Build idempotency and reconciliation into the first version of the architecture, and how does your team handle payment states when the gateway response is uncertain?

#PaymentGateway #SystemDesign #SoftwareArchitecture #EnterpriseArchitecture #DistributedSystems #Java #FinTech

MEDIA advertisement

#183 Hassan Zaib Hayat

System Design

Senior Backend Engineer | Python, Django & FastAPI | PostgreSQL, Redis & AWS | Maintainer of django-query-doctor

1mo · View on LinkedIn

A 10 GB upload should not be one HTTP request.

And your application servers should not become a very expensive file pipe.

The naive flow looks harmless:

Client → API → Application → Object storage

But under real traffic:

→ Every byte passes through the application infrastructure

→ Slow clients consume connections and backpressure capacity

→ A network interruption can restart the entire transfer

→ Retried completion requests can duplicate processing

→ Uploaded but unverified files may become publicly accessible

The architecture I’d use separates coordination from transfer.

Create an upload session

The API authenticates the user, validates limits, generates a server-owned object key, and returns an upload ID.

Upload directly to object storage

The client splits the file into parts and uploads them using short-lived presigned URLs.

Application servers never proxy the file bytes.

Persist resumable progress

For every verified part, store:

→ Part number

→ Size

→ Checksum

→ Storage ETag

After a disconnect, the client requests the server-side manifest and uploads only the missing parts.

Make completion idempotent

The completion API uses an idempotency key and verifies the exact part manifest against storage.

Retries return the same finalization operation instead of creating another workflow.

Treat integrity as a server decision

Client progress is advisory.

Storage state, recorded checksums, expected size, and the server manifest determine whether the upload is complete.

Separate “uploaded” from “ready”

My state machine would look like:

INITIATED → UPLOADING → COMPLETING → PROCESSING → READY

After completion, the object remains quarantined while asynchronous workers perform:

→ Malware scanning

→ File-type verification

→ Metadata extraction

→ Archive and decompression-bomb checks

Only a verified object can update the public asset pointer.

Clean up abandoned work

Expired sessions should abort incomplete multipart uploads and remove stale metadata.

Otherwise, unfinished parts quietly become a storage-cost leak.

The important distinction:

Successful byte transfer does not mean the asset is safe to publish.

My default when scanning is unavailable:

Keep the upload in PROCESSING, retry asynchronously, and never publish the unscanned object.

How long would you keep it quarantined before marking the upload as failed?

Post 3 of Systems Under Load.

Next: Designing a notification service with preferences, fan-out, retries, provider failover, and deduplication.

#SystemDesign #BackendEngineering #DistributedSystems #CloudArchitecture

Resumable Uploads

.

10 pages

MEDIA advertisement; document/carousel (possible)

#184 Java Developers Community (moderated)

Kafka / Messaging

Pavan Deshpande • 3rd+

1mo · View on LinkedIn

You think your Kafka publish is safe inside @Transactional.

It's not.

@Transactional

public Order placeOrder(...) {

orderRepository.save(order); // 1 DB write

kafkaTemplate.send("order.placed"); // 2 Kafka send

}

Two failure modes that WILL hit you in production:

DB commits → JVM crashes → Kafka never gets the event

Kafka send fails → Order saved → Event silently lost

The Transactional Outbox Pattern solves this permanently.

Write to DB + outbox table atomically.

Let a scheduler relay to Kafka independently.

Crash between the two? Scheduler retries on restart.

Never lose a Kafka message again.

Full implementation on my blog — link in comments ↓

#Kafka #SpringBoot #Java #Microservices #BackendEngineering #DistributedSystems #CloudNative #SystemDesign #SoftwareArchitecture #SoftwareEngineering

LINKS

- http://orderRepository.save

#185 Manoj Vurukonda

System Design

Technical Lead Java Full Stack | Reactive Programming and Cloud-Native Microservices Architecture

1mo · View on LinkedIn

Why 'synchronized' keyword cannot prevent duplicate payments? How production systems prevents duplicate payments in banking applications?

- Idempotency Keys
- @Transactional
- Database Row Locking
- Lock Ordering
- Kafka Events

The synchronized keyword in Java operates exclusively at the single-JVM level. In modern cloud-native banking, payment services run across multiple instances or pods (e.g., Pod A, Pod B, Pod C). Because synchronized locks an object on a local heap monitor, a request routed to Pod A and a duplicate request routed to Pod B will both execute concurrently without knowing the other exists, resulting in double debits.

To safely prevent duplicate payments, production systems combine distributed mechanics and database controls:

1. Idempotency Keys

Clients attach a unique header (clientRequestId or UUID) to every payment attempt. This key is stored in a fast distributed cache (like Redis) or database with a UNIQUE index constraint. If a network retry or double-click triggers a duplicate call, the API edge layer intercepts the key, rejects the execution, and safely returns the existing cached response without re-processing the payment.

2. @Transactional

Spring/JPA @Transactional boundaries group the balance check, debit, and transaction logging operations into a single ACID unit of work. By wrapping these operations within a transaction (e.g., using isolation = REPEATABLE_READ), the system ensures all ledger changes commit together or roll back completely if an error occurs.

3. Database Row Locking

Because distributed application servers cannot share JVM locks, the centralized database acts as the single source of truth. Production systems apply pessimistic locking (SELECT ... FOR UPDATE) or optimistic locking (@Version) on specific account rows. This forces concurrent transactions modifying the same account balance to wait in line at the database level regardless of which app instance initiated the request.

4. Lock Ordering

When a payment involves transferring funds between two accounts (Sender and Receiver), obtaining locks in an arbitrary order across concurrent requests causes deadlocks. Systems enforce deterministic lock ordering by sorting account IDs (e.g., locking min(accountA, accountB) first, then max(accountA, accountB)). This guarantees all instances acquire locks in the exact same sequence.

5. Kafka Events

For asynchronous downstream operations (posting to payment rails, ledger updates, or reconciliation), systems use Kafka events paired with the Transactional Outbox pattern. Event consumers process messages using the idempotency key to prevent duplicate downstream execution, while reconciliation consumers process streams to detect discrepancies and issue compensating auto-reversals.

MEDIA advertisement

#186 Dr. Sandeep Srivastava

Other

Director, Meander Software Private Limited

1mo · **View on LinkedIn**

Still confused about how to validate your startup idea?

You don’t need a huge team or expensive infrastructure to build your first MVP.

Build small. Test fast. Get real users before you spend big.

#StartupValidation #MVP #BuildInPublic #Entrepreneurship #SaaS #Startup #ProductDevelopment #IndieHackers

#187 Shubham Sharma

System Design

SDE-2 @ Dp-World | Ex-Paytm | Distributed Systems • GenAI • Cloud (Azure • AWS) | Scalable APIs

1mo · **View on LinkedIn**

If you want to Become Good at System Design? Start by understanding how these real systems actually works.

I’ve added high-quality YouTube resources to each topic so you can start learning immediately ↓

1. How Stock Exchange Works

Low latency, matching engines, fault tolerance

https://lnkd.in/gCXk-Pti

2. How Payment Systems Work

Transactions, idempotency, consistency, fraud handling

https://lnkd.in/gxmrEKS4

3. How YouTube Works

Video storage, CDN, encoding pipelines, scaling

https://lnkd.in/gWFXwe67

4. How Google Docs Works
Real-time collaboration, CRDTs, sync conflicts
https://lnkd.in/g-iF2XVm
5. How Kafka Works
Event streaming, partitions, brokers, replication
https://lnkd.in/gBiu9ePG
6. How URL Shortener Works
Database design, hashing, scaling
https://lnkd.in/gHmAvbHY
7. How WhatsApp Works
Message queues, delivery guarantees, presence systems
https://lnkd.in/g-iF2XVm
8. How Airbnb Works
Marketplace scaling, booking workflows
(Search in above playlist – Airbnb System Design)
9. How Spotify Works
Music streaming architecture, caching
(Search Spotify system design in same playlist)
How Slack Works
Chat architecture, WebSockets, event-driven systems
(Slack system design in same playlist)
11. How Reddit Works
Feed generation, ranking, caching
(Reddit system design in same playlist)
12. How Uber Finds Nearby Drivers
Geospatial indexing, proximity search
https://lnkd.in/gGcy4s3V
13. How Amazon S3 Works
Distributed storage, replication, durability
https://lnkd.in/gZbQ5Q4M
14. How LLM Systems (like ChatGPT) Work
Inference scaling, model serving, pipelines
https://lnkd.in/gZbQ5Q4M
How to Use This List
Don’t just watch.
1. Pause the video.
2. Draw the architecture yourself.
3. Identify bottlenecks.
4. Suggest improvements.
That’s how you actually improve at system design.
System design isn’t about knowing buzzwords.
It’s about understanding trade-offs.
If you go deep into even 6–7 of these systems, you’ll be ahead of 90% of candidates walking into interviews at companies like Google, Amazon, and Meta.
If this helps, save it for later and follow Shubham Sharma for more.
Big credit to Neo Kim for compiling this powerful list of real-world systems to study.

LINKS

- https://lnkd.in/gCXk-Pti
- https://lnkd.in/gxmrEKS4
- https://lnkd.in/gWFXwe67
- https://lnkd.in/g-iF2XVm
- https://lnkd.in/gBiu9ePG
- https://lnkd.in/gHmAvbHY
- https://lnkd.in/gGcy4s3V
- https://lnkd.in/gZbQ5Q4M

#188 Deepika Sidhanandan

Java / Spring

Architecting the Future of Enterprise Engineering with Java, Cloud & AI

1mo · View on LinkedIn

Your "@Transactional" is NOT broken. Your proxy is.
You added:
@Transactional
public void saveOrder() {
// DB operation
}
...and somehow the transaction doesn't start.
You blame Spring.
✗ “Spring @Transactional is buggy.”
Nope.
Your proxy probably never got a chance to intercept the call.
That’s the part many Java developers miss.

First: What does Spring actually do?
Spring AOP usually doesn't modify your original class directly.
Instead, Spring creates a proxy around your bean:
Caller
↓
Spring Proxy
↓
@Transactional / @Async / @Cacheable
↓
Target Bean
The proxy intercepts the method call and applies the required behavior.

(!) And here comes the classic trap
@Service
public class OrderService {
public void createOrder() {
saveOrder(); // ✗ self-invocation
}
@Transactional
public void saveOrder() {
// DB operation
}

}
You call:
orderService.createOrder();
The call enters through the proxy.
But then...
"createOrder()" → "this.saveOrder()"
The second call happens inside the target object itself.
The proxy is bypassed.
Therefore, the "@Transactional" interception doesn't happen the way you expect.
JDK Proxy vs CGLIB
Here's another interview favorite:
“If my class implements an interface, which proxy does Spring use?”
If interface-based proxying is enabled:
Interface
↑
JDK Proxy
↓
Concrete Class
→ JDK Dynamic Proxy
If class-based proxying is requested:
Concrete Class
↑
CGLIB Proxy
→ CGLIB
One more thing people get wrong
Does interface inheritance force CGLIB?
NO.
If Spring is using interface-based proxying, it can still create a JDK Dynamic Proxy.
The important question isn't:
«“Does the interface extend another interface?”»
The important question is:
«“Is Spring proxying through an interface or through the concrete class?”»
The mental model worth remembering
JDK Proxy
→ Interface-based
→ Proxy implements interfaces
→ Great for interface-oriented design
CGLIB
→ Class-based
→ Proxy subclasses the target
→ Useful when you need class-based proxying
And both have limitations.
"final class" ✗ CGLIB can't subclass it.
"final method" ✗ CGLIB can't override/intercept it.

The uncomfortable truth

When "@Transactional", "@Async", or "@Cacheable" “doesn't work”...
Don't immediately blame the annotation.
Ask:
1. Am I calling the method through the Spring proxy?
2. Is this a self-invocation?
3. Which proxy type is Spring creating?
4. Is the method/class "final"?
5. Am I injecting the Spring-managed bean or creating it with "new"?
Understanding Spring Proxies turns a lot of “Spring magic” into simple Java mechanics.
Your annotation isn't magic.
Your proxy is doing the work.
↓ What's the most confusing Spring AOP/proxy issue you've faced in production?
#Java #SpringBoot #SpringAOP #SpringFramework #JavaDeveloper #BackendDevelopment #Microservices #SoftwareArchitecture #SystemDesign #TechInterview

MEDIA advertisement

#189 Neo Kim System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1mo · **View on LinkedIn**

If I had to design a system,
here are 40 tradeoffs I'd consider:
1 Latency vs Throughput
2 Consistency vs Availability
3 Strong vs Eventual Consistency
4 Vertical vs Horizontal Scaling
5 SQL vs NoSQL
6 Normalization vs Denormalization
7 Replication vs Sharding
8 Sync vs Async Replication
9 Range vs Hash Partitioning
10 B-Tree vs LSM Tree
11 Local vs Shared Cache
12 Cache Freshness vs Hit Rate
13 Indexes vs Write Speed
14 Primary vs Replica Reads
15 Serializable vs Weaker Isolation
16 Optimistic vs Pessimistic Locking
17 Last-Write-Wins vs Merging
18 Monolith vs Microservices
19 Stateful vs Stateless
20 Shared Database vs Database Per Service
21 Single Model vs CQRS
22 Synchronous vs Asynchronous
23 Queue vs Event Log
24 Choreography vs Orchestration

25 2PC vs Saga
26 At-Least-Once vs Exactly-Once
27 Total vs Partitioned Ordering
28 Push vs Pull
29 Batch vs Stream Processing
30 REST vs GraphQL
31 Unary vs Streaming RPC
32 Sync vs Async APIs
33 Compression vs CPU Usage
34 Short vs Long Timeouts
35 Retries vs Load Amplification
36 Idempotency vs Simplicity
37 Deep Queues vs Backpressure
38 Load Shedding vs Request Completion
39 Active-Active vs Active-Passive
40 Single vs Multi-Region
Learn which tradeoffs your workload can afford,
not just know that they exist...
What else should make this list?
===
PS - Want a detailed breakdown of these tradeoffs?
Read right now:
→ https://lnkd.in/eAYRDhUy
===
Repost to help others become good at system design.
Follow Neo Kim + turn on notifications.

MEDIA 40 tradeoffs in system design

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/eAYRDhUy

#190 shujat khan

Java / Spring

Senior Technical Lead at Network People Services Technologies Ltd. (NPST- Banking and Payment Solutions)
1mo · **View on LinkedIn**

Yesterday I appeared for an interview with Accenture for a Java Developer (4+ YOE) role, and here are the questions I faced...
Honestly, it felt more like a discussion on how I think as a developer rather than just a Q&A session.
Sharing them here, maybe it will help someone preparing ↓
Core Java / Advanced
1. Explain the difference between Comparable and Comparator. Which one is more flexible in real projects?
2. How does Garbage Collector decide which objects to clean? What is the difference between Minor GC & Major GC?
3. Why String is immutable in Java? What benefits does it give in multi-threaded environments?
4. How does ClassLoader work in Java? Can we have multiple classloaders in one JVM?
5. Difference between StringBuffer, StringBuilder, and normal String. Which one to use where?

Collections & Performance

6. How does CopyOnWriteArrayList work internally? When would you prefer it?

7. What is the difference between fail-fast and fail-safe iterators?

8. If you have millions of records to process, would you use Stream API or traditional loops? Why?

9. How does TreeMap maintain ordering? Can we provide custom sorting logic?

10. How do you find duplicates in a list using Java 8 features?

Spring & Spring Boot

11. What is the difference between @Transactional at class level vs method level?

12. Can you explain how Spring Boot auto-configuration works internally?

13. How do you configure multiple datasources in Spring Boot?

14. What's the difference between @RequestParam, @PathVariable, and @RequestBody?

15. How do you monitor and optimize Spring Boot applications in production?

Microservices & System Design

16. How do you implement distributed logging in microservices?

17. What's the difference between synchronous vs asynchronous communication in microservices?

18. How would you design a service to handle 1000+ requests per second?

19. How do you handle transactions that span across multiple microservices?

20. Can you explain the circuit breaker pattern? When have you used it?

Database & Real-time Scenarios

21. How do you implement pagination in SQL + Java?

22. If a query is taking 5 seconds, how would you debug & optimize it?

23. What's the difference between optimistic and pessimistic locking? Which one is better in microservices?

24. How do you implement caching in Spring Boot + DB to reduce load?

25. Real-world scenario: If one service keeps failing in production, how will you identify whether it's a code issue, infra issue, or DB issue?

#Java #BackendDeveloper #SoftwareEngineering #SystemDesign #Microservices #SpringBoot #CodingInterview #DSA #DataStructures #Algorithms #TechCareers #CareerGrowth #TalentAcquisition #HiringTechTalent #SoftwareJobs #ProfessionalDevelopment #ContinuousLearning #Innovation #CloudComputing #FinTech #DevOps #TechCommunity #EngineeringCommunity #GrowthMindset #LinkedInCommunity

Accenture_Java_Developer_Maste

.

7 pages

MEDIA document/carousel (possible)

#191 Aashish Gupta System Design

Sr Software Engineer@Uber | Mentoring Engineers in the AI Era | Backend & System Design | Building Scalable Systems | Brand Collabs

1mo · **View on LinkedIn**

Everyone treats URL Shortener as a warm-up interview question.

It's actually a small, elegant distributed system, and most whiteboard answers skip the three parts that make it hard once it's live.

Here's how the pieces fit together:

Getting a URL into the system: API Gateway — auth, abuse rate limiting (typically 10–20 creates/min per API key), and routing before a create request ever reaches the write path.

- Shorten Service — owns ID generation and write-path validation: custom alias collisions, malicious-URL checks against a blocklist, max-URL-length enforcement. Nothing gets a short code without going through here first.

- ID Generator — each app node checks out a range of ~10,000 base62 IDs at a time from a counter service, so writes don't need a global lock, and a node crash mid-range just wastes the unused tail instead of corrupting anything.

Getting a URL to actually redirect:

- Redirect Service — a completely separate read path from the write side, because reads outnumber writes by roughly 100:1 and can't queue behind them.

Cache Layer (Redis) — hot short codes served from memory with a multi-hour TTL; a cache miss falls through to the DB and repopulates the cache on read, not the other way around.

CDN Edge — high-traffic redirects are cached at the edge (short TTL, since a link can be deactivated), so one viral link doesn't funnel 100k req/sec into origin.

When things go viral:

- Hot Key Handling — the hard case isn't average load, it's one link suddenly taking 90% of traffic on a single Redis key. A thin local in-process LRU cache (last few hundred hot codes, TTL'd in seconds) sits in front of Redis specifically to absorb this, since Redis itself can become the bottleneck on a single-key hot spot.

- Analytics Pipeline — click events get pushed to a Kafka topic async and consumed downstream for click counts, referrer data, and geo — completely off the critical redirect path, so counting a click never adds latency to serving one.

Making it queryable and durable: Database — short_code as the primary lookup key, hash-sharded once a single node can't hold the table comfortably (usually well past the tens-of-millions-of-rows mark).

- Expiry Worker — a background job that sweeps expired links out of both cache and DB on a schedule, so dead links don't sit around forever consuming storage and cache slots.

Resources: → System Design Primer — https://lnkd.in/gCh7XJBf → AWS Builders' Library (caching) — https://lnkd.in/dN7MvvAE → Designing Data-Intensive Applications, Ch. 3

#backend #systemdesign #interview #softwareengineering

LINKS

- https://lnkd.in/gCh7XJBf
- https://lnkd.in/dN7MvvAE

#192 **Mohammad Iliyas Bagali** **Java / Spring**

Senior Java Backend Engineer | 10+ Years | Java 17, Spring Boot, Microservices, Kafka, AWS | JVM & SQL Performance | Open to UAE & Saudi Relocation

1mo · **View on LinkedIn**

A method can have @Transactional and still run without the transaction you expected. (!)

One common reason is self-invocation: a method inside the same class calls another @Transactional method. Because the call bypasses Spring’s proxy, the transactional behaviour may never be applied.

But this is only one of several production traps.

I created this practical guide with code examples covering:

Self-invocation and Spring proxies

(!) Checked-exception rollback rules

Exceptions caught and silently swallowed

@Async and thread boundaries

REQUIRES_NEW and connection-pool exhaustion

Slow external calls inside transactions

Why readOnly=true is not a write firewall

For every trap, the guide explains:

✗ Problematic code

What happens at runtime

✓ A safer implementation

The architectural decision behind it

My key takeaway:

@Transactional protects a database unit of work. It does not automatically make threads, API calls, messages, or multiple microservices part of the same atomic transaction.

Swipe through the guide and let me know:

Which trap has caused the most difficult production issue for you?

A) Self-invocation

B) Rollback rules

C) Async boundaries

D) Connection-pool exhaustion

#Java #SpringBoot #SpringFramework #Microservices #BackendEngineering #JavaDeveloper #SoftwareArchitecture #SeniorJavaEngineer

spring-transactional-example-g

.

10 pages

MEDIA SENIOR JAVA BACKEND ENGINEERING SERIES SPRING @TRANSACTIONAL 7 PRODUCTION TRAPS PRACTICAL EXAMPLES + SAFER DESIGNS PROXY CALLER SPRING PROXY DATABASE CONSISTENCY WHAT YOU WILL LEARN 1 Why an annotation can be present while no transaction starts 2 Why some exceptions commit and some roll back 3 Why threads, messages, and services need different consistency patterns READ THE CODE, THEN ASK: WHERE DOES THE ATOMIC BOUNDARY END? SWIPE FOR WRONG -> WHAT HAPPENS -> SAFER APPROACH Mohammad Iliyas Bagali; SPRING @TRANSACTIONAL - EXAMPLE GUIDE PAGE 2/10 HOW THE TRANSACTION REALLY STARTS NORMAL EXTERNAL CALL PROXY BEGIN TX RUN METHOD CALLER PROXY TX MANAGER DB WORKING EXAMPLE - CALL CROSSES THE PROXY @RestController class OrderController { private final OrderService service; @PostMapping("/orders") void create(@RequestBody Order order) { service.create(order); // external call to Spring bean } } @Transactional public void create(Order order) { repository.save(order); } WHAT SPRING DOES The controller holds the proxied OrderService bean. The proxy opens a transaction before create(), runs the method, and then commits or rolls back according to the configured rules. IMPORTANT @Transactional is metadata. In default proxy mode, only a call that crosses the Spring proxy is intercepted. KEY TAKEAWAY First debugging question: Did this method call actually cross a Spring proxy? MOHAMMAD ILIYAS BAGALI | SENIOR JAVA BACKEND ENGINEERING SERIES 2/10; SPRING @TRANSACTIONAL - EXAMPLE GUIDE PAGE 3/10 TRAP 1: SELF-CALL BYPASSES THE PROXY WRONG - TRANSACTION NEVER STARTS @Service class OrderService { public void placeOrder(Order order) { validate(order); saveOrder(order); // same object: this.saveOrder(...) } @Transactional public void saveOrder(Order order) { orderRepository.save(order); } } WHAT HAPPENS placeOrder() calls saveOrder() directly on the same Java object. The call does not pass through the proxy, so the annotation on saveOrder() is not applied. SAFER - MOVE THE BOUNDARY TO ANOTHER SPRING BEAN @Service class OrderWriter { @Transactional public void save(Order order) { repository.save(order); } } @Service class OrderService { private final OrderWriter writer; public void placeOrder(Order order) { writer.save(order); } } KEY TAKEAWAY Do not solve this with hidden self-injection. Make the transaction boundary visible in the service design. MOHAMMAD ILIYAS BAGALI | SENIOR JAVA BACKEND ENGINEERING SERIES 3/10; document/carousel (possible)

#193 Yogesh Kshirsagar

Java / Spring

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

1mo · **View on LinkedIn**

Spring Boot Under the Hood- Vol. 16 of 30

**You Called One Method...

So Why Did So Much Happen? **

You write:
paymentService.transferMoney();
Simple.
Or at least... that's what you think.
The call begins its journey toward your method.
But before it reaches there...
Someone stops it.
“Security check complete.”
“Transaction started.”
“Logging enabled.”
Your method hasn't even started yet.
Finally, the call reaches:
transferMoney()
Your business logic runs.
Money is transferred.
But the journey isn't over.
As the method returns...
“Commit transaction.”
“Log the result.”
“Record execution time.”
Wait...
Who is doing all this?
You certainly didn't write all of that inside your method.
Meet the invisible character: The Proxy
The Proxy stands between the caller and your actual bean.
Your call thinks it is going directly here:
You → Service Method
But the real journey may look like this:
You → Proxy → Extra Work → Your Method → Extra Work → You
And now the other characters enter the story:
JoinPoint
“Here is a place where something can happen.”
Pointcut
“This is the exact place where I want something to happen.”
Advice
“This is the work I will perform.”
Proxy
“I'll make sure it all happens at the right time.”
Together, they quietly power some of Spring's biggest features:
@Transactional
Security
Logging
Caching
And your business method?
It keeps doing just one thing:
Business logic.

Everything else happens around it.
That is the magic of Spring AOP.
Next: Self Invocation & Proxy Limitations
Why does @Transactional sometimes appear to work...
but silently doesn't?
Follow along as we continue going deeper into Spring Boot — Under the Hood.
#SpringBoot #SpringFramework #Java #BackendDevelopment #Microservices #JavaDeveloper #SoftwareEngineering

MEDIA advertisement

#194 Yogesh Kshirsagar

Java / Spring

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL
1mo · View on LinkedIn

Spring Boot Under the Hood- Vol. 15 of 30
**“I need a bean.”
Spring replies: “Which lifetime?” **
Imagine five people walk into a hotel.
The first says:
“Give me one room. Everyone can share it.”
The second says:
“I want a brand-new room every time I ask.”
The third says:
“I only need it for this visit.”
The fourth says:
“Keep my room for my entire stay.”
The fifth says:
“One space for everyone in the building.”
Same hotel.
Same purpose.
Completely different lifetimes.
And surprisingly...
Spring does something very similar with your beans.
When you ask Spring for an object, it doesn't just decide:
“Should I create this bean?”
It also decides:
“How long should this bean live?”
“Who should share it?”
“When should I create a new one?”
That decision is called Bean Scope. ↓
Singleton → One shared instance
Prototype → New instance every time
Request → Lives for one HTTP request
Session → Lives for one user session
Application → Lives across the web application context
But here's the interesting part...

Most Spring developers know Singleton and Prototype.

The real question is:

What happens when a Prototype bean is injected into a Singleton?

And...

Does Singleton automatically mean thread-safe?

Not always.

Bean scopes may look like a small annotation-level concept.

But underneath, they control object lifetime, sharing, state, and sometimes some very unexpected production behaviour.

Next: AOP Internals

What if Spring could add behavior to your code...

without changing the code itself?

Proxy

JoinPoint

Advice

Pointcut

Who actually receives your method call?

We’re about to find out.

Follow along as we continue going deeper into Spring Boot — Under the Hood.

#SpringBoot #SpringFramework #Java #BackendDevelopment #Microservices #JavaDeveloper #SoftwareEngineering

46

1

4

Yogesh Kshirsagar

Author

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

1mo

Circular Dependencies: https://www.linkedin.com/posts/yogesh-kshirsagar-04080014_springboot-springframework-java-activity-7498599849198579712-udwz?utm_source=share&utm_medium=member_android&rcm=ACoAAAL3YDABrLwhcCq19IWx22YwJAGh4cG9UwQ

Spring Boot Singleton and Early Reference | Yogesh Kshirsagar posted on the topic | LinkedIn

linkedin.com

MEDIA advertisement

LINKS

- https://www.linkedin.com/posts/yogesh-kshirsagar-04080014_springboot-springframework-java-activity-7498599849198579712-udwz?utm_source=share&utm_medium=member_android&rcm=ACoAAAL3YDABrLwhcCq19IWx22YwJAGh4cG9UwQ

#195 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

Kafka / Messaging

Sohitha C • 3rd+ | Kafka becomes easier when you stop treating it like a simple message broker. | Real systems use different patterns depending on how events must be distributed, processed, replayed, scaled, and recovered. | Here are 12 Kafka patterns every Data Engineer should understand: | Publish / Subscribe | ↳ Multiple consumer groups independently receive the same events from a topic. | Work Queue | ↳ Consumers in one group divide messages so each event is processed once. | Event Sourcing | ↳ Store every state change as an event and rebuild

state through replay. | Stream Processing | ↳ Continuously consume, transform, enrich, and publish real-time data. | Change Data Capture | ↳ Stream database changes into Kafka for synchronization and analytics. | Dead Letter Queue | ↳ Move failed records into a separate topic for debugging and reprocessing. | Parent–Child Topics | ↳ Derive child-topic data from parent events using consistent keys. | Request / Reply | ↳ Connect request and response topics through a correlation ID. | Competing Consumers | ↳ Scale processing across consumers without duplicating message consumption. | Partitioning Strategy | ↳ Distribute events across partitions for throughput, order, and scalability. | Saga Pattern | ↳ Coordinate distributed transactions through events and compensating actions. | Time Windowing | ↳ Group events into fixed or rolling windows for time-based metrics. | The pattern matters, but implementation discipline matters too. | Choose partition keys carefully. | Design idempotent consumers. | Set retention intentionally. | Monitor lag and failures. | Test recovery before production does it for you. | Which Kafka pattern do you use most often in real systems? | #DataEngineer #DataEngineering #Data #Kafka #Workflows #ETL #ELT #KafkaTopics #CDC #StreamProcessing #DLQ Abhisek Sahu | 68 | 1 | 10 | | Sardar Noor Burhan Verified Profile 3rd+ | Sardar Noor Burhan | Lead Data Consultant @ Statistique.io | Automation, Analytics & AI

1mo · **View on LinkedIn**

the discipline point is the real story, most kafka messes trace back to undocumented event contracts, not the wrong pattern

MEDIA advertisement; document/carousel (possible)

#196 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · **View on LinkedIn**

You might think “caching” = Redis.

But in real system design...

Caching is a stack, not a single layer.

Different caches live in different places, solve different problems, and break in different ways.

Here are 8 types of caching you’ll actually use in system design ↓

1) Browser Cache

The first cache layer - stores static frontend files in the user’s browser so repeat visits feel instant.

2) CDN Cache

Caches images/videos/JS/CSS at edge locations worldwide, reducing latency and protecting the origin from traffic spikes.

3) Reverse Proxy Cache

Sits between client and backend (NGINX/Varnish) to cache API responses/pages and reduce backend load.

4) Application Cache

Lives inside your service layer - caches computed results, user sessions, feature flags, and frequent query outputs.

5) Database Cache

Caches query results / hot rows near the DB layer to reduce DB I/O and speed up repeated reads.

6) Distributed Cache

A shared cache layer (Redis/Memcached) used across services - essential for microservices and horizontal scaling.

7) Write-Through Cache

Writes go to cache + DB together - best for strong consistency where stale data is unacceptable.

8) Write-Back Cache (Write-Behind)

Writes go to cache first, DB later asynchronously - best for high-write systems, but needs durability + recovery planning.

✓ If you understand these 8 cache types...

you can design systems that are fast, scalable, and stable under load

LINKS
- https://www.youtube.com/@learnwithrockybhatia

#197 Alex Xu

Java / Spring

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

6mo · **View on LinkedIn**

How the JVM Works

We compile, run, and debug Java code all the time. But what exactly does the JVM do between compile and run?

Here's the flow:

Build: javac compiles your source code into platform-independent bytecode, stored as .class files, JARs, or modules.

Load: The class loader subsystem brings in classes as needed using parent delegation. Bootstrap handles core JDK classes, Platform covers extensions, and System loads your application code.

Link: The Verify step checks bytecode safety. Prepare allocates static fields with default values, and Resolve turns symbolic references into direct memory addresses.

Initialize: Static variables are assigned their actual values, and static initializer blocks execute. This happens only the first time the class is used.

Memory: Heap and Method Area are shared across threads. The JVM stack, PC register, and native method stack are created per thread. The garbage collector reclaims unused heap memory.

Execute: The interpreter runs bytecode directly. When a method gets called multiple times, the JIT compiler converts it to native machine code and stores it in the code cache. Native calls go through JNI to reach C/C++ libraries.

Run: Your program runs on a mix of interpreted and JIT-compiled code. Fast startup, peak performance over time.

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gauQcE45

#systemdesign #coding #interviewtips

LINKS
- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gauQcE45
- https://www.linkedin.com/feed/update/urn:li:activity:7500587900963598337
- https://lnkd.in/gAhvuxym

ALSO SAVED AS Alex Xu (1mo) link

#198 Sai Ram Somanaboina

System Design

Engineering Manager at NowFloats(Jio Group) | 15 years in Engineering

1mo · **View on LinkedIn**

If you want to land a software engineering job in 2026, bookmark these 18 resources:

1. Overleaf CV templates - clean resume formats

https://lnkd.in/d7uBXZYs

2. NeetCode Roadmap - pattern-based DSA

https://lnkd.in/diqE3c6Y

3. Striver's A2Z Sheet - DSA from basics to DP

https://lnkd.in/dnP_zXus

4. LeetCode Top Interview 150 - high-frequency questions
https://lnkd.in/deHmsbMc
5. Tech Interview Handbook - resume, coding, behavioural, negotiation
https://lnkd.in/dF4zgh2k
6. Coding Interview University - CS fundamentals
https://lnkd.in/gf4M3U8P
7. System Design Primer - free HLD foundation
https://lnkd.in/gvE5ApvJ
8. Hello Interview - system design and behavioural prep
https://lnkd.in/d7WfW6Xa
9. Jobs24x - curated tech and remote roles
https://jobs24x.com/jobs
10. Wellfound - startup jobs
https://wellfound.com/jobs
11. YC Work at a Startup - YC company roles
https://lnkd.in/daNHa__a
12. Instahyre - product company roles in India
https://lnkd.in/dZxiKFrU
13. Cutshort - Indian startup and tech jobs
https://lnkd.in/dNhQuDQK
14. LinkedIn Jobs - the broadest role pool
https://lnkd.in/d7qbvrMH
15. Naukri - Indian enterprise and IT roles
https://lnkd.in/dnnyCKuD
16. Hirist - tech-only jobs in India
https://www.hirist.tech/
17. Unstop - fresher, campus, and early-career roles
https://unstop.com/jobs
18. Internshala - internships and entry-level roles
https://lnkd.in/dsUV-hUb

Pick three preparation resources and three job boards for the next 30 days. Track every application, revision topic, rejection, and interview note.

Otherwise, this will become another useful post you save and never use.

LINKS

- https://lnkd.in/d7uBXZYs
- https://lnkd.in/diqE3c6Y
- https://lnkd.in/dnP_zXus
- https://lnkd.in/deHmsbMc
- https://lnkd.in/dF4zgh2k
- https://lnkd.in/gf4M3U8P
- https://lnkd.in/gvE5ApvJ
- https://lnkd.in/d7WfW6Xa
- https://jobs24x.com/jobs
- https://wellfound.com/jobs
- https://lnkd.in/daNHa__a
- https://lnkd.in/dZxiKFrU
- https://lnkd.in/dNhQuDQK
- https://lnkd.in/d7qbvrMH
- https://lnkd.in/dnnyCKuD
- https://www.hirist.tech/
- https://unstop.com/jobs

- https://lnkd.in/dsUV-hUb

#199 Raja Anand

Java / Spring

Lead Java Engineer · Spring Boot & Microservices · Building Scalable Backend Systems at HCL Technologies

1mo · **View on LinkedIn**

Are you a Java developer?
Give me one minute. I want to show you something Spring Framework 7 quietly does for us.
You used to reach for Resilience4j the moment you needed a retry.
Not anymore.
Retries and concurrency limits are built into the core framework now.
A retry is one annotation:

```
@Retryable(maxRetries = 3)
public Quote fetchQuote(String symbol) {
return restClient.get()
.uri("/quotes/{symbol}", symbol)
.retrieve()
.body(Quote.class);
}
```

Cap concurrent calls to a flaky downstream:

```
@ConcurrencyLimit(10)
public void callLegacyApi() { ... }
```

Turn it on once:

```
@EnableResilientMethods
@Configuration
class ResilienceConfig { }
```

No extra library. No wrapper config.
But here's the honest part.
Spring Framework 7 does not give you a circuit breaker or a rate limiter.
For a synchronous integration that has to fail fast when a dependency dies, you still want Resilience4j.
So this isn't "delete Resilience4j."
It's "reach for it less."
Retries and bulkheads: let Spring handle them.
Circuit breaking and rate limiting: Resilience4j still earns its place.
When do you still pull in Resilience4j?
#Java #SpringBoot #BackendDevelopment

MEDIA resilience

LINKS

- https://www.linkedin.com/feed/update/urn:li:activity:7483203085507481600

ALSO SAVED AS Ian Dancan (3mo) link

#200 Sudhir Dhangar

System Design

Specialist Programmer L2 | 12K+ Followers | Helping Students & Developers Land Software Engineering Jobs | Java | Spring Boot | DSA | #InfyTQ Certified

1mo · View on LinkedIn

Some engineers prepare for System Design interviews by memorizing architectures.
That’s the wrong approach.
You don’t need to memorize 50 different system designs.
Instead, go deep into 8–10 core systems:
→ Payments
→ Search
→ Feed
→ Booking / Inventory
→ Notifications
→ File Storage
→ Rate Limiter
→ Ride Matching
→ Recommendation
→ Metrics / Logging
But understanding the architecture is only the beginning.
The real skill is knowing how the design changes when the requirements change.
Take one system and change one constraint:
→ Traffic becomes 10x
→ One region goes down
→ Duplicate requests are no longer acceptable
→ Reads must be under 100ms
→ Data must stay within a specific country
→ A critical dependency becomes unreliable
Then ask yourself:
What changes?
What stays the same?
And most importantly, why?
That is System Design.
For Senior/Staff-level interviews, prepare your own projects even more deeply.
For every major system you have built, understand this flow:
Scale → Bottleneck → Decision → Trade-off → Incident → Outcome
For example, saying:
✗ “We used Kafka.”
is not a strong answer.
A better answer is:
✓ “Kafka was necessary because our traffic had increased significantly. We rejected X because of ___ and chose Kafka because ___. The trade-off was ___.”
That shows engineering judgment, not just tool knowledge.
And don’t ignore behavioural rounds.
At senior levels, interviewers also want to understand how you operate:
→ How do you handle disagreement?
→ How do you deal with ambiguity?
→ How do you influence people without authority?
→ How do you handle failure?
→ How do you respond to production incidents?

→ How do you balance speed vs quality?

→ How do you mentor others?

The goal isn’t to remember the perfect architecture.

The goal is to understand why your architecture should change when the constraints change.

That is what strong System Design preparation looks like.

Keeping this in mind, I went deep and documented everything into a System Design Interview Guide.
Get the Guide here:

https://lnkd.in/dsWQkfy4

Until the next post...

Stay Curious. Keep Coding.

#SystemDesign #SystemDesignInterview #InterviewPrep #SoftwareEngineering #Engineering #SystemDesignPreparation #TechInterviews #Java #BackendDevelopment #SoftwareDeveloper

38

1

10

Sudhir Dhangar

Author

Specialist Programmer L2 | 12K+ Followers | Helping Students & Developers Land Software Engineering Jobs | Java | Spring Boot | DSA | #InfyTQ Certified

1mo

Want more content like this?

I share:

✓ Java & Backend Development

✓ Interview Tips

✓ Hiring Updates

✓ Project Ideas

✓ Career Guidance

✓ Free Resources

Join my communities and never miss an update!

Topmate : https://topmate.io/sudhir_dhangar

Telegram: https://t.me/sudhirdhangar

WhatsApp Channel: https://whatsapp.com/channel/0029Vb8G7Hy65yDAPzWuYd2v

If you find my content helpful, do join and share it with your friends.

... more

Sudhir Dhangar

whatsapp.com

LINKS

- https://lnkd.in/dsWQkfy4
- https://topmate.io/sudhir_dhangar
- https://t.me/sudhirdhangar
- https://whatsapp.com/channel/0029Vb8G7Hy65yDAPzWuYd2v

#201 Kumari Mahima

System Design

Senior Software Engineer (Backend) | Java • Spring Boot • Kafka | Distributed Systems | AWS , Azure, Kubernetes | Built Platforms for 10M+ Users | Open to Europe Relocation

1mo · **View on LinkedIn**

Most system designs don’t start with Kafka, Redis, or 10 microservices.

They usually start with something much simpler:
Client → Backend → Database
And honestly, that’s often enough in the beginning.
But as the system grows, the problems start showing up.
More traffic hits the APIs.
Database reads increase.
Some operations take too long.
Users start uploading files.
Third-party services become dependencies.
And suddenly, our simple architecture needs to evolve.
This is where system design becomes interesting.
Instead of adding technologies just because they are popular, we start asking:
“What problem are we trying to solve?”
Traffic is increasing?
→ Load Balancer + Horizontal Scaling
Too many repeated reads?
→ Redis Cache
Database reads are becoming a bottleneck?
→ Read Replicas
Some operations don’t need an immediate response?
→ Kafka/RabbitMQ + Background Workers
Large files shouldn’t pass through application servers?
→ S3 / Object Storage
A third-party service is slow or unavailable?
→ Timeout + Retry + Circuit Breaker
APIs need protection?
→ Authentication + Authorization
Something fails in production and we need to know why?
→ Logs + Metrics + Alerts + Tracing
So our architecture gradually evolves:
Client
↓
API Gateway
↓
Load Balancer
↓
Backend Services
↓
Cache + Database
↓
Message Queue + Workers
↓
External Services
But the most important part isn’t the diagram.
It’s being able to explain why each component exists.
You shouldn’t add Kafka because “every scalable system uses Kafka.”

You add it because you have a problem that asynchronous processing can solve.
You shouldn’t add Redis because “Redis makes systems faster.”
You add it because reducing repeated database reads is actually a requirement.
Good system design isn’t about having more boxes in the architecture.
It’s about making the right trade-offs for the problem you’re solving.
And as backend engineers, one question should always be at the back of our mind:
“What problem does this component solve?”
That’s where real system design begins.
#SystemDesign #BackendEngineering #Java #SpringBoot #Microservices #DistributedSystems #SoftwareArchitecture #Kafka #Redis #AWS #BackendDevelopment

#202 **Neo Kim** System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1mo · **View on LinkedIn**

If you want to become good at system design,
learn these 15 case studies (save now):
1 How ChatGPT Works
↳ https://lnkd.in/emyP63Wx
2 How YouTube Works
↳ https://lnkd.in/e7q9F4Sg
3 How WhatsApp Works
↳ https://lnkd.in/eU2fswMi
4 How Uber Computes ETA
↳ https://lnkd.in/eVKV2ePC
5 How Google Docs Works
↳ https://lnkd.in/ehPNA7Az
6 How Spotify Works
↳ https://lnkd.in/eGbWVeNW
7 How Reddit Works
↳ https://lnkd.in/egmm_P7a
8 How Twitter Works
↳ https://lnkd.in/eniXMPfU
9 How Slack Works
↳ https://lnkd.in/eATMDjrK
10 How Airbnb Works
↳ https://lnkd.in/dGVfstQM
11 How Apache Kafka Works
↳ https://lnkd.in/eTtVAjTg
12 Top Redis Use Cases
↳ https://lnkd.in/ekJMjMG3
13 How Stock Exchange Works
↳ https://lnkd.in/eNf2QxVZ
14 How Bitly URL Shortener Works
↳ https://lnkd.in/evFTZVQq
15 How Bluesky Works

↳ https://lnkd.in/eEhB8V_k

Anything else you'd add?

===

PS - Want my System Design Playbook for FREE?

Join my newsletter with 245K+ software engineers:

→ https://lnkd.in/ehTrcyak

===

Repost to help others ace system design.

Follow Neo Kim + turn on notifications.

MEDIA 15 case studies for senior software engineers

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/emyP63Wx
- https://lnkd.in/e7q9F4Sg
- https://lnkd.in/eU2fswMi
- https://lnkd.in/eVKV2ePC
- https://lnkd.in/ehPNA7Az
- https://lnkd.in/eGbWVeNW
- https://lnkd.in/egmm_P7a
- https://lnkd.in/eniXMPfU
- https://lnkd.in/eATMDjrK
- https://lnkd.in/dGVfstQM
- https://lnkd.in/eTtVAjTg
- https://lnkd.in/ekJMjMG3
- https://lnkd.in/eNf2QxVZ
- https://lnkd.in/evFTZVQq
- https://lnkd.in/eEhB8V_k
- https://lnkd.in/ehTrcyak

#203 Pradeep Kumar Saini

System Design

Senior Engineering Manager @ Innefu Labs | Ex-Zee5, Shopclues, Paytm | AI · LLMs · Agentic AI · RAG | Backend · Java · Spring Boot · Distributed Systems · MySQL · DynamoDB · Elasticsearch · Redis · Cassandra . Kafka ·AWS

1mo · **View on LinkedIn**

Some engineers prepare for System Design interviews by memorizing architectures.

That’s the mistake.

You don’t need to memorize 50 system designs. Go deep on 8–10 core systems:

→ Payments

→ Search

→ Feed

→ Booking / Inventory

→ Notifications

→ File Storage

→ Rate Limiter

→ Ride Matching

→ Recommendation

→ Metrics / Logging

But don’t stop at the architecture.

Take one design and change one constraint:

→ Traffic becomes 10x

→ One region goes down

→ Duplicates become unacceptable
→ Reads must be <100ms
→ Data must stay in-country
→ A critical dependency becomes unreliable
Then ask:
What changes? What doesn’t? And why?
That is System Design.
And for Senior/Staff interviews, prepare your own projects even harder.
For every major system you’ve built, know:
Scale → Bottleneck → Decision → Trade-off → Incident → Outcome
“We used Kafka” is not an answer.
“Kafka was necessary because..., we rejected X because..., and accepted Y as the trade-off” is engineering judgment.
Finally, don’t ignore behavioural rounds.
At senior levels, they test how you operate, not just what you know:
→ Disagreement
→ Ambiguity
→ Influence without authority
→ Failure
→ Incidents
→ Speed vs quality
→ Mentorship
The goal isn’t to remember the perfect architecture.
The goal is to explain why your architecture should change when the constraints change.
#systemDesign #engineering #development #interviews #InterviewPrep #systemDesignInterview #maang #AI #AISystemDesign

#204 Vibhav Gupta

Java / Spring

Lead Backend Engineer | Java & Spring Boot | Microservices | Kafka | AWS | Building Scalable Systems That Solve Real Business Problems

1mo · View on LinkedIn

"@Transactional" in Spring Boot: The Annotation Is Easy. Understanding Its Boundaries Is Not.
One thing I’ve learned while working with Java/Spring Boot is that database transactions are not just about adding:
@Transactional
The real engineering challenge is understanding where the transaction starts, where it ends, and what happens when things go wrong.
Consider a typical order flow:
Order → Payment → Inventory → Commit
What if Payment succeeds but Inventory update fails?
Without proper transaction boundaries, you can easily end up with inconsistent data.
A few areas I always pay attention to:
Transaction Boundary
A transaction should cover the right business operation — not blindly an entire class or every database call.
Propagation

"REQUIRED" vs "REQUIRES_NEW" can completely change the behavior of nested service calls.

For example, audit logging may intentionally require a separate transaction:

@Transactional(propagation = Propagation.REQUIRES_NEW)

public void saveAuditLog(...) {

// independent transaction

}

Rollback Semantics

A common production misconception is:

«“Any exception automatically rolls back.”»

Not necessarily.

Spring's default rollback behavior primarily targets "RuntimeException" and "Error". Checked exceptions may require explicit configuration:

@Transactional(rollbackFor = Exception.class)

Isolation Level

Choosing an isolation level is a consistency vs concurrency decision.

Higher isolation can prevent anomalies, but it can also increase locking/contention and impact throughput.

The Self-Invocation Trap

public void methodA() {

methodB();

}

@Transactional

public void methodB() {

}

Calling "methodB()" from the same class can bypass Spring's proxy, so the expected transactional behavior may not be applied.

readOnly ≠ “Database Cannot Write”

@Transactional(readOnly = true)

is primarily a transaction optimization/hint. It should not be treated as a hard security mechanism preventing writes.

What matters in production?

Don't ask only:

“Where should I put "@Transactional"?”

Ask:

“What is the atomic business operation, what consistency guarantees do I need, and what should happen when one step fails?”

That's where understanding transactions moves from Spring knowledge → production engineering.

#Java #SpringBoot #BackendEngineering #SoftwareEngineering #Microservices #Database #SystemDesign #SpringFramework #SeniorSoftwareEngineer #TechLeadership

#205 Jainath Jaiswal System Design

Solution Architect®| AWS Certified® | Microservices | Cloud-Native | AWS | Azure | GCP | Kafka | Kubernetes | AI/ GenAI | Distributed Systems | OpenShift | Spring Boot | Applications Development

1mo · **View on LinkedIn**

Idempotency vs Deduplication — Two Concepts Every Microservices Engineer Should Know

#Java #Microservices #Kafka #SystemDesign #DistributedSystems #SpringBoot #SoftwareArchitecture #BackendDevelopment #EventDrivenArchitecture #CloudNative #TechLeadership

#206 Glynes D'Souza

Java / Spring

Java Developer || Full Stack || Card Lending || AWS

1mo · View on LinkedIn

Spring Boot quietly eliminated an entire category of boilerplate code: manual REST clients.

If you've ever built inter-service communication in Java, you know the routine. You've likely written dozens of RestTemplate or WebClient chains—building URLs, serializing parameters, chaining .retrieve(), handling response mappings, and dealing with generic collection types. Multiply that across every microservice endpoint, and it quickly becomes maintenance clutter.

With declarative HTTP interfaces in modern Spring, you define an interface and let the framework handle the rest:

@HttpExchange("/users")

public interface UserClient {

@GetExchange

List<User> findByStatus(@RequestParam String status);

}

Why this is a game-changer for backend architecture:

1. Zero Proxy Wiring: Define the method, path, and parameters—Spring generates the bean implementation under the hood.

2. Compile-Time Safety: Strong typing across query params and payload structures catches contract breaks early.

3. Underlying Flexibility: Easily swap the transport engine (RestClient for synchronous blocking or WebClient for reactive flows) without altering business logic.

4. Effortless Testing: Mocking an interface in unit tests is significantly cleaner than mocking chained client builders.

5. Native & Dependency-Free: No need for extra third-party wrappers like OpenFeign.

Are you already migrating your downstream HTTP clients to @HttpExchange, or are you still maintaining traditional client wrapper chains in production?

#Java #SpringBoot #Microservices #BackendEngineering #SoftwareArchitecture #CleanCode #JavaDeveloper #RESTAPI #CloudNative #DevCommunity

MEDIA advertisement

#207 Mukesh Dhale

System Design

Java Backend Developer | SaaS Platform R&D | Spring Boot | Microservices | AWS Cloud | RabbitMQ | Docker | REST APIs | Distributed Systems | Scalable Enterprise Applications

1mo · View on LinkedIn

I've been working with Java and Spring Boot, and one topic that keeps coming up in backend interviews is Microservices Architecture.

So I made this simple poster to explain the basic flow and some important interview points.

In a typical microservices setup:

Client → API Gateway → Multiple Services → Database

For example, we can have separate services for:

User

Order

Payment

Notification

Product

The main advantage is that each service can be developed, deployed and scaled independently.

But microservices also comes with its own problems — service communication, network failures, data consistency, logging, monitoring and distributed transactions.

For interviews, I think it's important to understand why we use each component instead of just remembering definitions.

Some questions I always recommend preparing:

→ Monolith vs Microservices

→ Why API Gateway?

→ What is Service Discovery?

→ How do services communicate?

→ What is Circuit Breaker?

→ How do you handle failures?

→ How do you maintain data consistency?

I’m sharing more such Java + Spring Boot + Microservices concepts while preparing and learning.

If you’re also a Java developer, feel free to share your experience.

Which topic should I cover next — Kafka, API Gateway or Circuit Breaker?

#JavaDeveloper #Java #SpringBoot #Microservices #BackendDeveloper #SoftwareDeveloper #SystemDesign #JavaInterview #SpringCloud #Kafka

#208 Shameer Ahamad

Java / Spring

Senior Java Developer @ Wipro | Java | Spring Boot | REST API | Microservices | GCP | Kafka | Docker | SQL | NoSQL | JUnit | Mockito | GenAI | IoT

1mo · View on LinkedIn

How HashMap Works Internally in Java

HashMap stores data as key-value pairs using hashing for fast lookups. The key's hashCode() determines the bucket, and equals() is used to identify the correct key when collisions occur.

When multiple keys map to the same bucket, entries are stored in a LinkedList. In Java 8+, if a bucket grows beyond a threshold, it is converted into a Red-Black Tree for better performance.

#Java #HashMap #Collections #CoreJava #JavaInternals #DataStructures #JavaDeveloper #BackendDevelopment #Programming #SoftwareEngineering #InterviewPreparation

#209 Parikh Jain

AI / LLM

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

1mo · View on LinkedIn

Trust me, cracking offers is not hard if you follow this ↓

You do not need 500 questions or 12 copied projects. You need a system.

[1] For DSA Prep

Follow this: https://lnkd.in/gqcDeDuB

For each pattern: learn the template, solve 3 easy + 5 medium problems, then explain the complexity aloud.

[2] For LLD Prep

1. OOP: abstraction, inheritance, polymorphism, composition

2. SOLID principles and clean interfaces

3. Patterns: Strategy, Factory, Observer, State, Command

4. Class diagrams, relationships, APIs and data models

5. Practice: Parking Lot, Elevator, Splitwise, Chess, Vending Machine

6. Add locks, race conditions and idempotency
7. Support new payments, vehicle types and pricing rules
8. Code the design and defend every trade-off

This is the loop 1500+ students are running inside the SDE Career Tracker (https://lnkd.in/gtzyBHkw).

Pattern-targeted DSA, LLD, HLD, GenAI basics, a Resume Analyzer that rewrites your resume, scored mock interviews,

a live Job-Ready Score, and job assistance with 200+ partner companies plugged in for referrals.

[3] For AI Fundamentals + Communication

1. Tokens, embeddings, transformers and context windows
2. Prompting, structured outputs and tool calling
3. RAG: chunking, retrieval, reranking and evaluation
4. Agents: planning, memory, tools and guardrails
5. Hallucinations, latency, cost, privacy and security
6. Prepare 8 STAR stories: ownership, conflict, failure, leadership
7. Think aloud while solving
8. Clarify requirements, compare options and state trade-offs

[4] For Building

1. Build a backend project and an AI project
2. Write requirements first
3. Design APIs, schema and failure cases
4. Add auth, caching, queues and background jobs
5. Write unit and integration tests
6. Deploy with logs, metrics and alerts
7. Write a README explaining your decisions
8. Record a demo and explain how it scales 10x

DSA gets you through coding.

LLD proves you can structure software.

AI shows you understand the market.

Communication and projects prove you can work as an engineer.

That combination gets offers.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gqcDeDuB
- https://lnkd.in/gtzyBHkw

#210 Ujjwal Sontakke

System Design

Senior Consultant - AWS Data Engineer at Capgemini | 4 Years of Experience | PySpark | Apache Spark | SQL | AWS Glue | S3 | Athena | Step Functions | ETL & Data Migration | Data Reconciliation | Big Data

1mo · **View on LinkedIn**

System Design becomes much easier when you stop memorizing components and start recognizing patterns.

When a system needs to handle more users, more data, higher traffic, and greater reliability, the same fundamental design decisions appear again and again.

A useful way to approach System Design is to think through six core patterns:

1. Scaling

Horizontal vs. vertical scaling — how do we handle increasing load?

2. Data

Sharding, partitioning, and indexing — how do we manage growing datasets efficiently?

3. Performance

Caching and CDNs — how do we reduce latency and avoid unnecessary database load?

4. Reliability

Load balancing and failover — how do we keep the system available when components fail?

5. APIs

Idempotency and rate limiting — how do we make APIs safer and more predictable?

6. Consistency

CAP and eventual consistency — what trade-offs are acceptable for the system?

The key is that System Design is not about drawing boxes.

It's about understanding the trade-offs behind those boxes.

For example, a typical user-to-service flow can look like:

User → DNS/CDN → Load Balancer/API Gateway → Application Servers → Cache → Service Layer → Database/Storage

But every component exists for a reason.

A Load Balancer distributes requests across multiple backend servers, helping prevent a single server from becoming overloaded while improving availability.

A Cache keeps frequently accessed data in faster storage, reducing latency and unnecessary database load.

And the broader architecture forces you to think about:

✓Scalability vs. cost

✓Performance vs. consistency

✓Availability vs. consistency

✓Simplicity vs. flexibility

That is where real System Design thinking begins.

The mindset I find most useful:

Don't start with:

✗ “Which technology should I use?”

Start with:

✓ What does the system need to do?

✓ How much traffic will it handle?

✓ Where could it become a bottleneck?

✓ What happens when a component fails?

✓ What data needs to be stored and how will it grow?

✓ Which trade-offs are acceptable?

Once you answer those questions, choosing the architecture becomes much more logical.

Good System Design isn't about using the most technologies.

It's about using the right patterns for the problem.

Repost if you found it useful. Follow Ujjwal Sontakke Jain for #Data related post.

#SystemDesign #DataEngineering #SoftwareEngineering #SystemArchitecture #DistributedSystems #Scalability #CloudComputing #DataArchitecture #BackendEngineering #TechInterview #AWS #Engineering

#211 Sudhir Dhangar System Design

Specialist Programmer L2 | 12K+ Followers | Helping Students & Developers Land Software Engineering Jobs | Java | Spring Boot | DSA | #InfyTQ Certified

1mo · View on LinkedIn

Which System Design resource should I follow?

I see this question all the time from students and developers preparing for Software Engineering interviews.

And honestly, I understand the confusion.

There are hundreds of YouTube videos.

Dozens of courses.

Multiple System Design books.

Different roadmaps.

HLD. LLD. Distributed Systems. Microservices. Cloud. Databases.

So you keep collecting resources...

But never actually start designing systems.

You don't need 20 resources.

You need to understand how to think about a system.

Here’s the roadmap I would follow ↓

1. Start with the fundamentals

Before jumping into “Design Netflix” or “Design Uber”, understand:

• Client-Server architecture
• HTTP & HTTPS
• REST APIs
• DNS
• Load Balancers
• Reverse Proxy
• Caching
• Databases
• SQL vs NoSQL
• Message Queues
• CDN
• Horizontal vs Vertical Scaling

2. Understand databases properly

Don't just learn:

SQL = relational

NoSQL = non-relational

Understand:

→ Indexing
→ Transactions
→ ACID
→ Replication
→ Sharding
→ Partitioning
→ Read replicas
→ Consistency
→ CAP theorem

3. Learn caching

Caching can completely change the performance of a system.

Understand:

• Cache-aside
• Write-through
• Write-back
• TTL
• Cache eviction
• Cache invalidation
• Cache consistency
• Distributed caching

4. Learn asynchronous communication

As your system grows, synchronous communication isn't always enough.

Learn:

→ Message queues
→ Pub/Sub
→ Kafka concepts
→ Producers & Consumers
→ Retry mechanisms
→ Dead-letter queues
→ Idempotency
→ Event-driven architecture

5. Learn scalability

Ask yourself:

“What happens when 100 users become 10 million users?”

Learn how to handle:

• Traffic spikes
• Database bottlenecks
• Read-heavy workloads
• Write-heavy workloads
• High availability
• Fault tolerance
• Horizontal scaling

6. Start designing real systems

URL Shortener
Rate Limiter
Notification System
Parking Lot
Food Delivery System
Ride Sharing System
Chat Application
YouTube
Instagram
Netflix
Payment System

For every problem, ask:

1. What are the requirements?
2. What APIs do we need?
3. How will we store the data?

4. How will the system scale?
5. Where can it fail?
6. How will we make it highly available?
7. What happens when traffic increases 10x?

And one important thing...

Don't spend 3 months just watching System Design videos.

Watching someone design a system ≠ knowing how to design one yourself.

Watch a concept.

Understand it.

Then close the video.

Pick a problem.

And try designing it yourself.

That's where the real learning happens.

Keeping this in mind, I went deep and documented everything into a System Design Interview Guide.
Get the Guide here:

https://lnkd.in/dsWQkfy4

Until the next post...

Stay Curious. Keep Coding.

MEDIA advertisement

LINKS

- https://lnkd.in/dsWQkfy4

#212 Vishal Singh

System Design

Applied AI Engineer & Developer Advocate • I Build Custom AI Voice Agents for Businesses • Helping 28K+ Devs Learn AI • Open to DevRel & AI Roles • Exploring Physical AI • DM for AI Builds & Paid Technical Collabs

1mo · **View on LinkedIn**

If I had 6 months to master System Design.

I'd do this.

Stage 1: Networking and Protocol Foundations

HTTP/HTTPS, TCP vs UDP, DNS, TLS handshakes, latency numbers every engineer should memorize.

Stage 2: Single-Node Fundamentals

Processes vs threads, memory hierarchy, disk I/O, concurrency, connection pooling.

Stage 3: API Design

REST, GraphQL, gRPC, pagination, versioning, rate limiting, idempotency keys.

Stage 4: Databases and Data Modeling

SQL vs NoSQL tradeoffs, schema design, normalization, indexing, transactions, ACID.

Stage 5: Caching Layers

Redis, cache-aside vs write-through, invalidation strategies, CDNs, edge caching.

Stage 6: Scaling Fundamentals

Vertical vs horizontal, load balancing, stateless design, replication, read replicas.

Stage 7: Sharding and Partitioning

Consistent hashing, hot spot handling, rebalancing, cross-shard queries.

Stage 8: Queues and Event-Driven Architecture

Kafka, pub/sub, backpressure, consumer lag, saga pattern, eventual consistency.

Stage 9: Distributed Systems Theory

CAP and PACELC, consensus (Raft), leader election, clocks, ordering, quorum.

Stage 10: Reliability Patterns

Circuit breakers, retries with backoff + jitter, failover, disaster recovery, chaos testing.

Stage 11: Observability and Capacity Planning

Distributed tracing, metrics, SLOs, error budgets, back-of-the-envelope estimation.

Stage 12: Practice on Real Systems

Design a URL shortener, news feed, chat system and an LLM inference gateway. Write the docs. Defend the tradeoffs.

Most people stay stuck watching tutorials.

Builders get hired.

(Bookmark & Repost)

I share tech jobs, project demos, and AI breakdowns on x.com/vishalsingh2972

LINKS

- http://x.com/vishalsingh2972

#213 Java Developers Community (moderated)

Java / Spring

Dnyaneshwar Ghodake • 3rd+

1mo · **View on LinkedIn**

(!) This Java statement looks thread-safe:

volatile int count = 0;

count++;

But it isn't.

This is one of the most common misunderstandings about "volatile".

What does volatile actually give us?

Two important guarantees:

✓ Visibility

When one thread writes a new value, another thread reading the volatile field can observe that write according to the Java Memory Model.

✓ Ordering

A volatile write establishes a happens-before relationship with subsequent reads of that same field.

But what it doesn't give:

✗ Atomicity for compound operations

Consider:

count++;

Conceptually, this involves:

READ count

ADD 1

WRITE count

Two threads can do this:

Thread 1 → reads 10

Thread 2 → reads 10

Thread 1 → writes 11

Thread 2 → writes 11

Expected: 12

Actual: 11

So what should we use?

For a counter:

AtomicInteger count = new AtomicInteger();

count.incrementAndGet();

For a critical section:

synchronized

Or an appropriate Lock when more control is required.

Where does volatile make sense?

When multiple threads need to observe a shared state change, but we don't need a compound read-modify-write operation.

For example:

private volatile boolean shutdownRequested;

One thread can set:

shutdownRequested = true;

while another thread observes the change and stops processing.

What about Spring Boot?

This becomes relevant because Spring commonly manages singleton beans.

If a singleton bean contains mutable state accessed by multiple threads, we must think about Java concurrency—not assume that Spring automatically makes the state thread-safe.

For simple runtime state where visibility is the requirement, "volatile" can be appropriate.

But if multiple operations must happen atomically, we need something stronger:

AtomicInteger / AtomicLong

synchronized

Lock

Concurrent collections

The key lesson:

Volatile ≠ mutual exclusive access.

Think:

Volatile → Visibility + Ordering

Atomic → Atomic operations

synchronized → Mutual exclusion + visibility

Lock → Explicit locking control

Understanding this difference becomes critical when building high-concurrency Java and Spring Boot applications.

What would you choose for a shared counter in a high-throughput Spring Boot service — "AtomicInteger", "synchronized", or "LongAdder"? Why?

#Java #CoreJava #Concurrency #JavaMemoryModel #JVM #SpringBoot #Multithreading #ThreadSafety #BackendEngineering #SoftwareEngineering #HighPerformance

MEDIA advertisement

#214 **nishant lakhara** **Java / Spring**

Staff Engineer at Walmart | Software Architecture, Distributed Systems, Core Java, Microservices & AWS | AI & GenAI Practitioner (IIT Kharagpur) | Ex-Morgan Stanley, Nomura & Apple

1mo · **View on LinkedIn**

Java Concurrency — CAS (Compare-And-Swap)

Starting a series of posts exploring Java Concurrency, focusing on what happens underneath the APIs we use every day.

Let's start with CAS — Compare-And-Swap, one of the fundamental mechanisms behind lock-free/ concurrent programming in Java.

Consider:

counter++;

It looks like a single operation, but conceptually it is:

Read → Modify → Write

With multiple threads, this can lead to a lost update:

Thread A: Read 10

Thread B: Read 10

Thread A: Write 11

Thread B: Write 11

Final value = 11 ✗

Expected = 12

CAS solves this differently.

A CAS operation essentially says:

"Update the value only if it is still the value I originally observed."

CAS(memory, expected, newValue)

If another thread has already changed the value, CAS fails instead of overwriting that update.

How AtomicInteger.incrementAndGet() uses CAS

Conceptually, it works like a retry loop:

```
while (true) {
int current = get();
int next = current + 1;
if (compareAndSet(current, next)) {
return next;
}
}
```

If two threads read 10:

Thread A → CAS(10 → 11) ✓

Thread B → CAS(10 → 11) ✗

↓

retry

↓

read 11

↓

CAS(11 → 12) ✓

Final value:

10 → 11 → 12

No update is lost.

Why CAS is interesting

CAS provides atomicity without traditional locking.

That means a thread doesn't need to acquire a lock and wait for another thread to release it.

However, CAS isn't automatically faster than locks. Under high contention, repeated CAS failures and retries can become expensive.

CAS is used extensively in Java concurrency, including mechanisms behind:

AtomicInteger

AtomicLong

AtomicReference

LongAdder

Parts of ConcurrentHashMap

The visual below breaks down the CAS operation, the incrementAndGet() retry loop, and what happens when multiple threads update the same value concurrently.

I'll continue exploring Java concurrency, multithreading, JVM internals, and distributed-system concepts through practical examples and visual explanations.

Follow me for more: https://lnkd.in/gBzc4qXi

#Java #JavaConcurrency #Multithreading #Concurrency #CAS #AtomicInteger #JVM #BackendEngineering #SoftwareEngineering #DistributedSystems #Programming #JavaDeveloper

MEDIA advertisement

LINKS

- https://lnkd.in/gBzc4qXi

#215 nishant lakhara

Java / Spring

Staff Engineer at Walmart | Software Architecture, Distributed Systems, Core Java, Microservices & AWS | AI & GenAI Practitioner (IIT Kharagpur) | Ex-Morgan Stanley, Nomura & Apple

1mo · **View on LinkedIn**

Java Concurrency | Why does HashMap allow null, but ConcurrentHashMap doesn't?

At first, it feels like an arbitrary Java design decision.

But there's a good reason — concurrency and ambiguity.

With HashMap:

- map.put("A", null);
- map.get("A"); // null
- map.get("B"); // null

You can use containsKey() to distinguish:

- map.containsKey("A"); // true
- map.containsKey("B"); // false

But imagine the same thing in a concurrent environment.

One thread does:

- map.containsKey("A");

Another thread removes "A".

- map.remove("A")

Then the first thread does:

- map.get("A");

What does null mean?

→ Was the value actually null?

→ Or was the key removed between the two operations?

There's no way to know.

So ConcurrentHashMap takes a simple approach:

- null means "key is absent" — and that's it.

This also makes atomic operations such as:

- putIfAbsent()
- computeIfAbsent()

much easier to reason about.

A small Java API difference, but a great example of how concurrency influences API design.

What other Java design decisions look arbitrary at first, but make sense once you understand the underlying problem?

Follow me for more such content: https://lnkd.in/gWtjb83W

#Java #Concurrency #HashMap #ConcurrentHashMap #SoftwareEngineering #Programming

LINKS

- https://lnkd.in/gWtjb83W

#216 Anuj Sharma

Kafka / Messaging

VP Technology | Engineering Leadership & Hands-On Architecture | Java/Spring Boot, .NET/C#, Microservices | Scaling Teams, Modernizing Legacy Systems | Banking & Beyond

1mo · **View on LinkedIn**

Apache Kafka — The Backbone of Real-Time Data Streaming

In modern distributed systems, applications generate millions of events every second. Apache Kafka helps organizations publish, store, process, and consume these events reliably at scale.

Simple Kafka Architecture:

Producer → Kafka Cluster → Consumer

Producer — Publishes events/messages

Topic — Logical category for events

Partition — Enables scalability & parallel processing

Broker — Stores and serves messages

Consumer — Reads and processes events

Offset — Tracks a record's position

Consumer Group — Allows consumers to work together

Replication — Provides fault tolerance

Why Kafka?

✓ High throughput

✓ Horizontal scalability

✓ Fault tolerance

✓ Durable event storage

✓ Real-time streaming

✓ Loose coupling between services

Common use cases:

• Event-driven microservices

• Real-time analytics

• Log aggregation

• Event sourcing

• Data pipelines

• Monitoring & alerting

• Stream processing

One important architectural lesson:

«Kafka is not just a messaging system—it can become the backbone of an event-driven architecture.»

The key is designing topics, partitions, consumer groups, replication, retention, and schemas carefully based on business requirements.

What Kafka concept do you think developers struggle with the most—Partitions, Consumer Groups, Offsets, or Replication? ↓

#ApacheKafka #Kafka #EventDrivenArchitecture #Microservices #DataEngineering #DistributedSystems #RealTimeStreaming #Java #SpringBoot #SoftwareArchitecture #DevOps #SoftwareEngineering

#217 Neo Kim Other

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

1mo · **View on LinkedIn**

If you want to get god-level with coding,
learn these 6 golden rules:
1 Separation of Concerns (SOC):
✓ Break down a complex program into smaller units.
✓ Each unit should focus on a specific task.
===
2 Document Your Code (DYC):
✓ Write code for your future self and others.
✓ Explain complex code sections with comments and documentation.
===
3 Don't Repeat Yourself (DRY):
✓ Don't waste time writing the same code again.
✓ Instead use functions, modules, and existing libraries.
===
4 Keep It Simple, Stupid (KISS):
✓ Simple is hard, but better.
✓ Readable code > clever code.
===
5 Test Driven Development (TDD):
✓ Write a failing test first.
✓ Write code to make the test pass.
✓ Then clean up the code without changing behaviour.
===
6 You Ain't Gonna Need It (YAGNI):
✓ Build only essential features.
✓ Don't build features you think you might need later.
===
The Bottom Line:
✓ Leave the codebase cleaner than you found it.
Anything else you'd add?
===
PS - Want my System Design Playbook (for free)?
Join my newsletter with 245K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others become better at coding.
Follow Neo Kim + turn on notifications.

MEDIA clean code rules for software engineers; document/carousel (possible)

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#218 Virendra Beniwal

System Design

Sr DevOps Engineer @ IQVIA | IIT (BHU) | DevSecOps & Cloud Automation | Azure • Kubernetes • Docker • Terraform • CI/CD

1mo · **View on LinkedIn**

Load Balancer ≠ NGINX. Here’s the difference.

Load balancers and NGINX are often used interchangeably, but they serve different purposes in a modern application architecture.

Here’s a simple way to understand the distinction:

Load Balancer

A load balancer is primarily responsible for distributing incoming traffic across multiple servers or application instances.

• Distributes traffic across multiple backend servers

• Improves availability and scalability

• Performs health checks and enables failover

• Can operate at Layer 4 (TCP) or Layer 7 (HTTP/HTTPS)

• Helps prevent a single server from becoming a bottleneck

NGINX as a Reverse Proxy

NGINX sits between clients and backend applications and controls how requests are handled before they reach the application.

• Routes requests to the appropriate backend service

• Terminates SSL/TLS connections

• Supports caching and compression

• Handles URL rewriting and redirects

• Can also perform Layer 7 load balancing

The easiest way to remember

A load balancer decides where the request should go.

NGINX can decide how the request should be handled before it reaches the application.

That said, the distinction is not always absolute.

NGINX itself can function as a load balancer, while many dedicated load balancers can also provide reverse-proxy capabilities.

In larger architectures, they may even work together:

Client → Load Balancer → NGINX → Application Servers

The key is not to memorize the definitions, but to understand which layer is responsible for which job.

That understanding becomes especially important when designing systems for scalability, high availability, security, and performance.

#LoadBalancing #NGINX #ReverseProxy #SystemDesign #DevOps #CloudComputing #SoftwareArchitecture #Scalability

#219 Puneet Patwari

System Design

Principal Software Engineer @Atlassian| Ex-Sr. Engineer @Microsoft | Sharing insights on SW Engineering, Career Growth & Interview Preparation

1mo · View on LinkedIn

I've cleared system design rounds at Amazon, Atlassian, Salesforce, and Walmart.

One of the biggest questions junior engineers ask me is: "What if the interviewer gives me a system I've never seen before? Is there a standard way to approach it?"

Yes. Here is the 4-phase approach I use:

[1] Start with what the system must NEVER get wrong

Before drawing anything, ask: "What are the invariants?"

Example: for a stock trading system:

→ an order must not execute twice

→ a user cannot spend money they do not have

→ shares cannot appear from nowhere

→ once a trade is confirmed, the state must be durable

For Airbnb:

→ one room cannot be confirmed for two people

For payments:

→ one logical payment must not charge twice

This immediately tells you where strong consistency, transactions, idempotency, or locking might matter.

Btw, if you're preparing for Senior to Principal-level system design interviews, I've put together 90+ fundamentals like this into a guide.

You can check it out here: puneetpatwari.in

[2] Map the unknown domain to familiar building blocks

Almost every system eventually needs some version of:

→ something that accepts requests

→ something that owns and stores state

→ something that handles slow work asynchronously

→ something that makes hot reads fast

→ something that tells you when things break

That could become:

API Gateway → Order Service → Database → Queue → Workers → Cache → Monitoring

The names change.

The fundamentals usually do not.

[3] Walk one critical user journey end to end

Pick the most important action and follow it.

For stock trading:

User places BUY order.

Then ask:

→ where is authentication checked?

→ where do we verify balance?

→ who owns the order state?

→ when is money reserved?

→ what happens if matching succeeds but the DB update fails?

→ what if the request is retried?

→ what does the user see while this is happening?

This is where the architecture starts building itself.

And once the happy path is clear, deliberately break it.

Kill the service halfway.

Delay the queue.

Duplicate the request.

Lose the response.

Now you are doing system design.

[4] Only then talk about scale and tradeoffs

After correctness is clear, ask:

→ What gets hot first?

→ Which reads can be cached?

→ What can be eventually consistent?

→ Where do we need partitioning?

→ What should happen during overload?

→ What can degrade without breaking the core product?

You do not need to know every detail of stock exchanges, food delivery, healthcare, or advertising before the interview.

Ask what must never go wrong.

Map the system to familiar primitives.

Trace the critical flow.

Then pressure-test it with scale and failures.

That is the cheat code.

System design becomes much less scary once you stop trying to memorize 50 architectures and start recognizing the same fundamentals hiding inside different products.

LINKS

- http://puneetpatwari.in

#220 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

1mo · **View on LinkedIn**

If your system keeps breaking as you scale... it’s not your code, it’s your tech stack.

Engineers usually focus on features, not foundations.

But at scale, architecture decisions matter more than anything you write in your editor.

This visual breaks down the core tech stack every system designer must understand - from storage to load balancing to message queues.

Here’s a simple breakdown anyone can follow:

Cloud Platforms

Where your system lives - AWS, Azure, GCP, DigitalOcean. The foundation that powers compute, storage, and global deployments.

Storage & File Systems

Services like Amazon S3, MinIO, HDFS, and Ceph store massive volumes of data reliably and cheaply.

Load Balancing

Nginx, Envoy, HAProxy, and Traefik distribute traffic so no server collapses under pressure.

Caching Layer

Redis, Memcached, Cloudflare Cache, and Fastly keep your system fast by reducing expensive reads.

Databases (SQL & NoSQL)

SQL: PostgreSQL, MySQL, MariaDB

NoSQL: MongoDB, Cassandra, DynamoDB, Couchbase

Choose based on consistency, scale, and access patterns.

Message Queues & Streaming

Kafka, RabbitMQ, Amazon SQS, Google Pub/Sub help systems communicate asynchronously and handle huge volumes.

Microservices & Orchestration

Docker, Kubernetes, Istio, Linkerd manage deployment, scaling, and service communication.

Monitoring & Logging

Prometheus, Grafana, Loki, ELK Stack track system health before things break.

CDN & Edge Computing

Cloudflare, Akamai, AWS CloudFront, Fastly deliver content ultra-fast across the world.

Distributed Systems Algorithms

Gossip, Paxos, Raft, Consistent Hashing - the backbone of reliable distributed architectures.

System design isn’t about buzzwords, it’s about choosing the right tool for the right job.

Master this tech stack, and you stop putting out fires... and start building systems that scale effortlessly.

LINKS

- https://www.youtube.com/@learnwithrockybhatia

#221 Sahilkumar Prasad

System Design

Software Development Engineer | Java & Spring Boot | REST APIs | PostgreSQL | ERP Integrations

1mo · **View on LinkedIn**

More application threads can improve throughput.

But if those threads are all waiting for database connections, adding more threads won't help.

This is where connection-pool sizing becomes important.

Consider a Spring Boot application:

Request

-> Tomcat threads

-> Service

-> Repository

-> HikariCP

-> PostgreSQL

Suppose:

200 application threads

HikariCP max pool size = 20

PostgreSQL can efficiently handle the workload

Only 20 requests can hold a database connection at the same time.

The other requests may simply wait for a connection.

Now imagine increasing HikariCP from 20 to 100.

That doesn't automatically mean 5× more throughput.

The database now has to handle more concurrent connections, which can increase:

CPU contention

memory usage

lock contention

context switching

query concurrency

So the right question isn't:

"How large should my connection pool be?"
It's:
"How much database concurrency can my system actually handle?"
There are several factors to consider:
Database CPU and memory
Query execution time
Number of application instances
Maximum database connections
Traffic patterns
Connection usage per request
Whether the workload is read-heavy or write-heavy
There's another important detail with multiple instances.
If you run:
10 application instances × 20 HikariCP connections
you could potentially have:
200 database connections.
The pool size configured in one application instance is not the total pool size of the system.
This is why connection-pool sizing needs to be considered at the architecture level, not just inside application configuration.
For larger deployments, connection proxies/poolers such as PgBouncer can also become useful depending on the workload and PostgreSQL architecture.
The goal isn't to maximize the number of connections.
The goal is to find the concurrency level where:
application throughput + database capacity + resource utilization remain balanced.
More connections ≠ more performance.
The database is still part of the system.
#Java #SpringBoot #BackendEngineering #PostgreSQL #PerformanceOptimization

#222 Parikh Jain Other

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

1mo · **View on LinkedIn**

If you are targeting Google or Uber:
focus harder on graphs, trees, DP, and follow-up heavy questions
If you are targeting Meta:
get very sharp at arrays, strings, two pointers, windows, and trees
If you are targeting Amazon:
do not neglect heaps, graphs, trees, and optimization
If you are targeting TCS Prime, Infosys SP, or Infosys DSE:
build strong fundamentals first, then push into the patterns that repeat most often
The biggest mistake in interview prep is treating all practice as equally valuable. It is not.
Ten well-chosen problems from the right pattern will help you more than fifty random ones.
P.S: This is the exact loop 1500+ students are running inside the SDE Career Tracker (https://lnkd.in/gtzyBHkw).
You get: Pattern-targeted DSA, LLD, HLD, GenAI basics, a Resume Analyzer that rewrites your resume, scored mock interviews, a live Job-Ready Score,
and the best part: job assistance with 200+ partner companies plugged in for referrals.

LINKS
- https://www.propeers.in/digitalProducts
- https://lnkd.in/gtzyBHkw

#223 Java Developers Community (moderated)

Java / Spring

Vahid Hasani • 3rd+
1mo · View on LinkedIn

Java Streams — The Mental Model
Most Java developers use Streams every day.
But using:
stream()
.filter(...)
.map(...)
.collect(...)
is very different from understanding what actually happens.
In Part 1 of Java Internals, I focused on the mental model behind Streams:
• A Stream is not a Collection.
• Intermediate operations are lazy.
• Terminal operations trigger execution.
• Elements can flow vertically through the pipeline.
• Stateful operations may require memory.
• Short-circuiting can stop traversal early.
• Parallel Streams divide work — they don't simply create one thread per element.
• Lambdas describe behavior; the pipeline controls execution.
The goal isn't to memorize the Stream API.
It's to build a mental model that lets you predict behavior.
That becomes especially important when you're debugging:
→ unexpected performance
→ memory pressure
→ parallel execution
→ unnecessary work
→ side effects
→ complex pipelines
This is Part 1 — The Mental Model of my Java Internals series, based on Java 21 LTS.
Part 2 will go one level deeper:
What actually happens when an element reaches filter(), map(), flatMap(), distinct(), sorted(), limit() and skip()?
If you've used Streams for years, the interesting question isn't:
"How do I use Streams?"
It's:
"What is actually happening underneath?"
#Java #Java21 #JavaStreams #JVM #BackendDevelopment #SoftwareEngineering #JavaDeveloper #Programming #Performance #SystemDesign
1/15

MEDIA document/carousel (possible)

#224 Kubernetes

System Design

Pawan C. • 3rd+

1mo · View on LinkedIn

“Autoscaling Cannot Fix Inefficient Architecture.”
Autoscaling sounds like the answer to almost every growth problem.
Traffic increases
Add replicas
CPU rises
Add more nodes
Demand drops
Scale back down ✓
→ The system looks intelligent.
(!) But here’s the real question:
→ What if we are simply scaling an inefficient architecture?
What Autoscaling Often Hides
A service consumes too much memory...
so we add more replicas.
But the real issue is inefficient application behaviour.
Kubernetes nodes keep scaling out...
but workloads have poorly defined requests and limits.
Traffic increases...
so more application instances are created.
But every request generates expensive database calls.
A queue grows...
so workers scale automatically.
But retries are multiplying the workload.
Cloud cost increases...
and autoscaling is blamed.
But the real problem started with architecture, sizing or workload design.
→ Autoscaling solves capacity pressure.
→ It does not automatically solve architectural inefficiency.
What Teams Should Understand Before Scaling
Before increasing capacity, ask:
→ Is the workload correctly sized?
→ Are CPU and memory requests realistic?
→ Is the bottleneck compute, database, network or storage?
→ Are retries creating unnecessary load?
→ Is caching being used effectively?
→ Are services communicating efficiently?
→ What does each additional replica cost?
→ Does scaling improve performance proportionally?
→ Is the architecture creating unnecessary cloud spend?
The goal should not be:
“Scale whenever utilisation rises.”
It should be:

“Understand why demand is rising, then scale the right layer.”

What We’re Building at CrftInfrai

CrftInfrai connects:

Application requirements

→ Architecture planning

→ Kubernetes platform design

→ Resource sizing

→ Multi-cloud cost comparison

→ Terraform generation

→ Architecture validation

→ Operational signals

→ AI-assisted diagnosis

The goal is to help teams reason about performance, architecture and cost together before simply adding more infrastructure.

https://crftinfrai.com

https://lnkd.in/dPHgduiv

The Shift

From:

→ “CPU is high. Add more infrastructure.”

To:

→ “Understand the bottleneck, validate the architecture, then scale the component that actually needs it.”

Right-size first.

Optimize second.

Autoscale where it makes architectural sense.

#Kubernetes #Autoscaling #PlatformEngineering #FinOps #CloudCost #CloudArchitecture #DevOps #SRE #AgenticAI #CrftInfrai

LINKS

- https://crftinfrai.com
- https://lnkd.in/dPHgduiv
- https://www.linkedin.com/feed/update/urn:li:activity:7497116301937700864

ALSO SAVED AS DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef (1mo) link

#225 Rakesh Gohel

AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

1mo · **View on LinkedIn**

Anthropic just launched Claude Academy, a free platform for using and building with Claude. No credit card, self-paced, most with completion badges.

I grouped the 15 courses by level so you know where to start:

BEGINNER

1. Claude 101: https://lnkd.in/gcBiRPe7
2. Claude Code 101: https://lnkd.in/g7ENig6N
3. Introduction to Claude Cowork: https://lnkd.in/gJWJydRD
4. Claude Platform 101: https://lnkd.in/giz-eXbT

INTERMEDIATE
5. Claude Code in Action: https://lnkd.in/g_qVbNVw
6. Intro to MCP: https://lnkd.in/gqWVrQ-y
7. Introduction to subagents: https://lnkd.in/gqMHqHNr
8. Introduction to agent skills: https://lnkd.in/gXejksbM
FLUENCY
9. AI Fluency: Framework & Foundations: https://lnkd.in/gGiGUBjp
10. AI Capabilities and Limitations: https://lnkd.in/gzt_7xjF
11. AI Fluency for Builders: https://lnkd.in/g2znfCfH
ADVANCED
12. Advanced MCP: https://lnkd.in/gyCwA2ei 13. Building with the Claude API: https://lnkd.in/gr4C-BNM 14. Claude with Amazon Bedrock: https://lnkd.in/gP-3h66w 15. Claude with Vertex AI: https://lnkd.in/gx9HKmaC
Full catalog: https://lnkd.in/gQqvb_TG
Which course you like the most, why ?
For more AI agent insights, subscribe to newsletter: https://lnkd.in/gatGAt9X
Grab your copy → The Human-Agent Orchestrator: https://amzn.to/4gEIzmv
Save → React → Share & follow for everything related to AI Agents
#AIAgents #AgenticAI #GenerativeAI #EnterpriseAI #ClaudeAI #Claudeacademy

MEDIA 15 Claude Courses fo AI Builders

LINKS

- https://lnkd.in/gcBiRPe7
- https://lnkd.in/g7ENig6N
- https://lnkd.in/gJWJydRD
- https://lnkd.in/giz-eXbT
- https://lnkd.in/g_qVbNVw
- https://lnkd.in/gqWVrQ-y
- https://lnkd.in/gqMHqHNr
- https://lnkd.in/gXejksbM
- https://lnkd.in/gGiGUBjp
- https://lnkd.in/gzt_7xjF
- https://lnkd.in/g2znfCfH
- https://lnkd.in/gyCwA2ei
- https://lnkd.in/gr4C-BNM
- https://lnkd.in/gP-3h66w
- https://lnkd.in/gx9HKmaC
- https://lnkd.in/gQqvb_TG
- https://lnkd.in/gatGAt9X
- https://amzn.to/4gEIzmv

#226 Deepak Kumar Kesharwani

Java / Spring

Engineering Lead @ Persistent Systems | Gen AI, Agentic AI & AI Automation Expert | Java Full Stack Architect | React, Angular, Spring Boot Microservices | Ex @ Infosys, Wipro

2mo · **View on LinkedIn**

Just came out of my Karat interview — and honestly, it was nothing like I expected.
They didn't ask me to reverse a linked list.
They didn't test if I memorized Java syntax.
Instead, they tested whether I *understand* Java.
Things like:
→ Why mutable Map keys are a silent killer
→ Why checked exceptions break Lambdas

→ How to spot a NullPointerException trap in 3 seconds

→ And a real production debugging challenge

I made an infographic breaking down all 5 questions with the answers + the KEY insight behind each one.

Save this if you have a Karat interview coming up.

#Java #KaratInterview #SoftwareEngineering #InterviewPrep #JavaDeveloper #CleanCode #Bangalore #IndianDevelopers

Java Developer Interview Ex

Subscribe

Karat Interview Experience! Here are the 5 Java Questions They Actually Asked Me

Deepak Kumar Kesharwani

MEDIA Karat Interview Experience! Here are the 5 Java Questions They Actually Asked Me

#227 Kubernetes

AI / LLM

Pawan C. • 3rd+

2mo · **View on LinkedIn**

“AI Cost Is Becoming a Platform Engineering Problem.”

Until recently, platform teams mostly worried about:

Compute cost

Storage cost

Network cost

Kubernetes cost

Database cost

→ Now another infrastructure bill is growing:

AI usage.

Input tokens.

Output tokens.

Cached tokens.

Batch processing.

Realtime inference.

Different models.

Different providers.

Different service tiers.

(!) But here’s the real question:

→ Are teams choosing AI models based only on capability—or also understanding the infrastructure economics behind that choice?

What AI Cost Decisions Often Miss

A model looks inexpensive per 1M tokens...

but output-heavy workloads can change the economics completely.

Another model has a higher headline price...

but caching may reduce repeated-input cost.

Batch processing looks slower...

but could be much cheaper for non-interactive workloads.

Realtime inference is selected everywhere...

even when the workload does not require realtime response.

A cheaper model is chosen...
but lower quality creates more retries and larger prompts.
AI usage grows...
but engineering teams cannot easily connect model choice to monthly application cost.
→ Token pricing alone does not explain AI economics.
→ Workload behaviour determines the real cost.
What AI Cost Comparison Should Understand
Teams should be able to compare:
→ Model provider
→ Exact model
→ Input-token pricing
→ Cached-input pricing
→ Output-token pricing
→ Expected request volume
→ Average prompt size
→ Average response size
→ Cache utilisation
→ Latency requirement
→ Model lifecycle and release stage
→ Whether the model is recommended for new workloads
Then answer the question that actually matters:
“Which model and service tier gives this workload the right balance of cost, performance and capability?”
What We’re Building at CrftInfrai
CrftInfrai is bringing AI cost intelligence alongside cloud infrastructure decisions.
Teams can compare:
Cloud infrastructure cost
→ AWS
→ Azure
→ GCP
→ OCI
And increasingly understand:
AI model cost
→ Provider
→ Model
→ Input / cached input / output pricing
→ Service tier
→ Workload economics
The longer-term goal is to connect these two decisions.
Because an AI application does not have just an AI bill.
It has:
Model cost + compute + storage + networking + Kubernetes + databases + operations.
That complete view is where platform engineering and FinOps start converging.
Explore CrftInfrai: https://crftinfrai.com
Try the platform: https://lnkd.in/dPHgduiv
The Shift

From:

→ “Which AI model has the cheapest token price?”

To:

→ “Which model, service tier and infrastructure architecture gives this workload the best total economics?”

AI cost should not be reviewed after the bill arrives.

It should be part of the architecture decision before the workload scales.

#ArtificialIntelligence #FinOps #PlatformEngineering #AICost #CloudCost #LLM #DevOps #CloudComputing #AgenticAI #AIInfrastructure #CrftInfrai

LINKS

- https://crftinfrai.com
- https://lnkd.in/dPHgduiv

#228 **Neo Kim** System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

If you want to become GOAT-level with system design,

learn these 21 concepts (save now):

1 System Design Concepts 101

↳ https://lnkd.in/eK87NkE5

2 Microservices Patterns

↳ https://lnkd.in/ehtUCF4R

3 How DNS Works

↳ https://lnkd.in/e8H3EGvp

4 How JWT Works

↳ https://lnkd.in/ek9_BTUc

5 How Does HTTPS Work

↳ https://lnkd.in/enM_pZYd

6 API Design Best Practices

↳ https://lnkd.in/eUEFzEC8

7 Redis Use Cases

↳ https://lnkd.in/ekJMjMG3

8 Distributed Systems 101

↳ https://lnkd.in/eZCC7KPx

9 How Message Queues Work

↳ https://lnkd.in/ezesi5N4

10 How Websockets Work

↳ https://lnkd.in/euHDJUfH

11 Frontend System Design Concepts

↳ https://lnkd.in/ehHuXpjx

12 How Databases Keep Passwords Securely

↳ https://lnkd.in/edd8-N5k

13 Modular Monolith Architecture

↳ https://lnkd.in/edUqkCKR

14 Saga Design Pattern

↳ https://lnkd.in/eFXC4-aJ

15 Microservices Lessons from Netflix
↳ https://lnkd.in/eZSM3CRB
16 How Consistent Hashing Works
↳ https://lnkd.in/eUP9DbCg
17 How Idempotent API Works
↳ https://lnkd.in/erMkqwq4
18 How RPC Actually Works
↳ https://lnkd.in/eScZS9F5
19 API Versioning - A Deep Dive
↳ https://lnkd.in/eg_fmBhu
20 How Bloom Filters Work
↳ https://lnkd.in/e2duES7s
21 How Service Discovery Works
↳ https://lnkd.in/eCYYwQfU
Anything else you'd add?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 245K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others ace system design.
Follow Neo Kim + turn on notifications.

MEDIA 21 Concepts to learn system design for software engineers

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eK87NkE5
- https://lnkd.in/ehtUCF4R
- https://lnkd.in/e8H3EGvp
- https://lnkd.in/ek9_BTUc
- https://lnkd.in/enM_pZYd
- https://lnkd.in/eUEFzEC8
- https://lnkd.in/ekJMjMG3
- https://lnkd.in/eZCC7KPx
- https://lnkd.in/ezesi5N4
- https://lnkd.in/euHDJUfH
- https://lnkd.in/ehHuXpjx
- https://lnkd.in/edd8-N5k
- https://lnkd.in/edUqkCKR
- https://lnkd.in/eFXC4-aJ
- https://lnkd.in/eZSM3CRB
- https://lnkd.in/eUP9DbCg
- https://lnkd.in/erMkqwq4
- https://lnkd.in/eScZS9F5
- https://lnkd.in/eg_fmBhu
- https://lnkd.in/e2duES7s
- https://lnkd.in/eCYYwQfU
- https://lnkd.in/ehTrcyak

#229 **Yash Vardhaan** Other

SDE-3 || PeopleStrong(Goldman Sachs Company) ||Spring Boot,Java,Kafka,Redis,Microservices,Rest APIs

2mo · View on LinkedIn

10:00 AM. Tatkal opens. Millions of users hit “Book Now.”

How would you design IRCTC to handle the traffic without selling the same seat twice?

My simplified HLD ↓

#SystemDesign #BackendEngineering #SoftwareEngineering #IRCTC

#230 DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef

Cloud / K8s / DevOps

Neel Shah • 3rd+

2mo · View on LinkedIn

DNS is invisible... until it breaks.

When Kubernetes services suddenly stop communicating, DNS is often the silent culprit.

CoreDNS → Config → Service → NetworkPolicy → CNI

Know the flow. Troubleshoot faster. Fix smarter.

Repost if this changed how you think about Kubernetes troubleshooting

Follow Neel Shah for more DevOps & Cloud insights

#Kubernetes #DevOps #CloudNative #SRE #CoreDNS #CloudComputing

MEDIA advertisement

LINKS

- https://sessionize.com/neel-shah/

#231 Sahilkumar Prasad

Java / Spring

Software Development Engineer | Java & Spring Boot | REST APIs | PostgreSQL | ERP Integrations

2mo · View on LinkedIn

More threads don't always mean more performance.

When a backend application becomes slow under load, one common reaction is:

"Increase the thread pool size."

Sometimes that helps.

Sometimes it makes the problem worse.

Consider a Spring Boot application handling requests like this:

Request

→ Thread

→ Database call

→ Response

Now imagine the application has 200 request threads, but the database connection pool has only 20 connections.

Most threads may end up waiting for a database connection.

Increasing the thread pool to 500 doesn't magically create more database capacity.

It can actually increase:

- Context switching
- Memory usage
- Contention
- Queueing

• Pressure on downstream systems

The same principle applies when calling external services.

If the downstream service can handle only a limited amount of concurrent traffic, continuously increasing concurrency can turn a bottleneck into a larger failure.

That's why performance tuning should start with:

Measure → Find the bottleneck → Understand the resource limits → Tune → Measure again.

Thread pools, database connection pools, CPU, memory, network connections and downstream services all have capacity limits.

A fast backend isn't created by simply increasing concurrency.

It's created by finding the right level of concurrency for the entire system.

#Java #SpringBoot #BackendEngineering #Concurrency #PerformanceOptimization

#232 Nafiul Islam

System Design

Senior Software Engineer | Building LLM-Integrated Production Systems at Scale — RAG, Multi-Agent, Guardrails | Event-Driven Architecture (Java · Kafka · K8s · AWS · LangChain4j)

2mo · **View on LinkedIn**

What Actually Happens When 10,000 Users Hit Your Spring Boot API?

Your API works perfectly with 10 users.

Then suddenly, 10,000 users hit it at the same time.

What happens next?

This is where writing a REST API and engineering a scalable system become two very different things.

1. Requests Hit the Load Balancer

The traffic usually doesn't go directly to one Spring Boot instance.

A load balancer distributes requests across multiple application servers.

10,000 requests → Load Balancer → Multiple Spring Boot instances

2. Tomcat Starts Handling Requests

Each Spring Boot instance has a limited number of request-processing threads.

If all threads are busy, new requests start waiting.

And suddenly, your "fast" API isn't so fast anymore.

3. Your Database Connection Pool Gets Tested

Your application doesn't create an unlimited number of database connections.

With thousands of concurrent requests, threads may start waiting for an available connection.

This can become one of your biggest bottlenecks.

4. The Database Gets Overloaded

Now imagine thousands of requests executing queries at the same time.

More connections ≠ unlimited database capacity.

Poor queries, missing indexes, N+1 queries, and unnecessary database calls can make things much worse.

5. Caching Can Save You

If thousands of users request the same data, hitting the database every time is unnecessary.

A properly designed Redis cache can absorb a large portion of those reads and reduce database pressure.

6. Some Requests Will Be Slow

Maybe one API calls another service.

Maybe a database query takes too long.

Maybe a thread is blocked by an expensive operation.

One slow dependency can create a chain reaction across your system.

7. You Need to Scale

At this point, you may need:

More application instances → Better load balancing → Proper connection pools → Caching → Database optimization → Monitoring

And this is why scalability isn't simply:

"Let's add more servers."

You need to understand where the bottleneck actually is.

The interesting part is that your Spring Boot code might not change at all.

The same:

Controller → Service → Repository

that worked perfectly for 100 users can behave very differently when 10,000 users arrive simultaneously.

10 users test your functionality.

10,000 users test your architecture.

MEDIA advertisement

#233 Dnyaneshwar Ghodake

System Design

Senior Java Backend Developer | 9+ Years | Java, Spring Boot, Spring Batch | Core Banking & Financial Systems | High-Volume Data Processing | REST APIs | Kafka | Docker | Kubernetes | AWS | Immediate Joiner

2mo · View on LinkedIn

How do you make a Java API handle 10K+ requests per second?

The first answer shouldn't be:

✗ Increase the thread pool

✗ Increase JVM heap

✗ Add @Async everywhere

✗ Buy a bigger server

The real answer?

10K+ RPS is not just a Java problem.

It's a system-design problem.

For a high-traffic Spring Boot API, I'd look at the entire request path:

Client → Load Balancer → API Gateway → Stateless Spring Boot instances → Cache/DB → Async processing

What matters most?

1. Keep the API stateless

Don't keep application state inside a particular instance.

Stateless services make horizontal scaling much easier:

1 instance → 5 instances → 20 instances

2. Optimize the database

A fast API won't help if every request waits on a slow query.

Think about:

• Proper indexes

• Query optimization

• Connection pool sizing

• Avoiding N+1 queries

• Pagination
• Read replicas where appropriate
3. Cache wisely
Redis can reduce database pressure for frequently accessed data.
But caching introduces its own challenges:
TTL + invalidation + consistency + cache stampede
4. Asynchronize where appropriate
If work doesn't need to complete before the response, consider:
API → Kafka → Consumer → Heavy Processing
This keeps the API responsive while downstream processing happens independently.
5. Control concurrency
More threads ≠ more throughput.
Too much concurrency can cause:
• CPU contention
• Context switching
• Connection pool exhaustion
• GC pressure
• Higher latency
The goal is the right concurrency level for the workload.
6. Protect the system
At high traffic, resilience matters as much as throughput.
Rate limiting, timeouts, circuit breakers, bulkheads, retries with backoff and backpressure all become important.
7. Measure what matters
Don't look only at average latency.
Watch:
p50 → p95 → p99
Along with throughput, error rate, CPU/memory, GC pauses, DB connections, cache hit ratio and Kafka lag.
And most importantly:
Load test before production.
10K RPS in an architecture diagram means very little if the system starts failing at 6K RPS under realistic load.
The bottom line:
10K+ RPS isn't achieved through one Java optimization.
It's the result of designing the entire system so that every layer can scale without becoming the bottleneck.
And remember:
10K lightweight GET requests/sec ≠ 10K financial transactions/sec.
Different workloads. Different bottlenecks.
#Java #SpringBoot #SystemDesign #Microservices #BackendEngineering #SoftwareArchitecture #Scalability #DistributedSystems #Kubernetes #Kafka

#234 Himanshu Sharma Java / Spring

Ex Software Engineer @ Sirion.ai | GenAI, LLM & RAG Systems | SaaS | NIT Kurukshetra

2mo · View on LinkedIn

How Spring Automatically Finds Implementations

Have you ever wondered:

“If I have an interface with multiple implementation classes, can Spring automatically find them and create beans?”

Yes! And this becomes very useful when implementing the Factory Pattern.

Imagine we have a "PaymentService" interface with multiple implementations like:

- UPI Payment

- Card Payment

- Net Banking Payment

If these classes are registered as Spring beans, Spring automatically detects them during application startup and creates their objects.

Now, instead of the Factory manually creating every object, Spring can provide all the available implementations to the Factory.

The Factory's job becomes very simple:

Spring → Creates and manages the objects

Factory → Decides which implementation to use

So when we add a new payment implementation, we don't need to change the object-creation logic inside the Factory.

This gives us a cleaner and more scalable design.

Simple flow

Interface → Multiple Implementations → Spring Beans → Factory → Select Required Implementation

This is one of the reasons why combining Spring Dependency Injection + Factory Pattern can make backend applications much easier to maintain.

#SpringBoot #Java #SpringFramework #FactoryPattern #DependencyInjection #BackendDevelopment #SoftwareEngineering

MEDIA advertisement

#235 Nishant Ojha

Java / Spring

Full-Stack Developer Enthusias || Spring boot || Java || DSA || Javascript || MySQL || DBMS || OOPs concepts

2mo · View on LinkedIn

Spring Boot Day 6 – Key Learnings

Learned about Circular Dependency and why it occurs between Spring beans.

Understood the role of Dependency Injection (DI), Inversion of Control (IoC), and Beans.

Explored how Spring manages bean creation based on dependency order.

Compared Constructor, Setter, and Field Injection in circular dependencies.

Understood why constructor-based circular dependency can cause BeanCurrentlyInCreationException.

Learned about Spring’s early reference mechanism for handling certain circular dependencies.

Studied why circular dependency is considered a design smell and how refactoring can provide a better solution.

Explored Bean Scopes: Singleton, Prototype, Request, Session, Application, and WebSocket.

Understood the difference between Singleton and Prototype scope.

Learned about Eager vs Lazy Initialization and the use of @Lazy.

Explored how @Lazy can inject a proxy and delay dependency creation.

Studied Spring Boot global lazy initialization using spring.main.lazy-initialization=true.

Learned the complete Spring Bean Lifecycle — Definition → Creation → Dependency Injection → Initialization → Ready → Destruction.

Key takeaway: Clean dependency design and clear separation of responsibilities are essential for maintainable Spring applications.

#SpringBoot #Java #SpringFramework #Learning #MCA #BackendDevelopment #SoftwareDevelopment #CodingJourney #100DaysOfCode

#236 Waseem Shaikh

Java / Spring

Java Backend Developer | Spring Boot | Microservices | REST APIs | Hibernate | Spring Batch | JPA | SQL | System Design | Enterprise Applications

2mo · View on LinkedIn

I've put together a collection of Java & SQL interview coding problems that I'm using for interview preparation and strengthening my problem-solving skills.

The collection covers:

Arrays & Collections

String-based problems

Java 8 Stream API & Functional Programming

Employee-based Stream problems

Stack, Queue & DSA problems

Multithreading & Concurrency

CompletableFuture & ConcurrentHashMap

REST API retry scenarios

Kafka duplicate message handling

Spring Boot pagination

Java OOP & Immutable Classes

SQL interview questions & queries

Some important problems include Longest Substring Without Repeating Characters, LRU Cache, Balanced Parentheses, Producer-Consumer, Nth Highest Salary, Comparator vs Comparable, map() vs flatMap(), and Pagination.

I'm sharing this as a practice resource for fellow Java developers preparing for interviews.

Consistency + hands-on coding + understanding the concepts = better interview confidence.

#Java #JavaDeveloper #SpringBoot #StreamAPI #Microservices #SQL #Coding #InterviewPreparation #SoftwareDevelopment #BackendDevelopment

Java Interview Preparation — C

.

29 pages

MEDIA Phase 1 — Arrays, Lists & Basic Collection Problems 1. Find duplicate elements in an array import java.util.*; public class Main { public static void main(String[] args) { int[] arr = {1, 2, 3, 2, 4, 1}; Set<Integer> set = new HashSet<>(); Set<Integer> duplicates = new HashSet<>(); for (int num : arr) { if (!set.add(num)) { duplicates.add(num); } } System.out.println(duplicates); } } 2. Find duplicate elements from a list using Stream API import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1); Set<Integer> set = new HashSet<>(); list.stream() .filter(n -> !set.add(n)) .forEach(System.out::println); } } 3. Remove duplicates from a list import java.util.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1); Set<Integer> set = new LinkedHashSet<>(list); System.out.println(set); } }; 4. Remove duplicates using Streams import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(1, 2, 3, 2, 4, 1); List<Integer> uniqueList = list.stream() .distinct() .collect(Collectors.toList()); System.out.println(uniqueList); } } 5. Filter even numbers from a list import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(1,2,3,4,5,6); list.stream() .filter(n -> n % 2 == 0) .forEach(System.out::println); } } 6. Filter odd numbers from a list import

java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9); List<Integer> oddNumbers = list.stream() .filter(n -> n % 2 != 0) .collect(Collectors.toList()); System.out.println("Odd Numbers: " + oddNumbers); } } 7. Sort a list in ascending order; import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(5, 2, 9, 1, 6); List<Integer> sortedList = list.stream() .sorted() .collect(Collectors.toList()); System.out.println("Ascending Order: " + sortedList); } } 8. Sort a list in descending order import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(5, 2, 9, 1, 6); List<Integer> sortedDesc = list.stream() .sorted(Comparator.reverseOrder()) .collect(Collectors.toList()); System.out.println("Descending Order: " + sortedDesc); } } 9. Find max and min numbers using Stream API import java.util.*; import java.util.stream.*; public class Main { public static void main(String[] args) { List<Integer> list = Arrays.asList(5, 2, 9, 1, 6); Integer max = list.stream() .max(Integer::compareTo) .get(); Integer min = list.stream() .min(Integer::compareTo) .get(); System.out.println("Max: " + max); System.out.println("Min: " + min); } }; document/carousel (possible)

#237 **Yogesh Kshirsagar** **Java / Spring**

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

2mo · **View on LinkedIn**

Spring Boot Under the Hood - Vol. 11 of 30

You write:

@Service

class PaymentService { }

Spring sees one annotation.

But behind the scenes?

A whole lifecycle begins.

The bean doesn't simply go:

Create → Done.

It goes through multiple stages before you can actually use it.

Instantiation

Spring creates the object.

Dependency Injection

Required dependencies are populated.

Aware & lifecycle callbacks

Spring gives the bean access to selected infrastructure and lifecycle hooks.

Initialization

@PostConstruct and other initialization callbacks execute.

BeanPostProcessor

Spring can inspect, modify or even wrap the bean.

✓ Ready

The bean is finally ready to be used.

But here's the part that makes Spring really interesting:

The object Spring creates initially may not be the object you finally receive.

Wait... what?

Then where does the proxy come from?

When does @PostConstruct run?

When are dependencies injected?

And where does @Transactional fit into this lifecycle?

That's where the Bean Lifecycle starts getting interesting.

Vol. 11 of 30 — Spring Boot Under the Hood

Follow along as we continue opening the Spring Boot hood—one internal mechanism at a time.

→ Next: Spring Proxies — Why does Spring sometimes create a proxy instead of returning your actual bean?

#SpringBoot #SpringFramework #Java #BackendEngineering #JavaDeveloper #Microservices

26

1

4

Yogesh Kshirsagar

Author

Assistant Vice President at Deutsche Bank |Senior Software Engineer | Java | SpringBoot | Microservices | Reactive Programming | Google Cloud (GCP) | Apache Beam/Dataflow | Oracle | PostgreSQL

2mo

BeanFactory Internals: https://www.linkedin.com/posts/yogesh-kshirsagar-04080014_springboot-springframework-java-activity-7494970567201234945-F7tA?utm_source=share&utm_medium=member_android&rcm=ACoAAAL3YDABrLwhcCq19IWx22YwJAGh4cG9UwQ

#springboot #springframework #java #backendengineering #javadeveloper #microservices #springinternals | Yogesh Kshirsagar

linkedin.com

MEDIA advertisement

LINKS

- https://www.linkedin.com/posts/yogesh-kshirsagar-04080014_springboot-springframework-java-activity-7494970567201234945-F7tA?utm_source=share&utm_medium=member_android&rcm=ACoAAAL3YDABrLwhcCq19IWx22YwJAGh4cG9UwQ

#238 Java Developers Community (moderated)

Other

Mohamed EL Laithy • 3rd+ | Every Java roadmap tells you what to learn. | Almost none tell you when you're actually done with a topic. | So I built one that does. | 14 stages, from Java Fundamentals to System Design. Each stage has three things: | → LEARN — the topics | → THEY ASK — the questions that actually come up in interviews | → MOVE ON WHEN — a clear signal you've got it | Example, Stage 11 (JPA & Hibernate): | You're done when you spot an N+1 in the SQL logs before it reaches production. | Not when you've watched a tutorial about it. | Full roadmap in the carousel ↓ | #Java #SpringBoot #BackendDevelopment #SoftwareEngineering | 17 pages | 66 | 1 | 16 | | Madhavan Ananthan Verified Profile 3rd+ | Madhavan Ananthan | Senior Software Engineer | 8 Years | Java 17/21, Spring Boot, REST APIs, Backend | Cloud | Microservices | Hibernate, JPA, SQL, NoSQL | Concurrency, Redis, Distributed Systems, Performance Optimization | Design Patterns

2mo · **View on LinkedIn**

Honestly one of the best resource , thanks and keep posting

MEDIA 2026M E L L A I T H Y · J A V A B A C K E N D R O A D M A P J A V A B A C K E N D D E V E L O P E R The Roadmap I Actually Interviewed On 14 stages · Zero to job ready Every stage has the topics to learn, the questions interviewers really ask, and a clear signal for when you're done with it. @mellaithymellaithy.gumroad.com; STAGE 01 / 14M E L L A I T H Y · J A V A B A C K E N D R O A D M A P 01 Java Fundamentals L E A R N Variables Primitives vs Objects Type Casting Operators Control Flow Loops Methods Arrays String StringBuilder Wrapper Classes Autoboxing I/O Basics T H E Y A S K == vs .equals() on Strings — what's actually being compared? Why is String immutable, and what does the String pool buy you? String vs StringBuilder vs StringBuffer? M O V E O N W H E N You can predict the output of s1 == s2 puzzles without running the code. @mellaithymellaithy.gumroad.com; STAGE 02 / 14M E L L A I T H Y · J A V A B A C K E N D R O A D M A P 02 OOP & Design Principles L E A R N Classes & Objects Constructors Encapsulation Inheritance Polymorphism Abstraction Interfaces vs Abstract Classes Access Modifiers static final Packages SOLID T H E Y A S K Interface or abstract class — when do you pick which? Overloading vs overriding: compile time or runtime? Give me a real Open/Closed example from your own

> code. M O V E O N W H E N You can refactor an if/else chain into polymorphism and defend why it's better. @mellaithymellaithy.gumroad.com; document/carousel (possible)

#239 Maxim Konstantinov

Java / Spring

Senior Java Engineer | Fintech & Banking | B2B Contractor | Distributed Systems | Highload | Microservices | Spring Boot · Kafka · K8s · AWS | 10M+ Users | Happy

2mo · **View on LinkedIn**

Spring Boot 4 quietly deleted a whole category of code most of us wrote by hand: the REST client.

If your service calls another service, you've got a WebClient or RestTemplate chain somewhere — build the URL, set the query params, retrieve(), map the body, wrestle the generic type, maybe .block(). Now multiply that across every endpoint you consume.

Spring Framework 7 turns the HTTP client into just an interface:

@HttpExchange("/users")

interface UserClient {

@GetExchange

List<User> byStatus(@RequestParam String status);

}

You describe the call — method, path, params, return type — and Spring generates the implementation and registers it as a bean. You inject it and call userClient.byStatus("active"). No proxy-factory wiring, no manual URL building.

It's type-safe (the compiler checks your params and return types), swappable (RestClient for blocking, WebClient for reactive — same interface), and trivially mockable in tests.

If OpenFeign is what you reached for before, this is the native, dependency-free version — and in Spring Boot 4 it's first-class.

Already moved your clients to @HttpExchange, or still on WebClient chains?

#Java #SpringBoot #Spring #BackendDevelopment

#240 Saleamlak Asrat Shiferie

System Design

Helping Business Implement Agentic AI Systems | AI Marketing & Building Agentic AI for Companies

2mo · **View on LinkedIn**

One of the best System Design writers on LinkedIn is Neo Kim.

His newsletter has helped hundreds of thousands of engineers learn distributed systems, architecture patterns, and real-world system design concepts.

If you're learning System Design, these 14 articles are a great place to start:

1. How Docker Works

https://lnkd.in/e9R-9U-g

2. Top Redis Use Cases

https://lnkd.in/ekJMjMG3

3. How Idempotent APIs Work

https://lnkd.in/erMkqwq4

4. How the Saga Design Pattern Works

https://lnkd.in/eFXC4-aJ

5. API Design Best Practices

https://lnkd.in/eUEFzEC8

6. How JWT Works

https://lnkd.in/ek9_BTUc

7. How Consistent Hashing Works
https://lnkd.in/eUP9DbCg
8. How Service Discovery Works
https://lnkd.in/eCYYwQfU
9. Capacity Planning Explained
https://lnkd.in/ev358pj3
How RPC Works
https://lnkd.in/eScZS9F5
11. How the Actor Model Works
https://lnkd.in/eqcb7MpP
12. How Apache Kafka Works
https://lnkd.in/eTtVAjTg
13. Modular Monolith Architecture
https://lnkd.in/edUqkCKR
14. How Bloom Filters Work
https://lnkd.in/e2duES7s
I've learned that system design becomes much easier when you focus on the underlying concepts before jumping into complex case studies.
Which System Design topic would you like to cover next?

LINKS

- https://lnkd.in/e9R-9U-g
- https://lnkd.in/ekJMjMG3
- https://lnkd.in/erMkqwq4
- https://lnkd.in/eFXC4-aJ
- https://lnkd.in/eUEFzEC8
- https://lnkd.in/ek9_BTUc
- https://lnkd.in/eUP9DbCg
- https://lnkd.in/eCYYwQfU
- https://lnkd.in/ev358pj3
- https://lnkd.in/eScZS9F5
- https://lnkd.in/eqcb7MpP
- https://lnkd.in/eTtVAjTg
- https://lnkd.in/edUqkCKR
- https://lnkd.in/e2duES7s

#241 **Aman Mishra** **System Design**

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

2mo · **View on LinkedIn**

"Design a URL Shortener."
The easiest System Design question, right?
I've asked this to 100+ candidates.
Less than 20% get it right.
And I keep seeing the same 5 mistakes.
1. Jumping straight into architecture
I haven't even told you the scale yet.
Maybe we only create 100 URLs/day.
You don't need Kafka.
Fix: Ask questions first:
→ How many URLs are created?

→ Do they expire?
→ Is the system read-heavy or write-heavy?
The answers completely change your architecture.
2. Using random strings for short URLs
"Generate a random 7-character string."
Okay... what happens when two requests generate the same string?
Now you need collision detection, retries, and uniqueness checks.
Fix: Use an ID + Base62 encoding.
Example:
ID: 1000 → Base62 → "g8"
The ID is unique, so the generated short code is unique by design.
3. Forgetting caching
Imagine 100M redirects/day.
That's roughly 1,200 requests/second hitting your database.
Your database will quickly become the bottleneck.
Fix: Cache frequently accessed URLs in Redis.
Hot URLs → Redis
Cold URLs → Database
The cache absorbs most redirect traffic and keeps the database healthy.
4. Not knowing 301 vs 302
This is a common interview follow-up.
301 — Permanent Redirect
→ Browsers can cache the redirect
→ Fewer requests reach your server
→ Better performance
→ Less control over tracking
302 — Temporary Redirect
→ Requests continue reaching your server
→ More control over analytics
→ More server traffic
Know the trade-off, not just the definitions.
5. Having no scaling plan
Interviewer:
"What happens when you reach 1 billion URLs?"
Candidates freeze.
Instead, explain how you'd evolve the system:
→ 0–10M: PostgreSQL + proper indexing
→ 10M–100M: Redis + read replicas
→ 100M–1B: Sharding
→ 1B+: Consider distributed databases like DynamoDB or Cassandra
The exact numbers aren't the point.
The interviewer wants to see that you understand how the architecture evolves with scale.
Why do interviewers love this question?
A "simple" URL Shortener tests:
→ Requirements gathering
→ Encoding & ID generation

→ Caching
→ HTTP concepts
→ Database design
→ Scalability & trade-offs
That's a lot of System Design packed into one seemingly simple problem.
Want to go deeper?
I've covered the complete URL Shortener design in my ebook:
System Design Simplified — Volume 2
Real-world architecture, trade-offs, scaling decisions, and interview-focused explanations.
→ Get it here: https://lnkd.in/gCPk6DFV
Use SDE20 to get 20% off.
Check the comments for the complete links to Volume 1 and Volume 2.
#SystemDesign #Backend #Java #SoftwareEngineering #Microservices #DistributedSystems #Scalability #Redis #Database #TechInterview #SystemDesignInterview

LINKS
- https://lnkd.in/gCPk6DFV

#242 Woongsik Dr. Su, MBA

System Design

AI | ML | NLP | Big Data | ChatGPT | Robotics | FinTech | Agent AI | Digital Transformation | AI Transformation | AX | DX | RX | AI Act | Robotics Transformation
2mo · **View on LinkedIn**

Preparing for a System Design Interview?
System Design can seem complicated at first.
There are databases, caches, load balancers, APIs, queues, distributed systems, fault tolerance—and somehow they all need to work together.
The best way to learn isn't to memorize hundreds of architecture diagrams.
It's to understand the problem first, then the reason behind each design decision.
This collection of 300 System Design questions and answers offers a practical way to build that understanding, covering:
→ Scalability and availability
→ Database and storage choices
→ Caching and performance
→ Load balancing
→ API design and communication
→ Distributed systems
→ Reliability and fault tolerance
→ Common architectural patterns
→ Real-world system design scenarios
→ Interview questions from beginner to advanced
The most important question during a system design interview isn't simply:
"Which technology would you use?"
It's:
"Why would you use it here?"
Once you understand the trade-offs, system design becomes much less about memorizing solutions and much more about solving problems.
Whether you're preparing for an interview or simply want to become a stronger backend engineer, this is a useful resource to keep nearby.

Save it for your next revision.

Which area would you find hardest?

1. Scalability
2. Caching
3. Distributed Systems
4. Database Design

Follow: Woongsik Dr. Su, MBA

#SystemDesign #SoftwareEngineering #BackendDevelopment #DistributedSystems #SoftwareArchitecture #TechInterviews #Programming #Engineering

AI

.

101 pages

MEDIA B E G I N N E R L E A R N I N G G U I D E System Design for Beginners 300 Questions and Answers — From Beginner to Advanced A practical, beginner-friendly guide to the core concepts behind scalable, reliable software systems. Created By: Kartik Mungole · @kartikmungole; System Design for Beginners Page 2 Created By: Kartik Mungole · @kartikmungole Q001 What is system design? System design is the process of planning how a software application will work before you start building it. It means deciding how the different parts, like the website, the database, and the servers, will fit together and talk to each other. Think of it like an architect drawing blueprints for a house before construction begins, instead of just starting to lay bricks randomly. Q002 Why is system design important? System design is important because it helps engineers build applications that can handle real users and real problems, not just work perfectly in a small test. Without good design, an app might work fine for 10 users but completely crash when 10,000 people try to use it at once. Planning ahead helps avoid expensive rebuilding later, saving both time and money. Q003 What is the difference between system design and coding? Coding is the act of writing the actual instructions a computer follows, line by line, in a programming language. System design is the bigger picture plan that comes before coding, deciding how different pieces of code and services should be organized and connected. You can think of coding as building individual rooms, while system design is the overall floor plan of the entire house.; System Design for Beginners Page 3 Created By: Kartik Mungole · @kartikmungole Q004 What is a system in software engineering? A system is a group of components that work together to perform a specific task, such as running a website, processing payments, or sending messages. For example, an online shopping system includes a website for browsing, a database to store products, a payment service, and a way to track orders. Each piece does its own job, but together they form one complete system. Q005 What is high-level design? High-level design is the overall plan for a system, showing the main components and how they connect, without going into small details. For example, a high-level design for a chat app might just show boxes labeled 'app,' 'server,' and 'database,' with arrows showing how messages flow between them. It is meant to give a quick, easy-to-understand overview before diving into specifics. Q006 What is low-level design? Low-level design is the detailed plan for how each individual part of a system actually works on the inside. While high-level design might just show a box labeled 'database,' low-level design would explain exactly which tables exist, what columns they have, and how the code reads and writes to them. It is the next step after highlevel design, filling in all the specific details needed to actually build the system.; document/carousel (possible)

#243 **Prachi Jain** System Design

19k+ LinkedIn | Software Engineer @ ServiceNow | Ex-Lloyds | Ex-IBM | SDE | Test Automation | Sharing insights on Java, Selenium, QA Strategy & Career Growth

2mo · **View on LinkedIn**

System Design is not about drawing boxes.

It’s about understanding why those boxes exist.

When designing a scalable system, you need to think about:

Scalability

Availability

Latency & Throughput

CAP Theorem

Load Balancing

Databases & Sharding

Caching & CDNs

Message Queues

Rate Limiting

Consistent Hashing

This 75-page System Design Interview Handbook brings these concepts together with architectural patterns, trade-offs, interview tips, and common system-design questions.

Learn the concepts → understand the trade-offs → design the system.

Save this for your next System Design interview.

Follow Prachi Jain for more!

#SystemDesign #SoftwareEngineering #SystemDesignInterview #BackendDevelopment #DistributedSystems #TechInterview #SoftwareDeveloper #Coding #Programming #Engineering

System Design

.

75 pages

MEDIA SYSTEM DESIGN 75 pages guide to ace your next System Design Interview ASHISH PRATAP SINGH AlgoMaster.io INTERVIEW HANDBOOK Nishant Kalkhanda Nishant Kalkhanda; Table of Contents Fundamentals 1. Scalability 2. Availability 3. Latency vs Throughput 4. CAP Theorem 5. Load Balancers 6. Databases 7. CDN 8. Message Queues 9. Rate Limiting 10. Database Indexes 11. Caching 12. Consistent Hashing 13. Database Sharding 14. Consensus Algorithms 6 7 8 9 10 11 12 13 14 15 16 18 19 20; 15. Proxy Servers 16. Heartbeats 17. Checksums 18. Service Discovery 19. Bloom Filters 20. Gossip Protocol 21 22 23 24 25 26 Trade-offs 1. Vertical vs Horizontal Scaling 2. Strong vs Eventual Consistency 28 29 30 31 33 35 3. Stateful vs Stateless Design 4. Read vs Write Through Cache 5. SQL vs NoSQL 6. REST vs RPC; document/carousel (possible)

#244 Achal Soni

System Design

Lead Software Engineer | Investment Banking | Java 21 | Spring Boot | Distributed Systems | AWS | Event-Driven Architecture | AI-Assisted Engineering

2mo · **View on LinkedIn**

Learn Easy System Design on One Page

System Design is not about drawing boxes : it’s about making the right trade-offs.

Scalability, caching, Kafka, databases, resilience, security, observability... every decision affects another.

I created this System Design: Learn Easy one-page guide as a quick reference for engineers and interview preparation.

Start simple. Design for failure. Scale when needed. Explain your trade-offs.

What’s the toughest part of a System Design interview for you?

#SystemDesign #SoftwareArchitecture #DistributedSystems #EngineeringLeadership #BackendEngineering

MEDIA advertisement

#245 Petar Ivanov

Kafka / Messaging

I help JavaScript devs think like architects in the AI era • React · Node · Architecture · System Design · Production AI • Senior AI Engineer • 100K+ Audience • Founder of petarivanov.tech

2mo · **View on LinkedIn**

7 message queue patterns every backend engineer should know:

Behind every system that survives a traffic spike, there's a queue.

The math: signups burst to 10,000 in one minute.

Your email provider allows 14/sec.

Without a queue, requests pile up on a third-party limit.

With one: the API answers in milliseconds, workers drain at exactly 14/sec.
The 7 patterns:
1. Background jobs
↳ Anything the user doesn't wait for leaves the request path.
↳ Emails, PDFs, resizing, outbound webhooks. Respond now, work later.
2. Burst buffering
↳ Traffic is bursty. Capacity is flat. The queue is the shock absorber between them.
↳ Absorb the spike, drain at your pace.
3. Retry with backoff
↳ Failed jobs re-run with exponential backoff, outside the request path.
↳ No user stares at a spinner while attempt # 3 sleeps.
4. Dead-letter queue
↳ Jobs that keep failing land somewhere visible, not in a forever-retry loop.
↳ No DLQ = silent data loss under a clean dashboard. Alert on its depth.
5. Fan-out
↳ One event, many independent consumers. Order placed → email + analytics + warehouse.
↳ Adding a consumer touches zero upstream code.
6. Idempotent consumers
↳ Queues promise at-least-once. Your job WILL run twice.
↳ Dedupe on a job ID before any side effect.
7. Depth as your early-warning metric
↳ Latency alerts fire when users are already hurt. Queue depth climbs minutes earlier.
↳ Alert on depth + oldest-job age. The cheapest leading indicator you'll ever get.
When NOT to queue: simple CRUD, and anywhere the caller needs the answer now.
> A queue you didn't need is just latency with extra steps.
And you don't need Kafka. Redis + BullMQ covers most Node backends.
A Postgres jobs table gets you further than you think.
Save this for your next design or backend interview.
What's the longest a job sat in your queue before anyone noticed?
———
Repost to help others learn how to build real systems.
+ Follow me (Petar Ivanov) to improve your JavaScript and Architecture skills.
7 message queue patterns for b
.
10 pages

MEDIA document/carousel (possible)

#246 **Alex Xu** AI / LLM

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

2mo · **View on LinkedIn**

Top 12 Agent Skills You Should Know

Agent skills are instructions and scripts that teach your LLM agent a new skill. The diagram below shows the 12 most-starred skill repos on GitHub as of August 2026.

1. Superpowers (obra/superpowers): This skill makes your agent plan before it writes code.

2. skills (mattpocock/skills): Matt Pocock's personal skill set makes your agent challenge your plan first. This is useful as agents can sometimes be too soft.

3. andrej-karpathy-skills: Multica AI distilled Karpathy's advice on AI coding pitfalls into one skill.

4. everything-claude-code: Skills that help you set up your coding agent. This is useful when you are starting Claude Code from scratch.

5. skills (anthropics/skills): This is Anthropic's official skills. It makes your agent capable of creating outputs like Word or PDF files.

6. ui-ux-pro-max-skill: This has instructions that teach your agent how to prevent AI-like designs.

7. caveman: Julius Brussee's skill makes your agent reply in short caveman speak.

8.ponytail: Dietrich Gebert's skill teaches your agent how to write code that is simple and clean.

9. agent-skills: Google's Addy Osmani included production-grade engineering practices in a skill

10. graphify (safishamsi/graphify): This skill converts a codebase into a knowledge graph, so an agent can navigate easier.

11. Understand-Anything: Egonex AI converts a codebase into visual maps to explore.

12. impeccable (pbakaus/impeccable): This skill makes an agent better at UI polish.

Over to you: Which skill would you add to this list?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gauQcE45

#systemdesign #coding #interviewtips

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gauQcE45

#247 Parikh Jain AI / LLM

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

2mo · **View on LinkedIn**

Zero-to-hired DSA curriculum (All free resources)

Arrays/Hashing: LC 1, 217, 242, 49, 347, 238, 128, 36, 169, 75

Two Pointers: LC 125, 167, 15, 11, 42, 283, 26, 344, 977, 392

Sliding Window: LC 121, 3, 424, 567, 76, 239, 209, 1004, 904, 438

Prefix Sum: LC 560, 303, 304, 525, 974, 523, 724, 930, 1310, 1588

Linked Lists: LC 206, 21, 141, 142, 19, 143, 2, 23, 25, 138

Stack/Queue: LC 20, 155, 150, 394, 739, 84, 853, 71, 232, 225

Binary Search: LC 704, 35, 74, 153, 33, 875, 1011, 410, 4, 162

Intervals: LC 56, 57, 435, 452, 986, 1288, 1094, 1109, 1851, 729

Trees/BST: LC 104, 100, 226, 102, 98, 230, 236, 124, 297, 501

Heaps: LC 215, 973, 703, 295, 621, 1046, 767, 502, 373, 692

Backtracking: LC 46, 47, 78, 90, 39, 40, 17, 79, 131, 51

Graph BFS/DFS: LC 200, 695, 733, 994, 1091, 1254, 130, 133, 417, 785

Topo Sort/DSU: LC 207, 210, 802, 1462, 1203, 547, 684, 1319, 1579, 990

Shortest Path: LC 743, 787, 1514, 1631, 1334, 1976, 778, 1368, 2290, 1928

Greedy: LC 55, 45, 134, 135, 763, 678, 846, 881, 1029, 406

1D DP: LC 70, 198, 213, 322, 279, 300, 152, 139, 91, 746

2D/String DP: LC 62, 63, 64, 221, 1143, 72, 115, 583, 97, 1312

Tries/Bits: LC 208, 211, 212, 648, 677, 1268, 136, 137, 191, 338

1500+ students are using my DSA + Dev + GenAI Placement System: https://lnkd.in/gtzyBHkw.

Doing exactly this: pattern-based DSA, full-stack Dev, GenAI, LLD, HLD, core CSE, plus job assistance built in.

5 posts I've written to go deeper:

-https://lnkd.in/gJMp8grU

-https://lnkd.in/g7wjY8hp

-https://lnkd.in/gyYVi2Xb

-https://lnkd.in/g6d2VXZD

-https://lnkd.in/g4nB4NV5

Do 3 patterns at a time. Solve 10, note the trigger, revisit failures after 3-4 days, then move on.

180 intentional problems > 1,000 random ones.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gJMp8grU
- https://lnkd.in/g7wjY8hp
- https://lnkd.in/gyYVi2Xb
- https://lnkd.in/g6d2VXZD
- https://lnkd.in/g4nB4NV5
- https://lnkd.in/gtzyBHkw

#248 Ali Raza Cloud / K8s / DevOps

Engineering Manager, Payments @ PostEx • I explain how AI, Web, Backend, and Security work • Building XPay • Built Pakistan's 1st Stripe-Like Payment Element • Building Payments, Security & Agentic AI Systems

2mo · **View on LinkedIn**

Can you answer these 10 practical backend questions?

1. Your API usually responds in 80ms. P99 jumps to 2 seconds, but CPU and memory look normal. What could cause it?

2. A "POST /payments" request times out after the server completes it. The client retries. How do you prevent a duplicate payment?

3. One API depends on three downstream services. Each is fast individually, but the API is slow. Why?

4. Traffic grows and requests start waiting for database connections. Will you increase the connection pool?

5. A user changes "/users/123/orders/456" to "/users/124/orders/456" and sees another user's order. Authentication is valid. What is missing?

6. A downstream service starts failing. Your service retries every request three times and makes the outage worse. How should retries work?

7. Your API returns "200", but the customer says the operation failed. How do logs, metrics, and traces help you find what happened?

8. A stolen JWT is still valid and is being replayed from another device. What can you do beyond validating its signature and expiry?

9. Error rate jumps from 0.1% to 3%, but only for one customer segment. How do you make that visible without exploding metric cardinality?

10. An endpoint that once returned 100 records now has 100,000. How do you keep it fast as the dataset grows?

When I interview backend engineers, I prefer questions around APIs, latency, security, failures, and observability over framework-specific knowledge.

They help me understand how someone thinks about systems once they reach production.

What would you add?

#backend #api #systemdesign #security #observability

#249 Satendra Rajput

System Design

Senior Software Engineer | Angular & Java Spring Boot Developer | Microservices | Banking & Fintech — Axis Bank LOS, ADCB LOS, AEPS, Micro-ATM, RBAC | 4.5+ Yrs Experience

2mo · View on LinkedIn

Mastercard – 60 Senior Software Engineer Interview Questions

Preparing for a Senior Software Engineer role at Mastercard with a focus on Angular + TypeScript + JavaScript + DSA?

I reviewed multiple publicly available Mastercard interview experiences and identified recurring patterns across coding, frontend, system design, debugging, projects, and behavioral rounds.

Key areas to prepare:

Angular

- Standalone Components
- Signals & Change Detection
- RxJS & Observables
- Dependency Injection
- Interceptors & Route Guards
- Performance Optimization

TypeScript

- Generics
- Type Narrowing
- Utility Types
- Interface vs Type
- any vs unknown
- Advanced TypeScript concepts

JavaScript

- Closures
- Hoisting
- Event Loop
- Promises & async/await
- Microtasks & Macrotasks
- this, Prototype & Prototypal Inheritance
- Debouncing & Throttling

DSA

- Two Sum
- Valid Parentheses
- Sliding Window
- Merge Intervals
- Group Anagrams
- Linked Lists
- Trees
- DFS/BFS
- Binary Search
- Dynamic Programming

System Design

- File Upload System
- EV Charging System

• Notification System
• URL Shortener
• Rate Limiter
• Scalable Frontend Architecture

Project & Behavioral

• Project deep dive
• Production challenges
• Performance optimization
• Debugging & critical issues
• Team conflicts
• “Why Mastercard?”

Estimated match for Angular + TypeScript + JavaScript + DSA: 65–75%

The biggest takeaway: don't prepare only Angular. For a Senior role, combine strong frontend fundamentals with DSA, system design, JavaScript internals, and detailed project knowledge.

60 questions. One focused preparation strategy.

Save this post if Mastercard is on your target-company list.

Prepared by Satendra Rajput

KudoEngineer.com

#Mastercard #InterviewPreparation #SoftwareEngineer #SeniorSoftwareEngineer #Angular #TypeScript #JavaScript #DSA #SystemDesign #FrontendDeveloper #AngularDeveloper #CodingInterview #TechInterview #SoftwareEngineering #InterviewQuestions #KudoEngineer

#250 Jeet Chatterjee

Kafka / Messaging

Software Engineer at Early-stage startup

2mo · **View on LinkedIn**

Kafka Duplicate Events in Production

Kafka commonly provides at-least-once delivery, which means duplicate events can happen during crashes, retries, rebalances, or offset commit failures.

So the goal isn't to prevent every duplicate.

The goal is to make duplicates harmless.

The production pattern

Kafka
↓
Consumer
↓
DB Transaction
├─ Check eventId
├─ Apply business logic
└─ Record eventId
↓
Commit DB
↓
Commit Kafka Offset

Use a unique eventId and a database constraint to guarantee that the same event can't apply its business side effect twice.

For external APIs such as payments or shipping, use the same eventId as an idempotency key whenever supported.

Also separate the concerns:

Retries → handle temporary failures

Idempotency → handle duplicate events

DLQ → handle permanently failing messages

Kafka transactions → useful for Kafka-to-Kafka processing, but not a universal exactly-once solution

Rule of thumb:

At-least-once delivery + idempotent side effects = reliable Kafka architecture.

#Kafka #ApacheKafka #KafkaConsumer #KafkaProducer #EventDrivenArchitecture #EventDrivenSystems #DistributedSystems #Microservices #MicroserviceArchitecture #SystemDesign #SoftwareArchitecture #BackendEngineering #BackendDevelopment #ScalableSystems #HighAvailability #FaultTolerance #DataConsistency #EventStreaming #MessageQueues #Messaging #AsyncProcessing #Idempotency #ExactlyOnce #AtLeastOnce #EventProcessing #Database #PostgreSQL #Java #SpringBoot #CloudArchitecture #DevOps #SoftwareEngineering #TechArchitecture #Engineering #Coding #Programming

#251 Puja Agrawalla

Kafka / Messaging

Tech Lead @ Applied Materials | 11+ Yrs | Java • Spring Boot • Microservices • System Design | AWS | Distributed Systems | GenAI | Backend Architecture | Engineering Leadership

2mo · **View on LinkedIn**

Kafka interviews aren’t just about Kafka-they’re about what happens when things break.

In a recent discussion, these are the production scenarios I’d expect in a Senior/Lead Backend interview:

1. Consumer crashes after processing but before committing offset.

Message is processed but offset isn’t saved.

→ What happens on restart? Duplicate processing.

How do you design for safe processing?

2. Consumer lag keeps increasing.

Producer: 10K msgs/sec, consumer: 7K/sec.

→ What is consumer lag? → How do you find the bottleneck? → Do you just scale consumers?

3. Multiple event types in one topic.

Same topic now has OrderCreated, OrderCancelled, OrderUpdated.

→ How do you design the consumer to handle this safely?

These matter more than:

✗ “What is Kafka?” ✗ “What is a topic?” ✗ “What is a partition?”

At senior level, the real question is:

“Something broke in productionwhat happens and how do you fix it?”

What’s the most interesting Kafka production scenario you’ve been asked?

#Kafka #Java #SpringBoot #Microservices #BackendDevelopment #SystemDesign #TechInterviews #SoftwareEngineering

#252 Aman Mishra

System Design

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

2mo · **View on LinkedIn**

"We use Redis for caching."

Every developer says this.

Then they stop learning Redis.
But caching is just one part of what Redis can do.
Here are 7 powerful Redis use cases you should know ↓
1. Caching — The obvious one
SET user:123 "{name: Aman}" EX 3600
Database read: ~5–10ms
Redis read: ~0.1ms
Fast reads. Less database load.
But this is just the beginning.
2. Session Store
3 servers behind a load balancer.
Sessions are stored in Redis.
Any server can read any user's session.
Result → easier horizontal scaling without relying on sticky sessions.
3. Rate Limiter
INCR user:123:requests
EXPIRE user:123:requests 60
Count requests per minute.
More than 100 requests?
→ Return 429 Too Many Requests
A simple Redis counter can power a basic distributed rate limiter.
4. Distributed Lock
SET lock:seat:A5 "user:123" NX EX 600
First user gets the lock.
Other users are rejected while the lock exists.
This pattern can be used for things like temporary seat reservations in booking systems.
5. Pub/Sub
PUBLISH notifications "new_order"
SUBSCRIBE notifications
Useful for lightweight real-time messaging.
For example:
→ Chat across multiple servers
→ WebSocket coordination
→ Real-time notifications
For simple use cases, you may not need Kafka.
6. Leaderboards
ZADD leaderboard 2300 "player:rahul"
ZREVRANGE leaderboard 0 9
Redis Sorted Sets automatically maintain scores in order.
Want the top 10 players?
One command.
Perfect for gaming, rankings, competitions, and similar use cases.
7. Geospatial Queries
GEOADD locations 77.2 28.6 "restaurant:123"
Store locations and query nearby points.
Useful for:

→ Finding nearby restaurants

→ Finding nearby drivers

→ Location-based services

The big takeaway

Redis is not just a cache.

It's an in-memory data platform that can support caching, sessions, rate limiting, distributed locks, Pub/Sub, leaderboards, and geospatial queries.

Most developers know:

SET

GET

Now you know 7 real-world use cases.

That's the kind of knowledge that helps in System Design interviews and, more importantly, in real backend systems.

Save this post for your next System Design interview.

Want to learn System Design in depth?

I've created System Design Simplified — Volume 1, covering core concepts, real-world architectures, interview questions, and practical examples in simple language.

→ Check it out here: https://lnkd.in/giT-sJSa

LINKS

- https://lnkd.in/giT-sJSa

#253 Sonam Yadav

System Design

Senior Software Engineer | Java | Spring Boot | Microservices | Spring AI | GenAI Integration | Distributed Systems | RabbitMQ | Docker | Building Scalable Backend & AI-powered Enterprise Applications

2mo · **View on LinkedIn**

Microservices Interview Preparation – Part 1

Here are the first 5 practical Microservices interview questions every backend engineer should be comfortable answering.

Instead of focusing only on theory, these questions cover real-world scenarios:

How to identify whether a performance issue is in the code, database, or network

How to prevent cascading failures when a dependent service goes down

How to handle sudden traffic spikes and protect your APIs

How to roll back safely after a problematic deployment

How to manage centralized configuration across multiple services

The goal is to understand not just “what tool to use”, but why and where to use it.

Save this for your next Microservices/Spring Boot interview preparation.

#Microservices #SpringBoot #Java #SystemDesign #BackendDevelopment #DistributedSystems #SoftwareEngineering #CloudNative #Resilience4j #DevOps #InterviewPrep #JavaDeveloper

#254 Neha Jain

Other

Senior ML eng. at Paypal | SDE @ex-Microsoft | 400+ @Linkedin| 60k+ @X| 100k+ @Instagram| Strategic Brand Advisor| Brand Marketer| Sharing insights on AI, Tech, SDE, ML, Marketing

2mo · **View on LinkedIn**

"DSA isn’t about solving 500+ problems — it’s about solving the right 25."

Even LeetCode has caught on — patterns matter more than numbers.

If you’re prepping for interviews, this list is your shortcut to success.

Arrays / Hashing / Two Pointers
• Two Sum – https://lnkd.in/g4fCe-r4
• Container With Most Water – https://lnkd.in/g6yMv-x4
• Longest Subarray with Sum K – https://lnkd.in/gNbZNMDA
• 3Sum Closest – https://lnkd.in/gmGJU64P
Strings
• Longest Substring Without Repeating Characters – https://lnkd.in/g67dVzAP
• Group Anagrams – https://lnkd.in/gtdb_Vum
• Valid Anagram – https://lnkd.in/g8PCp2Mk
Linked Lists
• Merge K Sorted Lists – https://lnkd.in/gbWKaAvs
• Detect Cycle in Linked List – https://lnkd.in/gqeNQ8UX
• Remove N-th Node From End – https://lnkd.in/gceShNRu
Stacks & Queues
• Daily Temperatures – https://lnkd.in/g4MJa5MT
• Min Stack – https://lnkd.in/ghdgKuf5
Dynamic Programming
• Climbing Stairs – https://lnkd.in/gkgj5SgV
• House Robber II – https://lnkd.in/gzhwv_aa
• Longest Increasing Subsequence – https://lnkd.in/gGsDwKWt
• Decode Ways – https://lnkd.in/gqjynYvz
Trees / BST / Binary Trees
• Invert Binary Tree – https://lnkd.in/gkU8JT_G
• Symmetric Tree – https://lnkd.in/gZr8yAeD
• Lowest Common Ancestor in Binary Tree – https://lnkd.in/gh8AKCmt
Backtracking
• N-Queens – https://lnkd.in/gNN7RxPD
• Palindrome Partitioning – https://lnkd.in/gEU72Tas
• Sudoku Solver – https://lnkd.in/gQ_smbz5
Graphs / BFS / DFS
• Number of Islands – https://lnkd.in/gD2NMcNc
• Course Schedule – https://lnkd.in/gEE4i8z8
• Rotting Oranges – https://lnkd.in/gWbgsNUg
Join my channels for more such job openings and resources:
X: https://lnkd.in/gmrp3tvZ
Whatsapp: https://lnkd.in/g83E7yt8
Check out my curated resources for your ultimate job preparation
Top 75 DSA Patterns to Crack PBC: https://lnkd.in/gWvxfaEM
Data Science Roadmap & Blueprint: https://lnkd.in/gQtc33Kn
Data Engineering Interview Prep. Kit & Sol.: https://lnkd.in/gjGiRXcN
Data Science Interview Prep. Kit + Solution: https://lnkd.in/g5qU74ej
Zero to AI Engineer Roadmap in 6 Months: https://lnkd.in/gkDMCCDr

LINKS

- https://lnkd.in/g4fCe-r4
- https://lnkd.in/g6yMv-x4
- https://lnkd.in/gNbZNMDA
- https://lnkd.in/gmGJU64P

- https://lnkd.in/g67dVzAP
- https://lnkd.in/gtdb_Vum
- https://lnkd.in/g8PCp2Mk
- https://lnkd.in/gbWKaAvs
- https://lnkd.in/gqeNQ8UX
- https://lnkd.in/gceShNRu
- https://lnkd.in/g4MJa5MT
- https://lnkd.in/ghdgKuf5
- https://lnkd.in/gkgj5SgV
- https://lnkd.in/gzhwv_aa
- https://lnkd.in/gGsDwKWt
- https://lnkd.in/gqjynYvz
- https://lnkd.in/gkU8JT_G
- https://lnkd.in/gZr8yAeD
- https://lnkd.in/gh8AKCmt
- https://lnkd.in/gNN7RxPD
- https://lnkd.in/gEU72Tas
- https://lnkd.in/gQ_smbz5
- https://lnkd.in/gD2NMcNc
- https://lnkd.in/gEE4i8z8
- https://lnkd.in/gWbgsNUg
- https://lnkd.in/gmrp3tvZ
- https://lnkd.in/g83E7yt8
- https://lnkd.in/gWvxfaEM
- https://lnkd.in/gQtc33Kn
- https://lnkd.in/gjGiRXcN
- https://lnkd.in/g5qU74ej
- https://lnkd.in/gkDMCCDr

#255 Ankita Sinha

Kafka / Messaging

AVP @ Deutsche Bank | Java | Spring Boot | Kafka | Distributed Systems | Enterprise Solutions | Tech Leadership | System Design & Architecture | Problem Solving | AI/ML | Gen AI | Agentic AI

2mo · **View on LinkedIn**

Apache Kafka Architecture — Simplified

#01/90 | Apache Kafka Architecture

Ever wondered what happens behind the scenes when millions of events need to be processed in real time?

Here’s a simplified view of Apache Kafka architecture:

Producer → Publishes messages/events to Kafka topics

Topic → Logical category where messages are stored

Partition → Splits a topic for parallel processing and scalability

Broker → Kafka server responsible for storing and serving messages

Replication → Copies partitions across brokers for fault tolerance

Consumer → Reads and processes messages from Kafka

Consumer Group → Multiple consumers work together to process partitions in parallel

Offset → Tracks the position of a consumer within a partition

KRaft → Manages Kafka cluster metadata and controller responsibilities in modern Kafka

How the flow works

Producer → Topic → Partition → Broker → Consumer Group → Consumer

For example, in an order-processing system:

Order Service → Kafka → Order Topic → Partitions → Payment / Inventory / Notification Services

The biggest advantage?

Kafka decouples producers and consumers, allowing systems to scale independently while supporting high throughput, durability, and fault tolerance.

Key takeaway:

Kafka isn't just a message queue — it is a distributed event-streaming platform designed to handle large-scale, real-time data.

Interesting Fact: Kafka consumers don't delete messages after reading them. They maintain an offset that represents their position in the partition, which allows messages to be replayed/reprocessed when required.

#ApacheKafka #Kafka #EventStreaming #Microservices #SystemDesign #Java #SpringBoot #DataEngineering #DistributedSystems

#256 Deshdeepak Rajpoot

System Design

Full-stack Developer | Java & Spring Boot | RESTful Services | CI/CD | Clean Code, Performance & Scalable Systems

2mo · View on LinkedIn

Choosing the right system design pattern isn't about knowing every buzzword — it's about matching the problem to the solution.

Here's a quick decision guide I put together for common scaling challenges:

More traffic? → Load Balancer + Horizontal Scaling

Slow static content? → CDN

Too many API entry points? → API Gateway

Database getting hammered with reads? → Caching + Replication

Database too large? → Sharding

Services tightly coupled? → Message Queue / Event-Driven Architecture

Too many microservices to manage? → Service Mesh

The biggest mistake I see engineers make isn't picking the wrong pattern — it's reaching for a pattern before they've actually diagnosed the problem. A service mesh won't fix a database bottleneck. Sharding won't help if your real issue is unauthenticated traffic hitting every service directly.

Good system design starts with asking:

what's actually breaking, and why? Then you work backward to the simplest pattern that solves it — not the most impressive one.

Bookmark this if you're prepping for system design interviews or just want a mental model for scaling conversations with your team.

What pattern do you find yourself reaching for most often? Curious to hear how others think about this in production. ↓

#SystemDesign #SoftwareEngineering #TechInterviews #Scalability #BackendDevelopment #Microservices #SoftwareArchitecture

MEDIA advertisement

#257 DevOps Professionals CI/CD Docker Kubernetes Serverless OpenShift Selenium Jenkins Ansible Chef

Cloud / K8s / DevOps

Neel Shah • 3rd+

2mo · View on LinkedIn

Most Kubernetes YAML looks complicated.

Until you know what each line actually does. ↓

apiVersion → What API to use

kind → What you’re creating

metadata → Name & labels

spec → The desired state

selector → What to target

template → What to create

Master these building blocks and Kubernetes YAML stops being scary; it becomes declarative infrastructure you can reason about.

Save this cheat sheet. You'll need it more than once.

Repost if this changed how you think about Kubernetes

Follow Neel Shah for more DevOps & Cloud insights

#Kubernetes #DevOps #CloudNative #PlatformEngineering #CNCF

MEDIA advertisement

LINKS

- https://sessionize.com/neel-shah/

#258 Java Developers Community (moderated)

AI / LLM

Likhitha V • 3rd+

2mo · **View on LinkedIn**

Building Enterprise AI Applications with Spring AI + MCP

Most AI applications can answer questions.

The next generation of Enterprise AI can take action.

Using Spring AI + Model Context Protocol (MCP), AI assistants can securely interact with enterprise systems instead of just generating responses.

As Java developers, we don't need to rewrite our applications for AI - we can extend our existing Spring Boot ecosystem with Spring AI and MCP to build secure, scalable, enterprise-ready AI solutions.

The future of Enterprise AI isn't just about choosing the right LLM.

It's about connecting AI to real business systems securely and intelligently.

What enterprise use case would you build with Spring AI + MCP?

#SpringAI #SpringBoot #MCP #Java #AgenticAI #EnterpriseAI #SoftwareArchitecture #Microservices #BackendDevelopment #LLM #AIEngineering #CloudNative #OpenAI #C2C #C2H

#259 Ashwani Gupta

System Design

Senior Data Engineer | Azure | Azure Databricks | Azure Data Factory | Data Vault 2.0 | PySpark | SQL | AI | AWS | ETL | CI/CD | Databricks Certified

2mo · **View on LinkedIn**

Most system design diagrams look complicated.

But the core idea is actually simple. ↓

A user opens an application and sends a request.

What happens next?

1. DNS

Converts the domain name into the address of the system.

2. Load Balancer

Distributes incoming traffic across multiple application servers so one server doesn't become the bottleneck.

3. Stateless Application Servers

Handle business logic and can scale horizontally as traffic grows.

4. Cache
Frequently accessed data can be served without hitting the database every time.
5. Message Queue + Workers
Heavy or non-immediate tasks can run asynchronously instead of making the user wait.
6. Database
Stores the system's persistent data.
As scale increases, we may introduce:
→ Read replicas for read-heavy workloads
→ Sharding/partitioning to distribute large datasets
→ NoSQL/Search systems for specialized access patterns
7. Logging, Metrics & Monitoring
Because a scalable system that you can't observe is still difficult to operate.
So the architecture gradually becomes:
User → DNS → Load Balancer → App Servers → Cache/DB
with
Queues → Workers
DB → Replicas/Shards
Everything → Monitoring
But here's the important part:
✗ You don't add every component just because it appears in a system design diagram.
You add them when the requirements and bottlenecks justify them.
A system handling 1,000 users and one handling 100 million users shouldn't automatically have the same architecture.
Good system design isn't about adding more components.
It's about knowing WHY and WHEN you need each one.
#SystemDesign #SoftwareArchitecture #DataEngineering #CloudComputing #Scalability #DistributedSystems #BackendEngineering

MEDIA advertisement

#260 AYAN HASNAIN System Design

Software Engineer @HARCORP | Building Artificial Super Intelligence & Matter.
2mo · **View on LinkedIn**

I used to think System Design was about memorizing big architecture diagrams.
It's not.
If I had to learn System Design again from scratch, this is the order I'd follow:
1. How the Web Works
HTTP, DNS, TCP/IP, HTTPS, request-response cycle
2. Backend Fundamentals
APIs, authentication, sessions, WebSockets
3. Database Fundamentals
SQL vs NoSQL, schema design, normalization, relationships
4. Database Performance
Indexes, query optimization, connection pooling
5. Caching
Redis, TTL, cache-aside, cache invalidation

6. Scaling
Vertical vs horizontal scaling, load balancers, stateless servers
7. Async Systems
Message queues, Kafka, RabbitMQ, background jobs, retries
8. Distributed Systems
CAP theorem, consistency, replication, partitioning, sharding
9. Reliability
Timeouts, retries, idempotency, circuit breakers, fault tolerance
10. Microservices
Service communication, API gateways, service discovery, distributed transactions
11. Observability
Logging, metrics, tracing, monitoring and alerting
12. Real-World System Design
→ URL shortener
→ Chat application
→ Instagram
→ YouTube
→ Uber
→ Netflix
And one rule I'd follow throughout:
Don't memorize the architecture.
Understand the problem first.
Why is the database becoming slow?
Why do we need a cache?
Why do we need a queue?
Why can't one server handle the traffic?
Why do we need replication?
Once you start asking these questions, System Design stops feeling like a collection of random technologies.
It starts feeling like problem solving.
Build → break → understand the bottleneck → improve → repeat.
That's how I'd learn System Design today.
#SystemDesign #SoftwareEngineering #BackendDevelopment #DistributedSystems #CodingJourney

#261 Aman Mishra

System Design

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

2mo · View on LinkedIn

5,000 people are trying to book the same Avengers seat on BookMyShow.
You click A5.
Someone else clicks A5 at the exact same time.
Who gets the seat?
This is the classic concurrency + locking problem.
There are 3 common ways to solve it ↓
1. Pessimistic Locking — Lock first, ask later
SELECT * FROM seats WHERE seat_id = 'A5' FOR UPDATE;

The database locks the row. Nobody else can modify it until your transaction finishes.

✓ Safe ✗ Can become a bottleneck when many users compete for the same resource.

Best for: Critical transactions, banking, low-to-moderate contention.

2. Optimistic Locking — Hope for the best, verify before saving

Don't lock the row upfront.

Both users can read seat A5. But when saving:

UPDATE seats SET status = 'HELD', version = 2 WHERE seat_id = 'A5' AND version = 1;

User 1: Version is 1 → update succeeds.

User 2: Version is already 2 → update fails.

Result: "Seat unavailable."

✓ Great for high-read workloads ✗ Conflicting writes need to be rejected or retried.

Best for: High reads with relatively low write conflicts.

3. Distributed Locking — Lock across multiple servers

BookMyShow runs on multiple servers.

Your request could hit Server 1 while another user's request hits Server 20.

A local lock won't protect the seat across all servers.

That's where a distributed lock comes in.

For example, Redis can coordinate access:

SET lock:show123:seatA5 <token> NX EX 600

User 1: Lock acquired → seat is held.

User 2: Lock already exists → cannot acquire the lock.

The lock expires after the configured TTL if it isn't released earlier.

Best for: Distributed systems and multi-server applications.

How a production booking system can combine these

→ User selects seat → Distributed lock prevents competing servers from claiming it → Database uses optimistic locking as another consistency check → Seat status is pushed to other users via WebSocket → Payment succeeds → HELD becomes BOOKED → Payment fails or hold expires → seat becomes available

Save this post. This is one of the most common concurrency and System Design interview problems.

Want to learn System Design from scratch?

I've created System Design Simplified — Volume 1, an in-depth guide covering core concepts, real-world architectures, interview questions, and practical examples in simple language.

If you're preparing for Backend or System Design interviews, check it out here:

https://lnkd.in/giT-sJSa

LINKS

- https://lnkd.in/giT-sJSa

#262 Swetha Telu

System Design

Lead Engineer @ Cognizant | 12+ Years in Java Development | Spring Boot | Microservices | REST | Event-Driven Architecture | Supply Chain Domain |Banking Domain | Health Care Domain

2mo · **View on LinkedIn**

30 Days of System Design for Java Developers – Day 4/30

API Gateway Explained

As your application grows from a few services to dozens of microservices, managing client requests becomes increasingly complex.

Should every client communicate directly with each service?

Not really.

That's where an API Gateway becomes the front door of your system.

What is an API Gateway?

An API Gateway is a single entry point that receives client requests and routes them to the appropriate backend services.

Instead of:

Client → User Service

Client → Order Service

Client → Payment Service

Use:

Client

↓

API Gateway

↓

User Service

Order Service

Payment Service

Notification Service

One endpoint.

One security layer.

One routing mechanism.

Key Responsibilities

✓ Request Routing

✓ Authentication & Authorization

✓ Rate Limiting

✓ SSL/TLS Termination

✓ Request & Response Transformation

✓ API Aggregation

✓ Logging & Monitoring

✓ Load Balancing Integration

Real-World Example

Imagine an e-commerce mobile app.

The Home screen requires:

User Profile

Recent Orders

Wallet Balance

★ Recommendations

Without an API Gateway:

The mobile app makes four different API calls.

With an API Gateway:

The client sends one request.

The gateway communicates with multiple services, combines the responses, and returns a single optimized response.

Fewer network calls.

Better performance.

Simpler client code.

(!) Common Mistakes

✗ Adding business logic inside the Gateway

✗ No authentication

✗ Missing rate limiting

✗ No monitoring

✗ Single Gateway without High Availability

Interview Questions

1. What is an API Gateway?

2. API Gateway vs Load Balancer?

3. What is API Aggregation?

4. Why is Rate Limiting important?

5. How does an API Gateway improve security?

6. Which API Gateway have you worked with? (Spring Cloud Gateway, Kong, NGINX, AWS API Gateway, Apigee)

7. Can an API Gateway become a bottleneck?

8. How do you make an API Gateway highly available?

Senior Developer Tip

Keep your API Gateway lightweight.

Its responsibility is routing, security, observability and request management—not business logic.

A clean API Gateway simplifies your architecture and allows backend services to evolve independently.

↓ Which API Gateway does your current project use? Spring Cloud Gateway, Kong, NGINX, AWS API Gateway, or another solution?

#SystemDesign #Java #JavaDeveloper #APIGateway #SoftwareArchitecture #Microservices #BackendDevelopment #DistributedSystems #CloudComputing #InterviewPreparation #30DaysOfSystemDesign

#263 Anand Gunasekaran System Design

Chief Manager – Technology & Project Delivery | Banking & Financial Services | Solution Architecture | Digital Banking | Microservices | Large Programs | Cross-Functional Leadership

2mo · **View on LinkedIn**

We broke our Java monolith into 47 microservices.

Then spent 2 years consolidating back.

Here's what architecture actually requires, and nobody tells you upfront.

1. Modular Monolith First. Microservices When Earned

Decompose only after >1M requests/day or 50+ developers.

Conway's Law will define your architecture whether you plan it or not.

Most teams distribute too early — and pay for it in operational complexity.

2. Virtual Threads changed everything. (Java 21+)

spring.threads.virtual.enabled=true is now the default in every new Spring Boot service.

40% higher throughput than WebFlux for DB-heavy workloads.

Write simple blocking code. Get reactive-level concurrency.

The reactive-by-default era is over.

3. DDD boundaries before service boundaries

Every service split without bounded contexts creates distributed coupling — not decoupling.

Define your domains first. Draw the service lines second.

4. Observability on Day 1, Not Sprint 47

Logs. Metrics. Distributed traces. From the first commit.

You cannot debug a distributed system you cannot observe.

This is not an infrastructure team problem. It is an architecture decision.

5. Design for failure - not for diagrams

Microservices fail in production. Not on whiteboards.

Circuit breakers, retries, saga patterns, and dead-letter queues are not optional.

If your architecture only works when everything is healthy, it is not production architecture.

The best architecture is not the most sophisticated one.

It is the one that matches your team size, domain maturity, and operational capability — right now.

What is the most expensive architecture mistake your team has made?

#Java #SoftwareArchitecture #SpringBoot #Microservices #BackendDevelopment #SystemDesign #TechLeadership

#264 Chandrakant Kumar **Kafka / Messaging**

SDE 3 | Java, Spring Boot, Kafka, AWS, Microservices, Distributed Systems MCP | Building high-scale fintech & backend systems (50M+ events/day) | Serving Notice

2mo · **View on LinkedIn**

Recently faced one of those incidents that reminds you why distributed systems are both fascinating and challenging.

A Kafka consumer group in one of our event-driven pipelines suddenly accumulated 2 crore (20 million) pending messages. Consumer lag kept climbing, and downstream processing fell behind.

What happened

Producers kept publishing at the usual rate, but the consumer group simply didn't have enough instances to keep up with the partition count. With fewer active consumers than partitions, several consumers ended up owning multiple partitions each — serializing work that should have been running in parallel. Throughput per partition dropped, and the backlog snowballed fast.

How we approached it

1. Checked consumer health and logs — ruled out processing failures or crashes. 2. Verified consumer group status and partition assignment — found partitions unevenly loaded across a shrunk consumer pool. 3. Confirmed the root cause: consumer count < partition count, capping real parallelism. 4. Scaled the consumer group to match partition count, restoring one-consumer-per-partition parallelism. 5. Monitored lag closely — it cleared completely within 20 minutes of the fix.

Key takeaway

Consumer lag is often a symptom, not the root cause.

The first question shouldn't be "how do we clear the lag?" It should be "why did the consumers stop keeping up?"

In this case, the fix was straightforward once we found it — but it's a good reminder that in Kafka, your consumer count and partition count aren't just configuration details. They directly define your ceiling for parallel throughput. Under-provision consumers relative to partitions, and lag is only a matter of time under load.

Scaling blindly buys you a temporary reprieve. Understanding the partition-to-consumer ratio is what actually prevents the next incident.

#Kafka #DistributedSystems #Java #SpringBoot #EventDrivenArchitecture

LINKS
- https://

#265 Pradeep Singh

System Design

Senior Software Engineer — Java | Spring Boot | Angular | Microservices | AI-Augmented Engineering

2mo · **View on LinkedIn**

Redis Cache with Spring Boot Microservices — From Basics to Production

One of the simplest ways to improve the performance of a microservice is to reduce unnecessary database calls.

That’s where Redis comes in.

I created this visual cheat sheet covering the Redis concepts I think every backend developer should understand:

@Cacheable, @CachePut, @CacheEvict

Cache-Aside Pattern

Cache Hit vs Cache Miss

TTL & Cache Expiration

Cache Invalidation

Cache Stampede / Thundering Herd

Redis Cluster & Replication

Kafka + Redis Cache Invalidation

Production-oriented Redis architecture

The flow I keep in mind:

Request → Redis → Cache HIT → Response

If there's a cache MISS:

Request → Redis → DB → Redis → Response

The real complexity starts when we move from a simple application to distributed microservices.

Then we need to think about:

→ How do we invalidate stale cache? → What happens when thousands of requests miss the same key? → How do we scale Redis? → How do multiple services keep their caches consistent? → When should we use Kafka for cache invalidation?

These are the concepts that turn "I know Redis" into "I can design Redis for production."

Save this for your next Spring Boot / Microservices interview.

What topic would you like me to cover next?

#Java #SpringBoot #Microservices #Redis #BackendDevelopment #SystemDesign #SoftwareEngineering #Kafka #DistributedSystems #Programming #Coding #TechLearning

#266 Aman Mishra

System Design

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

2mo · **View on LinkedIn**

You have been doing System Design in Spring Boot every day.

You just didn't know the names.

That @Cacheable you use? Caching.

That @CircuitBreaker you added? Fault tolerance.

That @Transactional you use? Database transactions and ACID guarantees.

Let me show you ↓

1. @Cacheable → Caching
Store frequently accessed results in Redis or another cache.
Next request → cache hit → skip the database.
That's the Cache-Aside pattern you discuss in System Design interviews.
2. @CircuitBreaker → Circuit Breaker
Downstream service keeps failing?
Stop calling it temporarily and return a fallback.
This prevents cascading failures across microservices.
3. @Async → Asynchronous Processing
Some work doesn't need to block the user.
Run it asynchronously and let the request continue.
Same fundamental idea behind background processing and message queues.
4. @RateLimiter → Rate Limiting
Limit how many requests a client can make in a given time.
For example: 100 requests/second.
This protects your API from abuse and traffic spikes.
5. @Transactional → Database Transactions
Multiple database operations should succeed together—or roll back together.
That's where ACID transactions, atomicity, and isolation come into play.
6. @LoadBalanced → Load Balancing
Requests can be distributed across multiple instances of a service.
That's client-side load balancing.
7. @FeignClient → Inter-Service Communication
Call another microservice through a clean interface instead of manually building HTTP requests.
Under the hood, you're still dealing with network calls, timeouts, retries, and failures.
8. Spring Cloud Gateway → API Gateway
A single entry point for your clients.
It can handle: Routing, Authentication, Rate limiting, Request filtering
That's the API Gateway pattern.
9. Eureka → Service Discovery
Services register themselves.
Other services discover them dynamically instead of relying on hardcoded URLs.
That's Service Discovery.
10. Spring Boot Actuator → Observability
Endpoints like:
/health
/metrics
/prometheus
Help you monitor the health and behavior of your application.
You just need to connect the dots. Save this post.

I've created an in-depth guide to System Design, covering the core concepts, architecture patterns, real-world examples, and interview questions in simple language.

If you're preparing for backend or System Design interviews, this is for you.

→ Get the Guide here: https://lnkd.in/giT-sJSa

LINKS

- https://lnkd.in/giT-sJSa

#267 Imran Nur

System Design

Backend-Focused Full-Stack AI Engineer | Software Architect | FastAPI, Django | AWS & DevOps | 4+ Years

2mo · View on LinkedIn

Building resilient backend architectures requires mastering core API principles. Here is a breakdown of 20 Core API Concepts for designing high-performance distributed systems:

1. Fundamentals & Traffic

• Endpoint: Specific URIs where APIs receive incoming requests to access resources.
• HTTP Methods: Standardized verbs (GET, POST, PUT, DELETE) defining the intended action.
• Request-Response: The basic client-server communication lifecycle over HTTP.
• Status Codes: Standardized response codes (e.g., 200, 404, 500) signaling outcome status.
• Rate-Limiting: Capping request counts per client to prevent abuse and denial of service.
• Throttling: Controlling incoming traffic spikes to safeguard downstream microservices.
• Pagination: Chunking massive datasets into manageable pages to optimize payload size.
• Caching: Storing frequent data in Redis or CDNs to reduce database workload.

2. Security & State

• Authentication: Validating the identity of the client or user making the request.
• Authorization: Enforcing access rights and permissions for authenticated entities.
• Access Tokens: Short-lived tokens (like JWTs) passed in headers for stateless validation.
• OAuth 2.0: Industry standard for secure, delegated access authorization.
• Idempotency: Ensuring multiple identical requests produce the exact same system state.

3. Architecture & Operations

• Webhooks: Asynchronous event-driven callbacks for real-time notifications.
• API Versioning: Managing breaking changes via URI paths or headers without breaking clients.
• OpenAPI: Machine-readable specifications for defining and documenting API contracts.
• REST vs GraphQL: Architectural trade-offs between structured endpoints and dynamic queries.
• API Gateway: Centralized entry point managing routing, rate-limiting, and auth across services.
• Microservices: Decoupling systems into small, independently deployable services using APIs.
• Error Handling: Standardizing structured error payloads for consistent debugging.

Key Takeaway: Real architecture balances fast execution, strict security boundaries, and clear contract specifications.

Which of these 20 concepts plays the biggest role in your current stack? Drop your thoughts below! ↓

#BackendEngineering #SystemDesign #APIDesign #Microservices #SoftwareArchitecture #CleanCode

> **MEDIA** document/carousel (possible)

#268 Javali Bathula

Java / Spring

SMTS @ Oracle | 35K+ Community | AI • Tech • Marketing | Content Creator | Top 1% at Topmate | Brand Partnerships | Career & Digital Trends | DM to Collaborate

2mo · View on LinkedIn

I asked a Spring Boot developer what @SpringBootApplication actually does.

She said, "It starts the app."

That's true. But it's not the answer.

That one annotation is actually doing three jobs at once, quietly, in the background. Most developers use it every single day and have never had to explain what it really does.

That's the real test in Spring Boot interviews. Not "do you know the annotation." But "do you know what it's doing behind the scenes."

A few more that trip people up:

→ What's the real difference between @Component, @Service, and @Repository?

→ What is the N+1 select problem, and how do you fix it?

→ @SpringBootTest vs @WebMvcTest — when do you use each one?

→ What happens if two beans match for @Autowired?

→ How do you secure Actuator endpoints before going to production?

You don't need more years of experience to answer these. You just need to have thought about them once, properly.

So I put together 50 Spring Boot interview questions, explained simply, in the order you would naturally learn them:

- Spring Boot Fundamentals
- Configuration & Properties
- Dependency Injection & Beans
- Autowiring & Web Layer
- REST APIs & Spring Data JPA
- Transactions, Queries & Actuator
- Security, Testing & Microservices

Read through them once. Then try explaining each answer out loud, without looking. That's the real test.

Save this for your next interview prep session.

If you are preparing for interviews and want a clear way to practice instead of solving random problems, I made a DSA sheet with a 2-month plan. It's based on what actually helped me.

Here's the link: https://lnkd.in/gzu2arC5

#SpringBoot #Java #BackendDevelopment #InterviewQuestions #TechCareers

1/8

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/gzu2arC5

#269 Ankit Pangasa

System Design

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

2mo · **View on LinkedIn**

20 API Concepts Every Software Engineer Should Know

1. Endpoint

The URL through which a client interacts with a resource. Example: GET /users/123

2. HTTP Methods

Define the operation: GET → Read, POST → Create, PUT/PATCH → Update, DELETE → Remove.

3. Request-Response

Client sends headers, parameters and body → server processes the request → returns a response.

4. Status Codes

Communicate the outcome: 2xx success, 3xx redirection, 4xx client error, 5xx server error. Examples: 200, 201, 400, 401, 404, 500.

5. Authentication

“Who are you?” Examples: passwords, API keys, tokens and certificates.

6. Authorization

“What are you allowed to do?” An authenticated user may still lack permission to access a resource.

7. Access Tokens

Credentials used to access protected APIs without repeatedly sending passwords.

8. OAuth 2.0

An authorization framework that lets applications access resources on a user's behalf without sharing their password.

9. Rate Limiting

Limits how many requests a client can make in a given period. Example: 100 requests/minute/user.

Throttling

Controls the rate at which requests are processed. Unlike rate limiting, throttling can regulate or slow down processing.

11. Pagination

Returns large datasets in smaller pages. Example: GET /users?page=2&limit=50

12. Caching

Temporarily stores frequently requested data to reduce database/API calls and improve latency. Redis is a common example.

13. Idempotency

Repeating a request produces the same intended result. Critical for payments and retries. An Idempotency-Key can prevent duplicate charges.

14. Webhooks

Instead of repeatedly asking “Did something happen?”, another service notifies you when it happens. Example: payment provider → your server.

15. API Versioning

Lets APIs evolve without breaking existing clients. Example: /api/v1/users and /api/v2/users

16. OpenAPI

A standard specification for describing APIs. It can generate documentation, client SDKs and testing tools.

17. REST vs GraphQL

REST uses resource-oriented endpoints. GraphQL lets clients specify the data they need. Neither is universally better.

18. API Gateway

A single entry point between clients and backend services. It can handle routing, authentication, rate limiting, logging and caching.

19. Microservices

Splits a large application into independently deployable services that communicate through APIs.

20. Error Handling

Good APIs provide meaningful status codes, error messages, error IDs and enough context to troubleshoot safely.

Don't learn these in isolation. Understand how they work together:

Client → API Gateway → Authentication → Rate Limiting → Service → Cache/DB → Response → Error Handling

That's where API knowledge becomes system design knowledge.

#SystemDesign #APIs #SoftwareEngineering #BackendDevelopment #TechInterview #Programming

Image credits: Neo Kim

MEDIA document/carousel (possible)

#270 Vibhu Agarwal

System Design

Lead Software Engineer | Java Backend | Distributed Systems | Microservices | Kafka | AWS | FinTech | JP Morgan | Ex-PayPal | Citi | UBS | ISRO

2mo · View on LinkedIn

Microservices Design Patterns Cheat Sheet

Building microservices is more than just splitting a monolith—it's about designing systems that are scalable, resilient, and fault-tolerant.

I've put together a visual cheat sheet covering 25 essential Microservices Design Patterns, including:

✓ Circuit Breaker

✓ Saga

✓ API Gateway

✓ CQRS

✓ Event-Driven Architecture

✓ Outbox

✓ Idempotency

...and many more.

Whether you're preparing for system design interviews or building production-grade distributed systems, I hope this serves as a handy reference.

Which microservices design pattern do you use the most in your projects?

#Microservices #SystemDesign #Java #SpringBoot #Backend #DistributedSystems #SoftwareArchitecture #Kafka #AWS #DesignPatterns

#271 Lokesh Alwani

AI / LLM

--

2mo · View on LinkedIn

MCP vs API – Understanding the Difference

I've been exploring AI infrastructure lately, and one concept that stood out is the difference between APIs and the Model Context Protocol (MCP).

While they may seem similar, they solve different problems.

API (Application Programming Interface)

APIs are designed for application-to-application communication.

Think of it like this:

A web or mobile app sends an HTTP request.

An API gateway routes the request.

Backend services process it and return a response.

APIs are ideal for: ✓ CRUD operations

✓ Business logic

✓ Microservices communication

✓ Traditional software development

MCP (Model Context Protocol)

MCP is designed for AI-to-tool communication.

Instead of developers writing custom integrations for every tool, an AI agent connects to MCP servers, which expose tools in a standardized way.

This allows AI models to: ✓ Access databases

✓ Read and write files

✓ Call web services

✓ Use enterprise tools

✓ Switch between different tools without changing the AI application

The Key Difference

API:

→ Apps call services.

MCP:

→ AI agents discover and use tools through a common protocol.

Think of it this way:

API is like speaking directly to one person using their language.

MCP is like having a universal translator that lets AI communicate with many different tools using the same language.

As AI agents become more capable, standards like MCP will make integrating enterprise systems significantly easier, reducing custom development and improving interoperability.

What's your take? Do you think MCP could become the "USB-C for AI integrations", just as REST became the standard for web services?

#AI #GenAI #MCP #ModelContextProtocol #API #ArtificialIntelligence #LLM #AITools #SoftwareArchitecture #Developers #TechInnovation #MachineLearning #Automation

#272 Nuzaim Noushad Thappi System Design

Software Engineer | Backend infrastructure, workload orchesation and observability | FastAPI + Kubernetes | Open to conversations

2mo · **View on LinkedIn**

"Exactly-once" message processing in distributed systems doesn't actually exist. It's a consequence of the fundamental limitations of asynchronous distributed systems.

If an engineer claims they provide exactly-once delivery, they're certainly referring to effectively-once processing, not a transport guarantee. That is because every distributed system eventually runs into the same problem:

Imagine a producer sends a message.

The consumer receives it, processes it successfully, and sends back an acknowledgment (ACK). Now imagine the ACK is lost due to a network failure. "Exactly-once" message processing in distributed systems doesn't actually exist.

What does the producer see? Nothing. From its perspective, the consumer may never have received the message, so the only safe option is to retry. The consumer now receives the same message twice. This approach is called At-least-once.

It's a consequence of the fundamental limitations of asynchronous distributed systems.

It gets worse.

When a service doesn't respond, you can't know whether:

• it's still processing,

• the network is slow,

• or it's completely dead.

That uncertainty is fundamental.

Every distributed system must choose between two imperfect guarantees.

✓ At-most-once

• No duplicate deliveries.

• Messages may be permanently lost.

✓ At-least-once

• Messages are eventually delivered.

• Duplicates are inevitable during failures.

There isn't a third option at the transport layer. Systems that advertise "exactly-once" solve the problem at the application layer, not the network layer.

The recipe is straightforward:

• Use unique idempotency keys so every message has a unique identity.

• Make state updates atomic so deduplication and business logic succeed or fail together.

• Write idempotent handlers where processing the same event multiple times produces the same final state.

For example:

✗ balance = balance + 10

✓ balance = 100

The lesson is simple.

Don't chase the myth of "exactly-once delivery." Design your systems assuming duplicates will happen. Because they will. Reliable distributed systems aren't built by preventing duplicate messages.They're built by making duplicate messages harmless.

#DistributedSystems #SystemDesign #BackendEngineering #SoftwareEngineering #Architecture #Kafka #Microservices

MEDIA advertisement

#273 Raul Junco

System Design

Simplifying System Design

2mo · **View on LinkedIn**

A distributed system can be perfectly consistent... and still completely wrong.

Consistency ≠ Accuracy.

You can have:

• Accurate but inconsistent → replication lag

• Consistent but inaccurate → wrong value replicated everywhere

Consistency guarantees agreement, not truth.

At scale the real job is choosing the least dangerous failure mode.

Full breakdown: ↓

https://lnkd.in/eKYiC8BG

LINKS

• https://lnkd.in/eKYiC8BG

#274 Nick Gejadze

System Design

Turning complexity into simple, scalable products

2mo · **View on LinkedIn**

Microservices sound great in theory.

Then you split the database and everything gets hard.

That is exactly what senior interviews probe: distributed data, partial failures, and whether you know when NOT to use microservices.

So I put together 100 Microservices interview questions with real answers. Full PDF right here ↓

Inside:

✓ Fundamentals: monolith vs microservices, bounded contexts, decomposition, when not to

✓ Communication: sync vs async, gRPC, events, API gateway, service mesh

✓ Data and transactions: database per service, saga, CQRS, event sourcing, outbox

✓ Resilience: circuit breakers, retries, bulkheads, tracing, the three pillars
✓ Deployment and security: containers, Kubernetes, mTLS, testing, anti-patterns
100 questions. Answers that lead with the trade-off, not the buzzword.
Save it for your next interview, and follow for more. A new deep dive drops every few days.
#Microservices #SoftwareArchitecture #Backend #DistributedSystems #DevOps #InterviewPrep #Programming #Developers
Microservices
.
77 pages

MEDIA advertisement; document/carousel (possible)

#275 Rani Dhage

Java / Spring

Senior Java Developer | Spring Boot | Microservices | AWS | Distributed Systems | AI | System Design
2mo · **View on LinkedIn**

I thought I understood HashMap...
until an interview proved me wrong.
If someone had asked me a year ago how HashMap works, I would've confidently said,
"It's just a key-value pair that uses hashing to store data."
Technically, I wasn't wrong.
But I wasn't completely right either.
While preparing for interviews, I decided to go beyond the definition.
That's when I discovered what actually happens inside a HashMap.
It isn't just a collection of random buckets.
At its core, HashMap uses a dynamic array.
Each position in that array is called a bucket.
When you insert a key, Java calculates its hashCode() to decide which bucket it belongs to.
But what if two different keys end up in the same bucket?
That's called a collision.
Instead of overwriting the existing value, HashMap stores both entries together.
Java then uses equals() to identify the correct key when retrieving data.
Earlier versions stored collided entries as a Linked List.
But in Java 8 and above, if too many entries accumulate in the same bucket (and the table is large enough), the linked list is converted into a Red-Black Tree.
That small optimization improves lookup performance from O(n) to O(log n) in worst-case scenarios.
Learning this completely changed the way I looked at Core Java.
I realized interviews aren't just testing whether you've used HashMap.
They're testing whether you understand why it works the way it does.
Once I understood the internals, I stopped memorizing interview answers.
I started understanding Java.
And that's when learning became much more enjoyable.
Keeping this in mind, I went deep and documented everything into a Java Backend Developer Guide.
Get the guide here: https://lnkd.in/dfhsJKMj
#Java #Java26 #SpringBoot #AI

LINKS
- https://lnkd.in/dfhsJKMj

#276 **Neo Kim** System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

If you want to get started with system design,
you must learn these 101 concepts (save now):

1 Scalability
2 Availability
3 Reliability
4 Latency
5 Throughput
6 Capacity
7 Client-Server
8 Database
9 SQL vs NoSQL
10 Load Balancing
11 Caching
12 Cache Invalidation
13 CDN
14 DNS
15 API Design
16 REST
17 GraphQL
18 gRPC
19 Authentication
20 Authorization
21 Rate Limiting
22 Fault Tolerance
23 High Availability
24 CAP Theorem
25 Consistency Models
26 Replication
27 Partitioning
28 Sharding
29 Indexing
30 Denormalization
31 ACID
32 BASE
33 Microservices
34 Monolith
35 Event-Driven
36 Message Queue
37 Pub/Sub
38 Sync vs Async
39 Idempotency
40 Backpressure

41 Circuit Breaker
42 Bulkhead
43 Retry Logic
44 Timeout
45 Service Discovery
46 API Gateway
47 Load Shedding
48 Autoscaling
49 Blue-Green
50 Canary Release
51 Feature Flags
52 Observability
53 Logging
54 Metrics
55 Tracing
56 Correlation ID
57 Monitoring
58 Alerting
59 Full-Text Search
60 Time Series
61 Vector DB
62 Materialized View
63 Query Optimization
64 Connection Pooling
65 Cache Stampede
66 Cache Warming
67 CDN Caching
68 Data Compression
69 Serialization
70 Deserialization
71 WebSockets
72 WebRTC
73 CQRS
74 Event Sourcing
75 Service Mesh
76 Sidecar
77 BFF
78 Strangler Pattern
79 LSM Trees
80 B-Trees
81 Merkle Trees
82 Bloom Filter
83 HyperLogLog
84 MapReduce
85 Batch Processing
86 Stream Processing

87 ETL
88 Data Pipeline
89 Data Lake
90 Data Warehouse
91 Secrets Management
92 RBAC
93 SSO
94 Encryption
95 Checksum
96 Erasure Coding
97 Consensus
98 Leader Election
(...and much more)
What else should make this list?
===
PS - Want a detailed breakdown of each concept?
Read my letters right now:
Part 1: https://lnkd.in/eK87NkE5
Part 2: https://lnkd.in/dicBp9N4
Part 3: https://lnkd.in/dDKk3-Xw
Challenges: https://lnkd.in/evqANxQv
===
Repost to help others master system design.
Follow Neo Kim + turn on notifications.

MEDIA System Design Concepts 101 for Software Engineers

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eK87NkE5
- https://lnkd.in/dicBp9N4
- https://lnkd.in/dDKk3-Xw
- https://lnkd.in/evqANxQv

#277 Mansi Kaushik

System Design

Java Technical Lead | 10+ Years Experience | Java, Spring Boot & Microservices | CI/CD | Banking & Financial Services | Scaling Systems for 1M+ Daily Transactions

2mo · **View on LinkedIn**

Inside Paytm’s System Design: A Public Architecture Deep Dive

As a Java backend engineer, I’ve always been fascinated by how large-scale fintech platforms handle millions of users, high transaction volumes, and strict security requirements.

To better understand these challenges, I created this Paytm System Design infographic by studying publicly available engineering resources, technical talks, industry practices, and publicly known architectural patterns used in modern fintech systems.

The infographic covers:

✓ User Channels & Edge Layer
✓ API Gateway & Load Balancers
✓ Domain-Driven Microservices
✓ Event-Driven Architecture with Kafka

✓ Service Discovery & Configuration
✓ SQL & NoSQL Databases
✓ Redis Caching & Object Storage
✓ Kubernetes & Cloud Infrastructure
✓ Security, Authentication & Encryption
✓ Monitoring, Logging & Distributed Tracing
✓ DevOps & CI/CD Pipeline
✓ Disaster Recovery & High Availability

Building payment platforms isn't just about processing transactions—it's about designing systems that are scalable, fault-tolerant, secure, observable, and resilient while maintaining low latency and high availability.

(!) Disclaimer: This diagram is an educational representation based on publicly available information and widely adopted distributed system patterns. It is not an official or proprietary Paytm architecture.

If you're preparing for System Design interviews, working with Spring Boot & Microservices, or interested in large-scale backend engineering, I hope this visual serves as a helpful learning resource.

What additional component or design pattern would you include in a payment platform of this scale? I'd love to hear your thoughts.

#SystemDesign #Paytm #Java #SpringBoot #Microservices #AWS #Kafka #Kubernetes #Docker #DistributedSystems #SoftwareArchitecture #BackendEngineering #CloudComputing #Scalability #HighAvailability #FinTech #DevOps #Observability #TechCommunity #CreatedByMansiKaushik

#278 Siddharth Kanojia System Design

DevOps Engineer @Neosoft | Ex-Guardians | Azure & AWS | Terraform | CI/CD | Docker | Kubernetes | Infrastructure as Code | Cloud Automation

2mo · **View on LinkedIn**

Load Balancer vs NGINX (Reverse Proxy) | What's the Difference?

One of the most common interview questions for Cloud, DevOps, and Platform Engineers is:

"If NGINX can distribute traffic, why do we still need a Load Balancer?"

The answer is simple:

→ They solve different problems, although their capabilities sometimes overlap.

Let's break it down.

Load Balancer

A Load Balancer is responsible for distributing incoming traffic across multiple backend servers.

✓ Key Responsibilities
✓ Distributes traffic evenly across servers
✓ Improves High Availability
✓ Prevents a single server from becoming overloaded
✓ Performs Health Checks
✓ Automatically removes unhealthy servers
✓ Supports Layer 4 (TCP/UDP) and Layer 7 (HTTP/HTTPS) load balancing

Common Examples

• Azure Load Balancer
• Azure Application Gateway
• AWS ALB / NLB
• Google Cloud Load Balancer

NGINX (Reverse Proxy)

NGINX sits between the client and your application.

Instead of simply distributing traffic, it adds several Layer 7 capabilities.

✓ Key Responsibilities

✓ Reverse Proxy

✓ SSL/TLS Termination

✓ URL Routing

✓ HTTP Compression

✓ Static File Serving

✓ Request Caching

✓ Security Headers

✓ Can also perform Layer 7 Load Balancing

So... Which One Should You Use?

→ If your goal is:

✓ High Availability

✓ Health Checks

✓ Traffic Distribution

→ Use a Load Balancer

→ If your goal is:

✓ SSL Offloading

✓ Reverse Proxy

✓ URL-based Routing

✓ Caching

✓ Security

→ Use NGINX

Real-World Architecture

A common production setup looks like this:

Internet -> Cloud Load Balancer ->NGINX (Reverse Proxy) -> Application Servers

✓ The Load Balancer distributes traffic across multiple NGINX instances.

✓ Each NGINX instance handles SSL termination, routing, caching, and forwards requests to the appropriate application.

This combination provides high availability, scalability, security, and better performance.

The best architecture isn't about choosing Load Balancer OR NGINX...

It's about knowing when to use each—and when to combine them.

Save this post for your next Cloud & DevOps interview.

Repost if you found it useful.

↓ Question: In your production environment, do you use NGINX, HAProxy, Traefik, Envoy, or a Cloud Load Balancer?

#DevOps #CloudComputing #NGINX #LoadBalancer #Azure #AWS #Kubernetes #ReverseProxy #Networking #Linux #PlatformEngineering #SRE #CloudEngineer #Terraform #InterviewPreparation

#279 Raul Junco System Design

Simplifying System Design

2mo · **View on LinkedIn**

I'm always learning from history, and one of my favorite teachers is Feynman.

He kept a list of 12 favorite problems in his mind. Whenever he learned something new, he tested whether it could help solve one of them.

These are mine for system design:

1. URL Shortener → Key generation and caching
2. Rate Limiter → Traffic control and fairness
3. Notification System → Queues and delivery guarantees
4. Chat Application → Real-time communication
5. News Feed → Fan-out and ranking
6. Search Engine → Indexing and retrieval
7. Payment System → Idempotency and consistency
8. Ticket Booking → Concurrency control
9. File Storage → Partitioning and replication
10. Video Platform → Processing pipelines and CDNs
11. Ride-Sharing App → Geospatial indexing and matching
12. Collaborative Editor → Conflict resolution

You don’t need to memorize 100 architectures.

Study a small set of problems deeply, then reuse the lessons everywhere.

What system design problem would you add to the list?

#280 **Nikki Siapno** AI / LLM

Eng Manager & Founder | Ex-Canva | 450k+ Community | Helping Senior Engineers Master System Design and AI Engineering | View my newsletter

2mo · **View on LinkedIn**

8 RAG Architectures Developers Should Know

RAG isn't one fixed architecture. It's a spectrum of retrieval approaches that determine what gets searched, how results are ranked, and how much control the LLM has over retrieval.

Knowing these architectures is important. But understanding how to evaluate and choose the right one for your application is just as important. Learn more about that here: https://lnkd.in/gvwvWMnD

1. Naive RAG

↳ Retrieves the top matching chunks from a vector store and passes them directly to the LLM.

2. Hybrid RAG

↳ Combines keyword search with vector search, which helps when both exact terms and semantic meaning matter.

3. Reranked RAG

↳ Retrieves a larger candidate set first, then scores and reorders chunks so the most relevant context gets used.

4. Multi-Query RAG

↳ Rewrites one question into multiple related queries to improve recall across different phrasings and document matches.

5. Hierarchical RAG

↳ Searches broad document sections first, then narrows into smaller chunks for more precise retrieval.

6. Graph RAG

↳ Represents knowledge as entities and relationships, which helps when the answer depends on connected facts.

7. Corrective RAG

↳ Checks whether retrieved context is useful and adjusts the retrieval process when the first results are weak.

8. Agentic RAG

↳ Lets the LLM plan retrieval steps, choose tools or sources, and reason through multiple searches before answering.

Here's a simple mental model: Naive = fast and simple. Hybrid + Reranked = better precision. Multi-Query + Hierarchical = broader coverage. Graph = connected facts. Corrective = self-checking. Agentic = autonomous reasoning.

RAG helps models retrieve knowledge. Memory systems help them retain it. If you want to learn how production AI evolves beyond RAG into stateful memory systems, here's a great deep dive → https://lnkd.in/gcjfge56

What else would you add?

——

Repost to help others learn RAG.

Thanks to Oracle for sponsoring this post.

+ Follow me (Nikki Siapno) to improve at system design.

MEDIA document/carousel (possible)

LINKS

- https://lucode.co/luc-system-design-newsletter-pl1
- https://lnkd.in/gvwvWMnD
- https://lnkd.in/gcjfge56

#281 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

I built a GitHub repo to help you learn system design.

It gives you:

• System design foundations,

• AI engineering fundamentals,

• Top System design interview questions,

• Deep dives into real-world architectures,

• Simplified engineering case studies with visuals.

PS - Star and/or save this repository now if you find it valuable:

→ https://lnkd.in/eKATU6QV

===

Repost to help others learn AI engineering & system design.

Follow Neo Kim + turn on notifications.

MEDIA GitHub repo for software engineers

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eKATU6QV

#282 Shalini Goyal

System Design

Executive Director, AI & Engineering @ JPMorgan | Amazon Alum | Author · Speaker · Professor | Helping Engineers Break into AI & High-Impact Careers

2mo · **View on LinkedIn**

12 System Design Patterns Simplified

Understanding system design patterns is essential for building reliable, scalable systems. Here’s a simplified breakdown with real-world analogies.

1. Failure Handling Patterns
Handle service crashes and recovery smartly.
- Circuit Breaker: Stops calling failing services to prevent bigger failures.
- Retry Pattern: Automatically retries failed requests after a delay.
- Timeout Pattern: Stops waiting after a set time to avoid blocking.
2. Traffic & Load Control
Avoid overloading your systems under high demand.
- Rate Limiter: Controls request frequency to prevent system overload.
- Bulkhead: Isolates failures to prevent cascading issues.
3. Caching Patterns
Improve speed and performance using smart data caching.
- Cache Aside: Loads data into cache only when needed.
- Write-Through Cache: Updates cache and database together for consistency.
4. Messaging & State Tracking
Ensure smooth communication and traceable actions.
- Publish-Subscribe: Sends messages to multiple listeners asynchronously.
- Event Sourcing: Logs events to recreate system state later.
5. Architecture & Data Flow
Make systems scalable, maintainable, and modular.
- CQRS: Splits reads and writes for performance.
- Strangler Fig: Gradually replaces legacy code without full rewrites.
- Saga Pattern: Manages multi-step transactions with rollback support.
Mastering these patterns gives you a strong edge in interviews and real-world systems.
Follow me on LinkedIn at Shalini Goyal
Follow me on Twitter at https://lnkd.in/eZ_uibMK

LINKS
- https://lnkd.in/eZ_uibMK

#283 Aritri Mukherjee

Java / Spring

Software Engineer Level 1 at Keross R&D
2mo · **View on LinkedIn**

5 Spring Boot Mistakes I Stopped Making After 4 Years as a Java Developer
After working with Java and Spring Boot for the past 4 years, I've realized that writing code is only a small part of building reliable applications.
Here are 5 mistakes I learned to avoid—sometimes the hard way.
1. Putting Business Logic Inside Controllers
Initially, my controllers became bloated with validations and business rules.
✓ Lesson: Keep controllers thin. Let the service layer handle business logic.
---
2. Ignoring Global Exception Handling
I used to wrap every API in multiple "try-catch" blocks.
✓ Lesson: Using "@ControllerAdvice" and centralized exception handling makes APIs cleaner, easier to maintain, and more consistent.
---
3. Fetching Too Much Data
Returning entire entities from APIs seemed convenient.

✓ Lesson: Use DTOs to expose only what clients need. It improves security, reduces payload size, and decouples your API from the database model.

---

4. Not Thinking About Database Performance

I once believed that if the API worked, it was good enough.

✓ Lesson: Understanding lazy vs. eager loading, proper indexing, avoiding N+1 queries, and writing efficient SQL makes a huge difference in production.

---

5. Skipping Proper Logging

Using "System.out.println()" felt quick during development.

✓ Lesson: Structured logging with meaningful log levels helps diagnose issues much faster in real-world applications.

One thing I've learned is that writing code is easy; writing maintainable code is what makes you grow as a developer.

I'm still learning every day, and that's what makes software development exciting.

What Spring Boot lesson took you the longest to learn? Share it in the comments—I'd love to hear your experience.

#Java #SpringBoot #BackendDevelopment #SoftwareEngineering #Programming #JavaDeveloper #Microservices #RESTAPI #Coding #Developers #TechCareer #Learning

MEDIA advertisement

#284 Neo Kim System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

Most people think Redis is just a cache.

It is not:

Redis is an in-memory database that stores data in RAM for very low latency.

Also it provides data structures & atomic operations.

Here are the most popular Redis use cases:

1 Caching

2 Queue

3 Distributed Locking

4 Throttling

5 Session Store

6 Rate Limiting

7 Leaderboards

8 HyperLogLog

9 Pub/Sub

10 Geospatial

11 Time Series

12 Timeline

13 Full-Text Search

14 JSON Storage

15 Graph Database

16 Job Scheduler

17 Feed Deduplication
18 Distributed Counter
19 Shopping Cart
20 Idempotency Keys
What else should make this list?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 245K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others learn Redis use cases.
Follow Neo Kim + turn on notifications.

MEDIA Top 20 redis use cases for software engineers

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#285 Harjeet Singh

System Design

Sr. Software Architect | Real-Time & Cloud-Native Platforms | FinTech | E-Commerce & Data Engineering | 11+ Years Experience

2mo · **View on LinkedIn**

Building a Microservices Architecture is Easy. Building a Team That Can Build One is the Real Challenge.
When we started our microservices journey, I wasn’t starting from zero.
I had experience designing distributed systems, but most of the team was completely new to microservices.
Many had never worked with:
* Service boundaries
* API Gateways
* Event-driven architecture
* Kafka/RabbitMQ
* Circuit Breakers
* Distributed transactions
* Observability
* Kubernetes
The easiest option would have been to design everything myself.
But that would have created a system dependent on one person.
Instead, I chose the harder path.
Every architecture decision became a discussion.
Every design review became a learning session.
Every production incident became a classroom.
There were mistakes.
Services that were too tightly coupled.
Synchronous calls where asynchronous events made more sense.
Missing monitoring.
Deployment challenges.

Database ownership issues.
Instead of asking, “Who made this mistake?”, we asked,
“What should we improve so nobody makes it again?”
Over time, something interesting happened.
The same engineers who once asked,
“What exactly is a bounded context?”
started debating:
* Event choreography vs orchestration
* Database-per-service trade-offs
* Saga patterns
* Idempotency
* API versioning
* Service scalability
That was the real success.
Not the architecture.
Not the technology.
But watching a team grow into architects.
A great system is temporary.
A team that knows how to build great systems creates value for years.
Technology scales systems. Knowledge scales teams.
#Microservices #SoftwareArchitecture #Leadership #Engineering #SystemDesign #Mentoring #Kafka #Kubernetes #DistributedSystems

#286 Niladri Chatterjee

AI / LLM

VP Domain Architect | Payments Modernization | API-First & Microservices | Data Mesh, Cloud Migration | EA Governance | Banking Payments & ISO 20022 | SRE Observability Leader | AI & Agentic AI | Digital Transformation

2mo · **View on LinkedIn**

We spent years making payments faster. Now we need to make them smarter.
The payments industry has moved from batch processing to real-time payments.
But speed alone doesn't create a great payment experience.
A payment can be completed in seconds and still be:
• Expensive
• High-risk
• Routed inefficiently
• Difficult to reconcile
• Poorly monitored
• Challenging to settle across borders
The next generation of payment architecture needs to make real-time decisions, not just real-time transactions.
Imagine a payment platform that can dynamically evaluate:
Routing → Which rail should process the transaction?
Risk → Is the transaction safe enough to proceed?
Cost → What is the most efficient processing route?
Liquidity → Where should funds be positioned?
Compliance → Can the payment move within regulatory requirements?
Customer Experience → Can all of this happen without adding friction?

This is where payment orchestration, AI, APIs, real-time analytics and multi-rail architectures become increasingly important.

Looking back, every major payment evolution solved a different problem:

Batch → Reliability

Cards → Convenience

Digital Payments → Accessibility

Real-Time Payments → Speed

Next Generation → Intelligent Decision-Making

The future of payments won't simply be about moving money faster.

It will be about deciding smarter, routing better and creating a resilient payment experience in real time.

The real question for payment leaders is: Are we building faster payment systems, or smarter payment ecosystems?

#Payments #PaymentArchitecture #PaymentOrchestration #PaymentModernization #FinTech #Banking #RealTimePayments #DigitalPayments #APIArchitecture #EnterpriseArchitecture #AI #PaymentInfrastructure #FinancialServices

MEDIA advertisement

#287 Saptarshi Dey

System Design

Senior Backend Engineer | Distributed Systems, REST APIs & Microservices | Python, Node.js, AWS | Scalable Data & Payment Systems | AI-Enabled Development

2mo · **View on LinkedIn**

Every payment looks simple to the customer. Behind the scenes, it's a series of smart decisions.

One diagram recently caught my attention because it explains payment orchestration in a very simple way.

When a customer clicks "Pay", the payment doesn't just go to a single processor. A payment orchestrator acts like an air traffic controller, deciding the best route in real time based on factors like card type, BIN, geography, currency, processor health, approval rates, latency, cost and even automatic failover.

The goal is straightforward: increase successful payments while keeping the experience fast and reliable.

As a Backend Engineer working with Python, AWS, Kafka, SQL, and NoSQL, I find this space incredibly exciting. Building systems that can process millions of events, make intelligent routing decisions in milliseconds and remain resilient during failures is a fascinating engineering challenge.

What I like most is that payment orchestration isn't just about moving money. It's about creating systems that learn, adapt, and make better decisions over time. Better routing means higher approval rates, lower costs for merchants, and a smoother experience for customers who never have to think about what's happening behind the scenes.

With AI, real-time analytics, and event-driven architectures becoming more powerful, I believe payment orchestration will continue to evolve into one of the most impactful areas of backend engineering.

There's still so much to learn, and that's what makes this journey exciting. Looking forward to building systems that are not just scalable, but also intelligent and resilient.

#BackendEngineering #Python #AWS #Kafka #SystemDesign #Payments #FinTech #SoftwareEngineering #CloudComputing #TechGrowth

#288 Deepak Kumar

AI / LLM

Senior Software Engineer L4 | Java • Spring Boot • Microservices • AWS • MySQL | Building scalable cloud systems

2mo · View on LinkedIn

Java + AI: The Next Evolution of Enterprise Banking

Artificial Intelligence isn't replacing enterprise Java applications - it's making them smarter.

By combining Spring Boot + Spring AI + LLMs + MCP + Vector Database + RAG, banks can build intelligent, secure, and context-aware applications without replacing their existing systems.

Real-world Banking Use Cases

AI Banking Assistant

Intelligent Document Processing (KYC, Statements)

Fraud Detection & Risk Analysis

Internal Knowledge Assistant

Automated Reports & Business Insights

Enterprise AI Architecture

✓ Spring Boot + Spring AI

✓ OpenAI / Claude / Gemini

✓ MCP (Model Context Protocol)

✓ Vector Database

✓ RAG (Retrieval-Augmented Generation)

The next generation of banking isn't about rewriting legacy systems - it's about enhancing them with AI.

The future belongs to engineers who can bridge Enterprise Java with Generative AI.

#Java #SpringBoot #SpringAI #ArtificialIntelligence #GenerativeAI #Banking #FinTech #RAG #MCP #OpenAI #SoftwareArchitecture #EnterpriseArchitecture #TechLeadership

MEDIA document/carousel (possible)

#289 Asha Shanmugam

Other

AML/KYC Analyst | Transaction Monitoring | CDD | EDD | KYB | actively seeking for new opportunity, immediate joiner

2mo · View on LinkedIn

What is Transaction Monitoring?

Transaction Monitoring means checking customer transactions regularly to find suspicious or unusual activities that may be related to:

Fraud

Money Laundering

Financial Crime

Example: A customer usually sends ₹10,000 every month, but suddenly transfers ₹10 lakh to another country. This is unusual, so the system creates an alert

Why is Transaction Monitoring Important?

Detects suspicious transactions

Prevents fraud

Identifies money laundering

Protects customers and banks

Follows AML regulations

Common Types of Fraud

1. Money Mule Fraud

A person allows their bank account to be used to receive or transfer illegal money.
Example: Someone offers ₹5,000 to use your bank account for transferring money.
2. Account Takeover (ATO)
A fraudster gets unauthorized access to a customer's account.
Example: Hacker steals the customer's password and logs into their bank account.
3. Identity Theft
A fraudster uses someone else's personal information.
Example: Using another person's Aadhaar or PAN card to open a bank account.
4. Card Fraud
Someone uses a debit or credit card without permission.
Example: Stolen card used for online shopping.
5. Phishing Scam
Fraudsters trick customers into sharing passwords or OTPs.
Example: Fake bank SMS asking you to click a link and enter your banking details.
6. Authorized Push Payment (APP) Fraud
The customer is tricked into sending money themselves.
Example: A fake investment agent convinces the customer to transfer money.
7. Business Email Compromise (BEC)
Fraudsters send fake business emails to steal payments.
Example: A fake email asks the finance team to transfer money to a new bank account.
8. Transaction Laundering
Illegal business transactions are processed through a legitimate business account.
Example: A fake online gambling business uses a genuine merchant account to receive payments.
Steps in Transaction Monitoring
Customer makes a transaction.
The system detects unusual activity and creates an alert.
Analyst reviews the alert.
Analyst investigates the case.
If suspicious, the case is reported (SAR/STR); otherwise, it is closed.
Common Red Flags
Large or unusual transactions
Frequent cash deposits or withdrawals
Rapid movement of money
Transactions with high-risk countries
Multiple failed login attempts
Sudden change in customer transaction behavior
Interview Answer (Simple)
Q: What is Transaction Monitoring?
Answer:
"Transaction Monitoring is the process of reviewing customer transactions to identify suspicious activities, prevent fraud and money laundering, and ensure AML compliance. If the system detects unusual activity, it generates an alert. The analyst reviews, investigates, and either closes the case or reports it if necessary."

#290 Rishi Jha AI / LLM

Backend Engineer | Core Banking & Payments | Java • Spring Boot • Kafka | Payment Systems • Distributed Systems | Banking & Fintech

2mo · View on LinkedIn

How Banks Detect Card Fraud in Under 100 ms

Every time you tap your card, an incredible amount of analysis happens before your transaction is approved—usually in less than 100 milliseconds.

Let's see what happens behind the scenes.

Step 1: Transaction Initiated

You tap your card at a POS terminal.

An ISO 8583 authorization request is created and sent through:

POS Terminal

↓

Acquirer Bank

↓

Visa / Mastercard

↓

Issuer Bank

The issuer now has only a few milliseconds to decide whether the transaction is genuine.

Step 2: Fraud Engine Takes Over

Before checking your account balance, the issuer's Fraud Detection Engine evaluates the transaction using hundreds of rules and AI models.

It analyzes signals such as:

Location Check

Is the transaction happening in a location consistent with your recent activity?

Example: A purchase in London just minutes after one in Delhi is suspicious.

Transaction Amount

Is the amount unusual for this cardholder?

Velocity Check

Have there been multiple transactions within a very short time?

Example: 5 purchases in 2 minutes.

Merchant Category (MCC)

Does the merchant type match your normal spending behavior?

Device & Channel

Is this a trusted device or payment channel?

Behavioral Analysis

Does this transaction fit your historical spending pattern?

Blacklist & Watchlists

Is the card, merchant, IP address, or device already flagged?

Step 3: AI Generates a Risk Score

All these checks are combined to calculate a risk score.

Risk Score < 30

↓

Approve ✓

Risk Score 30–70

↓

Step-up Authentication (OTP / 3DS)

Risk Score > 70

↓

Decline ✗

This decision is made in just a few milliseconds.

Example Timeline

0 ms → Card tapped

20 ms → Authorization reaches issuer

45 ms → Fraud engine evaluates risk

75 ms → Decision made

95 ms → Response reaches POS

The customer only notices a brief "Processing..." message, while the bank has already analyzed hundreds of data points.

Why It Matters

Modern fraud detection isn't based on a single rule.

Banks use a combination of:

Rule-based engines

Machine Learning models

Real-time behavioral analytics

Device fingerprinting

Historical transaction patterns

to stop fraudulent transactions before money leaves the account.

Key Takeaway

Banks don't just check your balance—they evaluate every transaction against hundreds of risk signals in under 100 milliseconds before deciding whether to approve or decline it.

Every time you tap your card, an AI-powered fraud engine races against the clock—analyzing hundreds of signals and making a decision in under 100 milliseconds. That's the invisible technology protecting billions of transactions every day.

#291 **Adarsh C.** System Design

DevRel Engineer | Sharing the latest developments in Tech, Web3, and AI | Content Creator | Exploring ways to excel at the game of life

2mo · **View on LinkedIn**

I spent the last few days turning Networking into a visual cheat sheet.

Most developers memorize commands.

Few understand how data actually moves across the internet.

That's where interviews, debugging, and system design become much easier.

So I created a 30 Networking Concepts Every Developer Should Know guide covering everything from networking fundamentals to real-world debugging.

Here's what you'll learn:

✓ Networking Fundamentals

* IP Address (IPv4 vs IPv6)

* MAC Address

* LAN, MAN & WAN

* Network Topologies

✓ Network Models

* OSI Model

* TCP/IP Model
* Packets & Frames
* Encapsulation vs Decapsulation
✓ Core Protocols
* TCP vs UDP
* HTTP vs HTTPS
* DNS
* Three-Way Handshake
✓ Routing & Communication
* Routers & Switches
* Default Gateway
* Routing Tables
* CIDR & Subnetting
✓ Performance & Scalability
* Latency
* Bandwidth vs Throughput
* Load Balancing
* CDN
✓ Security
* Firewall
* VPN
* Proxy vs Reverse Proxy
* TLS/SSL
✓ Debugging Tools
* ping
* traceroute
* nslookup / dig
* curl
* netstat / ss
* tcpdump
* Wireshark
Networking isn't just for DevOps or Network Engineers.
It's a core skill for:
* Software Engineers
* Backend Developers
* System Design Interviews
* Cloud Engineers
* SREs
* Security Engineers
The better you understand networking, the easier it becomes to debug production issues, optimize distributed systems, and design scalable applications.
What's the networking concept you struggled with the most when starting out?
Get the Cheatsheet in your DMs directly: https://lnkd.in/e-hJY5fG
#interview #interviewpro #interviewprep #dsa #coding #git #java #adarshchetan
Do follow for more such amazing stuff
30 Networking Concepts Every D

.

8 pages

MEDIA document/carousel (possible)

LINKS
- https://lnkd.in/e-hJY5fG

#292 Muhammad Nouman

System Design

Full-Stack Developer for European SaaS & Startups | Next.js · NestJS · React Native · PostgreSQL | AI/RAG Integration | Remote, CET-overlap | Open to freelance & contract projects

2mo · **View on LinkedIn**

15 Must-Know Networking Concepts Every Developer Should Know

Whether you're building APIs, deploying microservices, or debugging production issues- networking is a must-have skill.

Here's a quick cheat sheet ↓

IP Address -> Identifies every device on a network.

DNS -> Converts domain names into IP addresses.

Ports -> Allow multiple services to run on one machine.

MAC Address -> Unique hardware identifier for a network device.

NAT -> Maps private IPs to a public IP.

DHCP -> Automatically assigns IP addresses.

Packets -> Data travels across networks in small chunks.

OSI Model -> Understand how network communication works layer by layer.

Subnetting & CIDR -> Organize and optimize IP networks.

Router vs Switch -> Routers connect networks, switches connect devices.

TCP vs UDP -> Reliability vs speed.

HTTP vs HTTPS -> Secure web communication with TLS.

TLS -> Encrypts data between client and server.

Firewall -> Filters traffic and blocks unauthorized access.

VPN -> Creates a secure private tunnel over the internet.

Quick Takeaway:

Most production issues eventually come down to networking. Mastering these fundamentals, and debugging distributed systems becomes much easier.

Follow Muhammad Nouman for more such System Design & Software Engineering insights.

#Networking #SystemDesign #CloudComputing #TCPIP #HTTP #DNS

w3schools.com JavaScript Mastery JavaScript Developer

LINKS
- http://w3schools.com

#293 Ashish Pratap Singh

Other

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

2mo · **View on LinkedIn**

If you want to get good at computer networks, learn these 15 concepts:

1. IP Address: Identifies a device on a network

2. DNS: Converts domain names into IP addresses

3. Ports: Allow multiple services to run on the same machine

4. MAC Address: Identifies a network interface within a local network
5. NAT: Maps multiple private IP addresses to a public IP
6. DHCP: Automatically assigns network configuration to devices
7. Packets: Small chunks of data transmitted across a network
8. OSI Model: Explains networking through seven conceptual layers
9. Subnetting and CIDR: Divide networks and define IP address ranges
10. Router vs Switch: Routers connect networks; switches connect devices within a network
11. TCP vs UDP: Reliable ordered delivery versus faster connectionless communication
12. HTTP and HTTPS: Protocols used for communication between web clients and servers
13. TLS: Encrypts data exchanged over a network
14. Firewall: Controls incoming and outgoing network traffic
15. VPN: Creates an encrypted tunnel over a public network

I wrote an article explaining these concepts in detail. Read it here: https://lnkd.in/gTieAMNE

Which other networking concept would you add to this list? Let me know in the comments.

Repost to help others in your network

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/gTieAMNE

#294 **Pravin Chalak** **System Design**

IT Director / CIO Track | AI Strategy & Digital Transformation Executive | Senior Technical Program Manager | Core Banking, Lending, Payments & Digital Modernization with AI-Driven Transformation

2mo · **View on LinkedIn**

From Monoliths to Microservices to AI-Governed Architectures: The Next Evolution of Enterprise Banking Systems

One of my key learnings from banking and enterprise modernization programs is that architecture evolution has never been purely about technology. Monolithic systems delivered stability and transactional integrity, but over time they also created long release cycles, complex integrations, growing technical debt, and higher costs of change.

The industry responded with APIs, microservices, cloud-native platforms, and event-driven architectures. These brought agility, scalability, and faster innovation, but also introduced new challenges such as distributed complexity, service sprawl, data fragmentation, orchestration overhead, and governance concerns.

Today, organizations are realizing that breaking a monolith into microservices is only part of the journey. Core refactoring, technical debt reduction, security, observability, integration modernization, and enterprise controls are equally critical. The focus is shifting from processing transactions to generating intelligence and business value from data.

This is where AI-Governed Architecture emerges. Just as API Gateways became the control plane for integrations, AI Gateways are becoming the control plane for enterprise AI, providing governance, security, compliance, observability, auditability, and cost control.

We're seeing architecture evolve across multiple dimensions:

-> Integration: Point-to-Point → APIs → Event-Driven Ecosystems

-> Governance: Application → API → AI Governance

-> Security: Perimeter → Zero Trust → AI Security

-> Operations: Monitoring → Observability → AIOps

-> Data: Databases → Data Fabric → Intelligence Platforms

My biggest takeaway is that future competitive advantage may not come from AI models alone. It will come from how effectively organizations combine data, integration, orchestration, governance, security, and intelligence into a trusted enterprise platform.

The conversation is no longer Monolith vs Microservices. The strategic question is:

How do we evolve from Microservices to AI-Governed Enterprises while reducing technical debt, modernizing core systems, strengthening controls, and maintaining business trust?

APIs gave us connectivity. Microservices gave us agility. Event-driven architectures gave us responsiveness. AI brings intelligence. Governance provides trust.

The next decade of enterprise architecture may be defined by how well we bring all of these together.

#EnterpriseArchitecture #DigitalTransformation #Microservices #AIGovernance #AIGateway #APIGateway #IntegrationArchitecture #EventDrivenArchitecture #TechnicalDebt #CloudNative #AgenticAI #AIOps #TechnologyLeadership #CIO #CTO #BankingTechnology #FutureOfArchitecture

#295 Arpit Aggarwal

Other

Lead Software Engineer @ Mastercard | Engineering Leadership | Java Full Stack | Generative AI, RAG & Agentic AI

2mo · View on LinkedIn

Tokenization: The Invisible Technology That Makes Digital Payments Safer

Every time you save your card on an e-commerce website or make a one-click payment, there’s a good chance tokenization is working behind the scenes to protect your financial information.

Instead of storing your actual card number (PAN), merchants store a payment token—a unique, non-sensitive reference that has no value outside its intended environment. The real card details remain securely protected in a Token Vault, managed by the payment network.

Why is this a game changer?

✓ Protects sensitive card data by replacing it with tokens.

✓ Reduces the impact of data breaches since stolen tokens cannot be used elsewhere.

✓ Lowers PCI-DSS compliance scope, reducing operational complexity for merchants.

✓ Enables safer card-on-file experiences for subscriptions, recurring payments, and one-click checkout.

✓ Builds consumer trust without compromising payment convenience.

As digital payments continue to grow, security can no longer rely solely on encryption. Encryption protects data during transmission and storage, while tokenization ensures that sensitive data often doesn’t need to be stored at all. Together, they create a much stronger security model.

Sometimes the best security is making sure the most valuable data is never exposed in the first place.

#Tokenization #PaymentSecurity #CyberSecurity #FinTech #DigitalPayments #DataSecurity #PCI #PCIDSS #NetworkTokenization #Encryption #InformationSecurity #CloudSecurity #SoftwareEngineering #Technology #Mastercard #Innovation

#296 Sivasankar Natarajan

AI / LLM

Technical Director | GenAI Practitioner | Azure Cloud Architect | Data & Analytics | Solutioning What’s Next

2mo · View on LinkedIn

Everyone is talking about how powerful AI is. But very few are talking about the seatbelts of AI the guardrails that keep AI safe, reliable, and useful.

Without them, AI can:

- Go off-topic and produce nonsense

- Share unsafe or harmful advice

- Generate broken outputs that crash your app

This is where AI Guardrails come in. Think of them as traffic rules and quality inspectors for AI.

What is an AI Guardrail?

They are rules and checks that make sure AI's answers are safe, correct, and useful. Just like seatbelts protect you from accidents, guardrails protect users from biased, harmful, or wrong outputs.

What is Guard (from Guardrails AI)?

A tool that acts like a quality inspector for AI. Every response goes through checks

- Is it in the right format?

- Is anything missing?

- Is it safe to share?

If the AI fails, Guard can re-ask, filter, fix, or block until you get a reliable output.

How Guard Works (in simple steps)

1. Define what type of output you want (spec, model, or string).

2. Add prompts and instructions to guide the AI.

3. Initialize Guard with the rules.

4. Invoke Guard it calls the LLM.

5. The output is validated.

6. If invalid, Guard retries, fixes, or blocks.

7. If valid, the answer is returned safely.

8. Logs are stored so you can trace what happened.

The truth is, without AI Guardrails, every AI system is a ticking time bomb. With guardrails, AI becomes a trustworthy partner.

Do you think AI safety should be optional or mandatory in every AI system?

Repost this to help your network get started

+ Follow Sivasankar for more

#AI #AIagents #AISafety #AgenticAI

#297 **B Rajasekhar Reddy** **Java / Spring**

Java Full-Stack Engineer | Java | Spring Boot | Microservices | React | REST APIs | Spring Security | MySQL | Oracle | Docker

2mo · **View on LinkedIn**

Spring Boot Explained | Part 1

?Why was Spring Boot introduced?

To answer that, let's take a step back and look at how software development has evolved over the years.

Programming Language

↓

Technology

↓

Library

↓

Framework

↓

Spring Framework

↓

Spring Boot

Every evolution was driven by one goal: **solving the limitations of the previous stage.**

Programming languages like Java allow us to write business logic, but they don't provide enterprise features such as dependency injection, web servers, or security.

Technologies like JDBC, Servlets, and JPA solve specific problems, but integrating them into a complete application is still the developer's responsibility.

Libraries make development easier by providing reusable code, but you remain responsible for object creation, dependency management, and application flow.

Frameworks changed this by introducing **Inversion of Control (IoC)**, where the framework manages the application's lifecycle and invokes your code when needed.

Spring Framework revolutionized enterprise Java development with features like Dependency Injection, MVC, Transaction Management, and Security.

However, developers still spent a significant amount of time configuring applications instead of building business features.

That's where **Spring Boot** changed the game.

✓ Auto Configuration

✓ Starter Dependencies

✓ Embedded Server

✓ Production-Ready Features

Spring Boot didn't replace Spring Framework—it made it faster, simpler, and more productive.

Key Takeaway:

Every technology evolves to solve the limitations of its predecessor.

→ Swipe through the infographic to understand this evolution visually.

Which Spring Boot feature has saved you the most development time?

#Java #Spring #SpringBoot #BackendDevelopment #JavaDeveloper #SoftwareEngineering #LearningInPublic

1/8

MEDIA document/carousel (possible)

#298 Aashi Gangrade

Kafka / Messaging

Software Engineer 2 @ Intuit | Distributed Systems | Scalable Backend Services | Cloud & AI-Native Development

2mo · **View on LinkedIn**

If someone asked you to explain the difference between Kafka and RabbitMQ, how would you answer? They often get grouped together, but they solve fundamentally different problems.

Think of RabbitMQ as a busy restaurant kitchen, and Kafka as a radio broadcast.

Here is how I break it down:

RabbitMQ (The Restaurant Kitchen) Waiters put order tickets on a rail, and chefs (your worker apps) grab them as fast as possible. Once a meal is cooked, the ticket is thrown in the trash. It’s all about workers competing to empty a queue. If tasks finish out of order, that's completely fine.

Kafka (The Radio Broadcast) Kafka doesn't want workers to compete; it wants multiple systems to listen. If a user updates their profile, your shipping, billing, and marketing apps all need to hear that exact same message. Kafka doesn't throw the message away - it keeps it on a log so systems can replay the history later.

(Interestingly, I also learned that Apache Pulsar can actually do both of these patterns on the same platform, which is pretty cool).

I wrote up a short Medium article diving a bit deeper into this analogy and how the mechanics work under the hood.

Link to the full read is in the comments! ↓

#Kafka #RabbitMQ #DataArchitecture #EventStreaming #Learning

#299 Imran Shaikh

Java / Spring

Senior Specialist - Technology at Synechron

2mo · View on LinkedIn

How Spring Boot Works Internally – Complete Architecture Explained!

Many Java developers use Spring Boot every day, but understanding what actually happens behind the scenes can take your development skills to the next level.

That's why I created this detailed visual guide explaining the complete internal working of Spring Boot 3.x with the latest features.

What you'll learn:

✓ Application startup lifecycle (SpringApplication.run())

✓ How @SpringBootApplication works internally

✓ Auto Configuration and Conditional Beans

✓ Bean Creation & Dependency Injection

✓ ApplicationContext initialization

✓ Embedded Tomcat/Jetty/Undertow startup

✓ Complete HTTP Request Flow

DispatcherServlet

Handler Mapping

Controller

Service

Repository

Database

Response

✓ Spring Security Filter Chain

✓ Actuator & Production Monitoring

✓ Micrometer Metrics & OpenTelemetry Tracing

✓ Externalized Configuration

application.yml

Profiles

Environment Variables

Config Server

Vault

✓ Spring Boot Starters

✓ AOT Compilation & GraalVM Native Image Support

✓ Virtual Threads support (Project Loom)

✓ Problem Details (RFC 9457)

✓ Production-ready best practices

Why is this important?

When you understand the internals, you can:

✓ Debug issues faster

✓ Improve application performance

✓ Reduce startup time

✓ Write cleaner and more maintainable code

✓ Prepare confidently for Java & Spring Boot interviews

✓ Design scalable enterprise applications

Whether you're a Java Developer, Senior Engineer, Tech Lead, or preparing for interviews, mastering Spring Boot internals is one of the most valuable skills you can develop.

Save this post for future reference and share it with your team if you found it useful.

Which Spring Boot topic would you like me to explain next?

Spring Security Internals

Spring Transactions

Spring AOP

Spring MVC Internals

Spring Data JPA

Kafka with Spring Boot

Microservices Architecture

↓ Let me know in the comments!

#Java #SpringBoot #SpringFramework #JavaDeveloper #BackendDevelopment #SoftwareEngineering #Microservices #SystemDesign #SpringSecurity #SpringMVC #SpringDataJPA #Hibernate #RESTAPI #CloudNative #GraalVM #VirtualThreads #TechLead #Programming #Coding #Developer

LINKS

- http://SpringApplication.run

#300 Spring Users

Java / Spring

Suraj Mishra • 2nd

2mo · **View on LinkedIn**

Spring Boot Interview Quiz : Why Is CPU Low but Latency High?

Context

Your application reserves inventory (see attached screenshot)

During a flash sale:

>> Application CPU stays below 20%

>> Database CPU stays below 30%

>> API latency jumps from 80 ms to 12 seconds

>> Thread dumps show many threads blocked inside findById()

What is the BEST explanation?

A. Spring Data JPA is leaking database connections.

B. The pessimistic lock is held while the external payment validation executes, forcing other transactions to wait.

C. Hibernate retries failed transactions automatically.

D. The database optimizer stopped using indexes because of the lock.

Detailed explanation :

https://lnkd.in/gR4jr6Ne

LINKS

- https://lnkd.in/gR4jr6Ne

#301 Sathish Kumar Subramani

System Design

Senior Engineering Manager | Generative AI | Azure Cloud Solutions | Leadership Excellence | Driving Operational Efficiency

2mo · **View on LinkedIn**

Your app isn't slow because of your code. It's slow because of your database.

I spent two days optimizing an API endpoint last year. Rewrote the logic. Reduced memory allocation. Shaved off milliseconds everywhere I could.

The endpoint was still slow. Then a senior engineer looked at the query behind it and said: "You're missing an index."

One index. Response time dropped 94%. Two days of code optimization. Five minutes of database knowledge.

Here are 9 concepts that would've saved me those two days:

1. Indexing: B-Tree, Hash, Bitmap structures that let the database jump to results instead of scanning every row. The most impactful performance lever most developers ignore. But every index slows writes, so you're trading write speed for read speed.

2. Connection Pooling: Reuse database connections instead of opening a new one per request. Opening connections is expensive. Pooling keeps them warm. The fix for the "works locally, dies under load" problem most teams hit first.

3. Monitoring: Track query performance, CPU, memory, connections, disk I/O, replication lag. The slow query that crashes production at 3pm was visible in metrics at 9am.

4. Sharding : Split data across multiple nodes. Replication scales reads. Sharding scales writes. When one database can't handle the write volume, distribute the data. Powerful but complex, don't shard until you've exhausted simpler options.

5. ACID Transactions: Atomicity, Consistency, Isolation, Durability. Why your bank transfer doesn't vanish halfway through. The guarantees that make relational databases trustworthy for anything involving money, orders, or records.

6. Partitioning: Large tables split into smaller pieces within one database. Not the same as sharding. Partitioning is internal organization. Sharding is external distribution.

7. Backup and Recovery: Full backups plus incrementals. The rule nobody follows until the disaster: an untested backup isn't a backup. It's a hope. Test your restores.

8. Normalization: Reduce redundancy. Improve integrity. Normalize for correctness first. Then denormalize deliberately when read performance demands it.

9. Replication: Primary handles writes. Replicas handle reads. Buys read scale and failover at the cost of eventual consistency. The trade-off is worth understanding deeply, "eventually consistent" means "temporarily wrong."

How they connect:

Indexing makes individual queries fast. Connection pooling handles many queries efficiently. Monitoring catches problems before users do. Normalization keeps data clean. Replication scales reads. Sharding scales writes. Partitioning organizes within.

Get these nine right and most "performance problems" disappear. Because most performance problems were always database problems.

Which one would've saved you the most debugging time?

Repost to help your network get started

+ Follow Sathish for more

#Database #SystemDesign #BackendEngineering

#302 Neo Kim AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

If you want to become top-tier with AI engineering,

learn these 10 concepts (save now):

1 LLM (Large Language Model)

↳ A program that reads massive amounts of text & then learns to predict the next word

↳ Use case: draft text, write code, answer questions
↳ Pro: one model handles thousands of different tasks
↳ Con: expensive & states wrong answers with full confidence (aka hallucination)
2 AI Agent
↳ AI that does the task: plans steps and uses tools
↳ Use case: research reports, email triage, coding
↳ Pro: does multi-step work while you do other tasks
↳ Con: tiny errors stack up across steps
3 Prompt Engineering
↳ Writing instructions clearly so AI gives you exactly what you want
↳ Use case: system prompts, team templates, output formatting
↳ Pro: boosts quality at zero training cost
↳ Con: breaks across model versions
4 Context Engineering
↳ What AI sees before it answers: documents, tools & history
↳ Use case: agent pipelines, memory design, tool selection
↳ Pro: improves agent reliability more than prompt tweaks
↳ Con: no standardized tooling yet
5 Token
↳ Bite-size text chunks (parts of words) that AI actually reads & writes
↳ Use case: API pricing, context limits, speed estimates
↳ Pro: gives you a unit to measure cost
↳ Con: causes weird bugs, such as miscounting letters in a word
6 RAG (Retrieval-Augmented Generation)
↳ AI looks things up in your documents first, then answers using what it found
↳ Use case: docs chatbots, support desks, enterprise search
↳ Pro: pulls in current information without retraining the model
↳ Con: wrong retrieval produces confident wrong answers
7 Fine-Tuning
↳ Take a general AI & train it on your examples until it masters your specific task
↳ Use case: brand voice, domain formats, vertical small models
↳ Pro: changes behavior more consistently than RAG
↳ Con: costs compute & can reduce model's general skills
8 Embeddings
↳ AI turns text into lists of numbers, and similar meanings get placed near each other
↳ Use case: semantic search, recommendations, clustering
↳ Pro: matches meaning where keyword search fails
↳ Con: numbers make debugging hard
9 RLHF (Reinforcement Learning from Human Feedback)
↳ Humans rate AI answers, so AI learns to produce more of what people prefer
↳ Use case: assistant training, safety tuning, preference alignment
↳ Pro: turned raw text predictors into usable assistants
↳ Con: teaches models to sound agreeable, not always to be right
10 MCP (Model Context Protocol)
↳ Universal plug that connects AI to your apps & data
↳ Use case: database access, SaaS connections, file systems

↳ Pro: No need to build custom integrations

↳ Con: security & permissions are not mature yet

What else should make this list?

===

PS - Want my System Design Playbook for FREE?

Join my newsletter with 245K+ software engineers now:

→ https://lnkd.in/ehTrcyak

===

Repost to help others ace AI engineering.

Follow Neo Kim + turn on notifications.

MEDIA 10 AI engineering concepts; document/carousel (possible)

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#303 Java Developers Community (moderated)

System Design

Tauseef Fayyaz • 2nd | Book an appointment

2mo · **View on LinkedIn**

Low Latency Practices are essential

as you are expected to build fast, responsive systems.

Many engineers over-engineer infrastructure and ignore these battle-tested low-latency techniques used by top tech companies.

Here are the guiding principles to help you reduce latency across your stack:

1. For Frequently Accessed Data – Use In-Memory Caching
2. For Slow DB Queries – Use Database Indexing
3. For High Request Volume – Use Connection Pooling
4. For Large Payloads – Use Gzip/Brotli Compression
5. For Global Users – Use CDN for Static Assets
6. For Reducing Round-Trips – Use HTTP/2 Multiplexing
7. For Network Efficiency – Use Request Grouping/Batching
8. For Heavy Background Jobs – Use Async Message Queues
9. For Distributing Load – Use Traffic Load Balancing
10. For Reducing API Delays – Avoid External Dependencies
11. For Near-User Processing – Use Edge Computing
12. For Fast Serialization – Use Protobuf or Avro
13. For Local Performance – Use Vertical Scaling
14. For Faster UX – Use Lazy Loading
15. For Load Sharing – Use Client-Side Rendering
16. For Instant Readiness – Use Prefetching Critical Resources
17. For Read-Heavy Workloads – Use Read Replicas
18. For Multi-Join Queries – Use Denormalized Reads
19. For p99 Tail Latency – Use Hedged Requests
20. For Chatty Services – Colocate in the Same Region

These principles are not theory, they are the foundation of low-latency, high-performance systems at scale (Netflix, Google, Amazon).

Which one gave you the biggest latency win?

Repost to help others grow as software engineer

+ Follow me (Tauseef Fayyaz) for more.

LINKS

- https://www.sefism.com/mentorship/

#304 Java Developers Community (moderated)

Java / Spring

Likhitha V • 3rd+

2mo · View on LinkedIn

Circuit Breaker in Spring Boot

In a microservices architecture, services often communicate over the network, which can introduce failures such as high latency, timeouts, or service crashes.

The Circuit Breaker pattern helps prevent cascading failures by monitoring calls to a remote service and stopping calls if failures exceed a threshold. This prevents overwhelming a failing service and allows it to recover.

Spring Boot provides Circuit Breaker support via Resilience4j, which is a lightweight, functional library designed for fault tolerance.

States of Circuit Breaker

A Circuit Breaker operates in three primary states:

1. Closed State

In this state, all requests are allowed to pass through.

The Circuit Breaker monitors failures (such as timeouts, exceptions).

If the failure rate remains below the configured threshold (e.g., 50% failures out of the last 10 requests), the circuit remains closed.

If failures exceed the threshold, the circuit transitions to Open state.

2. Open State

When the circuit is open, it rejects all requests immediately without calling the remote service.

Instead of making failing requests, it returns a predefined fallback response or throws an exception.

The circuit remains open for a configured timeout period (e.g., 10 seconds).

After the timeout, the circuit transitions to a half-open state to test if the service has recovered.

3. Half-Open State

A limited number of test requests are allowed to pass through.

If these test requests succeed, the circuit transitions back to the Closed state, meaning the service is healthy again.

If the test requests fail, the circuit goes back to the Open state and remains open for another timeout period.

State Transition Diagram

CLOSED ---> OPEN ---> HALF-OPEN ---> CLOSED

| | | |

| Failure | Timeout | Success |

| threshold | period over | threshold |

This mechanism ensures that failing services do not receive excessive load while also periodically testing if they have recovered.

#fullstack #java #TechSimplified #SystemDesign #DistributedSystems #Kafka #Messaging #BackendEngineering #Microservices

#SoftwareArchitecture #EngineeringLessons #FullStackDeveloper #WebDevelopment #DeveloperHumor #TechLife #Programming #CodingLife #SoftwareEngineering #DevCommunity #WorkLife #TechCommunity #ProgrammerLife #FullStackLife #ITLife #SoftwareDeveloper #EngineerLife #DeveloperLife #TechWorld #CodingHumor

#FrontendDeveloper #BackendDeveloper #JavaScriptCommunity #WebDevCommunity #C2C #C2H #java #python #Angular #React #Aws #Azure #GCP

#c2c #c2h #jobs

#305 Arun Goel System Design

Staff Software Engineer @ Okta | Ex - Uber | Ex - Atlassian | Former Samsung R&D Bangalore| 57K+ | DTU

2mo · **View on LinkedIn**

I've prepared for LLD interviews across Uber, Facebook, Google, Amazon, Microsoft, Rippling, Apollo.io, and Atlassian.

One thing I realised: you don't need to solve 100 LLD problems.

Master these instead

Core

- Parking Lot
- Splitwise
- Elevator
- ATM
- Library
- Movie Ticket Booking
- Hotel Management
- Airline Reservation
- Cab Booking
- Food Delivery

Games

- Snake & Ladder
- Tic-Tac-Toe
- Chess
- Ludo
- Minesweeper

Real-world

- Vending Machine
- Coffee Machine
- Digital Wallet
- Inventory Management
- Shopping Cart

Advanced

- Logger
- Notification System
- LRU/LFU Cache
- Rate Limiter
- File System
- Linux Find
- Text Editor
- Task Scheduler
- Message Queue
- URL Shortener

And here is what interviewers actually evaluate:

→ Requirement gathering
→ OOP fundamentals
→ SOLID principles
→ Design Patterns
→ Extensible design
→ Abstraction & low coupling
→ Edge cases & error handling
→ Scalability & clean code
→ Communication & trade-offs
Please bookmark it.
If you're preparing for SDE interviews, they are helping multiple engineers land offers in these companies
Join here → https://lnkd.in/d2aj6p2H
Please add other interview questions that you have also seen ↓
#connections #interviews #hiring #faang #lld

LINKS
- https://lnkd.in/d2aj6p2H

#306 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter
2mo · **View on LinkedIn**

If you want to become god-level with system design,
learn these 19 case studies (save now):
1 How WhatsApp Works
↳ https://lnkd.in/eU2fswMi
2 How YouTube Works
↳ https://lnkd.in/e7q9F4Sg
3 How Instagram Works
↳ https://lnkd.in/ejBKTZPD
4 How Stripe Payment Integration Works
↳ https://lnkd.in/ehpbWAdy
5 How Google Docs Works
↳ https://lnkd.in/ehPNA7Az
6 How Cloudflare Scaled Postgres
↳ https://lnkd.in/eEQP6Apw
7 How Meta Serverless Works
↳ https://lnkd.in/eNyYKCQD
8 How Airbnb Works
↳ https://lnkd.in/dGVfstQM
9 How Spotify Works
↳ https://lnkd.in/eGbWVeNW
10 How Slack Works
↳ https://lnkd.in/eATMDjrK
11 How Reddit Works
↳ https://lnkd.in/egmm_P7a
12 How Bluesky Works

↳ https://lnkd.in/eEhB8V_k
13 How Tinder Works
↳ https://lnkd.in/en65fv-W
14 How Twitter Timeline Works
↳ https://lnkd.in/eniXMPfU
15 How Uber Finds Nearby Drivers
↳ https://lnkd.in/eeqH9Hjh
16 How Amazon Lambda Works
↳ https://lnkd.in/eNd3Z5Yn
17 How Zoom Works
↳ https://lnkd.in/edidhxZw
18 How Apple AirTags Work
↳ https://lnkd.in/ec75eiCH
19 How ChatGPT Works
↳ https://lnkd.in/eSd6fS7n
What else should make this list?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 245,001+ software engineers right now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others master system design.
Follow Neo Kim + turn on notifications.

MEDIA 19 system design case studies for software engineers

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/eU2fswMi
- https://lnkd.in/e7q9F4Sg
- https://lnkd.in/ejBKTZPD
- https://lnkd.in/ehpbWAdy
- https://lnkd.in/ehPNA7Az
- https://lnkd.in/eEQP6Apw
- https://lnkd.in/eNyYKCQD
- https://lnkd.in/dGVfstQM
- https://lnkd.in/eGbWVeNW
- https://lnkd.in/eATMDjrK
- https://lnkd.in/egmm_P7a
- https://lnkd.in/eEhB8V_k
- https://lnkd.in/en65fv-W
- https://lnkd.in/eniXMPfU
- https://lnkd.in/eeqH9Hjh
- https://lnkd.in/eNd3Z5Yn
- https://lnkd.in/edidhxZw
- https://lnkd.in/ec75eiCH
- https://lnkd.in/eSd6fS7n
- https://lnkd.in/ehTrcyak

#307 Java Developers Community (moderated)

System Design

Tauseef Fayyaz • 2nd | Book an appointment
2mo · **View on LinkedIn**

If you want to become top-tier at API design,

learn these 25 concepts:
1 Endpoint
→ A unique URL that names one resource.
2 HTTP Methods
→ GET, POST, PUT, PATCH, DELETE, what to do with it.
3 Request-Response
→ Client asks, server answers: status, headers, body.
4 Status Codes
→ 2xx worked, 4xx you broke it, 5xx we broke it.
5 Authentication
→ Proves who is calling, before anything else runs.
6 Authorization
→ Decides what that caller is allowed to touch.
7 Access Tokens
→ Short-lived credentials, so passwords never travel.
8 OAuth 2.0
→ An app acts for a user without seeing the password.
9 Rate Limiting
→ Caps calls per window. Over the line gets a 429.
10 Throttling
→ Slows callers down instead of rejecting them.
11 Pagination
→ Returns big result sets one page at a time.
12 Caching
→ Serves a stored copy. Less latency, less DB load.
13 Idempotency
→ Same request twice, same result. No double charges.
14 Webhooks
→ The API calls you the moment an event happens.
15 API Versioning
→ Ship breaking changes without breaking old clients.
16 OpenAPI
→ One spec that generates docs, SDKs and mocks.
17 REST vs GraphQL
→ Many fixed endpoints vs one client-shaped query.
18 API Gateway
→ One front door: routing, auth, limits, observability.
19 Microservices
→ Independent services talking only over APIs.
20 Error Handling
→ One predictable error shape clients can act on.
21 gRPC
→ Binary, contract-first RPC for service-to-service.
22 WebSockets & SSE
→ Keep the connection open and push updates live.
23 CORS

→ Which origins the browser lets call your API.
24 Retries & Backoff
→ Retry with growing, jittered delays, no stampede.
25 Circuit Breaker
→ Stop calling a sick dependency. Fail fast instead.
An API is just a contract between two systems.
The 25 above are what makes that contract hold up
in production.
Which one took you the longest to really get?
Repost to help someone else level up their API design.
+ Follow me (Tauseef Fayyaz) for more system design breakdowns.

LINKS

- https://www.sefism.com/mentorship/

#308 Arvind Kumar

System Design

Staff Engineer @Chegg Inc | Building codefarm.in
2mo · **View on LinkedIn**

If you're preparing for System Design interviews, focus less on memorizing architectures and more on understanding the engineering trade-offs behind real production problems.

I've been publishing a series of real-world System Design scenarios that are commonly discussed in interviews at product companies. Each article goes beyond the high-level design and explains the concepts interviewers actually expect you to understand.

1. Kafka Consumer Processes Same Event Twice
Offset Management • Idempotent Consumers • Exactly-Once Semantics • Dead Letter Queues
https://lnkd.in/g3hf6ZSZ
2. YouTube Video Processing Pipeline
Distributed Encoding • Async Workflows • Parallel Processing • Storage Optimization
https://lnkd.in/gQVyXndn
3. Payment Deducted But Order Failed
Saga Pattern • Distributed Transactions • Compensation • Idempotency Keys • Retry Handling
https://lnkd.in/ga2WZgSf
4. Instagram Notification Storm
Fanout Architecture • Push Notification Scalability • Backpressure • Batch Processing
https://lnkd.in/gnWdStMh
5. Amazon Cart Shows Old Data
Cache Invalidation • Distributed Cache • Session Consistency • Event-Driven Updates
https://lnkd.in/gYpE_qZw
6. Uber Surge Pricing During Rain
Real-Time Analytics • Stream Processing • Dynamic Pricing • Kafka Pipelines
https://lnkd.in/g6mrQaUp
7. Netflix Buffers After a New Season Release
CDN Architecture • Cache Prewarming • Auto Scaling • Traffic Spike Handling
https://lnkd.in/gsCQAsaJ
8. BookMyShow Sells the Same Seat Twice
Race Conditions • Distributed Locking • Optimistic vs Pessimistic Locking • Transactions
https://lnkd.in/g-unjnr8

9. Swiggy Shows Wrong Rider Location

Real-Time GPS Tracking • WebSockets • Eventual Consistency • Geo-Distributed Systems

https://lnkd.in/grqFhArx

I'm continuously adding new scenarios covering distributed systems, scalability, consistency, messaging, caching, and architecture patterns that frequently appear in backend engineering interviews.

Follow for more Java, Spring Boot, Microservices, Kafka, and System Design content.

#SystemDesign #SoftwareArchitecture #Java #SpringBoot #Microservices #Kafka #DistributedSystems #BackendDevelopment #Scalability #TechInterviews #Engineering

LINKS

- https://lnkd.in/g3hf6ZSZ
- https://lnkd.in/gQVyXndn
- https://lnkd.in/ga2WZgSf
- https://lnkd.in/gnWdStMh
- https://lnkd.in/gYpE_qZw
- https://lnkd.in/g6mrQaUp
- https://lnkd.in/gsCQAsaJ
- https://lnkd.in/g-unjnr8
- https://lnkd.in/grqFhArx

#309 Arslan Ahmad System Design

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

2mo · **View on LinkedIn**

Most system design interviews reward you for explaining how a system works when everything goes right.

The strongest answers explain what happens when it does not.

Distributed systems do not fail in infinite ways. They fail in a small number of recognizable patterns, and the same ones keep showing up in postmortems at every company:

1. Single Point of Failure
2. Cascading Failure
3. Retry Storm
4. Cache Stampede
5. Hot Partition
6. Replication Lag
7. Duplicate Processing
8. Queue Backlog
9. Poison Message
10. Split Brain

Once you can name a pattern, you can design against it before the interviewer has to ask. That is most of the distance between a mid-level answer and a senior one.

So when you propose a cache, mention stampede protection. When you shard, name your key and say why it will not go hot. When you add retries, say backoff and jitter in the same breath. When you add replicas, say which reads can tolerate lag.

Anyone can draw boxes. Senior engineers explain how the boxes behave on the worst day of the quarter.

I wrote up all ten, with a diagram and the fix for each one:

https://lnkd.in/gEGzJtMC

Help your friends by reposting this.

LINKS
- https://www.designgurus.io/course/grokking-the-system-design-interview
- https://lnkd.in/gEGzJtMC

#310 Chandra Sekhar

AI / LLM

I simplify AI for everyone | 56K+ Followers | Top 1% Linkedin India | Senior AI Engineer | Agentic AI Trainer | Claude Certified Architect | Corporate Trainer

2mo · **View on LinkedIn**

Accenture asked this in L3 round of Senior AI Engineer:

Your client has a project related to bank services. They want to run this AI on their private servers with ZERO internet access.

How do you build an AI that doesn't need the Cloud?

To solve this, you need to shift from Cloud LLMs to Local LLMs.

Here is how you Decode the “No Internet” problem:

1. Pick an Open-Source Model: Instead of OpenAI, use models like Llama 3 (Meta), Mistral, or Gemma (Google). These are weights you can actually download and own.

2. Quantization is Key: A massive model won’t fit on standard office servers. Use Quantization (4-bit or 8-bit) to compress the model so it runs fast on local hardware without losing much “intelligence.”

3. Local Serving Tools: Use frameworks like Ollama, vLLM, or LocalAI. These tools create a local API that works exactly like ChatGPT but stays 100% inside the company’s firewall.

4. Offline Vector Database: For the RAG pipeline, use local databases like ChromaDB or FAISS. This ensures that even the “search” part of your AI never hits the public web.

Privacy isn’t a feature; it’s a requirement.

==========

I’ve covered questions like these in my AI Engineering Interview Master Bundle, a comprehensive set of 22 courses designed for real interview prep.

Explore the full guide here → https://lnkd.in/gqFkWZd4

LINKS
- https://lnkd.in/gqFkWZd4

#311 Parikh Jain

Other

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

2mo · **View on LinkedIn**

Your first CTC is not a paycheck. It is the number every future recruiter starts from.

6 LPA does not stay 6. It becomes the 8, then the 11, then the 14 you keep getting offered, because every hike is just a percentage of your last number.

Raise a small number by 30% and it is still a small number.

So you are not underpaid for one year. You are underpaid off the same anchor for the next ten.

Same work, same late nights, just a lower base that follows you from job to job.

And here is the part that stings. That anchor got set in about 45 minutes in a DSA round.

One coding round you walked into underprepared, on a problem you could have recognized if you had drilled the right patterns first.

The good news is the same math works in reverse. Set a higher first number and every future hike compounds on top of it.

The 45 minutes that felt small end up being the most valuable 45 minutes of your career.

That first number is exactly what 1500+ students are protecting inside the SDE Career Tracker (https://lnkd.in/gtzyBHkw).

DSA mapped to 50 patterns, LLD, HLD, GenAI basics, a Resume Analyzer that rewrites your resume, scored mock interviews, a live Job-Ready Score, and job assistance with 200+ partner companies for referrals.

You will negotiate from this one number for ten years. Spend 6-8 months preparing to make it a big one.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gtzyBHkw

#312 Gaurav Nikumbh

System Design

Senior Product Manager - Payments & AI | IIM Calcutta | Ex - Founder | Payments Platform & Money Movement | Payment Processing, ACH, UPI, Cards, Fraud & Risk | Payment Orchestration | $2B+ TPV Across 50+ FI's

2mo · **View on LinkedIn**

Tokenization Explained - why replacing a 16-digit number changes everything about card payment security

Every time you tap your phone or click "pay with saved card," the merchant never sees your real card number.

They see a token. That distinction is what makes modern card payments structurally safer.

What tokenization actually does

Tokenization replaces the real card number (PAN) with a surrogate value called a token.

Instead of storing:

➥ 4111 1111 1111 1111 (real PAN)

The merchant stores:

➥ 5789 2345 9876 1234 (token)

The token has no value outside its intended environment. Breach the merchant database - all you get are tokens that are useless everywhere else.

How it works in a transaction

➥ Customer adds card to wallet or merchant checkout

➥ Card network generates a unique token

➥ Token stored by merchant, wallet, or device - never the real PAN

➥ At payment time, token is sent instead of card number

➥ Network maps token back to real card and routes authorization to issuer

The customer feels nothing different. The architecture underneath is transformed.

Why it matters beyond just security

➥ Fraud reduction - breached databases yield tokens unusable anywhere else

➥ Better UX - saved cards, one-click payments, mobile wallets without exposing card details

➥ Higher auth rates - networks auto-update tokens when cards are reissued

➥ PCI-DSS scope reduction - merchants handling tokens carry far less compliance burden

➥ Foundation for digital wallets - Apple Pay, Google Wallet, Samsung Pay are built on this model

Tokenization vs 3DS - two layers, two different jobs

Often confused. They solve different problems:

➥ 3DS answers: "Is this the genuine cardholder?"

➥ Tokenization answers: "How do we protect the card number from exposure?"

3DS verifies the person. Tokenization protects the credential.

Together they form the two foundational security layers of modern card payments - and neither replaces the other.

A decade ago, merchants stored real card numbers in databases that attackers knew exactly how to target.

Today the real number rarely touches the merchant at all.

From storing credentials to storing references to credentials - that architectural shift is what tokenization made possible.

Any conversation about card payment security that doesn't start here is starting in the wrong place.

#Payments #Tokenization #CardPayments #PaymentsEngineering #Fintech #PCIDSS #ApplePay #3DS #ProductManagement #PaymentSecurity

LINKS

- https://topmate.io/gaurav_nikumbh

#313 Neo Kim

AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

2mo · **View on LinkedIn**

If you want to become world-class with AI agents,

then watch these 10 videos (save now):

1 Deep Dive into LLMs like ChatGPT (by Andrej Karpathy)

↳ Explains full LLM training stack, the foundation agent builders need

https://lnkd.in/eiJeGN3n

2 AI Agents for Beginners (by Microsoft)

↳ Microsoft's agent curriculum in one video

https://lnkd.in/ep4Rgxjp

3 LangGraph Complete Course for Beginners (by freeCodeCamp)

↳ Builds agents from scratch

https://lnkd.in/emzaYing

4 GenAI Essentials Full Course (by freeCodeCamp)

↳ Covers the entire GenAI lifecycle

https://lnkd.in/eMqV9XRx

5 Build n8n AI Agents (by Nate Herk)

↳ Teaches you no-code agent builds & multi-agent architectures

https://lnkd.in/etGUZcwn

6 Complete Agentic AI Course (by Krish Naik)

↳ Teaches LangChain, LangGraph, RAG, guardrails & evals

https://lnkd.in/eRx6Kjkk

7 Agentic AI Crash Course using LangChain (by CampusX)

↳ Theory of generative & agentic AI

https://lnkd.in/e9FptHJY

8 AI Agents for Beginners (by KodeKloud)

↳ Builds agent personalities from scratch

https://lnkd.in/eb5Rvg-y

9 Generative AI Full Course (by freeCodeCamp)

↳ GenAI & agent courses in one playlist

https://lnkd.in/eJuJky4h

10 Master Agentic AI

↳ Condenses current agent concepts & tooling into 2 hours

https://lnkd.in/eq2KQ8fw

What else should make this list?

===

PS - Want my System Design Playbook for FREE?

Join my newsletter with 245K+ software engineers now:

→ https://lnkd.in/ehTrcyak

===

Repost to help others ace AI agent engineering.

Follow Neo Kim + turn on notifications.

MEDIA 10 videos to learn AI agent engineering

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eiJeGN3n
- https://lnkd.in/ep4Rgxjp
- https://lnkd.in/emzaYing
- https://lnkd.in/eMqV9XRx
- https://lnkd.in/etGUZcwn
- https://lnkd.in/eRx6Kjkk
- https://lnkd.in/e9FptHJY
- https://lnkd.in/eb5Rvg-y
- https://lnkd.in/eJuJky4h
- https://lnkd.in/eq2KQ8fw
- https://lnkd.in/ehTrcyak

#314 Ashish Pratap Singh

Kafka / Messaging

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

2mo · **View on LinkedIn**

Kafka is NOT just a messaging system. Here are the top 10 use cases you should know:

1. Asynchronous Messaging: Buffer traffic spikes and decouple producers from slower consumers.

2. Pub-Sub Fan-Out: Publish one event and let multiple independent systems consume it, such as billing, analytics, and notifications.

3. Activity Tracking: Capture clicks, searches, page views, and other user activity in real time.

4. Log Aggregation: Collect logs from multiple services into a centralized stream for storage, analysis, and monitoring.

5. Stream Processing: Filter, aggregate, enrich, and transform events while they are moving through the system.

6. Metrics and Alerting: Stream operational telemetry into monitoring systems for real-time dashboards and alerts.

7. Event Sourcing: Store state changes as an append-only sequence of events and rebuild the current state by replaying them.

8. Change Data Capture: Capture database changes and propagate them to caches, search indexes, analytics systems, and other services.

9. Data Pipelines: Continuously move data from applications and databases into warehouses, data lakes, and other storage systems.

10. Replay and Recovery: Reprocess historical events from a previous offset when consumers fail, logic changes, or data needs to be rebuilt.

Which other Kafka use case would you add to this list? Let me know in the comments.

Repost to help others learn these Kafka use cases

→ If you want to learn system design in-depth with visuals and real-world examples, checkout my comprehensive resource here: https://lnkd.in/gpWPT_DG

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/gpWPT_DG

#315 Manan Solanki

System Design

Software Engineer 2 @ Uber | Ex: Flipkart, Tekion, HashedIn | DAIICT'21 | Backend | Java | Go | Distributed Systems | AI Enthusiast | Top 1% @ LeetCode

3mo · **View on LinkedIn**

While interviewing across top PBC like Uber , Flipkart , Rippling ...

One backend interview question completely changed how I prepare!!

It wasn't about DSA or System Design.

The interviewer asked:

"How would you debug an intermittent production issue that's not reproducible?"

I paused.

Because there wasn't a "correct" answer.

What they wanted to understand was how I approached uncertainty.

→ Do I start with logs or metrics?

→ Would I look at distributed traces to identify where latency spikes?

→ How do I narrow the blast radius before jumping to conclusions?

That question taught me something I still apply in production today:

You can't debug what you can't observe.

Since then, observability has become part of how I think about backend systems—not just after deployment, but while designing them.

If a service doesn't expose the right:

→ Logs. Metrics. Traces

...you're making future incidents much harder to diagnose.

That's also why I've noticed senior backend interviews becoming far more production-oriented.

They're evaluating questions like:

→ Can you investigate an issue with incomplete information?

→ Can you reason through trade-offs under pressure?

→ Can you restore a system before optimizing it?

That's difficult to practice in day-to-day work because production incidents are unpredictable, while interviews compress those situations into a 45-minute discussion.

That's exactly where Bosscoder Academy's EVOLVE helps. It goes beyond DSA and System Design by helping experienced engineers build the engineering judgment, production-first thinking, and interview approach expected from Senior SDEs.

And if you're preparing for a serious switch to Senior roles, learning from industry experts at Bosscoder Academy can be highly profitable.

Explore EVOLVE for SDEs with 6+ YOE here → bcalinks.com/mqwIh6G

They've helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

LINKS
- http://bcalinks.com/mqwlh6G

#316 Kriti Rohilla

AI / LLM

Software Engineer @Microsoft | Ex-Coinbase, Goldman Sachs | Helping 70K+ professionals optimize career, health & performance | Brand partnerships

3mo · **View on LinkedIn**

I interviewed at Goldman Sachs, Coinbase, and now Microsoft.

One thing never changed↓

The DSA round decides whether you get the opportunity to prove everything else.

That's something I underestimated early in my career.

→ I used to think getting the correct solution was enough.

For senior roles, it isn't✗

Interviewers are paying attention to how you arrive at the solution just as much as the solution itself.

Over time, I noticed a few patterns that consistently showed up across interviews:

Trees & Graphs – Traversals are expected. What stands out is how you reason about edge cases and complexity.

Sliding Window & Two Pointers – Less about memorizing templates, more about recognizing when a brute-force solution can be optimized.

Dynamic Programming – Not every DP problem is difficult. The challenge is explaining the state transition clearly.

Intervals & Binary Search – These often look straightforward until constraints force you to rethink your approach.

The biggest shift in my preparation wasn't solving more questions.

It was reviewing every solution afterward.

→ Could I explain why I chose this approach?

→ Could I defend the time-space trade-offs?

→ Would another engineer understand my reasoning without looking at the code?

That's what interviewers are evaluating.

Your experience gets your resume shortlisted.

Your ability to communicate structured thinking is what gets you through the interview loop.

If you're preparing for senior SDE roles, don't do it alone.

Bosscoder Academy's EVOLVE Program is designed for working engineers like you who've built the experience but want structured guidance to crack top product companies.

Explore EVOLVE for SDEs with 6+ YOE here → bcalinks.com/mOd8xCy

They've helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#dsa #interview #microsoft #career #jobswitch #collab

LINKS
- http://bcalinks.com/mOd8xCy

#317 Ashish Pratap Singh

System Design

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

3mo · View on LinkedIn

If you want to get better at designing large scale systems, understand these 15 System Design Trade-offs:

1. Consistency vs Availability

During a network partition, do you return the latest data or continue serving requests?

2. Latency vs Throughput

Do you optimize for faster individual responses or more total requests per second?

3. SQL vs NoSQL

Do you need relational integrity and complex queries, or flexible schemas and easier horizontal scaling?

4. Vertical vs Horizontal Scaling

Do you upgrade to a larger machine or distribute the workload across more machines?

5. Monolith vs Microservices

Do you prefer simpler development and deployment, or independent scaling and team ownership?

6. Strong vs Eventual Consistency

Must every read return the latest write, or can replicas temporarily return stale data?

7. Normalization vs Denormalization

Do you reduce data duplication or duplicate data to simplify and accelerate reads?

8. Read vs Write Optimization

Indexes, caching, and precomputation improve reads but often increase write cost and complexity.

9. Caching vs Freshness

Do you serve data quickly from a cache or fetch the latest version from the source?

10. Push vs Pull Architecture

Should the server send updates automatically, or should clients request them when needed?

11. Synchronous vs Asynchronous Communication

Should the caller wait for a response, or should the work happen independently?

12. Replication vs Partitioning

Do you copy data for availability and read scalability, or split it for storage and write scalability?

13. Batch vs Stream Processing

Do you process data efficiently in large chunks or continuously as events arrive?

14. Stateful vs Stateless Services

Should a service maintain session state locally, or remain stateless for easier scaling and failover?

15. Redundancy vs Cost

More replicas, backups, and regions improve reliability, but they also increase infrastructure costs.

A strong system design answer explains: What the system needs, which trade-off was chosen, why it was chosen, and what disadvantages were accepted.

Which other system design trade-off would you like to add to this list?

Repost to help others learn these system design trade-offs

→ If you want to learn system design from scratch with visuals and real-world examples, checkout my comprehensive resource here: https://lnkd.in/gpWPT_DG

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/gpWPT_DG

#318 Krutam Hathi

System Design

Senior Software Engineer@Weave | Ex-Microsoft, Cohesity | IIIT-Hyderabad Alumini | Talks about Tech, AI, sports and keyboards.

3mo · **View on LinkedIn**

After switching from Microsoft → Cohesity and now Weave, one interview trend became impossible to ignore

The last two companies I interviewed with asked me the same question.

It wasn't about System Design!!

The discussion rarely stayed on architecture for long.

Instead, interviewers kept trying to break it.

Not with trick questions. With production scenarios.

"Your database starts throttling writes. What's your first move?"

"One downstream service is adding 700ms latency. Do you retry, fail fast, or queue the requests?"

"Your cache is serving stale data during a traffic spike. How do you investigate before fixing it?"

They wanted to see how I'd debug the system without access to logs.

That changed how I approached System Design.

I stopped treating it as an architecture exercise and started treating it like a production incident.

→ Where's the bottleneck?

→ What's the blast radius?

→ Which metrics would validate my hypothesis?

That's when my interview discussions started feeling much closer to real backend engineering.

If you're preparing for senior backend interviews, here's one thing I'd recommend:

Don't stop after drawing the architecture.

Spend the next 10 minutes trying to break it.

That's where the most valuable interview discussions usually begin.

And that's exactly the kind of production-first thinking Bosscoder Academy's EVOLVE helps experienced engineers build through real-world System Design, mentorship, and interview preparation.

So if you're preparing for a serious switch to Senior roles, learning from industry experts at Bosscoder Academy can be highly profitable.

Explore EVOLVE for SDEs with 6+ YOE here → https://lnkd.in/gP-C_Zfd

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#dsa #interview #microsoft #career #jobswitch #collab

LINKS

- https://lnkd.in/gP-C_Zfd

#319 Aashish Gupta

AI / LLM

Sr Software Engineer@Uber | Mentoring Engineers in the AI Era | Backend & System Design | Building Scalable Systems | Brand Collabs

3mo · **View on LinkedIn**

One thing that significantly improves your chances of getting selected for Senior SDE roles?

→ Stop preparing only for the interview✗

→ Start preparing for the industry✓

Over the years, from Adobe and Dell EMC to Walmart and now Uber, I've noticed that the engineers who consistently stand out aren't always the ones who solve problems the fastest.

They're the ones who think like owners.

In senior interviews, interviewers are constantly looking for signals beyond coding ability.

→ Can you break down an ambiguous problem?

→ Can you explain why you chose one design over another?

→ Can you identify trade-offs before someone points them out?

→ Can you influence decisions without being the loudest person in the room?

These are the skills that become increasingly important as you grow.

One mistake I see many experienced engineers make is spending months grinding DSA while ignoring everything else.

Yes, coding rounds matter.

But so do System Design, communication, stakeholder management, and your ability to make sound engineering decisions under constraints.

A simple habit that helps: whenever you work on a project, don't just think about what was built.

→ Think about why certain decisions were made, what alternatives were considered, and what would happen if the system had to scale 10x.

That's the kind of thinking senior interviewers look for✓

Because at senior levels, companies aren't just hiring engineers.

They're hiring people they can trust with bigger problems.

And if you're preparing for a serious switch to Senior roles, learning from industry experts at Bosscoder Academy can be highly profitable.

Explore EVOLVE for SDEs with 6+ YOE here → bcalinks.com/bnPN6dO

They've helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#collab #pbc #tech #career #jobswitch #seniorsde

LINKS

- http://bcalinks.com/bnPN6dO

#320 Neo Kim System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

3mo · **View on LinkedIn**

If you want to become S-tier with API design,

learn these 20 concepts:

1 Endpoint

↳ A unique URL that identifies & gives access to a specific API resource.

2 HTTP Methods

↳ Commands like GET, POST, PUT, PATCH & DELETE that define what action to perform.

3 Request-Response

↳ How clients send requests & servers return data, status codes, and headers.

4 Status Codes

↳ Three-digit codes that tell you whether an API request succeeded/failed.

5 Authentication

↳ Verify who the user or application is before allowing access.
6 Authorization
↳ Decide what an authenticated user or application is allowed to do.
7 Access Tokens
↳ Temporary credentials that prove permission to access an API.
8 OAuth 2.0
↳ A standard for delegated access without sharing user passwords.
9 Rate Limiting
↳ Restrict how many API requests a client can make in a given time.
10 Throttling
↳ Slow down requests instead of rejecting them when usage gets too high.
11 Pagination
↳ Return large datasets in smaller pages instead of one huge response.
12 Caching
↳ Store API responses temporarily to reduce latency and server load.
13 Idempotency
↳ Repeating same request produces the same result without unwanted "side effects".
14 Webhooks
↳ Let one system automatically notify another when an event happens.
15 API Versioning
↳ Release API changes without breaking existing applications.
16 OpenAPI
↳ A standard format for documenting and describing REST APIs.
17 REST vs GraphQL
↳ Two API styles with different approaches to requesting and returning data.
18 API Gateway
↳ A single entry point that routes, secures, and manages API traffic.
19 Microservices
↳ Small, independent services that communicate through APIs.
20 Error Handling
↳ Return clear, consistent error messages so clients know what went wrong.
API is a set of rules that allows different services to talk to each other.
What else should make this list?
===
PS - Want my System Design Playbook for FREE?
Join my newsletter with 245K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Repost to help others ace API design.
Follow Neo Kim + turn on notifications.

MEDIA 20 API concepts for senior engineers; document/carousel (possible)

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#321 Sivasankar Natarajan

AI / LLM

Technical Director | GenAI Practitioner | Azure Cloud Architect | Data & Analytics | Solutioning What's Next

3mo · View on LinkedIn

When Did "AI Engineer" Go From Calling One API to Orchestrating 11 Layers?

Two years ago the job was: call an API, parse the response, ship.

Today it's: models, agents, memory, RAG, tooling, guardrails, observability, and the infrastructure underneath. The role exploded. Here's the map.

The intelligence layers:

1. Models: OpenAI, Anthropic, Google, Meta, Mistral, Cohere. Increasingly interchangeable commodities. The smart move isn't betting on one, it's building so you can swap as capabilities and prices shift.

2. Agent Frameworks: LangGraph, CrewAI, AutoGen, LlamaIndex, AWS Strands. Competing opinions about the same problem. Learn one deeply. Mastery transfers most of the way to the rest.

3. Knowledge and RAG: LangChain, LlamaIndex, Haystack, DSPy, GraphRAG. How you ground models in your data. The field is moving beyond flat vector search toward relationship-aware retrieval.

The persistence layers:

4. Memory and Context: Mem0, Zep, Letta, Redis, PostgreSQL. What makes agents continuous instead of amnesiac. Redis and Postgres showing up here is telling sometimes the "AI memory layer" is just a well-used traditional database.

5. Vector DB and Storage: Pinecone, Weaviate, Qdrant, Milvus, pgvector, Elasticsearch. pgvector's presence says it all for many teams, adding vectors to Postgres beats adopting a new database entirely.

The connectivity layers:

6. Tooling and MCP: MCP SDK, FastMCP, servers for GitHub, Slack, Postgres, filesystem. Quietly becoming the connective tissue of the whole stack. The standard way agents reach tools without bespoke integration.

7. Automation and Workflows: n8n, Zapier, Make, Temporal, Apache Airflow. Where AI meets the rest of your systems. Durable, battle-tested orchestration matters as much as the flashy AI parts.

The trust layers:

8. Security and Guardrails: NeMo Guardrails, Presidio, Lakera Guard, Prompt Security, Bedrock Guardrails. An entire layer just for safety tells you how seriously production teams take it. What makes AI shippable in real organizations.

9. Observability and Evals: LangSmith, Langfuse, Arize Phoenix, W&B, TruLens. Separates "seems to work" from "provably works." The discipline serious teams live by.

The foundation:

10. Infrastructure: Docker, Kubernetes, AWS, GCP, Azure, Terraform. The unglamorous bedrock. AI-native or not, it still runs on containers, clouds, and infrastructure-as-code. Fundamentals never went away.

What's the real takeaway?

You don't need every tool.

You need to know what each layer does and how to pick one per layer.

Models on top.

Infrastructure at the bottom.

The operational layers: memory, tooling, guardrails, observability: in between.

Those middle layers are where most teams under-invest and where most production failures originate.

Which layer is your biggest gap right now?

Repost this to help your network get started

+ Follow Sivasankar for more

#AIEngineering #AgenticAI #MLOps

#322 Spring Users

Other

Imdad Areeph • 3rd+ | #Spring AI 2.0 is a major step forward for building production-ready AI applications with Java. | The release introduces a unified tool-calling architecture, progressive tool discovery to reduce token usage, structured output validation, improved #MCP support, and a more composable advisor-based runtime for agentic systems. | One of my favorite additions is #ToolSearchToolCallingAdvisor, which enables AI models to discover only the tools they need instead of loading entire tool catalogs—resulting in significantly lower token usage and better scalability. | I put together this infographic to summarize the key architectural changes, new features, and best practices in Spring AI 2.0 GA. | #SpringAI #SpringBoot #Java #AI #GenerativeAI #AgenticAI #AIEngineering #EnterpriseAI #LLM #MCP #ModelContextProtocol #SoftwareArchitecture #BackendDevelopment #JavaDeveloper #CloudNative #Microservices #OpenSource #SoftwareEngineering #TechLeadership #DeveloperTools #VectorSearch #AIAgents #SpringFramework #agenticcodingnewsletters | 115 | 1 | Nileshkumar Jani 3rd+ | Nileshkumar Jani • 3rd+ | AI Architect | Sr. Staff Software Engineer (100% Hands-On) | Distributed Systems • Enterprise AI & RAG • Agentic AI | Python, Java, Go | AWS, Azure,GCP | LLMOps & Observability,OTel | US Citizen (15+ Yrs Exp)

3mo · **View on LinkedIn**

thank you for sharing!

MEDIA advertisement

#323 Parikh Jain

Other

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

3mo · **View on LinkedIn**

Amazon loves: Heaps, Graph BFS, Intervals.

Google loves: DP, Backtracking, Trees.

Microsoft loves: Strings, Hashing, DP.

You don't have to be ready for all 50 patterns. You have to be ready for the 10-15 your target company keeps asking.

The DSA round is a pattern-targeting test, and the targets are fully public.

Three moves separate the candidates who clear from the ones still applying.

1. Pick the shortlist first. Three companies, not thirty. Until you name the three, every problem you solve is a guess and every sheet you open is the wrong one.

2. Read 30 recent interview reports per company on LeetCode Discuss, GFG, and Glassdoor. Tag the pattern in each. By the 90th report, the top 5-8 patterns repeat so loudly you cannot miss them.

3. Drill those 5-8 until you can write the template blind, spot them in fresh problems under 90 seconds, and code the optimal under 25 minutes. Six weeks of focused work, not six months of random sheets.

This is the loop 1500+ students are running inside the SDE Career Tracker (https://lnkd.in/gtzyBHkw).

Pattern-targeted DSA, LLD, HLD, GenAI basics, a Resume Analyzer that rewrites your resume, scored mock interviews,

a live Job-Ready Score, and job assistance with 200+ partner companies plugged in for referrals.

The Amazon round is not unpredictable.

The Google round is not unpredictable. Your prep is unfocused.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gtzyBHkw

#324 Ankit Pangasa

System Design

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

3mo · View on LinkedIn

"The customer clicked 'Pay' three times. How do you ensure they're charged only once?"

This is one of the most common backend and system design interview questions.

Many candidates know what idempotency is. Very few can explain why it's needed and how it's implemented.

Spend 5 minutes with this diagram and you'll never forget this concept.

Save this before your next interview.

The Problem

Imagine you're building a payment system.

The user clicks "Pay." The request reaches the server, but the response never reaches the user because of a network timeout.

What does the user do?

→ Clicks "Pay" again.

Without idempotency, your API may create:

Multiple orders

Duplicate payments

Incorrect inventory

Angry customers

The Solution

Generate a unique Idempotency Key for each logical request.

First request → Process it and store the response with the key.

Retry with the same key → Return the stored response.

No duplicate processing.

✓ Same request + Same key = Same result.

Where is it used?

Payment APIs

Order creation

Money transfers

Ticket booking

Inventory updates

Any operation where duplicates are unacceptable

Common Interview Questions

What is idempotency?

Why is POST not idempotent by default?

How do idempotency keys work?

Where should idempotency keys be stored?

How long should they be retained?

Which HTTP methods are idempotent?

Interview Tip

Don't say:

✗ "Idempotency prevents duplicate requests."

Say:

✓ "Idempotency ensures the same request produces the same outcome. A common implementation uses an Idempotency Key to identify retries and return the original response instead of processing the request again."

That shows you understand both the problem and the solution.

Interview Question: If your payment API stores idempotency keys in Redis and Redis crashes, how would you prevent duplicate payments?

#SystemDesign #BackendEngineering #API #Idempotency #DistributedSystems #Microservices #SoftwareEngineering #TechInterview #Payments #RESTAPI

#325 Venkatesh Patnala

System Design

Software Engineer @ Cars24 | Debugging and Fixing code written by AI | Java · Spring Boot · Distributed Systems (RabbitMq, Redis, MongoDB, SQL) | Ex-Sprinklr | IIT Jodhpur '23

3mo · **View on LinkedIn**

We added one annotation.

Our API latency doubled.

No slow SQL. No missing indexes. No traffic spike.

Just one innocent-looking @Transactional.

"@Transactional

public void updateOrder() {

Order order = repository.findById(id);

paymentService.verifyPayment(); // ~800ms

repository.save(order);

} "

At first glance, nothing looked wrong.

But here's what we missed.

The transaction starts before the external API call.

Which means:

✓ A database connection is acquired.

The transaction stays open while waiting ~800ms for another service.

That DB connection can't return to the connection pool.

At low traffic? Nobody notices.

At production scale?

Hundreds of requests were holding database connections far longer than necessary.

The result?

✗ Connection pool exhaustion

✗ Requests waiting for available connections

✗ API latency spiked across the system

The database wasn't slow.

The queries weren't slow.

We were just holding an expensive resource for too long.

The fix was surprisingly simple:

External API call
↓
@Transactional
fetch → update → save
Keep transactions for database work only.
The lesson?
@Transactional isn’t just about whether you use it.
It’s also about where you use it.
A few extra milliseconds inside a transaction might seem harmless..
Until they’re multiplied across thousands of concurrent requests.

LINKS
- http://repository.save

#326 Raja Anand

Java / Spring

Lead Java Engineer · Spring Boot & Microservices · Building Scalable Backend Systems at HCL Technologies
3mo · **View on LinkedIn**

Spring Boot - @Cacheable & @CacheEvict
Caching is not just about storing data...
You also have to keep the data consistent when it changes.
Spring Boot provides 2 annotations for this:
1. @Cacheable - Store & Reuse Data
@Cacheable caches the result of a method so repeated calls with the same parameters skip the execution.
→ Example (see attached image ↓)
→ Behavior
First call → executes method & caches result
Next calls (same id) → returned directly from cache
Improves performance by avoiding repeated DB calls
2. @CacheEvict - Keep Cache in Sync
When data changes, the cache must be updated or cleared. That’s where @CacheEvict comes in.
→ Example (see attached image ↓)
→ Why it matters
Removes stale data from cache
Ensures users don’t see outdated results
Keeps cache consistent with the database
#SpringBoot #Java #Caching #BackendDevelopment #WebDevelopment #SoftwareEngineering

#327 Niladri Chatterjee

System Design

VP Domain Architect | Payments Modernization | API-First & Microservices | Data Mesh, Cloud Migration | EA Governance | Banking Payments & ISO 20022 | SRE Observability Leader | AI & Agentic AI | Digital Transformation
3mo · **View on LinkedIn**

Designing Low-Latency Payment Systems: It's More Than Just Speed
Everyone wants faster payments.
But in reality, customers don't measure success in milliseconds.
They measure it by one simple question:

"Did my payment go through?"

As payment volumes continue to grow, reducing latency isn't just about buying faster infrastructure. It's about making smarter architectural decisions.

A low-latency payment platform is built on principles like:

• Intelligent payment routing

• Event-driven architecture

• In-memory processing where it matters

• Asynchronous communication patterns

• Resilient APIs with minimal network hops

• Real-time observability and proactive monitoring

But here's the catch...

Optimizing for speed alone can introduce new risks.

The best payment platforms strike the right balance between performance, resilience, security, and reliability.

A transaction completed in 80 milliseconds isn't a success if it compromises resilience or fails under peak load.

most dependable system at scale.

As architects, our goal isn't to build the fastest system.

It's to build the

Because in payments, consistent performance beats occasional speed.

If you had to optimize one metric for a payment platform, what would it be- latency, availability, scalability, or resilience?

#Payments #PaymentSystems #EnterpriseArchitecture #LowLatency #Banking #DigitalPayments #CloudArchitecture #EventDrivenArchitecture #FinTech #TechnologyLeadership

#328 Rakesh kumar Java / Spring

Java Backend Developer | Spring Boot | Microservices | Kafka | REST APIs | Sharing Java & AI Content | Helping Developers Crack Interviews

3mo · **View on LinkedIn**

Every expert was once a beginner.

Today, I'm starting my journey of sharing practical Java Backend Development content.

I'll be posting about:

✓ Java

✓ Spring Boot

✓ REST APIs

✓ Microservices

✓ Kafka

✓ MySQL

✓ Redis

✓ Docker & AWS

✓ System Design

✓ Interview Preparation

✓ AI Tools for Developers

My goal is simple:

Learn → Build → Share → Grow Together.

Which Java topic would you like me to cover first? ↓

#Java #SpringBoot #BackendDevelopment #Microservices #Kafka #Programming

#329 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Varun Gupta • 3rd+

3mo · View on LinkedIn

Everyone is building RAG systems.

But not everyone needs a Vector Database.

For the last few years, the default architecture has been:

Documents → Chunking → Embeddings → Vector DB → Retrieval → LLM

But a new approach is emerging:

Vectorless RAG

Instead of maintaining embedding pipelines and vector stores, the LLM itself performs retrieval, reasoning, re-ranking, and context selection directly from your data.

This means:

✓ No embedding generation

✓ No vector database management

✓ Lower infrastructure costs

✓ Faster deployment cycles

✓ Simpler architecture to maintain

For enterprises dealing with:

Policy documents

SOPs

Contracts

Knowledge bases

Government records

Enterprise portals

Vectorless RAG can significantly reduce operational complexity.

The question is no longer:

"Can we build a RAG system?"

The better question is:

"Do we really need vectors for this use case?"

What are your thoughts?

Will Vectorless RAG become mainstream, or will Vector Databases continue to dominate enterprise AI architectures?

#AI #GenerativeAI #RAG #VectorlessRAG #LLM #EnterpriseAI #ArtificialIntelligence #MachineLearning #AIAgents #DataEngineering #AIArchitecture #Innovation #VarunGuptaAIConsultant

MEDIA document/carousel (possible)

#330 Priyam Jain

Other

Data Engineer @ WestCx | Ex-Paytm, ThoughtWorks, CTS | Azure & Databricks Certified | Big Data | PySpark | SQL | Azure/AWS Data Engineer

3mo · View on LinkedIn

One PDF That Made DSA Revision Much Easier

After spending time going through Data Structures & Algorithms, I realized something important...

Learning a concept once isn't enough.

The real improvement happens when you revisit it until solving problems becomes second nature.

Recently, I found a well-organized handwritten DSA revision guide covering almost everything needed for interviews:

✓ Arrays & Strings

✓ Linked Lists, Stacks & Queues

✓ Hashing & Trees

✓ Graphs (BFS & DFS)

✓ Recursion & Dynamic Programming

✓ Sorting & Searching

✓ Common Interview Patterns

✓ Advanced DSA Concepts

What I liked most wasn't just the content—it was how every topic was summarized into concise notes with diagrams, examples, and key points.

These are the kind of notes you can revisit in 15–20 minutes before an interview instead of spending hours searching through different resources.

One thing I've learned is:

DSA isn't about memorizing solutions. It's about recognizing patterns.

The more problems you solve, the more you'll notice that many "new" questions are just variations of concepts you've already seen.

Small improvements every day eventually turn into confidence during interviews.

Which DSA topic challenged you the most?

For me, it's always been Dynamic Programming. Understanding the thought process behind it takes time, but once it clicks, it's incredibly rewarding.

For more such posts follow Priyam Jain

#dsa #datastructures #algorithms #codinginterview #softwareengineer #programming #computerscience #coding #leetcode #techcareers

DSA Handwritten

.

9 pages

MEDIA document/carousel (possible)

#331 Suchita Chaturvedi

System Design

Principal Software Architect | Senior Java Architect | Enterprise Applications | Performance & Monitoring Tools | AI-powered Applications (LLMs)

3mo · **View on LinkedIn**

Ever wondered what happens behind the scenes when millions of Indians hit "Book Now" at exactly 10:00 AM for Tatkal tickets?

Within seconds, the system receives an overwhelming surge of requests, yet it somehow ensures that:

✓ Only one person gets a seat

✓ No double booking occurs

✓ The platform remains available under extreme load

This makes the Tatkal booking platform one of the most fascinating examples of high-scale distributed system design.

Some key architectural challenges:

Handling millions of concurrent users

Preventing seat overselling under heavy contention

Managing traffic spikes using virtual waiting rooms and rate limiting

Maintaining low latency while integrating with external payment gateways

Detecting and blocking bots during peak booking windows

A simplified architecture typically includes:

→ API Gateway for authentication, routing & throttling

→ Virtual Queue to smooth sudden traffic bursts

→ Booking Service with distributed locking for seat consistency

→ Redis caching and read replicas for fast search responses

→ Message queues (Kafka/RabbitMQ) for asynchronous tasks like PNR generation and notifications

The most interesting part for me is the seat locking mechanism. Imagine 10,000 users trying to book the last available seat simultaneously—yet the system guarantees that only one succeeds.

That's distributed systems engineering at scale!

I've created a simplified architecture diagram explaining the complete flow. Would love to hear from fellow architects and engineers:

If you were designing this system today, what additional improvements or technologies would you introduce?

#SystemDesign #DistributedSystems #SoftwareArchitecture #Microservices #Scalability #HighAvailability #Java #Kafka #Redis #CloudArchitecture #Engineering #TechArchitecture #IRCTC #Tatkal #BackendEngineering

#332 Antrixsh Gupta

AI / LLM

Senior Solution & AI Architect & Data Science Leader @Genzeon Platform | Architecting LLM/GenAI Systems, Clinical Intelligence & Responsible AI for Healthcare & BFSI Industries | LinkedIn Top Voice

3mo · **View on LinkedIn**

Most teams pick a vector DB in the first hour of a RAG project.

They regret it six months later.

Not because the DB was bad. Because they picked for features instead of fit. Scale tier and RAG production fit are two completely different axes and almost no one maps both before committing.

Here is what actually matters:

→ Scale tier first

Chroma and LanceDB are prototype tools. They will not survive production load. FAISS is a library, not a database. Carrying any of these past the experiment stage is how re-architectures happen.

→ RAG fit second

Three dimensions decide whether your retrieval holds up in production. Hybrid search, metadata filtering, and reranking support. Most DBs are strong on one. Few are strong on all three.

→ Only two score High across all three RAG dimensions

Weaviate and Vespa. Every other DB has at least one gap. That gap becomes your bottleneck at scale.

→ Extend your existing stack before adding infrastructure

Already on Postgres? pgvector first. Already on MongoDB? Atlas Vector Search first. Do not add a new database until your current one actually breaks.

→ Speed versus depth tradeoff

Redis Vector wins on latency and caching. It loses on RAG fit. That tradeoff is fine for real-time retrieval. It is a problem for complex multi-turn agents.

→ Hyper-scale is a different category

Milvus and Vespa are built for billion-scale embeddings. Everything else degrades before you get there. Know your ceiling before you need it.

The vector DB decision is not reversible without pain.

Make it once, make it on the right axes.

P.S. Which vector DB has your team outgrown, and what finally forced the migration?

Follow Antrixsh Gupta for more insights

LINKS

- https://genzeon.one/

#333 AlgoMaster.io

Other

Visit website

3mo · View on LinkedIn

If you want to master Array problems for interviews, learn these 10 patterns:

1. Two Pointers: https://lnkd.in/gxVkhHvi
2. Sliding Window: https://lnkd.in/g_AapGpf
3. Prefix Sum: https://lnkd.in/gCHQegHZ
4. Kadane's Algorithm: https://lnkd.in/gMcB5M8q
5. Binary Search: https://lnkd.in/g93YZeEU
6. Cyclic Sort: https://lnkd.in/gUQxkz-P
7. Merge Intervals: https://lnkd.in/g45zauYS
8. Monotonic Stack: https://lnkd.in/gnx5JvUB
9. Hash Map Frequency: https://lnkd.in/gAefPAiH
10. Sorting + Greedy: https://lnkd.in/gWfq8B25

Repost to help others in your network

LINKS

- https://algomaster.io/
- https://lnkd.in/gxVkhHvi
- https://lnkd.in/g_AapGpf
- https://lnkd.in/gCHQegHZ
- https://lnkd.in/gMcB5M8q
- https://lnkd.in/g93YZeEU
- https://lnkd.in/gUQxkz-P
- https://lnkd.in/g45zauYS
- https://lnkd.in/gnx5JvUB
- https://lnkd.in/gAefPAiH
- https://lnkd.in/gWfq8B25

#334 Navya Shrri Bommu

Kafka / Messaging

Senior Data Engineer | AWS, Azure& GCP | Databricks| Data Governance| Spark | SQL | Python | Snowflake | Pyspark | Kafka | DBT | AI/ML| LLM | Ataccama | Fabric | Collibra| Informatica

3mo · View on LinkedIn

Kafka looks complicated until the core pieces click.

Most data engineers learn Kafka through scattered terms: topics, partitions, offsets, brokers, producers, consumers, replication, schema registry.

But Kafka becomes much easier when you understand how these concepts work together.

Topics - Logical streams where records are organized. Think of them as named event channels like orders, payments, clicks, or app events.

Partitions - A topic is split into partitions so Kafka can scale, process data in parallel, and preserve ordering inside each partition.

Producers - Applications that write events into Kafka topics. The message key often decides which partition receives the event.

Consumers - Applications that read records from Kafka and process them for analytics, alerts, pipelines, or downstream systems.

Consumer Groups - Consumers share work across partitions so workloads can scale without every consumer reading the same data.

Offsets - Each record has a position inside a partition. Offsets help Kafka track what has already been processed.

Brokers - Servers that store Kafka data and serve producers and consumers. A Kafka cluster usually has multiple brokers.

Replication - Copies of partitions improve durability and fault tolerance. If one broker fails, replicas help keep the system running.

Schema Registry - Keeps event data compatible and reliable as producers and consumers evolve over time.

Kafka is not just a messaging system.

It is the backbone for real-time data pipelines, event-driven systems, streaming analytics, and scalable data platforms.

#QuestionForGroup - Which Kafka concept was the hardest for you to understand at first?

MEDIA advertisement

#335 Shubham Srivastava Kafka / Messaging

Principal Data Engineer @ Microsoft CoreAI | ex-Amazon | Data Engineering

3mo · **View on LinkedIn**

Imagine running a library where 50 people are reading the exact same 500-page book at the same time.

One person is reading the final chapter in real-time, another is stuck three days ago on page 50, and a third is replaying the whole book from page 1.

If you had to print a brand-new copy of the book for every single reader, your printing costs would bankrupt you.

Yet, as shown in this question, Kafka handles this exact chaos effortlessly.

It serves 50 different consumers reading at entirely different speeds from a single 30-day pool of events.

It does not store 3 separate copies of the data, let alone 50. It stores exactly one.

Here is the deep engineering magic that allows Kafka to pull this off without breaking a sweat:

1. The Commit Log and Consumer Offsets

Kafka does not push data to consumers. It is a dumb storage system with a smart log. Data is written as an append-only commit log on the disk.

Every message gets a sequential ID called an offset. Kafka doesn't care who is reading what. Instead, each consumer group is responsible for keeping track of its own bookmark, its current offset.

The real-time reader's bookmark is at offset 1,000,000. The slow reader's bookmark is at offset 900,000. Kafka just sits back and lets them read from different spots on the same file.

2. The OS Page Cache Secret

If 50 consumers are hitting the disk at different points, wouldn't the hard drive explode from random I/O bottlenecks?

Kafka avoids this using the Operating System Page Cache. When data is written, it goes directly to the OS kernel memory cache before hitting physical storage.

Your real-time consumer is reading straight out of ultra-fast RAM.

3. Zero-Copy Data Transfer

This is where Kafka bypasses traditional JVM overhead.

Normally, moving data from disk to a network socket requires copying data four times across the user space and kernel space.

Kafka uses the Sendfile system call.

It bypasses the application completely, copying data directly from the OS page cache straight to the network socket. This means serving a historical reader takes almost zero CPU cycles.

Kafka is just a collection of giant, immutable log files.

Consumers don't get data handed to them. Instead, they just move their own bookmarks across the exact same file at whatever speed they want.

#336 Aashish Gupta System Design

Sr Software Engineer@Uber | Mentoring Engineers in the AI Era | Backend & System Design | Building Scalable Systems | Brand Collabs

3mo · View on LinkedIn

Distributed Job Scheduler is one of the most common interview questions, and during my tenure at Walmart, I designed and developed it in production.

What most people don't get to see is what it looks like once it's actually running in production, handling retries, crashes, and cancellations at scale.

Here's how the pieces fit together:

Getting a job into the system:

API Gateway - Auth, rate limiting, and routing sit here before a request ever touches internal services.

Job Service - owns create/edit/schedule/cancel and publishes every job execution event to Kafka. Nothing executes without going through this path first.

Job Search Service - a separate read path so users can look up jobs by jobId or status without hitting the write-heavy Job Service.

Getting a job to actually run:

Watcher Service - polls every 10–20 sec for jobs scheduled in [now, now+5 min] and flips them to Queued. It's also the first line of failure detection: any "running" job whose modified_time is stale by 10–15 sec is assumed dead.

Executor Service - runs the job (Python scripts, inside Docker) and pushes failures straight to Kafka for retry.

Docker Execution Layer - every job gets its own container. Fault isolation and a consistent runtime, without one bad script taking down a shared process.

Zookeeper - heartbeats from executors, used for health checks and cluster coordination.

When things fail:

Retry Mechanism - failed executions land on a Kafka retry topic, retry count gets bumped in the DB.

Failure Detection — the hard case isn't the job that fails cleanly, it's the executor that crashes mid-job and never gets to report anything. Stale modified_time on a "running" job is the only signal you get — so the Watcher treats it as a failure and moves on.

Killing a job on purpose:

Cancel Job Flow - future jobs are canceled via a Kafka event before they're ever queued. Running jobs are harder to manage: you can't unpublish a job that's already executing.

Redis - executors poll Redis for a cancel signal mid-run. TTL on the key means a stale cancel request expires instead of haunting a job that already finished.

Making it queryable:

Job Consumer Service - consumes the Kafka events and is the only writer of status, retry count, and execution metadata into the database. One writer, one source of truth.

Cassandra - user_id as partition key, job_id as clustering key. Job lookups per user stay fast no matter how wide the table gets.

S3 - scripts, files, and artifacts a job needs at execution time live here, not in the DB.

It's a small interview topic that hides a genuinely hard production problem: detecting failures, retrying safely, and cancelling something that's already running.

#interview #systemdesign

#337 Gourav A.

System Design

Principal Engineer @ Microsoft | Ex-Intuit | Building AI-Driven Cybersecurity & Threat Intelligence Platforms

3mo · View on LinkedIn

Optimistic Locking vs. Pessimistic Locking — How SQL Databases Handle Concurrent Updates

Have you ever wondered what happens when two users try to update the same database row at exactly the same time?

Without proper concurrency control, one update could overwrite the other, leading to inconsistent or incorrect data.

SQL databases typically solve this problem using two concurrency control strategies: Optimistic Locking and Pessimistic Locking.

The infographic below illustrates how each approach works.

✓ Optimistic Locking (Version-Based)

Instead of locking the row, each transaction reads the current version number.

When updating, the database verifies that the version hasn't changed:

UPDATE accounts

SET balance = ?, version = version + 1

WHERE id = ? AND version = 5;

If another transaction has already modified the row, the version check fails immediately.

Key characteristics:

✓ No row-level lock

✓ Better throughput for read-heavy systems

✓ Conflicts detected during UPDATE

✓ Application retries on failure

Think of it as: "Try first. If someone already changed the data, fail fast and retry."

Pessimistic Locking (Row-Level Lock)

Before modifying data, the transaction explicitly locks the row:

SELECT * FROM accounts

WHERE id = 101

FOR UPDATE;

While one transaction holds the lock:

Other transactions requesting the same row wait in a queue

They execute only after the lock is released

No version conflict occurs because updates happen sequentially

Key characteristics:

✓ Row locked immediately

✓ Other requests wait instead of failing

✓ Strong consistency for highly contended data

✓ Lower concurrency but predictable execution

Think of it as: "Wait your turn before making changes."

When should you use each?
Choose Optimistic Locking when:
Conflicts are rare
Your application is read-heavy
High throughput is important
Retries are acceptable
Examples:
E-commerce product catalog
User profiles
Content management systems
Choose Pessimistic Locking when:
Data integrity is critical
Many users update the same records
Lost updates are unacceptable
Examples:
Banking transactions
Digital wallets
Inventory management
Ticket booking systems
Key Takeaway
There isn't a universally "better" approach.
The right choice depends on your application's contention level, consistency requirements, throughput goals, and user experience.
Understanding these trade-offs is essential for designing scalable, reliable, and high-performance backend systems.
Which locking strategy have you used in production, and what trade-offs did you observe?
#SystemDesign #SoftwareEngineering #BackendEngineering #SQL #Databases #Concurrency #DistributedSystems #Java #Microservices #Architecture #Tech

#338 Puneet Patwari System Design

Principal Software Engineer @Atlassian| Ex-Sr. Engineer @Microsoft | Sharing insights on SW Engineering, Career Growth & Interview Preparation

3mo · **View on LinkedIn**

It's a Wrap! Here are all the links to 25 important system design concepts I explained on my Scaling and Architecture series I just finished today.
1) Load Balancing: https://lnkd.in/gH9rdjCx
2) CDN: https://lnkd.in/g83A7-rM
3) Caching: https://lnkd.in/gTjxhv2V
4) Cache Invalidation: https://lnkd.in/geC955AY
5) Rate Limiting: https://lnkd.in/gWqJzCNJ
6) API Gateway: https://lnkd.in/gBNKpecH
7) CAP Theorem: https://lnkd.in/g4yFYkEi
8) Sharding: https://lnkd.in/gFi23iNV
9) Replication: https://lnkd.in/gikkrmNp
10) Partitioning: https://lnkd.in/gQhJS8ii
11) Queues: https://lnkd.in/gPGiuxtu

12) Microservices: https://lnkd.in/gZfYV2Qu
13) Microservices Vs Monoliths: https://lnkd.in/gM-dKE3D
14) Fault Tolerance: https://lnkd.in/gdamMmtc
15) Database Scaling: https://lnkd.in/ghq4v_gQ
16) Service Discovery: https://lnkd.in/gjfbNVBe
17) Consistency models: https://lnkd.in/gGkMENA3
18) Eventual Consistency: https://lnkd.in/gdSn54SK
19) Distributed Transactions: https://lnkd.in/gTc8pSbH
20) Leader Election: https://lnkd.in/g-kwhzSb
21) Horizontal vs Vertical Scaling: https://lnkd.in/gW-Vi9Qt
22) Back of the Envelope Estimation: https://lnkd.in/gQ6vtM3U
23) Idempotency, Data Latency & Finale: https://lnkd.in/gapgNSgh

---

P.S: If you're preparing for interviews and want a detailed guide with 90+ fundamentals organized in one place, check out this link → puneetpatwari.in

LINKS

- https://lnkd.in/gH9rdjCx
- https://lnkd.in/g83A7-rM
- https://lnkd.in/gTjxhv2V
- https://lnkd.in/geC955AY
- https://lnkd.in/gWqJzCNJ
- https://lnkd.in/gBNKpecH
- https://lnkd.in/g4yFYkEi
- https://lnkd.in/gFi23iNV
- https://lnkd.in/gikkrmNp
- https://lnkd.in/gQhJS8ii
- https://lnkd.in/gPGiuxtu
- https://lnkd.in/gZfYV2Qu
- https://lnkd.in/gM-dKE3D
- https://lnkd.in/gdamMmtc
- https://lnkd.in/ghq4v_gQ
- https://lnkd.in/gjfbNVBe
- https://lnkd.in/gGkMENA3
- https://lnkd.in/gdSn54SK
- https://lnkd.in/gTc8pSbH
- https://lnkd.in/g-kwhzSb
- https://lnkd.in/gW-Vi9Qt
- https://lnkd.in/gQ6vtM3U
- https://lnkd.in/gapgNSgh
- http://puneetpatwari.in

#339 Marcos Vinicius Marcon

Java / Spring

Co-Founder and Principal Software Architect at Eurobotics Brasil

3mo · **View on LinkedIn**

Java 26 introduces a cleaner and safer way to propagate contextual data across a call tree: ScopedValue.

Before modern Java, values such as tenant IDs, request IDs, authentication details, and tracing information were commonly stored in a ThreadLocal.

That approach works, but it comes with an important responsibility: the value must always be removed. If cleanup is forgotten, the context may remain attached to a reused thread and leak into another request.

With ScopedValue, the value exists only within a clearly defined execution scope. Any method called inside that scope can access it, including deeply nested service and repository layers.

Once the execution finishes, the binding disappears automatically.

The key difference is simple:

- ThreadLocal attaches data to a thread.

- ScopedValue attaches data to an execution scope.

This makes contextual data easier to reason about, automatically cleaned up, effectively immutable within the scope, and better suited for virtual threads and structured concurrency.

It is a small API change, but it solves a very common backend problem in a much cleaner way.

#Java #Java26 #ScopedValue #Concurrency #VirtualThreads #BackendDevelopment #SoftwareEngineering

#340 Jay kumar

Other

Computer Science Engineering Student at Galgotias College of Engineering & Technology | Java Developer | Data Structures & Algorithms Learner | Web Development Enthusiast | AI Enthusiast

3mo · View on LinkedIn

3 Powerful Problem-Solving Patterns Every Programmer Should Know

Many coding interview problems can be solved by recognizing the right pattern rather than memorizing solutions.

Here's a quick comparison of three essential techniques:

Binary Search

✓ Best for sorted data

✓ Eliminates half of the search space in each step

Time Complexity: O(log n)

Sliding Window

✓ Ideal for subarray and substring problems

✓ Expands and shrinks a window to optimize performance

Time Complexity: O(n)

↑ Two Pointer

✓ Great for sorted arrays, pairs, and palindrome problems

✓ Uses two indices moving toward each other or in the same direction

Time Complexity: O(n)

MEDIA advertisement

#341 Thiago Fernandes

Java / Spring

Software Engineer | Java Software Engineer | Spring Boot | Microservices | Kafka | Cloud (GCP/AWS) | Banking Systems | AI Integration

3mo · View on LinkedIn

Java Streams are a powerful way to write cleaner, more readable, and functional-style code.

With methods like filter(), map(), flatMap(), collect(), and reduce(), Java developers can process data in a more declarative and efficient way, reducing boilerplate and improving code maintainability.

A must-know feature for modern Java development.

#342 AlgoMaster.io

Other

Visit website

3mo · View on LinkedIn

10 Must-Know Time Complexity Patterns for Interviews:

1. Hash Lookup - O(1) average

A lookup usually takes constant time, regardless of input size. In the worst case, it can degrade to O(n).

2. Halving Loop - O(log n)

The input size is divided by a constant factor in every step, such as in binary search.

3. Single Loop - O(n)

Each element is processed once.

4. Sequential Loops - O(n + m)

You iterate over two separate inputs one after another, not with one loop nested inside the other.

5. Loop + Binary Search - O(n log n)

A loop runs n times, and each iteration performs O(log n) work.

6. Divide & Conquer - O(n log n)

The problem is divided across roughly log n levels, with O(n) total work at each level. Merge sort is a common example.

7. Nested Loops - O(n2)

For every element, you iterate over all elements again.

8. Triangular Loop - O(n2)

The inner loop performs fewer iterations each time, but the total is still roughly n(n − 1) / 2, which is quadratic.

9. Branching Recursion - Exponential Time

When each recursive call creates multiple new calls, the work can grow exponentially.

10. Permutations - O(n!)

There are n! possible orderings, and producing or copying each complete permutation can take O(n) time.

If you want to learn Big O and time complexity in more detail, click here: https://lnkd.in/gKZJXAyg

Repost to help others in your network.

LINKS

- https://algomaster.io/
- https://lnkd.in/gKZJXAyg

#343 Shubham Adik Other

Automation Engineer | SDET | 5+ YOE | Cards & Payments Domain | 12k+ LinkedIn

3mo · **View on LinkedIn**

ISO 8583 Transaction Lifecycle – What Really Happens After You Tap Your Card?

Most people think a payment is complete the moment they see "Approved" on the POS terminal. In reality, that's only the first step. Every card transaction passes through four important stages before the merchant actually receives the money.

→ 1. Authorization

The issuer validates:

✓ Card details

✓ Available balance / credit limit

✓ PIN/CVV

✓ Fraud & risk checks

Common MTIs:

✓ 0100 – Authorization Request

✓ 0110 – Authorization Response

Money is NOT transferred at this stage. Only funds are authorized.

→ 2. Clearing

Transaction details are exchanged between:

✓ Merchant

✓ Acquirer

✓ Card Network

✓ Issuer

This stage confirms that the transaction took place successfully.

Common MTIs:

0200

0220

0230

→ 3. Settlement

This is when the actual movement of money happens.

Funds move from:

Issuer Bank → Card Network → Acquirer Bank → Merchant Account

The merchant finally gets paid.

→ 4. Reconciliation

All participants compare transaction records to ensure everything matches.

This helps detect:

✓ Missing Transactions

✓ Duplicate Transactions

✓ Settlement Mismatches

✓ Reversal Issues

End-to-End Payment Flow

Customer → POS Terminal → Authorization → Clearing → Settlement → Reconciliation → Merchant Receives Funds

Key Takeaway

Authorization approves the payment.

Settlement transfers the money.

#ISO8583 #Payments #FinTech #Banking #PaymentSwitch #CardPayments #DigitalPayments #SoftwareTesting #QA #AutomationTesting #TransactionProcessing #POS #ATM #PaymentLifecycle

MEDIA advertisement

#344 Suneel Kumar Manepalli

AI / LLM

Technical Lead | Java | Spring | Cloud & DevOps | SRE | Python | GenAI | Agentic AI | MCP | Banking Technology | AI Enthusiast

3mo · **View on LinkedIn**

How Modern AI Systems Actually Work

Many people think AI starts and ends with an LLM. In reality, production-grade AI is a complete ecosystem where every layer plays a critical role.

Here’s a simplified view of the modern AI stack:

LLM – The reasoning engine that generates, explains, and creates.

RAG (Retrieval-Augmented Generation) – Retrieves the most relevant information before the model responds, reducing hallucinations.

Vector Database – Stores embeddings for semantic search, enabling retrieval by meaning instead of keywords.

AI Agent – Orchestrates workflows, decides the next action, and interacts with tools autonomously.

MCP (Model Context Protocol) – A standardized way to connect AI models with external tools, APIs, databases, and enterprise systems.

Guardrails – Enforces security, compliance, safety, and business rules to ensure reliable AI behavior.

Evals – Measures accuracy, latency, cost, safety, and overall system performance for continuous improvement.

The biggest shift in AI isn't just better models—it's building complete AI systems where retrieval, orchestration, tooling, safety, and evaluation work together.

As AI applications mature, understanding this stack becomes essential for developers, architects, and business leaders alike.

#ArtificialIntelligence #GenerativeAI #LLM #RAG #AIAgents #VectorDatabase #MCP #MachineLearning #DataScience #AIEngineering #Tech #Innovation #AIArchitecture

#345 RAMAKRISHNA NOOLU

System Design

Payment Systems Engineer & Application Admin | 10 YOE / ISO 8583 / ATM 'State Table' / POS / Payment Switch / Narada & IST Switch / Tieto CardSuite RTPS / Immediate Joiner

3mo · **View on LinkedIn**

Understanding Payment Switch Architecture — Simplified for Everyone

Ever wondered what happens behind the scenes when you swipe your card or make an online payment? That's where a Payment Switch comes in — the invisible backbone that connects merchants, banks, and networks to make transactions seamless and secure.

A payment switch acts like a traffic controller for financial transactions, ensuring every payment request reaches the right bank, gets validated, and returns a response in milliseconds.

How a Transaction Flows Inside a Payment Switch

Input Channels – Transactions originate from ATMs, POS terminals, mobile apps, or online gateways.

Input Interface – Converts each request into a standard format (like ISO 8583).

Validation Module – Performs fraud checks, syntax validation, and limit verification.

Routing Engine – Decides which bank or network (Visa, Mastercard, RuPay) should process the transaction.

Authorization Module – Sends the request to the bank for approval or decline.

Response Handler – Converts the bank's reply and sends it back to the originating channel.

Logging & Settlement – Records every transaction for audit, reconciliation, and end-of-day settlement.

Simplified Block Diagram

Code

Input Channels → Input Interface → Validation Module → Routing Engine → Authorization Module → Response Handler → Logging & Settlement

Each module plays a critical role in maintaining speed, accuracy, and security across millions of daily transactions.

Key Checks at Each Stage

ModuleWhat It DoesInput InterfaceEnsures message format & mandatory fieldsValidationFraud detection, syntax & limit checksRouting EngineChooses correct bank/networkAuthorizationConfirms balance, PIN, card validityResponse HandlerSends approval/decline backLoggingMaintains audit trail & settlement data

In short: A payment switch is the heart of digital payments — connecting multiple systems, enforcing security, and ensuring that every transaction completes smoothly.

I recently created a visual Payment Switch Architecture Diagram to make this easier to grasp. It shows how each module interacts and how data flows from initiation to authorization.

→ If you’re building or integrating payment systems, understanding this flow is essential for reliability and compliance.

#FinTech #Payments #BankingTechnology #DigitalPayments #Architecture #PaymentSwitch #FinancialInnovation #TechExplained #ISO8583 #TransactionFlow

#346 Gaurav Nikumbh

Other

Senior Product Manager - Payments & AI | IIM Calcutta | Ex - Founder | Payments Platform & Money Movement | Payment Processing, ACH, UPI, Cards, Fraud & Risk | Payment Orchestration | $2B+ TPV Across 50+ FI’s

3mo · **View on LinkedIn**

Payment Gateway vs Payment Switch - most people in payments use these terms interchangeably. They're not the same thing.

One faces the customer. The other never will.

Here's a clean breakdown of what each does, where each sits, and why the distinction matters.

What they are

A Payment Gateway is merchant-facing software - the digital layer that collects payment details, handles authentication (OTP, CVV, 3DS), initiates the transaction, and manages settlement and reconciliation.

➥ Analogy: cashier + POS machine + accounting system for online payments.

A Payment Switch is a backend routing engine - invisible to merchants and customers entirely. It lives inside processors, banks, and gateways.

➥ Analogy: traffic controller for payment flows.

What each one actually does

Payment Gateway:

➥ Collects and validates payment info

➥ Initiates and securely transmits the transaction

➥ Handles refunds, chargebacks, reconciliation, and settlement

➥ Provides merchant-facing dashboards and APIs

Payment Switch:

➥ Routes transactions to the right PSP or acquirer

➥ Applies BIN-based, time-based, and amount-based smart routing rules

➥ Transforms message formats - ISO 8583, JSON, XML

➥ Manages failover routes, load balancing, and uptime

Where they sit in the payment flow

→ Gateway → at the edge, facing the user

→ Switch → inside the processing core, connecting systems

A gateway can contain a switch internally. A switch never replaces a gateway externally. They solve different problems at different layers.

Who uses them

Gateway users:

➥ Merchants (e-commerce, POS, apps)

➥ Fintechs building merchant payment products

➥ Customers at checkout

Switch users:

➥ Payment gateways routing downstream

➥ Banks and processors managing multi-acquirer infrastructure

➥ Large PSPs handling high transaction volumes across schemes

If you've ever debugged a failed transaction and found it died in routing rather than auth, you've already felt the switch layer - even if you didn't have a name for it.

At scale, these two always split. And when they do, the switch becomes one of the most critical - and least visible - pieces of the entire payments stack.

#Payments #PaymentGateway #PaymentSwitch #PaymentsEngineering #Fintech #PSP #PaymentsInfrastructure #ProductManagement #Acquirer #SmartRouting

LINKS

- https://topmate.io/gaurav_nikumbh

#347 Alok Sharan

AI / LLM

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author

3mo · **View on LinkedIn**

RAG looks simple when shown as one flow:

user query → retrieve context → generate answer.

In production, it is a much deeper architecture.

The quality of a RAG system depends on many small design decisions working together: how documents are parsed, how content is chunked, how embeddings are created, how vectors are stored, how retrieval is tuned, and how responses are grounded back to trusted sources.

From an architecture perspective, terms like embeddings, chunking, vector database, retriever, re-ranking, grounding, metadata filtering, and hybrid search are not just concepts.

They directly affect accuracy, latency, cost, trust, and scalability.

A weak ingestion pipeline creates poor retrieval.

Poor chunking reduces context quality.

Bad metadata filtering returns irrelevant results.

No re-ranking lowers answer precision.

No source attribution reduces trust.

No evaluation loop means the system cannot improve.

This is why RAG should not be treated as a quick add-on to an LLM application.

It is an information architecture problem.

A good RAG system needs:

→ clean document parsing

→ strong chunking strategy

→ reliable embeddings

→ efficient vector search

→ controlled retrieval

→ grounding and source attribution

→ latency and throughput monitoring

→ guardrails and feedback loops

The model is only one part of the system.

The real engineering work sits in the retrieval layer, data quality, controls, and continuous evaluation.

That is where RAG moves from a demo to something useful in production.

Which RAG architecture term do you think teams underestimate the most?

MEDIA document/carousel (possible)

#348 Chirag Jain

Java / Spring

Java Full Stack Developer | Microservices • Python • AI & ML | Exploring Quantitative Finance & Algorithmic Trading | NSUT EE'24 | Data Science @ IIT Madras'25

3mo · **View on LinkedIn**

Most Java developers use HashMap every day, but how many actually know what happens behind the scenes?

I used HashMap for a long time before I took the time to understand how it really works internally. Once I did, many interview questions—and even everyday coding decisions—started making much more sense. Here's a simple breakdown:

1. Hashing

When you insert a key-value pair, Java calculates the key's hash code using the hashCode() method. That hash is then transformed into an index where the data should be stored. Think of it like assigning every key a "home address."

2. Buckets

A HashMap is basically an array of buckets. Each bucket can hold one or more entries. If two different keys point to the same bucket, that's called a collision.

3. Collision Handling

What happens if two keys land in the same bucket? Java handles this in two ways:

- Linked List (for a small number of collisions)

- Red-Black Tree (when too many entries exist in the same bucket)

This change, introduced in Java 8, improves search performance from O(n) to O(log n) in heavily collided buckets.

4. Load Factor

The default load factor is 0.75. This means once the map becomes 75% full, Java decides it's time to grow the table. A higher load factor saves memory but increases collisions. A lower load factor uses more memory but generally provides faster lookups.

5. Resizing (Rehashing)

When the threshold is crossed:

- The bucket array size doubles.

- Existing entries are redistributed into the new buckets.

Although resizing is expensive, it doesn't happen often, so performance remains efficient overall.

Time Complexity:

- Insert: Average O(1), Worst Case O(n)

- Search: Average O(1), Worst Case O(n) (or O(log n) with tree buckets in Java 8+)

- Delete: Average O(1), Worst Case O(n)

Key Takeaway

HashMap looks simple from the outside, but internally it's a well-designed combination of hashing, arrays, linked lists, and balanced trees that makes data retrieval incredibly fast. Understanding these internals doesn't just help in interviews—it helps you write better, more efficient Java code.

#Java #Programming #SoftwareEngineering #DataStructures #HashMap #CodingInterview #JavaDeveloper #BackendDevelopment #DSA #TechCommunity #LearningInPublic #100DaysOfCode #Developers #Coding

#349 Apoorv Kapsime

Java / Spring

Java Backend Developer | Spring Boot, Microservices, PostgreSQL, Angular | Building Scalable APIs, Improving Production Stability & Optimizing System Latency

3mo · **View on LinkedIn**

10 Java Stream API Methods Every Java Developer Should Know (and When to Use Them)

When I first started using Java Streams, I mostly relied on filter() and map().

As I worked on more backend projects and prepared for interviews, I realized the real power of the Stream API isn't knowing every method—it's knowing when to use the right one.

Here are the Stream API methods I find myself using most often:

1. filter()

• Filters elements based on a condition.

• Great for selecting only the data you need.

2. map()

• Transforms each element into another object.

• I commonly use it to convert Entities into DTOs in REST APIs.

3. flatMap()

• Flattens nested collections into a single stream.

• A very common interview question.

```
orders.stream()
.flatMap(order -> order.getItems().stream())
.toList();
```

4. distinct()

• Removes duplicate elements from a stream.

• Useful when working with unique data.

5. sorted()

• Sorts elements naturally or with a custom Comparator.

• Keeps your results organized and predictable.

6. limit()

• Returns only the first N elements.

• Useful for pagination and previews.

7. skip()

• Skips the first N elements.

• Often paired with limit() for pagination.

8. peek()

• Helpful for debugging a stream pipeline.

• I avoid using it for business logic since it's meant for inspection.

9. collect()

• Converts a stream into a List, Set, Map, or another collection.

• Probably the Stream method I use the most.

10. reduce()

• Combines multiple elements into a single result.

• Perfect for totals, sums, concatenation, and custom aggregations.

Learning the Stream API isn't about memorizing every method.

It's about understanding which operation makes your code cleaner, more readable, and easier to maintain.

↓ Question for fellow Java developers:

Which Java Stream API method do you use the most, and why?

For me, it's map() because I use it in almost every REST API project.

Found this helpful?

Save it for future reference.

Share it with a fellow Java developer.

Comment your favorite Stream API method—I’m curious to know which one you use the most.

Follow Apoorv Kapsime for more Java, Spring Boot & Backend content.

#Java #JavaProgramming #StreamAPI #BackendDevelopment #SoftwareEngineering #Coding

LINKS

- http://orders.stream

#350 Akhila G

AI / LLM

Making AI feel less complex | 20k+ Followers | GenAI Trainer | Senior Software Engineer

3mo · **View on LinkedIn**

RAG Interview Question — PolicyBazaar (AI Engineer)

A policy doc gets updated this morning. But your assistant still serves the old answer.

The embedding was never refreshed... and a cached response is sitting there, stale.

The question:

How do you design index & cache invalidation so updated documents don't keep serving yesterday's answer?

This is where most RAG systems quietly break. Retrieval looks fine in the demo, then production starts lying to customers about coverage, premiums, and claim rules.

Here's how I'd answer ↓

✓ 1. Version every document

Each doc carries a content_hash + updated_at. On ingestion, compare hashes only re-embed chunks that actually changed (delta re-indexing), not the whole corpus. No silent drift.

✓ 2. Treat the vector index as mutable

On update: upsert new chunk vectors, then delete the old chunk IDs in the same transaction. Stale vectors lingering in the index are the #1 cause of "ghost" answers. Tie chunk IDs to doc_id:version so cleanup is deterministic.

✓ 3. Event-driven invalidation, not cron

A doc-update event (CDC / webhook / queue) triggers re-embedding immediately. Don't wait for a nightly batch — for insurance policy changes, "by tomorrow" is already wrong.

✓ 4. Make the cache version-aware

Cache key = hash(query) + knowledge_base_version. When a doc updates, bump the KB version → old cached answers become unreachable automatically. No manual flushing, no stale hits.

✓ 5. Safety net: TTL + verification

Short TTL on cached answers, and on cache hit, cheaply check whether the source chunks' versions still match. Mismatch → bypass cache, regenerate.

==========

We've covered questions like these in my AI Engineering Interview Master Bundle, a comprehensive set of 22 courses designed for real interview prep.

Explore the full guide here → https://lnkd.in/gqFkWZd4

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/gqFkWZd4

#351 Java Developers Community (moderated)

Java / Spring

Naveengandhi C. • 3rd+

3mo · **View on LinkedIn**

Our Java service was running inside a 2-core container.

CPU was sitting at 100%.

No traffic spike.

No obvious memory leak.

No runaway thread.

The application looked normal.

Then we checked what the JVM believed:

java -XX:+PrintFlagsFinal -version | grep -i ActiveProcessorCount

The result was the surprise:

ActiveProcessorCount = 64

The container had 2 CPUs.

The JVM thought it had 64.

That mismatch explained almost everything.

The JVM uses available CPU count for more than most people realize.

It affects GC thread sizing.

It affects JIT compiler threads.

It affects `ForkJoinPool.commonPool()`.

It can also influence frameworks that size worker pools using `Runtime.getRuntime().availableProcessors()`.

So if the JVM thinks it has 64 CPUs while the container only has 2, it may create far more background threads than the container can run efficiently.

Then the symptoms start.

GC runs more aggressively than expected.

Context switching increases.

Latency becomes unstable.

CPU stays high even when traffic does not justify it.

And everyone starts looking for a bug in the application logic.

The issue may simply be that the JVM was sized for the host, not the container.

This was especially common with older JDK versions before container awareness became reliable and enabled by default in modern Java.

The fix starts with making sure the JVM understands container limits:

-XX:+UseContainerSupport

On JDK 11 and newer, this is enabled by default.

But awareness is only step one.

You still need to verify what the JVM actually sees at startup:

java \

-XX:+PrintFlagsFinal \

-version 2>&1 | grep -E "ActiveProcessor|MaxHeap|RAM"

Run this in the same container image and runtime environment where the app actually runs.

Not just locally.

Not just in CI.

Memory has a similar trap.

If the JVM calculates heap from the host instead of the container limit, you may see sudden `OOMKilled` events in Kubernetes and assume the application has a memory leak.

Sometimes it does.

Sometimes the JVM simply sized itself against the wrong reality.

A safer container startup configuration usually includes:

java \

-XX:MaxRAMPercentage=75.0 \
-XX:InitialRAMPercentage=50.0 \
-XX:+UseG1GC \
-XX:+HeapDumpOnOutOfMemoryError \
-jar app.jar

For small CPU containers, also review GC thread settings instead of blindly trusting defaults.

The bigger lesson is simple:

Container limits are not just infrastructure settings.

They directly shape JVM behavior.

The infrastructure team may set CPU and memory limits.

The application team may tune the JVM.

But runtime behavior lives at the intersection of both.

Before blaming the code, ask one question:

What does the JVM think it is running on?

Sometimes your 2-core container is being treated like a 64-core machine.

And the JVM is doing exactly what it was configured to do.

What JVM flag or container setting has saved you during a debugging session?

#352 Alexandre Reis

Java / Spring

Senior Backend Engineer | Java & Spring Boot APIs | Generative AI for Software Engineering

3mo · View on LinkedIn

Most Java devs know Stream.collect(). Far fewer use Gatherer.

JEP 485 finalized Stream Gatherers in JDK 24, and they quietly solve a problem collect() never could: custom intermediate operations.

Before Gatherers, if you needed sliding windows, running totals, or stateful transforms mid-stream, you either wrote a manual loop or bolted on a library. Now it's built into the Stream API itself.

A few reasons this matters:

- Gatherers can be one-to-many, many-to-one, or many-to-many — collect() is strictly many-to-one

- Built-in options like windowFixed, windowSliding, fold, scan, and mapConcurrent cover most common cases out of the box

- They can short-circuit, so you can turn infinite streams finite mid-pipeline

- They compose with the rest of the Stream API — no separate abstraction to learn

The old way: accumulate into a List, then loop again to compute windows or running aggregates. The Gatherer way: express it as one intermediate step in the same pipeline, read top to bottom.

Takeaway: if you're still reaching for manual loops to do stateful stream processing, Gatherer is probably a one-line replacement.

#Java #JDK24 #StreamAPI #SoftwareEngineering #BackendDevelopment

#353 Chandrasekhar Y.

System Design

Senior Application Developer @ IBM | Java 8–25 | Spring Boot | Microservices | AWS | Kafka | React.js | Spring AI | Ex-Cognizant | Ex-TCS

3mo · View on LinkedIn

Two Microservices Update the Same Record at the Same Time... What Happens?

This is one of the most asked Java Microservices interview questions—and a real production challenge.

If not handled correctly, it can lead to:

✗ Lost Updates
✗ Data Inconsistency
✗ Duplicate Payments
✗ Race Conditions
The solution depends on the business use case:
✓ Optimistic Locking → Best for Inventory, Customer Profiles, Product Catalogs
Pessimistic Locking → Best for Banking, Payments, Seat Reservations
Idempotency → Best for Payment APIs, Order Creation, and Retry Mechanisms
Knowing when to use which approach is what separates someone who knows Spring Boot from someone who understands distributed systems.
I created this visual cheat sheet with:
Real-world examples
Spring Boot code snippets
Interview answer strategy
Pros & Cons
When to use each approach
I hope it helps you in your next Java Microservices interview!
Interview Question: If you were designing a payment service, which approach would you choose and why?
Let's discuss in the comments. ↓
#Java #SpringBoot #Microservices #SystemDesign #BackendDevelopment #SoftwareEngineering #JavaDeveloper #CodingInterview #TechInterview #DistributedSystems

#354 Mayur Gaikwad System Design

Software Engineer | Angular • TypeScript • Node.js • PostgreSQL | Enterprise Applications | Performance • APIs • System Design

3mo · **View on LinkedIn**

Offset pagination doesn't become a scalability problem because it's slow.
It becomes a correctness problem because page numbers are not stable.
Most discussions about offset pagination focus on performance. The explanation is familiar: as the offset grows, the database has to scan and discard more rows before returning the requested page. That's true, but it's only half the story.
The more interesting problem is that OFFSET is based on row positions, not on the records themselves.
Imagine a user opens the first page of a feed ordered by created_at DESC. Before they request the second page, a new post is published. The backend executes the exact same LIMIT and OFFSET query as before, but the first page now contains a different set of rows because every existing record has shifted down by one position.
Nothing is wrong with the SQL. Nothing is wrong with the index.
The meaning of Page 2 changed while the user was reading.
The result is subtle but important. The user may see a record twice because it shifted from the end of Page 1 to the beginning of Page 2. If a record is deleted instead, another record can shift into the previous page, causing the user to skip it entirely.
Cursor pagination approaches the problem differently.
It doesn't ask, What are the records currently at positions 11–20?
It asks, What are the next records after the last record the user actually saw?

That distinction is why cursor pagination is more than a performance optimization. It's a different consistency model. The cursor represents progress, not a page number.

However, that guarantee depends on another assumption that is often overlooked: the ordering must be stable.

Ordering by immutable fields such as (created_at, id) works well because those values never change. Ordering by mutable fields like likes_count, score, or popularity introduces a different failure mode. A post can move from below the cursor to above it while the user is scrolling. The cursor hasn't failed, and neither has the query. The data itself has changed its position in the ordering.

That's one of the reasons large-scale systems rarely paginate directly over live ranking signals. They typically paginate over a stable ordering often a snapshot or a precomputed ranked list and update that ordering independently.

The interesting tradeoff isn't offset vs. cursor.

It's position-based navigation vs. progress-based navigation.

One assumes the dataset is static.

The other assumes the dataset never stops changing.

#SystemDesign #DistributedSystems #BackendEngineering #ScalableSystems

#DatabaseInternals #SoftwareArchitecture #APIEngineering #PerformanceOptimization #PostgreSQL #Engineering #NodeJs #Java #Express

MEDIA advertisement

#355 Alex Xu System Design

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

3mo · **View on LinkedIn**

Pagination Strategies for Large Systems

Pagination looks simple when your table has a few thousand rows. It becomes a real problem once your data grows.

Here is how the main approaches compare when the dataset is large.

Most pagination implementations start with offset and page numbers. You ask for “?page=3&size=10” and the database skips 20 rows, then returns 10. Simple to build and easy to explain. The issue is that the database still reads every row it skips. Page 5 is fast. Page 5000 is slow.

Keyset pagination handles deep pages much better. Instead of counting rows, you tell the server to return items after ID 101. The query goes straight to that point using the index, so page 1 and page 10,000 take the same time.

Continuation tokens are common in large APIs like S3 and YouTube. The server returns a token, you send it back on the next request, and it resumes from where the last response ended. It hides the cursor logic from the client. The token is tied to the filters and sort order from the original request, so the client must keep using the same parameters until the pagination ends.

Time-based pagination works well for feeds and logs. You ask for items before a timestamp and read backwards. Add a secondary field, like an ID, for rows that share the same timestamp. Without it, you can lose records.

Over to you: Which one have you used?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gauQcE45

#systemdesign #coding #interviewtips

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gauQcE45

#356 Rishi Jha Kafka / Messaging

Backend Engineer | Core Banking & Payments | Java • Spring Boot • Kafka | Payment Systems • Distributed Systems | Banking & Fintech

3mo · View on LinkedIn

Australia to India in 37 Seconds: Is SWIFT Finally Becoming Real-Time?

A recent milestone in global payments caught my attention:

A cross-border payment from Australia to India was completed in just 37 seconds using SWIFT's new Retail Payments service.

At first glance, it sounds like SWIFT has become an instant payment network.

But that's not exactly what happened.

The Traditional SWIFT Model

Historically, an international payment looked like this:

Customer

↓

Sender Bank

↓

SWIFT

↓

Correspondent Bank(s)

↓

Receiver Bank

↓

Beneficiary

Because the payment often passed through multiple correspondent banks, it could take 1–3 business days due to:

Multiple intermediaries

Compliance and AML checks

Time zone differences

Manual processing in some cases

What Changed?

In this case:

The payment originated from Westpac (Australia).

It was routed through SWIFT's new Retail Payments service.

Standard Chartered India received the payment instruction.

The beneficiary was credited using IMPS, India's real-time domestic payment system.

Australia

|

Westpac

|

SWIFT Retail Payments

|

Standard Chartered India

|

IMPS

|

Beneficiary

The payment reached the beneficiary in 37 seconds.

Why Was It So Fast?

The speed wasn't because SWIFT replaced domestic payment systems.

Instead, SWIFT improved the cross-border messaging layer by enabling:

✓ Pre-validated payment information

✓ Direct connectivity between participating banks

✓ End-to-end tracking

✓ Better transparency

✓ Integration with domestic instant payment rails like IMPS

The final credit in India was completed over IMPS, which is already designed for instant domestic transfers.

The Technical Insight

Think of the payment journey in two parts:

1. Cross-Border Messaging

SWIFT securely carries the payment instruction between banks.

2. Domestic Settlement

Once the funds reach the destination country, a local real-time payment system (such as IMPS) completes the last-mile transfer.

This combination significantly reduces the overall payment time.

Why This Matters

For decades, international payments were considered slow, expensive, and difficult to track.

This milestone shows how global messaging networks and domestic instant payment systems can work together to make cross-border payments almost as seamless as local ones.

It may not be the end of correspondent banking, but it is a significant step toward a future where sending money internationally feels as fast as sending it across town.

Discussion

Do you think the future of cross-border payments will be driven by:

SWIFT's modernization,

regional instant payment networks,

or blockchain-based settlement?

I'd love to hear your thoughts.

#SWIFT #CrossBorderPayments #FinTech #Banking #ISO20022 #IMPS #Payments #FinancialTechnology #DigitalPayments #BankingTechnology

MEDIA advertisement

#357 Java Developers Community (moderated)

Java / Spring

Naveengandhi C. • 3rd+

3mo · **View on LinkedIn**

I once spent two hours debugging a `NullPointerException` on a line that looked impossible.

No method chain.

No object navigation.

No obvious dereference.

Just this:

int total = order.getDiscount();

How does a primitive assignment throw an NPE?

The answer was autounboxing.

We had an `Order` class with an optional discount:
public class Order {
private Integer discount;
}
`Integer`, not `int`, because a discount may not exist.
That part was fine.
The issue happened when the nullable wrapper was assigned to a primitive:
int total = order.getDiscount();
What Java effectively does is:
int total = order.getDiscount().intValue();
If `getDiscount()` returns `null`, Java calls `intValue()` on `null`.
Result: `NullPointerException`.
On a line that looks safe.
That is the trap with autoboxing and unboxing.
`int` to `Integer` is boxing.
`Integer` to `int` is unboxing.
The compiler inserts the conversion for you. You do not see it, so it is easy to forget that unboxing is still a method call.
Here are the places this shows up in real code.

Arithmetic with nullable wrappers
Integer a = getValueFromDb();
Integer b = getAnotherValue();
int result = a + b;
Both values are unboxed first. If either is `null`, it fails before the arithmetic starts.

Comparisons that look harmless
Integer x = null;
if (x == 10) {
// ...
}
This unboxes `x`.
If `x` is `null`, it throws.
Another trap:
Integer a = 1000;
Integer b = 1000;
System.out.println(a == b); // false
`==` compares object references for wrappers.
Small cached values may appear to work, usually from `-128` to `127`, but outside that range the result can surprise you.
Use:
Objects.equals(a, b)

Map lookups
Map<String, Integer> config = getConfig();
int timeout = config.get("timeout");

If the key does not exist, `Map.get()` returns `null`.
Assigning that to `int` triggers unboxing.
Safer:
int timeout = config.getOrDefault("timeout", 30);

Ternary expressions
Integer flag = condition ? 1 : null;
int value = flag;
If `condition` is false, `flag` is `null`.
The crash happens later during assignment.

The fix is not complicated.
The awareness is.
When a value can be missing, keep that visible:
Integer discount = order.getDiscount();
int total = discount != null ? discount : 0;
Or:
int total = Optional
.ofNullable(order.getDiscount())
.orElse(0);
The rule I follow now is simple:
Whenever a wrapper type becomes a primitive, ask:
Can this ever be `null`?
If yes, handle it before the assignment.
The compiler may not warn you.
The line may look innocent in review.
The stack trace may point to a place that makes no sense.
That is what makes autounboxing bugs frustrating.
They are not complex.
They are invisible.
Until production reminds you that `Integer` and `int` are not the same thing.

#358 Sathish Kumar Subramani AI / LLM

Senior Engineering Manager | Generative AI | Azure Cloud Solutions | Leadership Excellence | Driving Operational Efficiency

3mo · **View on LinkedIn**

Everyone's building on top of LLMs.
Far fewer can explain what happens between your prompt and the reply.
Strip away the math and it is 5 steps turning your words into the next word.
Step 1: Understand the Input (Tokenization)
• Text broken into smaller units called tokens, each mapped to a numeric ID.
• "Artificial intelligence is transforming" becomes [2987, 9231, 318, ...].
• The model never sees your words, only token IDs. This is why token count drives both your cost and your context limit. Every character you type is converted to currency before the model even starts working.
Step 2: Convert Tokens Into Context (Embeddings)

• Each token ID becomes a dense vector through an embedding layer.

• These vectors capture meaning and relationships in high-dimensional space.

• "King" and "queen" end up near each other not because of spelling but because of how they're used across billions of sentences. Raw IDs gain meaning here.

Step 3: Find What's Important (Attention)

• Queries ask questions. Keys store information. Values hold the details.

• The breakthrough that made modern LLMs possible. Every word weighed against every other word simultaneously. "It" knows what it refers to because attention connects them directly no matter how far apart they are.

Step 4: Process and Refine (Transformer Blocks)

• Multi-head attention plus feed-forward networks, wrapped in Add and Norm with skip connections.

• This block repeats across L layers: 12, 24, 48, and beyond.

• Depth is where capability comes from. Early layers catch syntax. Middle layers capture meaning. Deep layers handle reasoning. Stacking layers is largely why a bigger model "understands" more than a small one.

Step 5: Predict the Next Word (Generation)

• The model outputs probabilities for the next token: data 0.31, people 0.21, better 0.15...

• It picks one, appends it, and repeats, one token at a time until it hits an end token.

• An LLM isn't "thinking out" a full answer. It's predicting the next token, over and over, fast enough to look like thought. That single fact explains hallucinations (confident but wrong predictions), streaming (tokens arriving one by one), and why output tokens cost more (each one requires a full forward pass).

Why does understanding this matter practically?

• Cost control: tokens are the unit of billing. Fewer tokens, lower cost.

• Prompt design: the model attends to everything in context. Irrelevant context dilutes attention on what matters.

• Debugging hallucinations: the model predicts likely next tokens, not verified facts. Hallucination isn't a bug it's the mechanism working as designed on insufficient grounding.

• Model selection: more layers means more capability and more cost. Match depth to task complexity.

Which step reframed how you think about LLMs?

Repost this to help your network get started

+ Follow Sathish for more

#LLM #DeepLearning #AIEngineering

#359 Bijaya Bhaskar Swain

System Design

Tech Lead | Java | Spring Boot | Microservices | AWS

3mo · **View on LinkedIn**

Ever Wondered How Instagram Instantly Says "Username Already Taken"?

You type a username like bhaskar, and before you even click Sign Up, Instagram instantly tells you:

✗ Username already taken

It feels magical.

But behind the scenes, millions of users are checking usernames simultaneously.

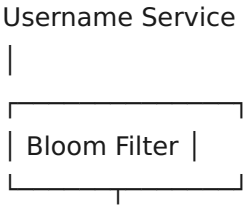
Here's how the system works.

Step 1: User Starts Typing

bh...

bhas...

bhaskar
The app doesn't wait until you press Submit.
Instead, after a short delay (around 300–500 ms), it sends a request to the backend.
This technique is called Debouncing, which prevents sending an API request for every keystroke.
Step 2: API Gateway
The request first reaches the API Gateway.
Responsibilities:
Authentication
Rate limiting
Routing
Load balancing
GET /username/check?name=bhaskar
Step 3: Bloom Filter (The Secret Weapon)
Searching a database for every request would overload the system.
Instead, Instagram first checks a Bloom Filter stored in memory.
A Bloom Filter is a super-fast probabilistic data structure.
It can answer:
✓ Definitely NOT taken
or
(!) Possibly taken
It never says "available" when the username actually exists.
If the Bloom Filter says Not Present, the server immediately returns:
✓ Username Available
No database query is needed.
Step 4: Database Verification
If the Bloom Filter returns Possibly Present, the Username Service checks the database.
Bloom Filter
↓
Possibly Present
↓
Username Database
If found:
✗ Username Taken
Otherwise:
✓ Username Available
This extra lookup happens only for a small percentage of requests.
Step 5: Cache the Result
Popular username checks are cached in Redis.
The next user checking the same username gets the answer almost instantly.
Final System Architecture
Instagram App
│
Debounce (300–500 ms)
│
API Gateway
│



Why Use a Bloom Filter?

✓ Extremely fast (memory lookup)

✓ Reduces database traffic

✓ Scales to billions of username checks

✓ Perfect for read-heavy systems

The next time Instagram instantly tells you a username is taken, remember:

It's not magic—it's smart system design powered by a Bloom Filter.

#360 Amit Raj System Design

Tech Lead @ Coforge | Ex-Deloitte | ASP.NET & .NET Core | Angular | Microservices & REST APIs | Databases (MSSQL, PostgreSQL, Snowflake) | Azure & AWS | Innovating with AI (Copilot, Claude) | 16.1k+ LinkedIn Family

3mo · **View on LinkedIn**

Most developers know how to code. Very few know these 15 design patterns, and that’s what sets apart good engineers from great system designers.

If you’re preparing for System Design interviews, bookmark this before your next round:

Singleton → https://lnkd.in/gTPiZV6X

Factory Method → https://lnkd.in/ghBca63c

Builder → https://lnkd.in/g6nW2bsv

Adapter → https://lnkd.in/gbVYyw7y

Decorator → https://lnkd.in/gv7aQpnC

Facade → https://lnkd.in/gJQiJu5v

Proxy → https://lnkd.in/gSavU2ZT

Composite → https://lnkd.in/g2nNVS-h

Observer → https://lnkd.in/gyBnaX9T

Strategy → https://lnkd.in/gakQYxub

Command → https://lnkd.in/gXY5a-WD

Iterator → https://lnkd.in/gmtKRahf

State → https://lnkd.in/gCm2YzWu

Template Method → https://lnkd.in/gjrBduDm

Chain of Responsibility → https://lnkd.in/g9q9QhFk

Once you start recognizing these patterns, you'll notice them everywhere — from backend frameworks to large-scale distributed systems.

Save this post, share it with your network, and revisit before your next interview.

Follow Amit Raj for more insights on cracking coding & system design interviews!

Hashtags for Reach

#SystemDesign #DesignPatterns #SoftwareEngineering #CodingInterview #LeetCode #DSA #CareerGrowth #InterviewPreparation #AmitRaj

LINKS

- https://lnkd.in/gTPiZV6X
- https://lnkd.in/ghBca63c
- https://lnkd.in/g6nW2bsv
- https://lnkd.in/gbVYyw7y
- https://lnkd.in/gv7aQpnC
- https://lnkd.in/gJQiJu5v
- https://lnkd.in/gSavU2ZT
- https://lnkd.in/g2nNVS-h
- https://lnkd.in/gyBnaX9T
- https://lnkd.in/gakQYxub
- https://lnkd.in/gXY5a-WD
- https://lnkd.in/gmtKRahf
- https://lnkd.in/gCm2YzWu
- https://lnkd.in/gjrBduDm
- https://lnkd.in/g9q9QhFk

#361 Namra Patel

System Design

Backend Engineer @ IBM | Java · Spring Boot · Microservices · REST APIs · GCP | Building Scalable, High-Throughput Systems

3mo · **View on LinkedIn**

Idempotency: The Unsung Hero of Reliable APIs

Imagine you click the "Pay Now" button...

The internet is slow, so you click it again.

And again.

Without idempotency, you might end up paying three times.

That's exactly why idempotency exists.

What is Idempotency?

An operation is idempotent if performing it multiple times produces the same result as performing it once.

For example:

✓ Updating a user's email to john@example.com

Whether the request is sent 1 time or 10 times, the final state remains the same.

But consider creating an order:

✗ POST /orders

Sending the same request multiple times could create multiple orders.

So how do companies like payment platforms prevent duplicate transactions?

They use an Idempotency Key.

How it works:

1. The client generates a unique key (for example, a UUID).

2. The server stores the key along with the response.

3. If the same request arrives again with the same key:

No duplicate processing happens.

The previously stored response is returned.

This makes retries safe, even if the client never receives the first response due to a timeout or network issue.

Where is idempotency commonly used?

Payment processing

Order creation

Ticket booking

Money transfers

Inventory reservation

Webhook event handling

A simple concept like an Idempotency Key can save businesses from duplicate payments, inconsistent data, and countless production issues.

Reliable distributed systems aren't just about handling success—they're about handling retries correctly.

Have you implemented idempotency in your applications? I'd love to hear your approach.

#Java #SpringBoot #BackendDevelopment #Microservices #SystemDesign #DistributedSystems #RESTAPI #SoftwareEngineering #Tech #Programming

#362 Ashish Pratap Singh

Other

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

3mo · **View on LinkedIn**

15 Must-Know Design Patterns:

1. Singleton: https://lnkd.in/g3VrJz-k
2. Factory Method: https://lnkd.in/gA6Uew8n
3. Builder: https://lnkd.in/gdTr2BBF
4. Adapter: https://lnkd.in/g_yB_CZn
5. Decorator: https://lnkd.in/g9zWv66w
6. Facade: https://lnkd.in/gHzPeaKG
7. Proxy: https://lnkd.in/g2MF2hvS
8. Composite: https://lnkd.in/gHwStDc3
9. Observer: https://lnkd.in/g4S_eGjy
10. Strategy: https://lnkd.in/gSjXJ3Cq
11. Command: https://lnkd.in/gffxnxih
12. Iterator: https://lnkd.in/g7F_PmD9
13. State: https://lnkd.in/gmfFnubm
14. Template Method: https://lnkd.in/gshGDpKE
15. Chain of Responsibility: https://lnkd.in/gayT82-s

Repost to help others in your network

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/g3VrJz-k
- https://lnkd.in/gA6Uew8n
- https://lnkd.in/gdTr2BBF
- https://lnkd.in/g_yB_CZn
- https://lnkd.in/g9zWv66w
- https://lnkd.in/gHzPeaKG
- https://lnkd.in/g2MF2hvS
- https://lnkd.in/gHwStDc3
- https://lnkd.in/g4S_eGjy

- https://lnkd.in/gSjXJ3Cq
- https://lnkd.in/gffxnxih
- https://lnkd.in/g7F_PmD9
- https://lnkd.in/gmfFnubm
- https://lnkd.in/gshGDpKE
- https://lnkd.in/gayT82-s

#363 Manu Agrawal

System Design

AI & Tech Creator | Ex-Google, Microsoft | AI Engineer | 143K+ LinkedIn | Tech Speaker | 50 Millions+ Impressions | View my portfolio

3mo · **View on LinkedIn**

Amazon & Airbnb Rejected Me 3 Times in System Design. These Free Articles Changed Everything.

- Cloudflare Engineering → https://lnkd.in/ggUaiQ47
- Stripe Engineering → https://lnkd.in/gbU_bfvj
- OpenAI Research & Engineering → https://openai.com/news/
- Anthropic Engineering → https://lnkd.in/gkgJhZ5T
- Databricks Engineering → https://lnkd.in/g8hirsXX
- Pinterest Engineering → https://lnkd.in/gTccjqfm
- Shopify Engineering → https://lnkd.in/gJ-cuTwv
- Discord Engineering → https://discord.com/blog
- Slack Engineering → https://slack.engineering
- Cloudflare Developers → https://lnkd.in/gY5NW9YC
- Snowflake Engineering → https://lnkd.in/gxGtTCRy
- Confluent Engineering → https://lnkd.in/gVPYGW3p
- CockroachDB Engineering → https://lnkd.in/g_UfDnS9
- ClickHouse Engineering → https://lnkd.in/gwqaPJzx
- Vercel Engineering → https://vercel.com/blog
- Figma Engineering → https://lnkd.in/giEGCpqY
- Instacart Engineering → https://lnkd.in/gz7rVFq5
- Airbnb Engineering → https://lnkd.in/gdSuCUWM
- Netflix TechBlog → https://lnkd.in/g6tvtWqe
- Meta Engineering → https://lnkd.in/gyFFD5AA
- Uber Engineering → https://lnkd.in/g45Q_vqk
- GitHub Engineering → https://lnkd.in/g9ua4u9F

If you're preparing for Google, Meta, Amazon, Microsoft, Uber, Stripe, Airbnb, or any Staff-level System Design interview...

Save this post for later.

#amazon #google #meta #microsoft #linkedin #jobs #jobs2026 #github #coding #airbnb

LINKS

- https://x.com/themanuagrawal
- https://lnkd.in/ggUaiQ47
- https://lnkd.in/gbU_bfvj
- https://openai.com/news/
- https://lnkd.in/gkgJhZ5T
- https://lnkd.in/g8hirsXX
- https://lnkd.in/gTccjqfm
- https://lnkd.in/gJ-cuTwv
- https://discord.com/blog
- https://slack.engineering
- https://lnkd.in/gY5NW9YC
- https://lnkd.in/gxGtTCRy

- https://lnkd.in/gVPYGW3p
- https://lnkd.in/g_UfDnS9
- https://lnkd.in/gwqaPJzx
- https://vercel.com/blog
- https://lnkd.in/giEGCpqY
- https://lnkd.in/gz7rVFq5
- https://lnkd.in/gdSuCUWM
- https://lnkd.in/g6tvtWqe
- https://lnkd.in/gyFFD5AA
- https://lnkd.in/g45Q_vqk
- https://lnkd.in/g9ua4u9F

#364 Srinath Vasudevan

Other

Global Payments Product Leader | Digital Payments | 20+ Years Leading Global Payment Platforms | Payment Acceptance | Expertise in Product Strategy, UX, Compliance & Multi-Country Expansion | Fintech & Payments Product

3mo · **View on LinkedIn**

Most people think payment success depends on the payment gateway.

In reality, it increasingly depends on Smart Routing.

Instead of sending every transaction through the same path, smart routing uses real-time data to choose the best route based on success rates, costs, issuer performance, and network availability.

The result?

- Higher payment success rates
- Lower transaction costs
- Fewer failed checkouts
- Better customer experience

As digital payments scale, the real competitive advantage won't just be processing payments, it will be sending every payment through the smartest possible path.

#FinTech #DigitalPayments #PaymentProcessing #PaymentOrchestration #SmartRouting #PaymentInnovation #FintechTrends #Payments

MEDIA advertisement

#365 Bhavesh Arora

Other

Fintech | NTT Data | Talks about Fintech and Marketing | 158k+ on Linkedin

3mo · **View on LinkedIn**

Most developers know how to code. Very few know these 15 design patterns.

If you're preparing for System Design interviews, bookmark this before your next round:

1. Singleton: https://lnkd.in/gTPiZV6X
2. Factory Method: https://lnkd.in/ghBca63c
3. Builder: https://lnkd.in/g6nW2bsv
4. Adapter: https://lnkd.in/gbVYyw7y
5. Decorator: https://lnkd.in/gv7aQpnC
6. Facade: https://lnkd.in/gJQiJu5v
7. Proxy: https://lnkd.in/gSavU2ZT
8. Composite: https://lnkd.in/g2nNVS-h
9. Observer: https://lnkd.in/gyBnaX9T
10. Strategy: https://lnkd.in/gakQYxub
11. Command: https://lnkd.in/gXY5a-WD
12. Iterator: https://lnkd.in/gmtKRahf

13. State: https://lnkd.in/gCm2YzWu
14. Template Method: https://lnkd.in/gjrBduDm
15. Chain of Responsibility: https://lnkd.in/g9q9QhFk

Once you start recognizing these patterns, you'll notice them everywhere - from backend frameworks to large-scale systems.

Credit: Ashish Pratap Singh

Comment "System Design", I will dm you this for Free.

Follow!

LINKS

- https://lnkd.in/gTPiZV6X
- https://lnkd.in/ghBca63c
- https://lnkd.in/g6nW2bsv
- https://lnkd.in/gbVYyw7y
- https://lnkd.in/gv7aQpnC
- https://lnkd.in/gJQiJu5v
- https://lnkd.in/gSavU2ZT
- https://lnkd.in/g2nNVS-h
- https://lnkd.in/gyBnaX9T
- https://lnkd.in/gakQYxub
- https://lnkd.in/gXY5a-WD
- https://lnkd.in/gmtKRahf
- https://lnkd.in/gCm2YzWu
- https://lnkd.in/gjrBduDm
- https://lnkd.in/g9q9QhFk

#366 NeetCode

Other

Visit website

3mo · **View on LinkedIn**

24 Problems To Practice 24 Coding Interview Patterns

1) Two Pointers

→ Reverse String - https://lnkd.in/dtXn-yuM

2) Prefix Sums

→ Range Sum Query - Immutable - https://lnkd.in/dtmJnUGe

3) Sliding Window

→ Longest Substring Without Repeating Characters - https://lnkd.in/dWFuyQAV

4) Kadane's Algorithm

→ Maximum Subarray - https://lnkd.in/dJjW-hmE

5) Find Number of Subarrays

→ Subarray Sum Equals K - https://lnkd.in/dHFm9ueV

6) Fast and Slow Pointers

→ Linked List Cycle -https://lnkd.in/dwyPRNvx

7) Depth-First Search

→ Maximum Depth of a Binary Tree - https://lnkd.in/dR98gSiB

8) Breadth-First Search

→ Binary Tree Level Order Traversal - https://lnkd.in/dpCvXsGQ

9) Matrix Traversal

→ Number of Islands - https://lnkd.in/dbRdNk6r

10) Adjacency List BFS / DFS

→ Clone Graph - https://lnkd.in/dhexaMUy

11) Two Heaps

→ Find Median From Data Stream - https://lnkd.in/eapQYHpJ
12) Binary Search - Search Array
→ Find Minimum in Rotated Sorted Array - https://lnkd.in/dnEpEEeM
13) Topological Sort
→ Course Schedule - https://lnkd.in/ekU3qbrn
14) Top K
→ Top K Frequent Elements - https://lnkd.in/dW5yJpmq
15) Linked List Pointer Manipulation
→ Reverse Linked List - https://lnkd.in/dTD5a3DU
16) Permutations
→ Permutations - https://lnkd.in/eWYFt8UM
17) Combinations
→ Combinations - https://lnkd.in/dvEVeuum
18) Tree Maze (Backtracking)
→ Word Search - https://lnkd.in/dKNVRuxn
19) LCS
→ Longest Common Subsequence - https://lnkd.in/etnXAYWU
20) Monotonic Stack
→ Daily Temperatures - https://lnkd.in/eZXbTVyx
21) Memoization
→ Climbing Stairs - https://lnkd.in/dEpHKfW7
22) Tabulation
→ House Robber - https://lnkd.in/dWz_nHMQ
23) Multi-Source BFS
→ Rotting Oranges - https://lnkd.in/dK4DDsRi
24) Intervals
→ Merge Intervals - https://lnkd.in/dDN_BGKB
-
Follow NeetCode for more content like this.

LINKS

- https://neetcode.io/?utm_source=li&utm_medium=company-profile-page&utm_content=visit-website-button
- https://lnkd.in/dtXn-yuM
- https://lnkd.in/dtmJnUGe
- https://lnkd.in/dWFuyQAV
- https://lnkd.in/dJjW-hmE
- https://lnkd.in/dHFm9ueV
- https://lnkd.in/dwyPRNvx
- https://lnkd.in/dR98gSiB
- https://lnkd.in/dpCvXsGQ
- https://lnkd.in/dbRdNk6r
- https://lnkd.in/dhexaMUy
- https://lnkd.in/eapQYHpJ
- https://lnkd.in/dnEpEEeM
- https://lnkd.in/ekU3qbrn
- https://lnkd.in/dW5yJpmq
- https://lnkd.in/dTD5a3DU
- https://lnkd.in/eWYFt8UM
- https://lnkd.in/dvEVeuum
- https://lnkd.in/dKNVRuxn
- https://lnkd.in/etnXAYWU
- https://lnkd.in/eZXbTVyx
- https://lnkd.in/dEpHKfW7

- https://lnkd.in/dWz_nHMQ
- https://lnkd.in/dK4DDsRi
- https://lnkd.in/dDN_BGKB

#367 AlgoMaster.io

Other

Visit website | 10 Must-Know Graph Algorithms for Interviews: | 1. Depth-First Search: https://lnkd.in/gQUpRGdk | 2. Breadth-First Search: https://lnkd.in/gFnJTxfH | 3. Topological Sort: https://lnkd.in/gqjyQU4H | 4. Union Find / DSU: https://lnkd.in/gdPiKc7q | 5. Cycle Detection: https://lnkd.in/gbJrZANe | 6. Connected Components: https://lnkd.in/gdFMuUhn | 7. Bipartite Check: https://lnkd.in/gru3tYDY | 8. Flood Fill: https://lnkd.in/gtSzgrXB | 9. Minimum Spanning Tree: https://lnkd.in/ge-BZAKZ | 10. Shortest Path: https://lnkd.in/ggGCX35B | Repost to help others in your network | 539 | 1 | 50 | | Ramalingam N, Open to work Verified Profile 3rd+ | Ramalingam N | Backend Engineer | Java 17, Spring Boot, Kafka, Microservices | Azure integrations @ Accenture | Event-Driven Systems, REST APIs

3mo · **View on LinkedIn**

Quick and useful

MEDIA advertisement

LINKS

- https://algomaster.io/
- https://lnkd.in/gQUpRGdk
- https://lnkd.in/gFnJTxfH
- https://lnkd.in/gqjyQU4H
- https://lnkd.in/gdPiKc7q
- https://lnkd.in/gbJrZANe
- https://lnkd.in/gdFMuUhn
- https://lnkd.in/gru3tYDY
- https://lnkd.in/gtSzgrXB
- https://lnkd.in/ge-BZAKZ
- https://lnkd.in/ggGCX35B

#368 Ashish Pratap Singh

Other

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

3mo · **View on LinkedIn**

DSA was HARD until I Learned these 15 Patterns:

1. Two Pointers: https://lnkd.in/gAs6WZ6X
2. Sliding Window: https://lnkd.in/gVqnKNdg
3. Binary Search: https://lnkd.in/gc3HzSgh
4. Frequency Counting: https://lnkd.in/gXzrFVg3
5. Matrix Traversal: https://lnkd.in/gTzGdT93
6. Monotonic Stack: https://lnkd.in/g2ckwXTW
7. Prefix Sum: https://lnkd.in/g2EXKs32
8. Overlapping Intervals: https://lnkd.in/gzzfpkkq
9. Greedy: https://lnkd.in/guCMb8k4
10. Top ‘K’ Elements: https://lnkd.in/gFPPYc6w
11. Backtracking: https://lnkd.in/gUc28SU6
12. Binary Tree Traversal: https://lnkd.in/gWpgwvGc
13. Depth-First Search (DFS): https://lnkd.in/gqvJNYE6
14. Breadth-First Search (BFS): https://lnkd.in/gJueWjk2
15. Dynamic Programming Patterns: https://lnkd.in/gmi-Ji8K

Repost to help others in your network

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/gAs6WZ6X
- https://lnkd.in/gVqnKNdg
- https://lnkd.in/gc3HzSgh
- https://lnkd.in/gXzrFVg3
- https://lnkd.in/gTzGdT93
- https://lnkd.in/g2ckwXTW
- https://lnkd.in/g2EXKs32
- https://lnkd.in/gzzfpkkq
- https://lnkd.in/guCMb8k4
- https://lnkd.in/gFPPYc6w
- https://lnkd.in/gUc28SU6
- https://lnkd.in/gWpgwvGc
- https://lnkd.in/gqvJNYE6
- https://lnkd.in/gJueWjk2
- https://lnkd.in/gmi-Ji8K

#369 Sathish Kumar Subramani

Kafka / Messaging

Senior Engineering Manager | Generative AI | Azure Cloud Solutions | Leadership Excellence | Driving Operational Efficiency

3mo · **View on LinkedIn**

Can You Draw Kafka's Architecture on a Whiteboard From Memory?

If someone asked you to explain Kafka's 9 core concepts in a job interview, could you connect them without hesitation?

Most engineers use Kafka daily. Fewer can explain why it works the way it does.

How does data get in?

1. Producer: Publishes records to topics. Records append to partitions across brokers. The producer decides : directly or via a key which partition each record lands in. This routing decision determines ordering guarantees downstream.

2. Topic: A named, append-only log. The logical channel producers write to and consumers subscribe to. "Orders," "events," "payments." Not a queue you drain, a log you read from.

3. Partition: Topics split into partitions for parallelism. Each partition is an ordered sequence distributed across brokers. More partitions, more parallel processing. This is the mechanical reason Kafka handles millions of events per second.

How does data get out?

4. Consumer : Reads records from topics at its own pace. Kafka retains records, so reading doesn't delete. A consumer can be fast or slow without affecting anyone else.

5. Consumer Group: Multiple consumers sharing the workload. Each partition assigned to exactly one consumer in the group. Add consumers to scale throughput. This is horizontal scaling for reads and the reason Kafka handles growing demand without redesign.

6. Offset: A unique position marker within a partition. Tracks the next record to read. How a consumer knows where it left off and resumes after restart without missing or duplicating records.

How does data stay safe?

7. Broker: A Kafka server storing data and serving clients. Producers write to it. Consumers read from it. A cluster is just multiple brokers working together and that togetherness is what provides resilience.

8. Replication: Partitions copied across brokers. A leader handles reads and writes. Replicas stay in sync, ready to take over. A broker dies, a replica is promoted, no data lost. Durability without manual intervention.

9. Retention: How long records are kept before deletion. Older records expire. Newer ones stay available. This is what makes Kafka both a real-time stream and a short-term replayable store, you can go back and re-read what happened yesterday.

How do all 9 connect?

Producers write to topics.

Topics are split into partitions across brokers.

Consumers organized into groups read from partitions tracked by offsets.

Replication keeps data safe across broker failures.

Retention manages the lifecycle.

Nine concepts.

One clean architecture.

Brokers store it, topics organize it, partitions parallelize it, producers and consumers move it, groups scale it, offsets track it, replication protects it, retention manages it.

Which concept took you the longest to fully understand?

Repost this to help your network get started

+ Follow Sathish for more

#370 Java Developers Community (moderated)

Java / Spring

Palmurugan C • 3rd+

3mo · View on LinkedIn

Spring Boot 4.1 just made Redis Pub/Sub a lot more “Springy”!

One of the features that caught my attention is @RedisListener — a cleaner, annotation-driven way to subscribe to Redis channels.

Before 4.1, integrating Redis Pub/Sub often meant:

✗ RedisMessageListenerContainer

✗ Manual listener adapters

✗ Verbose configuration

✗ More plumbing than business logic

With @RedisListener:

✓ Just annotate a method

✓ Auto-configuration handles the setup

✓ Cleaner and more readable code

✓ Better alignment with Spring’s programming model

My take

This is a small feature, but it significantly improves developer experience — especially for teams building reactive, event-driven, and cloud-native applications.

I’ve summarized the Why / When / Problem / Solution in the poster below ↓

#SpringBoot #Redis #Java #Microservices #EventDrivenArchitecture #WebFlux #Kubernetes #BackendDevelopment #SoftwareArchitecture #codewithpal

— Palmurugan Chandran (@palmuruganc)

#371 Java Developers Community (moderated)

Kafka / Messaging

Vladislav Grigorev • 3rd+

3mo · View on LinkedIn

You did not need Kafka. You needed a job queue.

I like Kafka.↕

But “we need async processing” does not always mean “we need Kafka”.

Sometimes you just need a worker to do a task later.

Example:
A user uploads a CSV file.
The system needs to:
• validate rows
• import data
• send notifications
• retry failed rows
• show progress in UI
This is not always an event streaming problem.
This is often a job processing problem.
You need:
• job status
• retry attempts
• max retries
• error reason
• worker ownership
• visibility timeout
• monitoring for stuck jobs
Kafka can be used here.
But it is not automatically the best fit.

Kafka makes sense when you need an event log.
For example:
• multiple services need the same event
• consumers process events independently
• replay matters
• ordering by key matters
• high throughput streaming is important
OrderCreated is published.
Payment, analytics, billing, and notifications can all consume it separately.
That is a good Kafka use case.

A job queue is different.
A job is work to be completed.
Usually one worker should take it.
If the worker dies, another worker should retry it.
If it fails too many times, it should be marked as failed or moved to DLQ.
Examples:
• send email
• generate PDF
• resize image
• retry webhook
• import file
• run scheduled cleanup
For many of these, Postgres, Redis, RabbitMQ, SQS, or another queue is enough.
The important part is the model.
Not the brand name of the tool.

Simple rule:

If it is a fact that happened, think event stream.

If it is work that must be completed, think job queue.

Kafka is powerful.

But power is not the same as fit.

Sometimes the better engineering decision is the boring queue everyone can understand, monitor, and fix at 2 AM.

Have you seen Kafka used where a job queue would have been enough?↓

#backend #java #kafka #microservices #systemdesign #softwareengineering #architecture #springboot

#372 Rajnikant Pandey

AI / LLM

AI Architect & Technical Leader | Staff/Principal AI & Full-Stack Engineer | Enterprise GenAI & Agentic AI | AI Platform Strategy & Architecture | LLM • RAG • MCP • LangGraph | Distributed Cloud-Native Systems

3mo · View on LinkedIn

Harness Engineering: The Missing Layer Between LLMs and Production AI

Building a great AI application is no longer just about choosing the best LLM. The real challenge is harnessing models, tools, data, memory, and evaluation into a reliable production system. This is where Harness Engineering comes in.

Instead of asking: "Which model should we use?" engineering teams are asking: "How do we orchestrate multiple models, tools, guardrails, and feedback loops to consistently deliver the right answer?"

A production AI harness typically includes:

✓ Intent Detection & Query Classification

✓ Adaptive Model Routing (cost vs latency vs quality)

✓ Context Retrieval (Hybrid RAG / Graph RAG / MCP)

✓ Tool & Agent Orchestration

✓ Prompt & Context Optimization

✓ Memory Management

✓ Guardrails & Policy Enforcement

✓ Structured Outputs & Validation

✓ LLM-as-a-Judge Evaluation

✓ Observability & Tracing

✓ Human Feedback Loop

✓ Continuous Evaluation & Improvement

The result isn't just a chatbot. It's an AI Operating System capable of routing every request through the most appropriate execution path. Future AI platforms won't compete on who has the biggest model. They'll compete on who builds the smartest AI harness. The LLM becomes just one component. The harness becomes the product.

#373 Rakesh Gohel

AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

3mo · View on LinkedIn

Prompt engineering gets the headlines.

Loop engineering is what keeps AI agents running on their own..

Most teams fund the first and wonder why nothing reaches production.
Here is the four-layer stack that actually ships AI agents.
Layer 1 - Prompt Engineering
The message you send the AI.
Who it should act as, what you want, an example, the format.
One good prompt → one good answer. Most teams stop here.
Layer 2 - Context Engineering
What the AI gets to see.
You decide what goes in: the question, documents, past messages, tool results.
Keep what matters. Drop what does not. A perfect prompt inside a bad context still fails.
Layer 3 - Harness Engineering
The machine around it all. It gathers information, lets the AI act, then checks the work.
Wrong answer? It tries again until it passes.
This is what separates a demo from a product.
Layer 4 - Loop Engineering
The layer most teams have never heard of.
A loop is not a prompt.
It is a cycle the agent runs on its own. Five steps, repeating until the work is done:
1. Discovery - scans for what it needs to know
2. Planning - maps steps before acting
3. Execution - acts, calls tools, writes output
4. Verification - quality gate: tests, linters, CI/CD
5. Iteration - fails? Tries again. No human required.
Open loops burn tokens and drift. Closed loops catch their own mistakes.
The Maturity Scale:
Prompter → Operator → Loop Engineer → System Architect
Most teams are stuck at Operator. The ones shipping in production are Loop Engineers.
These four layers are not steps in a sequence. They are nested. Pull one out and the others collapse.
When your AI pilot dies in production, the model was not the problem. One of these layers was never built.
If the answer is a person → you have a prompt. If the answer is a system → you have a product.
For more AI agent insights, subscribe to newsletter: https://lnkd.in/gatGAt9X
Grab your copy → The Human-Agent Orchestrator: https://amzn.to/4gEIzmv
Follow for weekly AI agent insights Repost to share this with your team. Save before your next AI build
#AIAgents #PromptEngineering #ContextEngineering #HarnessEngineering #LoopEngineering

MEDIA The-difference-between-a-Prompter-and-a-Loop-Engineer for AI Agents; document/carousel (possible)

LINKS
- https://lnkd.in/gatGAt9X
- https://amzn.to/4gEIzmv

#374 Sumit Srivastava

Other

Software Engineer |Marketing |AI |Tech |Open to Collab | Founder - “Tycoons in Trade”| Helping businesses to grow with premium trade solutions

3mo · **View on LinkedIn**

Most people learn DSA by solving random coding questions.

That works...

Until you face a real coding interview.

Then you realize interviews don't test whether you've seen the exact question before.

They test whether you can identify the pattern behind the problem.

That's why mastering coding patterns is one of the fastest ways to improve your problem-solving skills.

Here are some of the most important DSA patterns covered in technical interviews:

1. Sliding Window & Two Pointers

Common questions:

• Longest Substring Problems • Maximum Sum Subarray • Pair Sum Problems • Window-Based Optimization

These patterns help you solve array and string problems efficiently without using brute force.

2. Linked List Patterns

Interview favorites include:

• Detect a Cycle • Find the Middle Node • Reverse a Linked List • Rotate a Linked List

These questions strengthen your understanding of:

Fast & Slow Pointers

In-Place Reversal

Pointer Manipulation

3. Tree & Graph Traversal

Almost every coding interview includes traversal problems.

Examples:

• Binary Tree Level Order Traversal • Path Sum Problems • Number of Islands • Flood Fill

You should be comfortable with:

Breadth-First Search (BFS)

Depth-First Search (DFS)

Matrix Traversal

Graph Traversal

4. Heap-Based Problems

Companies often test your ability to process large datasets efficiently.

Common questions:

• K Closest Elements • Kth Largest Element • Median of a Data Stream • Top K Frequent Elements

These require strong knowledge of:

Priority Queues

Min Heap

Max Heap

Two Heaps Pattern

5. Binary Search & Sorting Patterns

Simple-looking questions often test optimization skills.

Examples:

• Search in Rotated Array • Bitonic Array Problems • Missing Numbers • Duplicate Detection

Key concepts include:

Modified Binary Search

Cyclic Sort

Bit Manipulation

6. Dynamic Programming

This is where many candidates struggle.

Popular questions:

• 0/1 Knapsack • House Thief • Staircase • Minimum Subset Difference • Longest Common Subsequence

These problems teach you how to:

Break large problems into smaller ones

Reuse previous computations

Optimize time complexity

7. Advanced Problem-Solving Patterns

Top companies frequently ask questions based on:

• Merge Intervals • Topological Sort • K-Way Merge • Subsets & Permutations • Palindromic Subsequences

Recognizing these patterns can dramatically reduce solving time.

8. Coding Interviews Test Pattern Recognition, Not Memorization

The goal isn't to solve thousands of questions.

The goal is to understand:

• How to recognize patterns quickly • How to choose the right algorithm • How to optimize solutions • How to handle edge cases • How to communicate your approach clearly

That's what separates a DSA beginner from a strong problem solver.

Master the patterns.

Practice the applications.

Understand the logic.

That's how you crack coding interviews.

Follow Sumit Srivastava for more such content.

Comment "DSA" and I'll share the complete DSA Coding Patterns PDF.

DSA

.

23 pages

MEDIA document/carousel (possible)

#375 AlgoMaster.io

Other

Visit website

3mo · **View on LinkedIn**

13 Must-Know Data Structures for Coding Interviews:

1. Array
2. Linked List
3. Stack
4. HashMap
5. Matrix
6. Queue
7. Deque
8. Binary Tree
9. Binary Search Tree (BST)
10. Heap (Priority Queue)
11. Trie

12. Graph

13. Union Find

If you want to master coding interviews fast, check out algomaster.io for visual and pattern-based resources.

Repost to help others in your network

LINKS

- https://algomaster.io/
- http://algomaster.io

#376 AKASH BHATNAGAR

AI / LLM

Consultant Software Engineer Tech Lead & Solutions Architect AI & Cloud Specialist ML & Data Science .NET Core | GoLang | Angular | Python | Kubernetes | Docker | AWS/Azure | Microservices | DevOps

3mo · **View on LinkedIn**

MCP vs RAG vs AI Agents

• MCP → standard way for AI models to connect with external tools, APIs, and data sources in a consistent, structured format. It removes the need for custom integrations, it acts as USB-C port for AI tools

• RAG → Makes LLMs accurate by retrieving real-time knowledge

• AI Agents → Autonomous systems that plan, reason & execute goals

Three powerful ways to build smarter AI

#AI #RAG #AgenticAI #MCP #GenerativeAI #ArtificialIntelligence #Tech

LINKS

- https://www.linkedin.com/feed/update/urn:li:activity:7474910539798700032

ALSO SAVED AS Umair khawas (3mo) link

#377 Ashish Pratap Singh

System Design

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

3mo · **View on LinkedIn**

Top 6 Load Balancing Algorithms You Should Know:

1. Round Robin

- Requests are sent to servers one by one in order.

- Best for: simple stateless services with similar servers.

2. Weighted Round Robin

- Each server gets traffic based on its assigned weight (CPU, memory etc.).

- Best for: clusters with mixed server sizes.

3. Least Connections

- Traffic goes to the server with the fewest active connections. A server handling many long-running requests will receive less new traffic.

- Best for: APIs, databases, WebSockets, and long-lived connections.

4. Least Response Time

- Traffic goes to the server with the lowest response time, often combined with active connection count.

- Best for: latency-sensitive systems where server performance changes over time.

5. IP Hash

- The load balancer hashes the client IP address and routes that client to the same server.

- Best for: systems where a user should consistently hit the same backend server.

6. Consistent Hashing

- Both servers and keys are mapped onto a hash ring. When a server is added or removed, only a small portion of keys move to a different server.

- Best for: systems where minimizing remapping matters.

Which of these algorithms have you used in your work? Let me know in the comments.

Repost to help others learn this

→ If you are preparing for interviews, checkout algomaster.io for pattern-based and visual learning resources.

LINKS

- https://algomaster.io/lnv
- http://algomaster.io

#378 Ashish Pratap Singh

Java / Spring

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

3mo · **View on LinkedIn**

If you want to master concurrency and multi-threading interviews, learn these 20 concepts:

1. Concurrency 101: https://lnkd.in/gbXNtx8i
2. Concurrency vs Parallelism: https://lnkd.in/ggm3zxKi
3. Processes vs Threads: https://lnkd.in/gjD7BE9D
4. Thread Lifecycle: https://lnkd.in/gnraugyb
5. Race Conditions: https://lnkd.in/gJBaBWa5
6. Mutex: https://lnkd.in/gU-6SvJh
7. Semaphores: https://lnkd.in/gpcMAvvC
8. Condition Variables: https://lnkd.in/gzD3RdWy
9. Coarse-grained vs Fine-grained Locking: https://lnkd.in/giHCvjbK
10. Reentrant Locks: https://lnkd.in/gjjtEZrm
11. Try-Lock/Timed-Lock: https://lnkd.in/gc9Frvik
12. Compare-and-Swap (CAS): https://lnkd.in/gzzvSW_W
13. Deadlock: https://lnkd.in/gKtDZ9gD
14. Livelock: https://lnkd.in/g8UTb86c
15. Signaling Pattern: https://lnkd.in/gg8RxZBf
16. Thread Pool Pattern: https://lnkd.in/g52TJUT3
17. Producer-Consumer Pattern: https://lnkd.in/g9P_Rh-6
18. Reader-Writer Pattern: https://lnkd.in/geAvZkRy
19. Thread-Safe Cache: https://lnkd.in/gPW9JnkH
20. Thread-Safe Blocking Queue: https://lnkd.in/gtKrDJiE

Repost to help others learn these concurrency concepts.

LINKS

- https://algomaster.io/lnv
- https://lnkd.in/gbXNtx8i
- https://lnkd.in/ggm3zxKi
- https://lnkd.in/gjD7BE9D
- https://lnkd.in/gnraugyb
- https://lnkd.in/gJBaBWa5
- https://lnkd.in/gU-6SvJh
- https://lnkd.in/gpcMAvvC
- https://lnkd.in/gzD3RdWy
- https://lnkd.in/giHCvjbK
- https://lnkd.in/gjjtEZrm

- https://lnkd.in/gc9Frvik
- https://lnkd.in/gzzvSW_W
- https://lnkd.in/gKtDZ9gD
- https://lnkd.in/g8UTb86c
- https://lnkd.in/gg8RxZBf
- https://lnkd.in/g52TJUT3
- https://lnkd.in/g9P_Rh-6
- https://lnkd.in/geAvZkRy
- https://lnkd.in/gPW9JnkH
- https://lnkd.in/gtKrDJiE
- https://www.linkedin.com/feed/update/urn:li:activity:7452208173014253568

ALSO SAVED AS Ashish Pratap Singh (6mo) link

#379 Ganesh Kumbhar

AI / LLM

Senior Staff Engineer | AI Application Architect | .NET & Azure | LLM Apps, RAG, Agentic AI, Azure OpenAI | Cloud & DevSecOps| Python | IIT Madras Pravartak — Agentic AI and Application Program

3mo · **View on LinkedIn**

AI Architect Interview Patterns Repo

I have started building a structured GitHub repo for engineers preparing for GenAI / RAG / Agentic AI / AI Architect / Solution Architect interviews.

The goal is simple:

Help engineers move from basic AI definitions to architect-level interview answers.

Many people know terms like RAG, Vector DB, AI Agents, Tool Calling, Fine-tuning, MCP, Semantic Kernel, LangChain, etc.

But in interviews, the real challenge is explaining:

? When to use what?

? When NOT to use it?

? What are the tradeoffs?

? What can go wrong in production?

? How do you explain it like a senior engineer or architect?

Having sat on both sides of the interview table — as a candidate answering architecture questions and as an interviewer evaluating them — I have seen one common gap:

Many engineers know the concepts, but struggle to explain the tradeoffs clearly.

So I am building this repo step by step.

GitHub Repo:

https://lnkd.in/dsnqAsy3

What will this repo cover?

✓ Agentic AI Fundamentals

AI Agent vs Workflow vs Chatbot, Tool Calling, Agent Memory, Single vs Multi-Agent, Human-in-the-loop

✓ RAG System Design

RAG basics, chunking, metadata filtering, tenant isolation, top-k issues, hallucination reduction

✓ AI System Design Tradeoffs

Cost, latency, accuracy, P95/P99, fallbacks, retries, observability, production failure points

✓ Enterprise GenAI Architecture

RBAC, PII handling, audit logs, model selection, Azure OpenAI, Azure AI Search, Semantic Kernel vs LangChain

✓ Interview Answer Frameworks

Agentic AI system design, enterprise document Q&A, AI support assistant, invoice/expense AI agent, measuring AI quality

Each topic will include:

1. Interview question
2. Why interviewer asks this
3. Basic answer
4. Architect-level answer
5. Must mention points
6. Real-world example
7. Common mistake
8. Memory formula

I will keep updating this repo with latest AI topics and trends useful for both AI learners and interview preparation.

As AI evolves, the repo will also evolve.

If you are learning GenAI or preparing for AI Architect / Solution Architect interviews, follow along.

If this helps you, please give it a ★ on GitHub and share it with someone preparing for GenAI interviews.

https://lnkd.in/dsnqAsy3

#GenAI #AgenticAI #RAG #AIAgents #AISystemDesign #AIArchitecture #AIArchitect #SolutionArchitect #InterviewPreparation

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/dsnqAsy3

#380 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

3mo · **View on LinkedIn**

Arguably the 20 most influential books for software engineers.

Most of us spent years on a CS degree and were assigned almost none of them.

1 The Pragmatic Programmer

2 Designing Data-Intensive Applications

3 Clean Code

4 The Mythical Man-Month

5 Refactoring: Improving the Design of Existing Code

6 Working Effectively with Legacy Code

7 Software Architecture: The Hard Parts

8 Database Internals

9 Staff Engineer: Leadership Beyond the Management Track

10 Extreme Ownership: How U.S. Navy SEALs Lead and Win

11 A Philosophy of Software Design

12 Why Programs Fail: A Guide to Systematic Debugging

13 Code Complete

14 Introduction to Algorithms

15 The C Programming Language

16 The Art of Computer Programming, Volume 1: Fundamental Algorithms

17 Building Microservices: Designing Fine-Grained Systems

18 Structure and Interpretation of Computer Programs
19 Peopleware: Productive Projects and Teams
20 Design Patterns: Elements of Reusable Object-Oriented Software
How many have you read?
===
PS - Want my system design playbook (for free)?
Join my newsletter with 200K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Save & repost to help others become better software engineers.
Follow Neo Kim + turn on notifications.

MEDIA 20 powerful books for software engineers

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#381 Rakesh Gohel AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

3mo · **View on LinkedIn**

Most enterprises talk about responsible AI. Santander shipped it.
Their open-source stack shows exactly how AI agents get built safely...
Not a demo. Real tooling for the hard parts of banking.
Banco Santander just open-sourced 13 repositories from their AI Labs fraud graph generation, LLM guardrails, governance frameworks for high-stakes model decisions, RAG alignment, discrimination testing. Synthetic and anonymised data only. No customer information leaves the building.
They stood up an Open Source Programme Office and a two-track review to ship it responsibly.
A bank did this. On purpose.
What they shipped
1. gen-fraud-graph : Scales past 100M accounts for fraud detection benchmarking
2. mech-gov-framework : Risk tiers, hard gates, entropy controls for regulated LLM decisions
3. autoguardrails : Fast iteration on guardrail policy
4. llm_bridge : One interface for OpenAI, AWS Bedrock, Google Gemini
5. linear-adapter-trainer : Aligns retrieval embeddings with your actual queries
6. mutatis-mutandis : Counterfactual comparators for AI discrimination testing
7. Sota-stressed-datasets : Stressed benchmarks to test model resilience
The argument for every bank
The models you train on your own data stay proprietary. That is the edge. Nobody is asking you to give it away.
The scaffolding around them is shared cost fraud graph generators, evaluation harnesses, and anonymisation toolkits. Right now every bank pays for it alone.
AI agents are moving into credit decisions, fraud flagging, compliance review.
The governance layer is not a differentiator. It is infrastructure. Build it once, in the open.
Santander ran this play. Others should follow.
What would it take for more banks to open-source their AI scaffolding? ↓

For more AI agent insights, subscribe to newsletter: https://lnkd.in/gatGAt9X

Grab your copy → The Human-Agent Orchestrator: https://amzn.to/3QBQRkI

Repost if open-source AI infrastructure is matters to you. Save the governance framework alone is worth studying.

#AIAgents #ResponsibleAI #OpenSource #FinancialServices #LLM #AIGovernance #BankingAI #Santander

MEDIA 13-GitHub-Repos-Fraud-Detection-to-LLM-Governance for AI Agents

LINKS

- https://lnkd.in/gatGAt9X
- https://amzn.to/3QBQRkI

#382 Bapurao Patil

AI / LLM

Software Engineer at Photon

3mo · **View on LinkedIn**

Interview Insight: Beyond RAG – Understanding KAG & CAG

Today, during an AI/ML interview, the discussion went far beyond the traditional RAG (Retrieval-Augmented Generation) architecture. The panel explored how modern GenAI systems are evolving with KAG (Knowledge-Augmented Generation) and CAG (Cache-Augmented Generation).

RAG – Retrieves relevant information from external sources before generating a response.

KAG – Uses structured knowledge graphs and relationships, enabling deeper reasoning, better context understanding, and higher accuracy.

CAG – Reuses cached context and previously processed responses to reduce latency, improve efficiency, and lower operational costs.

Key Takeaway:

RAG fetches what it needs.

KAG understands what it knows.

CAG reuses what it has.

As AI systems move toward production-scale deployments, understanding these architectures is becoming essential for AI/ML Engineers, Data Scientists, and GenAI Developers.

Every interview teaches something new. Today's lesson: GenAI is no longer just about prompts and RAG; it's about building intelligent, scalable, and efficient knowledge systems.

#AI #MachineLearning #GenerativeAI #LLM #RAG #KAG #CAG #KnowledgeGraphs #DataScience #MLEngineer #AIEngineer #InterviewExperience #ArtificialIntelligence #TechLearning #CareerGrowth

#383 Sherry Jasal

AI / LLM

AI Engineer | I teach engineers and help companies build AI that survives production | Founder @TheSherryCode | 11+ yrs Data & AI

3mo · **View on LinkedIn**

Top 6 RAG architectures every AI engineer should know

Most teams ship one RAG pattern and blame the model when it breaks.

The model is rarely the problem. The retrieval design is.

RAG = RETRIEVAL + GENERATION

→ Reduces hallucinations

→ Grounds answers in real data

→ Adds knowledge without retraining

Here are 6 architectures, simplest to most autonomous:

1. Simple RAG

Retrieve top-k chunks from a vector store, then generate.

Use cases: FAQ bots, internal knowledge assistants, support.

2. Hybrid RAG

Combines semantic and keyword search, then reranks the results.

Use cases: enterprise search over messy or technical docs where pure vector search misses exact terms.

3. Corrective RAG (CRAG)

Scores retrieved content for relevance and triggers a fallback search when it is weak.

Use cases: medical, legal, financial. Anywhere a wrong answer is expensive.

4. Self-RAG

The model decides when to retrieve and critiques its own output before answering.

Use cases: technical documentation, deep research, exploratory writing.

5. Graph RAG

Retrieves over a knowledge graph of entities and relationships, not just isolated chunks.

Use cases: scientific discovery, legal reasoning, multi-hop questions where connections matter.

6. Agentic RAG

An agent routes queries, runs multi-step retrieval, and validates across sources and APIs.

Use cases: automated research, market intel, executive dashboards.

The trap is treating these as a ranking.

They are a decision tree.

Start with Simple RAG. Move to Hybrid the moment exact-match terms get missed. Add Corrective or Self-RAG when accuracy is non-negotiable. Reach for Graph or Agentic RAG only when the questions are genuinely multi-step.

Complexity should follow the problem, not the trend.

Which of these are you running in production, and where did it start to break?

I'm breaking the full RAG pipeline down, layer by layer, in my newsletter series The AI Engineering Stack. Retrieval Layer eval drops this week. Link in comments.

MEDIA document/carousel (possible)

#384 Alok Sharan

AI / LLM

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author

3mo · **View on LinkedIn**

After building and reviewing multiple AI systems, one pattern shows up every time:

The model is rarely the problem.

The retrieval layer is.

Most teams spend weeks comparing LLMs...

and minutes deciding where their data lives.

That decision?

Vector databases.

If you’re building RAG systems, AI agents, or semantic search...

this layer decides whether your AI feels intelligent or broken.

Pick the wrong one → slow retrieval, irrelevant answers, weak outputs.

Pick the right one → fast, precise, production-ready systems.

Here’s a breakdown of 15 vector databases every AI builder should know ↓

Pinecone / Weaviate / Qdrant
Built for production-grade semantic search and scalable retrieval.
Milvus / FAISS
High-performance engines for handling massive embedding workloads.
Chroma
Great for local development, prototyping, and quick RAG setups.
Redis Vector / Elasticsearch / OpenSearch
Perfect when you want vector search + existing infra (caching, search, analytics).
LanceDB / pgvector
Developer-friendly options for local workflows or extending SQL databases.
Vespa / SingleStore
Real-time systems combining search, ranking, and analytics at scale.
MongoDB Atlas Vector Search / Astra DB
Cloud-native solutions integrating vector search into operational databases.
What actually matters when choosing:
→ Latency and retrieval speed
→ Scalability with embeddings
→ Filtering + hybrid search support
→ Ease of integration with your stack
Because at the end of the day:
Your AI is only as good as the context it retrieves.
Which one are you currently using or planning to try? ↓

#385 Java Developers Community (moderated)

Java / Spring

Naveengandhi C. • 3rd+
3mo · View on LinkedIn

Java 21 shipped with 15 JEPs.
Most of the attention went to virtual threads.
But one feature changed how I model domain logic more than I expected:
Pattern matching for `switch`.
Before Java 21, type-based logic often ended up as an `instanceof` chain:

```
if (txn instanceof CardTransaction card) {
return processCard(card.network(), card.last4());
} else if (txn instanceof BankTransfer bank) {
return processBank(bank.routingNumber());
} else if (txn instanceof CryptoTransaction crypto) {
return processCrypto(crypto.walletAddress());
}
```

It works, but the compiler cannot confirm that every possible transaction type is handled.
Add a new subtype, forget to update this code, and the bug may only appear at runtime.
Java 21 makes it cleaner:

```
String result = switch (txn) {
case CardTransaction card ->
processCard(card.network(), card.last4());
case BankTransfer bank ->
```

processBank(bank.routingNumber());
case CryptoTransaction crypto ->
processCrypto(crypto.walletAddress());
};
The casts disappear. The switch returns a value. No mutable result variable. No `break`.
The real value appears when this is combined with sealed types.
sealed interface Transaction
permits CardTransaction,
BankTransfer,
CryptoTransaction {}
Now the domain defines a closed set of valid variants.
The compiler knows every permitted subtype and can check the switch for exhaustiveness.
Add another transaction type and forget to handle it?
Compile-time error.
Guarded patterns make the logic even clearer:
BigDecimal fee = switch (txn) {
case CardTransaction card
when card.network().equals("AMEX") ->
PREMIUM_CARD_FEE;
case CardTransaction card ->
STANDARD_CARD_FEE;
case BankTransfer bank ->
BANK_TRANSFER_FEE;
case CryptoTransaction crypto
when crypto.confirmations() < 6 ->
PENDING_CRYPTO_FEE;
case CryptoTransaction crypto ->
SETTLED_CRYPTO_FEE;
};
The `when` clause keeps the condition with the pattern instead of hiding it inside another `if`.
Java 21 also allows explicit null handling:
String label = switch (txn) {
case null -> "NO_TRANSACTION";
case CardTransaction card -> "CARD";
case BankTransfer bank -> "BANK";
case CryptoTransaction crypto -> "CRYPTO";
};
Without `case null`, a null value still throws `NullPointerException`. The difference is that null can now be handled intentionally inside the switch.
Pattern matching for `switch` is not just cleaner syntax.
Combined with sealed types, it gives the compiler ownership of the domain contract.
Records represent the variants.
Sealed types define what is allowed.
Switch expressions make every outcome explicit.
That is more than readability.
It is a correctness guarantee.

If you are on Java 21 and still using long `instanceof` chains for closed hierarchies, this is one of the most valuable refactors available.

Which Java 21 feature has made the biggest difference in your codebase?

MEDIA advertisement

LINKS

- http://card.network

#386 Prasad Rao

AI / LLM

Principal Solutions Architect at AWS | I help people excel in their Cloud Career Journeys

3mo · View on LinkedIn

I shared a 6-week Claude Architect certification roadmap I created for myself couple of months ago.

It got good traction. People saved it. A few messaged me saying they'd started following it.

Here it is - https://lnkd.in/eXdGXKUP

I never followed it myself.

Every week I'd open the roadmap, look at Week 1, and close my laptop.

Something more urgent always won. The plan sat untouched for two months.

Then a two-week window opened up where my workload eased off.

I compressed the entire six-week plan into those two weeks and passed the exam.

Here's what I actually did:

1/ 5 Skilljar courses (not all 15+)

https://lnkd.in/eZw9VB4g

Anthropic Academy has a lot of free courses. I picked five that covered the core ground:

→ Building with the Claude API

→ Claude Code in Action

→ Introduction to Model Context Protocol

→ Introduction to Agent Skills

→ Introduction to Sub Agents

2/ Official practice test, cold: 700

I sat the exam with no additional prep after the courses. Passing score is 720. That one result told me exactly where my gaps were.

3/ The book, cover to cover

https://lnkd.in/e45AJ45Q

Architecting Agentic Solutions with Claude by Vladimir Provorov.

I'd recommend this even if you're not sitting any exam. It's genuinely useful for understanding architectural principles that apply to any production LLM system.

4/ Official practice test, second attempt: 950+

That jump from 700 to 950+ made me confident enough to take the real exam.

One honest caveat: I come from a Solution Architecture background, so two weeks worked for me.

If you're newer to agentic concepts or can only carve out a few hours a week, the original six-week plan is the right target. Calibrate to your own starting point.

The gap between "the plan" and "the execution" is often more useful than the plan itself. This is mine.

Publishing a full deep-dive in my newsletter on Thursday. Subscribe here - https://lnkd.in/eX6cstvS

Repost to help others find this post

+ Follow Prasad Rao to excel in your cloud and AI career

LINKS

- https://lnkd.in/eXdGXKUP
- https://lnkd.in/eZw9VB4g
- https://lnkd.in/e45AJ45Q
- https://lnkd.in/eX6cstvS

#387 Alex Xu AI / LLM

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

3mo · **View on LinkedIn**

RAG vs Graph RAG vs Agentic RAG

RAG connects LLMs to your data and there are three different ways to do it.

Standard RAG

- The query is converted into an embedding and matched against a vector database.

- The top-K closest chunks are pulled out and passed to the LLM as context.

- The LLM writes a grounded answer using only what was retrieved.

Graph RAG

- The query is classified: specific questions route to local search, broad questions route to global search.

- Local search: query embedded → vector DB finds matching entities → pipeline traverses across the knowledge graph collecting linked context → LLM synthesis final answer.

- Global search: no vector search, no graph traversal → community reports loaded in batches → LLM scores each for relevance → top-ranked context → LLM synthesizes final response.

Agentic RAG

- A reasoning agent reads the query, breaks it into sub-questions and picks the sources.

- The context across multiple sources is retrieved, depending on the sub-query.

- Another agent checks whether the retrieved context answers the question. If not, it re-retrieves.

- Once satisfied, the final answer is synthesized by LLM based on the prompt.

Standard RAG is fast and cheap but if the wrong chunk is retrieved, the answer is wrong and nothing catches it.Use it when the answer lives in your documents and speed matters.

Graph RAG is expensive to build and slow to update. Use it for structured knowledge like legal, compliance, or biomedical data.

Agentic RAG is more capable and flexible but slower, expensive, and harder to debug. Use it when the question needs multi-step reasoning and self-correction.

Over to you: Which of these are you running in production?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gF7Vsw6X

#systemdesign #coding #interviewtips

MEDIA document/carousel (possible)

LINKS

- https://bit.ly/bbg-a-li-profile

• https://lnkd.in/gF7Vsw6X

#388 Java Developers Community (moderated)

System Design

Palmurugan C • 3rd+

3mo · **View on LinkedIn**

How we removed a synchronous dependency in an e-commerce system

Imagine an e-commerce application with these services:

Order Service

Customer Service

Shipping Service

Initial Design

When an order is created:

Order Service publishes an OrderCreated event.

Shipping Service consumes the event.

Shipping Service needs the customer's address.

It makes a REST call to Customer Service.

Shipping Service --> REST --> Customer Service

This works, but creates a dependency:

✗ Shipping cannot work if Customer Service is slow or unavailable.

✗ Increased latency due to network calls.

✗ More service-to-service coupling.

Improved Design: Event-Carried State Transfer

Instead of querying Customer Service every time:

Customer Service publishes customer updates as events.

Shipping Service maintains a local copy of customer data.

When OrderCreated arrives, Shipping Service already has the customer information.

Customer Service --> Kafka --> Shipping Service

Now:

✓ No synchronous REST call

✓ Lower latency

✓ Better resilience

✓ Services are more autonomous

✓ Fully event-driven workflow

Key Lesson

Events are not only for notifications.

They can also be used to transfer state between services.

This pattern is called "Event-Carried State Transfer" and is commonly used in large-scale microservice architectures to reduce service coupling and improve reliability.

Have you used local data replication instead of service-to-service REST calls in your microservices?

#Microservices #Kafka #EventDrivenArchitecture #SystemDesign #DistributedSystems #SoftwareArchitecture #Java #Engineering #codewithpal

#389 Anshika Sinha

AI / LLM

Engineering Leader@NetApp | Ex- Google | Gen & Agentic AI Platforms | Multi-Cloud Infrastructure | Organizational Scaling | Product & Platform Strategy

4mo · View on LinkedIn

I lost 3 senior-level offers from Amazon, DE Shaw, & Salesforce between TCS → Google...

But each rejection taught me what senior-level prep really means.

I was qualified for all the Engineering Lead positions, all offering 50+ LPA, & yet I failed.

And this is what actually went wrong (in DSA rounds itself)↓

At Amazon, it was a Graph Traversal problem with multiple edge cases.

→ I froze while explaining my optimization approach, not because I didn't know the logic, but because I couldn't structure my reasoning clearly under pressure.

At DE Shaw, it was a mix of Sliding Window and Binary Search.

→ I reached the brute-force solution quickly but took too long to identify the right pattern.

At Salesforce, it was a DP variation that required understanding overlapping subproblems, something I hadn't revised deeply enough.

Each of these interviews showed me the same truth → solving problems isn't the same as solving them live.

At senior levels, you're also judged on how you think, reason, & communicate.

Once I learned to:

✓ Explain trade-offs clearly,

✓ Talk through optimizations confidently,

✓ And stay calm while debugging live,

everything changed.

And that's exactly what structured mentorship can do for you too.

So if you're an experienced SDE like me aiming to grow beyond coding into leadership or senior roles, try EVOLVE by Bosscoder Academy, built esp. for SDEs with 6+ YOE.

Explore EVOLVE here: bcalinks.com/jvS5bOK

They've helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#career #growth #sdeprep #google #systemdesign #interviews

LINKS

- http://bcalinks.com/jvS5bOK

#390 Shivam Garg

System Design

SWE-III @ Google | IIT Ropar | 125K+ @Linkedin | AI for scaling businesses & Marketing | Hackathon Judge | Mentor| Technical Content Creator | DM for Collaboration

4mo · View on LinkedIn

In most SDE-3 interviews I've seen at Google, one thing becomes very obvious:

Candidates don't fail because they can't code.

They fail because their thinking doesn't shift from "solution mode" to "ownership mode" during the interview.

Let me break this down.

At SDE-2 level, interviews are still fairly structured:

You're given a problem, you identify a known pattern, and you implement a solution.

At SDE-3 level (especially in companies like Google), that structure quietly changes.

Now the evaluation starts much earlier than coding.

For example:

→ In DSA rounds, problems are intentionally open-ended

Before writing anything, you’re expected to define constraints, clarify assumptions, and even shape the problem statement

→ In system design rounds, correctness is no longer the goal

There are multiple valid designs , what matters is how clearly you can justify trade-offs like latency, cost, consistency, and scalability

→ Even follow-ups are different

They are not “harder questions” - they test whether your thinking stays stable when assumptions keep changing

One pattern I’ve noticed across many experienced candidates:

They jump to solutions too early because they are used to execution-heavy roles.

But SDE-3 interviews reward something different:

→ Problem framing first

→ Structured thinking under ambiguity

→ Clear justification of every decision

That’s the real gap.

Not experience. Not coding ability.

But how you think when nothing is fully defined.

And if you're an experienced SDE preparing for senior roles while balancing a full-time job, I'd highly recommend Bosscoder Academy's EVOLVE program, built specifically for SDEs with 6+ YOE.

Explore EVOLVE here: bcalinks.com/7fInPVc

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#collab #career #growth #seniorsde #jobswitch #tech #leadership

LINKS

- http://bcalinks.com/7fInPVc

#391 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · **View on LinkedIn**

Most people are using Claude like Google.

Become better at AI in just 1 minute a day.

Get the AI newsletter smart leaders read.

Sign up free now → aiforleaders.com

_____

Here’s the 7-day system to make Claude useful for real work:

Most people stay stuck at basic prompts.

They ask one-off questions.

They start from scratch every time.

They never build a repeatable setup.

Claude gets much better when you stop treating it like a search box.

Here’s the breakdown:
• Day One to Two: Stop using Chat for everything
Chat is fine for quick questions.
But it is not where the real work happens.
Use Claude like this:
→ Chat for quick answers
→ Projects for ongoing work
→ Cowork for deeper execution
↳ Most people never leave Chat, so they miss the best parts of Claude.
• Day Two to Three: Build your Claude OS
Create a simple folder structure.
Use:
→ ABOUT ME for your identity, tone, and writing rules
→ PROJECTS for active work
→ TEMPLATES for repeatable formats
→ OUTPUTS for finished drafts and deliverables
↳ Random prompting becomes a working system.
• Day Three to Four: Replace prompts with files
Stop rewriting your instructions every time.
Start with two files:
→ about-me.md
→ anti-ai-style.md
↳ These give Claude memory, taste, and boundaries before the task starts.
• Day Four to Five: Let Claude think with you
Stop giving Claude tiny tasks only.
Ask it to:
→ generate options
→ rank ideas
→ compare approaches
→ plan execution
↳ You make better calls when Claude does the first round of thinking.
• Day Five to Six: Add tools
Connect Claude to the places where your work already lives.
Start with:
→ Google Docs
→ Slack
→ Notion
↳ Claude becomes more useful when it can work with your real context.
• Day Six to Seven: Automate the repeatable work
Start small.
Use Claude to:
→ schedule recurring tasks
→ run repeat workflows
→ prepare work before you start your day
↳ The goal is to wake up with work already moving.
Avoid these mistakes:

✗ Asking random one-off questions forever

✗ Treating every prompt like a fresh start

✗ Using Chat when the work belongs in Projects or Cowork

Build one layer at a time.

Claude only becomes powerful when your system around it gets better.

Start with Day One.

Which layer are you building first?

MEDIA Most people are using Claude like Google. Become better at AI in just 1 minute a day. Get the AI newsletter smart leaders read. Sign up free now → aiforleaders.com _____ Here's the 7-day system to make Claude useful for real work: Most people stay stuck at basic prompts. They ask one-off questions. They start from scratch every time. They never build a repeatable setup. Claude gets much better when you stop treating it like a search box. Here's the breakdown: • Day One to Two: Stop using Chat for everything Chat is fine for quick questions. But it is not where the real work happens. Use Claude like this: → Chat for quick answers → Projects for ongoing work → Cowork for deeper execution ↳ Most people never leave Chat, so they miss the best parts of Claude. • Day Two to Three: Build your Claude OS Create a simple folder structure. Use: → ABOUT ME for your identity, tone, and writing rules →

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com
- http://about-me.md
- http://anti-ai-style.md

#392 Parmeshwar Pardeshi

Java / Spring

Java Backend Developer | 5.6 Years Experience | Spring Boot | Microservices | Kafka | REST APIs | AWS | Docker | SQL | FinTech & Payment Systems

4mo · **View on LinkedIn**

Complete OOP in Java — Concepts, Rules & Corner Cases

A concise notebook-style guide covering the 4 pillars of OOP, Java-specific behaviors, tricky output scenarios, access modifier rules, inheritance hierarchy, polymorphism internals, abstraction concepts, and commonly misunderstood corner cases every Java developer should know.

#Java #OOP #ObjectOrientedProgramming #JavaDeveloper #Programming #BackendDevelopment #SpringBoot #SoftwareEngineering #CodingNotes #JavaCommunity #DeveloperLife #LearnJava #TechNotes #JavaFundamentals

MEDIA advertisement

LINKS

- https://youtu.be/lvR9HPcT_8E
- https://www.youtube.com/watch?v=lvR9HPcT_8E

#393 Pradeep Kumar Tyagi

AI / LLM

GenAI Architect | Cloud based Digital Transformation | Azure/AWS | Enterprise Security (Entra ID, RBAC)

4mo · **View on LinkedIn**

Most viral AI architecture at Internet

Ran across this production AI architecture breakdown recently (IC Chorouk Malmoum) and it highlights a critical truth:

moving from a cool AI demo to a robust, enterprise-grade system requires real architectural discipline.

Building a production-ready AI application isn't just about calling an API; it requires a structured, multi-layered approach:

Services & Routing: Breaking down the core logic into distinct components like RAG pipelines, semantic caching, and intelligent query routers rather than keeping it monolithic.

Self-Correcting Agents: Utilizing document graders and adaptive routers to build resilient, autonomous workflows.

Strict Security & Evaluation: Implementing multi-stage guardrails (input, content, output) and robust evaluation layers (golden datasets, offline evals) instead of shipping blind.

Observability: Granular, per-stage tracing linked to cost-per-query analysis for predictable operations.

For anyone architecting AI platforms right now, this structure is a fantastic blueprint for what "production-grade" actually looks like.

What does your current AI stack look like?

Are you implementing a dedicated evaluation layer, or is that still a work in progress?

Let's discuss in the comments.

#GenerativeAI #SoftwareArchitecture #EnterpriseAI #LLMs #AIEngineering

MEDIA document/carousel (possible)

#394 Anmol S

AI / LLM

Digital Product Creator at Gumroad

4mo · **View on LinkedIn**

AI Agents are only as powerful as the architecture behind them.

Most people think MCP is just "another API wrapper."

It's not.

MCP is the nervous system of modern agentic AI — and there are 7 distinct architectural roles that separate toy demos from production-grade systems.

Here's the breakdown every builder needs to know ↓

-

1. Tool Lean-in (One-to-One Transport)

One service. One MCP server. Simple, debuggable, versionable.

→ Example: GitHub MCP for repo operations

2. Context Provider (Resource-First)

Agents that read before they act.

Context informs reasoning, not just execution.

→ Example: Docs, files, DB rows feeding into agent memory

3. Gateway Connector (Multi-Service Hub)

One MCP layer managing many services. Centralizes auth, rate limiting, observability.

→ Example: Jira + Salesforce under a single gateway

4. Stateful Session Manager

Maintains state across calls. Handles sessions, transactions, browser contexts.

→ Example: Playwright MCP managing persistent browser sessions

5. Sandboxed Executor

Executes code in isolation. The trust boundary between "agent thinking" and "agent doing."

→ Example: E2B sandboxing for safe code execution

6. Workflow Packager (Runbook-as-a-Tool)

Wraps multi-step workflows. Server handles orchestration so the agent doesn't have to.

→ Example: Deployment pipelines, employee onboarding flows

7. Delegated Reasoner (Agent-Inside-Agent)

The MCP server becomes an agent. Handles its own tools, context, and reasoning.

→ Example: Research sub-agents, code review agents

-
Why this matters:
→ Purpose-Built: Each role solves a specific architectural problem
→ Composable: Combine roles to build complex enterprise systems
→ Secure by Design: Isolation, auth, and state management where it counts
→ Production Ready: From simple tools to mission-critical workflows
The teams building with these 7 roles in mind?
They're the ones shipping agentic systems that actually work at scale.
The ones treating MCP as "just an API wrapper"?
They're debugging auth failures at 2 AM.
-
Which architectural role are you implementing first in your agent stack?
Drop a number (1-7) in the comments — curious where the community is focused

#395 Balaji Mani

AI / LLM

AI Governance & Agentic AI Leader | Trusted Advisor for Enterprise AI Transformation | Responsible AI, AI Risk & Enterprise Architecture | Helping Organizations Govern, Scale & Secure AI

4mo · View on LinkedIn

Dear LinkedIn family,
Future of Agentic AI is Being Standardised — Enter MCP Architecture
The shift in AI is no longer about better models.
It’s about systems that can think, decide, and act autonomously.
Welcome to the era of Agentic AI ↓
(!) The Challenge We Faced
Until recently, building agentic systems meant dealing with:
Fragmented tool integrations
Inconsistent context handling
Vendor-locked frameworks
→ Result: Complex, brittle, and hard-to-scale architectures
✓ The Shift: Model Context Protocol (MCP)
A major breakthrough is emerging: MCP — the universal connectivity layer for AI agents
→ Think of it as:
“USB-C for AI systems”
It standardises how agents:
Access tools
Retrieve data
Execute actions
Share context
MCP Architecture (Simplified)
1. Host Layer
AI Apps (Copilot, IDEs, enterprise platforms)
2. Agent Layer
Reasoning, planning, orchestration
3. MCP Layer (Connector Fabric)
Unified access to APIs, DBs, SaaS, workflows

→ Agents focus on what to do
→ MCP handles how to execute

From GenAI → Agentic Systems

Traditional AI

Agentic AI + MCP

Prompt-driven

Goal-driven

Static responses

Action execution

Custom integrations

Standard protocol

Single agent

Multi-agent systems

Multi-Agent Future

With MCP + emerging A2A protocols:

Agents collaborate like teams

Tasks are decomposed dynamically

Workflows run in parallel

Example:

Planner → breaks tasks

Researcher → gathers data

Executor → performs actions

Validator → ensures compliance

What This Means for Architects

✓ Interoperability by design
✓ Plug-and-play enterprise integrations
✓ Traceable, governed AI workflows
✓ Scalable multi-agent ecosystems

Industry Direction

Major players are converging toward:

MCP (tool & data standardisation)

A2A (agent collaboration)

Open agent ecosystems

→ This is the HTTP moment for AI agents

Final Thought

The future of AI won’t be a single powerful model...
It will be an ecosystem of specialised agents connected through standard protocols.

MCP is becoming the foundation of that future.

#AgenticAI #MCP #AIArchitecture #FutureOfAI #EnterpriseAI #MultiAgentSystems #AIStandards #DigitalTransformation Tech Mahindra Kunal Purohit Nikhil Malhotra

#396 **Maryam Miradi, PhD** **AI / LLM**

Chief AI Scientist | 20+ Yrs in AI | 400+ Production AI Agents Built | AI Agents Instructor | Teaching 3,200+ students Agentic Python Systems (Claude Code, LangGraph, Google ADK, CrewAI, MCP, OpenAI) | 46k+ Newsletter

4mo · View on LinkedIn

How to Use Claude Code Like a Senior AI Engineer. → Even If You Start from Zero.
This is my 6-Step roadmap to turn a raw idea into spec-driven AI engineering.
I have built 400+ production AI agents.
My advice:
Do not start with code in Claude Code.
Start with specifications.
“Build this.” “Fix that.” “Add this feature.” works for small tasks, not for production AI agents.
Here is the workflow I combine 5 Power Claude Code Plugins:
Step 1: Use BMAD to Define the Mission
✸ Turn a rough idea into a PRD.
✸ Define goals, users, constraints, and risks.
✸ Get alignment before coding starts.
→ Think: Product Manager + Architect in one workflow.
Step 2: Use Spec Kit to Write the Rules
✸ Create a constitution for the project.
✸ Define principles, requirements, and decisions.
✸ Turn ideas into specifications.
→ Think: What must always be true?
Step 3: Use GSD to Research the Space
✸ Explore tools, frameworks, and trade-offs.
✸ Document risks and implementation options.
✸ Gather context before building.
→ Think: Research before implementation.
Step 4: Use GStack to Challenge the Plan
✸ Review architecture decisions.
✸ Stress-test assumptions.
✸ Find gaps before coding.
→ Think: Senior design review.
Step 5: Use Superpowers to Critique Everything
✸ Ask harder questions.
✸ Find missing requirements.
✸ Improve the plan before implementation.
→ Think: Red team your own design.
Step 6: Let Claude Code Build
✸ Now generate the code.
✸ Use the approved specification.
✸ Keep humans responsible for architecture and judgment.
→ Coding is the last step, not the first.
Please Remember:
Claude Code is not the architecture.
Claude Code is not the specification.
Claude Code is not the evaluation strategy.
Claude Code writes the code.
AI engineers design the system.

--

Skip the trial-and-error of building production AI agents.

Watch my free 30-min training + get 88 pages of production guides.

Join 46,000+ AI engineers, architects, and directors.

→ https://lnkd.in/eqDxeGFR

MEDIA Infographic titled “How to Use Claude Code Like a Senior AI Engineer” by Dr. Maryam Miradi. The visual presents a 6-step workflow that combines BMAD, Spec Kit, GSD, GStack, Superpowers, and Claude Code. The roadmap starts with defining the mission and requirements, creating specifications and project rules, researching tools and implementation options, challenging the architecture through design reviews, critiquing assumptions and plans, and finally letting Claude Code generate the implementation. The key message is that successful AI engineering starts with alignment, specifications, research, and evaluation before coding. The framework emphasizes that Claude Code accelerates execution, while AI engineers remain responsible for architecture, requirements, and decision-making.; document/carousel (possible)

LINKS

- https://www.maryammiradi.com/ai-agents-mastery?utm_source=linkedin_premiumbutton
- https://lnkd.in/eqDxeGFR

#397 Rakesh Gohel

AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

4mo · **View on LinkedIn**

AI agents are only as good as the engineers who build them.

10 free GitHub repositories that replace a $10,000 AI engineering courses...

No paywalls. No fluff. No outdated content. Just the best hands-on resources in AI engineering today. Bookmark this before you scroll past.

Here are 10 GitHub repositories AI Engineers should bookmark right now.

1. AutoResearch by Andrej Karpathy 630 lines of Python. AI modifies training code, runs ~100 experiments overnight on a single GPU. Autonomous research in action. https://lnkd.in/g4kv6Zs7

2. LLM Course Hands-on roadmap with Colab notebooks. The most structured free course for learning LLMs deeply. https://lnkd.in/gTtJKDhE

3. Hands-On Large Language Models Full code from the book. Notebooks covering LLM basics through fine-tuning. Learn by running it. https://lnkd.in/gp3FYtz5

4. AI Agents for Beginners Free 11-lesson course from Microsoft. Zero to agent builder structured and beginner-friendly. https://lnkd.in/g5txJ39j

5. GenAI Agents Tutorials from basic to advanced agent techniques. A hands-on guide to building intelligent AI systems. https://lnkd.in/gAvTcZ23

6. Made With ML Design, develop, deploy, and iterate on production ML apps. Real-world ML engineering, end to end. https://lnkd.in/gXbqudpH

7. Learn Harness Engineering Project-based course on agent environment setup, state management, and control mechanisms. https://lnkd.in/gJKWj8Kz

8. Awesome LLM Inference Curated papers with code on Flash-Attention, Paged-Attention, quantization, KV cache, and deployment. https://lnkd.in/g9Z3bhG9

9. Designing Machine Learning Systems Summaries and resources from Chip Huyen's essential ML systems book. Architecture thinking for engineers. https://lnkd.in/gJ7CpANT

10. Hands-On AI Engineering Curated collection of AI-powered apps and agentic systems practical LLM use cases, not theory. https://lnkd.in/gQHxRyMB

If you want to go deeper on AI Agents, my free newsletter breaks down everything you need to know: https://lnkd.in/esJDdA5Q

save this post. Your future self will thank you.

Which one are you starting with? Drop it in the comments ↓

#AIEngineering #AIAgents #LLMs #MachineLearning #GenerativeAI #ArtificialIntelligence #AgenticAI

MEDIA 10-Github-repos-for-AI-Engineers for AI Agents

LINKS

- https://lnkd.in/g4kv6Zs7
- https://lnkd.in/gTtJKDhE
- https://lnkd.in/gp3FYtz5
- https://lnkd.in/g5txJ39j
- https://lnkd.in/gAvTcZ23
- https://lnkd.in/gXbqudpH
- https://lnkd.in/gJKWj8Kz
- https://lnkd.in/g9Z3bhG9
- https://lnkd.in/gJ7CpANT
- https://lnkd.in/gQHxRyMB
- https://lnkd.in/esJDdA5Q

#398 Anusha Sundaramurthi

AI / LLM

AI & Automation Engineer | n8n, LLMs, RAG, FastAPI | Building AI-Powered Workflow Automation Systems

4mo · **View on LinkedIn**

Stop calling everything an AI Agent

A chatbot with tool access is not automatically an AI Agent.

Neither is a RAG pipeline.

Yet many teams are rebuilding their architecture because they think "agent" simply means "LLM + tools."

It doesn't.

Here's the expensive mistake:

A RAG system retrieves information.

An AI Agent decides what to do next.

Those are different problems.

Imagine a customer support system.

A RAG application receives a question, retrieves documents, and generates an answer.

The workflow is largely predetermined.

The model is helping with retrieval and generation.

Now consider a real agent.

It can decide:

whether it needs more information

which tool to use

whether a previous step failed

when to ask the user for clarification

how to revise its plan

The difference is not intelligence.

The difference is autonomy.

"Tool use is a capability. Decision-making is an architecture."

Many organizations jump straight to agents because it sounds more advanced.

Then they discover they didn't need planning, memory, feedback loops, or multi-step reasoning in the first place.

They needed better retrieval.

What to do before building an agent

Audit the decisions
List every decision your system must make. If there are none, you probably don't need an agent.
Map the workflow
If every step is known beforehand, a workflow often beats an agent.
Count the feedback loops
Agents become valuable when outcomes influence future actions.
Test RAG first
Many "agent" use cases disappear after retrieval quality improves.
What's one use case where a RAG system fails, but an autonomous agent becomes necessary?
The fastest way to waste an AI budget is to buy autonomy before proving you need decisions.
#AIAgents #RAG #GenerativeAI #LLM #AIArchitecture

MEDIA document/carousel (possible)

#399 **Aman Mishra** **System Design**

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

4mo · **View on LinkedIn**

How do you route a million food orders a day without your servers catching fire?
That's what Zomato's engineering team solves every single day.
Here's how they do it ↓
You open Zomato.
Search "biryani"
47 restaurants appear in milliseconds.
You place an order. Pay.
A delivery partner gets assigned.
You track them live.
Food arrives in 25 minutes.
Looks simple.
Behind the scenes, dozens of distributed systems are working together.
Search
Your query doesn't hit a traditional database.
Elasticsearch handles search, typo correction, ranking, and filters.
Redis caches restaurant and menu data for lightning-fast responses.
Order Processing
One click triggers multiple services:
• Order Service
• Payment Service
• Restaurant Service
• Notification Service
They communicate through Kafka events.
This is Event-Driven Architecture.
If one service fails, the entire system doesn't.
Rider Assignment
The platform must find a rider who is:
• Nearby

• Available
• Reliable

This is a real-time optimization problem powered by geospatial indexing and location updates every few seconds.

Live Tracking

That moving dot on the map?

WebSockets.

Your phone receives live updates instantly instead of constantly asking for them.

Handling Peak Traffic

Lunch and dinner can increase traffic by 10x.

To stay online, Zomato relies on:

✓ Load Balancing
✓ Auto Scaling
✓ Database Sharding
✓ Rate Limiting
✓ Circuit Breakers

Every feature maps to a core system design concept:

Search → Elasticsearch + Caching

Orders → Event-Driven Architecture

Rider Assignment → Geospatial Systems

Tracking → WebSockets

Scale → Sharding + Auto Scaling

The next time your biryani arrives in 25 minutes, remember:

You're not just ordering food.

You're interacting with one of the most fascinating distributed systems in production today.

Which part of the architecture would you like to see a deep dive on next?

→ Follow Aman Mishra for more backend insights,content, and interview-focused tech breakdowns!

I've covered this in depth, GC tuning, JVM flags, and real interview Q&As — in my Java Core & Advanced Interview Mastery Guide.Offering 50% off for a limited time! ↓

Get the guide here: https://lnkd.in/gn3AG7Cm

Use code JAVA50

#SystemDesign #BackendEngineering #SoftwareEngineering #Microservices #Kafka #Redis #Elasticsearch

LINKS

- https://lnkd.in/gn3AG7Cm

#400 Rocky Bhatia

Other

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

4mo · **View on LinkedIn**

Behind the Scenes of UPI: How It Handles ₹18.41 Trillion Transactions!

India's UPI has revolutionised digital payments - an interoperable, public infrastructure enabling private innovation. It's simple, fast, and widely adopted.

How UPI Works

UPI links virtual payment addresses (VPAs) like rahul@hdfc directly to bank accounts, eliminating the need for IFSC codes. But what happens behind the scenes when you send money?

Let’s break it down:

1. Rahul (Sender) uses PhonePe to send ₹100 to Amit (Receiver).

2. He enters Amit’s UPI ID.
3. PhonePe encrypts the request and forwards it to its acquiring bank (SBI).
4. SBI sends it to NPCI, which validates Rahul’s account and fetches Amit’s bank details.
5. NPCI routes the request to Amit’s bank (ICICI) for validation.
6. ICICI confirms the details, and NPCI authorizes Rahul’s bank (HDFC) to debit ₹100.
7. HDFC checks balances, debits the amount, and confirms back to NPCI.
8. NPCI instructs ICICI to credit ₹100 to Amit’s account.
9. ICICI completes the credit and notifies NPCI.

NPCI ensures settlements between banks via RBI, completing the transaction.

All of this happens in seconds!

Why UPI Works So Well

✓ Banks: Hold funds & process transactions

✓ Payment Apps: Link bank accounts & facilitate transactions

✓ NPCI: The trusted switch ensuring seamless routing & security

This blend of innovation and security has made UPI the backbone of India’s digital economy.

What’s your take on UPI’s future?

LINKS

- https://www.youtube.com/@learnwithrockybhatia

#401 Abdul Rehman

AI / LLM

Full-Stack Agentic AI Architect | AI Agents, Automation Workflows | APIs, Data Pipelines, RAG & Cloud Infrastructure | Python, React, Node | AWS/GCP | Open to Full-Time Roles & Freelance Projects

4mo · **View on LinkedIn**

LLMs → RAG → Agents → Agentic AI

This is the map most businesses need to understand modern AI.

These are not separate trends.

They are layers of the same intelligence stack.

1. LLMs are the foundation

LLMs understand language.

They follow instructions.

They reason.

They generate outputs.

But on their own, they do not have real business context.

They do not know your company data.

They do not understand your workflows unless you connect them to the right systems.

2. RAG adds your data

RAG connects LLMs with external knowledge.

→ Documents → Databases

→ Company files → Knowledge bases

→ Internal systems

This makes AI answers more grounded, accurate, and useful.

But RAG still mostly answers.

It does not always act.

3. AI agents add action

Agents move AI from answering to doing.

They can:

→ Use tools → Call APIs
→ Follow steps → Manage tasks
→ Trigger workflows → Work with business systems
This is where AI becomes more practical for operations.
Not just “tell me what to do.”
But “help me do it.”
4. Agentic AI adds orchestration
This is where multiple agents or workflows work together.
1. One agent plans. 2. One retrieves.
3. One executes. 4. One verifies.
One asks for human approval when needed.
That is how AI starts supporting more complex business workflows.
Here is the real point:
The future of AI is not just better prompts.
It is better architecture.
A business may not need full agentic AI on day one.
Sometimes it only needs a strong RAG system.
Sometimes it needs one AI agent connected to a workflow.
Sometimes it needs multi-agent orchestration.
At Services Ground, we help businesses think through these layers practically.
Not by chasing trends.
But by identifying which layer actually solves the business problem.
Because successful AI adoption is not about using the most advanced term.
It is about building the right system for the right workflow.
Where is your business right now: LLMs, RAG, AI agents, or full agentic workflows? Come hang out on Agentic Ops Hub Discord: https://lnkd.in/dF6nNhK4
Subscribe for free: https://lnkd.in/dzQpf5uQ
#RAG #AgenticRAG #AIAgents #AgenticAI #LLMs #AIEngineering #AISystems #VectorSearch #GenerativeAI #MLOps

MEDIA document/carousel (possible)

LINKS
- https://lnkd.in/dF6nNhK4
- https://lnkd.in/dzQpf5uQ

#402 Smarter with AI

AI / LLM

4mo · **View on LinkedIn**

Free credentials won’t make you AI-ready. A learning path will.
The reason “AI rollout” feels chaotic in many companies is not a lack of tools.
It’s a lack of shared competence.
People don’t know what to ask, how to evaluate outputs, or where the risks live.
This is why structured, short programs matter.
Not because a certificate impresses anyone.
Because it forces a baseline: vocabulary, prompting habits, responsible use, and measurable application.
If you want these courses to translate into business impact, treat them like a delivery sprint:
Choose one track per persona: individual contributor, manager, technical builder, executive.

Block a fixed weekly slot.

And require one artifact after each module: a reusable prompt template, a meeting-notes workflow, an evaluation checklist, a policy snippet, or a small automation.

Two rules keep it real:

Measure outcomes, not hours watched. Time saved per workflow beats “completion rate.”

And set guardrails early: what data can be used, when citations are required, and when humans must approve.

A good learning program reduces rework, improves judgment, and makes adoption predictable.

That’s the real return on “free.”

If your team had to pick just one certificate to start, would you optimize for productivity gains, leadership alignment, or technical depth? ✓

Learn AI in 3 Minutes a Day → https://lnkd.in/eC_XFZJJ

LINKS

- https://lnkd.in/eC_XFZJJ

#403 **Rishi Jha** Other

Backend Engineer | Core Banking & Payments | Java • Spring Boot • Kafka | Payment Systems • Distributed Systems | Banking & Fintech

4mo · **View on LinkedIn**

Tokenization in Cards — The Invisible Security Layer Behind Modern Payments

Every time you tap your card, use Apple Pay, Google Pay, or save your card on an e-commerce website, chances are:

Your actual card number is not being used.

Instead, a token is used.

What is Tokenization?

Tokenization is the process of replacing the real card number (PAN) with a unique substitute value called a token.

Example:

Actual Card PAN:

4761 73XX XXXX 9012

Token:

4892 45XX XXXX 7685

The token:

looks like a card number,

can be used for transactions,

but has no value if stolen outside its intended use.

How Card Tokenization Works

Customer

↓

Adds Card to Apple Pay / Merchant

↓

Card Network Token Service

(Visa Token Service / Mastercard MDES)

↓

Generates Token
↓
Token Stored on Device/Merchant
↓
Payment Uses Token Instead of PAN
Example: Apple Pay Transaction
Suppose your HDFC Visa card is added to Apple Pay.
During card provisioning:
Real PAN
↓
Visa Token Service
↓
Device Token Generated
The iPhone stores:
Token + Cryptographic Keys
During payment:
iPhone
↓
NFC
↓
POS Terminal
↓
Acquirer
↓
Visa Network
Visa then performs:
Token
↓
Detokenization
↓
Actual PAN
↓
Issuer Bank
The issuer never knows whether the transaction came from:
physical card,
Apple Pay,
Google Pay.
It simply receives the real PAN for authorization.
Why Tokenization Matters
Without tokenization:
Merchant Database Breach
↓
Actual PAN Stolen
↓
Fraud Risk
With tokenization:

> Merchant Database Breach
> ↓
> Only Tokens Exposed
> ↓
> Limited Usefulness
> Types of Tokenization
> 1. Device Tokenization
> Used by:
> Apple Pay
> Google Pay
> Samsung Pay
> Token is linked to:
> Specific Device
> 2. Merchant Tokenization
> Used by:
> Amazon
> Netflix
> Uber
> Token is linked to:
> Specific Merchant
> 3. Network Tokenization
> Provided by:
> Visa
> Mastercard
> American Express
> The card network manages the token lifecycle.
> Tokenization vs Encryption
> Encryption
> PAN
> ↓
> Encrypted PAN
> ↓
> Decryptable
> If you have the key:
> ✓ Original PAN can be recovered.
> Tokenization
> PAN
> ↓
> Token
> The token itself does not mathematically reveal the PAN.
> A separate token vault maintains the mapping.
> Real-World Example
> When you replace your iPhone:
> Old Device Token → Deactivated
> New Device Token → Generated
> Even though:

Same Physical Card

each device gets its own token.

Final Take

Tokenization transformed card security by ensuring that merchants and devices don't need to store the actual card number.

The result?

✓ Reduced fraud risk

✓ Safer digital payments

✓ Better customer experience

#404 Rajesh Kumar Cloud / K8s / DevOps

Principal Architect | Data, AI & Cloud Transformation | Agentic AI Architecture & Strategy | Hands-On Enterprise Architect | 12× AWS & Azure | TOGAF Certified

4mo · View on LinkedIn

A Production-Ready Agentic AI Architecture Pattern on AWS

Overall architecture highlights how AgentCore’s components fit together (Runtime, Gateway, Memory, Identity, Observability), and the hybrid search, RAG pattern and grounded responses.

This architecture solves grounded retrieval, secure tool execution, and responsible AI governance in one unified design.

I built this architecture for real world use cases, and this pattern can easily scale across diverse use cases.

End-to-End Architecture Flow:

User submits a legal research query through a web, mobile, or API interface-fronted by API Gateway and WAF for edge protection.

AgentCore Runtime provides serverless execution, memory, scaling, security, and observability for enterprise agents.

Agent frameworks(Strands, LangGraph, CrewAI, Or LlamaIndex) enable orchestration and tool integration. Choose one primary agent framework-e.x., LangChain.

AgentCore Gateway securely connects agents to enterprise tools and MCP-compatible services.

Memory & Context

AgentCore Memory maintains short-term session context and long-term semantic memory.

Knowledge Ingestion

Enterprise content lands in Amazon S3, gets chunked, embedded, and indexed into OpenSearch-the pipeline that actually feeds retrieval.

Hybrid Retrieval (RAG)

Amazon OpenSearch combines:

• Vector search for semantic understanding

• BM25 for exact terms, citations, and keywords.

Relevance Optimization

Bedrock Reranker re-scores retrieved content so only the most relevant context reaches the model.

Guardrails & Responsible AI

Amazon Bedrock Guardrails enforce content filtering, PII redaction, denied topics before and after generation.

Tool Calling & Enterprise Actions

AgentCore Gateway orchestrates tools and business actions through AWS Lambda and enterprise integrations. High-risk actions route through a human-in-the-loop approval step before execution.

Identity & Security

Identity and Cognito handle authentication, authorization, and non-human identity for agents.

Observability & Evaluation

AgentCore Observability CloudWatch, X-Ray provide observability, tracing and monitoring.

Governance and Approval Gates

Centralized governance enforces policies, auditability, compliance, model controls, Human-in-the-Loop and responsible AI guardrails.

Amazon S3 serves as the secure repository for legal documents, policies, memos, contracts, and research artifacts.

PostgreSQL maintains legal metadata for document discovery.

Amazon Bedrock generates the final response, with trusted legal references and citations.

Key AWS Services:

Amazon Bedrock | AgentCore (Runtime, Gateway, Memory, Identity, Observability | OpenSearch Service | Amazon Bedrock Guardrails | Bedrock Reranker | AWS Lambda | Cognito | S3 | PostgreSQL | CloudWatch | API Gateway | AWS WAF

#ArtificialIntelligence #AgenticAI #GenerativeAI

#AWS #AmazonBedrock

#EnterpriseArchitecture

#CloudArchitecture

#SystemDesign

#DataEngineering

MEDIA document/carousel (possible)

#405 AI Architecture Insights

AI / LLM

4mo · **View on LinkedIn**

Today’s topic: Enterprise AI Architecture — From Demo to Production

Enterprise AI architecture is not just about connecting an application to an LLM.

A simple demo may look like:

User → App → LLM → Response

But a production-ready enterprise AI system usually needs multiple layers working together:

User experience layer

API gateway and authentication

AI orchestrator

LLM layer

RAG and vector database

Memory layer

Tool/API calling

Governance and guardrails

Monitoring and observability

The real value is not only in these components.

The real value comes from how they interact safely and reliably.

A practical flow can look like:

User request → AI orchestrator → context retrieval → LLM reasoning → approved tool call → validation → response → audit log

This is where enterprise AI moves from a prototype to a real production system.

For me, the key takeaway is simple:

LLMs generate answers, but enterprise architecture makes those answers reliable, secure, and usable in real workflows.

#AI #MachineLearning #GenerativeAI #AgenticAI #EnterpriseAI #MLOps #AIArchitecture #RAG

#406 Daniel Moka

AI / LLM

I help 200K+ developers craft better software | View my newsletter

4mo · **View on LinkedIn**

Most developers don't use agentic loops.

But they are the best way to automate work with AI.

Here's 8 tips on how to write quality Loops:

- Use Closed loops. Open loops burn a lot of tokens and money
- Only loop repeatable work with checkable "done."
- Run parallel agents in separate Git worktrees
- Use a separate agent to verify work is done
- Add quality gates with tests and linters, not on LLM output
- Use a RULES.md for repeated mistakes
- Manage RULES.md yourself. It's where human input matters
- Use Claude's built-in /goal commands to get started

To learn more about loops, check out my latest article: https://lnkd.in/dk9npy6c

Average developers write prompts for agents.

Great developers design loops that prompts agents.

MEDIA Image shared with post

LINKS

- https://craftbettersoftware.com/
- http://RULES.md
- https://lnkd.in/dk9npy6c

#407 Startup Adviser

AI / LLM

4mo · **View on LinkedIn**

The smartest job seekers aren’t applying randomly anymore.They’re↓

using AI, networking, ATS tools, interview prep platforms & remote job websites strategically.

Here are 60 powerful platforms every job seeker should know ↓

Job Search

• LinkedIn — linkedin.com/jobs
• Indeed — indeed.com
• Glassdoor — glassdoor.com
• Wellfound — wellfound.com
• Naukri — naukri.com
• FlexJobs — flexjobs.com
• RemoteOK — remoteok.com
• Remotive — remotive.com
• WeWorkRemotely — weworkremotely.com
• Foundit — foundit.in

Resume & ATS Tools

• ChatGPT — chat.openai.com

- Claude — claude.ai
- Gemini — gemini.google.com
- Teal — tealhq.com
- Jobscan — jobscan.co
- ResumeWorded — resumeworded.com
- Rezi — rezi.ai
- Kickresume — kickresume.com
- Enhancv — enhancv.com
- Canva — canva.com

Interview Preparation

- Pramp — pramp.com
- Interviewing.io — interviewing.io
- BigInterview — biginterview.com
- Yoodli — yoodli.ai
- Google Warmup — grow.google
- LeetCode — leetcode.com
- HackerRank — hackerrank.com
- Exponent — tryexponent.com
- Gainlo — gainlo.co
- Prepfully — prepfully.com

Networking & LinkedIn

- LinkedIn — linkedin.com
- Taplio — taplio.com
- Waalaxy — waalaxy.com
- Meetup — meetup.com
- Lunchclub — lunchclub.com
- Polywork — polywork.com
- Contra — contra.com
- Topmate — topmate.io
- ADPList — adplist.org
- CoffeeMug — coffeemug.ai

Free Learning

- Coursera — coursera.org
- Udemy — udemy.com
- edX — edx.org
- freeCodeCamp — freecodecamp.org
- GreatLearning — mygreatlearning.com
- FutureLearn — futurelearn.com
- Simplilearn — simplilearn.com
- Skillshare — skillshare.com
- KhanAcademy — khanacademy.org
- Codecademy — codecademy.com

Freelancing & Remote Work

- Upwork — upwork.com
- Fiverr — fiverr.com
- Freelancer — freelancer.com

• Toptal — toptal.com
• PeoplePerHour — peopleperhour.com
• Guru — guru.com
• Arc — arc.dev
• Outsourcely — outsourcely.com
• WorkingNomads — workingnomads.com
• SolidGigs — solidgigs.com

Portfolio & Productivity

• Notion — notion.so
• Behance — behance.net
• Dribbble — dribbble.com
• GitHub — github.com
• Medium — medium.com
• Substack — substack.com
• Carrd — carrd.co
• Framer — framer.com
• Webflow — webflow.com
• Canva — canva.com

Bonus

Get access to:

* 1k+ Advance prompts
* 30+ AI resources and guides.
* ChatGPT Cheatsheet and More.

100% FREE ↓

https://lnkd.in/edmK7AG8

–

1. Access the 10k+ prompts library here:

↳ https://lnkd.in/eQrDuXtv.

LINKS

- http://indeed.com
- http://glassdoor.com
- http://wellfound.com
- http://naukri.com
- http://flexjobs.com
- http://remoteok.com
- http://remotive.com
- http://weworkremotely.com
- http://foundit.in
- http://chat.openai.com
- http://claude.ai
- http://gemini.google.com
- http://tealhq.com
- http://jobscan.co
- http://resumeworded.com
- http://rezi.ai
- http://kickresume.com
- http://enhancv.com
- http://canva.com
- http://pramp.com
- http://Interviewing.io
- http://interviewing.io

- http://biginterview.com
- http://yoodli.ai
- http://grow.google
- http://leetcode.com
- http://hackerrank.com
- http://tryexponent.com
- http://gainlo.co
- http://prepfully.com
- http://taplio.com
- http://waalaxy.com
- http://meetup.com
- http://lunchclub.com
- http://polywork.com
- http://contra.com
- http://topmate.io
- http://adplist.org
- http://coffeemug.ai
- http://coursera.org
- http://udemy.com
- http://edx.org
- http://freecodecamp.org
- http://mygreatlearning.com
- http://futurelearn.com
- http://simplilearn.com
- http://skillshare.com
- http://khanacademy.org
- http://codecademy.com
- http://upwork.com
- http://fiverr.com
- http://freelancer.com
- http://toptal.com
- http://peopleperhour.com
- http://guru.com
- http://arc.dev
- http://outsourcely.com
- http://workingnomads.com
- http://solidgigs.com
- http://notion.so
- http://behance.net
- http://dribbble.com
- http://github.com
- http://medium.com
- http://substack.com
- http://carrd.co
- http://framer.com
- http://webflow.com
- https://lnkd.in/edmK7AG8
- https://lnkd.in/eQrDuXtv

#408 Antrixsh Gupta

AI / LLM

Senior Solution & AI Architect & Data Science Leader @Genzeon Platform | Architecting LLM/GenAI Systems, Clinical Intelligence & Responsible AI for Healthcare & BFSI Industries | LinkedIn Top Voice

4mo · **View on LinkedIn**

Most people think LLMs have memory.

They don't.

Every time you start a new conversation, the model starts from zero. Everything it "remembers" is an engineering decision made outside the model.

Here are the 5 types of AI memory what they do, how they wire into an LLM, and when to use each.

1. In-context memory

This is the simplest form. Everything in the active prompt window: your instructions, the conversation history, any docs you injected. It exists only for the duration of that request. When the context closes, it's gone.

Use it for: single-turn tasks, short conversations, inline document Q&A.

2. External memory / RAG

Your documents live in a vector database. At inference time, the user's query gets embedded, similar chunks are retrieved, and only the relevant pieces get injected into the prompt. The model never sees your full database - just what's relevant.

Use it for: internal knowledge bases, compliance docs, product catalogs - anything too large for context and too dynamic for fine-tuning.

3. Episodic memory

Past conversations get compressed into summaries, stored, and retrieved at the start of new sessions. The model doesn't replay full transcripts. It receives curated history - enough context to maintain continuity without overloading the prompt.

Use it for: personal assistants, coaching tools, long-running agents where prior decisions affect current ones.

4. Semantic memory

Structured facts extracted from interactions - user preferences, entity relationships, business rules - stored in key-value stores or knowledge graphs. More reliable than episodic for stable, structured lookups.

Use it for: personalization engines, CRM-integrated AI, recommendation systems.

5. Parametric memory

Knowledge baked directly into the model weights during pre-training or fine-tuning. No retrieval step at inference. This is the model's built-in intelligence - the fastest and cheapest to query, but the hardest to update.

Use it for: domain-specific language, stable proprietary workflows. Never use it for data that changes.

Follow Antrixsh Gupta for more insights

MEDIA document/carousel (possible)

LINKS

- https://genzeon.one/

#409 Sagar Agarwal

AI / LLM

1M+ Impressions | Senior Data Engineer | Gen AI Engineer | Building Production-Ready AI & Data Systems | Azure | SQL | Python

4mo · **View on LinkedIn**

Most Claude users are stuck at 10% capacity.

Here's the cheat code list nobody talks about

I've been using Claude daily for months, and the biggest unlock wasn't a better prompt - it was learning these simple command prefixes that completely change how Claude responds.

Start with these

Output style:

→ /brief — kills the fluff, gives you the answer only

→ /ghost — makes it sound human, not AI-generated

→ /10x — rewrites anything 10x sharper and tighter

Deeper thinking:

→ /devil — steelmans the opposing side (great for debate prep)

→ /scout — hunts for risks and blind spots in your plan

→ /ooda — structured framework for complex problem-solving
→ /compare — instant side-by-side analysis
Learning faster:
→ /explainlikeim5 — breaks down any concept for total beginners
→ /teacher — turns Claude into a mentor who debates and quizzes you
→ /socratic — guides you through questions instead of just answering
Building & creating:
→ /artifacts — builds live apps, charts, and tools right in chat
→ /pitch — generates a crisp 30-second investor pitch
→ /critique — tears apart your work and shows you how to fix it
→ /godmode — removes the diplomatic sugar-coating entirely
→ /debug — finds and fixes code bugs with full context
Quick note: these aren't built-in slash commands — they're prompt conventions. Putting them at the start of your message primes Claude to respond in that mode. Works surprisingly well.
Which one are you trying first? Drop it below ↓
#Claude #AITools #PromptEngineering #GenAI #Productivity

#410 Viktoriia Horokh

Other

Head of Payments | Payment Strategy & Operations | Approval Rate & Revenue Optimisation | PSP & Acquirer Partnerships | EU FinTech & Banking

4mo · View on LinkedIn

Reversal, refund and chargeback are often confused.

For a customer, they may look similar: “I want my money back.”

But for a payment team, these are three very different processes.

Reversal happens when the payment is cancelled before the money has fully moved through the payment chain. For example, the transaction was authorised, but not yet finally settled. This is usually the fastest and least expensive scenario for the business.

Refund happens after the payment has already been completed. The customer paid for a product or service, and the merchant agrees to return the money fully or partially. A refund may take several business days because the money goes back through the PSP, acquirer, card scheme and the customer’s issuing bank.

Chargeback is different.

It is a formal dispute initiated by the customer through their bank. The customer tells the bank that they do not recognise or do not agree with the transaction.

Chargebacks can happen for different reasons: fraud, non-delivery, service issues, an unrecognised charge, a subscription renewal the customer did not expect, or a refund that was not processed on time.

And usually, chargeback is the most expensive scenario.

The cost is not only the disputed amount. It can also include chargeback fees, investigation time, evidence preparation, communication with the PSP or acquirer, the risk of losing the dispute and the impact on the merchant’s dispute ratio.

That is why not every “money-back” case should be treated as a simple refund.

Sometimes the payment can still be cancelled quickly.

Sometimes it requires a standard refund.

And sometimes it has already become a formal dispute.

In payments, correct classification helps businesses control timing, cost and customer experience.

#Payments #HeadOfPayments #PaymentStrategy #PaymentOperations #Chargebacks #Refunds #CardPayments #Acquiring #PSP #Fintech

#411 Sivasankar Natarajan

AI / LLM

Technical Director | GenAI Practitioner | Azure Cloud Architect | Data & Analytics | Solutioning What’s Next

4mo · View on LinkedIn

Traditional AI vs Agentic AI vs Agentic RAG

Most teams treat AI like a static model.

Agentic AI treats it like an autonomous system that reasons, acts, and improves.

Traditional AI

Specify Task → Collect Data → Refine Data (Vector DB) → Retrieval Index → Model Training → Evaluate Outcomes → Deploy Model → Generate Results.

Traditional AI is a pipeline. You train once, deploy once, and hope it works. When data changes or requirements shift, you retrain from scratch.

Agentic AI

Specify Task → Choose LLM Model → Integrate Tools & APIs → External Operation → Search/APIs → Embed Logic & Iterations → Agent-Led Choices → Self-Decisions → Implement Actions → Improve & Evolve → Self-Decisions (loop back).

Agentic AI does not just predict. It reasons, takes action, evaluates results, and adapts. The agent makes decisions autonomously, learns from outcomes, and improves over time without retraining.

Agentic RAG

Specify Task → Fetch Useful Data → Search/APIs → External Operation → Design Multi-Step Process → Apply Iterative Logic → Implement Actions → Query APIs → Refresh Memory → Produce & Verify Results → Adapt for Future Use (loop back).

Agentic RAG combines retrieval with autonomous reasoning. The agent does not just fetch documents it designs multi-step processes, applies iterative logic, queries APIs, verifies results, and adapts its memory for future queries.

The Key Differences

Traditional AI: Static pipeline. Train, deploy, hope.

Agentic AI: Autonomous reasoning. Acts, evaluates, improves.

Agentic RAG: Retrieval + reasoning. Multi-step processes, iterative logic, memory refresh.

Traditional AI gives you answers. Agentic AI solves problems. Agentic RAG solves problems with context.

Most AI systems today are still traditional. The teams winning with AI are building agentic systems that reason, act, and evolve.

Which approach are you building today: traditional, agentic, or agentic RAG?

Repost this to help your network get started

+ Follow Sivasankar Natarajan for more

#RAG #AIEngineering #LLMs #AIAgents

MEDIA document/carousel (possible)

#412 Java Developers Community (moderated)

Java / Spring

Alexander Sidorov • 3rd+

4mo · **View on LinkedIn**

Many developers think GC solves all memory problems. Naive. The garbage collector doesn’t read minds and won’t fix sloppy code. It only removes objects that are unreachable. Everything else is your responsibility.

Static collections are a classic trap. Drop data into a static Map and forget about it? It lives until the JVM shuts down. Unlimited caches grow silently, and before you know it – OutOfMemoryError.

Event listeners are another favorite: subscribe but never unsubscribe, and the object stays alive through the callback, even though the business logic died long ago. ThreadLocal in a pool is even worse – values outlive requests, users, and your optimism.

Collections kill quietly. An ArrayList that only grows, a HashMap where keys accumulate “just in case” – they eat memory for months. In application servers, classloader leaks bring old classes, drivers, singletons, and threads that prevent module unloading, bleeding memory with every redeploy.

GC sees only the reference graph. If your object is reachable, it stays. Don’t rely on magic – use profilers, WeakReference, SoftReference, and common sense. Memory leaks are your architectural debt, and interest compounds.

#java #backend #softwareengineering #jvm #programming #devlife

#413 Aravind Kumar bysani

Java / Spring

"System Engineer at TCS| Experienced QA Manual & Automation Engineer | Expertise in Design of Test Approach, Test cases & Test execution, Test Scripts Agile | Banking Domain Expertise | Delivering Quality with precision"

4mo · **View on LinkedIn**

Java Evolution: Key Features from Java 8 to Java 22

Java has continuously evolved, introducing powerful features that improve developer productivity, code readability, performance, and scalability. Understanding these enhancements is essential for every Java Developer, Automation Engineer, and SDET.

Java 8 (Game Changer)

✓ Lambda Expressions
✓ Stream API
✓ Date & Time API (java.time)
✓ Optional Class
✓ Default & Static Methods in Interfaces

Impact: Introduced Functional Programming in Java and revolutionized collection processing.

Java 11 (Enterprise Favorite)

✓ Standard HTTP Client API
✓ Local Variable Syntax for Lambda Parameters
✓ String API Enhancements
✓ HTTP/2 Support

Impact: Simplified web service integration and improved application performance.

Java 17 (Long-Term Support)

✓ Records
✓ Sealed Classes & Interfaces
✓ Pattern Matching for instanceof
✓ Enhanced Switch Expressions

Impact: Reduced boilerplate code and improved code maintainability.

Java 21 (Latest LTS)

✓ Virtual Threads (Project Loom)
✓ Sequenced Collections
✓ Pattern Matching for Switch
✓ Record Patterns
✓ String Templates (Preview)

Impact: Massive improvements in concurrency and modern application development.

Java 22

✓ Unnamed Variables & Patterns
✓ Statements Before super()
✓ Stream Gatherers
✓ String Templates (Preview)

Impact: Cleaner code, improved stream processing, and enhanced developer experience.

Recommended Learning Path

Java 8 → Java 11 → Java 17 → Java 21 → Java 22

Focus first on:

✓ Lambda Expressions
✓ Stream API
✓ Collections Framework
✓ Functional Programming
✓ Records
✓ Virtual Threads

These topics are frequently asked in Java Developer, Selenium Automation, and SDET interviews.

Interview Tip: If you're preparing for Automation Testing roles, strong knowledge of Java 8 Streams, Collections, Exception Handling, OOPs, and Java 17+ features can significantly improve your interview performance.

Follow me: https://lnkd.in/gAJ9-6w3

— Aravind Kumar Bysani

#Java #Java8 #Java11 #Java17 #Java21 #Java22 #JavaDeveloper #Programming #SoftwareEngineering #Selenium #AutomationTesting #SDET #CodingInterview #StreamAPI #LambdaExpressions #VirtualThreads #OOP #TechLearning #InterviewPreparation #Developers

MEDIA advertisement

LINKS

- https://lnkd.in/gAJ9-6w3

#414 Santosh Patidar

Java / Spring

Java Backend Developer | Spring Boot | Microservices | REST APIs | PostgreSQL | Docker

4mo · **View on LinkedIn**

Day 11 of #30DaysOfJavaMastery | HashMap Internals Explained

Most Java developers use HashMap every single day.

But very few can confidently answer questions like:

? How does HashMap find data so quickly?

? What exactly are buckets?

? What happens when two keys generate the same hash?

? Why was Treeification introduced in Java 8?

? What is the purpose of Load Factor and Resize?

And that's exactly why HashMap Internals remains one of the most frequently asked Java interview topics.

In this infographic, you'll learn:

✓ How hashing works behind the scenes

✓ The role of buckets in storing key-value pairs

✓ How HashMap handles collisions efficiently

✓ Linked List → Red-Black Tree conversion (Treeification) in Java 8

✓ Why resizing happens and how rehashing impacts performance

✓ The importance of Load Factor and capacity tuning

✓ Time complexity of common HashMap operations

✓ Common mistakes developers make while using HashMap

✓ Best practices followed by experienced Java developers

✓ Interview questions that can help you crack your next Java interview

Pro Tip:

Always override both equals() and hashCode() when using custom objects as keys. Ignoring this can lead to unexpected behavior and difficult-to-debug issues.

Understanding HashMap isn't about memorizing definitions. It's about understanding the design decisions that make Java applications fast and scalable.

Save this post for interview preparation and quick revision.

Comment below: Have you ever been asked HashMap internals in an interview? What was the toughest question?

Share this with a Java developer preparing for interviews.

Follow Santosh Patidar for daily learning on Java, Spring Boot, React, Spring Security, JWT Authentication, Microservices, Backend Development, System Design, and Full Stack Software Engineering.

#Java #HashMap #JavaDeveloper #CollectionsFramework #JavaInterview #SpringBoot #BackendDeveloper #SoftwareEngineer #Programming #Coding #JavaProgramming #DataStructures #TechLearning #DeveloperCommunity #SystemDesign #FullStackDeveloper #InterviewPreparation #30DaysOfJavaMastery

MEDIA advertisement

#415 Java Developers Community (moderated)

Java / Spring

Eugen H. • 3rd+

4mo · View on LinkedIn

Performance problems in Java Spring Boot applications rarely come from one magical setting.

They usually come from small bad decisions repeated across the codebase.

A few examples:

* Loading entire tables instead of using pagination.
* Creating N+1 JPA queries without noticing.
* Saving records one by one instead of batching.
* Calling external REST services inside a loop.
* Creating ObjectMapper, HTTP clients, or other expensive objects per request.
* Logging too much in hot paths.
* Returning JPA entities directly from controllers.
* Not caching stable reference data.
* Starting unbounded async work from request threads.
* Writing queries that cannot use indexes.

None of these problems looks dramatic in development.

With 100 records, everything works.

With 10,000 records, the endpoint becomes slow.

With 10 million records, the system becomes unstable.

This is why performance is not only a “DevOps issue” or a “database issue.”

It is an engineering design issue.

Every layer crossing has a cost:

* Controller to service.
* Service to database.
* Service to network.
* Application to logs.
* Entity to JSON.
* Thread to async executor.

Good Spring Boot performance comes from asking simple questions during code review:

* How much data are we loading?
* How many SQL queries are executed?
* Are we returning DTOs or full entities?
* Are external calls bulked, cached, timed out, and bounded?
* Are logs useful, or are they creating noise and cost?
* Are async jobs controlled, or are they just moving the problem elsewhere?

The biggest performance wins often come before profiling.

They come from better code structure, better boundaries, better data access, and better production thinking.

I put together a short PDF with 10 bad Java Spring Boot examples that hurt performance, with better alternatives for each one.

#Java #SpringBoot #SoftwareEngineering #BackendDevelopment
#JavaPerformance #CodeReview #CleanCode #SystemDesign
#Microservices #ApplicationPerformance
#DatabasePerformance #JPA #Hibernate #Scalability
#EngineeringLeadership

5 pages

MEDIA Java Spring Boot Performance Examples - Page 1 10 Bad Examples That Impact Performance in Java Spring Boot Applications Concrete code smells, why they hurt, and better approaches for production systems. Performance problems in Spring Boot applications rarely come from one magical setting. They usually come from small design decisions repeated across controllers, services, repositories, logging, serialization, and database access. The examples below are common in real applications. Each one can look harmless in development and become expensive under production load. 1. Loading too much data into memory Why it hurts performance: A Spring Boot endpoint that loads an entire table can consume heap, trigger long GC pauses, and slow down the database. This becomes dangerous when the table grows from thousands to millions of rows. Bad example: @GetMapping("/orders") public List<Order> orders() { return orderRepository.findAll(); } Better approach: @GetMapping("/orders") public Page<OrderSummaryDto> orders(Pageable pageable) { return orderRepository.findOrderSummaries(pageable); } @Query("select new com.acme.OrderSummaryDto(o.id, o.status, o.createdAt) " + "from Order o order by o.createdAt desc") Page<OrderSummaryDto> findOrderSummaries(Pageable pageable); Review question: Does this endpoint use pagination and return only the fields the client really needs? 2. The JPA N+1 query problem Why it hurts performance: One request can accidentally produce hundreds or thousands of SQL statements. The application looks fine in development but collapses with real production data. Bad example: @GetMapping("/customers") public List<CustomerDto> customers() { return customerRepository.findAll().stream() .map(c -> new CustomerDto(c.getName(), c.getOrders().size())) .toList(); } Better approach: @Query("select c from Customer c left join fetch c.orders") List<Customer> findAllWithOrders(); @GetMapping("/customers") public List<CustomerDto> customers() { return customerRepository.findAllWithOrders().stream() .map(c -> new CustomerDto(c.getName(), c.getOrders().size())) .toList(); } Review question: How many SQL statements does this endpoint generate for 10, 100, and 1,000 records?; Java Spring Boot Performance Examples - Page 2 3. Saving one row at a time instead of batching Why it hurts performance: Looping over records and calling save repeatedly creates many round trips, many transactions, and unnecessary flush work. This is expensive in import jobs, APIs, and data pipelines. Bad example: @Transactional public void importTrades(List<TradeDto> trades) { for (TradeDto dto : trades) { tradeRepository.save(mapper.toEntity(dto)); } } Better approach: @Transactional public void importTrades(List<TradeDto> trades) { List<Trade> entities = trades.stream() .map(mapper::toEntity) .toList(); tradeRepository.saveAll(entities); } // Also configure JDBC batching for Hibernate where appropriate: // spring.jpa.properties.hibernate.jdbc.batch_size=100 Review question: Can this write path batch database operations instead of sending one statement at a time? 4. Calling external services inside a loop Why it hurts performance: Sequential REST calls multiply latency. If one external service call takes 150 ms and the loop runs 100 times, the endpoint can spend 15 seconds waiting. Bad example: public List<AccountDto> accounts(List<String> ids) { List<AccountDto> result = new ArrayList<>(); for (String id : ids) { AccountDto dto = restTemplate.getForObject("https://risk/api/accounts/" + id, AccountDto.class); result.add(dto); } return result; } Better approach: public List<AccountDto> accounts(List<String> ids) { return riskClient.getAccounts(ids); // bulk endpoint } // If no bulk API exists, use bounded concurrency and timeouts. // Do not create unbounded parallel calls from a request thread. Review question: Can we use a bulk endpoint, caching, or bounded concurrency instead of sequential network calls?; document/carousel (possible)

#416 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Lucia Moretti • 3rd+
4mo · **View on LinkedIn**

<> 7000+ Course Free Access: https://lnkd.in/gbeCT2sb

Google Data Analytics:-

https://lnkd.in/gBhq8EXw
Repost so that all can take benefits
1. Google Data Analytics
■https://lnkd.in/gBhq8EXw
2. Google Project Management
https://lnkd.in/gbwqe7f3
3. Google IT Support
■ https://lnkd.in/gd6Wbufg
4. Google Digital Marketing & E-commerce
https://lnkd.in/gfA4GiAr
5. Google IT Automation with Python
■ https://lnkd.in/gwrA-QkY
6. Google Business Intelligence
https://lnkd.in/gvDvVKrF
7. Google Advanced Data Analytics
■ https://lnkd.in/gBhq8EXw
8. Google Cybersecurity
https://lnkd.in/gVuDwxaj
9. Meta Database Engineer
■https://lnkd.in/gwEAFUnD
10. IBM Data Science
https://lnkd.in/gzpTAxpM
11. Indigenous Canada
■ https://lnkd.in/gfhdbyVA
12. Machine Learning
https://lnkd.in/gx7Bk2aE
13. IBM Data Analyst
■ https://lnkd.in/guz8AvRS
14. Deep Learning
https://lnkd.in/geSH2-E4
15. Python for Everybody
■ https://lnkd.in/gUqkvnGG

MEDIA advertisement

LINKS

- https://lnkd.in/gbeCT2sb
- https://lnkd.in/gBhq8EXw
- https://lnkd.in/gbwqe7f3
- https://lnkd.in/gd6Wbufg
- https://lnkd.in/gfA4GiAr
- https://lnkd.in/gwrA-QkY
- https://lnkd.in/gvDvVKrF
- https://lnkd.in/gVuDwxaj
- https://lnkd.in/gwEAFUnD
- https://lnkd.in/gzpTAxpM
- https://lnkd.in/gfhdbyVA
- https://lnkd.in/gx7Bk2aE
- https://lnkd.in/guz8AvRS
- https://lnkd.in/geSH2-E4
- https://lnkd.in/gUqkvnGG

#417 Maryam Miradi, PhD

AI / LLM

Chief AI Scientist | 20+ Yrs in AI | 400+ Production AI Agents Built | AI Agents Instructor | Teaching 3,200+ students Agentic Python Systems (Claude Code, LangGraph, Google ADK, CrewAI, MCP, OpenAI) | 46k+ Newsletter

4mo · View on LinkedIn

How to Build a Supply Chain AI Agent System → Even If Everything Still Runs Manually.
My 7-Step map to build AI Agents for supermarket stock and delivery.
Standard Retail AI models predicts demand which is fine.
However a real supermarket is slightly more complicated.
It needs to know:
- what will sell
- what is low
- what to order
- who can deliver
- which store needs stock
- what might go wrong
Meaning, you need an AI agent workflow.
I spent 4 Years building supply chain AI models.
I know the complexity and messy data quite well.
This is How I would build a Supply chain Multi-Agent System
that Actually work:
Step 1: Map the Real Work
✸ Start with what people do today.
✸ Track each step from low stock to new delivery.
✸ Do not automate a messy process too soon.
→ Example: A store is low on tomatoes. What happens next?
Step 2: Give Each Agent One Job
✸ Use small agents with clear roles.
✸ One checks demand. One checks stock. One writes orders.
✸ Do not build one huge supply chain chatbot.
→ Example: The Stock Agent should not plan delivery trucks.
Step 3: Connect Agents to Real Data
✸ Agents need live business data.
✸ Sales, stock, suppliers, prices, and delivery times.
✸ Do not copy and paste files into prompts forever.
→ Example: The Stock Agent reads the inventory system directly.
Step 4: Let Agents Pass Work Forward
✸ Each agent gives clean output to the next agent.
✸ Demand becomes a stock signal.
✸ Stock signal becomes a draft order.
✸ Draft order becomes supplier confirmation.
→ Example: “Order 240 kg tomatoes from Supplier A.”
Step 5: Keep Humans in Control
✸ Humans approve important actions.
✸ Orders, supplier changes, and delivery plans need review.
✸ Do not let agents spend money alone.

→ Example: A manager checks the order before it is sent.
Step 6: Make Every Decision Explainable
✸ Each agent must explain why.
✸ Why this item?
✸ Why this supplier?
✸ Why this delivery plan?
→ Example: “Supplier A was chosen because it is faster and more reliable.”
Step 7: Test the Whole Workflow
✸ Do not only test one agent.
✸ Test the full chain from demand to delivery.
✸ Check if humans can review the result quickly.
→ Example: Can the system catch a late supplier before shelves go empty?
Please Remember:
For supply chain forecast AI agent says what might happen.
But an agent workflow helps the business decide what to do next.
That's where the ROI is.
--
Skip the trial-and-error of building production AI agents.
Watch my free 30-min training + get 88 pages of production guides.
Join 46,000+ AI engineers, architects, and directors.
→ https://lnkd.in/eqDxeGFR

MEDIA Infographic titled “How to Build a Supply Chain AI Agent System: Replenishment to Human-Controlled” by Dr. Maryam Miradi. The visual presents a workflow that moves from demand prediction and inventory checks to supplier selection, delivery planning, approval, and execution. A 7-step roadmap shows how to build a production-ready supply chain agent system: map the workflow, split work into specialist agents, connect real business tools and databases, pass structured state between agents, add human approval for critical actions, explain every decision, and evaluate the workflow. The diagram highlights collaboration between demand, inventory, procurement, supplier, and exception agents, with human oversight to improve reliability, reduce supply chain disruptions, and support better inventory replenishment decisions.

LINKS
- https://www.maryammiradi.com/ai-agents-mastery?utm_source=linkedin_premiumbutton
- https://lnkd.in/eqDxeGFR

#418 Imran Shaikh

Java / Spring

Senior Specialist - Technology at Synechron
4mo · **View on LinkedIn**

Java Virtual Threads: The Future of Concurrency in Java
For years, Java developers had to choose between simplicity and scalability when building highly concurrent applications. Traditional platform threads were powerful, but they came with significant memory overhead and limitations.
With Java Virtual Threads (Project Loom), that changes.
What are Virtual Threads?
Virtual Threads are lightweight threads managed by the JVM rather than the operating system. They allow developers to create thousands or even millions of concurrent tasks without exhausting system resources.
✓ Why Virtual Threads Matter
Lightweight
Consume significantly less memory than platform threads.

Created and destroyed quickly.

Massive Scalability

Run millions of virtual threads on a small number of carrier (OS) threads.

Ideal for applications handling large numbers of concurrent requests.

Simple Programming Model

Write familiar blocking code.

No need for complex reactive programming or callback-heavy implementations.

Better Resource Utilization

When a virtual thread blocks (I/O, sleep, waiting), the JVM detaches it from the carrier thread, allowing the carrier to execute other tasks efficiently.

Example (Java 21+)

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
for (int i = 0; i < 1000; i++) {
executor.submit(() -> {
String result = callExternalService();
System.out.println(result);
});
}
}
```

Simple code. Massive concurrency.

Best Use Cases

✓ REST APIs and Microservices

✓ Database Operations

✓ External Service Integrations

✓ Messaging Systems

✓ File Processing Applications

✓ Batch Jobs and ETL Pipelines

✓ High-Concurrency I/O Workloads

(!) Keep in Mind

Virtual Threads are excellent for I/O-bound workloads, but CPU-intensive tasks still require careful design and appropriate parallelism strategies.

Key Takeaway

Virtual Threads make concurrent programming in Java easier, more scalable, and more developer-friendly.

If you're using Java 21, it's time to explore how Project Loom can simplify your architecture while improving performance.

Have you started using Virtual Threads in production? Share your experience in the comments! ↓

#Java #Java21 #VirtualThreads #ProjectLoom #Concurrency #Multithreading #SoftwareEngineering #BackendDevelopment #Microservices #Programming #JVM #DeveloperTips

#419 Ankit Pangasa System Design

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

4mo · **View on LinkedIn**

8 Redis Use Cases Every Backend Engineer Should Know

Most engineers know Redis as a cache, but it's much more than that.

Redis powers some of the world's fastest applications by providing ultra-low latency and specialized data structures that solve real-world system design challenges.

Here are 8 common Redis patterns every backend engineer should know:

✓ Caching – Reduce database load and improve latency

✓ Session Store – Share user sessions across servers

✓ Rate Limiting – Protect APIs from abuse

✓ Leaderboards – Real-time rankings using Sorted Sets

✓ Vector Search & Semantic Caching – Power AI and RAG applications

✓ Queues & Streams – Asynchronous job processing

✓ Pub/Sub – Real-time notifications and messaging

✓ Distributed Locks – Prevent race conditions in distributed systems

Common System Design Questions These Patterns Help Solve

How would you reduce database load in a high-traffic application?

Design a rate limiter for millions of users.

Build a real-time gaming leaderboard.

Design a notification or chat system.

Prevent multiple services from updating the same resource simultaneously.

Speed up AI/RAG applications while reducing LLM costs.

Learning Resources

Redis Crash Course (Traversy Media)

https://lnkd.in/g8QQ27jK

Redis Tutorial for Beginners (freeCodeCamp)

https://lnkd.in/gVpbVwkR

Redis & System Design (Gaurav Sen)

https://lnkd.in/gyHserze

Caching Strategies Explained (ByteByteGo)

https://lnkd.in/gQ4wnm2f

Redis Documentation

https://redis.io/docs

Redis University (Free Courses)

https://lnkd.in/g--t5TsG

Quick Interview Answer

Why Redis?

"Redis is an in-memory data store that provides extremely low latency and specialized data structures such as Hashes, Sorted Sets, Streams, and Pub/Sub channels, making it ideal for caching, session management, rate limiting, leaderboards, messaging, distributed locking, and modern AI workloads."

Big thanks to Level Up Coding for the excellent visual illustration.

Which Redis use case have you implemented in production, and what was your biggest learning?

#Redis #SystemDesign #BackendEngineering #SoftwareEngineering #DistributedSystems #Architecture #TechInterview #Programming #AI #RAG #Caching #Microservices

And thanks to Nikki Siapno for this really nice illustration

MEDIA document/carousel (possible)

LINKS

- https://lnkd.in/g8QQ27jK
- https://lnkd.in/gVpbVwkR
- https://lnkd.in/gyHserze

- https://lnkd.in/gQ4wnm2f
- https://redis.io/docs
- https://lnkd.in/g--t5TsG

#420 Ravena O

AI / LLM

AI Researcher and Data Leader | Healthcare Data | GenAI | Driving Business Growth | Data Science Consultant | Data Strategy

4mo · **View on LinkedIn**

Why do so many AI agents become expensive at scale?

Not because the models are bad.

But because every agent step adds more context, more tool calls, more reasoning, and more tokens.

In production AI systems, token efficiency is no longer an optimization.

It’s part of the architecture.

Here are 6 practical ways to reduce token usage in agentic workflows:

→ Prompt Caching

Reuse repeated system prompts, instructions, and workflows instead of reprocessing the same context every time.

→ Semantic Caching

Return previous answers when queries have the same meaning, even if phrased differently.

→ Routing

Use the right model for the right task.

Simple tasks do not need your most expensive LLM.

→ Clean Context

Remove noisy, duplicated, outdated, or irrelevant information before sending it to the model.

→ Context Compaction

Summarize long conversations and documents while preserving critical information.

→ Lazy Context Loading

Load only the information the agent needs at that moment instead of injecting everything upfront.

The biggest lesson:

More context ≠ better AI.

Better context = better AI.

The most scalable agentic systems are built on smarter context management, efficient routing, selective memory, and optimized token usage.

Useful if you’re building:

• AI agents
• RAG systems
• Automation workflows
• Production GenAI applications

CC: Greg Coquillo

MEDIA document/carousel (possible)

#421 Neo Kim

AI / LLM

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

4mo · **View on LinkedIn**

If you want to elevate your AI engineering career

(in 2026), save these 20 GitHub repos:

1 OpenClaw
↳ Runs a personal AI agent locally that can browse, plan & take actions on your device.
2 TensorFlow
↳ Provides a production-ready framework to build, train & deploy machine learning models at scale.
3 AutoGPT
↳ Automates multi-step tasks by chaining LLM reasoning into autonomous agents.
4 n8n
↳ Automates workflows with a visual builder that integrates APIs, data & AI tools.
5 Ollama
↳ Runs open LLMs locally with simple commands & optimized performance.
6 Stable Diffusion WebUI
↳ Generates images locally with a powerful UI for Stable Diffusion models.
7 Hugging Face Transformers
↳ Offers thousands of pretrained models for NLP, vision & multimodal AI tasks.
8 Langflow
↳ Builds & tests LLM pipelines visually using a drag-and-drop interface.
9 Dify
↳ Creates production-ready AI apps with built-in orchestration, prompts & APIs.
10 LangChain
↳ Orchestrates LLM workflows, tools, memory & agents in applications.
11 Open WebUI
↳ Delivers a self-hosted ChatGPT-style interface with local & API model support.
12 DeepSeek-V3
↳ Provides a high-performance open-weight LLM optimized for reasoning and coding.
13 PyTorch
↳ Builds & trains deep learning models with flexible, research-friendly APIs.
14 Gemini CLI
↳ Interacts with Google’s Gemini models directly from the command line.
15 llama cpp
↳ Runs LLaMA-style models efficiently on CPUs & local hardware.
16 Whisper
↳ Transcribes & translates speech with high accuracy using deep learning.
17 ComfyUI
↳ Designs advanced image generation workflows using node-based pipelines.
18 CrewAI
↳ Coordinates multiple AI agents to collaborate on complex tasks.
19 RAGFlow
↳ Implements retrieval-augmented generation pipelines for enterprise search & QA.
20 Claude Code
↳ Assists coding with deep repository understanding & agent-style workflows.
What else should make this list?
===
PS - Want my System Design Playbook (for free)?
Join my newsletter with 201K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===

Save & repost to help others get good at AI engineering.

Follow Neo Kim + turn on notifications.

MEDIA 20 AI GitHub repos for Software Engineers

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#422 Anchal Sharma

Other

Engineering @Myntra (Flipkart) || Ex-SDE@PayPal || 210K+ @LinkedIn || Mentor

4mo · **View on LinkedIn**

Everyone wants the “perfect roadmap” for DSA.

But interviews are rarely about memorized patterns.

They’re about how you THINK under pressure.

{{ Get Interview Ready with me in the next 3 Months!!!

Next batch is getting started on SUNDAY, 21st June.

Link - https://lnkd.in/g7HGBtwh

Guys, I run a PERSONAL GUIDANCE PROGRAM where Me and Ikshit .. guide very limited students/ batch to crack the SDE Interviews. We will guide you from the very scratch: learning programming language, DSA, Computer Concepts, LLD + HLD, Resume Review, Projects Creation & at the end Job Hunting Templates & Tricks!!! }}

Some rules I personally followed while solving coding problems:

- If brute force works → first understand WHY it’s slow
- If constraints are huge → optimize before coding
- If you’re confused → dry run on paper
- If a problem looks hard → break it into smaller states
- If you can explain your approach simply → you actually understand it
- Edge cases > fancy code
- Clean logic always beats clever shortcuts
- Most interviewers judge communication more than syntax
- Revising old questions > solving 100 random new ones
- Consistency in DSA > motivation spikes

And the biggest one:

Getting stuck is part of the process, not proof that you’re bad at coding. :)

Keep practicing. One problem at a time!

LINKS

- https://lnkd.in/g7HGBtwh

#423 Sanjay M

Cloud / K8s / DevOps

B.Tech (Information Technology) | VTMT(2025) | Java Developer | MERN Stack | Devops

4mo · **View on LinkedIn**

Learning Networking Fundamentals

One concept that often confuses beginners is the difference between a Proxy and a Reverse Proxy.

Proxy protects the client.

Reverse Proxy protects the server.

Understanding these concepts is important for web development, Docker deployments, system design, and scalable architectures.

Today I explored how Nginx acts as a reverse proxy, helps with load balancing, security, and routing requests to backend services.

Always learning, always building.

#Networking #DevOps #Nginx #ReverseProxy #Docker #WebDevelopment #SystemDesign #LearningJourney

MEDIA advertisement

#424 Kamlesh Kawadkar AI / LLM

API Architect & Agentic AI | GenBI Insights Platform | Backend & Distributed Systems

4mo · **View on LinkedIn**

The Workflow Doesn’t Exist Until The User Asks

While building an AI application, I noticed that the system felt slow.

The user would submit a request and the system would spend several seconds doing work before the model even started generating tokens.

At first, I assumed this was a performance problem. But after looking more closely, it turned out to be an architecture problem.

Traditional software usually executes predefined workflows.

When a customer places an order, generates an invoice, or submits a form, the system already knows what happens next.

The workflow was designed ahead of time. The request simply triggers it.

AI systems are often different.

Consider a request like:

• “Investigate declining sales in Germany.”

• “Find the root cause of this incident.”

• “Generate insights from this dataset.”

The system doesn’t immediately know what to do?

It first has to determine:

• Which data sources are relevant?

• Which tools should be called?

• What information is missing?

• What sequence of actions should be executed?

• When is enough evidence available to answer?

Before any useful response can be generated, the system has to construct a workflow. Only then it can execute.

This is why orchestration becomes such an important part of AI systems.

The challenge isn’t simply generating tokens. It is turning an ambiguous human request into a concrete execution plan.

Once I started looking at AI systems through that lens, a lot of architectural decisions began to make sense.

Planning.

Tool calling.

Memory.

Retrieval.

Agent coordination.

They all exist because the workflow often doesn’t exist when the request arrives. The system has to build it first.

Sharing my journey from backend to AI, I write to make AI less intimidating.

#BackendToAI #AIBuilders #AITech #AIProductEngineering #Backend #LearningInPublic #AI #AIArchitecture #Architect #Mindset #TechJourney

MEDIA KAMLESH KAWADKAR AI ARCHITECT. AI WORKFLOW. LATENCY. MODEL. SOLUTION.; advertisement

#425 Kubernetes

Cloud / K8s / DevOps

Sunke Nagarani • 3rd+

4mo · **View on LinkedIn**

Most repeated Kubernetes interview question:

Liveness Probe vs Readiness Probe

Many beginners get confused, but the difference is simple:

Liveness Probe = Is the app alive?

Readiness Probe = Is the app ready to receive traffic?

If liveness probe fails:

Kubernetes restarts the container.

If readiness probe fails:

Kubernetes stops sending traffic to the Pod.

MEDIA advertisement

#426 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Sanidhya Charan Srivastava • 3rd+

4mo · **View on LinkedIn**

Want to make Claude Code understand your company's knowledge?

Here's the simplest way to set up RAG in Claude Code:

1. Collect Your Knowledge

Gather:

• Documentation
• PDFs
• Architecture Diagrams
• API Specifications
• Coding Standards
• Jira Tickets
• Internal Wikis

2. Convert Everything into Searchable Chunks

Break large documents into smaller pieces that can be efficiently retrieved.

3. Create Embeddings

Transform each chunk into vector representations so semantic search becomes possible.

4. Store in a Vector Database

Use a vector database such as Chroma, Qdrant, Pinecone, or Azure AI Search.

5. Build a Retrieval Tool

Create a tool that can search your vector database and return the most relevant information.

6. Connect the Tool to Claude Code using MCP

Expose your retriever through MCP (Model Context Protocol) so Claude Code can access your knowledge base.

7. Define Retrieval Rules

Tell Claude Code:

✓ Search the knowledge base first

✓ Use retrieved information as the source of truth

✓ Mention when information is unavailable

8. Test with Real Questions

Ask:

• How does our authentication flow work?

• What is our deployment process?

• Which API should I use for payments?

If Claude answers correctly using company knowledge, your RAG pipeline is working.

Final Result

Without RAG:

Claude Code knows general programming.

With RAG:

Claude Code becomes an AI Engineer that understands YOUR company.

That's where the real productivity gains begin.

Repost this to help more AI Engineers learn practical RAG implementation.

Follow Sanidhya Charan Srivastava for AI Engineering, Agents, RAG, and Claude Code content.

#AI #ClaudeCode #RAG #MCP #GenerativeAI #AIEngineering #AgenticAI #LLM

MEDIA advertisement; document/carousel (possible)

#427 Swetha Telu

Java / Spring

Lead Engineer @ Cognizant | 12+ Years in Java Development | Spring Boot | Microservices | REST | Event-Driven Architecture | Supply Chain Domain |Banking Domain | Health Care Domain

4mo · **View on LinkedIn**

For a Java 8 interview, these 30+ programs test your understanding of:

✓ Collections

✓ Streams API

✓ Lambda

✓ Functional Interfaces

✓ Optional

✓ Sorting

✓ Grouping

✓ Mapping

✓ Real-world data processing

A good Java backend engineer should be able to write these without memorizing - by understanding the flow:

Source ← Stream → Operation → Result.

#428 Sivasankar Natarajan

AI / LLM

Technical Director | GenAI Practitioner | Azure Cloud Architect | Data & Analytics | Solutioning What’s Next

4mo · View on LinkedIn

What is MCP (Model Context Protocol)?

Most AI agents are trapped inside their own walls.

MCP is the protocol that connects them to the outside world data sources, tools, and workflows.

What is MCP?

• MCP is an open-source standard that connects AI applications to external systems like data sources, tools, and workflows.

• It enables seamless integrations, allowing AI models like ChatGPT to access data, use tools, and perform tasks like web app creation or database queries.

• MCP simplifies development, reducing complexity and time by providing a standardized way to connect AI systems to various resources.

• It enhances AI capabilities, making models more powerful and personalized by allowing them to interact with external systems and data on behalf of users.

Before MCP:

LLM → Slack, Google Drive, GitHub (separate connections for each).

Every integration is custom. Every tool requires its own API client. Every agent reinvents the wheel.

After MCP:

LLM → Unified API (MCP) → Slack, Google Drive, GitHub.

One protocol. One connection layer. Every tool accessible through a standardized interface.

How MCP Works?

User → User Query → MCP Client → Invoke Graph → LangGraph → Route Request → OpenAI GPT → Tool Decision → Call MCP Tool → MCP Server → External API Call → External APIs → API Response → MCP Server → Tool Result → OpenAI GPT → Generate Response → MCP Client → Natural Language Response → Final Result User → Agent Response → User.

The Flow:

1. User sends a query to the MCP Client.

2. MCP Client invokes LangGraph to route the request.

3. OpenAI GPT makes a tool decision and calls the MCP Tool.

4. MCP Server makes an external API call to the appropriate service (Slack, Google Drive, GitHub, etc.).

5. External API returns a response to the MCP Server.

6. MCP Server sends the tool result back to OpenAI GPT.

7. OpenAI GPT generates a natural language response.

8. MCP Client delivers the final result to the user.

Before MCP, every agent built its own integrations. After MCP, every agent shares the same connection layer.

MCP is the protocol that turns isolated AI models into connected AI agents.

Are you building AI agents with custom integrations or with MCP?

Repost this to help your network get started

+ Follow for Sivasankar more

#MCP #LLMs #OpenSource

#429 Gabriel Millien

AI / LLM

Enterprise AI Execution Architect | Closing the AI Execution Gap | $100M+ in AI-Driven Results | Trusted by Fortune 500s: Nestlé • Pfizer • UL • Sanofi | AI Transformation |Board Member | Fractional CAO | Keynote Speaker | View my services

4mo · View on LinkedIn

Everyone is rushing to deploy AI agents.

Almost no one is asking what they give up to get one.

Not capability. You gain that.

What you trade away is the checkpoint before consequence.

The diagram below draws the three types of AI as data flows. Save it. Then look at where the human stands in each.

Predictive AI. The forecaster.

Labeled data in, a prediction out. Demand, churn, fraud, risk. It proposes a number. You decide what to do with it. You check before anything happens.

Generative AI. The responder.

A model tuned on your data, wired to your tools. You ask, it produces. The answer is fluent, which makes it harder to verify, but you still read it before you act on it. The checkpoint is still there, just weaker.

Agentic AI. The actor.

A coordinator running other agents that use tools and take steps on their own. It does not propose or produce. It does. By the time you see the result, the action already happened.

Now look at what moved.

It is not just that capability climbed.

It is that the moment where a human could catch the error moved from before the consequence to after it.

Predictive lets you verify, then act.

Generative lets you read, then act.

Agentic acts, and lets you review.

That is the real decision hiding inside this diagram.

Choosing an AI type is choosing when you get to intervene.

Earlier is safer and slower. Later is faster and far less forgiving.

This is why agentic AI is not simply the advanced version of the other two.

It is the version that moves your control from prevention to cleanup.

That can be the right trade. For low-stakes, reversible work, reviewing after is perfectly fine, and the speed is worth it.

It is the wrong trade for anything you cannot easily undo.

So before you approve an agent, ask the question the diagram is really posing.

If this acts before I can check it, can I live with what it does when it is wrong.

If the answer is yes, build the agent.

If the answer is no, you do not need a better model.

You need the checkpoint back.

Save this as your AI systems map.

Repost so a leader in your network chooses AI by where they can intervene, not by what demos best.

Follow Gabriel Millien for daily insights on closing the AI Execution Gap

Credit: original diagram by Rahul Agarwal, give him a follow.

#430 Vikram Kumar AI / LLM

Senior Product Manager | FinTech + AI | Card & Wallet Payments | Transaction Systems & Authorization | API, ISO8583 | CSPO® & CSM®

4mo · View on LinkedIn

Tokenization in Modern Payments

Most customers think their card number travels through every payment transaction.

In many modern payment systems, it doesn’t.

Instead, a secure token is used in place of sensitive card data.

This is one of the most important technologies powering digital payments today.

Why?

Because payments must balance three goals:

Security.

Convenience.

Conversion.

Tokenization helps achieve all three.

A token is a surrogate value that represents card data without exposing the actual card number.

Even if intercepted, it is generally useless outside the environment where it was issued.

Today, tokenization is widely used across:

• E-commerce
• Mobile Apps
• Digital Wallets
• Subscription Platforms
• Recurring Payments
• Card-on-File Merchants

One common misconception is that tokenization replaces encryption.

It doesn’t.

Encryption protects data by making it unreadable.

Tokenization reduces exposure by replacing sensitive data altogether.

The strongest payment ecosystems use both.

Beyond security, tokenization delivers business value.

It can help:

• Reduce fraud
• Lower PCI compliance scope
• Improve customer trust
• Enable smoother checkouts
• Support recurring payment success
• Improve approval performance

Modern tokenization involves multiple participants:

• Merchants
• Payment Platforms
• Acquirers
• Card Networks
• Issuers
• Token Service Providers

Together, they manage token issuance, storage, validation, and lifecycle controls.

Where AI can help

AI can assist with fraud analytics, anomaly detection, token lifecycle monitoring, risk scoring, and operational insights.

However, AI complements tokenization.

It does not replace vault security, governance controls, or compliance frameworks.

Product Management Insight

The best payment technologies are often invisible.

Customers rarely notice tokenization.

They simply experience secure, seamless, and reliable payments.

That is exactly the outcome the industry wants.

The infographic explains token types, tokenization flow, ecosystem participants, security controls, compliance considerations, business impact, common misconceptions, and future trends used across modern payment platforms.

What do you think has been the biggest driver of tokenization adoption: security, compliance, customer experience, or approval performance?

Like | Comment | Repost

For deeper learning, review the infographic and follow the series.

#Payments #FinTech #Tokenization #DigitalPayments #CardPayments #MerchantAcquiring #PaymentSecurity #ProductManagement #Banking #PaymentGateway #PaymentProcessing #PaymentArchitecture #Visa #MasterCard #Rupay #FintechInnovation #CustomerExperience

#431 Madhur Shyam Maurya

System Design

AI Solutions Architect | GenAI & LLM Engineer | RAG & Agentic AI Expert | Engineering Manager at TATA Advanced Systems Limited

4mo · **View on LinkedIn**

15 Architecture Types

1) Monolithic

2) Layered

3) Hexagonal

4) Clean Architecture

5) Microservices

6) Event-Driven

7) CQRS

8) Serverless

9) Pipeline

10) Space-Based

11) RAG

12) Agentic AI

13) Multi-Agent Systems

14) Data Mesh

15) Cloud-Native

#432 Alex Xu

AI / LLM

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

4mo · **View on LinkedIn**

SLMs vs. LLMs, Clearly Explained

Big models cost more. Small models do less. Here's how SLMs and LLMs differ across the dimensions that matter in production:

1. Architecture: SLMs are usually under 10B parameters and run on a laptop or phone. LLMs sit at 10B+ with deeper layers and more attention heads, built for broad reasoning across tasks.

2. Task Complexity: SLMs work well on simple tasks but fail on complex multiple reasoning steps. LLMs handle difficult math, multi-step code, and long-horizon planning.

3. Long Context Recall: SLMs lose the thread across long documents or extended conversations. LLMs reliably track and connect information across large inputs.

4. Latency and Cost: SLMs run on consumer hardware with low response times and significantly lower inference costs. LLMs require GPU and carry higher costs per request.

5. Deployment and Privacy: SLMs run on-device or on-premise. LLMs are typically cloud-hosted, which adds data governance complexity.

6. Where each fits:

SLMs: on-device assistants, real-time classification, or privacy-sensitive applications

LLMs: complex reasoning, agent workflows, or broad knowledge tasks.

Are you using SLMs, LLMs, or a hybrid setup in production?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gF7Vsw6X

#systemdesign #coding #interviewtips

MEDIA document/carousel (possible)

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gF7Vsw6X

#433 Venkata Yugandhar Babu Teki

System Design

Senior Full Stack Java Developer @ Wells Fargo | Core Java ,Microservices, Spring boot ,Restful API, Agentic AI, React ,KAFKA,AWS/GCP, AI, hibernate, Oracle,

4mo · **View on LinkedIn**

The most expensive architecture mistake isn't choosing Kafka.

It's choosing Kafka when you needed Redis.

Or choosing Redis when you needed Kafka.

Every developer eventually reaches that moment.

The application is growing.

Traffic is increasing.

Microservices are multiplying.

And someone asks:

"Should we use Kafka or Redis?"

Suddenly the meeting becomes a battlefield.

Team Kafka:

Event Streaming

Durability

Scalability

High Throughput

Team Redis:

Blazing Fast

Microsecond Latency

Caching

Real-Time Operations

And then a senior engineer quietly says:

"Why are we acting like they're competitors?"
Because the truth is:
Kafka and Redis solve completely different problems.
Kafka is like a highly organized logistics company.
It records every event.
Stores everything safely.
Delivers messages reliably.
And lets you replay history whenever needed.
Redis is like having a genius assistant sitting beside your application.
Need data?
Here.
Need a session?
Here.
Need a leaderboard?
Here.
Need a cache hit in milliseconds?
Already done.
The funniest thing about system design interviews is that candidates often try to replace one with the other.
Meanwhile, real-world architectures use both.
A common production flow looks like:
Events → Kafka → Services → Redis Cache → Users
One handles the movement of data.
The other handles the speed of data.
And together they're a ridiculously powerful combination.
That's why experienced engineers don't ask:
✗ "Which technology is better?"
They ask:
✓ "Which problem am I trying to solve?"
Because great architecture is rarely about choosing the strongest tool.
It's about choosing the right tool.
And sometimes...
The right answer is both.
So let's settle a backend debate.
#Kafka #Redis #SystemDesign #BackendDevelopment #SoftwareEngineering #Microservices #DistributedSystems #Java #SpringBoot #SoftwareArchitect #BackendEngineer #EventDrivenArchitecture #ScalableSystems #CloudArchitecture #TechLead #EngineeringManager #DevOps #PerformanceEngineering #TechnicalRecruiter #HiringDevelopers

MEDIA advertisement

#434 Utsav Bhanderi

System Design

.
4mo · **View on LinkedIn**

Modern applications are not powered by APIs + databases alone anymore.
At internet scale, millions of events are generated every second:
-> Orders

-> Payments
-> Notifications
-> Tracking updates
-> User activity
-> Recommendations
Directly writing everything into SQL becomes a bottleneck.
This is where Apache Kafka becomes the backbone of real-time systems.
Why Kafka is used by large-scale systems:
✓ High Throughput
✓ Low Latency
✓ Event Streaming
✓ Distributed Architecture
✓ Horizontal Scalability
✓ Fault Tolerance
✓ Real-Time Data Processing
✓ Decoupled Microservices
✓ Reliable Message Queues
✓ Stream Processing
Real-world use cases:
- Food Delivery Apps
- Ride Booking Platforms
- Banking Systems
- E-Commerce Platforms
- Fraud Detection
- Analytics Pipelines
- IoT Systems
- Notification Engines
- Recommendation Systems
Core Idea:
Traditional Architecture:
- Application → Database → Bottleneck
Modern Event-Driven Architecture:
- Application → Kafka → Multiple Independent Consumers
Kafka helps systems:
- process millions of events reliably
- reduce service dependency
- scale independently
- survive heavy traffic spikes
- build resilient distributed systems
The more I explore system design, the more I realize:
Scalability is not about handling more users only.
It is about designing systems that continue performing under pressure.
#Kafka #ApacheKafka #SystemDesign #BackendDevelopment #Microservices #DistributedSystems #EventDrivenArchitecture #SoftwareEngineering #Java #SpringBoot #ScalableSystems #RealTimeSystems #BigData #CloudComputing #DevOps #Programming #Coding #Developer #Tech #Engineering #DataEngineering #Architecture #FullStackDeveloper #AI #Automation #OpenSource #LinkedInTech #TechCommunity

MEDIA Infographic explaining how Apache Kafka powers real-time scalable systems. The poster shows an event-driven architecture where producers such as driver, restaurant, and product services send events into Apache Kafka. Kafka streams and stores millions of events, which are then consumed by services like notifications, analytics, payments, recommendations, and monitoring. The image highlights Kafka benefits including high throughput, scalability, fault tolerance, decoupled architecture, and real-time processing. It also compares traditional database-centric architecture versus modern Kafka-based distributed systems. Social media links for LinkedIn and GitHub are displayed at the bottom in a vertical 9:16 layout suitable for LinkedIn and mobile viewing.

#435 Abdulla Basha Shaik

AI / LLM

Principal Software Engineer | Cloud-Native Software Architect | Java, Spring Boot, Kafka & Kubernetes | AI Agents & Enterprise Modernization

4mo · View on LinkedIn

Spring AI Ecosystem: Building Production-Ready AI Applications in Java

Many developers think building an AI application is simply:

User → LLM → Response

That may work for demos.

It won’t work in production.

As AI applications mature, the architecture is becoming much bigger than just model integration.

A production-ready AI system requires:

✓ Model Providers (OpenAI, Gemini, Claude, Llama, Mistral)

✓ Spring AI Components (Chat, Embeddings, Vector Stores, Tool Calling)

✓ Agent Frameworks (Spring AI Agents, LangChain4j, LangGraph4j, CrewAI)

✓ Vector Databases (pgvector, Qdrant, Weaviate, Pinecone)

✓ Memory & State Management

✓ Workflow Orchestration (Temporal, Camunda, Argo)

✓ Observability & Monitoring (Micrometer, OpenTelemetry, Grafana)

✓ Evaluation & Guardrails

✓ Cloud-Native Deployment

What I find interesting is how Spring AI is evolving beyond being an LLM integration framework.

It is becoming the central orchestration layer that allows Java developers to combine:

Models

Knowledge Sources

Agent Workflows

Memory

Observability

Guardrails

into a single production-ready platform.

The AI stack is starting to resemble what happened with microservices years ago.

Initially, everyone focused on APIs.

Later, we realized we needed:

• Service Discovery

• Security

• Observability

• Resilience

• Distributed Tracing

AI applications are following the same path.

Connecting to an LLM is only the beginning.

The real engineering challenge starts when you need to build:

→ Reliable Agents

→ Long-Running Workflows

→ Memory Systems

→ Evaluation Pipelines

→ Production Monitoring

The future of Java development is not just microservices.

It’s AI-powered, agent-driven applications built on the Spring ecosystem.

Which area of the Spring AI ecosystem are you currently exploring?

#SpringAI #Java #SpringBoot #AIEngineering #GenAI #LLM #AIAgents #RAG #SoftwareArchitecture #AgenticAI

#436 Karthikeyan Yuvaraj

AI / LLM

Senior Solutions Architect | Cloud & GenAI Architect | Driving Enterprise-Scale AI Adoption | Data & Analytics | RPA

4mo · **View on LinkedIn**

15 Practical LLM Cost Optimization Techniques Every AI Engineer Should Know

As organizations move from AI experimentation to production, the biggest challenge is no longer model capability—it's cost efficiency at scale.

Here are some techniques I've seen consistently deliver value in enterprise AI implementations:

✓ Use the smallest model that can solve the task

✓ Route requests dynamically based on complexity (Multi-Model Routing)

✓ Trim conversation history and optimize context windows

✓ Summarize long conversations instead of storing full chat history

✓ Control output token limits

✓ Implement RAG to retrieve only relevant information

✓ Cache responses for repeated queries

✓ Cache prompts and reusable system instructions

✓ Use semantic caching for similar requests

✓ Enforce structured outputs (JSON/XML) to reduce token usage

✓ Batch requests wherever possible

✓ Add agent guardrails to prevent runaway executions

✓ Adopt a Tool-First architecture

✓ Classify queries before sending them to expensive models

✓ Continuously monitor AI spend with cost dashboards

My experience:

In enterprise AI projects, the biggest savings often come from a combination of:

1. Smart model routing

2. Context optimization

3. Effective RAG implementation

4. Caching strategies

These four areas alone can reduce LLM costs significantly while maintaining response quality.

As AI adoption grows, cost optimization is becoming as important as prompt engineering.

What techniques have worked best in your AI projects?

#ArtificialIntelligence #GenerativeAI #LLM #AIEngineering #MachineLearning #OpenAI #AzureOpenAI #AWSBedrock #RAG #AIAgents #PromptEngineering #MLOps #AITech #DigitalTransformation

#437 Mithun Venkatesan

AI / LLM

M.Tech CSE (Data Science) @ VIT | GenAI / LLM | Python | FastAPI | Backend Development | Prompt Engineering | AI/ML

4mo · View on LinkedIn

Everyone is talking about smarter AI models.

But after understanding Model Context Protocol (MCP), I started thinking the bigger challenge might not be intelligence.

It might be interoperability.

Imagine a world without common standards.

Every AI application would need its own custom integration for every database, API, SaaS platform, and internal company system.

Cursor would need one integration.

Claude would need another.

ChatGPT would need another.

The complexity grows faster than the intelligence.

That's what made MCP click for me.

MCP isn't trying to make models smarter.

It's trying to make them easier to connect.

A common language between AI systems and the tools they need to interact with.

In many ways, it reminds me of what HTTP did for the web.

Websites became powerful because they could communicate through a shared standard.

AI systems may scale for the same reason.

The more I learn AI engineering, the more I realize that the future may not belong to the model with the highest benchmark score.

It may belong to the ecosystem that connects models, tools, APIs, databases, and workflows most effectively.

Infrastructure rarely gets the spotlight.

But infrastructure is often what changes an industry.

The internet wasn't built on websites.

It was built on protocols.

AI might be heading in the same direction.

#MCP #AIEngineering #GenAI #LLM #ArtificialIntelligence #SoftwareArchitecture #EnterpriseAI #Tech

#438 Ravena O

AI / LLM

AI Researcher and Data Leader | Healthcare Data | GenAI | Driving Business Growth | Data Science Consultant | Data Strategy

4mo · View on LinkedIn

AI agents don’t fail loudly.

They fail quietly when memory is ignored.

Context drift is one of the biggest reasons AI agents produce weak or inconsistent results in production.

If we want reliable enterprise AI agents, memory cannot be an afterthought.

1. Working Memory

Keeps active context inside the model window.

✓ Best for: Real-time reasoning and short tasks

(!) Watch for: Context limits causing silent failures

2. Episodic Memory
Stores and retrieves past interactions.
✓ Best for: Conversational AI with recall
(!) Watch for: Irrelevant or outdated retrievals
3. Semantic Memory
Captures structured facts from interactions.
✓ Best for: Personalization and long-term agents
(!) Watch for: Incorrect or stale information
4. Procedural Memory
Retains reusable workflows and processes.
✓ Best for: Automating repeatable business tasks
(!) Watch for: Scaling flawed workflows without validation
5. Hierarchical Memory
Uses hot, warm, and cold storage layers.
✓ Best for: Large-scale, long-running systems
(!) Watch for: Slow retrieval and latency issues
6. Prospective Memory
Tracks future tasks, reminders, and commitments.
✓ Best for: Autonomous multi-step workflows
(!) Watch for: Missed triggers or scheduling gaps
7. Shared Memory
Enables multiple agents to work from common knowledge.
✓ Best for: Multi-agent collaboration
(!) Watch for: Conflicts without version control or locking
Most teams are still focused on improving models.
Enterprise teams are designing better memory systems.
Because in production, what an AI agent remembers often matters more than what it knows.
CC: Rakesh Gohel
#AI #AIAgents #EnterpriseAI #LLM #MachineLearning #Automation #GenAI

#439 **Surajit P.** AI / LLM
Sr Technical Specialist @ Volkswagen - IT | Software Development | SAFe Certified
4mo · **View on LinkedIn**

Which LLM should you actually use? ChatGPT vs. Grok vs. Gemini vs. Claude vs. Perplexity
The AI landscape is moving so fast that picking the right tool for the job has become a skill in itself. Choosing the wrong model can mean missing critical facts, getting dry writing, or losing integration with your business stack.
Here is a quick cheat sheet on how the top 5 models stack up based on their architecture and optimal strengths:
1. ChatGPT (OpenAI)
• Best for: Creative exploration, content generation, coding, and day-to-day productivity automation.
• Core Edge: Superior multimodal inputs, custom GPT extensions, and robust long-term context memory.
• Pro-Tip: Use custom GPTs to lock down your exact tone, data formatting, and repeating workflows.
2. Grok (xAI)
• Best for: Real-time trend discovery, pop culture tracking, and unfiltered conversational insight.

• Core Edge: Native, instant access to the X (formerly Twitter) data stream for breakneck news cycles.

• Pro-Tip: Leverage Grok when you need sharp, punchy social copy that captures a live global narrative.

3. Gemini (Google)

• Best for: Deep ecosystem integration within Google Workspace (Docs, Sheets, Slides, Drive).

• Core Edge: Flawless processing of structured business files and live web search anchoring.

• Pro-Tip: Use it to instantly organize unstructured data files straight from your Google Drive into ready-made docs.

4. Claude (Anthropic)

• Best for: Advanced text comprehension, long-form logic, and highly nuanced legal/research briefs.

• Core Edge: Massive 200k+ token window and an ultra-balanced, human-sounding tone with safety guardrails.

• Pro-Tip: Drop in massive 100-page reports or entire codebases to let Claude rewrite, summarize, or audit them with absolute precision.

5. Perplexity (Perplexity AI)

• Best for: Academic research, conversational search, deep fact-checking, and rapid verification.

• Core Edge: Inline citation engines that back up every single sentence with direct web sources.

• Pro-Tip: Stop Googling manually when researching complex things; Perplexity construct a cited source-backed outline instantly.

#ArtificialIntelligence #GenerativeAI #ChatGPT #Productivity #TechLeadership #LearnAI #Post_1

#440 Chris Donnelly

AI / LLM

Co Founder of Searchable.com | Follow for posts on Business, Marketing, Personal Brand & AI

4mo · **View on LinkedIn**

All people want to know is which LLM is "the best."

There's no simple answer. But here's my take:

You might've seen me do this post before a couple of times.

And I like to do an updated version every few months.

Because AI is evolving so fast that recommendations from 6 months ago are ancient news.

6 months ago, ChatGPT was generally seen as the leading product.

Then, as you might've seen... Claude took over the market.

However, even now, we're seeing a mini resurgence for GPT.

With the likes of GPT 5.5, Image 2.0, and Codex.

But they all have their strengths and weaknesses.

So here's how I see it in 2026:

(See the sheet for further details ↓)

1. Claude Opus 4.8

The model I still rely on the most.

Key features: Strong reasoning and long-form thinking.

Pros: Consistent and dependable outputs.

Cons: Can feel slower for rapid brainstorming.

New updates: Opus 4.8, Claude in Excel, PowerPoint & Chrome, and Claude Design.

2. ChatGPT (GPT 5.5)

The all-rounder with the biggest ecosystem.

Key features: Build as an agentic worker that plans and checks its own work.

Pros: The largest tool and integration ecosystem.
Cons: Older models retired fast, around three months' access.
New updates: GPT 5.5, Images 2.0, and Codex.
3. Gemini 3.5 Flash
Google's everywhere AI, now genuinely agentic.
Key features: The new default in the Gemini app and AI Mode in Search.
Pros: Best speed-to-intelligence ratio on the market.
Cons: The flagship 3.5 Pro is not out yet.
New updates: Gemini Omni, Nano Banana 2 Pro, Gemini Spark, Daily Brief.
4. Perplexity
The cited-answer engine, now doing the work too.
Key features: Real-time answers with clear citations.
Pros: Transparent sourcing cuts the guesswork.
Cons: Lighter on creative generation.
New updates: Perplexity Computer, Computer in Microsoft 365, Personal Computer on Mac, and iOS Launch.
5. Grok 4.3
Shipping faster than anyone else in AI.
Key features: New architecture with a 1M-token context.
Pros: Live data straight from X and the web.
Cons: Best features locked to SuperGrok and Premium+ tiers.
New updates: Skills, Grok Build, Connectors, and Imagine Agent Mode.
6. Kimi K2.6 (Moonshot AI)
The open-source powerhouse from China.
Key features: A trillion-parameter open-source model.
Pros: Open-source and far cheaper than the Western flagships.
Cons: Smaller ecosystem, with English-first polish still catching up.
New updates: Agent Swarm, Vibe Coding & Design, Kimi Work, WebBridge.
All those updates have landed in the last 3 months. Absolutely wild.
P.S. If you want to keep up to date on all features,
Every tool here has an updates page on their website (bar Kimi).
Worth keeping an eye out on!
Which LLMs are you using? Disagree with my takes?
Let me know in the comments ↓

If you want a high-res PDF of this guide:
1. Follow Chris Donnelly
2. Save the post.
3. Repost to your network.
4. Subscribe to: https://lnkd.in/eUTCQTWb

LINKS

- http://searchable.com
- https://lnkd.in/eUTCQTWb

#441 Rajesh Ravindran

AI / LLM

Business Intelligence & Data Analytics Professional | Dashboards, MIS & Process Automation

4mo · View on LinkedIn

7 AI agent concepts that will define how you build in 2026:

(Save this — you'll reference it more than once)

1. Orchestration Layer

The conductor of your agent system. It decides which agents run, in what order, and what gets passed between them. Without a solid orchestration layer, multi-agent systems turn into chaos fast.

2. Tool Calling

How agents actually do things. When an LLM decides it needs to search the web, run code, or query a database — it doesn't do it directly. It calls a tool. The difference between a chatbot and an agent lives here.

3. Context Window Management

The silent bottleneck nobody talks about enough. Every agent has a limit on what it can "see" at once. Smart agents chunk, summarize, and prioritize information so they don't hit the ceiling mid-task.

4. Human-in-the-Loop (HITL)

The checkpoint. Agentic systems can run autonomously — but the best ones know when to pause and ask a human before taking an irreversible action. This is what separates reliable agents from risky ones.

5. Agent Handoff

When one agent finishes its job and passes state to the next. Done badly, you lose context. Done well, it's invisible. Most production failures in multi-agent systems trace back to broken handoffs.

6. Guardrails

The rules baked into your agent to prevent it from going off-script. Input validation, output filtering, scope limits. Not optional. Not an afterthought.

7. Deterministic vs. Non-deterministic Steps

Some parts of your pipeline should always return the same output (API calls, database reads). Others are intentionally variable (LLM reasoning). Knowing which is which — and designing accordingly — is the mark of a senior AI engineer.

Most people are still building agents without thinking about steps 4, 5, 6 and 7.

That's where things break.

What would you add to this list? ↓

#AIAgents #ArtificialIntelligence #MachineLearning #LLM #GenAI #AgenticAI

#442 Janjirala Anusha

AI / LLM

Technical Program Lead | AI, DevOps & Cloud | 15.5k+ Followers

4mo · View on LinkedIn

Top 8 AI Models That Can Help You Work Smarter in 2026

Artificial Intelligence is evolving rapidly, and choosing the right AI model can significantly improve productivity, learning, research, and development workflows.

Here are 8 powerful AI models worth exploring:

ChatGPT – Great for content creation, coding, problem-solving, and productivity.

Claude – Excels in long-form writing, document analysis, and detailed reasoning.

Gemini – Strong multimodal capabilities with seamless integration across Google tools.

DeepSeek – Open-source powerhouse for coding, mathematics, and technical tasks.

Grok – Real-time insights and trending information with a unique conversational style.

Perplexity – Perfect for research, fact-checking, and source-backed answers.

Mixtral – Efficient open-weight model delivering strong performance with fewer resources.

Llama – Flexible open-source model enabling developers to build custom AI applications.

Key Takeaway:

There is no single "best" AI model. The right choice depends on your specific use case:

✓ Content Creation → ChatGPT, Claude

✓ Research & Learning → Perplexity, Gemini

✓ Coding & Development → ChatGPT, DeepSeek, Mixtral

✓ Open-Source Projects → Llama, Mixtral, DeepSeek

✓ Real-Time Information → Grok, Perplexity

The future belongs to those who learn how to collaborate with AI, not compete against it.

Which AI model do you use the most, and why? Share your experience in the comments! ↓

#ArtificialIntelligence #AI #ChatGPT #Claude #Gemini #DeepSeek #Perplexity #Llama #Grok #Mixtral #MachineLearning #GenerativeAI #Tech #Innovation #Productivity #Learning #AnushaJanjirala

MEDIA document/carousel (possible)

#443 Ajit Kumar

System Design

Java Architect • Payments Domain • AWS Certified Solutions Architect/Developer/AI Practitioner(Foundational)• Agentic AI (Antigravity & Codex) • Microservices

4mo · **View on LinkedIn**

A standard, local in-memory lock (like Java's synchronized block or ReentrantLock) only works within a single Java Virtual Machine (JVM). In a modern cloud-native system, our microservice is scaled out across multiple independent instances (containers or nodes) running behind a Load Balancer. A Distributed Lock ensures that only one microservice instance can access a shared resource across the entire distributed cluster.

MEDIA No alternative text description for this image

#444 Abhishek Pandey

Other

Payments SME | AI Product Owner | API | ISO20022 | AI-Native Digital Transformation | ISO20022

4mo · **View on LinkedIn**

Not All Payments Are the Same... And That's Exactly What Makes Payments So Interesting.

From a simple UPI transfer to a complex cross-border corporate payment, every transaction follows a different path based on geography, speed, payment method, and business purpose.

Understanding these payment types isn't just theory—it's the foundation for designing better products, solving production issues, improving customer experience, and building scalable payment systems.

Some key categories: ✓ Domestic vs Cross-Border Payments

✓ Real-Time, Near Real-Time & Deferred Settlement

✓ Account-to-Account, Cards, Wallets & Cash

✓ P2P, B2B, B2C & Government Payments

The more you understand the payment landscape, the easier it becomes to connect technologies like ISO 20022, SWIFT, SEPA, UPI, RTGS, NEFT, ACH, FedNow, and RTP into one complete picture.

Every payment tells a story. The better we understand that story, the better systems we build.

Follow Payment Inovation for simple, practical insights into Payments, ISO 20022, SWIFT, FinTech, and Banking Architecture.

#Payments #ISO20022 #FinTech #Banking #PaymentInnovation

MEDIA advertisement

#445 Hina Arora

AI / LLM

Helping Tech Leaders Build Their Personal Brand and Industry Authority | Founder @BrandBuilders | Tech / AI / Career Content Creator (500K+) | 10+ YEO in Tech & AI | Ex-Engineering Manager @Jio

4mo · View on LinkedIn

Are You Still Writing Custom Integrations for Every Tool Your AI Agent Needs?

Every agent eventually hits the same wall: it needs databases, email, search, chat and writing custom integrations for each becomes a full-time job.

MCP (Model Context Protocol) is the standard quietly fixing this.

What is the mental model?

• MCP Host: The environment running your agent (Claude Desktop, IDE, your app).

• MCP Clients: One per tool, each speaking the MCP protocol.

• MCP Services: Wrappers around actual tools.

• External Tools: The real systems like Postgres, Gmail, Calendar, Slack.

Same protocol, every connection. The agent connects to MCP clients, each client talks to one service, each service connects to one external tool.

How does a request actually flow?

Twelve steps that look like one from the user's perspective: "I asked, it answered."

Why does MCP matter?

1. Open and Standard: Same protocol regardless of model, tool, or framework. Connect once, reuse everywhere.

2. Easy Integrations: No rewriting when you switch from OpenAI to Claude or vice versa. Connect once, reuse forever.

3. Faster Development: The boilerplate around "how do I let my agent use this tool" mostly disappears.

4. Smarter Systems: The model isn't smarter. The system around it can do much more.

What patterns are worth noticing?

• MCP is doing for AI tools what USB did for hardware. Before USB, every device had its own connector. Before MCP, every integration was custom. You stop solving the same problem 50 times.

• The protocol separates intent from implementation. The model says "send an email." The MCP server figures out how against the specific API. Change the API the agent doesn't care.

• It enables ecosystems, not just integrations. Community-built MCP servers install like browser extensions. That's already happening.

MCP is not a flashy capability. It is plumbing and it is quickly becoming the plumbing that makes agents portable, composable, and deployable across stacks. Teams understanding this early build agents that survive the next model swap. Teams ignoring it build integrations they will rewrite in 12 months.

How are you connecting your agents to tools today?

————————————————————————

I (Hina Arora) help tech executives to position them online that helps them to grow professionally.

DM me ‘Branding’ if you want to position yourself on LinkedIn.

#MCP #AgenticAI #AIArchitecture

#446 Saju Thomas

AI / LLM

Cloud & AI Solution Architect | Azure Solution Architect | Microservices | AKS | Azure OpenAI | Enterprise Integration | DevOps | 16+ Years

4mo · View on LinkedIn

Enterprise AI Architecture — From Prompt to Production

Most Enterprise AI systems fail not because of the model, but because the architecture around the model is incomplete.

A prompt alone is not an enterprise AI system.

Production-grade AI requires trusted data, orchestration, security, observability, governance, and execution layers working together.

Here’s a practical view of Enterprise AI Architecture ↓

1. User & Experience Layer

Web, mobile, copilots, APIs, enterprise apps

→ Secure and seamless AI interaction

2. API Gateway & Security Layer

Authentication, RBAC, token governance, rate limiting

→ Govern AI before execution

3. Prompt Orchestration Layer (The Brain)

Prompt routing, agent workflows, tool calling, model selection

→ Where prompts become intelligent workflows

4. Enterprise Knowledge Layer (RAG)

Documents, SOPs, SharePoint, databases, enterprise context

→ AI becomes context-aware, not generic

5. AI Model Layer

GPT models, fine-tuned models, small/domain-specific models

→ The model is important — but not the architecture

6. Guardrails & Responsible AI

Prompt injection protection, hallucination mitigation, compliance, content filtering

→ Trust cannot be optional

7. Execution & Tool Layer

ERP, CRM, APIs, automation, workflows

→ AI should not only answer — it should act

8. Observability & AgentOps

Latency, cost, prompt quality, failures, grounding accuracy

→ If you can’t observe AI, you can’t trust AI

9. Memory & Feedback Layer

Session memory, feedback loops, personalization

→ Continuous learning over time

DevSecOps & Governance

CI/CD, prompt/model versioning, approvals, rollback strategies

→ Enterprise AI is a lifecycle, not a demo

Enterprise AI Success Formula:

Prompt + Trusted Data + Orchestration + Guardrails + Observability + Governance = Production AI

The future of AI isn’t just better models, it’s better architecture.

What layer do you think enterprises underestimate the most today?

#EnterpriseAI #AIArchitecture #AzureAI #AgenticAI #SolutionArchitecture #CloudArchitecture #GenerativeAI

MEDIA document/carousel (possible)

#447 Kubernetes

Cloud / K8s / DevOps

Sunke Nagarani • 3rd+

4mo · View on LinkedIn

Kubernetes ConfigMap vs Secret: A Common DevOps Interview Question

One frequently asked question in Kubernetes interviews for DevOps engineers is: What is a ConfigMap, what is a Secret, and when should we use each one? While many candidates are familiar with the terms, they often struggle to articulate the specific use cases.

Here's a straightforward explanation:

- **ConfigMap**: Used to store non-sensitive configuration data for applications running in Kubernetes. Examples include:
- APP_ENV=prod
- LOG_LEVEL=info
- API_URL=https://api.example.com
- FEATURE_FLAG=true
- **Secret**: Used to store sensitive data. Examples include:
- DB_PASSWORD=******
- API_TOKEN=******
- TLS_KEY=******
- PRIVATE_KEY=******

The primary purpose of using ConfigMap and Secret is to keep configuration separate from application code and Docker images, allowing the same Docker image to be used across multiple environments. For example:

- Same image + Dev ConfigMap = Dev deployment
- Same image + INT ConfigMap = INT deployment
- Same image + Prod ConfigMap = Prod deployment

Applications can utilize ConfigMaps and Secrets in various ways:

- As environment variables
- Using envFrom
- As mounted files inside the container

Key Distinction:

- ConfigMap = Normal application configuration
- Secret = Sensitive application configuration

Important Interview Point: Kubernetes Secrets are base64-encoded by default, not fully encrypted. Therefore, for production, Secrets should be protected with:

- RBAC
- Encryption at rest
- External Secrets Manager
- Sealed Secrets
- Proper namespace access control
- Secret scanning in Git

Simple Rule to Remember:

- If it is not sensitive → Use ConfigMap
- If it is sensitive → Use Secret

This topic, while small, is crucial in real DevOps, GitOps, and production deployments.

MEDIA advertisement

LINKS

- https://api.example.com

#448 Tarun Vegi

System Design

Software Engineer II at Carelon Global Solutions India | View my portfolio

4mo · View on LinkedIn

Continuing my journey of building a Wallet & Payment system from scratch.

In my previous post, I talked about the consistency challenges that appear once a transaction spans multiple services.

This week I focused on making the synchronous flow more resilient.

The first issue was straightforward:

What happens if the Payment Service becomes slow or temporarily unavailable?

To handle this, I introduced:

• Request timeouts

• Retries for transient failures

At first, this seemed sufficient.

But retries immediately exposed another problem.

Suppose the Wallet Service sends a payment request.

The Payment Service successfully processes it.

However, the response times out before reaching the Wallet Service.

From the caller's perspective, the request failed.

A retry is triggered.

Now the same payment is processed again.

This led me to implement idempotency on the Payment Service side, ensuring that multiple requests for the same transaction result in only one successful logical operation.

While implementing this, I ran into an interesting Spring behavior.

I initially tried to catch a database exception inside a method annotated with @Transactional and return a custom response.

What I learned was that some exceptions mark the transaction as rollback-only.

Even if the exception is caught, Spring still prevents the transaction from being committed and propagates a failure to the caller.

At first this felt surprising.

But it makes sense.

Without this behavior, a transaction could partially fail while appearing successful to the rest of the application.

The framework is protecting consistency.

A few key takeaways from this exercise:

• Timeouts prevent threads from waiting forever

• Retries should be used only for transient technical failures

• Retries without idempotency are dangerous

• Distributed systems introduce uncertainty, not just failures

• Understanding transaction boundaries is just as important as understanding business logic

Building these scenarios from scratch has been far more valuable than simply reading about the patterns.

Every failure mode introduces a new design decision.

#SoftwareEngineering #BackendDevelopment #SystemDesign #DistributedSystems #Microservices #Java #SpringBoot #Idempotency #Transactional #EngineeringJourney #LearningInPublic

MEDIA advertisement

LINKS
- http://tarunvegi-dev.web.app

#449 **Neo Kim** System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

4mo · **View on LinkedIn**

If you want to get dangerously good at system design,
learn these concepts:

1 Scalability
2 Availability
3 Reliability
4 Latency
5 Throughput
6 Database
7 SQL vs NoSQL
8 Load Balancing
9 Caching
10 Cache Invalidation
11 API Design
12 REST
13 GraphQL
14 gRPC
15 Authentication
16 Fault Tolerance
17 High Availability
18 CAP Theorem
19 Consistency Models
20 Replication
21 Erasure Coding
22 Consensus
23 Leader Election
24 Secrets Management
25 RBAC
26 Sharding
27 Indexing
28 Denormalization
29 ACID
30 BASE
31 Event-Driven
32 Message Queue
33 Pub/Sub
34 Sync vs Async
35 Idempotency
36 Bulkhead
37 Retry Logic
38 Timeout

39 Service Discovery
40 API Gateway
41 Blue-Green Deployment
42 Canary Release
43 Feature Flags
44 Observability
45 Logging
46 Correlation ID
47 Monitoring
48 Alerting
49 Full-Text Search
50 Time Series
(...and many more!)
What else should make this list?
===
PS - Want a simple breakdown of each concept?
Read right now on my newsletter:
→ Part 1: https://lnkd.in/eK87NkE5
→ Part 2: https://lnkd.in/dicBp9N4
→ Part 3: https://lnkd.in/dDKk3-Xw
===
Save & repost to help others get good at system design.
Follow Neo Kim + turn on notifications.

MEDIA System Design Master Tree 2026

LINKS

- https://newsletter.systemdesign.one/join
- https://lnkd.in/eK87NkE5
- https://lnkd.in/dicBp9N4
- https://lnkd.in/dDKk3-Xw

#450 Tarun Khandagare

AI / LLM

SDE2 @Microsoft | YouTuber | 140K+ Followers | Not from IIT/NIT | Public Speaker
4mo · **View on LinkedIn**

After 2+ yrs at Visa as a Senior SDE, I decided to switch again so I gave an interview at Microsoft...
& Recursion almost took away my dream offer from me!!
I'd already worked at tech giants like Ola, Cognizant, and Visa.
And I thought, "How hard could my next senior-level interview be?"
But the moment Recursion came up, I froze, not because I didn't know it, but because I couldn't always explain it.
We all know that DSA is a must, and everyone says to prepare it.
But no one talks about the one thing that quietly haunts even experienced devs → Recursion.
Base case, Recursive case, Stack frames, Backtracking, and what not.
For the longest time, recursion felt like some kind of magic, it worked... but I couldn't tell how or why it worked.
It only started to make sense when I sat down and unpacked it, layer by layer.
No shortcuts. Just honest practice.

And since then I have learned that recursion isn’t impossible, it’s just never explained in a way that clicks.

But here’s what helped me a lot ↓

→ Drawing recursive trees instead of writing code first

→ Revisiting common patterns like Subsets, N-Queens, and Tree problems

→ Visualizing stack frames and return values at each level

So if you’re struggling with it too, trust me, you’re not alone.

Don’t pretend to “get it.” Just slow down. Trace the calls. Visualize the stack. It will come together.

One concept at a time. One dry run at a time. That’s the only way to build confidence.

And if you're working full-time and trying to level up for senior-level SDE roles in top PBCs, having a structured prep plan makes all the difference.

That’s exactly where guidance by industry experts at Bosscoder Academy can make a difference.

Explore EVOLVE for SDEs with 6+ YOE here → bcalinks.com/cMBkSnh

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and job switch support.

#seniorsde #jobswitch #microsoft #dsa #recursion #pbc #collab

LINKS

- http://bcalinks.com/cMBkSnh

#451 Java Developers Community (moderated)

Kafka / Messaging

Vladislav Grigorev • 3rd+

4mo · **View on LinkedIn**

Every backend engineer should know how the Transactional Outbox works.

Saving to the database and publishing an event looks simple.

Until one succeeds and the other does not.

That is how distributed workflows break.

1. The problem is the DB → event gap

A common flow looks like this:

• save order in DB

• publish OrderCreated to Kafka

Seems fine.

But these are two different systems.

There is a gap between them.

2. If the DB commit succeeds and publishing fails, the system is inconsistent

The order exists.

But downstream services never hear about it.

Example:

Order is stored in Postgres.

OrderCreated is never published.

Payment, notification, or inventory flow never starts.

3. If publishing succeeds but the DB write fails, you publish a lie

Now the opposite happens.

Other services think the order exists.
But the database says it does not.

4. The outbox pattern closes that gap
Instead of publishing directly to Kafka, you do this in one DB transaction:
• save business data
• save event to an outbox table
If the transaction commits, both are stored.
If it rolls back, neither exists.

5. A separate relayer publishes outbox rows
A background worker reads unpublished outbox rows and sends them to Kafka.
After success, it marks them as sent.

6. It does not mean “exactly once”
The relayer can still retry.
Consumers can still receive duplicates.
So outbox solves DB ↔ event consistency.
It does not remove the need for:
• idempotent consumers
• deduplication
• retry handling

7. In prod, the outbox is about recovery
If Kafka is temporarily unavailable, the request can still succeed.
The outbox row stays in DB.
The relayer publishes it later.
That makes the system recoverable.

8. What the outbox usually needs
A practical setup usually includes:
• outbox table
• event ID
• aggregate ID
• payload
• created_at
• status / published flag
• retry logic
• monitoring for stuck events
Because an outbox nobody watches is just another silent failure point.

Simple example:
User creates an order
→ app writes orders row + outbox row in one transaction
→ relayer reads outbox
→ publishes OrderCreated
→ marks row as sent
Now the workflow is recoverable even if Kafka is down for a while.

Simple rule:
Do not trust “save to DB, then publish event.”

If the flow must be consistent, close the gap.

Have you seen a real production issue caused by the DB → event gap? ↓

#backend #java #microservices #systemdesign #softwareengineering #kafka #architecture

#452 Divyang Panchasara

Cloud / K8s / DevOps

Cloud Architect | Agentic AI + GenAI/MLOps | LLM Productization | Distributed Systems | Integration & APIs | Data/Streaming | Observability | Semiconductor/FPGA

4mo · View on LinkedIn

As a Solution Architect, one of the biggest challenges is not designing the solution — it's communicating it quickly and clearly to stakeholders, developers, and business teams.

Recently came across this interesting open-source repo:

Cocoon AI Architecture Diagram Generator

Imagine describing your system architecture in plain English and getting a clean, professional architecture diagram in seconds. Instead of spending hours in Visio, Draw.io, or PowerPoint, architects can iterate rapidly, visualize ideas faster, and improve collaboration with teams. The output can even be shared easily as standalone HTML/PNG/PDF. (GitHub)

I see huge potential here for:

✓ Faster solution proposal discussions

✓ Better stakeholder communication

✓ Quick architecture brainstorming during workshops

✓ Keeping solution diagrams updated with evolving systems

✓ Accelerating pre-sales and architecture documentation

This could be especially powerful when combined with codebase analysis or LLM-based system understanding for cloud-native architectures (AWS / Azure / GCP / Hybrid). Curious to see where AI-driven architecture communication goes next.

#SolutionArchitecture #AI #CloudArchitecture #EnterpriseArchitecture #GenAI #SoftwareArchitecture #AWS #Azure #GCP #OpenSource

https://lnkd.in/gVXPHMQZ

MEDIA document/carousel (possible)

LINKS

- http://Draw.io
- https://lnkd.in/gVXPHMQZ

#453 Gabriel Millien

AI / LLM

Enterprise AI Execution Architect | Closing the AI Execution Gap | $100M+ in AI-Driven Results | Trusted by Fortune 500s: Nestlé • Pfizer • UL • Sanofi | AI Transformation |Board Member | Fractional CAO | Keynote Speaker | View my services

4mo · View on LinkedIn

The best AI courses are free.

Not the cheap version of good.

The actual best, published by the people building the models.

The companies behind the chips, models, and infrastructure shaping this wave run their own learning platforms. Open. Free. Maintained by the people closest to the work.

The infographic below has ten of them. A few worth flagging.

NVIDIA goes deepest on the technical side. Hugging Face is where you go to build with your hands, not just watch. Anthropic and OpenAI teach you how the model makers think their own tools should be used. Start where your actual work lives, not where the hype is loudest.

The list:

1. Anthropic: anthropic.skilljar.com
2. Google: grow.google/ai
3. Meta: ai.meta.com/resources
4. NVIDIA: https://lnkd.in/g_RgCxEq
5. Microsoft: https://lnkd.in/gAWvm9Z3
6. OpenAI: academy.openai.com
7. IBM: skillsbuild.org
8. AWS: skillbuilder.aws
9. DeepLearningAI: deeplearning.ai
10. Hugging Face: huggingface.co/learn

The platform was never the bottleneck.

I have watched people collect a folder of free certificates and learn nothing that sticks. I have watched others take one course and turn it into a working tool inside a month.

The difference was the method.

Pick one track and finish it before opening a second.

Build the thing the lesson describes, the day you learn it.

Put the result somewhere public, where being wrong costs you something.

You can buy a certificate. You cannot buy the thing it was supposed to prove.

Now the part the courses do not cover.

Knowing how a model works is not the same as making it work inside an organization.

I have seen skilled people finish every certificate on a list like this and still stall in production.

The gap was never knowledge.

It was ownership. Evaluation. Governance. The architecture that turns a working demo into a system the business will actually trust.

That distance between what you can build and what an enterprise will run is what I call the AI Execution Gap.

These platforms close the first half of it. For free.

The second half is decided by how you apply what you learn, and whether anyone owns the result once it ships.

So pick one track this week. Build something small by Friday. Then ask the question the courses skip.

Who owns this when it breaks?

Save this for your learning plan

Repost so someone in your network stops paying for what is already free

Follow Gabriel Millien for daily insights on closing the AI Execution Gap

Image Credit: Andreas Horn

LINKS

- http://anthropic.skilljar.com
- http://grow.google/ai
- http://ai.meta.com/resources
- https://lnkd.in/g_RgCxEq
- https://lnkd.in/gAWvm9Z3
- http://academy.openai.com
- http://skillsbuild.org
- http://skillbuilder.aws
- http://deeplearning.ai
- http://huggingface.co/learn

#454 SOORAJ VIDYASAGAR

AI / LLM

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

4mo · View on LinkedIn

Spring Boot + Spring AI MCP Server Architecture: Building AI-Native Applications

The Model Context Protocol (MCP) is quickly becoming a standard way to expose tools and capabilities to AI agents. If you’re a Java developer, Spring Boot + Spring AI provides a clean and scalable foundation for implementing an MCP server.

A recommended architecture looks like this:

1.Spring Boot Application

Build your MCP server as a standard Spring Boot application. It can run locally, in Docker, Kubernetes, or Azure Container Apps without architectural changes.

2.MCP Endpoint

Expose an /mcp endpoint over HTTP or Streamable HTTP to communicate with AI clients and agents.

3.AI Tool Discovery

Define tools using Spring AI annotations (such as @Tool) or registered beans. The framework can automatically expose these capabilities to the AI model.

Examples:

* calculateTotal()
* getCustomerDetails()
* generateInvoice()
* getCurrentTime()

4.Keep Business Logic in Services

The MCP layer should be a thin protocol adapter. Place all business rules inside @Service classes and inject them into your tools.

Benefits:

* Better separation of concerns
* Easier unit testing
* Reusable business logic
* Cleaner codebase

5.Test Everything

Validate:

* Tool behavior
* Service logic
* Endpoint responses
* End-to-end integrations

Use JUnit, Spring Boot Test, and integration testing to ensure reliability.

Operational Best Practices

* Use /actuator/health for health probes
* Secure MCP endpoints with Spring Security, OAuth2, JWT, or API keys
* Keep services stateless for horizontal scaling
* Add observability with Micrometer and OpenTelemetry
* Containerize with Docker and deploy to Kubernetes or Azure Container Apps

Why this architecture works

* Thin protocol layer with minimal MCP-specific code
* Clear separation between AI interface and domain logic
* Dependency Injection promotes modular design
* Easy to test and maintain

* Cloud-native and horizontally scalable

* Works seamlessly with modern DevOps and CI/CD pipelines

Key takeaway

Treat your MCP server as an adapter, not your application.

Keep the AI-facing layer lightweight, encapsulate business logic in Spring services, and let Spring Boot’s ecosystem handle dependency injection, security, testing, and scalability.

This approach results in AI-enabled applications that are clean, maintainable, and production-ready.
#SpringBoot #SpringAI #MCP #Java #AIEngineering #SoftwareArchitecture #CloudNative #Microservices #GenerativeAI #BackendDevelopment

LINKS

- http://3.AI

#455 Rignor Labs

AI / LLM

4mo · **View on LinkedIn**

AI Engineer is no longer a single role.

One AI Engineer builds products with LLMs using RAG, agents, APIs, and system design.

Another AI Engineer builds the models themselves through architecture design, training, optimization, and research.

Same title.

Different skills.

Different interviews.

Different career paths.

Know which one you’re preparing for.

#AIEngineer #ArtificialIntelligence #GenerativeAI #LLM #MachineLearning #AIAgents #PromptEngineering #RAG #MCP #SystemDesign #LLMOps #MLOps #AIProducts #AIResearch #DeepLearning #TechCareers #SoftwareEngineering #DataScience #OpenAI #FutureOfWork

#456 Rakesh Gohel

AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

4mo · **View on LinkedIn**

Vector databases just became optional for AI agents.

Reasoning-based retrieval is replacing the stack most teams spent years building...

Most teams think RAG = vectors. That assumption is now being challenged.

Vectorless RAG is a different retrieval architecture and it's worth understanding why it exists.

Here's the core problem with traditional vector RAG:

You take a 200-page document. Chunk it into fragments. Convert to embeddings. Retrieved by similarity.

The system finds what sounds relevant. Not what is relevant.

For simple Q&A - fine. For complex professional documents like financial filings, legal contracts, or technical manuals that gap between similarity and relevance is where accuracy breaks down.

So what is Vectorless RAG?

Instead of chunking and embedding, it reads the document structurally, building a hierarchical tree index that mirrors how the document is actually organized.

Chapters. Sections. Subsections. A structured Table of Contents that an LLM can reason over.

Retrieval becomes a reasoning process, not a similarity match.

Here's how it works step by step:
1. Document Indexing
- Parses the full document into a hierarchical tree structure
- No chunking, no embedding, no vector DB required
2. Tree-Based Reasoning
- LLM traverses the tree from root to relevant leaf nodes
- Evaluates context at each level before going deeper
3. Context-Aware Retrieval
- Incorporates full conversation history into the search
- Returns exact page and section references fully traceable
4. Agentic Execution
- VectifyAI built a working demo using the OpenAI Agents SDK
- End-to-end agentic pipeline with no vector layer in between
Why does this matter?
PageIndex- the leading open-source vectorless RAG framework scored 98.7% on FinanceBench, the most rigorous financial document QA benchmark available. For context: ChatGPT 4o with search scored 31% on the same test.
30,000+ GitHub stars.
The RAG debate has shifted. It's no longer "which vector DB is best."
It's whether vectors should be the default assumption at all.
If you want to go deeper on AI Agents and the infrastructure powering them, my free newsletter breaks it all down: https://lnkd.in/g5-QgaX4
Save → React → Share
& follow for everything related to AI Agents
#AIAgents #VectorlessRAG #RAG #LLM #OpenAI #AgenticAI

MEDIA The-End-of-Chunking-Vectorless-RAG-Explained; document/carousel (possible)

LINKS
- https://lnkd.in/g5-QgaX4

#457 Asim Kumar

AI / LLM

Senior .NET & Azure AI Engineer | ASP.NET Core | Agentic AI | RAG | EF | SQL | Azure OpenAI | Azure AI Search | Building AI-Driven Enterprise Apps | Open to Collaborations

4mo · **View on LinkedIn**

I built a RAG pipeline from scratch — zero hallucination, full auditability.
Here's everything I learned (and the exact architecture I used):
Most LLM apps fail in production because they let the model "guess."
Mine doesn't.
I built a full RAG system on Azure + .NET Core 8 that:
✓ Ingests PDFs automatically on Blob upload
✓ Chunks text with 50-word overlap (no boundary blindspots)
✓ Detects + skips TOC and cover pages that pollute results
✓ Runs a cosine similarity gate at 0.60 before ever calling GPT-4o
✓ Falls back to GPT-4o-mini (40x cheaper) when no relevant chunk is found
✓ Returns answerSource + topScore on every response — fully auditable
The result?
Ask it a document question → "rag" with a grounded answer.

Ask it something off-topic → "llm_fallback" with a clear signal.
Ask about something not in the docs → "I don't have enough information."
No hallucinations. No silent failures. No surprise token bills.

The stack:
• Azure Blob Storage → triggers ingestion
• Azure Document Intelligence → PDF text extraction per page
• text-embedding-3-large (3072 dims) → embeddings
• Azure AI Search (HNSW, cosine) → vector retrieval
• GPT-4o → RAG answers | GPT-4o-mini → fallback
• IMemoryCache → same question twice = zero API calls

The one insight most tutorials skip:
text-embedding-3-large uses WIDER cosine distances than smaller models.
Set your threshold at 0.60 — not 0.75.
At 0.75 you'll miss half your relevant chunks.
At 0.50 you'll let garbage in.
0.60 is the sweet spot. Test it with /api/search/debug first.

Cost breakdown per question:
Embedding only → ~$0.0000013 (use this for tuning!)
RAG path (GPT-4o) → ~$0.0033
Fallback (GPT-4o-mini) → ~$0.000076
With $150 Azure credits you get ~45,000 RAG queries.

I wrote the full implementation guide covering:
→ Architecture overview
→ Ingestion pipeline (chunking, embedding, indexing)
→ Query pipeline (score gate, RAG vs fallback)
→ All key service classes explained
→ Cost optimisation tips
→ End-to-end testing strategy
→ Troubleshooting (the stream bugs that burned me)
Check my demo video in the comments to see the chatbot in action.
#RAG #DotNet #Azure #AzureOpenAI #LLM #AIEngineering #MachineLearning #GenerativeAI #SoftwareEngineering #CloudArchitecture
RAG Pipeline Implementation Gu
.
13 pages
14
1

Asim Kumar
Author
Senior .NET & Azure AI Engineer | ASP.NET Core | Agentic AI | RAG | EF | SQL | Azure OpenAI | Azure AI Search | Building AI-Driven Enterprise Apps | Open to Collaborations
4mo

https://www.linkedin.com/posts/asimkumarjaiswal_azureopenai-rag-dotnet-ugcPost-7464642161087692800-RMgC/?utm_source=share&utm_medium=member_desktop&rcm=ACoAABNF3mUBZU_KquCH2tH-etSN3YqjyGgbyLY

#azureopenai #rag #dotnet #genai #artificialintelligence #azureaisearch #gpt4o #semanticsearch #docker #openai #softwarearchitecture #cloudcomputing #backenddevelopment #vectorsearch #aiengineering... | Asim Kumar

linkedin.com

MEDIA advertisement; document/carousel (possible)

LINKS

- https://www.linkedin.com/posts/asimkumarjaiswal_azureopenai-rag-dotnet-ugcPost-7464642161087692800-RMgC/?utm_source=share&utm_medium=member_desktop&rcm=ACoAABNF3mUBZU_KquCH2tH-etSN3YqjyGgbyLY

#458 ASHISH KUMAR

AI / LLM

AI Engineer @ Flixstock | GenAI & LLMs | AI Agents | RAG | Automation | Ex-DRDO |

4mo · View on LinkedIn

Pure Coding Vibe Coding

With AI tools, anyone can generate a working feature in minutes.

Need a login page? ✓

Need an API? ✓

Need a dashboard? ✓

Just describe it and AI starts writing code.

But here's what many developers discover when that code reaches production:

The feature works... until an edge case appears.

Debugging becomes difficult because nobody fully understands the generated code.

Different AI prompts create different coding patterns across the project.

Technical debt grows faster than the feature list.

A small change in one module unexpectedly breaks another.

Vibe coding is amazing for: ✓ Prototyping

✓ Learning

✓ MVPs

✓ Repetitive tasks

But in large-scale projects, success still depends on:

✓ System design

✓ Architecture decisions

✓ Code reviews

✓ Testing strategies

✓ Debugging skills

✓ Understanding why the code works

The most valuable developers in 2026 are not the ones writing every line manually.

They're the ones who can say:

"AI wrote this code. I know exactly why it works, where it can fail, and how to fix it when production breaks at 2 AM."

That's not vibe coding.

That's engineering.

#AI #VibeCoding #SoftwareEngineering #Programming #Developers #Coding #Tech #ArtificialIntelligence #SystemDesign #Engineering

#459 Aiswarya Venkitesh

AI / LLM

Principal Cloud Solution AI Architect @Microsoft | AI, Data and Tech Content Creator | Global Speaker | Worldwide Ranked 4th in the World’s Top Female Tech Creators | ★ Top AI Voice | Opinions are my own!

4mo · View on LinkedIn

Master RAG with Azure – from documents to answers

Retrieval-Augmented Generation (RAG) isn’t magic.

It’s a well-designed pipeline — and Azure gives you all the building blocks.

This visual breaks RAG into 4 clear stages ↓

Indexing

• Parse documents (PDFs, docs, etc.)

• Chunk text for better recall
• Generate embeddings using Azure embedding models
• Store them in a vector database
Retrieval (R)
• User query → encoded into embeddings
• Semantic search over the vector store
• Fetch the most relevant chunks
Augmentation (A)
• Combine retrieved chunks
• Build context-aware prompts
• Ground the query with trusted data
Generation (G)
• Send prompt + context to the LLM
• Generate accurate, hallucination-reduced responses
Key takeaway:
RAG is not about “asking the LLM harder questions”
it’s about feeding the LLM the right context at the right time.
If you’re building enterprise-grade GenAI on Azure, mastering this flow is non-negotiable.
Save this for reference
Drop a comment if you want a deep dive on any stage
Become better at AI in just 1 minute a day. Join my weekly newsletter where I document the real-world journey of AI transformation.
→ Sign up free now → https://avsl.beehiiv.com/
Follow Aiswarya Venkitesh for more such insights!!

MEDIA document/carousel (possible)

LINKS
- https://www.instagram.com/aiswaryavenkitesh?utm_source=ig_web_button_share_sheet&igsh=ZDNlZDc0MzIxNw==
- https://avsl.beehiiv.com/

#460 Vinod Thube

System Design

Senior Java Developer | Microservices, Spring Boot | System design | HLD and LLD
4mo · **View on LinkedIn**

Building Resilient Microservices with Resilience4j & Circuit Breaker in Spring Boot
In distributed systems, failures are inevitable.
A single slow or unavailable service can impact the entire microservice ecosystem.
That’s where the Circuit Breaker Pattern with Resilience4j becomes essential.
Key Benefits of Resilience4j:
✓ Prevents cascading failures across services
✓ Stops repeated calls to unhealthy services
✓ Improves fault tolerance & system stability
✓ Provides fallback responses for better user experience
✓ Reduces latency and thread exhaustion
✓ Supports automatic recovery when service becomes healthy
✓ Lightweight and designed for Java 8+ functional programming
✓ Easy integration with Spring Boot microservices

✓ Helps improve overall application availability
Circuit Breaker States:
Closed → Requests work normally
Open → Requests blocked after failures exceed threshold
Half-Open → Allows limited test requests for recovery
Commonly Used Resilience4j Modules:
✓ Circuit Breaker
✓ Retry
✓ Rate Limiter
✓ Bulkhead
✓ Time Limiter
✓ Cache
Example:

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackResponse")
public String callPaymentService() {
return restTemplate.getForObject(url, String.class);
}
```

A resilient microservice architecture is not about avoiding failures — it’s about handling failures gracefully and recovering quickly.
#SpringBoot #Microservices #Java #Resilience4j #CircuitBreaker #BackendDevelopment #SystemDesign #SoftwareEngineering #CloudNative #DistributedSystems

#461 Rajendra M G

AI / LLM

Associate Director - AI Platform amp; Security @ Bloom Value | Azure, M365, Fabric, MS Entra, AI, Power Platform, Google Workspace, IT Compliance & Security, AWS
4mo · **View on LinkedIn**

Still confused between LLMs, RAG, AI Agents, and MCP?
Let’s simplify this once and for all ↓
Think of modern AI systems like a human body

LLM = Brain (Thinks)
Large Language Models are trained on massive text data to understand and generate language.
What they do well:
• Pure reasoning & text generation
• Language understanding
• Pattern recognition
Limitations:
• No real-time data
• No access to private or external systems
• Can hallucinate without context
→ LLMs think, but they don’t know beyond training.

RAG = Brain + Books (Knows)
Retrieval-Augmented Generation enhances LLMs with external knowledge.
How it works:
• LLM + documents (PDFs, DBs, APIs)
• Retrieves relevant context before answering

• Grounds responses in real data
Benefits:
• ✓ Fewer hallucinations
• More accurate, context-aware answers
• Essential for enterprise use cases
→ RAG knows, because it can look things up.

AI Agents = Brain + Hands (Acts)
Agents don’t just answer — they take action.
Core components:
• LLM
• Planning & reasoning
• Memory
• Tool usage (APIs, DBs, services)
What agents can do:
• Take decisions
• Execute workflows
• Automate tasks
• Coordinate multiple systems
Used in:
• Automation
• Copilots
• DevOps
• Enterprise GenAI systems
→ Agents act, not just respond.

MCP (Model Context Protocol) = Nervous System (Connects)
MCP is the missing backbone for scalable AI systems.
Why MCP matters:
• One standard way for models to access tools & data
• Unified interface for GitHub, Slack, files, APIs
• Secure, consistent, production-ready integration
→ MCP connects and coordinates everything.

In short:
LLM thinks.
RAG knows.
Agents act.
MCP connects.

Real-world insight (from production systems):
• LLM alone = demo
• RAG = usable
• Agents = powerful
• MCP = scalable & enterprise-ready
Modern GenAI is systems engineering, not prompt engineering.

Where are you right now?
• Just LLMs?

• Building RAG?
• Experimenting with agents?
• Moving toward MCP-based architectures?
↓ Drop your thoughts in the comments.
#GenAI #LLM #RAG #AIAgents
Repost for more like this
↓ Follow me Rajendra M G
Feel free to reach Rajendra M G for Best High Quality IT Infra Support to manage your End to end IT Environment and save cost and time. Secure your system by Global Experts in industry with 20+ yr experience.
#ITCareer #TechCommunity #ContinuousLearning #ITInfra #ITSupport #ITServices #RajendraMG
Find this helpful? [follow my profile/ repost]
Anything to add about this subject? [invited to comment]

MEDIA document/carousel (possible)

#462 Kashish Kathuria

AI / LLM

Software Engineer @Wipro
4mo · **View on LinkedIn**

Everyone is talking about Agentic AI... but most are still building glorified workflows.
LLM chatbots ≠ Agentic AI
RPA ≠ Agentic AI
RAG ≠ Agentic AI
Real Agentic AI is different.
It plans, remembers, collaborates, and acts - not just responds.
It’s
It’s not a single model.
It’s a system of agents working together:
→ Orchestrating tasks
→ Using tools autonomously
→ Learning from feedback
→ Sharing context across agents
The shift?
From asking AI questions → to assigning AI outcomes.
And the ones who understand this early... will build 10x leverage.
Are you building prompts or systems?

#463 Aymane Mrini

AI / LLM

CEO of AgentX Academy | Agentic AI Expert | Building and teaching agentic AI
4mo · **View on LinkedIn**

This CLAUDE.md file will make you 10x engineer ↓
It combines all the best practices shared by Claude Code creator:
Boris Cherny (creator of Claude Code at Anthropic) shared on X internal best practices and workflows he and his team actually use with Claude Code daily. Someone turned those threads into a structured CLAUDE.md you can drop into any project.
It includes:

- Workflow orchestration
- Subagent strategy
- Self-improvement loop
- Verification before done
- Autonomous bug fixing
- Core principles

This is a compounding system. Every correction you make gets captured as a rule. Over time, Claude's mistake rate drops because it learns from your feedback.

If you build with AI daily, this will save you a lot of time

#464 Srinivas Mahakud

AI / LLM

Generative AI Architect & Engineering Leader | LLM · RAG · Multi-Agent Systems | Scaling AI from PoC to Production | EY

4mo · View on LinkedIn

Most AI agents fail not because they lack intelligence,

but because they lack the right memory architecture.

Working memory, episodic memory, semantic memory, procedural memory, and hierarchical memory all play different roles in making agents reliable, contextual, and scalable.

The future of Agentic AI will depend on how well systems remember, retrieve, and reason over time.

Memory is no longer a feature in AI systems.

It is becoming the architecture itself.

#AgenticAI #AIArchitecture #GenAI #LLM #MemorySystems #RAG #EnterpriseAI #AIEngineering #SystemDesign #FutureOfAI

#465 Avinash Jha

AI / LLM

AVP - Axis Bank | Product Management | Product Design | Productising AI in Banking | Revenue Generation | Enterprise AI Capability | Business Value Delivery | AI Use Case Discovery | RAG | | Six Sigma BB Professional

4mo · View on LinkedIn

We all use ChatGPT/Copilot/Claude every day. But did we ever wondered what's actually happening the moment you press send along with your query?

I dug into the research — here's the whole journey in plain English.

1. Your message gets chopped up

The moment you hit Send, your sentence is broken into tiny chunks called tokens. Think of tokens like puzzle pieces — words, parts of words, even punctuation get broken into one one word called token get it’s identifiable token ID in the system.

Examples: You query: How do I prepare for a PM interview?

Becomes : "How", "do", "I", "prepare", "for", "a", "PM", "interview", "?".

Each piece gets its own ID number.

2. Words get placed on a map of meaning

Each token ID is converted into a long list of numbers — called a vector. Imagine a giant map where every word has a location. Words with similar meanings sit close together. So "prepare" and "practice" are neighbours. But "prepare" and "pizza"? Miles apart.

3. Every word watches every other word

This is the clever bit. Each word in your sentence asks every other word: "How much should I care about you right now?" This is called Self-Attention.

In our example, the model quickly figures out that "prepare", "Product Manager", and "interview" are strongly related — and gives them extra weight.

4. It peels through 30–100 layers of understanding

The message doesn't just pass through once. It travels through 30 to 100 layers inside the model — like peeling an onion. Each layer refines the understanding a little more: surface meaning → context → intent → nuance. By the last layer, the model has a rich picture of exactly what you need.

5. It picks the best next word, over and over . The model doesn't write the whole reply at once. It predicts one word at a time. After each word, it looks at everything written so far and picks the next most likely word. Then again. Then again — until the answer is complete. That's why you see text appearing word by word on your screen.

6. All of this — in under a second

Billions of tiny math operations. Hundreds of layers. One word at a time. And your full, detailed answer lands in under a second. That's not just software — that's one of the most impressive engineering feats of our generation.

#ArtificialIntelligence
#GenerativeAI
#LLM
#MachineLearning
#TechExplained
#AILiteracy

#466 Shivani Virdi AI / LLM

AI Engineering | Founder @ NeoSage | ex-Microsoft • AWS • Adobe | Teaching 70K+ How to Build Production-Grade GenAI Systems

4mo · **View on LinkedIn**

Everyone talks about context engineering. No one shows you the memory stack underneath it. Here are the 5 layers of agent memory.

1. Working Memory

↳ In-context state for this turn. Conversation buffer, tool calls, intermediate reasoning

↳ Bounded by the context window.

↳ Compaction strategies (truncation, summarisation, hierarchical pruning) each lose different things

↳ No compaction policy means you drop critical state or blow the budget.

↳ Resets at session end.

2. Episodic Memory

↳ Cross-session record of what happened: tool calls, results, errors, user state, time-indexed

↳ Stored as append-only traces. Retrieved by semantic search, time window, or user scope

↳ Most teams log traces for debugging but never query them at planning time

↳ Without retrieval at planning time, the agent treats every session as its first

3. Semantic Memory

↳ An entity store. Users, accounts, documents and their typed attributes

↳ Queried at planning time. By entity ID or similarity search

↳ Extraction jobs read episodes. They write structured facts to entity records

↳ Most teams append facts indefinitely. Without deduplication, entity resolution, or contradiction handling

↳ Without those ops, retrievals serve contradictions.

4. Procedural Memory

↳ A skill registry. Versioned procedures for recurring task patterns

↳ Queried at planning time. Indexed by task type or input pattern

↳ Most teams hard-code prompts and freeze them.

↳ No reflection loop, no procedure store, no versioning
↳ Without those ops, same mistakes repeat session after session
5. Meta-Memory
↳ A control plane over the other four layers.
↳ Policies for retention, decay, compression, and forgetting
↳ Runs offline. Scheduled or triggered by memory thresholds
↳ Reviews all four layers. Archives stale entries, dedupes duplicates, deprecates unused procedures
↳ Without it, the stack rots. Storage grows forever, retrieval slows, stale wins
Context engineering decides what the model sees in this turn.
Memory decides whether your agent compounds or rots.
Working, episodic, semantic, procedural, meta.
Each layer feeds the next.
Models change. Memory is what compounds.
Save this for your next agent architecture review.
Repost to help an engineer build beyond storage and retrieval.

MEDIA document/carousel (possible)

LINKS
- https://academy.neosage.io/

#467 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · **View on LinkedIn**

Everyone wants to start with Agentic AI, but the first layer is still Traditional AI.
Become better at AI in just 1 minute a day.
Get the AI newsletter smart leaders read.
Sign up free now → aiforleaders.com

_____
Here’s the simple way to explain AI types:
There are 3 main categories.
Traditional AI.
Generative AI.
Agentic AI.
And they do very different jobs.
1. Traditional AI
This is the AI many companies already use.
→ Predictive analytics forecasts customer behaviour, demand patterns, and business risk
→ Classification systems sort emails, transactions, support tickets, and internal requests
→ Anomaly detection spots fraud, system failures, security issues, and unusual patterns
Why it matters:
↳ Traditional AI helps the business understand what is happening
↳ It turns messy activity into patterns leaders can measure
↳ It often has clearer ROI than the newer AI tools
2. Generative AI
This is the AI most teams started testing after ChatGPT.

→ Content generation creates drafts, reports, emails, code, images, and documents
→ Workflow automation plugs AI into daily tasks like notes, triage, and data cleaning
→ Knowledge systems use company data so AI can answer business questions
Where it helps:
↳ Generative AI speeds up work that used to take hours
↳ It works best when the underlying data is clean and organised
↳ It breaks quickly when teams feed it messy files or vague context
3. Agentic AI
This is the category everyone is talking about now.
→ AI agents use APIs, tools, and external systems to execute tasks
→ Multi-agent orchestration lets agents work together and coordinate steps
→ AI product integration embeds AI directly inside products and services
The risk:
↳ Agents can take action, so weak checks create bigger problems
↳ Multi-agent systems need clear workflows or they become hard to control
↳ Product AI needs evaluation rules before customers depend on it
The order matters.
Traditional AI helps the business understand patterns.
Generative AI helps the business create and answer faster.
Agentic AI helps the business take action through tools.
The mistake is skipping straight to agents.
Because an agent still needs clean inputs.
It still needs clear rules.
It still needs a way to spot when something has gone wrong.
That is why the first layer matters.
Predictive analytics.
Classification.
Anomaly detection.
Those are not outdated AI.
They are the foundation.
Leaders do not need to memorise every AI term.
They need to know what type of AI they are funding.
And whether the business is ready for that layer.
Start with the foundation.
Then add generation.
Then scale agents carefully.
Which AI type is your company spending the most time on right now?

MEDIA document/carousel (possible)

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com

#468 SOORAJ VIDYASAGAR AI / LLM

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

4mo · View on LinkedIn

Spring Boot + Spring AI: Building Enterprise-Grade RAG Applications

Many AI tutorials focus on Python, but enterprise organizations often run on Java. That’s where Spring AI changes the game.

A modern RAG (Retrieval-Augmented Generation) architecture can now be built entirely within the Spring ecosystem:

✓ Spring Boot 3.5

✓ Spring AI

✓ PostgreSQL + pgvector

✓ Azure OpenAI / OpenAI

✓ Spring Security

✓ Kubernetes (AKS/EKS/GKE)

Typical Flow

1. User submits a question from a React or Angular application
2. Spring Boot validates the JWT token
3. Spring AI performs vector search against pgvector
4. Relevant document chunks are retrieved
5. Context is added to the prompt
6. Azure OpenAI generates a grounded response
7. Citations and chat history are stored for future reference

Why Spring AI?

• Built-in RAG support

• AI Agents and Tool Calling

• Conversation Memory

• Multiple LLM providers (Azure OpenAI, OpenAI, Claude, Gemini, Ollama)

• Native Spring Security integration

• Production-ready deployment model

Enterprise Benefits

Leverage existing Java skills

Secure by design with OAuth2 and Entra ID

Cloud-native and Kubernetes-ready

Scalable ingestion pipelines using Spring Batch

Seamless integration with enterprise systems

The architecture is simple:

React → Spring Boot → Spring AI → Vector Store → Azure OpenAI

Java teams no longer need to switch ecosystems to build advanced AI solutions.

#SpringBoot #SpringAI #Java #GenAI #RAG #AzureOpenAI #PostgreSQL #pgvector #AIArchitecture #SoftwareArchitecture #EnterpriseAI #AKS #Kubernetes #LLM #Developers

MEDIA advertisement; document/carousel (possible)

LINKS

- https://www.youtube.com/playlist?list=PLUg9hRlm7gxS_jszh-RCtU6Q3L3Vt80be
- http://www.youtube.com/playlist?list=PLUg9hRlm7gxS_jszh-RCtU6Q3L3Vt80be

#469 Brij Kishore Pandey

AI / LLM

AI Architect & Engineer | Agentic systems, RAG, AI infrastructure, Data Engineering | 738K+ LinkedIn, 294K+ Instagram | Newsletter for 250K AI builders

4mo · View on LinkedIn

MCP is powerful.

But I think many people are confusing connection with production readiness.

MCP makes it easier for AI agents to connect with tools, APIs, data sources, files, and external systems.

That is a big step forward.

But once an agent can actually take action, the real engineering problems begin.

It is no longer just:

Can the agent call a tool?

It becomes:

Should the agent call this tool?

Is it acting on behalf of the right user?

Does it have the right permission?

What if the tool fails?

Can we retry safely?

Can we trace what happened?

Can a human approve risky actions?

Can security teams audit the full flow later?

This is the part that often gets missed in agent conversations.

MCP gives us a clean way to connect.

But production agents need much more around it:

- Identity
- Authorization
- Policy
- Observability
- Audit logs
- Error handling
- Cost controls
- Human approval
- Incident response

My simple view:

MCP is the connection layer.

The control layer is what makes agents production-ready.

Without that control layer, agents can become unpredictable, expensive, and hard to govern.

With it, they become safer, more reliable, and much easier to scale inside real enterprises.

The next phase of agentic AI will not be about connecting every possible tool.

It will be about building systems that can safely control, observe, and govern those tools.

That is where real production AI will be won.

I created this infographic to explain the difference visually.

What do you think is the most overlooked part of production AI agents?

LINKS

- https://brij.media/live

#470 Nikki Siapno System Design

Eng Manager & Founder | Ex-Canva | 450k+ Community | Helping Senior Engineers Master System Design and AI Engineering | View my newsletter

4mo · View on LinkedIn

Things Every Engineer Should Know: SQL Execution Order.

Most engineers read SQL top-down, but that only explains intent. Performance requires a second pass that follows execution order.

A SQL query executes its statements in the following order:

1) FROM / JOIN

2) WHERE

3) GROUP BY

4) HAVING

5) SELECT

6) DISTINCT

7) ORDER BY

8) LIMIT / OFFSET

The techniques you implement at each step help speed up the following steps. This is why it's important to know their execution order. To maximize efficiency, focus on optimizing the steps earlier in the query.

With that in mind, let's take a look at some optimization tips:

1) Maximize the WHERE clause

This clause is executed early, so it's a good opportunity to reduce the size of your data set before the rest of the query is processed.

2) Filter your rows before a JOIN

Although the FROM/JOIN occurs first, you can still limit the rows. To limit the number of rows you are joining, use a subquery in the FROM statement instead of a table.

3) Use WHERE over HAVING

The HAVING clause is executed after WHERE & GROUP BY. This means you're better off moving any appropriate conditions to the WHERE clause when you can.

4) Don't confuse LIMIT, OFFSET, and DISTINCT for optimization techniques

It's easy to assume that these would boost performance by minimizing the data set, but this isn’t the case. Because they occur at the end of the query, they make little to no impact on its performance.

Query optimization isn’t about memorizing tricks. It’s about knowing where the work happens. Optimize early, measure often, and adapt to your actual data and access patterns.

What else would you add?

——

PS: Want to improve at system design? Download my free System Design Handbook and join 34,000+ engineers who get my free weekly newsletter → https://lnkd.in/gueNmru7

——

Repost to help others learn and grow.

+ Follow me (Nikki Siapno) + turn on notifications.

LINKS

- https://lucode.co/luc-system-design-newsletter-pl1
- https://lnkd.in/gueNmru7
- https://

#471 Rakesh Gohel AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

4mo · View on LinkedIn

Agentic Engineering in 2026 is not about mastering the latest model release.

It is about building the system around it that actually works in production...

Most people start with prompts, APIs, or a new framework. That is a fine entry point.

But the deeper skill, the one that separates an AI user from an AI engineer, is knowing how all the layers fit together and why each one exists.

Here is how I think about the stack:

→ LLM fundamentals + reasoning models Understand how models think before you orchestrate them. Reasoning models changed what a single agent call can do autonomously that changes your architecture decisions downstream.

→ Context Engineering Most people are still optimizing their prompts. The engineers shipping reliable systems are thinking one level up, designing the full context environment the LLM operates in, not just the message they send it.

→ Memory architecture Memory is now a first-class engineering primitive. Session memory, vector retrieval, and episodic memory across agents solve different problems. Knowing which to reach for is a real skill.

→ Agentic workflows + multi-agent orchestration Single agents hit ceilings fast. Production systems run networks of specialized agents handing off tasks. The engineering challenge is state management and failure handling across that network.

→ MCP + tool connectivity MCP standardized how agents connect to external tools and data sources. If you haven't mapped your tool layer against MCP yet, you're building on a foundation that will need rebuilding.

→ AI gateways, routing + cost control Routing requests across models by cost, latency, and capability is its own discipline. At scale, getting this wrong is expensive.

→ Guardrails + safety Agents operating autonomously at scale introduce vulnerabilities no human reviewer can catch manually. Safety needs to be embedded in the architecture, not added at the end.

→ Observability + evaluation When an agent fails, you don't get a stack trace. You get a reasoning loop. Tracing why an agent made a specific decision requires tools and mental models most engineers don't have yet.

The best agentic engineers are not model experts with a RAG pipeline bolted on.

They are system builders who understand every layer between the LLM and the user and can debug any one of them when production breaks at 2am.

That is where the real engineering begins.

My free newsletter breaks down each of these layers in depth: https://lnkd.in/g5-QgaX4

Save → React → Share

& follow for everything related to AI Agents

#AgenticEngineering #AIAgents #LLM #TechLeadership #ContextEngineering

MEDIA The-AI-Agent-Engineering-Roadmap

LINKS

- https://lnkd.in/g5-QgaX4

#472 Yash Wadpalliwar

AI / LLM

↳ C++(DSA) | Python| DSML & AIML | Generative AI | Adv-SQL | Snowflake| Apache Airflow | Git & GitHub | Full-Stack(MERN) | The Analytics Accelerator| Databricks | 10k+@YouTube | Corporate Trainer | Eco-Activist

4mo · View on LinkedIn

BEFORE vs AFTER: How AI Tools Are Evolving

A few years ago, we used different tools for different tasks.

Today, AI platforms are expanding beyond a single use case and becoming complete ecosystems.

✦ ChatGPT → Claude

Deep reasoning, analysis, and advanced AI assistance.

✦ Cursor → Claude Code

AI-powered coding support with stronger context understanding and code insights.

✦ Figma → Claude Design

AI-assisted design generation and creative workflows.

✦ n8n → Claude Routine

Natural language automation for workflows and repetitive tasks.

What's Driving This Shift?

✦ More powerful AI models

✦ Better reasoning and contextual understanding

✦ Faster execution of complex tasks

✦ Reduced manual effort through automation

✦ End-to-end workflows inside a single platform

A Critical Observation

The image suggests a direct replacement of existing tools, but the reality is more nuanced.

Tools like ChatGPT, Cursor, Figma, and n8n are still highly specialized and continue to lead in their respective domains. The trend is not necessarily replacement, but convergence—AI platforms are expanding into areas that previously required multiple tools.

The real question is:

Will future professionals use a collection of specialized tools, or will a few AI platforms become the operating systems for most digital work?

What do you think—are we moving toward AI consolidation or will specialized tools continue to dominate?

#AI #ArtificialIntelligence #Claude #ChatGPT #Cursor #Figma #n8n #Automation #Coding #Design #Productivity #TechTrends #Innovation #FutureOfWork

MEDIA advertisement

#473 Nagarjuna Reddy

AI / LLM

Sr SRE & Platform Engineer | AIOps •GenAI/LLM Infra | DevSecOps | AWS •Azure •Kubernetes •Terraform •GitOps | IEEE Senior Member •Sigma Xi •Forbes Tech Council | Tech Author/Researcher | High-Scale Production Systems

4mo · **View on LinkedIn**

Most people use AI every day.

But very few understand what happens behind the scenes.

Here's a simple breakdown of how Large Language Models (LLMs) work:

1. Tokenization: Text is broken into smaller pieces called tokens.

2. Embeddings: Tokens are converted into numerical representations that capture meaning and context.

3. Transformer Architecture: The model analyzes relationships between words and concepts.

4. Self Attention: The model focuses on the most relevant information in the context.

5. Parameters: Billions of learned parameters help the model recognize patterns from training.

6. Context Window: Determines how much information the model can consider at once.

7. Next Token Prediction: The model predicts the most likely next token.

8. Output Generation: Tokens are generated one by one to create the final response.

Key building blocks of modern LLMs:

- Tokens
- Embeddings
- Transformer Architecture
- Self Attention
- Parameters
- Context Windows
- Next Token Prediction
- Output Generation

How are these models trained?

Pre Training: Learning from massive amounts of text data

Fine Tuning: Adapting the model for specific tasks

RLHF (Reinforcement Learning from Human Feedback): Improving responses through human evaluation and feedback

Whether you're a creator, developer, marketer, founder, or student, learning how LLMs work is becoming an essential skill in the AI era.

What's one AI concept you'd like to understand better?

#AI #LLM #GenerativeAI #Technology #MachineLearning

#474 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · **View on LinkedIn**

Your AI stack is probably heavier than it needs to be.

Become better at AI in just 1 minute a day.

Get the AI newsletter smart leaders read.

Sign up free now → aiforleaders.com

_____

Most teams do not have an AI problem.

They have a tool-switching problem.

The old stack looked simple.

ChatGPT for ideas.

Cursor for coding.

Figma for UI.

n8n for automations.

Then every workflow started crossing tool boundaries.

That is where Claude becomes interesting.

→ Ideas → Claude

Use Claude when the job needs reasoning, precise instructions, and a usable first draft.

Before: brainstorm in ChatGPT, then rewrite for execution.

After: ask Claude for the brief, constraints, edge cases, and first deliverable.

→ Coding → Claude Code

Use Claude Code when the work touches full features, refactors, architecture, or tests.

Before: prompt a coding tool one file at a time.

After: describe the outcome and let Claude reason across the codebase.

→ UI/UX → Claude Design

Use Claude Design when you need a dashboard, workflow, or prototype direction fast.

Before: explain the idea, wireframe it, then translate it again.

After: describe the user, screen, states, and decisions in one pass.

→ Automations → Claude routines

Use Claude when the task should run on a schedule with context.

Before: build logic in n8n before the workflow is clear.

After: define the morning run, checks, outputs, and escalation rules first.

→ The diagnostic

Count how many times one task moves between ChatGPT, Cursor, Figma, n8n, Docs, and Slack.

If it crosses 3 or more tools, consolidation is worth testing.

If ownership gets unclear, consolidation is worth testing.

If context keeps getting re-explained, consolidation is worth testing.

→ The 30-minute test

Pick one live workflow this week.

Ask Claude to plan it, draft the output, identify missing context, and define the next action.

Then compare that against your current ChatGPT, Cursor, Figma, or n8n handoff path.

If Claude removes 2 handoffs without lowering quality, you found a real consolidation candidate.

The caution

Do not replace specialist tools just because Claude can draft the work.

Keep Figma, Cursor, n8n, or production systems where governance, scale, or integrations demand them.

The real win is not fewer logos.

It is fewer handoffs.

Less context loss.

Faster execution.

Which part of your AI stack is mostly tool-switching?

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com

#475 Ankit Pangasa

AI / LLM

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

4mo · **View on LinkedIn**

Bookmark This: 12 Agentic AI Terms Every Engineer Should Know in 2026

A year ago, everyone was talking about prompts.

Today, the conversation has shifted to:

→ Agents

→ MCP

→ Tool Calling

→ Memory

→ Multi-Agent Systems

And if you're building AI applications, interviewing for AI roles, or simply trying to keep up with the ecosystem, these are no longer "nice-to-know" concepts.

They're becoming foundational.

Here's a quick cheat sheet of the 12 terms you'll encounter repeatedly:

MCP (Model Context Protocol)

A standardized way for AI agents to connect with tools, APIs, databases, and external systems.

Think of it as:

"USB-C for AI Agents."

Agent Loop

The core reasoning cycle:

Perceive → Plan → Act → Observe

This loop is what makes an AI behave more like an agent than a chatbot.

Tool Use

The ability of an AI model to invoke APIs, execute code, search the web, access databases, or interact with external systems.

Without tools, an agent can only think.

With tools, it can act.

Orchestrator

The coordinator agent.

Breaks a large goal into smaller tasks and delegates work to specialized agents.

Think:

Project Manager for AI.

Subagent

A specialized worker agent focused on a specific responsibility.

Examples:

Research Agent

Coding Agent

Testing Agent

Summarization Agent

Memory

Allows agents to retain information across interactions.

Two common types:

Short-Term Memory

Long-Term Memory

Memory is what makes agents feel personalized rather than stateless.

Grounding

Connecting AI outputs to verified external data.

This reduces hallucinations and improves reliability.

Guardrails

Safety boundaries that define what an agent can and cannot do.

Examples:

✓ Approval workflows

✓ Access restrictions

✓ Action limits

Sandboxing

Running code in isolated environments to protect production systems.

Critical when agents execute code autonomously.

Human-in-the-Loop

AI proposes.

Humans approve.

Especially important for high-risk decisions.

Context Window
The amount of information an AI model can process in a single interaction.
A larger context window enables better reasoning across large documents and codebases.
Multi-Agent Systems
Multiple specialized agents collaborating to solve a larger problem.
One researches.
One writes.
One validates.
One orchestrates.
Together they outperform a single general-purpose agent.
Why This Matters
The future of AI is not just bigger models.
It's:
Better tools
Better memory
Better orchestration
Better collaboration between agents
Understanding these concepts today is similar to understanding:
APIs in 2010
Cloud Computing in 2015
Containers in 2020
They are rapidly becoming core building blocks of modern software systems.
Save this post.
You'll likely encounter most of these terms in the next AI article, conference talk, architecture discussion, or interview.
Image: Brij
#AI #AgenticAI #MCP #LLM #GenerativeAI #AIAgents

MEDIA document/carousel (possible)

#476 Java Developers Community (moderated)

Java / Spring

Naveengandhi C. • 3rd+
4mo · **View on LinkedIn**

We had a thread leak in production for months.
No crash.
No alerts.
No obvious symptoms.
Just a slow accumulation of threads... doing work nobody needed anymore.
At first, everything looked fine.
Requests were completing. Metrics were stable. Nothing screamed “incident.”
But over time, thread counts kept creeping up.
Until one day, they didn’t come back down.
The pattern will look familiar.
You submit multiple tasks. Wait for one. Handle the result.
It works.
Until one of those tasks fails.

Then something subtle happens.
The failure is handled... but the other tasks keep running.
No one cancels them.
No one waits for them.
No one owns them anymore.
They just continue in the background.
Individually, that’s harmless.
At scale, it isn’t.
A few orphaned threads per request turns into hundreds.
Hundreds turn into resource contention.
And suddenly your system is busy doing work that nobody is waiting for.
The core problem isn’t the code.
It’s the model.
Traditional concurrency lets you create work anywhere... without enforcing how that work ends.
There’s no built-in structure tying tasks to the request that created them.
Which means it’s easy to forget cleanup.
And even easier for that mistake to stay invisible.
What changed recently is not just performance.
It’s structure.
The newer approach to concurrency introduces a simple but powerful idea:
If a task is created inside a block...
it must finish — or be cancelled — before that block exits.
No exceptions. No leftovers.
That one constraint changes everything.
Now tasks have ownership.
Their lifetime is bounded.
Failures propagate cleanly.
If one task fails, the others don’t keep running pointlessly.
They’re cancelled automatically.
The interesting part isn’t just correctness.
It’s clarity.
You can finally reason about concurrent code the same way you reason about normal code.
Start here. End here. Everything in between is contained.
It also changes how systems behave under stress.
Instead of leaking work, they fail fast and clean.
Instead of accumulating hidden load, they release resources immediately.
The biggest win is something most teams don’t notice until it’s gone:
orphaned work disappears.
Looking back, the leak we had wasn’t caused by bad code.
It was caused by a model that made it easy to forget the lifecycle of work.
Once you see that, you start designing differently.
Not just “how do I run tasks in parallel?”
But:
“Who owns this work... and when does it end?”
That question alone prevents a surprising number of production issues.
Have you ever run into work that kept running long after the request that created it was gone?

#477 Srinivas Mahakud

AI / LLM

Generative AI Architect & Engineering Leader | LLM · RAG · Multi-Agent Systems | Scaling AI from PoC to Production | EY

4mo · View on LinkedIn

Building AI Agents with MCP? Don’t just connect... design smart.

Most teams plug MCP into agents.

Few think in design patterns — that’s where scale happens.

Here are 6 MCP patterns you should know

Direct API Wrapper

Fastest way to integrate → 1:1 API mapping

→ Best for quick wins & low effort

Composite Service

Orchestrate multiple APIs into one flow

→ Best for complex workflows

MCP-to-Agent

Agents calling agents

→ Best for modular, multi-agent systems

Event-Driven

Async triggers, non-blocking flows

→ Best for real-time & scalable systems

Hierarchical MCP

Layered MCP with routing control

→ Best for enterprise-scale architecture

Local Resource Access

Secure local data access

→ Best for offline / sensitive workloads

(!) Design smart, not just fast:

Avoid tight coupling, agent overuse, event floods, and security gaps.

#AI #AgenticAI #MCP #SystemDesign #AIArchitecture #GenAI #SoftwareArchitecture #TechLeadership #Engineering #ScalableSystems

#478 Ashish Sahu

AI / LLM

GenAI Architect

4mo · View on LinkedIn

Most organisations are focused on building more powerful AI agents.

The more important challenge is building AI agents that can be trusted.

That is the real enterprise conversation in 2026.

As AI agents move beyond experimentation and begin interacting with enterprise systems, workflows, APIs, and operational data...

Security stops being a technical afterthought.

It becomes a business-critical leadership concern.

The strongest AI organisations are no longer asking:

“How autonomous can the agent become?”

They are asking:

“How controlled should the agent remain?”

Because autonomous systems without strong security architecture create new forms of operational exposure:
→ Uncontrolled tool access
→ Prompt injection risks
→ Data leakage pathways
→ Unsafe autonomous actions
→ Invisible decision chains
Which is why mature AI organisations are prioritising:
→ Identity-aware AI systems
→ Controlled execution environments
→ Human oversight for critical actions
→ Continuous behavioural monitoring
→ Security embedded directly into agent workflows
The future AI stack will not only require intelligent models.
It will require intelligent control systems around them.
And the companies that understand this early will scale AI more confidently, responsibly, and sustainably.
Because in enterprise AI...
Trust is not created by capability alone.
It is created by control.
P.S. Many teams are still treating AI security like traditional application security.
But autonomous systems introduce entirely new operational risks that require a different security mindset.
Follow Ashish Sahu for more insights

#479 Brij Kishore Pandey AI / LLM

AI Architect & Engineer | Agentic systems, RAG, AI infrastructure, Data Engineering | 738K+ LinkedIn, 294K+ Instagram | Newsletter for 250K AI builders

4mo · **View on LinkedIn**

AI in 2026 is no longer just about prompts.
It is becoming a full software system.
And honestly, this is where things are getting interesting.
A few years ago, most AI apps looked simple:
User asks a question.
Model gives an answer.
But production AI does not work that way anymore.
Today, the model is only one part of the stack.
You also need RAG to bring the right knowledge.
Tools to connect with real systems.
Memory to keep useful context.
Agents to plan and take action.
Multi-agent systems to divide work.
Skills to reuse common capabilities.
Governance to keep everything safe.
Observability to understand what happened.
And humans in the loop when judgment really matters.

This is why AI engineering feels different now.

It is not just about asking better questions to a model.

It is about designing systems that can reason, retrieve, act, remember, coordinate, and still stay reliable.

That is also where many AI projects struggle.

Not because the model is weak.

But because the system around the model is missing.

No clear evaluation.

No visibility.

No permission model.

No fallback path.

No human review.

No proper grounding.

In 2026, the teams that win with AI will be the ones who understand the full stack.

Because the model can generate intelligence.

But the architecture turns that intelligence into something useful.

The model gets the attention.

The system creates the value.

What layer do you think most teams still underestimate?

LINKS

- https://brij.media/live

#480 Vrushabh Gawale

AI / LLM

EXL- Digital Transformation | Product M @ Yes Bank & IDFC First Bank

4mo · **View on LinkedIn**

Every one of us must be using chatbots such as GPT/Co-pilot/Claude for daily use irrespective of office work or personal tasks.

I wanted to understand the architecture of how it exactly works, and I did some research on LLMs.

When we type a message, for example: "How do I prepare for a Product Manager interview?" and hit send, it basically gets broken into pieces called tokens. That message can be split into tokens like "How", "do", "I", "prepare", "for", "a", "Product", "Manager", "interview", "?". Each token gets a numeric ID. We call it a Token ID.

These Token IDs then get converted into vectors, think of a vector as a list of thousands of numbers that captures the "meaning" of that word. Words with similar meanings end up close to each other in this space. So "prepare" and "practice" would be neighbours. "prepare" and "pizza" would be far apart.

Now here's where it gets interesting every word looks at every other word and asks: "how much should I care about you right now?" This is called Self-Attention. In our example, the model figures out that "prepare", "Product Manager", and "interview" matter a lot together.

This whole process passes through 30 to 100 layers inside the model each layer refining the understanding a little more, like peeling an onion of context.

Finally, the model doesn't write a full reply at once. It predicts one word. Then the next. Then the next until your answer is complete. That's why you see responses appearing word by word on your screen.

And all of this from your interview question to a detailed preparation plan happens in under a second.

The more I learn about LLMs, the more I appreciate how much innovation is hidden behind a simple "Send" button.

#ArtificialIntelligence #GenerativeAI #LLM #ProductManagement #BusinessAnalysis #DigitalTransformation #Technology #Innovation

#481 Maryam Miradi, PhD

AI / LLM

Chief AI Scientist | 20+ Yrs in AI | 400+ Production AI Agents Built | AI Agents Instructor | Teaching 3,200+ students Agentic Python Systems (Claude Code, LangGraph, Google ADK, CrewAI, MCP, OpenAI) | 46k+ Newsletter

4mo · View on LinkedIn

How to Use Knowledge Graphs for AI Coding Agents → Even If Your Codebase Is Huge.

This is 7 Step roadmap from raw codebase to queryable agent context.

Most AI coding agents still treat your repo like a pile of files.

But production codebases are actually graphs.

My 7 Step roadmap how to treat it like a graph:

Step 1: Choose What to Index

✸ Start with the parts your agent actually needs.

✸ Code, docs, configs, tests, or selected folders.

✸ Do not dump everything blindly into context.

→ Example: Index only the docs folder before asking the agent to explain product logic.

Step 2: Convert Files into a Knowledge Graph

✸ Turn files, functions, routes, modules, and docs into nodes.

✸ Turn imports, calls, references, and workflows into edges.

✸ Now your agent can follow structure, not just text.

→ Tool idea: Graphify-style codebase graph.

Step 3: Map Hidden Dependencies

✸ Which component calls which service?

✸ Which route touches which database table?

✸ Which admin page connects to which AI feature?

→ This is where the agent stops guessing.

Step 4: Add a Query Layer

✸ Let the agent ask the graph before editing code.

✸ Find related files.

✸ Explain a feature.

✸ Show the shortest path between two modules.

→ Example: “Explain how the AI chat connects to the admin dashboard.”

Step 5: Use the Graph Before Refactoring

✸ Before changing code, ask what depends on it.

✸ Before deleting code, ask what might break.

✸ Before adding a feature, ask where it belongs.

→ This turns code editing into system navigation.

Step 6: Keep the Graph Updated

✸ Reindex changed files.

✸ Avoid stale architecture maps.

✸ Treat the graph as living context, not a one-time artifact.

→ A stale graph makes the agent confidently wrong.

Step 7: Add Verification Before Shipping

✸ Check affected files.

✸ Check dependency paths.
✸ Check tests.
✸ Check human review points.
→ Better context does not remove production judgment.
Please Remember:
AI coding agents need context architecture.
A raw repo tells the agent what exists.
A knowledge graph tells the agent how everything connects.
--
Building production AI agents?
Start with the architecture.
Watch my free 30-min training + get 88 pages of production guides.
Join 46,000+ AI engineers, architects, and directors.
→ https://lnkd.in/eqDxeGFR

MEDIA Infographic titled “How to Use Knowledge Graphs for AI Coding Agents” by Dr. Maryam Miradi. The diagram shows a workflow where a raw codebase is transformed into a Code Knowledge Graph, which serves as the central context layer for AI coding agents. The process flows from raw code and documentation to graph building, knowledge graph creation, query layer, coding agent, verification, and safer code changes. Key benefits highlighted include reducing token waste, improving codebase navigation, discovering hidden relationships, querying before editing, and making code research and refactoring safer and more reliable.; document/carousel (possible)

LINKS
- https://www.maryammiradi.com/ai-agents-mastery?utm_source=linkedin_premiumbutton
- https://lnkd.in/eqDxeGFR

#482 Prasanth N.

AI / LLM

Managing Director @ WellsFargo | Gen AI |AI advocate
4mo · **View on LinkedIn**

AI engineering is rapidly becoming an optimization problem—not just a modeling problem.Everyone is talking about Agentic AI, Recursive Agents, RL, Distillation, LoRA, QLoRA, and Quantization. But the real enterprise question is:

How do we achieve frontier-model quality without frontier-model costs?

Consider a customer service operation handling millions of interactions per month. A frontier model may deliver the best accuracy, but running every request through it is often economically unsustainable.

The future of enterprise AI is not a single giant model. It is an ecosystem of:

• Orchestrator Agents
• Specialist Agents
• Reward Models
• Memory Systems
• Distilled Models
• Continuous Learning Loops

Here’s how the key techniques fit together:

Offline Distillation

Train smaller models (Qwen, Llama, Mistral) using trajectories generated by frontier models (GPT-5, Claude, Gemini).

Outcome: Retain most of the quality at a fraction of the inference cost.

On-Policy Distillation

The student model generates its own trajectories and learns from rewards, feedback, and selective teacher audits.

Outcome: Better planning, delegation, tool usage, and adaptation to real-world workflows.

LoRA

Fine-tune only lightweight adapters instead of billions of model parameters.

Outcome: Faster experimentation, lower training cost, easier governance, and rapid domain adaptation.

Examples:

• Banking LoRA
• Fraud LoRA
• Compliance LoRA
• Customer Service LoRA

QLoRA

Combine quantization and LoRA to fine-tune large models using significantly less GPU memory.

Outcome: Enterprise-grade model adaptation without massive infrastructure investments.

Quantization (PTQ/QAT)

Compress models from FP16 to INT8 or INT4 for deployment.

Outcome:

• Lower memory footprint
• Higher throughput
• Reduced latency
• Significant infrastructure savings

The trade-off is balancing cost, accuracy, and model perplexity.

The winning AI architecture is no longer:

Bigger Model = Better Outcome

It is:

Accuracy × Cost × Latency × Governance × Scalability

Organizations that can retain 90–95% of frontier-model performance while reducing inference costs by 10x–100x will have a significant competitive advantage.

#AI #AgenticAI #GenAI #LLM #ArtificialIntelligence #MachineLearning #EnterpriseAI #AIArchitecture #LoRA #QLoRA #Distillation #Quantization #MLOps #LLMOps #ReinforcementLearning #DataScience

#483 Satyam Jyottsana Gargee AI / LLM

Software engineer | AI & Tech | LinkedIn Top Voice 2025 | Ex-Microsoft | walmart | 450k+ community | Featured on Time Square | Josh Talk speaker

4mo · **View on LinkedIn**

My team reduced their LLM bill from $5,000 to $1,000 without changing a single prompt.

My friend told me recently that made me rethink how LLM APIs actually work.

I asked him what they optimized, it wasn’t the model or architecture, it was prompt caching.

Most people assume every API call starts from scratch sending system prompts, tools, and context again and again.

That’s exactly where the money goes.

Models build an internal representation.

Without caching → recomputed every request

With caching → reused

It’s same output in Lower cost.

Where prompt caching saves the most (and how to implement it right):

→ System prompts (biggest win)

If your system prompt is long and doesn’t change, you’re paying for it on every request.

Just cache it once instead of reprocessing it every time.

→ Tool / function definitions

Same tools on every call = same cost repeated. Cache them so the model doesn’t process them again and again.

→ RAG pipelines

If your system keeps pulling similar docs for different queries, you’re reprocessing the same context.

Caching here can give you 60%+ reuse.

→ The mistake that breaks everything

Caching only works if your prompt is structured right. Put static content first, dynamic input at the end.

Wrong order → no savings.

→ How to know it’s working

Check your cache hit rate (cache_read_input_tokens).

If it’s low, your setup is off.

If you’re spending big on LLM APIs,

a good chunk of it is just repeated prompts.

One small change can cut that fast.

#PromptCaching #AIEngineering #LLMCosts #Developers2026 #TechCareers #WorkingProfessionals #ProductionAI #RAG #GenerativeAI #MLOps

1/3

MEDIA document/carousel (possible)

#484 Java Developers Community (moderated)

System Design

Gurpreet Singh • 3rd+

4mo · **View on LinkedIn**

Strangler Pattern in Microservices

Migrating a monolith to microservices doesn’t need a big-bang rewrite.

The Strangler Pattern helps modernize applications step by step by gradually extracting features into independent microservices.

Incremental migration

Lower migration risk

Better scalability

Faster deployments

Easier maintenance

Real-world example:

E-commerce systems gradually move Orders, Payments, Users, and Notifications from a monolith into separate services while routing traffic through an API Gateway.

Modernize safely. Replace gradually. Scale confidently.

#Microservices #StranglerPattern #SystemDesign #SoftwareArchitecture #BackendDevelopment #Java #SpringBoot #DistributedSystems #CloudNative

#485 SOORAJ VIDYASAGAR

AI / LLM

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

4mo · View on LinkedIn

Spring AI for Java Developers: The Missing Guide

“Java developers don’t need to switch to Python to build AI applications anymore.”

For years, most AI frameworks and tutorials have been centered around Python. While Python remains dominant in AI research, enterprise applications are often built on Java and Spring.

That’s where Spring AI comes in.

Spring AI brings AI development into the familiar Spring ecosystem, allowing Java developers to build intelligent applications using the same patterns, tools, and practices they already know.

What is Spring AI?

Spring AI is a framework that simplifies integration with Large Language Models (LLMs), embedding models, vector databases, and AI providers such as:

OpenAI

Azure OpenAI

Anthropic Claude

Ollama

Vertex AI

Amazon Bedrock

It provides Spring-style abstractions for chat models, prompts, embeddings, vector stores, and AI agents.

Why Java Developers Should Care

Instead of learning an entirely new ecosystem, you can leverage:

* Spring Boot
* Dependency Injection
* Configuration Management
* Security
* Observability
* Testing Frameworks
* Production Deployment Patterns

This means AI applications can be developed and operated using proven enterprise practices.

Spring AI Architecture

A typical Spring AI application consists of:

User → Spring Boot Application → Spring AI → LLM Provider

Additional components can include:

* Vector Databases
* RAG Pipelines
* Tool Calling
* MCP Servers
* Agent Workflows
* Enterprise APIs

Spring AI vs LangChain

Area Spring AI LangChain

Language Java Python

Enterprise Integration Excellent Good

Spring Ecosystem Native N/A

Learning Curve for Java Teams Low Medium

Production Readiness High High

The goal isn’t to replace LangChain.

The goal is to give Java teams a first-class AI framework that fits naturally into existing enterprise architectures.

My Take

If your organization already runs Spring Boot microservices, Spring AI is one of the fastest paths to building:

RAG Applications

AI Agents

Knowledge Assistants

AI-Powered APIs

Enterprise Automation Solutions

The future of AI isn’t just models.

It’s integrating intelligence into the systems we already build.

#SpringAI #Java #SpringBoot #AI #GenerativeAI #LLM #AzureOpenAI #OpenAI #SoftwareArchitecture #AgentEngineering #EnterpriseAI

#486 Java Developers Community (moderated)

Java / Spring

Palmurugan C • 3rd+

4mo · View on LinkedIn

Ever wondered why arrays start at 0 and not 1?

It's not a quirk — it's math. The index is literally an offset from the base memory address. The first element is 0 steps away, so it's [0].

No subtraction. No overhead. Pure efficiency.

Languages like Lua, MATLAB, and R start at 1 — designed for mathematicians.

Languages like Java, C, Go, Python start at 0 — designed for systems and performance.

#Java #Programming #Arrays #CSFundamentals #SoftwareEngineering #TechTips #codewithpal

#487 Janjirala Anusha

AI / LLM

Technical Program Lead | AI, DevOps & Cloud | 15.5k+ Followers

4mo · View on LinkedIn

Unlocking the Power of AI with Claude Commands!

AI becomes much more powerful when you know how to communicate with it effectively.

I recently explored a comprehensive Claude Commands Cheat Sheet covering 90 useful commands that help improve productivity, content creation, coding, research, data analysis, and workflow automation.

Key Areas Covered:

✓ Start & Create

✓ Focus & Context

✓ Think & Solve

✓ Write & Edit

✓ Organize & Structure

✓ Code & Tech

✓ Data & Analysis

✓ Automate & Integrate
✓ Personalize & Control
✓ Learn & Research
✓ Collaborate & Share
My Key Takeaways:
Provide clear context for better responses.
Break complex problems into smaller tasks.
Use AI to accelerate learning and research.
Automate repetitive workflows whenever possible.
Structure prompts effectively to get higher-quality outputs.
Combine commands to unlock advanced use cases.
The future belongs to professionals who can effectively collaborate with AI. Learning prompt engineering and AI command frameworks is becoming as important as learning new technical skills.
The better your prompts, the better your results.
What is your favorite AI productivity trick or prompt? Share it in the comments! ↓
#AI #ClaudeAI #PromptEngineering #ArtificialIntelligence #Productivity #GenerativeAI #Automation #MachineLearning #TechCommunity #Learning #Innovation #Developer #DataAnalytics #FutureOfWork #AnushaJanjirala

#488 Mohd Salim

AI / LLM

Senior Architect | Enterprise Solutions | Java | Spring Boot | AWS | AI & Agentic AI Enthusiast | Driving Digital Transformation.

4mo · **View on LinkedIn**

How a Modern AI Agent Works
Most people think AI is just an LLM like GPT or Qwen.
In reality, an enterprise AI agent is a combination of multiple components working together to deliver accurate and useful responses.
User
↓
API/UI Layer
* Receives user requests
↓
Agent Orchestrator
* Coordinates the entire workflow
* Decides what information is needed
↓
Memory
* Conversation history
* User profile
* Preferences
RAG
* Vector Database
* Documents
* Runbooks
Tools
* APIs & Enterprise Systems

* AWS
* Database
* Jira
* GitHub
* Linux

↓

Prompt Builder

* Combines context from Memory, RAG, and Tools

↓

LLM Model (GPT / Qwen)

* Understands the request
* Generates the response

↓

Response Generator

* Formats and delivers the final answer

↓

User

Why this architecture matters

A good AI agent doesn't rely only on the model's knowledge.

It can:

• Remember previous interactions
• Access company documents and knowledge bases
• Connect with business systems and tools
• Gather real-time information
• Generate more accurate and contextual responses

The real power of AI comes from combining Memory + RAG + Tools + LLMs into a single intelligent system.

This is the foundation of next-generation enterprise AI solutions.

What component do you think is the most challenging to implement: Memory, RAG, Tool Integration, or Agent Orchestration?

#AI #AIAgents #AgenticAI #RAG #LLM #GenerativeAI #EnterpriseAI #SoftwareArchitecture #AWS #Technology #Innovation #ArtificialIntelligence #TechLeadership

MEDIA document/carousel (possible)

#489 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Yogana S • 3rd+ | Book an appointment

4mo · **View on LinkedIn**

Welcome to the next level of RAG Optimization!

Most RAG pipelines fail not because of the model...

But because of how the data is indexed.

Here are 4 powerful ways to optimize your RAG Indexing Pipeline:

Typical RAG

Parent–Child Chunking

Hypothetical Queries

Summary-Based Indexing

Better indexing = Better retrieval = Better AI responses

If you're building AI applications, LLM products, or GenAI workflows, optimizing retrieval is no longer optional.

Which indexing strategy are you currently using in your RAG pipeline?

Drop your thoughts in the comments ↓

And follow for more AI + GenAI architecture insights.

#RAG #GenerativeAI #LLM #ArtificialIntelligence #MachineLearning #AIEngineering #DataEngineering #VectorDatabase #LangChain #OpenAI #MLOps #AIAgents #Tech #AI #DeepLearning

LINKS

- https://topmate.io/yoganawithai/2324050

#490 Vanshika Agarwal

System Design

SDE-2 @Atlassian | Ex-Google, Oracle | Helping Engineers Crack Tier-1 Tech Interviews | AI & DevTools Creator | workwithvanshiagarwal@gmail.com

4mo · **View on LinkedIn**

I’ve sat in interviews at 3 top product giants like Google , Atlassian , & Oracle ...

And here are 3 things you NEED to know before your next SDE-2/3 level interview ↓

TIP #1: Give Data Structures & Algorithms (DSA) fundamentals PRIORITY.

→ Most tech giants, esp. Google, expect you to be strong in Arrays, Strings, Trees, Graphs, and DP.

→ Don’t just solve random problems. Instead, focus on mastering core patterns, Sliding Window, Binary Search, DFS/BFS, Trie, DP state transitions.

TIP #2: Don’t think Behavioral Rounds are “OPTIONAL”

At PBCs like Amazon, your real contributions and leadership will matter.

So always quantify your impact → “Led a team of 4 engineers to reduce API latency by 30%” is far better than “Worked on APIs”.

TIP #3: System Design will become UNAVOIDABLE (even mid-level)

Interviewers will expect you to design scalable, reliable systems, and explain tradeoffs clearly, especially at Microsoft.

Practice structuring your thought process ↓

→ Start with requirements → High-level architecture → Data modeling → API contracts → Scalability challenges.

This is what I’ve learned along the way as a software engineer with 5+ YOE, & these actionable tips can help you make your next switch to MAANG-level PBCs!

So if you’re working a 9-5 and struggling to prepare for your next big role...

I highly recommend giving Bosscoder Academy a try.

Explore their programs here: bcalinks.com/1YY737e

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, & AI

Real world, AI-powered projects & 24/7 doubt-solving

1:1 mentorship from industry experts

Resume reviews, mock interviews, & job switch support

#sdeprep #systemdesign #interviewtips #tech #jobswitch #maang #pbc #collab

LINKS

- http://bcalinks.com/1YY737e

#491 Balaji Mani

AI / LLM

AI Governance & Agentic AI Leader | Trusted Advisor for Enterprise AI Transformation | Responsible AI, AI Risk & Enterprise Architecture | Helping Organizations Govern, Scale & Secure AI

4mo · View on LinkedIn

Agentic AI Engineering Stack – The Future of Enterprise AI Systems

Most organisations still think AI = chatbot + LLM.

That model is already outdated.

We are now moving into the era of Agentic AI — where systems don’t just respond... they plan, act, and autonomously execute business tasks.

What is an Agentic AI Stack?

An Agentic AI system is a closed-loop autonomous system that:

Understands intent

Plans multi-step actions

Uses tools & APIs

Learns from memory

Continuously evaluates and improves

This requires a full engineering stack, not just a model.

The Modern Agentic AI Engineering Stack

Here’s how enterprise-grade AI systems are structured:

1. Infrastructure Layer

Cloud / On-Prem compute, storage, vector databases
→ Foundation for scalability & performance

2. Foundation Models (Brain)

LLMs like GPT, Llama, Claude
→ Reasoning, planning, decision-making

3. Agent Runtime & State

Session handling, memory persistence
→ Enables long-running, context-aware agents

4. Orchestration Layer

LangGraph, AutoGen, Agent frameworks
→ Multi-step workflows & multi-agent coordination

5. Memory & Knowledge (RAG)

Vector DB + enterprise data
→ Context grounding + intelligence

6. Tooling & Action Layer

APIs, enterprise systems, automation
→ Converts AI → real business actions

7. Application Layer

Customer service, network ops, finance bots
→ Business value realisation

8. Observability & Guardrails

Monitoring, evaluation, safety controls
→ Trust, reliability & compliance

9. Governance & Identity

RBAC, audit logs, policy enforcement
→ Enterprise control & accountability

How it works (simple view)

User request → Agent reasoning → Plan → Retrieve context → Use tools → Evaluate → Act → Log & govern

This loop continues until the goal is achieved.

Key Shift for Enterprises

✗ Old mindset:
“Build a chatbot using LLM APIs”

✓ New reality:
“Build autonomous, governed, multi-agent systems”

Why this matters (especially for Telecom & Enterprises)

Automates complex workflows (not just conversations)

Reduces operational cost & manual effort

Enables real-time decisioning

Creates AI-native operating models

Critical takeaway

The real competitive advantage is NOT the model
...it’s how you design:

→ Orchestration
→ Memory
→ Governance
→ Observability

#AgenticAI #AIArchitecture #EnterpriseAI #LLMOps #AIEngineering #DigitalTransformation #TelecomAI #FutureOfWork

#492 Akhila G

AI / LLM

Making AI feel less complex | 20k+ Followers | GenAI Trainer | Senior Software Engineer

4mo · View on LinkedIn

Asked by Deloiite for an AI Engineer role Round-2

Your RAG system works great... until you see the invoice: $20,000/month — just on embeddings. And the worst part? Most of it is wasted. ↓

Here's how I'd fix it:

1. Find the root cause: you're re-embedding unchanged content

→ Tweaking chunk size shouldn't re-embed millions of identical paragraphs

→ The fix is content-addressable caching — embed by what the text is, not when you ran the job

2. Hash every chunk → cache the embedding

→ Key = hash(chunk_text + model + model_version)

→ Before embedding, check the cache. Hit → reuse. Miss → embed once, store it.

→ Same text + same model = never pay twice

3. Version everything

→ Track embedding_model, model_version, and chunking_config as metadata

→ A new chunking strategy creates a new version namespace — old vectors stay reusable, you don't blow away the index

4. Selective / incremental re-embedding

→ Diff old vs new chunks → only embed the delta (added/changed/re-split chunks)

→ Unchanged chunks carry over untouched → experimentation cost drops from "whole corpus" to "what actually changed"

5. Make experiments cheap

→ Run new chunking strategies on a sample / subset first, validate retrieval quality, then roll out

→ Batch embedding calls + use a cheaper/smaller embedding model for experiments

6. Monitor the spend

→ Track cache hit-rate, $ per re-index, and tokens embedded per run

→ A healthy pipeline should be hitting cache 90%+ of the time on re-runs

The core insight: never re-embed what hasn't changed. Hash-based caching + versioning + delta re-embedding turns a $20K full re-index into a few dollars of incremental work.

==========

We've covered questions like these in my AI Engineering Interview Master Bundle, a comprehensive set of 22 courses designed for real interview prep.

Explore the full guide here → https://lnkd.in/gqFkWZd4

LINKS

- https://lnkd.in/gqFkWZd4

#493 saisuresh yerragogu System Design

Senior Cloud & DevOps Engineer | AWS | Azure | Kubernetes | Terraform | GitHub Actions | CI/CD Automation | Docker | Infrastructure as Code

4mo · **View on LinkedIn**

Why IRCTC Faces Challenges During Tatkal Booking While Amazon Handles Prime Day Traffic at Scale

Both IRCTC and Amazon experience enormous traffic surges.

Millions of users.

Millions of requests.

Thousands of transactions per second.

Yet, IRCTC often encounters performance bottlenecks during Tatkal booking windows, while Amazon continues to deliver a seamless experience during Prime Day, Black Friday, and other major sales events.

The key difference lies not in server capacity, but in system architecture and traffic distribution.

Understanding the Challenge

During Tatkal booking, a massive number of users attempt to access the same resource at the same moment:

- Same train
- Same quota
- Same seat inventory

This creates intense contention on backend systems, leading to:

Database row locking

Inventory conflicts

Connection pool exhaustion

Sudden traffic spikes

Increased response times and failures

The challenge is not simply handling millions of users—it's handling millions of users competing for the exact same resource simultaneously.

Why Amazon Scales More Effectively

Amazon's traffic patterns are fundamentally different:

- Users browse millions of products
- Requests are distributed across thousands of services
- Inventory is spread across multiple systems
- Workloads are processed asynchronously where possible

This distributed model significantly reduces contention and enables horizontal scaling.

Architectural Principles Behind Large-Scale Systems

Modern internet-scale platforms rely on:

✓ Horizontal Auto Scaling
✓ Load Balancing Across Multiple Regions and Availability Zones
✓ Caching Layers to Reduce Database Load
✓ Microservices Architecture for Independent Scaling
✓ Queue-Based Processing for Traffic Absorption
✓ Multi-Region Deployments for High Availability

These patterns allow systems to absorb demand spikes without overwhelming critical backend components.

Key Takeaway

Scalability is not achieved by deploying larger servers.

It is achieved by designing systems that can:

• Distribute load efficiently
• Minimize resource contention
• Scale horizontally on demand
• Isolate failures
• Maintain performance under peak traffic conditions

As engineers, our goal is not just to handle growth but to build architectures that remain resilient when demand is at its highest.

Architecture, not infrastructure alone, is what enables systems to scale reliably.

#DevOps #CloudComputing #AWS #Azure #SystemDesign #Scalability #Architecture #Kubernetes #SRE #PlatformEngineering #DistributedSystems #CloudArchitecture #Engineering #Technology #SoftwareArchitecture #PerformanceEngineering

MEDIA advertisement

#494 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · **View on LinkedIn**

Most AI teams are losing time because every task goes into the same box.

Become better at AI in just 1 minute a day.

Get the AI newsletter smart leaders read.

Sign up free now → aiforleaders.com

The fastest AI upgrade is not another subscription.

It is routing work to the right engine.

Most leaders still ask ChatGPT to write, research, analyze, browse, summarize, and sense-check everything.

That works for simple requests.

It breaks down when the work gets specific.

Here is the practical split I would use.

→ ChatGPT

Use it as the creator.

✓ Draft complex SOPs, policies, legal-style documents, and first-pass strategy memos.
✓ Rewrite a rough email into something clear, direct, and professional.
✓ Brainstorm 10 campaign angles before your team commits to one direction.
✓ Turn messy notes into a structured plan with owners, next steps, and open questions.

Consequence:
If the task needs reasoning, structure, or transformation, ChatGPT should probably be first.
→ Perplexity
Use it as the researcher.
✓ Find answers with citations instead of relying on unsupported model memory.
✓ Compare competitor claims, pricing pages, product features, and market language.
✓ Dig through academic papers, PDFs, reports, and source-heavy questions.
✓ Verify a claim before it goes into a board deck, sales page, or investor update.
Consequence:
If the answer needs sources, Perplexity reduces the risk of confident nonsense.
→ Grok
Use it as the news desk.
✓ Track viral topics before they peak.
✓ Read the live mood around launches, controversies, creators, and markets.
✓ Spot what people are actually saying, not just what polished summaries report.
Consequence:
If the work depends on current conversation, stale context will mislead you.
→ Gemini
Use it as the ecosystem layer.
✓ Pull signals from Gmail, Drive, Docs, Sheets, Slides, images, and video.
✓ Turn long threads into action items your team can actually execute.
✓ Summarize decks, extract chart data, and create spreadsheet formulas.
Consequence:
If the work lives in Google Workspace, Gemini belongs in the workflow.
The point is simple.
Stop asking one model to be the whole stack.
Build a routing habit.
Creator.
Researcher.
News desk.
Workspace layer.
Which AI task in your company is going to the wrong tool today?

> **MEDIA** Most AI teams are losing time because every task goes into the same box. Become better at AI in just 1 minute a day. Get the AI newsletter smart leaders read. Sign up free now → aiforleaders.com _____ The fastest AI upgrade is not another subscription. It is routing work to the right engine. Most leaders still ask ChatGPT to write, research, analyze, browse, summarize, and sense-check everything. That works for simple requests. It breaks down when the work gets specific. Here is the practical split I would use. → ChatGPT Use it as the creator. ✓ Draft complex SOPs, policies, legal-style documents, and first-pass strategy memos. ✓ Rewrite a rough email into something clear, direct, and professional. ✓ Brainstorm 10 campaign angles before your team commits to one direction. ✓ Turn messy notes into a structured plan with owners, next steps, and open questions. Consequence: If the task needs; document/carousel (possible)

LINKS
- http://aiforleaders.com/jb
- http://aiforleaders.com

#495 Gabriel Millien

AI / LLM

Enterprise AI Execution Architect | Closing the AI Execution Gap | $100M+ in AI-Driven Results | Trusted by Fortune 500s: Nestlé • Pfizer • UL • Sanofi | AI Transformation |Board Member | Fractional CAO | Keynote Speaker | View my services

4mo · **View on LinkedIn**

Prompt engineering gets the headlines.
Harness engineering ships the product.
Most teams are funding the first and wondering why nothing reaches production.
The infographic above explains the three skills people mix up constantly.
Here is each one in plain terms.
1. Prompt engineering. The message you send the AI.
The instruction itself.
Who it should act as, what you want, an example or two, the format to answer in.
One good prompt, one good answer.
2. Context engineering. What the AI gets to see.
An AI can only hold so much at once.
You decide what goes in: the question, documents, past messages, tool results.
Keep what matters. Drop what does not.
Get this wrong and even a perfect prompt fails.
3. Harness engineering. The machine around it all.
The system that runs in the real world.
It gathers the information, lets the AI act, then checks the work.
Wrong answer? It tries again until it passes.
Now the part the headlines skip.
These three are not steps in a line.
They are layers, one inside the other.
A good prompt sits inside good context.
Good context runs inside a good harness.
Pull out one layer and the other two collapse.
So when an AI pilot dies in production, the model usually was not the problem.
The harness was never built.
That missing layer is what I call the AI Execution Gap.
The distance between an AI that demos well and one that survives a real customer.
Prompt skills get you a clever answer.
Harness skills get you something you can actually ship.
So before you fund another prompt workshop, ask one question:
When your AI gets it wrong in front of a customer, what catches it before they do?
If the answer is a person, you have a prompt.
If the answer is a system, you have a product.
Repost to help someone in your network avoid this mistake.
Save it for your next AI initiative.
Follow Gabriel Millien to stay ahead in AI while everyone else plays catch-up.
Credit: Rathnakumar Udayakumar, give him a follow.

MEDIA document/carousel (possible)

#496 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · View on LinkedIn

ChatGPT is not average. Most prompts are.
Become better at AI in just 1 minute a day.
Get the AI newsletter smart leaders read.
Sign up free now → aiforleaders.com

_____

The fastest way to improve ChatGPT is to improve the question.
Most people treat it like Google.
They type a loose request.
They get a loose answer.
Then they blame the model.
The better move:
Use prompts that force structure.
Use prompts that force feedback.
Use prompts that force the model to think with you.
Here are 10 worth keeping close:
→ 80/20 Learning
Ask ChatGPT to find the 20% of a topic that unlocks the remaining 80%.
Use it before opening another course, article, or tutorial.
→ Writing Feedback
Paste your draft and ask for grammar fixes plus clarity improvements.
The diagnostic check: does the rewrite make the point faster?
→ Research Intern
Give it a subject, audience, and outcome.
Ask for a report with steps, not just a summary.
→ 30-Day Skill Plan
Name the skill.
Ask for a beginner schedule with daily practice.
Vague ambition becomes a visible plan.
→ Knowledge Test
Ask ChatGPT to quiz you on the topic.
Then make it explain the gaps after each answer.
That beats passive reading.
→ Prompt Generator
Ask for 10 prompts around one task or outcome.
You are not looking for one answer.
You are building a reusable prompt set.
→ Mastery Roadmap
Ask for phases, exercises, tools, resources, and timelines.
This turns scattered learning into a path.
→ Expert Reasoning
Assign a role, give the question, and ask it to reason step by step.
Use this when the cost of a shallow answer is high.
→ Idea Engine

Ask for options against a specific goal.
Then rank them by effort, risk, or upside.
→ Summarizer
Paste the text.
Ask for key insights and important facts.
Use it to reduce noise before deciding.
Try one prompt on a real task today.
Then save the version that works.
Small prompt systems compound faster than random chats.
The pattern is simple:
Bad prompts ask for outputs.
Better prompts create thinking systems.
Which one would improve your work today?

> **MEDIA** ChatGPT is not average. Most prompts are. Become better at AI in just 1 minute a day. Get the AI newsletter smart leaders read. Sign up free now → aiforleaders.com _____ The fastest way to improve ChatGPT is to improve the question. Most people treat it like Google. They type a loose request. They get a loose answer. Then they blame the model. The better move: Use prompts that force structure. Use prompts that force feedback. Use prompts that force the model to think with you. Here are 10 worth keeping close: → 80/20 Learning Ask ChatGPT to find the 20% of a topic that unlocks the remaining 80%. Use it before opening another course, article, or tutorial. → Writing Feedback Paste your draft and ask for grammar fixes plus clarity improvements. The diagnostic check: does the rewrite make the point faster? → Research Intern Give it a subject, audience, and outcome. Ask for a rep

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com

#497 **vaibhav sable** **Java / Spring**

AI Engineer | 12.7K+ LinkedIn Followers | RAG ,Agentic AI (LangChain • LangGraph • FastAPI) | MLflow • Docker • K8s | Open to Work

4mo · **View on LinkedIn**

Most Java Backend interviews don't fail candidates on coding.
They fail candidates on internals they haven't revised in years.
After working through real backend projects and interview preparation, I noticed a common pattern:
Developers are comfortable writing APIs.
But many struggle when the discussion moves to:
(!) JVM Internals
(!) Concurrency & Multithreading
(!) Spring Boot Internals
(!) Microservices Patterns
(!) Kafka Reliability
(!) Database Performance
(!) System Design Trade-offs
That's why I compiled a list of 150 Java Backend Interview Questions.
The collection covers:
✓ Core Java & JVM
✓ Multithreading & Concurrency
✓ Java 8 Streams
✓ Spring Boot

✓ JPA & Hibernate
✓ Microservices
✓ Apache Kafka
✓ SQL & Databases
✓ Security
✓ System Design
A few topics that frequently appear in backend interviews:
HashMap Internals
ConcurrentHashMap
CompletableFuture
@Transactional Internals
Saga Pattern
Kafka Consumer Groups
JWT vs OAuth2
URL Shortener Design
Save this post for your next interview preparation session.
What's the toughest Java Backend interview question you've faced?
↓ Share it in the comments.
#Java #SpringBoot #Microservices #Kafka #SQL #SystemDesign #BackendDeveloper #InterviewPreparation
Java_Backend_Interview_Questio
.
6 pages

MEDIA advertisement; document/carousel (possible)

#498 **Harsh Arora** **Kafka / Messaging**

Java Developer @ Morgan Stanley | Tech Content Creator | Sharing Software Engineering & Career Insights | 18K+ Linkedin Followers

4mo · **View on LinkedIn**

Apache Kafka Interview Preparation Sheet
If you're a Java Backend, Spring Boot, or Microservices developer preparing for interviews, this cheat sheet can save you a lot of revision time.
Topics covered:
✓ Kafka Architecture
✓ Producer & Consumer
✓ Topics & Partitions
✓ Offsets & Offset Management
✓ Consumer Groups & Rebalancing
✓ Replication, Leader/Follower & ISR
✓ Serializers & Deserializers
✓ Retry Mechanism & DLT
✓ Idempotent Producer
✓ Exactly Once Semantics (EOS)
✓ KafkaTemplate & @KafkaListener
✓ Kafka vs RabbitMQ
✓ Real-world Kafka Use Cases

Whether you're preparing for a job switch, refreshing your Kafka fundamentals, or getting ready for a system design round, this sheet can serve as a quick revision guide.

Which Kafka topic do you find the most challenging in interviews?

#ApacheKafka #Kafka #Java #SpringBoot #Microservices #BackendDeveloper #JavaDeveloper #SoftwareEngineer #InterviewPreparation #SystemDesign #DistributedSystems #TechCareers

Follow for more Harsh Arora

#499 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

4mo · View on LinkedIn

Claude certification is not a prompt test.

Become better at AI in just 1 minute a day.

Get the AI newsletter smart leaders read.

Sign up free now → aiforleaders.com

_____

It is closer to a systems exam for people who can reason through real Claude workflows.

Anthropic's Claude Architect Certification has:

→ 60 multiple-choice questions

→ 5 competency areas

→ 1 single session

→ no external resources

→ no breaks

That changes how you prepare.

You cannot just watch videos.

You need working fluency across APIs, agents, MCP, Claude Code, and evaluation.

Here is the roadmap I would use:

→ Week 1: Learn the foundation

Complete the recommended courses:

Building with the Claude API.

Introduction to Model Context Protocol.

Claude Code in Action.

Claude 101.

Courses link:

https://lnkd.in/eZw9VB4g

→ Week 2: Build with the stack

Use Claude Code.

Use the Agent SDK.

Use the Anthropic API.

Build something with MCP.

The diagnostic check:

If you cannot explain what your tool call does, when it should escalate, and what failure mode it creates, you are not ready yet.

→ Week 3: Study the exam structure

Review the 6 exam scenarios.

Map the 5 domains to the decisions they test.

Connect each task assessment to the skill being tested before you practise.

→ Week 4: Do the guide exercises

Build a multi-tool agent with escalation logic.

Configure Claude Code for a team workflow.

Create a structured data extraction pipeline.

Debug a multi-agent research pipeline.

→ Week 5: Take the practice exam

Aim for more than 850 / 1000.

Then review what you missed and rebuild the weak scenario from scratch.

→ Week 6: Take the real exam

Only one attempt is allowed.

Treat it like a real certification, not a casual quiz.

Registration link:

https://lnkd.in/eEYwUGV5

One caveat:

Early access is limited to eligible Anthropic Partners right now, so check eligibility before planning your exam date.

But the courses are useful either way because the same skills show up in real Claude implementation work.

The real point is simple.

Claude expertise is moving from prompting into architecture.

Tools.

Agents.

Workflows.

Evaluation.

Systems thinking.

That is where the advantage is, especially for teams moving from casual usage to production workflows.

Are you preparing like a user or like an architect?

> **MEDIA** Claude certification is not a prompt test. Become better at AI in just 1 minute a day. Get the AI newsletter smart leaders read. Sign up free now → aiforleaders.com _____ It is closer to a systems exam for people who can reason through real Claude workflows. Anthropic's Claude Architect Certification has: → 60 multiple-choice questions → 5 competency areas → 1 single session → no external resources → no breaks That changes how you prepare. You cannot just watch videos. You need working fluency across APIs, agents, MCP, Claude Code, and evaluation. Here is the roadmap I would use: → Week 1: Learn the foundation Complete the recommended courses: Building with the Claude API. Introduction to Model Context Protocol. Claude Code in Action. Claude 101. Courses link: https://lnkd.in/eZw9VB4g → Week 2: Build with the stack Use Claude Code. Use the Agent SDK. Use the Anthropic API. Build som

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com
- https://lnkd.in/eZw9VB4g
- https://lnkd.in/eEYwUGV5

#500 **Srinath P.** **Java / Spring**

Senior Solution Architect | Distributed Systems | Java (17/21/25) | Microservices | Kubernetes | AI-Enabled Systems | High-Scale Platforms

4mo · **View on LinkedIn**

Did You Know? You Can Build Machine Learning Models in Pure Java Without Python.

When most developers think about Machine Learning, they immediately think of Python, Scikit-Learn, or PyTorch.

But for Java-first enterprises, there's an interesting open-source alternative: Tribuo.

✓ Native Java Machine Learning Library

✓ Classification, Regression & Clustering

✓ Anomaly Detection Support

✓ XGBoost Integration

✓ Easy Spring Boot Integration

✓ No Python Runtime Required

A modern Java AI architecture can look like this:

Kafka → Spring Boot → Tribuo → Predictions → Business Applications

This makes Tribuo a compelling option for organizations that want to keep their AI and ML workloads within the Java ecosystem while reducing operational complexity.

While Python continues to dominate the ML landscape, Tribuo proves that Java developers don't have to leave their ecosystem to leverage Machine Learning.

Have you used Tribuo or any Java-based ML frameworks in production?

#Java #MachineLearning #Tribuo #SpringBoot #Kafka #ArtificialIntelligence #OpenSource #SoftwareArchitecture #DataEngineering #EnterpriseAI #JavaDevelopers #TechInnovation

#501 SOORAJ VIDYASAGAR

System Design

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

4mo · **View on LinkedIn**

Stop Memorizing. Start Thinking Like an Architect.

One of the biggest shifts in a software engineer’s career is moving from remembering technologies to understanding the principles behind them.

Senior engineers and architects don’t solve problems by memorizing frameworks. They rely on a mental model built around a few fundamental concepts:

• Statelessness

• Caching

• CAP Theorem

• Message Queues

• Databases

• API Design

Every large-scale system challenge eventually becomes a trade-off between these pillars:

* Consistency vs Availability

* Performance vs Freshness

* Simplicity vs Scalability

* Reliability vs Cost

The engineers who understand these trade-offs can design systems that remain resilient when traffic spikes, services fail, networks partition, or data grows exponentially.

This is also why system design interviews focus less on code and more on reasoning. Interviewers want to see how you think, not what you can memorize.

The path from developer to architect is not learning more tools. It’s developing better mental models for making technical decisions.

Master the principles. The technologies will change, but the trade-offs remain the same.

#502 Sai Ram Somanaboina

System Design

Engineering Manager at NowFloats(Jio Group) | 15 years in Engineering

4mo · View on LinkedIn

JWTs are used by 67% of modern web apps for authentication. But the moment a user clicks "logout," your stateless token is still valid and there is nothing your server can do about it.

JWT is stateless by design. But most engineers forget one terrifying fact: if you cannot reach the server to invalidate a token, the token is still a valid key to everything.

That is not a bug.

That is how JWTs work.

[1] Deleting the token from the client is not logout

Most apps handle logout by removing the JWT from localStorage or cookies on the frontend. The user feels logged out. The UI redirects to the login page.

But the token itself is still valid. If an attacker copied that token before logout, they can keep making authenticated requests until the token expires. The server has no idea the user "logged out" because the server never stored the session in the first place.

Stateless means the server trusts the token blindly. It checks the signature, reads the expiry, and lets the request through. There is no session table to delete from.

[2] The real answer: short-lived tokens + a revocation layer

Since you cannot truly invalidate a signed JWT without breaking statelessness, you work around it.

Use short-lived access tokens with a 5 to 15 minute expiry. The shorter the token life, the smaller the window of abuse after logout.

Pair them with refresh tokens stored server-side in a database or Redis. When the user logs out, you delete the refresh token. The access token may survive a few more minutes, but the user can never get a new one.

For immediate revocation, maintain a token blacklist or blocklist. On logout, push the token's jti (unique ID) into Redis with a TTL matching the token's remaining life. On every request, check the blacklist before trusting the token.

Yes, this adds a database lookup. Yes, this breaks pure statelessness. But pure statelessness and instant logout cannot coexist. You pick which tradeoff you accept.

[3] Other strategies worth knowing

Token versioning: store a tokenVersion on each user record. Increment it on logout. If the JWT's version does not match the database, reject it.

Event-driven invalidation: publish a logout event through a message broker so all services know the token is dead.

Gateway-level enforcement: let your API gateway check a centralized revocation list so individual services stay stateless while the gateway handles security.

JWT gives you stateless authentication.

But logout is a stateful action.

If you pretend otherwise, a "logged out" user's token is still an open door to their account.

#503 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Lucia Moretti • 3rd+

4mo · View on LinkedIn

FREE AI Courses!

1. Introduction to Large Language Models– https://lnkd.in/gfKGzHSE

2. Generative AI with Large Language Models– https://lnkd.in/gwPN9VCH

3. Large Language Models (LLMs) Concepts– https://lnkd.in/g84xe52n
4. Prompt Engineering for ChatGPT– https://lnkd.in/gP6qsPWY
5. Introduction to LLMs in Python– https://lnkd.in/ggd7-wGR
6. ChatGPT Teach-Out– https://lnkd.in/gcKuQEPV
7. Large Language Models for Business– https://lnkd.in/g2b5RYdu
8. Introduction to Large Language Models with Google Cloud– https://lnkd.in/gu5m56MJ
9. Finetuning Large Language Models– https://lnkd.in/ga8qpuQ8
10. LangChain with Python Bootcamp– https://lnkd.in/g3SwUDAq

MEDIA advertisement

LINKS

- https://lnkd.in/gfKGzHSE
- https://lnkd.in/gwPN9VCH
- https://lnkd.in/g84xe52n
- https://lnkd.in/gP6qsPWY
- https://lnkd.in/ggd7-wGR
- https://lnkd.in/gcKuQEPV
- https://lnkd.in/g2b5RYdu
- https://lnkd.in/gu5m56MJ
- https://lnkd.in/ga8qpuQ8
- https://lnkd.in/g3SwUDAq

#504 AI For SEO

AI / LLM

4mo · **View on LinkedIn**

RAG vs AI Agents

Become better at AI in just 1 minute a day.

Get the AI newsletter smart leaders read.

Sign up free now → aiforleaders.com

Original post:

_____

Retrieval systems answer questions. AI Agents take actions.

If you are still confusing RAG (Retrieval-Augmented Generation) with AI Agents, you are missing the shift happening in enterprise AI right now.

While both leverage Large Language Models (LLMs), their architecture, capabilities, and purposes are fundamentally different. Here is the breakdown you need to know:

RAG: The Smart Librarian

Think of RAG as a hyper-efficient research assistant. It takes a user query, searches an enterprise database or document archive, finds the right context, and uses an LLM to generate a grounded, accurate answer.

Workflow: Linear and single-step (Query → Search → Generate → Answer).

Memory: Limited to the immediate context window.

Autonomy: Reactive. It only speaks when spoken to.

Best For: Enterprise search, document Q&A, and customer support chatbots.

AI Agents: The Intelligent Operator

AI Agents don’t just find information. They execute multi-step workflows autonomously to achieve a high-level goal. They reason, plan, adapt, and use external tools.

Workflow: Dynamic and recursive loops (Goal → Plan → Use Tool → Evaluate → Repeat until done).

Memory: Persistent (remembers past actions and learns over time).

Autonomy: Proactive. It determines its own sub-tasks and calls APIs, writes code, or automates browsers.

Best For: Autonomous research, end-to-end workflow automation, and AI copilots.

The Bottom Line:

RAG is phenomenal for bridging the gap between static LLM knowledge and your private enterprise data. But if you want to automate actual business processes, orchestrate workflows, and let AI execute tasks independently, you are building an AI Agent.

Credit to Kavi Priyan R. Follow him for more.

MEDIA Infographic comparing RAG and AI Agents with flowcharts and example use cases.; document/carousel (possible)

LINKS

- http://aiforleaders.com

#505 Mansi Kaushik

System Design

Java Technical Lead | 10+ Years Experience | Java, Spring Boot & Microservices | CI/CD | Banking & Financial Services | Scaling Systems for 1M+ Daily Transactions

4mo · **View on LinkedIn**

Understanding the Saga Pattern in Microservices

In a distributed microservices architecture, maintaining data consistency across multiple services can be challenging. Traditional distributed transactions (2PC) often introduce performance bottlenecks and tight coupling.

The Saga Pattern addresses this by breaking a business transaction into a series of local transactions. Each service performs its operation independently and publishes an event for the next step. If a failure occurs, compensating transactions are executed to undo previously completed actions.

Example: E-Commerce Order Processing

✓ Create Order

✓ Reserve Inventory

✓ Process Payment

✓ Create Shipment

If payment fails, the system triggers compensating actions such as releasing inventory and cancelling the order, ensuring eventual consistency.

Key Benefits:

✓ Improved scalability

✓ Better fault tolerance

✓ Loose coupling between services

✓ No dependency on distributed transactions (2PC)

Implementation Approaches:

Choreography – Event-driven communication between services

Orchestration – A central coordinator manages the workflow

The Saga Pattern is one of the most important patterns for building resilient, cloud-native microservices.

What other microservices design patterns do you use frequently in production systems?

#Microservices #Java #SpringBoot #SystemDesign #SoftwareArchitecture #DistributedSystems #Kafka #EventDrivenArchitecture #TechLeadership #BackendDevelopment

#506 Java Developers Community (moderated)

Other

Naveengandhi C. • 3rd+ | I used Reflection for years before discovering a JDK feature that solved the same problem more efficiently. | MethodHandle. | It's been around since Java 7, yet many developers never touch it. | Like Reflection, it lets you invoke methods dynamically. Unlike Reflection, it gives the JVM far more room to optimize. | Most Reflection code looks familiar: | Method method = MyService.class | .getDeclaredMethod("processOrder", Order.class); | Object result = method.invoke(serviceInstance, order); | It works, but every invocation carries overhead: | Runtime access checks | Limited JIT optimization | Object casting | String-based lookups | Runtime-only errors | For occasional use, that's perfectly fine. | For serializers, mappers, event processors, or proxy layers handling thousands of calls per second, it can become surprisingly expensive. | That's where MethodHandles shine. | MethodHandle handle = lookup.findVirtual(| MyService.class, | "processOrder", | MethodType.methodType(Result.class, Order.class) |); | Result result = | (Result) handle.invoke(serviceInstance, order); | The setup is more verbose, but the cost is paid upfront. Once created, repeated invocations are significantly cheaper, and the JVM can optimize them far more effectively. | What I find most valuable isn't even the performance. | It's the failure model. | Reflection often fails at runtime. | MethodHandles frequently fail during lookup, which means problems show up during startup or testing instead of during a production incident. | The trade-off? | The API is more complex, and if the code runs only occasionally, Reflection is usually good enough. | But when profiling points to Reflection in a hot path, MethodHandle is one of the first optimizations worth exploring. | It's one of those JDK features that's been available for years, quietly powering many modern frameworks behind the scenes. | Have you used MethodHandles in production, or is Reflection still your default choice? | #Java #JVM #PerformanceEngineering #SoftwareEngineering #BackendEngineering #JavaPerformance #MethodHandles | 78 | 1 | 4 | | Karl Heinz Marbaise Premium Profile 3rd+ | Karl Heinz Marbaise | Java Champion, Developer/DevOps owner of SoEBeS, Apache Software Foundation Member, Apache Maven PMC

4mo · **View on LinkedIn**

Based on which measurements are method-handles faster? Do you have a reference project which shows such things?

> **MEDIA** advertisement

#507 Priyanka Logani

System Design

Senior Full Stack Engineer | Java 17 • Spring Boot •.NET Core • Microservices • Kafka • Angular | AWS • Azure • GCP | Cloud-Native Architecture • CI/CD • Kubernetes • Event-Driven Platforms • APIs | LLMs

4mo · **View on LinkedIn**

Every Database Transaction Relies On These 4 Principles

When money is transferred, orders are placed, or user data is updated, databases need guarantees.

That's where ACID comes in.

ACID ensures transactions remain reliable, even when systems crash, requests fail, or multiple users access data simultaneously.

Atomicity

↳ A transaction is completed entirely or not at all.

Transfer $100 between accounts?

Both debit and credit succeed together, or neither happens.

Consistency

↳ Every transaction moves the database from one valid state to another.

Constraints, rules, and relationships are always preserved.

Isolation

↳ Concurrent transactions don't interfere with each other.

Each transaction behaves as if it's running alone.

Durability

↳ Once a transaction is committed, it's permanent.

Even a server crash won't undo the change.

ACID principles have been around for decades, yet they remain the foundation of modern systems from banking platforms to e-commerce applications.

As systems scale, understanding ACID becomes just as important as understanding APIs or microservices.

Which ACID property do you think is the hardest to maintain in distributed systems?

#Java #SpringBoot #Microservices #SystemDesign #SoftwareArchitecture #DistributedSystems #CloudNative #EventDrivenArchitecture #DevOps #ApacheKafka #Kubernetes #Docker #AWS #ScalableSystems #BackendDevelopment #DatabaseDesign #PostgreSQL #DataConsistency #SoftwareEngineering #TechLeadership #PlatformEngineering #FullStackDeveloper

#508 vaibhav sable

Java / Spring

AI Engineer | 12.7K+ LinkedIn Followers | RAG ,Agentic AI (LangChain • LangGraph • FastAPI) | MLflow • Docker • K8s | Open to Work

4mo · **View on LinkedIn**

Java Streams Internals — How filter(), map(), and collect() Actually Work

Most Java developers use Streams every day.

But very few understand what happens behind the scenes when this code runs:

```
list.stream()
.filter(x -> x > 10)
.map(x -> x * 2)
.sorted()
.collect(Collectors.toList());
```

Many assume each operation executes immediately.

That's not what actually happens.

Behind the scenes, Java creates a Stream Pipeline:

Collection
↓
Source
↓
filter()
↓
map()
↓
sorted()
↓
collect()
↓
Result

Key Concepts

✓ Source Stage

The stream starts from a collection, array, or data source.

✓ Intermediate Operations

filter(), map(), sorted(), distinct(), limit()

These operations are lazy and do not execute immediately.

✓ Pipeline Creation

Each operation is added to a processing pipeline.

✓ Terminal Operation

Execution begins only when a terminal operation is called.

Examples:
• collect()
• count()
• reduce()
• forEach()
• findFirst()
✓ Lazy Evaluation
Without a terminal operation, nothing runs.
This helps Java optimize performance and avoid unnecessary work.
Bonus: Parallel Streams
Using parallelStream(), Java can split work across multiple CPU cores using the ForkJoinPool framework.
But parallel processing isn't always faster.
Understanding when to use it can significantly improve application performance.
Interview Question
Why does this code produce no output?
list.stream()
.filter(x -> x > 10)
.map(x -> x * 2);
Answer:
Because there is no terminal operation. The pipeline is created, but execution never starts.
The next time you use Streams, remember:
filter(), map(), and sorted() don't process data immediately.
They build a pipeline.
The real work starts only when a terminal operation is invoked.
#Java #JavaStreams #JavaDeveloper #Programming #SoftwareEngineering #BackendDevelopment #SpringBoot #JVM #Coding #TechCommunity #LearnJava #JavaProgramming #SystemDesign

MEDIA advertisement

LINKS
• http://list.stream

#509 Ashok Bollepalli

Java / Spring

Gen AI & Agentic AI Developer | Full Stack Developer | Cloud & DevOps | FinTech | Agile | Wells Fargo
4mo · View on LinkedIn

Java Stream API makes data processing simple, clean, and powerful.
Here are some commonly used Stream API methods every Java developer should know for filtering, transforming, sorting, collecting, and processing data efficiently.
Master these methods to write better and more readable Java code. ✓
#Java #StreamAPI #Java8 #JavaDeveloper #BackendDevelopment #SpringBoot #Programming #Coding #SoftwareDevelopment #LearnJava #AshokIT

#510 Devarasetty Badarinath

AI / LLM

Senior Technology Architect | Enterprise Architecture | Cloud & DevOps | Engineering Leadership | Scalable Systems
4mo · View on LinkedIn

AI Explained Like a Human Body — The Simplest Way to Understand Modern AI Systems

Many people hear terms like LLM, RAG, AI Agents, and MCP and feel overwhelmed.

Here's a simple analogy:

LLM = The Brain

The thinking engine. It generates answers based on what it learned during training. Powerful, but it doesn't automatically know the latest information.

RAG = Brain + Books

RAG (Retrieval-Augmented Generation) gives AI access to documents, databases, and company knowledge before it answers. This makes responses more accurate, contextual, and reliable.

AI Agents = Brain + Hands

Agents don't just answer questions—they take action. They can use tools, remember context, make decisions, and execute multi-step workflows.

MCP = The Nervous System

Model Context Protocol (MCP) connects AI to external tools, APIs, databases, and applications through a standardized interface. It enables enterprise-scale AI ecosystems.

The Evolution of AI

→ LLM thinks

→ RAG knows

→ Agents act

→ MCP connects

The future of AI isn't just smarter models—it's intelligent systems that can access knowledge, take action, and seamlessly interact with the digital world.

Organizations that understand this stack today will build the most powerful AI solutions tomorrow.

#ArtificialIntelligence #AI #GenAI #LLM #RAG #AIAgents #MCP #MachineLearning #EnterpriseAI #DigitalTransformation #Technology #Innovation #CloudComputing #DevOps #FutureOfWork #AIEngineering

MEDIA document/carousel (possible)

#511 Janjirala Anusha

AI / LLM

Technical Program Lead | AI, DevOps & Cloud | 15.5k+ Followers

4mo · **View on LinkedIn**

AI Engineer vs Generative AI Engineer — Do you know the difference?

These two roles are closely related, but they are not the same.

An AI Engineer usually focuses on:

• Building and training models

• Working with structured data

• Improving model performance

• Using tools like Python, TensorFlow, PyTorch, and scikit-learn

A Generative AI Engineer focuses more on:

• Building applications powered by LLMs

• Creating AI experiences for users

• Working with RAG, vector databases, agents, and APIs

• Using tools like LangChain, OpenAI APIs, Vertex AI, and Bedrock

• Simple way to understand it:

• AI Engineer builds the intelligence

• Generative AI Engineer turns that intelligence into useful real-world applications

Both roles are valuable, and both are growing fast in today’s market.

If you're planning your AI career, understanding this difference can help you choose the right learning path.

Which role interests you more — AI Engineer or Generative AI Engineer?

#AI #GenerativeAI #AIEngineer #GenAI #MachineLearning #LLM #CareerInAI #ArtificialIntelligence #TechCareers #AnushaJanjirala

MEDIA advertisement

#512 Vishal Sharma

AI / LLM

Building in AI, Product Intelligence & Developer Tools | PureLogicLabs Learning and AI applications | View my services

4mo · View on LinkedIn

The Future of Java Enterprise Architecture: Agentic, Intelligent & Event-Driven

Over the last decade, enterprise architecture has transformed from monolithic applications to microservices, cloud-native platforms, and event-driven systems. The next evolution is already underway: Agentic Enterprise Architecture.

In the future, enterprise systems will not only execute business logic but will also understand intent, reason, plan, and orchestrate actions across multiple services and workflows.

Consider a Relationship Manager asking: "Why is this counterparty at risk of downgrade?" Instead of navigating multiple systems, an AI Agent will:

- Understand the request
- Retrieve risk and collateral data
- Invoke Java microservices through APIs
- Query knowledge bases and policies using RAG
- Trigger workflows through Camunda/Temporal
- Generate an explainable, auditable response

Key Architectural Shifts include:

- Microservices → Intelligent Service Ecosystems
- API-First → Event-First Architecture (Kafka)
- Automation → AI Agents
- Databases → Knowledge Graphs + Vector Stores
- DevOps → Platform Engineering
- Static Workflows → Adaptive Orchestration

A Future Java Enterprise Stack will consist of:

- Users & Business Teams
- AI Agent Layer (Reasoning • Planning • Memory • Tools)
- API Gateway & Tool Layer
- Java Enterprise Services (Spring Boot)
- Kafka Event Backbone
- Camunda / Temporal Workflow Orchestration
- AI Decision Services (RAG, ML Models, Decision Engines)
- PostgreSQL + Neo4j + Vector Databases
- AWS Cloud-Native Platform

All governed through security, compliance, observability, and human-in-the-loop controls.

The future isn't about AI replacing enterprise systems; it's about AI Agents orchestrating them. With Spring AI, Kafka, Virtual Threads, cloud-native runtimes, and workflow orchestration platforms, Java is well positioned to remain at the center of this transformation.

What architectural shift do you think will have the biggest impact on enterprise systems?

#513 Nagoor Meeran S.

Kafka / Messaging

Senior Technical Lead | Java · Spring Boot · Microservices · Kafka | 20+ Years Building Enterprise Systems | Exploring AI-Assisted Engineering (Claude Code, Codex)

4mo · View on LinkedIn

Java + Microservices + Kafka (Part 5 — Scaling Event-Driven Systems to Millions of Events)

Building an event-driven system is relatively easy.

Scaling it to millions of events per day while maintaining reliability, throughput, and operational simplicity is where architecture truly matters.

In this carousel, I cover:

✓ Capacity planning

✓ Kafka topic design

✓ Partition strategy

✓ Idempotent consumers

✓ Real-time vs batch workloads

✓ Consumer lag management

✓ Event payload optimization

✓ Failure recovery strategies

✓ Observability at scale

✓ Automation

The biggest lesson I've learned:

→ Scaling is less about Kafka and more about architecture discipline.

Save this if you're building large-scale event-driven systems.

#Java #Kafka #Microservices #SystemDesign #EventDrivenArchitecture #SpringBoot #SoftwareArchitecture #DistributedSystems #BackendEngineering #TechLeadership

Part 5-Scaling Event-Driven Sy

.

4 pages

MEDIA Part 5 — Scaling Event-Driven Systems to Millions of Events Scaling event-driven systems requires architecture discipline, not just more infrastructure. 1. Start with Capacity Planning • Estimate events/sec and peak load • Understand event size and retention • Measure consumer processing time 2. Design Kafka Topics for Scale • Separate high-volume and low-volume workloads • Use business-driven topic boundaries • Avoid giant catch-all topics; 3. Partition Strategy Determines Throughput • Partitions are Kafka's scaling unit • Balance throughput and overhead • Avoid hot partitions 4. Build Idempotent Consumers • Handle duplicates safely • Use idempotency keys and deduplication • Reliability starts with idempotency 5. Separate Real-Time and Batch Workloads • Real-time: workflows, notifications • Batch: analytics, reporting, ML • Different workloads need different scaling; 6. Control Consumer Lag • Monitor lag, retries, DLQ growth • Treat lag as a business metric 7. Optimize Event Payloads • Keep events small and focused • Use schema evolution and compression 8. Design for Failure & Recovery • Safe replay strategy • Recover from outages gracefully • Recovery is part of scalability; document/carousel (possible)

#514 Sunil Reddy

System Design

Lead Product Engineer @ Harman | Java · Spring Boot · Microservices · AI/LLM Integration | 7+ Years

4mo · **View on LinkedIn**

One of the most common questions in system design:

"Do we need Nginx, a Load Balancer, or an API Gateway?"

The answer?

Sometimes all three.

Because they solve different problems.

Load Balancer

Its job is simple:

Request → Server 1

Request → Server 2

Request → Server 3

Goal:
✓ Distribute traffic
✓ Improve availability
✓ Prevent server overload

Nginx
Acts as a reverse proxy.
✓ SSL termination
✓ Static content serving
✓ Request routing
✓ Basic load balancing
Think of it as a traffic manager.

API Gateway
Designed for microservices.
✓ Authentication
✓ Rate limiting
✓ Request transformation
✓ Service aggregation
✓ API policies
Think of it as a traffic manager with business rules.

The mistake?
Using one tool to do everything.
I've seen:
✗ Nginx turned into an API Gateway
✗ API Gateway turned into a backend service
✗ Load Balancer expected to handle authentication
Result?
Complexity.
Simple way to remember:
Load Balancer = "Where should traffic go?"
Nginx = "How should traffic enter?"
API Gateway = "What rules should traffic follow?"
Golden rule:
The best architecture is not the one with the most components.
It's the one where every component has a clear responsibility.
Be honest ↓
What are you using in production today:
Load Balancer, Nginx, API Gateway... or all three?
#nginx #apigateway #loadbalancer #microservices #systemdesign #backend #java #softwarearchitecture

#515 Parikh Jain AI / LLM
Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

4mo · View on LinkedIn

2 patterns will save you from 200 Sliding Window rejections.

3 patterns will save you from 300 Two Pointer rejections.

5 patterns will save you from 500 Array rejections.

8 patterns will save you from 700 DP rejections.

6 patterns will save you from 400 Tree rejections.

7 patterns will save you from 350 Graph rejections.

Every rejection email you have collected this year was a recognition gap. The pattern was sitting in front of you in the 45-minute round, wearing a new name, and your brain had no map to point at it.

Here is what changes when you flip the math:

a) You stop grinding random problem sheets and start compressing them into 50 patterns

b) Every new problem feels familiar because your brain matches it to one you already solved

c) Your speed in the 45-minute round doubles because you are recognizing, not rebuilding

d) Your DSA round panic disappears because the problem stops being "new"

e) You finally have headspace for the rounds after DSA, system design, core CSE, GenAI

That last part is what most people miss in 2026. Clearing DSA is the gate. The offer happens in rounds that come after.

1500+ students are using the DSA + Dev + GenAI Placement System https://lnkd.in/gut39Xsr to fix exactly this:

pattern-based DSA, system design from LLD up, core CSE, GenAI fundamentals, and application strategy in one place.

You do not need 3,500 problems.

You need 50 patterns and the full picture around them.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gut39Xsr

#516 Umashanker Pandey

Kafka / Messaging

Technical Lead @ Morningstar | Technical Leadership, Full-Stack Development | View my services

4mo · View on LinkedIn

I lost data in production because of 2 lines of code.

Here's what went wrong — and how to never let it happen again.

We were saving an order to the database and publishing an event to Kafka right after.

DB write ✓

Kafka publish ✗

Order existed. Inventory never updated. Customer paid. Stock didn't move.

This is called the dual-write problem.

Two independent systems. One transaction. Zero atomicity guarantee between them. And no — @TransactionalEventListener doesn't save you. If Kafka fails after DB commits, the event is gone forever.

The fix: Outbox Pattern.

Instead of writing to Kafka directly, you write an event record into your own database — inside the same transaction as your business data.

Orders table → updated

Outbox table → event saved

Both in one atomic commit. Either both succeed, or neither does.

A separate Relay process then picks up outbox entries and forwards them to Kafka. Two approaches:

Polling — a scheduled job reads unpublished rows every N seconds. Simple, zero extra infra, safe with FOR UPDATE SKIP LOCKED.

CDC (Debezium) — reads the DB Write-Ahead Log directly. Near real-time, no polling overhead, your service never touches Kafka at all.

The diagram above shows the full flow visually — from REST request all the way to consumers.

The crossed-out arrow says it all: avoid direct write.

Save this for the next time someone asks why the notification didn't send.

Have you hit this in production? Drop your story below ↓

#softwaredevelopment #java #springboot #kafka #microservices #distributedsystems #systemdesign #backend #outboxpattern #eventdriven #debezium #softwarearchitecture

#517 **Varun Sharma** **Cloud / K8s / DevOps**

Technology Leader | .NET & Azure Architect | Python | GenAI | GitHub Copilot | Building Intelligent, Scalable, Cloud-First Systems

4mo · **View on LinkedIn**

Understanding Circuit Breaker:

One slow microservice can crash your entire application.

That’s exactly why the Circuit Breaker Pattern exists.

Imagine this:

Your Order Service depends on a Payment API.

Suddenly the Payment service becomes slow during peak traffic.

Now every request:

✗ waits endlessly

✗ consumes threads

✗ blocks resources

✗ spreads failures across services

This is called a cascading failure.

The Circuit Breaker pattern prevents this by:

- Detecting repeated failures

- Stopping unnecessary requests

- Failing fast instead of timing out

- Allowing graceful recovery through HALF-OPEN state

In this carousel, I covered:

✓ Real-world failure scenario

✓ Circuit Breaker states (Closed/Open/Half-Open)

✓ Industry best practices

✓ Production-grade Polly implementation in .NET

✓ ASP.NET Core integration

✓ Common interview questions

✓ Key architectural takeaways

If you're working with:

Microservices

Cloud-native systems

Distributed APIs

.NET backend systems

...this pattern is essential.

Comment “PDF” and I’ll share the detailed guide.

Follow for more:

Cloud Design Patterns

.NET Architecture

Azure & Microservices

AI + Cloud Engineering

For detailed video explanations and hands-on backend engineering content, check out my YouTube channel.

Follow @FundamentallyClear for content drops.

https://lnkd.in/gJG26Sw6

→ Want exclusive access? Drop your name and email → https://lnkd.in/gtyPHZ65

1/9

MEDIA document/carousel (possible)

LINKS

- http://ASP.NET
- https://lnkd.in/gJG26Sw6
- https://lnkd.in/gtyPHZ65

#518 Md Ashikur Rahman

Other

Manager(Card systems & ADC), Brac Bank Plc | FinTech | ISO 8583 | Emv | 3DS | Tokenization | POS | QR Payment | CMS |

4mo · **View on LinkedIn**

E-Commerce Transaction Flow Through ACS (3D Secure)

This diagram illustrates the complete 3D Secure (3DS) e-commerce card payment authentication and settlement lifecycle. It demonstrates how online card transactions are securely authenticated, authorized, and settled within international card schemes such as Visa, Mastercard, and Discover.

1. Transaction Initiation

The process begins when the cardholder selects products/services on an e-commerce website and proceeds to checkout. The merchant captures the card credentials and initiates the payment request through the acquiring bank.

2. Authentication Request Routing

The Acquirer forwards the authentication request to the relevant payment network (Visa, Mastercard, Discover, etc.), which then routes the request to the issuing bank (Issuer) responsible for the card.

3. 3D Secure Enrollment Verification

The Issuer verifies whether the cardholder is enrolled in the 3D Secure authentication program. If enrolled, the transaction is redirected to the ACS (Access Control Server) for Strong Customer Authentication (SCA).

4. Cardholder Authentication via ACS

The ACS performs multi-factor authentication using mechanisms such as:

• One-Time Password (OTP) via SMS.EMAIL.

This step ensures that the legitimate cardholder is initiating the transaction.

5. CAVV / UCAF Validation

After successful authentication:

• For Visa, a CAVV (Cardholder Authentication Verification Value) is generated.

• For Mastercard, a UCAF (Universal Cardholder Authentication Field) is generated.

The Issuer validates these cryptographic authentication values to confirm the integrity and legitimacy of the authentication response before proceeding with authorization.

6. Authentication Response Flow

The ACS sends the authentication result (Success/Failure) to the Issuer. The Issuer then forwards the response through the Payment Network to the Acquirer, which finally communicates the outcome to the Merchant.

If authentication succeeds:

• The merchant proceeds with transaction authorization and order completion.

If authentication fails:

• The transaction may be declined or challenged based on issuer risk policies.

7. Authorization and Risk Mitigation

The 3DS authentication process significantly reduces:

• Card-not-present (CNP) fraud

• Unauthorized online transactions

• Chargeback exposure for merchants

It also enables liability shift benefits under card scheme rules where applicable.

8. Settlement Process (Post-Authorization)

Once authorization is successful, the settlement cycle begins:

1. Merchant batches transactions and submits them to the Acquirer.

2. Acquirer sends settlement files to the Payment Network.

3. Payment Network clears and routes transactions to the Issuer.

4. Issuer settles funds to the Acquirer based on scheme settlement timelines (T+1 / T+2).

5. Acquirer credits the Merchant account after deducting applicable MDR and processing fees.

MD ASHIKUR RAHMAN

MANAGER,CARD SYSTEM

BRAC BANK PLC

LINKS

- http://SMS.EMAIL

#519 Patrick Al Kreidy Other

Senior Cards & Payments Consultant | Lead Product Support & Solutions Engineer | Fintech | Core Banking Systems (CORTEX, IST Switch)

4mo · **View on LinkedIn**

Understanding the End-to-End Card Payment Lifecycle

Every card transaction involves far more than the approval message displayed to the customer. Behind each successful payment is a complex ecosystem of financial institutions, payment networks, and security controls working together to process transactions seamlessly and securely.

Key participants in the payment flow include:

Cardholder

Merchant

Acquiring Bank / Payment Service Provider (PSP)

Card Network (mada, Visa, Mastercard)

Issuing Bank

A typical card transaction progresses through three core stages:

✓ Authorization – Real-time validation of the transaction, including card status, available funds, risk checks, and authentication controls.

✓ Clearing – Exchange, validation, and reconciliation of transaction data between the issuer, acquirer, and payment network.

✓ Settlement – The actual transfer of funds from the issuing bank to the acquiring bank, followed by payment to the merchant.

Beyond the primary transaction flow, several supporting processes are critical to maintaining a resilient payments ecosystem:

Reversals – Cancellation of authorized transactions before settlement.

Refunds – Return of funds to the customer after settlement has been completed.

Chargebacks & Disputes – Formal mechanisms for handling fraud claims, transaction disputes, and merchant-customer disagreements.

Contactless Payments – Secure and efficient transactions enabled through tokenization and NFC technology.

3D Secure (3DS) – An additional authentication framework designed to reduce fraud and strengthen security for e-commerce transactions.

While customers experience payment approval in a matter of seconds, the transaction lifecycle continues through clearing and settlement before funds are ultimately received by the merchant.

A strong understanding of these payment processes is essential for professionals involved in Payments, Banking, FinTech, Digital Channels, and Enterprise Architecture, as they form the foundation of modern payment ecosystems.

#Payments #PaymentProcessing #Cards #FinTech #Banking #DigitalPayments #Visa #Mastercard #Mada #PaymentGateway #EnterpriseArchitecture #BankingTechnology #FinancialServices #SaudiBanking

#520 Prakash Sethia Other

Senior Consultant| Payments Banking | Swift Payments | Java | Spring Boot | Microservices | Kafka | Event-Driven Architecture | Distributed Systems | Cloud | Performance Optimisation | ISO 20022 | RTP | NRT

4mo · **View on LinkedIn**

A ₹500 Coffee Purchase: The Hidden Journey Through the Global Payments Ecosystem

Every time a customer taps, inserts, swipes, or enters card details online, a complex ecosystem works behind the scenes to ensure the payment is processed securely, accurately, and in real time.

This infographic breaks down the complete Card Payment Transaction Lifecycle into four critical stages:

✓ Leg 1 – Transaction Initiation

Cardholder → Merchant → Acquirer

✓ Leg 2 – Authorization Request

Acquirer → Card Network → Issuer

✓ Leg 3 – Authorization Response

Issuer → Card Network → Acquirer → Merchant

✓ Leg 4 – Clearing & Settlement

Merchant → Acquirer → Network → Issuer → Acquirer → Merchant

Example: A simple ₹500 coffee purchase triggers multiple systems, validations, routing decisions, and settlement processes before funds finally reach the merchant.

What part of the payment lifecycle do you find most interesting—the real-time authorization process or the clearing and settlement stage?

#Payments #FinTech #Banking #CardPayments #Mastercard #Visa #PaymentProcessing #ISO8583 #DigitalPayments #FinancialServices #Acquiring #Issuing #Settlement #Clearing #Technology #SoftwareEngineering #FintechInnovation #Leg1 #Leg2 #Leg3 #Leg4 #TransactionLifeCycle #Authorization #ClearingAndSettlement #CardProcessing #PaymentsArchitecture

MEDIA advertisement

#521 Srinivas Mahakud

System Design

Generative AI Architect & Engineering Leader | LLM · RAG · Multi-Agent Systems | Scaling AI from PoC to Production | EY

4mo · View on LinkedIn

Your architect keeps saying things like Idempotency, Immutability, or Zero Trust... and everyone in the room nods politely.

But what do these actually mean for business leaders?

Here are 11 architecture principles translated into plain English ↓

They’re the design principles that keep modern digital platforms secure, scalable, and reliable.

#SoftwareArchitecture #EnterpriseArchitecture #SystemDesign #TechLeadership #CloudArchitecture #PlatformEngineering #DigitalTransformation

#522 Alok Sharan

System Design

Technology Leader and Architect || AI & Data Transformation at Scale || Fintech || Published Author

4mo · View on LinkedIn

Your architecture isn't tested by traffic.

It's tested by unpredictable traffic.

And that’s where most systems quietly break - not because they couldn’t scale,

but because they scaled the wrong layer at the wrong time.

From real-world systems, one thing becomes clear:

scaling is not one strategy - it’s a combination of patterns working together.

This breakdown covers the ones that actually matter ↓

→ Queue-Based Scaling

Decouple workloads and process asynchronously.

Critical when handling spikes without overwhelming systems.

→ Microservices Scaling

Scale only what needs scaling - not the entire application.

Efficiency comes from isolating bottlenecks.

→ Vertical vs Horizontal Scaling

More power vs more machines.

Good engineers know when to use each - great engineers combine both.

→ Auto Scaling

Let systems respond to real-time demand automatically.

No manual intervention, no guesswork.

→ Load Balancing

Distribute traffic intelligently to avoid single points of failure.

→ Caching Strategy

Reduce repeated computation and database load.

Often the simplest way to get massive performance gains.

→ CDN (Content Delivery Network)

Move content closer to users - latency drops instantly.

→ Database Scaling

Replication, sharding, partitioning.

Because most scaling problems eventually become data problems.

→ Serverless Scaling

Event-driven, auto-managed scaling without worrying about infrastructure.

Here’s what consistently goes wrong:

→ Teams rely only on auto scaling and ignore architecture

→ Compute scales, but databases become the bottleneck

→ Asynchronous patterns are introduced too late

That’s why systems fail exactly when demand increases.

Real scalability isn’t about handling more traffic.

It’s about handling uncertainty without breaking.

If your system had 10x traffic tomorrow,

which layer would struggle first? ↓

#523 Spring Users

AI / LLM

John Ferguson Smart • 3rd+

4mo · View on LinkedIn

So I've been experimenting with AI-assisted development on a Spring Boot API.

One lesson keeps coming back: AI is way better at implementing requirements than discovering them by itself.

In other words, if you tell AI: "Build me a cashback rewards API", you'll often get plausible-looking code. It might even look quite good. But there will be subtle bugs, missing rules, and assumptions you didn't ask for.

But if you give it a precise specification - with rules, examples, counter-examples, and acceptance tests - the results are much, much better.

BUT, yeah, there's always a but.

This isn't your grandpa's spec-driven development.

You can't just dump a detailed spec with 20 features into an AI coding tool and expect magic.

When you do that, you tend to get bloated code, confused responsibilities, missed edge cases, and tests that basically just confirm whatever the implementation already does.

So what actually works?

The workflow I'm getting the best results with looks something like this:

Discover — use Example Mapping with AI to surface rules, edge cases, assumptions, and open questions before writing a line of code.

✓ Acceptance — turn those examples into contractual acceptance tests that drive and frame the code that AI will write.

TDD — let AI build the code through strict red-green-refactor cycles against those tests.

Review — AI is actually quite good at putting on another hat and checking its own work. Beyond the passing tests, what else might it have missed?

Want to see an example?

I've open-sourced a sample project where I've been applying this practice with Claude Code.

It's a Spring Boot cashback rewards API with hexagonal architecture, Claude Code commands for each step of the workflow, and branches for each section:

https://lnkd.in/exXC5BJE

Still work in progress, but enough to give you the idea.

I'd be interested to hear how other Spring developers are using AI coding tools without letting the design get away from them.

MEDIA advertisement

LINKS
- https://lnkd.in/exXC5BJE

#524 Shalini Goyal

AI / LLM

Executive Director, AI & Engineering @ JPMorgan | Amazon Alum | Author · Speaker · Professor | Helping Engineers Break into AI & High-Impact Careers

4mo · **View on LinkedIn**

GenAI vs AI Agents vs Agentic AI vs ML vs Data Science vs LLM

AI has many layers, from data science foundations to intelligent, autonomous systems.

Each concept plays a unique role in shaping today’s intelligent technology stack.

Let’s break down how these six AI domains connect yet differ at their core :

1. Generative AI – Core Concepts

-Focuses on creating new content - text, images, music, or video.

-It uses diffusion models, GANs, and transformers to generate outputs from patterns it learns.

-Think ChatGPT, Midjourney, or Runway - all powered by creative generation.

2. AI Agents – Core Concepts

-AI Agents act autonomously, performing tasks and making decisions.

-They use context, reasoning, and environment interaction to execute workflows.

-These agents can use APIs, tools, and feedback loops to reach goals intelligently.

3. Agentic AI – Core Concepts

-Takes AI agents to the next level — self-improving, reasoning, and planning systems.

-It introduces chain-of-thought reasoning, self-reflection, and multi-agent collaboration.

-Agentic AI focuses on autonomy, feedback, and human-in-the-loop alignment.

3. Machine Learning – Core Concepts

-ML trains models to learn patterns from data and make predictions.

-It involves supervised, unsupervised, and reinforcement learning, powered by algorithms like regression and clustering.

-The focus: accuracy, feature engineering, and model optimization.

4. Data Science – Core Concepts

-The backbone of AI - focused on data collection, analysis, and visualization.

-It combines statistics, hypothesis testing, and data ethics to extract insights.

-Data science powers every stage — from data cleaning to predictive analytics.

5. Large Language Models (LLMs) – Core Concepts

-LLMs are language-based neural networks trained on massive text datasets.

-They use transformers, embeddings, and attention mechanisms to understand and generate language.

-LLMs like GPT and Gemini form the core engine of today’s AI assistants.

In Summary:

- Data Science → builds the data foundation.
- Machine Learning → finds patterns.
- LLMs & GenAI → create outputs.
- AI Agents & Agentic AI → take intelligent action.

Together, they form the complete AI ecosystem driving automation and intelligence today.

#525 Hina Arora

AI / LLM

Helping Tech Leaders Build Their Personal Brand and Industry Authority | Founder @BrandBuilders | Tech / AI / Career Content Creator (500K+) | 10+ YEO in Tech & AI | Ex-Engineering Manager @Jio

4mo · View on LinkedIn

Everyone is talking about AI Agents.

Very few are talking about Agentic AI.

And they are not the same thing.

An AI Agent is like an employee who completes a task when you assign it.

"Summarize this document."

"Generate this email."

"Analyze this CSV."

Task received.

Task completed.

Agentic AI behaves more like a team.

It understands the goal, creates a plan, delegates work, checks progress, learns from feedback, and adapts when things change.

Think of it this way:

AI Agent = Execution

Agentic AI = Execution + Planning + Memory + Reflection + Coordination

The progression usually looks like this:

Level 0 → Runs commands

Level 1 → Assists users

Level 2 → Completes small tasks independently

Level 3 → Multiple agents collaborate

Level 4 → Self-corrects using memory

Level 5 → Operates with minimal supervision

When should you use each?

Use an AI Agent when:

• The task is simple and well-defined

• Speed matters more than autonomy

• You want predictable outputs

Use Agentic AI when:

• The workflow involves multiple steps

• Decisions depend on context

• The system needs memory and feedback loops

• Human intervention should be minimized

A lot of companies are building agents today.

The next wave will be building systems of agents that can reason, collaborate, and continuously improve.

That's where Agentic AI starts becoming interesting.

Have you built a simple AI Agent or experimented with Agentic AI systems yet? ↓

————————————————————————

I (Hina) help tech executives to position them online that helps them to grow professionally.

DM me ‘Branding’ if you want to position yourself on LinkedIn

MEDIA document/carousel (possible)

#526 Java Developers Community (moderated)

AI / LLM

Martin Manriquez Leon • 3rd+ | Stop trusting AI blindly. How to document your Claude Code sessions. | AI coding tools like Claude Code are changing the game, but they don't replace the need for engineering rigor. As we integrate these assistants into our daily workflows, the way we audit and document their output becomes a critical skill. | Here are 4 best practices to keep your codebase healthy: | Treat AI code as "Unverified PRs": Never push code generated by AI without a local test run. Review it with the same skepticism you would apply to a junior developer’s contribution. | Keep a "Decision Log": If an AI suggests a non-obvious refactor or a specific library implementation, document the why in the PR description or code comments. This saves your team hours of "Why did we do this?" in the future. | Version your prompts: If a specific prompt solves a complex bug, save that context in a dedicated docs/ai-prompts/ folder. It helps your team reproduce the logic and improves consistency. | Audit the dependencies: AI can hallucinate packages or suggest deprecated versions. Always verify the package.json or requirements.txt changes before confirming the install. | AI is your pair programmer, not your replacement. The final responsibility for the architecture and security of the code remains yours. | How are you keeping your AI-assisted workflows clean? Let’s share tips in the comments! ↓ | #SoftwareEngineering #AI #ClaudeCode #DevLife #BestPractices #CodingTips | 101 | 1 | 3 | | Shani Tiwari Verified Profile 3rd+ | Shani Tiwari | I help businesses turn their ideas into fast, intuitive web experiences. Build Framer Websites and Web Apps. Full-Stack Dev | Creative Strategist

4mo · **View on LinkedIn**

Dumb move.... Say this to Claude, explain this code to me like I'm 5 yo

MEDIA advertisement; document/carousel (possible)

#527 Konica Dhingra

Java / Spring

Senior Backend Engineer | Java | Spring Boot | Microservices | Distributed Systems | Cloud-Native Architectures

4mo · **View on LinkedIn**

Java Garbage Collection Explained

Garbage Collection (GC) in Java is an automatic memory management process that removes unused objects from Heap memory.

How It Works Internally : Java Heap is mainly divided into

- Young Generation
- Old Generation
- Metaspace

GC Flow:

- New objects are created in Eden Space
- Minor GC removes short-lived unused objects
- Surviving objects move to Survivor spaces
- Long-living objects move to Old Generation
- Major/Full GC cleans old unused objects

Why Garbage Collection Matters

✓ Prevents memory leaks
✓ Eliminates manual memory cleanup
✓ Improves application stability
✓ Optimizes memory usage automatically

Modern Java Garbage Collectors

- G1 GC (Default)
- ZGC
- Shenandoah
- Parallel GC

Understanding Garbage Collection helps developers build high-performance and memory-efficient Java applications.

MEDIA advertisement

#528 Anubhav Sethi

System Design

SWE 3 - Google • Helps you learn System Design & GenAI • 2× Gold Medalist• Tech events, Hackathon & Podcast

4mo · **View on LinkedIn**

You know your system design answer felt incomplete.
Here’s where it often breaks down:
It’s your database.
Most “scale” problems are database problems in disguise.
A common scaling progression looks like this:
→ Single DB starts struggling
First ask: is it a read bottleneck or a write bottleneck?
→ Read-heavy workload → Add read replicas
Reads scale horizontally, but writes still hit the primary.
→ DB still overloaded or latency spikes → Introduce caching (Redis)
Cache hot reads to reduce DB load and improve latency.
→ Cache returns stale data → Add invalidation strategy
TTL, cache-aside, write-through, event-based invalidation — depends on consistency requirements.
→ Write pressure increases → Offload work using queues/events
Move async workflows (emails, analytics, notifications, fanout) out of request path.
→ Queries slow down at scale → Fix queries + indexing first
Bad queries on a bigger cluster are still bad queries.
→ One database is no longer enough → Partitioning / sharding
Choose shard keys carefully or you’ll create hot partitions.
→ Shards become uneven → Better distribution strategy
Consistent hashing helps reduce rebalancing pain during scaling.
→ One region becomes a single point of failure → Multi-region replication + failover
Now you trade simplicity for latency, consistency, and conflict-resolution complexity.
Most candidates jump to sharding in 2 minutes.
Strong system design answers show why each bottleneck appears and what breaks next.
Learn this sequence before your next system design round.
If this was useful, follow Anubhav Sethi for more of this.
Repost if you found this helpful, it might help someone else too.
1/3

MEDIA document/carousel (possible)

LINKS

- https://yt.openinapp.co/0gj3u

#529 Revanth Boggarapu

System Design

Building web solutions as a Software Developer with No-Code expertise.

4mo · **View on LinkedIn**

Everyone wants to build scalable systems.

Very few take the time to understand what happens behind a single user request.
Over the last few weeks, I explored the foundations of System Design, and one thing became clear:
Great systems are not built by frameworks.
They are built by engineering decisions.
A simple request follows a surprisingly complex journey:
Client → DNS → Server
But what happens when millions of users arrive?
That's where System Design begins.
Scaling
• Vertical Scaling → Bigger machine, single point of failure
• Horizontal Scaling → Multiple servers, higher availability
Load Balancing
Distributes traffic intelligently using:
• Round Robin
• Least Connections
• IP Hashing
And yes, it continuously performs health checks to avoid routing traffic to failed servers.
Reducing Latency
Users don't want fast systems.
Users want instant systems.
That's why CDNs exist.
Edge servers cache:
• Images
• Videos
• Audio
• Fonts
• Static HTML
Techniques like:
• Cache Miss Handling
• TTL
• Versioned Assets
help keep content fresh and fast.
Caching with Redis
When databases become bottlenecks:
Database → Redis → User
Key concepts:
• Write Through
• Write Back
• LRU
• LFU
• TTL
• Cache Stampede Prevention
Database Scaling
Handling millions of reads?
→ Read Replicas
Handling massive writes?

→ Sharding
Strategies:
• Range-Based Sharding
• Hash-Based Sharding
Event-Driven Architecture
Not everything should happen synchronously.
Uploading an image may involve:
• Compression
• Resizing
• Thumbnail Generation
• Metadata Processing
Instead of making users wait:
Queue → Process → Respond
Technologies:
• Apache Kafka
• RabbitMQ
Real-Time Communication
Different use cases require different approaches:
• Short Polling
• Server-Sent Events (SSE)
• WebSockets
Choosing the wrong one can dramatically impact scalability.
Object Storage
Large files don't belong in databases.
Use:
• Object Storage
• Blob Storage
• S3-compatible systems
Reliability Matters
One of my favorite concepts:
Idempotency Keys
Without them:
Duplicate payments happen.
With them:
✓ Safe retries
✓ Better fault tolerance
✓ Consistent transactions
Used heavily by payment platforms worldwide.
CAP Theorem
Every distributed system eventually faces a tradeoff:
C → Consistency
A → Availability
P → Partition Tolerance
The challenge isn't choosing all three.
The challenge is choosing the right compromise.
The deeper I go into AI Engineering and System Design, the more I realize:

Building features is easy.

Building systems that survive scale, failures, latency, retries, spikes, and real-world traffic is where engineering truly begins.

#SystemDesign #SoftwareEngineering #DistributedSystems #BackendDevelopment #Scalability #Architecture #CloudComputing #Redis #Kafka #RabbitMQ #LoadBalancing #Microservices #WebSockets #DatabaseDesign #Sharding #Caching #CDN #DevOps #AIEngineering #TechCareer #LearningInPublic #EngineeringExcellence

MEDIA advertisement

#530 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Jaden Sim • 3rd+

4mo · **View on LinkedIn**

AI - aitagem.com/login.php - Join 10 million+ people excited about learning AI.

The Big Picture: AI is the Universe aitagem.com/login.php

Artificial Intelligence (AI) is the broadest category. It contains everything else:

Machine Learning

Generative AI

Deep Learning

Neural Networks

Computer Vision

NLP

Think of AI as the entire field of making machines intelligent.

Machine Learning: The Engine aitagem.com/login.php - Join 10 million+ people excited about learning AI.

ML is a subset of AI. Instead of hard-coding rules, machines learn from data.

Three main flavors:

Unsupervised Learning – Detection, clustering (finding hidden patterns)

Supervised Learning – Ranking, regression (learning from labeled examples)

Reinforcement Learning – Policy, model-free (learning by trial and error)

Deep Learning: The Brain Simulator aitagem.com/login.php - Join 10 million+ people excited about learning AI.

Deep Learning is a subset of ML using multi-layer neural networks.

The visual simplifies it:

Prediction network → "This is a car"

Deep learning excels at recognizing patterns in images, sound, and text—like identifying a car in a photo.

Generative AI: The Creator aitagem.com/login.php - Join 10 million+ people

Generative AI sits under the deep learning umbrella (and often uses large language models).

The diagram shows:

User + Large Language Model + Data sources → Output

GenAI creates new content—text, images, code, music—rather than just classifying or predicting.

RAG: The Knowledge Augmenter aitagem.com/login.php - Join 10 million+ people

RAG (Retrieval-Augmented Generation) isn't a separate discipline—it's a pattern that combines retrieval with generation.

The visual flow:

User query + Embedding → Vector DB

Retrieved Data + System Prompt + User Prompt → Output
RAG grounds GenAI in external knowledge, reducing hallucinations and enabling up-to-date answers.
AI Agents: The Autonomous Actor aitagem.com/login.php Join 10 million+ people
AI Agents go beyond generation. They use:
Database – persistent knowledge
Memory – short-term and long-term context
Reasoning – planning and decision-making
Tools – APIs, code, web search
Planning & Feedback – iterative improvement
The diagram also includes: System Prompt → User → Planning → Feedback loop.
Agents don't just answer—they act in the world.
Why This Matters
You don't need to master every layer. But understanding how they nest helps you:
Pick the right tool for the job
Communicate clearly with technical teams
Spot buzzwords vs. real capabilities
Next time someone says "we're using AI", you can ask: Which part? ML? GenAI? Agents? AI universe - aitagem.com/login.php - Join 10 million+ people
Which discipline are you digging into right now? Let me know in the comments! ↓ AI universe - aitagem.com/login.php - Join 10 million+ people excited about learning AI.
#AI #MachineLearning #DeepLearning #GenerativeAI #RAG #AIAgents #TechExplained #ArtificialIntelligence

LINKS
- http://aitagem.com/login.php

#531 Java Developers Community (moderated)

System Design

Mohamed EL Laithy • 3rd+
4mo · **View on LinkedIn**

Monolithic, Microservices, Serverless, or Event-Driven?
There's no "best" architecture — only the right choice for your scale, team, and business needs.
✓ Monolith → Fast to build and ideal for startups
✓ Microservices → Independent scaling and team autonomy
✓ Serverless → Pay only for what you use
✓ Event-Driven → Highly scalable, real-time systems
Want to learn how to build production-grade Microservices with Spring Boot?
Follow for more backend engineering, system design, and microservices content: https://lnkd.in/dd965_rG
#SpringBoot #Java #Microservices #JavaDeveloper #DistributedSystems

MEDIA advertisement

LINKS
- https://lnkd.in/dd965_rG

#532 Mahesh Mallikarjunaiah ↗

System Design

AI Executive & Generative AI Transformation Leader | Driving Enterprise Innovation & AI Community Growth | From Idea to Intelligent Product | Driving Technology Transformation | AI community Builder

4mo · View on LinkedIn

Do you practice system design by memorizing architectures ?
URL shortener → Redis
Feed → Fanout
Scale → Sharding
Its a very wrong approach
Switching from **pattern recall** to **deliberate practice** is what will help you.
Here’s the loop to follow if you want to pass solid system design interviews :
1. Pick a boring system
Not Netflix. Not Instagram.
URL shortener. Rate limiter. Audit log. API gateway.
2. Start with numbers, not boxes
Users, QPS, read/write ratio, p99 latency, availability.
If you’re guessing — say it out loud.
3. Design the smallest thing that could work
One service. One database. No cache. No async.
Yes, it feels wrong. That’s the point.
4. Break it on purpose
Ask only “what happens when”:
– traffic 10×
– DB writes succeed but reads lag
– cache evicts faster than it fills
If you can’t explain how you’d *detect* the failure, you don’t understand it yet.
5. Add exactly ONE fix
One problem → one solution.
Then immediately ask: what new failure did I introduce?
6. Remove a component
Take out the cache. Or replication.
If the system still works under your assumptions, you never needed it.
Stop chasing “correct architectures”
and started defending **trade-offs with numbers and failure modes**.
That’s the gap between pattern fluency and real judgment.
what’s one design choice you make by default, without really defending it ?
#systemdesign

#533 **Prasanth V** Cloud / K8s / DevOps

AWS DevOps Engineer | Kubernetes (EKS) Expert | CKA Certified | Docker | CI/CD | Terraform | GitHub Actions | Argo CD | Linux || Open to Work

4mo · View on LinkedIn

One of the BIGGEST challenges for Kubernetes beginners is
→ Managing clusters using only kubectl commands
Because remembering dozens of commands is not always easy
That's why tools like Kubernetes Dashboard and Lens became extremely popular
Both tools help you visualize and manage Kubernetes clusters
But they work very differently

What is Kubernetes Dashboard?
Kubernetes Dashboard is a web-based UI that runs inside the cluster.
Think of it as:
Kubernetes Control Panel
What can you do with Dashboard?
View Pods
View Deployments
View Services
Monitor resources
Troubleshoot issues
Deploy applications
These capabilities are highlighted in the Dashboard overview section.
What is Lens?
Lens is a desktop application for Kubernetes.
Many engineers call it:
"VS Code for Kubernetes"
Simple understanding
Dashboard = Runs INSIDE cluster
Lens = Runs on your laptop
Why Lens is so popular
✓ No installation inside cluster
✓ Uses existing kubeconfig
✓ Supports multiple clusters
✓ Rich debugging experience
One-Line Interview Answer
→ "Kubernetes Dashboard is a web-based UI running inside the cluster, while Lens is a desktop-based Kubernetes IDE that provides advanced observability, troubleshooting, and multi-cluster management."
Final Takeaway
Both tools make Kubernetes easier...
Dashboard → Simple cluster visualization
Lens → Full Kubernetes management experience
And that's why most production engineers eventually move from Dashboard to Lens for day-to-day operations
#Kubernetes #DevOps #Lens #KubernetesDashboard #CloudComputing
62. Kubernetes Dashboard & Len
.
6 pages

MEDIA advertisement; document/carousel (possible)

#534 Ashish Pratap Singh

System Design

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon
4mo · **View on LinkedIn**

How to Design URL Shortener in a System Design Interview:
1. Requirements

Functional:

→ Shorten a long URL into a short code.

→ Redirect a short code back to the original URL.

→ Support optional custom aliases and expiry dates.

Non-functional:

→ Read-heavy: links are created once and clicked many times.

→ Low latency on redirects.

→ High availability and horizontal scalability.

2. API Design

You mainly need two endpoints.

→ POST /shorten: takes long_url, plus optional custom_alias and expiry_date, and returns the short URL.

→ GET /{short_code}: looks up the original URL and returns a 301 redirect.

Handle the error cases too: 404 when the code does not exist, 410 when the link has expired.

3. High-Level Architecture

Since the system is read-heavy, split reads from writes.

→ A URL Generation Service handles writes.

→ A Redirection Service handles reads.

A load balancer routes traffic to each. The redirection path sits behind a Redis cache, so popular links are served from memory. On a cache miss, you read from the database and populate the cache.

4. Database Schema

The access pattern is a simple lookup by short code, so a NoSQL store like DynamoDB fits well and scales to billions of rows.

url_mappings:

→ short_code (primary key)

→ long_url

→ user_id

→ created_at

→ expiry_date

→ click_count

users:

→ user_id (primary key)

→ email, name, password

5. Short Code Generation

Skip random strings, since they cause collisions and force extra database checks.

Use a counter instead.

→ A Redis counter creates a unique number with an atomic INCR.

→ Base62 encoding turns it into a short string like 8ryaqQ.

INCR is atomic, so no two requests ever get the same number. With a 7-character code, you get about 62^7, roughly 3.5 trillion combinations.

What else would you like to add to the design? Let me know in the comments.

Repost to help others get better at System Design

→ If you are preparing for interviews, checkout my website algomaster.io for pattern-based learning resources

LINKS

- https://algomaster.io/lnv
- http://algomaster.io

#535 Pramod Ahirwar

AI / LLM

Java Backend Developer | 2+ Years of Experience | Spring Boot | Microservices | REST APIs | Hibernate | Docker | JWT Authentication | MySQL | Building Scalable Backend Systems | M.Tech (CSE)

4mo · View on LinkedIn

Exploring Java + AI Integration

Recently, I started learning how AI can be integrated with Java Spring Boot applications, and honestly, it feels like the future of backend development is changing very fast.

In this demo/video, I explored how a Java backend can interact with AI models to process user input and generate intelligent responses in real time.

Concepts Covered:

✓ Java Spring Boot Backend

✓ AI API Integration

✓ Request → Response Flow

✓ Prompt Processing

✓ Service Layer Communication

✓ AI Response Handling

✓ Backend + AI Architecture

One thing I realized while learning this:

AI is not replacing backend development — it is making backend systems smarter and more powerful.

As a Java Backend Developer, understanding how AI integrates with APIs, services, and scalable architectures is becoming extremely valuable.

Currently exploring:

Spring Boot

REST APIs

AI Integrations

Kafka

Microservices

Scalable Backend Systems

Would love to hear your thoughts on Java + AI integration

#Java #SpringBoot #AI #ArtificialIntelligence #BackendDevelopment #JavaDeveloper #Microservices #Programming #SoftwareEngineering #RESTAPI #Tech #Developers

#536 Chandrakumar R Pillai

AI / LLM

AI Advisor & Enterprise Architect | Co-Founder & President, Gate2ASI | Guiding Europe’s AI Innovation & Compliance | Board Member, Author, Speaker | Based in Brussels, EU

4mo · View on LinkedIn

Most people are still paying ₹20,000–₹2,00,000 for AI courses...

Meanwhile, these YouTubers are quietly doing a better job for free.

No fluff.

No outdated theory.

Just real AI skills, taught in public.

We curated 7 YouTubers who explain AI better than most paid courses ever will.

From:

→ Building with AI tools

→ Understanding core concepts simply

→ Shipping real projects fast

If you’re still “thinking about learning AI” — this is your sign to start free, start now.

Swipe through.

Pick one.

And go build.

Follow Chandrakumar R Pillai for more such posts

7 YOUTUBERS WHO TEACH AI BETTE

.

8 pages

MEDIA document/carousel (possible)

#537 Kubernetes

Cloud / K8s / DevOps

Ravi Ranjan • 3rd+

4mo · View on LinkedIn

K8s Services and types ? #kubernetes.

#538 Zoran Milosevic

AI / LLM

Senior Software Engineer / Architect | Python | FastAPI | JavaScript | React | Next.js | TypeScript | C# | .NET Core | AI | Docker | Kubernetes | Microservices | Software Architecture | Databases | Automation

4mo · View on LinkedIn

MCP vs RAG vs AI Agents

Everyone is talking about MCP, RAG, and AI Agents. Most people are still mixing them up. They’re not competing ideas. They solve very different problems at different layers of the stack.

MCP (Model Context Protocol) is about how LLMs use tools. Think of it as a standard interface between an LLM and external systems. Databases, file systems, GitHub, Slack, internal APIs.

Instead of every app inventing its own glue code, MCP defines a consistent way for models to discover tools, invoke them, and get structured results back. MCP doesn’t decide what to do. It standardizes how tools are exposed.

RAG (Retrieval-Augmented Generation) is about what the model knows at runtime. The model stays frozen. No retraining. When a user asks a question, a retriever fetches relevant documents (PDFs, code, vector DBs), and those are injected into the prompt.

RAG is great for:

- Internal knowledge bases

- Fresh or private data

- Reducing hallucinations

But RAG doesn’t take actions. It only improves answers.

AI Agents are about doing things. An agent observes, reasons, decides, acts, and repeats. It can call tools, write code, browse the internet, store memory, delegate tasks, and operate with different levels of autonomy.

--

#systemdesign

#coding

#interviewtips

MEDIA document/carousel (possible)

#539 Sivasankar Natarajan

AI / LLM

Technical Director | GenAI Practitioner | Azure Cloud Architect | Data & Analytics | Solutioning What's Next

4mo · View on LinkedIn

Is Your AI Agent Architecture Just a Box Labeled "LLM"?

Most people think an AI agent is just a model with a prompt.

It's actually an 8-layer stack. The model sits in the middle. The other 7 layers are what make it reliable, connected, secure, and production-ready.

What does the full stack look like, bottom to top?

1. Infrastructure Layer

• Servers, cloud platforms, storage, and monitoring to run AI at scale.

• Tools: AWS EC2, Google Cloud Storage, Kubernetes, Terraform, Grafana, Prometheus.

Everything above this layer assumes this one works. Get infrastructure wrong and nothing else matters.

2. Agent Internet Layer

• Connects multiple agents and services so they share data and communicate in real time.

• Tools: Docker, Kubernetes, Kafka, ZeroMQ.

3. Protocol Layer

• Defines how agents talk to each other communication methods and data formats.

• Tools: REST API, gRPC, GraphQL, Protobuf, WebSockets, MQTT.

Without standardized protocols, every agent integration becomes custom glue code.

4. Tooling Layer

• Gives AI access to external tools, APIs, databases, and workflows.

• Tools: LangChain, Transformers, Jupyter, FAISS, RASA.

This is where agents go from thinking to doing.

5. Cognition Layer

• The brain thinking, reasoning, decision-making, problem-solving.

• Tools: TensorFlow, PyTorch, ChatGPT, IBM Watson, scikit-learn.

The layer everyone over-invests in. Important, but useless without the other seven.

6. Memory Layer

• Stores past conversations, user data, and preferences for personalization.

• Tools: MongoDB, FAISS, Chroma, Weaviate, Pinecone, Redis.

An agent without memory starts from zero every time.

7. Application Layer

• The AI-powered apps users actually interact with.

• Tools: Botpress, Dialogflow, RASA, Power Apps, Notion AI.

Great cognition with a bad interface still fails.

8. Governance Layer

• Rules, security, privacy, permissions, and monitoring to keep the system safe.

• Tools: HashiCorp Vault, Cloud Custodian, Open Policy Agent, Datadog.

The layer everyone adds last and should add first.

Skip any one layer and the agent breaks somewhere you didn't expect. The teams shipping reliable agents aren't obsessing over which LLM to use. They're building all eight layers.

Which layer is your agent architecture missing?

Repost this to help your network get started

+ Follow Sivasankar Natarajan for more

#AgenticAI #AIArchitecture #AIEngineering

#540 Shalini Goyal

AI / LLM

Executive Director, AI & Engineering @ JPMorgan | Amazon Alum | Author · Speaker · Professor | Helping Engineers Break into AI & High-Impact Careers

4mo · View on LinkedIn

An Agentic AI system is not just a chatbot connected to tools.

It is a full architecture where users, agents, tools, memory, security, monitoring, and infrastructure work together.

At the top, the User / Client Layer receives requests from web apps, chat, voice, APIs, SDKs, or enterprise systems.

The Orchestration Layer acts like the control plane. It breaks down tasks, selects the right agent, plans execution, manages state, and applies guardrails.

The Agent Layer is where the real work happens.

Research Agents search and analyze information.

Reasoning Agents plan, decide, and solve complex problems.

Action Agents call tools and execute tasks.

Data Agents query, process, and transform information.

Communication Agents summarize, explain, and respond clearly.

Then comes the Tools & Integrations Layer, where agents connect with APIs, databases, web search, code execution, files, documents, and external services.

The Memory & Knowledge Layer gives the system context through short-term memory, long-term memory, vector databases, knowledge bases, event history, and user profiles.

A serious agentic system also needs strong support layers:

Monitoring & Observability to track traces, logs, latency, cost, failures, and anomalies.

Reliability & Failure Management to handle retries, fallbacks, human review, and circuit breakers.

Governance & Security to manage authentication, authorization, privacy, policy enforcement, guardrails, and audit trails.

The foundation is the Infrastructure Layer: LLM providers, model gateways, vector databases, storage, queues, cache, secrets management, and deployment systems.

Agentic AI becomes powerful when every layer is designed properly.

Not just agents.

The entire system around them.

MEDIA document/carousel (possible)

#541 Gabriel Millien

AI / LLM

Enterprise AI Execution Architect | Closing the AI Execution Gap | $100M+ in AI-Driven Results | Trusted by Fortune 500s: Nestlé • Pfizer • UL • Sanofi | AI Transformation |Board Member | Fractional CAO | Keynote Speaker

4mo · View on LinkedIn

9 steps to ship an AI agent.

3 more to keep it in production.

The guide above shows the 9.

Here are the 3 the guide does not.

That is the difference

between an agent that demos

and an agent a CFO will let touch a customer.

First, the 9. This is the build path most teams follow:

1. Define the agent's role

What it does, who it helps, what it outputs.
2. Structure input and output
Pydantic or JSON schemas. Think like an API.
3. Tune the behavior
Role-based system prompts. Prefix or prompt tuning.
4. Add reasoning and tools
ReAct, Chain-of-Thought, search, code, retrievers.
5. Orchestrate multiple agents
Planner, researcher, reporter. CrewAI, LangGraph, Swarm.
6. Add memory and RAG
Vector stores. Conversational and summary memory.
7. Add voice or vision (optional)
ElevenLabs, GPT-4o, LLaMA 3.2.
8. Format the output
Markdown, PDF, or JSON. Human or machine readable.
9. Wrap in a UI or API (optional)
Gradio, Streamlit, FastAPI.
Follow that and you have a working agent.
You do not yet have an enterprise system.
Here are the 3 that turn one into the other:
10. Evaluation
Not unit tests. Continuous quality scoring.
Drift detection. Regression catches when the model updates.
11. Governance
Who can use it. What data it can touch.
Audit trail at runtime. Policy enforcement before output ships.
In regulated industries this is the layer
that decides whether the agent ever goes live.
12. Human oversight
Defined escalation when confidence is low.
Override authority named upfront.
A person on the call when the agent gets it wrong.
Skip these three and the pattern is always the same.
The agent works in demo.
It breaks quietly in production.
And the postmortem cannot find an owner.
These five layers are what I call the Enterprise Agent StackTM.
Infrastructure. Orchestration. Evaluation. Governance. Human oversight.
Builder guides like this cover Infrastructure and Orchestration.
The other three are where the AI Execution Gap lives.
I founded an enterprise AI incubator. Built the ML lifecycle from scratch.
Infrastructure and orchestration were never the reason an agent stalled.
Evaluation, governance, and human oversight were the reason it never left the lab.
Anyone can build an agent that demos.
Very few can build one a CFO will let near a customer.
The difference is not the model.

It is the three layers no one wires until something breaks.

Save this for your next agent design review.

Repost to help a builder in your network.

Follow Gabriel Millien for daily insights on closing the AI Execution Gap.

Image Credit: Arfa Farheen, give her a follow.

#542 Brij Kishore Pandey

AI / LLM

AI Architect & Engineer | Agentic systems, RAG, AI infrastructure, Data Engineering | 738K+ LinkedIn, 294K+ Instagram | Newsletter for 250K AI builders

4mo · View on LinkedIn

Everyone's throwing around the same 12 words right now.

RAG. Agent. Agentic AI. MCP. Tools. Skills. Hooks. Subagents. Memory. Context. Orchestration. Eval.

Master these 12 terms and watch the whole game change. You stop guessing why your agent breaks and start seeing exactly which layer failed.

Each one sits at a different layer, and the whole thing only works when they stack.

So I drew the entire stack out — one picture per idea, plain English, no jargon ↓

The foundation

→ RAG — pulls in outside knowledge before answering

→ Context — everything the model can see right now

→ Memory — what it carries across turns and sessions

The agent itself

→ Agent — an AI that acts, not just answers

→ Agentic AI — systems that plan, decide, and self-correct

→ Tools — external functions the agent can call

→ MCP — one open standard to connect tools and data

The capabilities

→ Skills — reusable know-how, loaded on demand

→ Hooks — triggers that run logic at set points

→ Subagents — specialist helpers the main agent hands work to

Running the system

→ Orchestration — coordinates the whole multi-step workflow

→ Eval — measures whether any of it actually works

Here's the part most people skip:

The foundation layer decides what your agent knows.

The agent layer decides what it can do.

The capability layer decides how well it scales.

And Eval is the only layer that tells you the truth about the other three.

You can have perfect tools and brittle memory and still ship something that fails in production. The pieces don't compete — they compose. Get one wrong and the whole stack feels broken.

The teams winning with agents right now aren't the ones who picked the best single primitive. They're the ones who understand how all twelve fit together.

If the field standardized on one shared vocabulary for this stack, which of these 12 terms do you think is still the most misunderstood?

LINKS

- https://brij.media/live

#543 Lakshmi Venkatesan

Other

Vice President, ANZ

4mo · View on LinkedIn

Problem solving in software engineering is rarely about memorising solutions, but about recognising patterns.

Recently, I consolidated a structured roadmap of important DSA patterns/types that repeatedly help in breaking down complex problems into reusable approaches.

Once we start recognising the pattern, many difficult problems suddenly become far more approachable.

Arrays & Strings → foundations for traversal, indexing, transformations, and pattern recognition

Sliding Window → efficient handling of subarray/substring optimization problems

Two Pointers → comparisons, partitioning, pair problems, and sequence processing

Slow & Fast Pointers (Floyd Cycle Detection) → cycle detection, middle node discovery, pointer-based optimizations

Prefix Sum → cumulative calculations, range queries, and optimized aggregation logic

Binary Search Pattern → efficient search, optimization problems, and “search on answer” techniques

Stack / Monotonic Stack → dependency tracking, next greater/smaller patterns, expression evaluation

Queue / Monotonic Queue → BFS traversal, scheduling, and streaming problems

Heap / Priority Queue & Top K Elements → ranking, prioritization, streaming data, nearest/top-k computations

Linked Lists → pointer manipulation, reversal, merging, traversal logic

Trees & Binary Tree Traversals → hierarchical problem solving using DFS, BFS, preorder, inorder, and postorder patterns

Graphs → traversal, shortest paths, connectivity, dependency resolution

Trie → prefix matching, autocomplete, dictionary search optimization

Union Find / DSU → grouping, connectivity, component detection

Intervals → overlap detection, scheduling, merging strategies

Matrix Traversal → grid-based reasoning, flood fill, pathfinding, island-style problems

Recursion & Backtracking → combinations, subsets, search trees, constraint solving

Dynamic Programming → optimization through overlapping subproblems and reusable states

Greedy Algorithms → locally optimal decisions for globally efficient outcomes

Sorting, Searching & Bit Manipulation → foundations behind performance optimization and elegant solutions

Inference:

Most difficult problems are often combinations or variations of a handful of reusable patterns.

The change happens when the question changes from:

“How do I solve this exact problem?”

to

“Which pattern can simplify this problem?”

That mindset makes problem solving more structured, reusable, and scalable & helps in solving complex problems?

DSA Top Algorithms By Type

.

31 pages

MEDIA document/carousel (possible)

#544 SOORAJ VIDYASAGAR

System Design

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

4mo · View on LinkedIn

Caching improves performance and scalability by storing frequently accessed data closer to applications and users. It reduces latency, lowers database load, and improves response times — but every caching strategy introduces trade-offs.

Common caching strategies include

Cache Aside,

Read Through,

Write Through,

Write Back,

Distributed Cache,

and CDN/Edge Cache.

Each approach balances speed, consistency, complexity, and operational cost differently.

Key trade-offs in caching:

* Performance vs Consistency
* Cost vs Scalability
* Freshness vs Availability
* Simplicity vs Complexity

Best practices for scalable caching:

* Cache frequently read data
* Use TTL values carefully
* Monitor cache hit/miss ratios
* Design proper invalidation strategies
* Avoid caching highly volatile data

At scale, caching is not only about faster systems — it is about maintaining reliability, consistency, and efficiency while handling massive workloads.

#Caching #DistributedSystems #SystemDesign #Scalability #CloudComputing #SoftwareArchitecture #Microservices #Redis #BackendDevelopment #DevOps

#545 Ganesh Bankar

System Design

Java Microservice Expert | 1M+ post impressions | Writes to 4K+ | AI | 11+ Years | Spring Boot | AWS | Azure | Development Manager@Mastercard | Guest Speaker @CDAC Pune

4mo · View on LinkedIn

A well-designed caching strategy can reduce database load, improve response time, and make systems far more scalable.

Here’s a quick breakdown of what every backend engineer should know about caching

✓ Cache Hit vs Cache Miss

Cache Hit → Data found instantly in cache

Cache Miss → Fetch from DB and store in cache

✓ Common Caching Strategies

Cache-Aside

Write-Through

Write-Behind

✓ Popular Cache Eviction Algorithms

LRU (Least Recently Used)
LFU (Least Frequently Used)
FIFO (First In First Out)
✓ What should be cached?
Frequently accessed data
User sessions
Configurations
Aggregated results
(!) Common Challenges
Stale data
Cache stampede
Cache penetration
Cache breakdown
Best Practices
Set proper TTL
Monitor hit/miss ratio
Avoid caching highly volatile data
Design graceful fallback mechanisms
Tools commonly used in real-world systems:
Redis
Memcached
Caffeine
CDN caching
In distributed microservices architecture, caching is often the difference between a system that survives traffic spikes and one that crashes under load.
What caching strategy have you used most in production systems?
#Java #SpringBoot #Microservices #SystemDesign #Caching #Redis #BackendDevelopment #SoftwareEngineering #DistributedSystems #PerformanceOptimization
#LearnWithGaneshBankar

#546 Sudeep Srivastava Kafka / Messaging

Associate Principal Architect | AI-Driven Enterprise Solutions | Generative AI, RAG, AI Agents | Azure Cloud | .NET | Microservices | Angular | 18+ Years

4mo · **View on LinkedIn**

Pub/Sub Pattern in Distributed Systems Explained Simply
As applications grow, direct service-to-service communication becomes difficult.
Instead of:
Order Service ➔ Notification Service
Order Service ➔ Inventory Service
Order Service ➔ Analytics Service
You introduce an event broker.
Now:
Order Service ➔ Event Bus ➔ Subscribers
This is the Publish–Subscribe (Pub/Sub) Pattern.
What is Pub/Sub?
• Publishers emit events

• Subscribers consume only relevant events
• Services stay independent of each other
Result → Loose coupling + easier scaling
Real-World Example (E-Commerce)
Customer places an order.
Order Service publishes:
OrderPlaced
Subscribers react independently:
• Inventory Service ➔ Reserve stock
• Notification Service ➔ Send email
• Analytics Service ➔ Update dashboard
• Fraud Service ➔ Validate order
Need another consumer later?
Just subscribe.
Order Service stays unchanged.
Key Distributed System Concepts
• Topic → Where events are published
• Consumer Group → Parallel processing
• Partitioning → Horizontal scaling
• Offset → Track processed events
• Idempotency → Handle duplicates safely
• DLQ → Capture failed events
• Eventual Consistency → Data syncs over time
(!) Trade-offs
Pub/Sub improves scalability but introduces:
• Duplicate events
• Ordering challenges
• Consumer lag
• Retry handling
• Schema evolution
Common Technologies
Kafka • RabbitMQ • Azure Service Bus • AWS SNS/SQS
Big Takeaway
Pub/Sub is not just messaging.
It enables systems that are:
• scalable
• loosely coupled
• resilient
The hard part is not publishing events...
It’s building consumers that behave correctly when events arrive late, out of order, or more than once.

#547 Chinmaya Lenka Java / Spring

Java Backend Developer | Spring Boot | Microservices | REST APIs | AWS | MySQL | Kafka | Redis | Docker | Scalable Backend Systems

4mo · View on LinkedIn

Improving API Performance in Spring Boot (Real Optimization Approach)

While working on a Spring Boot service, I observed higher response latency than expected under normal load.

Instead of guessing, I broke down the system and analyzed the actual bottlenecks.

Issues Identified:

✗ Multiple unnecessary DB calls

✗ N+1 query problem

✗ No caching for frequently accessed data

✗ Inefficient query structure

Optimizations Applied:

✓ Reduced DB round trips by restructuring service logic

✓ Optimized queries using JOIN FETCH & proper indexing

✓ Introduced caching using @Cacheable for read-heavy endpoints

Impact:

Response time improved from ~2.5 sec → ~800 ms

Better throughput and more stable performance under load

Key Takeaways:

→ Always measure before optimizing

→ Database is often the main bottleneck

→ Caching should be applied strategically

→ Small improvements can create big impact

Engineering Insight:

Performance optimization is not just about speed—

it’s about building systems that remain reliable and scalable in real-world scenarios.

#SpringBoot

#Java

#BackendEngineering

#PerformanceOptimization

#Microservices

#SoftwareEngineering

#548 Anis Rahman Java / Spring

Java Full Stack Developer | @AnisCodex - Peer Guide for Aspiring Developers | Spring Boot · Hibernate · React.js · REST APIs | DSA in Java · 200+ Pattern-Based Problems Solved | Open to opportunities

4mo · View on LinkedIn

Most backend devs think they’re interview-ready... until the first “explain how it works internally” question hits.

Core Java & Internals

• How memory works in Java (Heap, Stack, GC cycles)

• What happens during garbage collection

• When finalize() fails you

• The four principles of OOP with real examples

• equals() vs hashCode()

• Fail-fast vs fail-safe collections

• How HashMap works internally

- Why ConcurrentHashMap is different
- Immutable objects
- Serialization vs Deserialization
- Deep copy vs shallow copy

Multithreading & Performance

- What is concurrency vs parallelism
- Thread lifecycle
- ExecutorService and thread pools
- Deadlock, livelock, starvation
- volatile vs synchronized
- CountDownLatch vs CyclicBarrier
- CompletableFuture

Spring Boot & Enterprise Java

- How Spring Boot auto-configuration works
- What happens at startup inside Spring
- Bean life cycle
- Dependency Injection vs Inversion of Control
- Profiles and environments
- Filters vs Interceptors
- Exception handling in production
- Actuator and monitoring
- Logging best practices
- Externalized configuration

Databases & ORM

- How JPA and Hibernate actually work
- Lazy vs eager loading
- N+1 problem
- First-level vs second-level cache
- Dirty checking
- Transactions in Spring
- Isolation levels
- Indexing strategy
- ACID properties
- Normalization vs denormalization
- Pagination at scale

API Design & Security

- What makes a RESTful API
- PUT vs POST vs PATCH
- Versioning strategies
- Idempotency
- OAuth vs JWT
- Token expiration strategies
- Handling rate limiting
- CORS
- Secure password storage
- API gateway role

System Design & Architecture
• Monolith vs microservices
• Docker basics
• Containerization strategy
• Service discovery
• Load balancing
• Caching strategies
• Message queues
• Horizontal vs vertical scaling
• Circuit breakers
• Failover strategy
• Handling millions of requests
Backend engineering is not about writing code.
Companies don’t pay you for frameworks.
They pay you for decisions that don’t break systems.
Most people fail backend interviews
not because they don't know Spring...
but because they ignored DSA.

#549 Nitesh Kumar Saw Other

Helping Founders & Professionals Build Authority on LinkedIn | Personal Branding | LinkedIn Growth Strategist | Lead Generation | Building Upliftly

4mo · **View on LinkedIn**

Most candidates do not fail coding interviews because they lack talent.
They fail because they prepare randomly.
Data Structures and Algorithms are not just interview topics anymore. They are a test of problem-solving ability, consistency, and structured thinking.
I came across a powerful collection of 100 essential DSA interview coding questions covering:
• Arrays
• Strings
• Linked Lists
• Stacks & Queues
• Trees
• Graphs
• Dynamic Programming
• Heaps and more
What makes these questions valuable is not the difficulty level.
It is the pattern recognition they build over time.
If you are preparing for software engineering interviews, focus less on solving hundreds of random problems and more on mastering the core concepts repeatedly.
A few important lessons every aspiring developer should remember:
• Consistency beats intensity
• Understanding beats memorization
• Revision beats one-time practice
• Problem-solving ability compounds daily
The candidates who stay disciplined with preparation eventually stand out in interviews.

Keep building. Keep learning. Keep solving.

#DSA #CodingInterview #SoftwareEngineering #Programming #DataStructures #Algorithms #TechCareers #InterviewPreparation #CodingJourney

Top 100 DSA Interview Coding

.

4 pages

MEDIA document/carousel (possible)

LINKS

- https://www.upliftly.in/

#550 Adarsh C.

Java / Spring

DevRel Engineer | Sharing the latest developments in Tech, Web3, and AI | Content Creator | Exploring ways to excel at the game of life

4mo · **View on LinkedIn**

10 Golden Rules for Solving a Coding Question in an Interview

1 If we are dealing with

top/maximum/minimum/closest

'K' elements among 'N' elements, we will be using a Heap

2 If the given input is a sorted array or a list, we will either be using Binray Search or the Two Pointers strategy.

3 If we need to try all combinations (or permutations) of the input, we can either use Backtracking or Breadth First Search.

4 Most of the questions related to Trees or Graphs can be solved either through Breadth First Search or Depth First Search.

5 Every recursive solution can be converted to an iterative solution using a Stack.

6 For a problem involving arrays, if there exists a solution in O(n^2)time and O(1) space, there must exist two other solutions: 1) Using a HashMap or a Set for O(n) time and O(n) space, 2) Using sorting for O(n log n) time and O(1) space.

7 If a problem is asking for optimization (e.g., maximization or minimization), we will be using Dynamic Programming.

8 If we need to find some common substring among a set of strings, we will be using a HashMap or a Trie.

9 If we need to search/manipulate a bunch of strings, Trie will be the best data structure.

10 If the problem is related to a LinkedList and we can't use extra space, then use the Fast & Slow Pointer approach.

Do follow AdarshChetan for more such amazing stuff

#interview #interviewprep #interviewpro #dsa #coding #java #git #dp #dynamicprogramming

#551 Sahil Kumar

Other

Building Scalable Backend Systems | Spring Boot Developer | ML + Real-World Applications

4mo · **View on LinkedIn**

Sorting algorithms are one of those DSA topics everyone learns...

but keeps revisiting again and again

Because during interviews, it’s not enough to just know how they work.

You also need to remember:

- Which one is stable?

- Which one is in-place?
- Which one is fastest in practice?
- Which one works best for nearly sorted arrays?
So I made this Sorting Algorithms Cheat Sheet for quick revision.
It covers:
Bubble Sort
Selection Sort
Insertion Sort
Merge Sort
Quick Sort
Heap Sort
Counting Sort
Radix Sort
Along with:
Time Complexity
Stability
In-place sorting
Best use cases
Interview tips
One thing I realized while learning sorting:
Understanding why an algorithm is used matters way more than memorizing steps.
For example:
- Merge Sort → stable + great for large datasets
- Quick Sort → insanely fast in practice
- Insertion Sort → surprisingly useful for small/nearly sorted arrays
And yes... Quick Sort worst case still scares me sometimes
Save this for your next DSA revision
What topic should I simplify next?
#DSA #SortingAlgorithms #CodingInterview #Algorithms #DataStructures #Java #Programming #InterviewPreparation #LeetCode

#552 Sai Roshan Neelam Java / Spring

Software Engineer @ Gamyam | MERN Stack Developer | Building Scalable Web Apps & AI-Powered Features | Python • SQL • DSA | NCC Cadet

4mo · **View on LinkedIn**

If you’re preparing for coding interviews...
Memorizing patterns helps.
But recognizing time complexity instantly?
That’s a superpower
Because interviewers don’t just care if your code works.
They care if it scales.
That’s why I created a quick DSA Time Complexity Cheat Sheet for interview revision.
It covers ↓
Arrays
Strings
Linked Lists

Stack / Queue
HashMap / HashSet
Binary Search
Heap / Priority Queue
Trees
Graph Traversals
Sorting Algorithms
Common Interview Patterns
Recursion Rules
(!) Some interview truths most beginners miss:
→ HashMap average case ≠ worst case
→ Binary Search ONLY works on sorted data
→ BFS / DFS complexity = O(V + E)
→ Sliding Window often reduces:
✗ O(n2)
→ ✓ O(n)
One thing I learned while doing DSA:
Choosing the right approach matters FAR more than typing code fast.
Because:
• brute force solves problems
• optimized thinking clears interviews
Strong DSA candidates don’t memorize everything.
They recognize:
✓ patterns
✓ complexity trade-offs
✓ optimization opportunities
That’s what makes problem-solving feel easier over time.
Which complexity or pattern confused you the most initially?
O(log n)?
Graphs?
DP?
Save this before your next coding round
Share with someone preparing for interviews
#DSA #CodingInterview #Algorithms #DataStructures #SoftwareEngineering #Programming

#553 Sai Krishna Chivukula Other

Principal Data Engineer @ Altimetrik | 75K+ Followers | ★Top 1% Cloud & Data Engineering Mentor | Certified in SNOWFLAKE | 1x FABRIC | 2x AZURE | 2x Databricks | SQL | Informatica | Databricks | Ex Carelon, ADP, CTS
4mo · **View on LinkedIn**

Most people fail DSA not because they can’t code...
But because they never learn the PATTERNS behind the problems.
You do NOT need to solve 3,500 questions.
You need to recognize the RIGHT PATTERN at the right time. ✓
Once that clicks, problem-solving becomes 10x faster.
Sharing the 15 most important DSA patterns every developer should master

1. Two Pointers
✓ Traverse arrays/strings efficiently using two indices
Examples: 3Sum, Container With Most Water
2. Sliding Window
✓ Maintain a moving window to optimize subarray/substring problems
Examples: Longest Substring Without Repeating Characters
3. Fast & Slow Pointers
✓ Detect cycles or find middle nodes efficiently
Examples: Linked List Cycle, Middle of Linked List
4. Binary Search
✓ Divide search space into halves
Examples: Search in Rotated Sorted Array
5. Binary Search on Answer
✓ Search possible answer range instead of array directly
Examples: Koko Eating Bananas
6. Merge Intervals
✓ Merge overlapping intervals smartly
Examples: Merge Intervals, Insert Interval
7. Backtracking
✓ Try all possibilities and backtrack when needed
Examples: N-Queens, Sudoku Solver
8. DFS (Depth First Search)
✓ Go deep before exploring alternatives
Examples: Number of Islands, Clone Graph
9. BFS (Breadth First Search)
✓ Explore level-by-level using queues
Examples: Shortest Path in Binary Matrix
Topological Sort
✓ Solve dependency-based problems
Examples: Course Schedule, Alien Dictionary
11. Dynamic Programming
✓ Break problems into reusable subproblems
Examples: Coin Change, Climbing Stairs
12. Greedy
✓ Take the best local choice at each step
Examples: Activity Selection, Huffman Coding
13. Graph
✓ Work with nodes and edges efficiently
Examples: Dijkstra, Union Find
14. Tries (Prefix Tree)
✓ Fast string searching and prefix matching
Examples: Implement Trie, Word Search II
15. Heap / Priority Queue
✓ Efficiently retrieve min/max elements
Examples: Kth Largest Element, Merge K Sorted Lists
Most “hard” interview problems are just patterns in disguise.

Save this before your next coding round.

Tag someone preparing for placements or SDE interviews. ↓

Follow me here - Sai Krishna Chivukula

Repost this to help someone crack DSA interviews faster Drop a if you want Part 2 with advanced patterns

#DSA #CodingInterview #DataStructures #Algorithms #SoftwareEngineer #CareerGrowth #SDE #TechByKrish

#554 Java Developers Community (moderated)

System Design

Gurpreet Singh • 3rd+ | Prototype Design Pattern Explained Simply | Create new objects by cloning existing ones instead of creating them from scratch. | The Prototype Design Pattern helps improve performance and reduce repetitive object creation by copying existing objects and modifying them when needed. | Learn how Prototype Pattern makes object creation faster, reusable, and efficient using simple Java examples and real-world use cases. | #Java #PrototypePattern #DesignPatterns #SystemDesign #Programming #SoftwareEngineering #BackendDevelopment #Coding #Developers | 45 | 1 | 3 | | Millan Mohanty Premium Profile 3rd+ | Millan Mohanty | Tech Strategy & Software Consultant | I Help Businesses Reform Their Digital Presence & Accelerate Growth.

4mo · View on LinkedIn

Great breakdown of the Prototype pattern. In enterprise software architecture, this pattern is an absolute lifesaver when dealing with costly database calls for object initialization. When you are dynamically generating complex digital assets or standardizing data structures across different business segments, cloning a pre-configured prototype and tweaking the specific fields is exponentially more efficient than querying the database from scratch every time. The inclusion of the 'Deep Clone' warning at the end is crucial—messing up nested object references during a clone is a classic source of massive production bugs

MEDIA advertisement

#555 Parikh Jain

AI / LLM

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

4mo · View on LinkedIn

Solve this self-testing DSA cheatsheet with 13 groups of patterns & 65 problems to check whether you’re ready for FAANG-level DSA rounds in 2026.

On-site interviews are making a comeback at many companies, so I’ve spent 30+ hours collecting these problems from real interviews at Amazon, Uber, Flipkart, JP Morgan, DE Shaw, and many other FAANG+ companies.

The goal should be to do this:

- Take any 2 random problems

- Set a 45-minute timer, simulate interview conditions

- See how far you get, after 45 mins, check where you fell short.

- Improve the gaps in your preparation and improve for the better

Btw, this cheatsheet is based on my DSA + Dev + GenAI Placement System: https://lnkd.in/gut39Xsr.

I’ve just released a new version where, now, we have:

-50+ videos (15+ hours), all recorded by me

-Built-in code editor + AI assisted feedback also built-in

-Personal notes, patterns, code & solutions all stored in one place

Back to the cheatsheet:

[1] Arrays and Hashing Basics

- Problems: Two Sum, Product of Array Except Self, Contains Duplicate, Majority Element, Sort Colors

- Keywords: hash map, frequency, prefix product, counting, in place

- Trigger: Use when the question talks about counts, presence, or rearranging array values without complex rules.

[2] Two Pointer Techniques

- Problems: 3Sum, Container With Most Water, Move Zeroes, Remove Duplicates from Sorted Array, Valid Palindrome

- Keywords: left right pointers, shrinking range, swapping, partitioning

- Trigger: Use when you can sort or already have sorted data and need pairs, ranges, or in place cleanup.

[3] Sliding Window

- Problems: Longest Substring Without Repeating Characters, Subarray Sum Equals K, Minimum Window Substring, Sliding Window Maximum, String Compression

- Keywords: window start end, expand shrink, running count, best window

- Trigger: Use when you scan a string or array for the best subarray or substring that satisfies a condition.

[4] Binary Search on Sorted or Answer Space

- Problems: Search in Rotated Sorted Array, Find First and Last Position of Element in Sorted Array, Median of Two Sorted Arrays, Find Peak Element, Koko Eating Bananas

- Keywords: mid index, sorted property, search space, monotonic answer

- Trigger: Use when the input or the answer behaves in a sorted or monotonic way and time limit is tight.

[5] Matrix and Grid Traversal

- Problems: Number of Islands, Rotting Oranges, Spiral Matrix, Rotate Image, Valid Sudoku

- Keywords: 2D grid, visited set, boundaries, four directions

- Trigger: Use when the problem is on a board or grid and you need to explore neighbors or layers.

[6] Intervals and Timeline Scans

- Problems: Merge Intervals, Meeting Rooms II, Best Time to Buy and Sell Stock, Best Time to Buy and Sell Stock II, Daily Temperatures

- Keywords: sort by start, sweep line, prefix max, monotonic structure

- Trigger: Use when you handle time ranges, bookings, or prices that evolve over a timeline.

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gut39Xsr

#556 Marcel Dybalski

System Design

AI & Data Platform Engineer | GCP & Fabric Certified

4mo · **View on LinkedIn**

Your architect keeps using words like Idempotency or Immutability?

I got you... Here's what that actually means:

11 architecture principles translated for non-tech leaders:

1. Least Privilege

→ Only give access to what someone truly needs. Not what's easiest.

2. Separation of Concerns

→ Each system does one job well. Not everything poorly.

3. Defense in Depth

→ Multiple security layers. One failure doesn't sink the ship.

4. Fail Fast

→ Catch problems early. Before they become expensive.

5, Single Source of Truth
→ One place for core data. Not 12 competing versions.
6. Loose Coupling
→ Systems talk without being entangled. Change one without breaking five.
7. Immutability
→ Don't modify data. Create new versions. Easier to trace what happened.
8. Idempotency
→ Run the same action twice, same result. No surprises.
9. Scalability by Design
→ Built to grow. Not rebuilt every 6 months.
10. Observability
→ See what's happening inside. Not just when it breaks.
11. Zero Trust
→ Verify everything. Trust nothing by default.
These aren't just tech buzzwords.
They're the difference between systems that scale and systems that collapse.
Your architect isn't gatekeeping.
They're protecting your infrastructure from becoming a liability.
What principle do you hear most often?
➤ Follow Marcel for insights that bridge tech and leadership.
Tap the bell to get notified when I post.
Repost to help an exec decode their next architecture review.

#557 Ashutosh Bhondele Java / Spring

Java Full Stack Developer
4mo · View on LinkedIn

Most Java developers know how to sort.
But fewer understand how sorting works internally inside Java Collections.
That’s where Comparable and Comparator become truly powerful.
Comparable vs Comparator is not just an interview question — it’s about designing flexible, scalable, and maintainable systems.
Comparable defines the natural ordering of objects.
Comparator gives you custom sorting logic without modifying the class.
But what happens internally?
When you call:
→ Collections.sort(list)
Java’s TimSort algorithm starts comparing objects using:
✓ compareTo() → for Comparable
✓ compare() → for Comparator
That comparison logic directly controls how objects move during sorting operations.
In this infographic, I simplified:
Internal working of Comparable
Internal working of Comparator
TimSort comparison flow
Real-world use cases

Advantages & limitations
Why Comparator is more flexible in enterprise systems
Understanding internals like this changes how you write Java code forever.
Because great developers don’t just use APIs — they understand what happens underneath them.
Created by Ashutosh Bhondele
Which one do you prefer in real-world projects: Comparable or Comparator?
#Java #JavaDeveloper #Programming #SoftwareEngineering #BackendDevelopment #Coding #Developers #DataStructures #Algorithms #CollectionsFramework #Comparator #Comparable #Tech #ComputerScience #SpringBoot #ProgrammingLife #SoftwareDeveloper #DSA #CleanCode #Developer

MEDIA advertisement

#558 Java Developers Community (moderated)

AI / LLM

Dineshchand G R • 3rd+
4mo · View on LinkedIn

New article: RAG Can't Cancel Orders — Building a Transactional AI Agent with Spring Boot (Part 1)
The core insight: RAG-based systems are read-only by nature. If you want AI that takes action — cancels orders, updates records, triggers workflows — you need an agentic architecture.
Here's the stack I used:
- Spring AI 1.0 — ChatClient + @Tool for the agentic loop
- Google Gemini 2.5 Flash via OpenAI adapter — decides which tool to call
- MySQL 8 — live orders DB with JPA + Flyway
- React (SSE streaming) — real-time chat UI
The agent receives a natural language request, Gemini selects the right tool, and Spring AI executes it — writing directly to the database.
Part 1 is live - https://lnkd.in/g5J6dp-K
14
1
1

Dineshchand G R
Author
Principal Engineer | Solution Architect | GenAI | AWS 4x Certified | Enterprise Integrations | AWS Cloud | Java | Spring Boot | Singapore
5mo
Link to my github repo - https://github.com/dineschandgr/spring-ai-order-support-agent
GitHub - dineschandgr/spring-ai-order-support-agent: Spring AI 1.0 + Google Gemini 2.5 Flash — agentic customer support with @Tool calling, SSE streaming, MySQL, Flyway, and React frontend
github.com

MEDIA advertisement

LINKS
- https://lnkd.in/g5J6dp-K
- https://github.com/dineschandgr/spring-ai-order-support-agent

#559 Raghavendra Bagalkoti

Cloud / K8s / DevOps

Building the Future of Finance with Cloud + AI | Simplifying Capital Markets Technology | Thought Leader in Tech-Driven Finance

4mo · View on LinkedIn

Nobody teaches you AWS this way.

They throw 200+ services at you and wish you luck.

Here are the only 48 you actually need as a Cloud Engineer — mapped by domain.

Compute — Where your code runs → EC2, Lambda, ECS, EKS

Storage — Where your data lives → S3, EBS, EFS, FSx

Database — How your data is structured → RDS, Aurora, DynamoDB, ElastiCache

Networking — How everything connects → VPC, Route 53, CloudFront, Global Accelerator

Security — How you don't get fired → IAM, KMS, GuardDuty, Secrets Manager

Observability — How you know something broke → CloudWatch, CloudTrail, X-Ray, Config

DevOps — How you ship without breaking things → CloudFormation, CodePipeline, CodeBuild, CodeDeploy

Serverless — How you scale without thinking about servers → API Gateway, Step Functions, EventBridge, AppSync

Analytics — How you make sense of the data → Redshift, Athena, EMR, Glue

AI & ML — How you add intelligence → Bedrock, SageMaker, Rekognition, Comprehend

Migration — How you move without breaking everything → App Migration Service, DMS, Snowball, DataSync

End User — How your users connect → WorkSpaces, AppStream 2.0, Secure Browser, Amazon DCV

200+ services. 48 that matter most.

Start with the map. Learn how the services connect.

The rest is just detail.

Which domain/service tripped you up the most when learning AWS? ↓

Follow Raghavendra Bagalkoti for more on AI × Cloud × Capital Markets

#560 Sai Kiran

AI / LLM

Software Engineer @ Google | Ex-Microsoft & Deloitte | AI Engineer | Agentic AI | Full-Stack Developer | Cloud | Distributed Systems | Technical Consulting | Corporate Training | Building Scalable Products

4mo · View on LinkedIn

HOW MUCH YOU SHOULD ASK

AS AN AI ENGINEER IN 2026

1. BASIC AI KNOWLEDGE

• Prompting

• ChatGPT usage

• Basic AI tools

→ ₹4L – ₹8L

2. RAG BASICS

• Build basic RAG pipelines

• Vector search systems

• FastAPI backends

→ ₹12L – ₹20L

3. AI AGENTS & ORCHESTRATION

• LangGraph

• AI agents

• Orchestration
• Memory systems
• Multi-agent workflows
→ ₹25L – ₹40L
4. PRODUCTION AI SYSTEMS
• Production AI deployment
• Observability (Langfuse, etc.)
• Inference pipelines
• Scalable AI infra
→ ₹40L – ₹1Cr
5. ADVANCED AI ENGINEERING
• Fine-tuning LLMs
• Custom model training
• Advanced RAG (Hybrid Search)
• LLMOps / MLOps
• Scalable, reliable AI architecture
→ ₹60L – ₹1Cr+
IMPORTANT:
The market is no longer paying based on years of experience.
It's paying based on AI leverage.
Two developers with the same experience can now have:
a ₹6L salary... or a ₹60L salary...
Purely based on their AI engineering depth.

MEDIA advertisement

#561 Java Developers Community (moderated)

Other

Arman Shaikh • 3rd+ | Kafka Interview Questions PDF | Sharing a handy Kafka Interview Questions PDF covering important concepts, commonly asked questions, and key topics for Java/Backend developers. | Hope this helps anyone preparing or revising Kafka concepts. | #Kafka #Java #SpringBoot #BackendDevelopment #Microservices #SoftwareDevelopment | 24 pages | 281 | 1 | 52 | | Saikumar Gatla Premium Profile 3rd+ | Saikumar Gatla | Backend Developer | Java | Spring Boot | Microservices | Kafka | Building Scalable APIs | SQL | PostgreSQL | MySQL | Redis | JPA(Hibernate) | Query & API Optimization | JWT | Javascript | React js

4mo · **View on LinkedIn**

Helpful..

MEDIA advertisement; document/carousel (possible)

#562 Rakesh Gohel

AI / LLM

Builder @ JUTEQ | Agent governance & evidence infrastructure for regulated AI | Proof that an AI agent acted inside its limits, shown to anyone | Gartner CIO Summit speaker | Agentic AI Strategy, Adoption, Build 0->1 | View my services

4mo · **View on LinkedIn**

The real risk in AI Agents is not capability. It is control.

As enterprises move from experimentation to deployment, the governing question is no longer whether AI agents can perform tasks.

It is whether they can do so reliably, safely, and within defined business boundaries.

That is where most organizations underestimate the challenge.

AI agents operate on data, policies, permissions, and model behaviour. If any one of those layers is weak, the entire system inherits that weakness.

At the board level, this comes down to two control domains:

Data Governance

- Ensures the inputs are accurate, secure, traceable, and appropriately governed
- Protects the enterprise from poor-quality, incomplete, or unauthorized data entering the system

AI Governance

- Ensures the outputs are explainable, auditable, compliant, and accountable
- Protects the enterprise from opaque decisions, bias, and uncontrolled automation

For AI agents, the implication is clear: Without governance, autonomy becomes an operational risk. With governance, autonomy becomes scale.

The enterprise model should be built around five controls:

1. Trusted data pipelines with clear lineage and ownership
2. Access controls aligned to business policy and regulatory requirements
3. Model guardrails for transparency, bias mitigation, and decision oversight
4. Human escalation for high-impact or exception-based actions
5. Continuous monitoring across both input integrity and output behaviour

This is no longer a technical nuance. It is an operating model decision.

Organizations that treat governance as part of the architecture will scale AI with confidence.

Organizations that treat it as a post-deployment control will scale risk.

The companies that win with AI Agents will not simply move fastest.

They will design for trust, accountability, and control from day one.

If you want to go deeper on AI Agents, my free newsletter breaks down everything you need to know: https://lnkd.in/esJDdA5Q

Save → React → Share

& follow for everything related to AI Agents

MEDIA Data and AI Governance Stack

LINKS

- https://lnkd.in/esJDdA5Q

#563 Vinod Alabana

Kafka / Messaging

Mainframe Consultant at JPMorgan Chase | Software Engineer | Ex-Cognizant | JCL, COBOL, VSAM,DB2,IMS DB,CICS, MQ | API, KAFKA, AWS | OpenText Designer | Master's In Computer Science | IOT |

4mo · **View on LinkedIn**

In today's real-time systems, APIs and Apache Kafka play a major role in building scalable and event-driven architectures.

APIs are mainly used for synchronous communication — when an application needs an immediate response.

Kafka is used for asynchronous, real-time event streaming between systems.

#Kafka #RESTAPI #SpringBoot #Microservices #EventDrivenArchitecture #Java #AWS #DataEngineering #BackendDevelopment #DistributedSystems

MEDIA advertisement

#564 Ambadas Garje

Java / Spring

Senior Software Developer | 8 Years Experience in Spring Boot Ecosystem | Java, Spring MVC, Microservices, REST APIs | Nelito Systems Pvt. Ltd.

4mo · View on LinkedIn

Spring Framework | Day 27
Internal Working of Profiles in Spring Boot
Most developers use Profiles in Spring Boot for DEV, QA, and PROD environments...
But during interviews, many cannot explain:
→ “What happens internally when a profile is activated?”
Here’s a simple breakdown ↓
What is a Profile?
Profiles allow Spring Boot to load environment-specific configurations.
Example:
spring.profiles.active=dev
This tells Spring Boot:
✓ Load DEV configurations
✓ Create DEV-specific beans
✓ Ignore other environment beans
Internal Working Flow
Application Starts
↓
SpringApplication.run()
↓
Environment Object Created
↓
Active Profile Detected
↓
Profile Properties Loaded
↓
Beans Filtered Using @Profile
↓
Application Context Ready
Example Property Files
application.properties
application-dev.properties
https://lnkd.in/d-YaDbEG
If active profile is:
spring.profiles.active=dev
Spring Boot loads:
✓ application.properties
AND
✓ application-dev.properties
Property Override Mechanism
application.properties
server.port=8080
application-dev.properties
server.port=9090
Final value becomes:

9090
Because profile-specific properties override default values.
Bean Loading with @Profile
@Service
@Profile("dev")
public class DevService {
}
@Service
@Profile("prod")
public class ProdService {
}
If active profile = dev
✓ DevService gets created
✗ ProdService is ignored
Internally Spring uses:
ProfileCondition
to decide whether bean should load or not.
Why Profiles Are Powerful
✓ Separate environment configs
✓ Safer production deployments
✓ No hardcoded values
✓ Cleaner microservices setup
✓ Easy cloud deployment
Interview Tip
If interviewer asks:
How do Spring Boot Profiles work internally?
Don’t stop at:
✗ “Profiles manage configurations.”
Explain:
✓ Environment object
✓ Property source loading
✓ Override priority
✓ Bean filtering
✓ @Profile annotation
✓ ProfileCondition
#SpringBoot #Java #Microservices #BackendDeveloper #SoftwareArchitecture #JavaDeveloper #CodingInterview #Programming #TechLearning #DeveloperTips

MEDIA advertisement

LINKS
- http://SpringApplication.run
- http://application.properties
- http://application-dev.properties
- https://lnkd.in/d-YaDbEG

#565 Java Developers Community (moderated)

AI / LLM

Mohamed EL Laithy • 3rd+

4mo · View on LinkedIn

100+ AI tools. One screen. Organized by what they actually do.

I got tired of bookmarking AI tools in 15 different places and never finding them again — so I built this map.

It groups 100+ tools into the categories that matter for a real workflow:

→ Generative AI (ChatGPT, Claude, Gemini, Mistral, Llama)

→ Text to Image & Text to Video (Midjourney, Sora, Runway, Veo)

→ Voice & Text (ElevenLabs, Murf, superwhisper)

→ Code (Cursor, v0, Replit, Devin, Lovable)

→ Marketing, SEO, Design, Website builders

→ Productivity, Meetings, Operations

→ Workflow automation (Zapier, Lindy, Cassidy, Magical)

The point isn't to use all of them. It's to know which tool to reach for the moment a task lands on your desk — and to stop reinventing your stack every few months.

The "Optimized Core" in the middle is the real lesson: a handful of tools wired together beats a graveyard of 50 free trials.

Save this one. You'll come back to it.

Repost this to help someone fix their AI stack.

+ And follow me (Mohamed El Laithy) for more AI tools, workflows, and cheat sheets every week.

Which category is carrying your workflow right now — and which tool would you add that I missed? Drop it below. ↓

#AITools **#ArtificialIntelligence

MEDIA advertisement

#566 Parikh Jain

AI / LLM

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

4mo · View on LinkedIn

DSA Preparation Roadmap (From Zero to OA Ready)

1. Sliding Window: 3, 76, 209, 424, 567, 904
2. Two Pointers: 11, 15, 16, 18, 42, 167
3. Fast/Slow Pointers (Linked List): 141, 142, 19, 876, 160, 234
4. Binary Search on Sorted Data: 33, 34, 35, 153, 162, 704
5. Binary Search on Answer: 875, 1011, 410, 774, 1283, 1482
6. Hashing / Frequency Maps: 1, 49, 128, 217, 242, 347
7. Prefix Sum / Running Sum: 303, 560, 724, 930, 974, 523
8. Difference Array / Range Updates: 370, 1094, 1109, 1893, 1943, 2381
9. Monotonic Stack: 739, 496, 503, 84, 85, 901
10. Monotonic Queue / Deque: 239, 862, 1425, 1438, 1499, 1696
11. Heap / Top K: 215, 347, 692, 703, 973, 1046
12. Intervals: 56, 57, 252, 253, 435, 452
13. Greedy Scheduling / Sorting: 45, 55, 406, 621, 763, 134
14. Linked List Manipulation: 21, 23, 24, 25, 92, 138
15. Tree DFS: 104, 112, 113, 543, 124, 226
16. Tree BFS / Level Order: 102, 103, 199, 515, 637, 116
17. BST Problems: 98, 99, 230, 235, 450, 700

18. Backtracking Basics: 46, 47, 77, 78, 90, 39
19. Backtracking with Constraints: 40, 17, 79, 131, 51, 52
20. Graph BFS / DFS: 200, 695, 733, 994, 1091, 1254
21. Topological Sort / DAG: 207, 210, 802, 1462, 1203, 2115
22. Union Find / DSU: 547, 684, 1319, 1579, 990, 1202
23. Shortest Path: 743, 787, 1514, 1631, 1334, 1976
24. MST / Graph Greedy: 1584, 1135, 1168, 1489, 778, 1102
25. Trie: 208, 211, 212, 648, 677, 1268
26. Bit Manipulation: 136, 137, 191, 338, 268, 190
27. 1D DP Basics: 70, 198, 213, 322, 279, 300
28. Knapsack / Subset DP: 416, 494, 518, 474, 1049, 879
29. Grid DP: 62, 63, 64, 221, 931, 120
30. String DP / Sequence DP: 1143, 72, 115, 583, 97, 1312

While we're talking about interviews, check out my DSA + Dev + Gen AI system. Link: https://lnkd.in/gut39Xsr

1000+ students are already using the new version of it that comes with everything.

• 50+ DSA pattern videos that build your thinking
• Mock tests and practice rounds to train you for interview pressure
• System design prep so you can answer in-depth questions
• Core CS, GenAI, full stack, all structured so nothing is left as a gap

How to use this list?

- These numbers are Leetcode problem numbers
- Do 3 patterns at a time, not all 30 together.
- For each pattern, solve the first 2 to understand the idea, the next 2 to get repetition, and the last 2 to stretch yourself.
- After every pattern, write one reusable template from memory.
- Do not just "solve and move on." Ask: what signal in the question pointed to this pattern?
- If you get stuck, revisit the same pattern after 3 to 4 days. Pattern recognition is built by spacing, not cramming.

If I had to simplify this even more:

Start with these 10 first:

- Sliding Window
- Two Pointers
- Binary Search
- Hashing
- Prefix Sum
- Monotonic Stack
- Heap
- Intervals
- Trees
- 1D DP

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/gut39Xsr

#567 Ankur Kesharwani

System Design

Senior AI engineer | 140K+ @LinkedIn | AI creator for Businesses | AI, Tech & Marketing | AI tools | Helping brands with growth and conversions | Views are personal

5mo · View on LinkedIn

In 2025, I made my 2nd switch with a 70% hike to Senior MTS at Oracle!

And with every interview I sat for, I realised that 80% of top PBCs repeat the same 20% of design problems.

That’s why I say this loud and clear to every engineer I talk to who’s preparing for senior/leadership roles↓

DSA will surely get you shortlisted. But System Design gets you the offer.

The questions might change slightly, but the core patterns? They repeat across companies like Microsoft, Amazon, Uber, Oracle, Atlassian and more.

Here are some of the most common design problems that I’ve seen come up:

Low-Level Design (LLD)

→ Parking lot system (with levels, entry/exit points, and slot assignment)

→ Chess or Tic Tac Toe (OOP, moves, win conditions)

→ LRU Cache (Eviction logic, O(1) operations)

→ Thread-safe HashMap

→ Design a scalable rate limiter

High-Level Design (HLD)

→ URL Shortener like TinyURL

→ Real-time Chat App like WhatsApp

→ Distributed Key-Value Store

→ Uber Ride-Matching System

→ Notification System (fan-out messaging, retries)

Once I shifted my prep to focus on the “why” behind each component, trade-offs, constraints, and real-world use, I started handling variations confidently.

So if you’re prepping for design rounds asked in senior interviews, don’t keep circling theory.

Practice what shows up, understand how to break it down, and get feedback.

And if you're looking for a structured way to do that or need guidance from industry experts while juggling a 9-5, I recommend checking out Bosscoder Academy.

Explore their program here: bcalinks.com/RfUVzIu

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#systemdesign #techinterviews #pbc #jobswitch #collab

LINKS

- http://bcalinks.com/RfUVzIu

#568 Ayushmaan Chandra

System Design

Founder & CEO| FinTech & Digital Payments | Payment Gateway Workflows | UPI & Card Payments | Merchant KYC | Risk & Compliance Foundations | Payments Infrastructure | Cloud Architecture

5mo · View on LinkedIn

Why Most Payment Failures Are Actually False Failures in Digital Payment Systems

Most people assume that if a payment fails, the system has broken somewhere.

But in real digital payment infrastructure, many “failed” transactions are not actual failures. They are temporary states inside a complex multi-system flow.

Here is what actually happens in practice:

A “failed” payment is often just a timing issue, not a failure

In payment systems, a transaction can appear failed due to timeout or delayed response, but still get successfully processed in the background.

Bank and network latency creates false failure signals

UPI, card networks, and banking systems operate across multiple layers. Even a small delay in one system can trigger a failure response on the user side while backend processing continues.

Retry mechanisms change the final outcome

Many payment systems automatically retry transactions through alternate routes or same rails. What looks like a failure in the first attempt may succeed in the second or third attempt.

Authorization and settlement are not the same stage

A transaction may be authorized but not confirmed in the user interface due to settlement delays or communication gaps between systems.

Frontend status does not always reflect backend reality

Apps and gateways often show simplified statuses like success or failure. The actual backend flow includes pending, queued, reversed, or delayed states that are not visible to users.

Risk systems can temporarily block and later release transactions

Some payments are held for risk checks and later cleared, which makes the initial failure message misleading.

System design prioritizes safety over instant accuracy

Payment infrastructure is designed to avoid fraud and system overload. In some cases, it prefers temporary failure signals over incorrect approvals.

What users see as failure is often just an incomplete state

In real systems, payment flow is not binary. It is a continuous process that completes across multiple systems over time.

In most cases, payment failures are not system breakdowns.

They are temporary mismatches between distributed systems trying to stay in sync in real time.

Understanding digital payments properly means looking beyond success and failure, and understanding how states evolve across the ecosystem.

A strong payment system is not one that never shows failure. It is one that eventually resolves every transaction correctly across systems.

#Payments #DigitalPayments #UPI #FinTech #PaymentGateways #FinTechOperations #PaymentSystems #PaymentInfrastructure #RiskManagement #SystemDesign

#569 AlgoMaster.io

Other

Visit website

5mo · **View on LinkedIn**

10 Must-Know Graph Algorithms for Coding Interviews:

1. DFS (Depth-First Search)
2. BFS (Breadth-First Search)
3. Topological Sort
4. Union Find (Disjoint Set)
5. Cycle Detection
6. Connected Components
7. Bipartite Check

8. Flood Fill

9. Minimum Spanning Tree

10. Shortest Path (Dijkstra)

→ If you want to master all important algorithms and patterns for coding interviews checkout algomaster.io

Repost to help others in your network

LINKS

- https://algomaster.io/
- http://algomaster.io

#570 Java Developers Community (moderated)

Java / Spring

Sarvesh Kumar Singh • 2nd

5mo · **View on LinkedIn**

Blocking vs Non-Blocking in Java —

Most developers write code like this:

→ Call API → wait

→ Call DB → wait

→ Process → wait

Looks normal... right?

✗ But this is exactly why your app doesn’t scale.

Let’s break it down ↓

The Core Difference

Blocking

→ Thread waits until task completes

→ Nothing else happens meanwhile

Simple... but wastes resources

Non-Blocking

→ Thread does NOT wait

→ Continues executing other tasks

Efficient + scalable

Simple Analogy

Blocking= Waiting in line for coffee

Non-Blocking = Order coffee → sit → get notified

In Java Terms

✓ Blocking:

→ Traditional Spring MVC

→ JDBC

→ Thread-per-request model

✓ Non-Blocking:

→ WebFlux

→ CompletableFuture

→ Reactive Streams

Where Developers Go Wrong

✗ Using blocking calls inside async code

✗ Mixing blocking + non-blocking blindly

✗ Not understanding thread usage

Interview Killer Question
“Why is non-blocking better for scalability?”
Strong Answer:
→ Because threads are not wasted waiting — they can handle more requests efficiently.
Real-World Insight
If your app:
Has low traffic → blocking is fine
Handles thousands of requests → go non-blocking
↓ Final Thought
Blocking code is easy to write...
Non-blocking code is what makes systems scale.
hashtag
#Java hashtag
#ReactiveProgramming hashtag
#WebFlux hashtag
#BackendDevelopment hashtag
#Scalability hashtag
#SoftwareEngineering

#571 SHIVAM SOMANI System Design

SDE2 @ Microsoft | Tech, AI & Career Coach | Top 1% LinkedIn | Gen AI | Mentor | Content Creator | Backend Engineer | 50K+ Followers | Open for Brand Collaborations reach here : somani2025shivam@gmail.com

5mo · **View on LinkedIn**

Before I made my 2nd switch to Microsoft , I gave an interview at Walmart for a senior SDE role...
And in the round focusing on LLD + HLD, the problem was: Design the Feeds section of Instagram / Facebook.
Sounds familiar, right? But the depth expected is very different.
Here’s how it went ↓
Step 1: Requirement clarity
Started by narrowing scope:
• post types, metadata
• comments, replies
• consistency expectations
Step 2: Data modeling
Discussion around:
• SQL vs NoSQL choices
• structuring posts, users, likes, comments
• handling relationships at scale
Step 3: Scaling the system
Focus shifted to:
• feed delivery at scale
• caching strategies (LRU, LFU, TTL)
• handling heavy read traffic
Step 4: LLD task
Implement LFU cache
• clean class design

• extensibility
• separation of concerns
Step 5: Design quality
• SOLID principles
• use of patterns where needed
• clear structure over complex code
So for such rounds, focus on communicating your thought process, how you handle ambiguity, structure systems, and balance between scalability and consistency matters more than just perfect code.
And if you're an experienced SDE with 6+ YOE aiming for senior/lead roles, this level of clarity needs structured prep and feedback.
That's exactly what Bosscoder Academy's EVOLVE Program is built for.
Explore EVOLVE here → bcalinks.com/WDOrrHX
They've helped 2200+ engineers crack top PBC roles through:
Structured curriculum to help you master DSA, System Design, and AI
Real world, AI-powered projects & 24/7 doubt-solving
Leadership skills for Senior roles with 1:1 mentorship from industry experts
Resume reviews, mock interviews, and 100% job switch support.
#sdeprep #systemdesign #lld #jobswitch #seniorsde #interviewexperience #collab

LINKS
• http://bcalinks.com/WDOrrHX

#572 Shivam Garg

AI / LLM

SWE-III @ Google | IIT Ropar | 125K+ @Linkedin | AI for scaling businesses & Marketing | Hackathon Judge | Mentor| Technical Content Creator | DM for Collaboration

5mo · **View on LinkedIn**

What does it take to go from SDE-1 → 40-50+ LPA Senior SDE at Google, Amazon, Microsoft, etc?
Is it:
→ Solving harder DSA problems?
→ Switching to a new tech stack?
→ Or building AI-integrated side projects?
From what I've seen working at Google as an SDE-3 after 4 yrs in the industry, I've realised that the jump from mid-level → senior isn't about more coding...
It's about thinking like an owner.
And here's what actually shifts when engineers move toward senior roles ↓
1. Senior interviews don't ask "Which framework?" They ask "Why this architecture?"
The focus moves from syntax to trade-offs:
caching strategy
■ concurrency control
■ data modeling & storage choices
■ cost vs performance decisions
2. It's not just about writing clean code, it's about explaining decisions clearly under time pressure. Hiring panels look for:
■ structured thinking
■ clarity of trade-offs
■ ability to collaborate across teams

3. A lot of engineers talk about what they built, very few talk about:

■ what broke

■ how they fixed it

■ how they prevented it from happening again

Ownership shows up in failure cases, & that’s what signals maturity.

So if you're a dev like me aiming to switch to more senior or leadership level roles, focusing on these 3 will make a huge difference.

And if you want guidance from industry experts who’ve walked this path...

I highly recommend giving EVOLVE by Bosscoder Academy a try, built esp. for SDEs with 6+ YOE, is a great start.

Explore EVOLVE here: bcalinks.com/9CamH31

They’ve helped 2200+ engineers crack top PBC roles through:

Structured curriculum to help you master DSA, System Design, and AI

Real world, AI-powered projects & 24/7 doubt-solving

Leadership skills for Senior roles with 1:1 mentorship from industry experts

Resume reviews, mock interviews, and 100% job switch support.

#seniorsde #systemdesign #leadership #pbc #jobswitch #collab

LINKS

- http://bcalinks.com/9CamH31

#573 Narendra Sahoo System Design

Senior Technical Lead @ HCL | Java • Spring Boot • Microservices • Kafka | AWS & Cloud-Native Architectures | AI Integration (LLM & RAG) | Tech Mentor | YouTuber @ Naren Java Help Line

5mo · **View on LinkedIn**

Most developers say they know microservices.

Very few understand how production systems actually work.

If you want to become strong in Microservices, master these concepts:

✓ API Gateway

✓ Service Discovery

✓ Load Balancing

✓ Circuit Breaker

✓ Event-Driven Architecture

✓ CQRS

✓ Saga Pattern

✓ Service Mesh

✓ Distributed Tracing

✓ Containerization

✓ Database per Service

✓ Bulkhead Pattern

✓ BFF Pattern

✓ Blue-Green Deployment

✓ Strangler Fig Pattern

These are not just interview topics.

These are the concepts used in real enterprise systems handling millions of requests daily.

The difference between a beginner and an experienced backend engineer is understanding:

→ Scalability

→ Fault tolerance
→ Distributed transactions
→ Observability
→ Deployment strategies
→ Resiliency patterns
Microservices are easy to draw on a whiteboard.
Hard part is making them reliable in production.
Which microservices concept do you use most in production?
Repost to help developers learn
Follow Narendra Sahoo for Java, Spring Boot, System Design & Microservices
Connect with me if you need step by step guidance.
#Java #Microservices #SpringBoot #SystemDesign #BackendDevelopment #SoftwareArchitecture #Kafka #Docker #Kubernetes #DevOps #DistributedSystems #Programming #JavaDeveloper #Tech #SoftwareEngineer

#574 Ramya A K Java / Spring

Java Full Stack Developer | Spring Boot | React | Vue | Microservices | JWT | Performance Optimization | 6+ YOE | PayPay

5mo · **View on LinkedIn**

The Spring Framework is a powerful Java framework used for building enterprise applications with features like Dependency Injection (DI), IoC, MVC, Security, and Data access.
But traditional Spring applications required:
Heavy XML configuration
Manual dependency setup
External server deployment
Longer project setup time
That’s where Spring Boot changed everything.
Why Spring Boot Became Popular?
✓ Auto Configuration
Automatically configures application components based on dependencies.
✓ Embedded Servers
No need to deploy WAR files separately. Comes with Tomcat/Jetty built-in.
✓ Starter Dependencies
Simplifies dependency management using starters like:
spring-boot-starter-web
spring-boot-starter-data-jpa
✓ Less Configuration No large XML setup. Mostly annotation and properties-based.
✓ Microservices Friendly Perfect for cloud-native and REST API applications.
✓ Production Ready Features Includes monitoring, health checks, metrics, and logging using Actuator.
Simple Difference
→ Spring gives flexibility
→ Spring Boot gives speed and productivity
Key Takeaway
Spring Boot follows “Convention over Configuration”, making Java backend development faster, cleaner, and easier for modern applications.
#Java #Spring #SpringBoot #BackendDevelopment #Microservices #JavaDeveloper #SoftwareEngineering #FullStackDeveloper

#575 Ashish Pratap Singh

System Design

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

5mo · View on LinkedIn

Redis is NOT just a cache. Here are 10 use cases you should know:

1. Caching

The most common use case. Your app checks Redis before hitting the database. If the key exists, return it instantly. If not, query the DB, store the result in Redis, and serve it.

2. Session Store

When a user logs in, store their session data (user ID, roles, preferences) in Redis. Every subsequent request just looks up the session key.

3. Rate Limiting

Redis makes rate limiting trivial. Use INCR to increment a request counter and EXPIRE to reset it after a fixed window. If the count exceeds the limit, throttle or block the request.

4. Leaderboards

Redis Sorted Sets keep users ranked by score in real time. You can update scores, fetch the top N users, and find a user's rank efficiently in O(log N) operations.

5. Pub/Sub Messaging

Publishers send messages to a channel, and subscribers receive them instantly. Great for chat, live notifications, and dashboards. Trade-off: messages are fire-and-forget.

6. Real-Time Analytics

Use counters and HyperLogLog to track clicks, views, searches, unique visitors, and active users with very low memory usage.

7. Distributed Locking

Redis SETNX (SET if Not eXists) lets you acquire a lock atomically. Set it with a TTL so locks auto-release if a process crashes. For stronger guarantees, use the Redlock algorithm.

8. Geospatial Queries

Redis GEO commands let you store coordinates and query nearby locations, such as restaurants within 5 km or drivers near a user. It uses sorted sets with geohash encoding under the hood.

9. Message Queue

Redis Lists with LPUSH and BRPOP can act as a lightweight queue for simple background jobs. For more advanced use cases, Redis Streams are a better fit.

10. Real-Time Notifications

Redis Streams provide an append-only log with consumer groups. Keyspace Notifications let you subscribe to changes on specific keys. Useful for notifications, activity feeds, background processing, and real-time pipelines.

How have you used Redis in production? Let me know in the comments.

Repost to help others in your network.

→ If you are preparing for Software Engineering interviews, checkout my website algomaster.io for comprehensive resources

LINKS

- https://algomaster.io/lnv
- http://algomaster.io

#576 Ashish Pratap Singh

Other

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

5mo · View on LinkedIn

SOLID Principles Explained with Clear Examples:

S - Single Responsibility Principle

A class should have only one reason to change.

- Example: Instead of one giant User class that handles authentication, profile updates, and sending emails, split it into UserAuth, UserProfile, and EmailService.

O - Open/Closed Principle

Classes should be open for extension but closed for modification.

- Example: Define a Shape interface with an area() method. When you need a new shape, just add a Circle or Triangle class that implements it.

L - Liskov Substitution Principle

Subtypes must be substitutable for their base types without breaking behavior.

- Example: If Bird has a fly() method, then Eagle and Sparrow should both work anywhere a Bird is expected.

I - Interface Segregation Principle

Don't force classes to implement interfaces they don't use.

- Example: Instead of one fat Machine interface with print(), scan(), and fax(), break it into Printable, Scannable, and Faxable. A SimplePrinter only implements Printable.

D - Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

- Example: Your OrderService should depend on a PaymentGateway interface, not directly on Stripe or PayPal.

The real power of SOLID is not in following each principle in isolation. It's in how they work together to make your code easier to change, test, and extend.

Repost to help others in your network

→ If you want to master technical interviews in less time, checkout algomaster.io for pattern-focused resources

LINKS

- https://algomaster.io/lnv
- http://algomaster.io

#577 AlgoMaster.io

Other

Visit website

5mo · **View on LinkedIn**

20 must-know DSA patterns for coding interviews:

1. Prefix Sum
2. Two Pointers
3. Sliding Window
4. Fast & Slow Pointers
5. LinkedList In-place Reversal
6. Frequency Counting
7. Monotonic Stack
8. Bit Manipulation
9. Top ‘K’ Elements
10. Overlapping Intervals
11. Binary Search Variants
12. Binary Tree Traversal
13. Depth-First Search (DFS)
14. Breadth-First Search (BFS)

15. Shortest Path
16. Matrix Traversal
17. Backtracking
18. Prefix Search (Trie)
19. Greedy
20. Dynamic Programming Patterns

→ Checkout the full list of patterns with detailed explanations and problems at algomaster.io

Repost to help others master coding interviews

LINKS

- https://algomaster.io/
- http://algomaster.io

#578 Java Developers Community (moderated)

Java / Spring

Sarvesh Kumar Singh • 2nd

5mo · **View on LinkedIn**

Concurrency vs Parallelism in Java

If you think these two are the same...

→ You're not alone

→ But you are missing a critical concept

Let's fix it ↓

The Core Difference

Concurrency

→ Dealing with multiple tasks at the same time

→ Tasks may NOT run simultaneously

It's about structure

Parallelism

→ Running multiple tasks simultaneously

→ Requires multiple CPU cores

It's about execution

Simple Analogy

Concurrency= One chef cooking multiple dishes (switching tasks)

Parallelism= Multiple chefs cooking at the same time

In Java Terms

✓ Concurrency:

→ Threads

→ ExecutorService

→ Async programming

✓ Parallelism:

→ Parallel Streams

→ ForkJoinPool

→ Multi-core processing

Where Developers Get It Wrong

✗ Thinking multithreading = parallelism

✗ Ignoring CPU limitations

✗ Overusing threads → performance drops

Interview Killer Question

“Can you have concurrency without parallelism?”
Answer:
→ YES
Real-World Insight
If your app:
Handles many tasks → use concurrency
Needs speed for heavy computation → use parallelism
↓ Final Thought
Concurrency is about managing work
Parallelism is about doing work faster
#Java #Concurrency #Parallelism #Multithreading #BackendDevelopment #SoftwareEngineering

#579 Julia Danyal

AI / LLM

Become a credible Chief AI Leader. Earn the AI certification built for leaders. Get AI certified now → chiefaileader.com

5mo · View on LinkedIn

AI isn't magic. It's a stack.
Become better at AI in just 1 minute a day.
Get the AI newsletter smart leaders read.
Sign up free now → aiforleaders.com

_____
Most people only see the newest layer.
Chatbots.
Image generators.
Agents.
Automations.
Tools that seem to do the work in seconds.
But none of this appeared overnight.
AI has been built layer by layer for decades.
Here is the simple version.
1. Classical AI
This was rule-based software.
If this happens, do that.
The system did not learn.
It followed instructions written by humans.
Useful for structured tasks.
Weak when the real world got messy.
2. Machine Learning
This changed the approach.
Instead of writing every rule, teams gave the system examples.
The model learned patterns from data.
That made prediction possible.
Fraud detection.
Recommendations.
Forecasting.

Risk scoring.
This is where AI started becoming useful for business.
3. Neural Networks
Neural networks made AI better at messy inputs.
Images.
Speech.
Language.
Unstructured data.
They helped machines find patterns that older systems could not handle well.
This made AI feel less rigid.
4. Deep Learning
Deep learning scaled neural networks.
More data.
More compute.
More layers.
This is where AI started improving quickly in areas like language, vision, and speech.
A lot of the AI tools people use today sit on top of this progress.
5. Generative AI
This is the layer most people know.
AI stopped only predicting or classifying.
It started creating.
Text.
Images.
Code.
Audio.
Video.
Presentations.
This is why AI reached a mainstream audience so fast.
People could see the output immediately.
6. Agentic AI
This is where the next wave is forming.
AI can now plan steps, use tools, remember context, and move through a workflow.
An agent can research, decide what to do next, call a tool, check the result, and continue.
That is a different level of usefulness.
The important point is simple.
Each layer made the next layer possible.
Rules made structure possible.
Data made learning possible.
Neural networks made messy inputs possible.
Deep learning made scale possible.
Generative AI made creation possible.
Agentic AI is making execution possible.
That is why AI feels like it is moving so fast.
The foundation has been building for a long time.
Which layer are you paying most attention to right now?

MEDIA AI isn't magic. It's a stack. •••••••••••••• Become better at AI in just 1 minute a day. Get the AI newsletter smart leaders read. Sign up free now → aiforleaders.com •••••••••••••• Most people only see the newest layer. Chatbots. Image generators. Agents. Automations. Tools that seem to do the work in seconds. But none of this appeared overnight. AI has been built layer by layer for decades. Here is the simple version. 1. Classical AI This was rule-based software. If this happens, do that. The system did not learn. It followed instructions written by humans. Useful for structured tasks. Weak when the real world got messy. 2. Machine Learning This changed the approach. Instead of writing every rule, teams gave the system examples. The model learned patterns from data. That made prediction possible. Fraud detection. Recommendations. Forecasting. Risk scoring. This is where AI started beco

LINKS

- http://aiforleaders.com/jb
- http://aiforleaders.com

#580 Spring Users

Java / Spring

Suraj Mishra • 2nd

5mo · **View on LinkedIn**

Java/Spring Boot Scheduler Cheatsheet

- Scheduling is one of those features that looks trivial on the surface but hides significant complexity the moment you move from a single-server prototype to a multi-instance production deployment.

- This guide walks through every major scheduling approach in the Java/Spring Boot ecosystem: when to reach for each one, how it works under the hood, and the pitfalls that cause bugs in production.

https://lnkd.in/ehWPmHRn

LINKS

- https://lnkd.in/ehWPmHRn

#581 Sagar Palai

Other

45K+ Impressions | GSSoC'25 Campus Ambassador & Contributor | Infosys Springboard Intern | Freelancer | Contributor @ OpenSourceConnectIndia | 4x Hackathon Participant | AIML | Data Analyst

5mo · **View on LinkedIn**

One thing that completely changed how I approach DSA interviews ↓

Most coding interview questions are not testing whether you memorized solutions.

They test whether you can recognize the underlying pattern.

Because once you spot the pattern, the solution path becomes much clearer

That's why I started focusing less on:

"Have I solved this exact question before?"

...and more on:

"What type of problem is this?"

That small shift made interview prep feel much easier and far less stressful.

Here are some of the most important DSA patterns that appear again and again in interviews:

Two Pointers

Sliding Window

Fast & Slow Pointers

Binary Search

DFS / BFS

Backtracking

Hashing

Dynamic Programming

Greedy Algorithms

When you understand these patterns deeply, you can solve completely new questions with confidence instead of relying on memorization.

That’s the real game in coding interviews

Bookmark this for your next placement or interview prep session

Which DSA concept should I break down next in a simple way?

#DSA #CodingInterview #Algorithms #DataStructures #LeetCode #Programming #SoftwareEngineering #Java #InterviewPrep #SoftwareDeveloper #CompetitiveProgramming #ProblemSolving #PlacementPreparation #CodingCommunity #Developers #InterviewTips #Python #Coding #Developer #Tech #CodingLife #LearnToCode

#582 Ankit Kumar

System Design

Java Backend Developer | 4.8 Years Experience | Spring Boot, Microservices, REST APIs | SQL | AWS (EC2, S3) | Docker | Open to Work

5mo · **View on LinkedIn**

SAGA PATTERN in Spring Boot — The Backbone of Distributed Transactions in Microservices

When working with microservices, one failed service should NOT break the entire business flow.

That’s where the Saga Pattern becomes powerful.

In this post, I covered:

✓ What is a Saga Pattern

✓ Event-driven architecture with Kafka

✓ Orchestration vs Choreography

✓ Compensation Transactions

✓ Real-world Order Processing Flow

✓ Failure Recovery Strategy

✓ Spring Boot + Kafka + PostgreSQL Setup

✓ Observability with OpenTelemetry & Grafana

Key Learning:

Distributed systems are not about avoiding failures.

They are about handling failures gracefully.

Why Saga Pattern matters:

✓ Eliminates distributed locks

✓ Enables eventual consistency

✓ Improves fault tolerance

✓ Supports scalable microservices

✓ Handles rollback using compensating events

Tech Stack:

Spring Boot 3 | Apache Kafka | PostgreSQL | Docker | Kubernetes | OpenTelemetry | Grafana

Real-world use cases:

• E-Commerce Order Processing

• Banking Transactions

• Insurance Workflows

• Logistics Systems

• Subscription Billing Platforms

Biggest takeaway:

Design for failure.

Recover gracefully.

Deliver consistently.

#Microservices #SpringBoot #Kafka #SystemDesign #Java #SoftwareArchitecture #BackendDevelopment #DistributedSystems #SagaPattern #EventDrivenArchitecture #PostgreSQL #Docker #Kubernetes #OpenTelemetry #Developers #Ankitkumar

#583 Anis Rahman

Java / Spring

Java Full Stack Developer | @AnisCodex - Peer Guide for Aspiring Developers | Spring Boot · Hibernate · React.js · REST APIs | DSA in Java · 200+ Pattern-Based Problems Solved | Open to opportunities

5mo · **View on LinkedIn**

HashMap vs ConcurrentHashMap in Java

HashMap

✓ Not Thread-Safe

✓ Allows one null key & multiple null values

✓ Faster in single-threaded applications

✓ Best for normal/non-concurrent operations

ConcurrentHashMap

✓ Thread-Safe

✓ Does NOT allow null keys/values

✓ Supports high concurrency

✓ Best for multi-threaded applications

Use HashMap when thread safety is not needed.

Use ConcurrentHashMap when multiple threads access data simultaneously.

#Java #HashMap #ConcurrentHashMap #BackendDevelopment #JavaCollections #Multithreading #SoftwareEngineering

MEDIA advertisement

#584 Sangram Sekhar Panda

System Design

Software Engineer I || Java || Spring Boot || Microservices || React || Apache Kafka || GCP || Fintech || UPI || Serving Notice period

5mo · **View on LinkedIn**

Recently explored Apache kafka with SpringBoot and learned how real-time event streaming works in modern microservices architecture.

#kafka #SpringBoot #Java #Microservices

MEDIA advertisement

#585 Omkar S.

AI / LLM

Head of SRE & Applied AI | Top 1% Engg Leader AI | Ex-Microsoft | Platform & SRE | AI-First Teams | Applied AI | Data Engineering | Nominated MSFT MVP | Azure Dev Community Lead

5mo · **View on LinkedIn**

Most "AI agent failures" are not AI failures. They are org failures wearing an AI label.

The model is rarely the bottleneck. \Scattered knowledge, undocumented workflows, poor data, no ownership, and missing guardrails will sink any agent project, no matter how good the underlying LLM.

15 signs your organization is not ready for AI agents:

1. Scattered Knowledge

• Information lives across multiple tools and teams

2. Documentation Doesn't Match Reality

• Documented processes do not reflect how work actually happens leads to incorrect decisions, fast

3. No Real-Time Data Access

• Agents rely on live signals to act effectively, decisions become outdated the moment they are made

4. Workflows Hidden in People

• Processes exist only in employees' heads and if it cannot be written down, it cannot be automated

5. Multiple Versions of Truth

• Different files, tools, and teams create conflicting data

6. No Approval or Review Layer

• If nobody validates outputs, mistakes scale quickly

7. Security Built Only for Humans

• Traditional access controls do not account for autonomous behavior

8. No Metrics for AI Performance

• If you are not measuring outcomes, you cannot improve them beacuse what gets measured gets optimized

9. Leadership Is Not Driving Adoption

• If leaders do not actively use AI, teams will not prioritize it

10. No Post-Launch Iteration

• AI systems improve with feedback without iteration, performance stagnates and quietly degrades

11. No Clear Ownership

• If no one is responsible for AI outcomes, issues remain unresolved

12. Poor Data Quality

• Incomplete or inconsistent data leads to unreliable outputs

13. No Defined Boundaries

• Agents need clear limits on what they can and cannot do

14. No Human-in-the-Loop

• Without oversight, small errors escalate quickly

15. No Cost Monitoring

• Untracked usage leads to unexpected costs as agents scale. Efficiency requires visibility into every token, call, and tool invocation

The takeaway

You cannot deploy your way out of these gaps. You have to fix them. The orgs winning with agents in 2026 are not the ones with the smartest models.

They are the ones with clean data, documented workflows, clear ownership, real guardrails, and leaders who actually use the systems they fund.

Audit these 15 honestly. The agent will not save you from the answers.

Repost to help your leadership team self-assess

+ Follow Omkar S. for more on architecting AI agents at scale

#AIAgents #EnterpriseAI #AIStrategy

MEDIA document/carousel (possible)

#586 Himanshu Sharma

System Design

Enterprise & Technology Architect | TOGAF Enthusiast | Banking Transformation and Strategy

5mo · **View on LinkedIn**

7 Architecture Decisions That Quietly Kill Scalability

Most systems do not fail because of traffic.
They fail because of architecture decisions that looked harmless when the system was small.
The scary part?
Many scalability problems are designed into the system years before the first outage happens.
Here are 7 architecture decisions that quietly destroy scalability:
---
1. Shared Database for Everything
It feels efficient in the beginning.
But over time:
every service becomes tightly coupled,
schema changes become political events,
and one slow query impacts everyone.
A shared database creates organizational bottlenecks long before technical bottlenecks.
---
2. Synchronous Calls Everywhere
One service waits for another. Which waits for another. Which waits for another.
Soon:
latency multiplies,
retries explode,
and small failures cascade across the platform.
Distributed systems fail in chains.
---
3. Premature Microservices
Many teams split into microservices before they even understand their domain boundaries.
What they gain in “modern architecture” they lose in:
debugging,
deployment simplicity,
observability,
and operational sanity.
A modular monolith often scales further than people expect.
---
4. Ignoring Observability Early
If you cannot trace:
requests,
dependencies,
failures,
and latency,
then scaling becomes guesswork.
At scale, visibility is not optional infrastructure. It is survival.
---
5. Over-Engineering for Imaginary Future Scale
Some systems are designed for:
100 million users,
multi-region failover,
extreme throughput,
before they even have product-market fit.

Complexity added too early becomes permanent architecture debt.

---

6. Centralized Decision Bottlenecks

When every architecture decision requires:

approvals,

governance meetings,

review boards,

or hierarchy escalation,

engineering speed collapses.

Scalable systems require scalable decision-making.

---

7. Optimizing for Technology Instead of Simplicity

Many architectures become collections of trends:

Kubernetes,

event streaming,

service mesh,

CQRS,

serverless,

AI integrations,

without solving actual business problems.

The simplest architecture that solves the problem usually wins.

At scale, simplicity is not laziness.

It is an engineering advantage.

The biggest scalability killer is rarely traffic.

It is accumulated complexity.

The best architects are not the ones who add the most technology.

They are the ones who remove unnecessary complexity before it becomes permanent.

#EnterpriseArchitecture #SoftwareArchitecture #Scalability #DistributedSystems #Microservices #SystemDesign #TechLeadership #EngineeringLeadership #CloudArchitecture #ArchitectureReview #TOGAF

#587 Srinath P.

AI / LLM

Senior Solution Architect | Distributed Systems | Java (17/21/25) | Microservices | Kubernetes | AI-Enabled Systems | High-Scale Platforms

5mo · **View on LinkedIn**

AI is no longer limited to Python ecosystems.

Modern Java applications are rapidly evolving with the power of Large Language Models (LLMs).

Recently explored how Java integrates seamlessly with enterprise AI workflows using:

✓ Spring Boot + AI APIs

✓ Retrieval-Augmented Generation (RAG)

✓ Vector Databases

✓ Prompt Engineering

✓ AI-powered REST APIs

✓ Secure enterprise integration with Azure/OpenAI

Why this matters?

Java already powers mission-critical enterprise systems.

Adding LLM capabilities enables:

Intelligent document search

AI copilots for internal platforms

Automated summaries & recommendations

Smart customer support

Natural language interfaces for legacy systems

The best part?

You don’t need to rebuild everything in Python. Java ecosystems are becoming AI-ready faster than ever.

Enterprise AI + Java = Scalable, secure, production-grade intelligence.

#Java #LLM #ArtificialIntelligence #SpringBoot #GenerativeAI #OpenAI #AzureAI #MachineLearning #SoftwareEngineering #BackendDevelopment #RAG #AIEngineering #CloudComputing #JavaDeveloper #TechInnovation

MEDIA document/carousel (possible)

#588 Rajesh Kumar

System Design

Engineering and Program Leader | 21+ Years | AI, SaaS & Product Delivery | Scaling High-Performance Teams | Driving Agile Transformation | Full Stack Observability

5mo · **View on LinkedIn**

Microservices Design Patterns: Building Scalable & Resilient Systems

Modern distributed systems are no longer just about splitting monoliths into smaller services. The real engineering challenge begins when services need to communicate reliably, recover gracefully, scale independently, and remain observable in production.

That’s where Microservices Design Patterns become critical.

Some of the most impactful patterns I’ve seen transform enterprise systems are:

• API Gateway Pattern → Centralized routing, authentication, throttling, and request aggregation

• Circuit Breaker Pattern → Prevents cascading failures during downstream outages

• Saga Pattern → Maintains distributed transaction consistency without tight coupling

• CQRS + Event Sourcing → Enables scalable reads, auditability, and event-driven architectures

• Service Discovery Pattern → Dynamic service registration and lookup in cloud-native systems

• Bulkhead Pattern → Isolates failures to protect system stability

• Strangler Fig Pattern → Incrementally modernizes legacy monoliths safely

• Sidecar Pattern → Adds observability, security, and networking capabilities without changing business code

In large-scale platforms like payroll, banking, payments, and trading systems, design patterns are not theoretical concepts — they directly influence:

✓ Reliability

✓ Latency

✓ Scalability

✓ Fault tolerance

✓ Deployment agility

✓ Operational visibility

The key lesson:

Microservices alone do not guarantee scalability or resilience. Well-implemented patterns do.

Successful architecture is less about choosing “microservices vs monolith” and more about choosing the right patterns for the right operational challenges.

#Microservices #SoftwareArchitecture #SystemDesign #CloudNative #DistributedSystems #EngineeringLeadership #Java #SpringBoot #Kubernetes #DevOps #EventDrivenArchitecture #Scalability #Observability

#589 Manish Kala Jain

Other

Engineering & Technology Leader | CTO | Head of Engineering | VP Engineering | AI, Platform Engineering, Digital Commerce, Payments & Cloud | Led 100+ Engineers | 100M+ Scale | Dubai Golden Visa | Immediately Available

5mo · View on LinkedIn

Visa vs Mastercard vs American Express ↓

The card networks — Visa, Mastercard, and American Express are the biggest players in the global payments landscape, but they operate differently.

Card Payment Models:

► 4-Party Card Network (Visa, Mastercard, GIE Cartes Bancaires)

This model involves consumers, issuers, merchants, & acquirers. Merchants interact with their customers and acquirers, while card networks serve as intermediaries.

► 3-Party Card Network (American Express, Discover Financial Services)

In this model, one entity acts as the issuer, acquirer, & network. Merchants now pay a single fee, often higher compared to the 4-party model.

—

Authorization Flow at Point of Sale (4-Party vs 3-Party Model)

→ 4-Party Model

► The Issuing Bank, Issuer, issues a debit/credit cards to its customer.

► The cardholder wants to buy a product and swipes the credit card at the Point of Sale (POS) terminal

► The POS terminal sends the transaction to the Acquirer that provided the terminal (a token is shared)

► The Acquirer sends the transaction to the card network who sends the transaction to the Issuer for approval

► The Issuer freezes the money if the transaction is approved. The approval or rejection is sent back to the acquirer, and POS. Funds are then transferred.

→ 3-Party Model — The first 3 steps are the same... but not step 4

► American Express and Discover Financial Services perform here the function of Acquirer, Issuer and Card Network. This is so called closed loop networks are more efficient with all functions processed in one place

► In recent years the closed loop networks have partnered with issuers and acquirers to scale their circulations

► The approval or rejection is sent back to the acquirer, then to the POS terminal. The funds are then transferred

—

Key Numbers:

► In 2023, U.S. merchants paid ~$224B in fees for accepting card payments (interchange, network, and processor fees combined).

► U.S. merchants would have saved $49B in 2023 if fees had remained at 2009 levels.

► In Europe, 45% of the Merchant Discount Rates (MDR) are attributable to interchange fees

—

Key Differences:

► Visa & Mastercard are payment networks, not issuers. They partner with banks and financial institutions to offer credit, debit, and prepaid cards.

► Both are widely accepted in over 200 countries. The main difference lies in their rewards and offers (depending on issuer).

► American Express is both a card issuer and a payment network. This gives AMEX more control over customer service and rewards programs, often making it a premium choice

► While AMEX is accepted in fewer locations than Visa and Mastercard, it offers exclusive perks for its users

#590 Ravindra Shinde Kafka / Messaging

Technical Lead at Mastercard | Payments Domain | AI Enthusiast

5mo · View on LinkedIn

Apache Kafka Architecture — Explained for Engineers

Most modern systems are event-driven.

Payments, notifications, analytics, fraud detection, logs, and microservices all generate continuous streams of events.

Apache Kafka acts as the backbone that moves this data reliably and in real time.

High-Level Kafka Architecture

Producers

Applications publish events into Kafka topics.

Topics

Logical channels that organize events (orders, payments, logs).

Partitions

Topics are split into partitions for:

* Parallelism
* Scalability
* Ordered processing within a partition

Brokers

Kafka servers that store and manage partitions.

Kafka Cluster

Multiple brokers working together for distributed storage and fault tolerance.

Consumers

Applications reading events from Kafka.

Consumer Groups

Consumers share partitions for horizontal scaling and failover.

How Data Flows

Producer → Topic → Partition → Broker → Consumer Group

Kafka persists events durably, allowing:

✓ Real-time processing
✓ Replay capability
✓ Async communication
✓ Loose coupling between services

Replication & Reliability

Each partition has:

* Leader Replica → Handles reads/writes
* Follower Replicas → Replicate data

If a broker fails, Kafka automatically promotes another replica.

This provides:
✓ High availability
✓ Fault tolerance
✓ Data durability

ACKS Configuration
* acks=0 → fastest, no guarantee
* acks=1 → leader acknowledgment
* acks=all → safest, all replicas confirm
Critical systems usually prefer acks=all.

Why Kafka Scales So Well
Kafka achieves massive scale because:
* Data is partitioned
* Consumers scale independently
* Sequential disk writes are optimized
* Brokers distribute workload horizontally
This enables millions of events per second with low latency.

Real-World Use Cases
Payment systems
Event-driven microservices
Real-time analytics
CDC pipelines
Log aggregation
IoT streaming
Fraud detection systems

Key Engineering Takeaway
Kafka is not just a messaging queue.
It is a distributed event streaming platform designed for scalable, fault-tolerant, real-time architectures.
Understanding Kafka architecture is becoming essential for:
* Backend Engineers
* Platform Engineers
* Data Engineers
* DevOps Engineers
* Cloud Architects
#ApacheKafka #Kafka #SystemDesign #DistributedSystems #Microservices #BackendEngineering #DataEngineering #EventDrivenArchitecture #SoftwareArchitecture #CloudComputing #DevOps #StreamingData

#591 Anshul Agarwal System Design

Software Engineer III | AI-Driven Architect ✓ | Designing enterprise-grade, scalable test frameworks → Driving modern transformation | Mentoring engineers to become AI-ready

5mo · **View on LinkedIn**

Most engineers prepare for System Design interviews by memorizing diagrams.
But real System Design understanding is not about remembering architectures.
It’s about understanding:
✓ why systems are designed a certain way

✓ tradeoffs
✓ scaling patterns
✓ bottlenecks
✓ reliability decisions
That’s the mindset interviewers actually look for.
Here’s a simple System Design interview roadmap I wish more engineers followed ↓
Step 1: Understand the Core Concepts First
Before jumping into “Design YouTube” or “Design Uber”…
learn the fundamentals:
✓ Load Balancer
✓ CDN
✓ API Gateway
✓ Caching (Redis)
✓ Databases (SQL vs NoSQL)
✓ Replication & Sharding
✓ Message Queues (Kafka/RabbitMQ)
✓ Rate Limiting
✓ Microservices
✓ CAP Theorem
✓ Scalability
✓ Fault Tolerance
✓ Observability
Without these concepts, large designs become confusing.
—
Step 2: Learn Tradeoffs
This is where interviews become interesting.
For example:
• Why use Kafka instead of REST?
• Why Redis for caching?
• Why NoSQL for feeds/search?
• Why eventual consistency?
• Why microservices over monolith?
There is rarely one “perfect” answer.
Good engineers explain tradeoffs.
—
Step 3: Understand Data Flow
One of the biggest mistakes people make:
They memorize components…
but don’t understand request flow.
Always ask:
✓ How does data enter?
✓ Where is it processed?
✓ Where is it stored?
✓ What happens during failures?
✓ What scales first?
System thinking matters more than fancy diagrams.

—
Step 4: Learn Real-World Architectures
Study systems like:
✓ Amazon
✓ Flipkart
✓ Netflix
✓ Uber
✓ WhatsApp
✓ YouTube
✓ Instagram
But focus on:
• scaling challenges
• bottlenecks
• caching strategies
• async processing
• reliability patterns
—
Step 5: Think Like a QA/SDET Too
This is underrated.
A strong engineer also thinks:
✓ How will this fail?
✓ What happens during retries?
✓ How do queues behave?
✓ What breaks during traffic spikes?
✓ How is observability handled?
✓ How will this system be tested?
That mindset makes your answers much stronger.
—
Common Interview Topics
Here are the concepts most frequently asked:
✓ Caching
✓ Rate Limiting
✓ Messaging Queues
✓ Database Scaling
✓ API Design
✓ CAP Theorem
✓ Consistency Models
✓ WebSockets
✓ Search Systems
✓ Notification Systems
✓ Event-Driven Architecture
✓ Distributed Systems Basics
✓ Retry & Idempotency
✓ High Availability
✓ Monitoring & Logging
—

One thing I realized recently:

AI can help generate system design diagrams now.

But interviews still test:

✓ thinking

✓ architecture understanding

✓ scalability reasoning

✓ tradeoff analysis

And that’s why fundamentals still win.

System Design is not about memorization.

It’s about learning how large systems think.

Follow for more such info : Anshul Agarwal

#SystemDesign #SoftwareEngineering #SDET #QA #Microservices #DistributedSystems #AI #Tech #Engineering #InterviewPreparation

#592 Arslan Ahmad

System Design

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

5mo · **View on LinkedIn**

Most candidates draw the same vague boxes-and-arrows for every system design question.

Interviewers notice.

The fix isn't memorizing more theory. It's this: have a small set of specific diagrams ready to draw on demand. So when someone says "design a feed" or "how do you handle traffic spikes," your hand already knows where to start.

Here are 6 worth burning into memory:

Universal Web Architecture — the default skeleton for almost any question

Microservices Architecture — when and how to split a system into services

Cache-Aside Flow — the read pattern behind most "make it faster" answers

Sharding with Consistent Hashing — how you scale data past one machine

Circuit Breaker — how a system fails gracefully instead of cascading

API Gateway — the single front door for auth, routing, and rate limiting

Practice is the whole trick. Draw each one until you can do it without thinking. In the interview, the diagram becomes your outline, and you just talk through it.

These are 6 of 20 in our full breakdown, which also covers write-through vs write-behind, fan-out on write vs read, CQRS, multi-AZ deployment, and more.

Full post in the comments. ↓

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview

#593 Aravind Kumar bysani

Java / Spring

"System Engineer at TCS| Experienced QA Manual & Automation Engineer | Expertise in Design of Test Approach, Test cases & Test execution, Test Scripts Agile | Banking Domain Expertise | Delivering Quality with precision"

5mo · **View on LinkedIn**

Java Interview Programs — Basic to Advanced With Code Examples

A complete Java interview preparation guide covering core programming concepts, OOPs, collections, multithreading, Java 8 features, algorithms, and coding interview programs with runnable examples.

What this document covers:

• Java basics including input handling, arithmetic operations, leap year logic, palindrome, Armstrong number, prime number, factorial, Fibonacci, and recursion programs (Pages 1–3)

• Pattern programs including pyramid, triangle, diamond, number, and alphabet patterns frequently asked in interviews (Pages 4–5)

• Array and matrix programs including sorting, reversing, duplicate detection, second largest element, missing number logic, and matrix operations (Pages 5–6)

• String-based interview programs including anagram checks, palindrome validation, substring search, duplicate removal, and non-repeated character logic (Pages 6–7)

• Core OOP concepts including classes, objects, constructors, inheritance, abstraction, interfaces, encapsulation, polymorphism, method overloading, and overriding (Pages 8–9)

• Exception handling concepts including try-catch-finally, throw/throws, and custom exceptions (Pages 9–10)

• File handling programs using Java IO for reading, writing, copying files, and word counting (Pages 10–11)

• Collections Framework programs using ArrayList, LinkedList, HashMap, TreeMap, HashSet, sorting, Set operations, and frequency counting (Pages 11–12)

• Searching and sorting algorithms including binary search, linear search, bubble sort, insertion sort, quick sort, and merge sort (Pages 13–14)

• Multithreading concepts including producer-consumer problem, deadlock, synchronization, inter-thread communication, and ExecutorService thread pools (Pages 15–16)

• Java 8 features including lambda expressions, functional interfaces, Streams API, method references, and Optional class usage (Pages 16–17)

• Mini project implementations including Library Management System, Banking System, Employee Management System, and Student Management System (Pages 17–18)

I’ll continue sharing high-value interview and reference content.

Follow me: https://lnkd.in/gAJ9-6w3

— Aravind Kumar Bysani

#Java #CoreJava #JavaInterview #CodingInterview #DSA #CollectionsFramework #Multithreading #Java8 #StreamsAPI #OOPs #Algorithms #Programming #Developers #InterviewPreparation #JavaDeveloper

java

.

18 pages

MEDIA advertisement; document/carousel (possible)

LINKS

- https://lnkd.in/gAJ9-6w3

#594 Parth Sarthi

System Design

Central Execution Trader @Graviton Research Capital LLP | Ex-SWE @Accenture | Ex-Intern @Salesforce | Python | Linux | SQL | AWS |

5mo · **View on LinkedIn**

Sharing My Recent Technical Interview Experience at Visa!

I recently went through an interview for the Software Engineer role at Visa (Post Purchase team), and it was one of the most well-rounded technical interviews I've experienced.

Instead of a standard Q&A, the conversation quickly transformed into a deep, hands-on architectural and coding session. Here are the core challenges we dove into:

• End-to-End API Orchestration & Backend Design:

Designing the complete backend architectural flow across controllers, services, and repositories for a real-world CRUD use case, while thinking through optimization strategies like pagination and caching to handle database operations at scale.

• Declarative GitOps Architecture & Pod Lifecycles:

Going beyond surface-level tool knowledge to explain how declarative GitOps systems reconcile cluster states under the hood, the internal mechanics of Kubernetes control planes, and drafting multi-resource manifests that enable secure internal microservice communication.

• Live coding under pressure:

Solving a non-trivial set operations problem, with iterative feedback from the interviewer on input handling, code structure, and execution, a great test of how you think out loud under pressure.

Beyond the technical questions, the conversation touched on event-driven architecture, the difference between synchronous and asynchronous microservice communication, cloud migration, and AI-assisted development tools.

Huge thanks to the interviewers for the sharp questions and great energy.

#Visa #SoftwareEngineering #InterviewExperience #BackendDevelopment #CareerGrowth #TechInterviews

#595 Neo Kim Cloud / K8s / DevOps

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter

5mo · **View on LinkedIn**

How to get ahead of 99% of people at deployments

Learn these 5 patterns:

1 Big Bang:

→ Deploy on all servers at once, replacing old version.

→ Simple to set up.

→ But high risk of downtime and failures.

===

2 Rolling:

→ Slowly update servers in batches.

→ Zero downtime & reduced risk.

→ But rollbacks might become complex mid-way.

===

3 Blue Green:

→ Maintain 2 environments & switch traffic between them.

→ Instant rollback and zero downtime.

→ But it increases infrastructure costs and complexity.

===

4 Canary:

→ Deploy to a few users first, and slowly increase traffic with monitoring.

→ Reduces the failure impact.

→ But it increases complexity with advanced monitoring & traffic control.

===

5 Feature Toggle:

→ Hide new features behind conditional code logic.

→ Separate deployment from release and easy to rollback.

→ But it adds code complexity and tech debt.

What else should make this list?

===
PS - Want my System Design Playbook (for free)?
Join my newsletter with 200K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Save & repost to help others learn deployment patterns.
Follow Neo Kim + turn on notifications.

MEDIA deployment patterns for senior software engineers

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#596 Sami Najim

AI / LLM

AI Agents & AI Automation | Personal Brand | Content Creator | CE + AI @ UofT | Prev @ Ecobee
5mo · **View on LinkedIn**

BREAKING: Everyone wants to learn AI Agents in 2026...
But 99.9% are learning the skills in the wrong order.
Here's the complete AI Agent Roadmap from zero to production ↓
The truth?
Most people still can't tell the difference between an AI Agent and a fancy chatbot.
And in 2026, that confusion is getting more expensive, not less.
Here's the clearest breakdown you'll find today:
Become better at AI in 5 minutes a day
Join our AI community with other business leaders to stay ahead of AI
Sign Up & Get Started for Free here → https://lnkd.in/e9tUTaMW
The Original post:

_____
These are NOT Agentic AI:
1. LLM Chatbots
- Advanced calculators for text. You ask → they answer.
- No planning, no dynamic tool use, limited adaptability.
2. RPA (Robotic Process Automation)
- Scripted bots in fixed sequences. Works for repetitive tasks, breaks on anything unexpected.
3. Simple RAG (Retrieval-Augmented Generation)
- Pulls info from a database or web, feeds it to an LLM.
- Gives answers, not strategies. No multi-step planning.
- Note: Agentic RAG — which plans and self-corrects — is a different story.
This IS Agentic AI:
- Remember context: short-term for current tasks, long-term for scheduled ones.
- Plans: breaks goals into smaller tasks using prompt engineering or reasoning models.
- Uses and schedules tools dynamically based on your prompt.
- Self-improves via feedback loops like ReACT and Reflexion.
- Collaborates in multi-agent teams where each agent specializes.
- Communicates across agents using MCP & A2A Protocol, the new interoperability standards.
- Operates with human-in-the-loop oversight, what separates pilots from production.

The terminology is noisy. The fundamentals aren't. Now you know the difference.

Save → React → Comment your thoughts below

Repost to share with your community

Also Credit to Rakesh G. Follow him for more.

& follow for everything related to AI Agents

LINKS

- https://lnkd.in/e9tUTaMW

#597 DataLinkk

AI / LLM

5mo · View on LinkedIn

Choosing the right workflow orchestration framework is becoming one of the most important decisions in modern AI system design.

At DataLinkk, we've been deeply evaluating both n8n and LangGraph in real-world AI engineering environments — and the distinction goes far beyond "visual vs code."

The real question is:

How do you design workflows that are not only automated — but also intelligent, stateful, scalable, and production-ready?

Here's how we see the ecosystem evolving:

• n8n

A powerful low-code automation platform designed for connecting APIs, services, and operational workflows with speed and simplicity.

• LangGraph

A graph-based orchestration framework built for stateful AI agents, cyclical reasoning, and multi-step LLM workflows with full Python control.

What excites us most is this shift:

AI workflows are moving from linear automation → to dynamic, reasoning-driven orchestration systems.

We're currently deep into:

→ Hybrid Architectures (n8n + LangGraph)

Using n8n for operational automation while LangGraph powers intelligent reasoning loops.

→ Stateful Agent Workflows

Building cyclical retrieval-reasoning agents with memory, checkpointing, and conditional execution.

→ Agentic RAG Pipelines

Combining retrieval, tool use, and reasoning into iterative AI systems.

→ LangGraph + Claude Integrations

Exploring structured MCP tool calling and multi-agent coordination patterns.

→ Workflow Evaluation Frameworks

Benchmarking latency, reliability, observability, and cost efficiency across orchestration stacks.

→ Automation at Scale with n8n

Integrating Slack, Notion, CRMs, databases, and internal APIs without heavy engineering overhead.

Key insight from our engineering exploration:

"n8n moves fast on operations; LangGraph thinks deeply on reasoning — the real power comes from knowing when to use each, and when to combine both."

If you're working in:

→ AI Engineering

→ Workflow Automation

→ Agentic AI Systems
→ RAG Pipelines
→ LLM Infrastructure
→ AI Orchestration Frameworks
Let’s connect and exchange insights on building production-grade AI systems.
Open to collaborations, architecture discussions, and enterprise AI deployments.
#n8n #LangGraph #AIEngineering #WorkflowAutomation #AgenticAI #RAG #LLM #LangChain #Python #Automation #GenerativeAI #DataLinkk
n8n VS LangGraph
.
3 pages

MEDIA document/carousel (possible)

#598 Docker Professionals Network Container Infrastructure Architects Developers DevOps Engineers

Other

Hemant Sawant • 3rd+
5mo · **View on LinkedIn**

https://lnkd.in/dxQSkx5a

MEDIA advertisement

LINKS
- https://lnkd.in/dxQSkx5a

#599 Alex Xu

System Design

Author of 4 Bestselling Books | Co-Founder of ByteByteGo
5mo · **View on LinkedIn**

Forward Proxy, Reverse Proxy, and API Gateway Explained
People mix these up all the time, since they all sit between a client and a server. The real difference is which side they represent and what problem they solve.
A forward proxy sits next to the client. Your laptop sends a request, the proxy forwards it out, and the destination never sees your real IP. Corporate networks use this to enforce policy, block sites, and cache traffic.
A reverse proxy sits next to the server. The client has no idea how many machines are behind it. The proxy decides who handles the request, terminates TLS, and keeps your backend off the public internet. NGINX and HAProxy are commonly used here, typically paired with a load balancer in front.
An API gateway is a reverse proxy that does more than route traffic. It also handles auth, rate limits, API keys, versioning, and request shaping. Without it, each microservice has to implement its own version of validation, throttling logic, and request logging.
A forward proxy represents the client, a reverse proxy represents the server, and an API gateway is what you add when ten services need the same authentication and rate limiting rules applied consistently.
In most real systems, all three are running at different layers. The forward proxy filters outbound traffic, the reverse proxy fronts the application servers, and the API gateway sits in front of your APIs to enforce policies before requests reach them.
Over to you: What's your proxy + gateway combo? Always interesting to see what teams pair together.
--

We just launched the Build with Claude Code cohort-based course. Check it out here: https://lnkd.in/g-6TScjm

#systemdesign #coding #interviewtips

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/g-6TScjm

#600 Ravena O

AI / LLM

AI Researcher and Data Leader | Healthcare Data | GenAI | Driving Business Growth | Data Science Consultant | Data Strategy

5mo · **View on LinkedIn**

Integrating Agentic RAG Systems with MCP ↓

As RAG systems evolve, retrieval is no longer just about fetching data, it's about making intelligent decisions on what data to retrieve and when.

That's where MCP becomes a key layer in Agentic RAG architectures.

How the flow works:

1. A query first goes through an LLM-powered agent for analysis.

The agent can:

• Rewrite the query for better retrieval

• Generate multiple queries if needed

• Decide whether external or additional data sources are required

2. If more context is needed, retrieval is triggered across:

• Real-time user data

• Internal enterprise documents

• Web data

• Other connected knowledge sources

This is where MCP adds real value:

✓ Each domain can manage its own MCP server with domain-specific rules

✓ Security and compliance stay isolated at the server level

✓ New data domains can be plugged in without rewriting the agent layer

✓ Standardized exposure of data enables scalable integrations

✓ AI engineers can focus on agent orchestration instead of custom integrations

3. Retrieved information is consolidated and reranked using stronger reasoning models to improve relevance.

4. If no external retrieval is required, the agent can directly generate responses or actions through the LLM.

5. The final response is evaluated for quality and relevance:

• If sufficient → returned to the user

• If not → the query is refined and the loop repeats

MCP is quickly becoming an important building block for scalable and modular Agentic RAG systems.

Are you integrating MCP into your AI stack yet?

CC: Aurimas

MEDIA document/carousel (possible)

#601 Shameer Ahamad

Java / Spring

Senior Java Developer @ Wipro | Java | Spring Boot | REST API | Microservices | GCP | Kafka | Docker | SQL | NoSQL | JUnit | Mockito | GenAI | IoT

5mo · **View on LinkedIn**

Deadlock in Java — One of the Silent Killers in Multithreading

Your application is running fine...

Suddenly it hangs forever

No errors. No logs. Just stuck.

→ That's a Deadlock.

What is Deadlock?

A situation where two or more threads are waiting on each other indefinitely.

• Thread 1 holds Resource A → waiting for B

• Thread 2 holds Resource B → waiting for A

Result: Both are blocked forever

How It Happens

1. Thread 1 locks Resource A

2. Thread 2 locks Resource B

3. Both try to acquire each other's resource

No progress → Deadlock

(!) Why It's Dangerous

• Application freeze (no response)

• Hard to debug in production

• Impacts performance & scalability

How to Prevent Deadlocks

✓ Maintain consistent lock ordering

✓ Use tryLock() with timeout

✓ Avoid nested synchronized blocks

✓ Minimize shared resources

✓ Use higher-level concurrency APIs (Executors, Locks)

#Java #Multithreading #Concurrency

#JavaDeveloper #BackendDevelopment

#SoftwareEngineering #InterviewPrep

#Threading #Deadlock #Programming

#602 Tanmay Khilari

Java / Spring

Java Developer | Spring Boot | Microservices | REST APIs | SQL | GraphQL | AWS | Docker | Kotlin | 2 YOE | Immediate Joiner

5mo · **View on LinkedIn**

Heap vs Stack in Java Memory Management

Understanding where Java stores data internally helps you write cleaner, more efficient, and optimized applications.

Stack Memory

Stack stores:

1. Method calls

2. Local variables

3. Function execution frames

Each thread gets its own separate stack memory.

public void test() {
int x = 10; // stored in Stack
}
✓ Very fast access
✓ Automatically cleared after method execution
✗ Limited memory size
Heap Memory
Heap stores:
1. Objects
2. Instance variables
3. Arrays
Objects created using new keyword are stored in Heap.
Student s = new Student();
Here:
s → reference stored in Stack
new Student() → object stored in Heap
✓ Shared across threads
✓ Larger memory area
✓ Heap memory is managed by the Garbage Collector
✗ Comparatively slower access
Remember this
1. Primitive local variables, Method frames → Stack
2. Objects, Instance Variables and Arrays → Heap
3. Object references → Stack (actual object lives in Heap)
→ If you are preparing for Java backend interviews, connect & follow - I share short, practical backend concepts regularly.
#Java #SpringBoot #BackendDevelopment #JavaDeveloper #Programming #SoftwareEngineering #JVM #CodingInterview

#603 AlgoMaster.io

Other

Visit website
5mo · **View on LinkedIn**

14 must-know Data Structures for coding interviews:
1. Array
2. Queue
3. Deque
4. Matrix
5. Stack
6. Binary Tree
7. Linked List
8. Doubly Linked List
9. HashMap
10. Binary Search Tree (BST)
11. Heap (Priority Queue)
12. Trie
13. Graph

14. Union Find

If you want to master DSA and coding interviews, check out algomaster.io for comprehensive, pattern-focused resources.

Repost to help others in your network

LINKS

- https://algomaster.io/
- http://algomaster.io

#604 Subhash Kumar

Other

MBA Finance | CIA | PGDRB From Manipal University Bangalore| Content Creator , Open for collabs

5mo · View on LinkedIn

The best things in life are free.

But in the world of finance, "free" usually has a hidden price tag.

Except for UPI.

India’s Unified Payments Interface is currently processing over ₹300 Trillion annually.

That is more than the GDP of many developed nations.

And for the 300 million+ people using it?

The cost is exactly ₹0. No convenience fees.

No hidden processing charges.

No "platform maintenance" tax.

So, if you aren't paying for it, who is?

Here is the "Aha!" moment most people miss:

The Government Footing the Bill: To keep India "cashless," the government pays thousands of crores in subsidies to banks to offset their losses.

The Data Play: Apps like PhonePe and Google Pay use your transaction history (anonymously) to offer you loans, insurance, and credit cards.

That’s where the real money is.

The Efficiency Gain: By reducing the need for physical cash, the RBI saves billions on printing, transporting, and securing paper money.

UPI isn't just a payment tool.

It’s a massive social experiment in financial inclusion.

We’ve moved from "Chai-wala only takes cash" to "Scan the QR for ₹5" in less than a decade.

It’s proof that when you remove friction, you unlock an entire economy.

India has built the world’s most advanced digital rail.

And we're just getting started.

Do you think UPI should stay free forever, or should there be a small fee for high-value transactions to make it sustainable?

Let’s discuss in the comments. ↓

#UPI #DigitalIndia #Fintech #IndianEconomy #FutureOfPayments #FinancialInclusion #TechRevolution

#605 Ashish Pratap Singh

System Design

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

5mo · View on LinkedIn

If you want to level up as a backend engineer, consider reading these 7 books:

1) Clean Code

2) Head First Design Patterns

3) Designing Data-Intensive Applications
4) Fundamentals of Data Engineering
5) Database Internals
6) Building Microservices
7) Head First Software Architecture
Which other book would you add to this list?
Repost to help others in your network
→ If you are preparing for Software Engineering Interviews, checkout algomaster.io for pattern-based resources.

LINKS
- https://algomaster.io/lnv
- http://algomaster.io

#606 Java Developers Community (moderated)

Java / Spring

Arvind Kumar • 3rd+
5mo · **View on LinkedIn**

Most explanations of Java Virtual Threads stop at:
“Virtual Threads are lightweight threads.”
That barely explains what actually changed inside the JVM.
Traditional Java threads follow:
1 Java Thread = 1 OS Thread
Each platform thread gets:
▸ native stack memory
▸ kernel scheduling
▸ dedicated OS resources
Which makes threads expensive.
A single platform thread may reserve ~1 MB stack memory.
So:
100k requests -> 100k platform threads
can quickly create:
▸ memory pressure
▸ context switching overhead
▸ scheduler contention
Virtual Threads completely change this model.
Internally:
Many Virtual Threads
↓
Few Carrier Threads (OS Threads)
But Virtual Threads are NOT just “smaller threads”.
They are implemented using Continuations.
A continuation is essentially:
A paused computation that can resume later
When a Virtual Thread blocks on:
▸ database calls
▸ HTTP requests
▸ socket I/O

▸ sleep()
the JVM:
captures thread state
stores stack frames into heap-based stack chunks
unmounts the Virtual Thread
releases the carrier thread
So:
Blocked request != Blocked OS thread
This is the core scalability breakthrough of Project Loom.
Another major difference:
Platform thread stacks: Contiguous native memory
Virtual thread stacks: Heap-based stack chunks
This allows millions of waiting threads with far lower memory overhead.
Another important evolution:
In Java 21–23, blocking inside synchronized sections could pin carrier threads.
Java 24 fixed most of this via JEP 491 by making JVM monitors virtual-thread aware.
Now, in most synchronized blocking scenarios:
▸ Virtual Threads can unmount
▸ carrier threads can be reused
▸ scalability is preserved
Some pinning scenarios still exist:
* JNI/native calls
* foreign function boundaries
because native execution cannot always be safely suspended.
Also:
Virtual Threads do NOT improve CPU-bound performance.
They primarily optimize:
▸ blocking workloads
▸ concurrency efficiency
▸ memory usage
▸ scalability
The fascinating part is:
developers still write simple blocking code while the JVM transparently handles suspension, scheduling, continuation capture, and remounting underneath.
Project Loom is one of the deepest JVM runtime changes in years.
#Java #JVM #VirtualThreads #ProjectLoom #Java24 #BackendDevelopment #Concurrency #SoftwareEngineering #codefarm

#607 Arslan Ahmad Cloud / K8s / DevOps

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

5mo · **View on LinkedIn**

The "Sidecar" Pattern.
Your application code should do one thing: business logic.
It shouldn't know how to handle retries, circuit breaking, TLS termination, logging, tracing, or rate limiting.

The sidecar pattern puts all that cross-cutting infrastructure into a separate process that runs alongside your application.

Same server. Different container.

Your app sends a request to the sidecar. The sidecar handles encryption, retries, load balancing, and observability.

Your app doesn't even know it's happening.

This is exactly how Istio and Envoy work.

Every service in your mesh gets a sidecar proxy.

Zero changes to application code.

Full infrastructure control.

In interviews, when you mention service meshes, explaining the sidecar pattern shows you actually understand how they work, not just what they do.

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview

#608 Alok Kumar AI / LLM

Principal Engineer | Startup Product Builder | Java Technical Architect | Distributed Systems | Cloud-Native Engineering | AI/GenAI Integration | Tech Lead

5mo · **View on LinkedIn**

Designing a Modern AI-Powered Daily Activity Tracker using Java, Spring Boot & RAG Architecture

Recently explored an enterprise-grade architecture for building a scalable SaaS platform combining:

Java 21

Spring Boot Microservices

ChatGPT/OpenAI

RAG (Retrieval-Augmented Generation)

Redis Cache

PostgreSQL + Vector Database

Mobile & Web UI

High-Level Architecture Flow

Mobile/Web UI

↓

API Gateway

↓

Spring Boot Microservices

↓

RAG + ChatGPT Integration

↓

Redis Cache + PostgreSQL + Vector DB

Key Microservices

User Service

Activity Tracking Service

Goal & Task Service

Notification Service

Analytics Service

AI Assistant Service

How AI/RAG Works

1. User asks question from mobile/web
2. AI Assistant retrieves relevant data from Vector DB
3. Builds contextual prompt
4. Sends to ChatGPT/OpenAI
5. Returns intelligent response

This helps reduce hallucination and provides context-aware AI responses.

Technology Stack

Spring Boot 3 + Java 21

Spring Cloud Gateway

Redis Cache

PostgreSQL

pgvector / Vector DB

RabbitMQ

Docker & Kubernetes

OpenAI / Azure OpenAI

React / Mobile UI

Key Benefits

✓ Scalable microservices architecture

✓ AI-powered SaaS platform

✓ Fast response with Redis caching

✓ Context-aware AI assistant using RAG

✓ Event-driven processing with RabbitMQ

✓ Cloud-native & production-ready architecture

The future of enterprise applications is moving toward: → AI-integrated microservices

→ Intelligent SaaS platforms

→ Context-aware enterprise systems

#Java #SpringBoot #Microservices #RAG #ChatGPT #AI #GenerativeAI #SoftwareArchitecture #JavaDeveloper #TechLead #Redis #PostgreSQL #Kubernetes #OpenAI #SaaS

#609 Narendra Sahoo

Java / Spring

Senior Technical Lead @ HCL | Java • Spring Boot • Microservices • Kafka | AWS & Cloud-Native Architectures | AI Integration (LLM & RAG) | Tech Mentor | YouTuber @ Naren Java Help Line

5mo · **View on LinkedIn**

Most developers learn Java syntax.

Few learn how to write scalable software.

That’s where SOLID principles change everything.

S → Single Responsibility

One class = One job

O → Open/Closed

Extend code without breaking existing logic

L → Liskov Substitution

Child classes should behave correctly as parents

I → Interface Segregation

Don’t force unnecessary implementations

D → Dependency Inversion

Depend on abstractions, not concrete classes

SOLID is the reason great systems stay maintainable for years.

And yes...

These concepts are heavily asked in Java, Spring Boot, and System Design interviews.

Don’t just memorize definitions.

Understand the real-world use cases.

That’s what separates a coder from a software engineer.

Repost to help developers grow

Follow Narendra Sahoo for Java, Spring Boot, Microservices & Interview Preparation

#Java #SOLIDPrinciples #SpringBoot #Microservices #SystemDesign #BackendDevelopment #SoftwareEngineering #Programming #Coding #JavaDeveloper #Tech #Developers #InterviewPreparation #CleanCode #SoftwareDeveloper

#610 Java Developers Community (moderated)

Java / Spring

Alexander Sidorov • 3rd+

5mo · View on LinkedIn

Ok, bust the myth right away - virtual threads are NOT a magic "make it faster" button. Sorry.

But damn, they are the most practical thing Java got in years. Here's the deal: they shine when your service spends most of its life waiting. Database calls, HTTP clients, message brokers, file I/O, slow third-party APIs - you name it. Old model? Wastes platform threads while everyone just sits there. Virtual threads let you keep that sweet thread-per-request style but scale like crazy with less memory and zero thread-pool tuning headaches.

Where do they suck? CPU-heavy stuff. JSON parsing, encryption, image processing, sorting huge datasets - anything that actually burns CPU. Virtual threads won't magically give you more cores. You still need profiling, better algorithms, batching, back pressure, and concurrency limits.

Mental model: virtual threads help you wait better, not compute faster.

Also - they don't fix bad architecture. Blocking calls become cheaper, not free. Databases still have connection limits. External services still fail. Too much concurrency? Congrats, you just melted your own system.

Use them when they simplify code and boost throughput for I/O-bound work. Don't use them as an excuse to stop measuring.

That's it. No magic, just solid engineering.

#java #softwareengineering #programming #backenddevelopment #webdevelopment

#611 Java Developers Community (moderated)

Java / Spring

Ahmet Emre Demirşen • 3rd+

5mo · View on LinkedIn

Spring Boot Developers: Stop writing response wrapper code for every endpoint!

After years of writing the same response wrapper code across Spring Boot projects, I got tired and built Spring Response Envelope. Just add @ResponseEnvelope annotation and get standardized API responses with pagination, error handling, request ID propagation and metrics integration automatically. Available on Maven Central, set up in 5 minutes!

Table Of Contents

1. The Problem with Inconsistent API Responses
2. Quick Start: Your First Wrapped Response
3. Advanced Features: Pagination, Error Handling, and Metrics
4. Configuration and Customization
5. Real-World Use Cases

Get this maven dependency here: https://lnkd.in/eMEgrAuK

Read my article here: https://lnkd.in/dbtk79bz

#SpringBoot #Java #API #Microservices #OpenSource

MEDIA Java Spring Boot Generic Response, Response Envelope, API; advertisement

LINKS

- https://lnkd.in/eMEgrAuK
- https://lnkd.in/dbtk79bz

#612 Java Developers Community (moderated)

Kafka / Messaging

Ivan Franchin • 3rd+

5mo · **View on LinkedIn**

Solace PubSub+ vs Kafka: Different Approaches to Topic Routing

Comparing topic routing: how Solace’s hierarchical wildcards differ from Kafka’s flat topic model

Link: https://lnkd.in/dciQWPJA

Not a Medium member? Read it for free using my friend link: https://lnkd.in/dF6bVQmg

#Solace #SolacePupSub #Kafka #EventDrivenArchitecture #MessageBroker #Java #SpringBoot #SpringCloudStream #TopicRouting

MEDIA advertisement

LINKS

- https://lnkd.in/dciQWPJA
- https://lnkd.in/dF6bVQmg

#613 Reshma Sriraman

AI / LLM

Director of AI products | GTM | 10M+ Impressions |Helped 5K+ Students professional to Build Next-Gen AI Agents 50k+ community| Mentor Empowering Students & IT Pros | Passionate About Smart, Scalable SystemsD | View my services

5mo · **View on LinkedIn**

The Complete Agentic AI Stack Every Developer Must Know in 2026

Most people think AI is just ChatGPT.

That mindset is already outdated.

We are moving from:

“AI answering questions”

to:

“AI systems that can reason, plan, use tools, automate workflows, remember context, communicate in voice, and collaborate autonomously.”

The future of software is no longer static.

It’s agentic. ↓

---

1. FOUNDATION MODELS

The intelligence layer powering modern AI.

Examples:

→ GPT-4o

→ Claude

→ Gemini

→ Llama 3

→ Mistral

These models enable:
• reasoning
• coding
• planning
• conversations
• multimodal understanding

---

2. LIGHTWEIGHT MODELS

Smaller models like:
→ Phi-3
→ TinyLlama
→ Hermes
are enabling:
• edge AI
• on-device inference
• low-latency systems
• cost-efficient deployments

---

3. GENERATIVE AI TOOLS

Text:
→ ChatGPT
→ Claude
→ Gemini
Image:
→ Midjourney
→ DALL·E 3
→ Stable Diffusion
Video:
→ Sora
→ Runway
→ Pika
Voice:
→ LuMay.ai
→ ElevenLabs
→ PlayHT
→ Suno AI
AI is now multimodal:
text + image + video + voice together.

---

4. AGENTIC FRAMEWORKS

Frameworks like:
→ LangGraph
→ CrewAI

→ AutoGen
help build agents that can:
• think step-by-step
• use tools
• call APIs
• browse the web
• execute workflows
• collaborate with other agents
This is the shift from:
“AI assistant”
to:
“AI workforce.”

---

5. OPEN-SOURCE AGENT ECOSYSTEM

Projects like:
→ MetaGPT
→ Flowise
→ BabyAGI
→ Camel-AI
are accelerating autonomous AI development rapidly.

WHAT’S HAPPENING NOW?

The industry is moving from:
✗ Simple prompting
To:
✓ AI systems with:
• memory
• reasoning
• tool calling
• voice interaction
• long-term context
• multi-agent collaboration
The next generation of engineers will not just write code.
They will design intelligent systems.

SKILLS THAT WILL MATTER IN 2026

• AI Agents
• RAG Systems
• Vector Databases
• LangGraph / CrewAI
• Voice AI
• OpenAI APIs
• Multimodal AI

• AI Automation
• Fine-Tuning
• AI System Design

We are still early.

Which part of the AI stack are you exploring the most?

Follow Reshma Sriraman || #reshmawithai for more AI insights

Repost if you’re building serious AI systems in 2026 to help others learn.

#AI #GenAI #LLM #AIAgents #ArtificialIntelligence #MachineLearning #OpenAI #LangGraph #CrewAI #Automation #AIEngineering

LINKS

• http://LuMay.ai

#614 Ashish Pratap Singh

Other

Founder @ AlgoMaster.io | YouTube (275k+) | Prev @ Amazon

5mo · **View on LinkedIn**

10 Golden Rules to Write Clean Code:

1. Avoid Magic Numbers and Strings

Hard-coded literals hide intent. Instead of writing 7, ADMIN, or ACTIVE directly in multiple places, extract them into named constants or enums.

2. Use Meaningful Names

A good name tells you what something represents and why it exists. If you need a comment to explain a variable, the name probably needs improvement.

3. Favor Early Returns Over Deep Nesting

Handle invalid cases upfront. This keeps the main logic flat, readable, and easier to reason about.

4. Avoid Long Parameter Lists

Too many arguments usually mean the function is doing too much or the data belongs together. Wrap related values into a single object, record or config.

5. Keep Functions Small and Focused

A function should do one thing well. If you can’t summarize it in one sentence, it may be time to split it.

6. Keep Code DRY

Don’t Repeat Yourself.

Repeated logic often leads to repeated bugs. If the same logic appears in multiple places, extract it into a reusable function, class, or module.

7. Apply KISS Principle

Keep it Simple Stupid.

Prefer simple, obvious solutions over clever abstractions.

If someone can’t understand it quickly, it isn’t simple.

8. Prefer Composition Over Inheritance

Inheritance creates tight coupling when overused. Use it only when there is a true “is-a” relationship. For flexible behavior, composition is often cleaner and easier to change.

9. Comment Only When Necessary

Good code explains what. Good comments explain why. If a comment only repeats the code, the code probably needs refactoring.

10. Write Good Commit Messages

Commit messages document your project’s history. Avoid vague messages like fix bug or update code. A good commit message clearly explains what changed and why it changed.

What other clean code rule would you add to this list?

Repost to help others learn these clean code rules.

→ If you are preparing for Software Engineering Interviews, checkout my website algomaster.io for pattern-based learning resources.

MEDIA document/carousel (possible)

LINKS

- https://algomaster.io/lnv
- http://algomaster.io

#615 Priyanka Logani

System Design

Senior Full Stack Engineer | Java 17 • Spring Boot •.NET Core • Microservices • Kafka • Angular | AWS • Azure • GCP | Cloud-Native Architecture • CI/CD • Kubernetes • Event-Driven Platforms • APIs | LLMs

5mo · **View on LinkedIn**

Optimistic locking vs pessimistic locking

Two users update the same database row at the same time.

What should happen?

This is one of the classic concurrency problems in backend systems, and most applications solve it using either optimistic locking or pessimistic locking.

With optimistic locking, the system assumes conflicts are rare.

Both transactions read the same row without locking it.

Each row carries a version number.

When an update happens, the database checks:

“Is the version you read still the latest version?”

If another transaction already updated the row and changed the version, the update fails with a version mismatch.

No blocking.

High concurrency.

Great for read-heavy systems.

With pessimistic locking, the system assumes conflicts are likely.

The first transaction acquires a lock on the row, and every other transaction must wait until the lock is released.

This prevents conflicting writes upfront, but it can reduce throughput and increase the risk of deadlocks under heavy contention.

A simple way to think about it:

• Optimistic locking → “Conflicts probably won’t happen.”

• Pessimistic locking → “Conflicts will happen, so block them early.”

In production systems:

✓ Optimistic locking works well for:

read-heavy applications

low write contention

scalable distributed systems

✓ Pessimistic locking works well for:

financial transactions

inventory management

high-conflict write scenarios

systems where consistency matters more than throughput

Choosing the wrong strategy can quietly become a scalability bottleneck.

The interesting part is that many large-scale systems end up using both depending on the business operation.

Have you ever dealt with deadlocks or version-conflict issues in production systems?

#SystemDesign #BackendDevelopment #Java #SpringBoot #Microservices #Database #SoftwareEngineering #SeniorFullStackDeveloper #C2C #DistributedSystems #PostgreSQL #MySQL #Engineering

MEDIA advertisement

#616 Ved Vekhande

AI / LLM

I (actually) build AI Agents & Workflows | open for freelance & collabs

5mo · **View on LinkedIn**

Someone built an AI from scratch.

And (literally) put it on Github for free.

This book's code repo teaches you to build Chatgpt like llm from zero:

Ch 2 → Working with Text Data

- Tokenization from absolute scratch
- How AI reads your words

Ch 3 → Attention Mechanisms

- The magic behind every LLM
- Multi-head attention, coded clean

Ch 4 → Build a GPT Model

- Actual GPT architecture, from scratch
- Not a wrapper. The real thing.

Ch 5 → Pretrain on Real Data

- Train your own model
- Load weights from bigger models

Ch 6 → Finetune for Classification

- Spam detection, sentiment, labels
- Your model, your task

Ch 7 → Instruction Finetuning

- Make it follow your commands
- The secret behind ChatGPT behavior

Bonus material inside:

- Llama 3.2 from scratch
- Qwen3 architecture walkthrough
- Gemma 3 implementation
- KV Cache explained with code
- Direct Preference Optimization (DPO)
- LoRA for efficient finetuning

Repo: https://lnkd.in/dMRSvPuJ

Save this post.

Repost for anyone learning AI.

Check my profile for more free resources

LINKS

- https://lnkd.in/dMRSvPuJ

#617 Technical Spark

Other

5mo · View on LinkedIn

Apple Pay vs Google Pay: Which is More Secure?

How Apple Pay Works

Your card info is stored securely on your device’s chip. Apple sends it to the bank, which creates a Device Account Number (DAN). Payments use the DAN, keeping your card details private and never shared.

How Google Pay Works

Your card info is sent to Google’s servers, where a payment token is generated. The token is sent to your phone, then to the merchant, and back to Google. Google decrypts it and sends your card info to the bank.

Google Pay stores your card info and tracks every transaction. Apple Pay doesn’t store or share your details, making it a more secure and private option.

Credit: Chirag Goswami

#Cybersecurity #ApplePay #GooglePay

#618 Ana Pedra

AI / LLM

AWSx15 · Azurex14 · GCPx7 · NVIDIAx4 · Red Hatx2 | Golden Kubestronaut | #2 Cloud DevOps LinkedIn Female Creator (Favikon) | AI Cloud DevOps Advisor | Helping Professionals Break into AI, Cloud & DevOps

5mo · View on LinkedIn

Most people still think AI = ChatGPT answering questions

But the real shift is happening somewhere else:

- Traditional AI → predicts
- Agentic AI → acts
- Agentic RAG → reasons + acts + learns

This changes EVERYTHING.

Traditional AI workflows were linear:

Collect data → train model → deploy → generate output.

Useful...

But mostly static.

Now look at Agentic AI:

- AI systems can use tools
- Call APIs
- Make decisions
- Execute actions
- Iterate autonomously

And then comes Agentic RAG, the next evolution.

Instead of relying only on training data, these systems:

- Retrieve fresh information
- Design multi-step workflows
- Maintain memory
- Verify outputs
- Adapt from previous interactions
- Continuously improve decisions

This is why AI agents are exploding right now.

We are moving from:
“AI that answers”
to
“AI that operates.”
The winners in the next 5 years won’t just build models.
They’ll build intelligent systems that can:
reason, retrieve, decide, and execute in real environments.
That’s the future of enterprise AI.
And honestly?
Most companies are still not ready for what’s coming.
#AI #AgenticAI #RAG #GenerativeAI #MachineLearning #LLM #ArtificialIntelligence #Tech

#619 Ashish Pratap Singh Other

Passionate Payments Professional | Writes to 30+ | Java Backend Developer | Exploring SWIFT, RTGS, RTP & ISO 20022

5mo · **View on LinkedIn**

Most people think money moves instantly when they swipe a card. It doesn’t.
In card payments, there are actually TWO important stages:-
1. Authorization
2. Settlement
And many people in tech confuse
Easiest Explanation
-> Authorization = “Can this payment happen?”
When you tap your card:-
- Merchant sends payment request
- Acquirer forwards it to Visa/Mastercard
- Network sends it to Issuing Bank
- Issuer checks:
✓ Balance / credit limit
✓ Fraud risk
✓ Card validity
✓ PIN/OTP/CVV
If everything looks fine:-
→ Transaction gets APPROVED
But Remember at this point, Money is NOT fully transferred yet.
The issuer only places a temporary hold on funds.
-> Settlement = “Now move the actual money”
After authorization:-
- Merchant submits approved transactions in batches
- Networks calculate obligations
- Funds move between banks
- Merchant finally receives money
This process may take:-
- A few hours
- T+1 day

- Sometimes longer depending on the bank/network
Simple example↓
You order food online for ₹500.
Authorization:
→ Bank checks whether you can pay ₹500.
Settlement:
→ Later, the ₹500 is actually transferred to the restaurant’s account.
This is why sometimes:
✓ Payment shows “successful”
BUT
✗ Merchant says “payment not received”
Because authorization succeeded...
but settlement is still pending.
Modern payments look instant to users.
Behind the scenes, multiple systems coordinate:
- Issuers
- Acquirers
- Networks
- Fraud engines
- Settlement systems
And all happening within seconds.
That’s the hidden engineering of card payments.
#Payments #Fintech #Banking #Visa #Mastercard #UPI #PaymentSystems #ISO8583 #CardPayments #FintechExplain Visa Mastercard RuPay #rbi #payments

#620 Java Developers Community (moderated)

Cloud / K8s / DevOps

Naveengandhi C. • 3rd+
5mo · View on LinkedIn

Java serialization broke our rolling deployment for 4 hours.
Two versions of the same class. One serialVersionUID mismatch.
That’s all it took.
Nothing crashed. Services were up. Health checks were green.
But requests started failing in weird, inconsistent ways.
Some nodes could read the data. Others couldn’t.
Retries made it worse. Traffic kept bouncing between versions.
It looked like a network issue at first. Then a cache issue.
It took time to realize — this wasn’t infrastructure.
It was serialization.
What actually happened was simple.
We had two versions of the same class deployed at the same time during a rolling update.
One version had a slightly modified field. The other didn’t.
The serialVersionUID changed.
Now objects serialized by one version couldn’t be deserialized by the other.
That’s when this shows up:
InvalidClassException

But only on certain nodes. Only for certain requests.
Which makes it painful to trace.
The part that’s easy to miss is this:
Java serialization isn’t just about converting objects to bytes.
It’s a contract.
If two versions of a class don’t agree on that contract, they can’t talk to each other.
And in a distributed system, that assumption breaks very quickly.
We fixed it by aligning the versions and redeploying.
But the real takeaway wasn’t the fix.
It was what we changed after.
We stopped relying on default serialization across service boundaries.
We made serialVersionUID explicit wherever serialization was unavoidable.
We became more careful about class evolution — adding fields is fine, but not without thinking about compatibility.
And most importantly, we stopped assuming that “minor changes” are safe during rolling deployments.
Serialization bugs don’t show up in local testing.
They show up when two versions of your system coexist.
And that’s exactly what happens during a deployment.
The uncomfortable truth is this:
Everything can look healthy, and still be broken.
If you’re still using Java serialization between services, it’s worth asking:
Is this something you can version safely?
Or is it time to move to a format designed for evolution?
What’s the most unexpected thing that’s broken your deployment?

MEDIA advertisement

#621 Java Developers Community (moderated)

System Design

Vladislav Grigorev • 3rd+
5mo · **View on LinkedIn**

Every backend engineer should know how payment retries work.
Retries in payments are not just “try again.”
They decide whether the system stays reliable - or charges the user twice.

1. Not every failure should be retried
Some failures are temporary:
- timeout
- network issue
- deadlock / transient DB error
Some failures are not:
- invalid card
- insufficient funds
- validation error
If you retry everything, you are not building resilience.
You are just repeating the wrong action.

2. Payment state matters

Before retrying, the system must know the current state:
CREATED, PROCESSING, CAPTURED, FAILED, REFUNDED.
Without state, retries are blind.

3. Idempotency is mandatory
The same request must not create two charges.
Use an idempotency key and return the same result on retry.

4. Retry queue handles transient failures
Do not retry instantly in a loop.
Send retryable failures to a queue and process them with delay/backoff.

5. DLQ handles repeated failures
If retries keep failing, move the message to a Dead Letter Queue.
That keeps bad messages from blocking healthy traffic.

6. Retries need limits
No retry should be infinite.
Use max attempts, backoff, and clear retry rules.

7. Reconciliation still matters
Even with retries, state tracking, and queues, things can still go wrong.
That is why real payment systems usually need reconciliation jobs:
- compare internal state with provider state
- detect mismatches
- repair missing updates
- trigger refunds or manual review when needed
Because in payments, “probably succeeded” is not good enough.

State Store example:
Payment provider returns Insufficient Funds → the payment is marked as FAILED in the state store with the final reason. No retry is needed.
DLQ example:
Payment provider keeps timing out, the message is retried several times, and still fails → after max retry attempts, the message is sent to the Dead Letter Queue for investigation.

Simple rule:
A retry is not just another attempt.
It is a business decision.
Have you seen retries save a payment flow — or break it even more? ↓
#backend #java #payments #microservices #systemdesign #softwareengineering #kafka

#622 Ankit Pangasa System Design

Engineering Manager at Adobe | Ex-Google | Breaking down interviews, system design & career growth | Sharing only verified job opportunities | Opinions my own | DM for collab

5mo · **View on LinkedIn**

Most database interviews focus on SQL syntax.
But senior-level interviews?
They focus on this ↓
How do you optimize databases at scale?
Because writing queries is easy.

Writing queries that survive:
millions of rows
high traffic
low latency
concurrent users
production workloads
...that’s where real engineering begins.
This infographic covers some of the most important database optimization techniques every backend engineer should know ↓
These topics show up everywhere:
System Design Interviews
Backend Engineering Interviews
Production Incident Reviews
Staff/Architect Discussions

Indexing
The first optimization most engineers learn.
Without indexes:
✗ full table scans
✗ slow queries
✗ high latency
But over-indexing also hurts:
✗ slower writes
✗ extra storage
✗ maintenance overhead
Know:
clustered vs composite indexes
B-tree vs hash indexes

Query Optimization
Sometimes the DB isn’t slow...
...the query is
Classic mistakes:
✗ SELECT *
✗ unnecessary joins
✗ missing filters
✗ N+1 queries
Good engineers:
✓ analyze query plans
✓ reduce scans
✓ optimize joins
One optimized query can save thousands in infra costs.

Normalization vs Denormalization
Normalization
✓ reduces redundancy

✓ improves consistency
BUT:
✗ too many joins can slow reads
Denormalization
✓ faster reads
✓ better analytics performance
BUT:
✗ duplicate data
✗ harder updates
Real systems usually use BOTH strategically.

Caching
Why hit the DB repeatedly for the same data?
Use:
Redis
Memcached
query caching
Benefits:
✓ lower latency
✓ reduced DB load
✓ better scalability
And yes...
→ cache invalidation is still one of the hardest problems

Partitioning
Large tables eventually become painful.
Partitioning helps split data by:
date
region
user
hash
Critical for large-scale systems.

Connection Pooling
Opening DB connections repeatedly is expensive.
Pooling improves:
✓ throughput
✓ concurrency
✓ resource utilization
Common tools:
HikariCP
PgBouncer
ProxySQL

Archiving & Cleanup
Old unused data silently kills performance.

Strong engineering teams:
✓ archive old records
✓ clean stale data
✓ separate hot vs cold storage
Because sometimes...
the fastest query is the one that never runs

Biggest lesson?
Database optimization is all about tradeoffs.
Every optimization improves something...
while making something else harder.
That’s what separates:
→ developers who write queries
from
→ engineers who design scalable systems.
Bookmark this for:
Backend Interviews
System Design
Database Design Rounds
Real-world scaling discussions

#623 Arslan Ahmad System Design

Author of Bestselling ‘Grokking’ Series on System Design, Software Architecture & Coding Patterns | Founder DesignGurus.io

5mo · **View on LinkedIn**

10 Must Read Papers for Senior Software Engineers

1. Dynamo - https://lnkd.in/dg48rQJB - Highly Available Key-value Store
2. Kafka - https://lnkd.in/dXA6HYib - A Distributed Messaging System for Log Processing
3. Consistent Hashing - https://lnkd.in/dE-w7KRG - Data distribution with minimal disruption during scaling.
4. Bigtable - https://lnkd.in/dyvF2RsW - A Distributed Storage System for Structured Data
5. Gossip protocol - https://lnkd.in/d5WZex_j - For failure detection and more.
6. Chubby - https://lnkd.in/dyF_u5Pa - Lock service for loosely-coupled distributed systems
7. ZooKeeper - https://lnkd.in/dZAVvK7x - Wait-free coordination for Internet-scale systems
8. MapReduce - https://lnkd.in/dyyQs7Fu - Simplified Data Processing on Large Clusters
9. HDFS - https://lnkd.in/d9q5wvHm - A Distributed File System
10. Cassandra - https://lnkd.in/dpFnWEMX - A distributed, decentralized, scalable, and highly available NoSQL database.

Ref: https://lnkd.in/g88SUxZr

LINKS

- https://www.designgurus.io/course/grokking-the-system-design-interview
- https://lnkd.in/dg48rQJB
- https://lnkd.in/dXA6HYib
- https://lnkd.in/dE-w7KRG
- https://lnkd.in/dyvF2RsW
- https://lnkd.in/d5WZex_j
- https://lnkd.in/dyF_u5Pa
- https://lnkd.in/dZAVvK7x

- https://lnkd.in/dyyQs7Fu
- https://lnkd.in/d9q5wvHm
- https://lnkd.in/dpFnWEMX
- https://lnkd.in/g88SUxZr

#624 Feezan Khattak

System Design

Backend Engineer | Java, Spring Boot, Microservices | AWS, Kubernetes | Scalable APIs for SaaS & Startups

5mo · View on LinkedIn

Multi-Region Payment Systems

One region goes down.

Do your payments stop?

If yes, you don’t have a payment system; you have a single point of failure.

Why multi-region matters:

- High availability (failover when a region dies)
- Lower latency for global users
- Resilience against outages

But it’s not that simple:

- Data consistency across regions
- Duplicate processing during failover
- Conflicting states between systems

Now you’re not just building payments — you’re dealing with distributed truth.

What good systems do:

1. Active-passive or active-active setups
2. Strong idempotency across regions
3. Region-aware routing (stick to one region per payment)
4. Async replication + reconciliation

#SystemDesign #Payments #BackendEngineering #DistributedSystems

MEDIA advertisement

#625 Feezan Khattak

Cloud / K8s / DevOps

Backend Engineer | Java, Spring Boot, Microservices | AWS, Kubernetes | Scalable APIs for SaaS & Startups

5mo · View on LinkedIn

Observability in Payments

A payment fails.

The user says it didn’t go through.

The provider says it did.

Now what?

Without observability,

You’re guessing.

What you actually need:

- Logs → what happened
- Metrics → how often it happens
- Tracing → where it broke

Because a single payment isn’t one call — it’s a chain across multiple services.

Where things get messy:

- Partial failures across systems
- Async flows (webhooks, queues)
- No single place with the full picture

How good systems handle this:

1. Correlate everything with a request/payment ID
2. Use distributed tracing across services
3. Log meaningful state transitions (not just errors)

If you can’t trace a payment end-to-end, you don’t control your system.

#SystemDesign #Payments #BackendEngineering #DistributedSystems

#626 SOORAJ VIDYASAGAR Java / Spring

Senior Technical Architect | Enterprise Architecture | AI-First Engineering | Cloud Strategy | Technology Leadership | Business Value|AI Architect

5mo · **View on LinkedIn**

Large switch statements are one of the most common maintainability problems in enterprise Spring Boot applications.

They usually begin as simple logic:

```
switch (type) {
case "CARD":
case "UPI":
case "PAYPAL":
}
```

But over time they become:

* harder to extend
* harder to test
* tightly coupled to business rules
* filled with duplicated logic

This is where the Strategy Pattern becomes powerful.

Instead of centralizing all behaviors in one service, each behavior gets its own strategy implementation.

Spring Boot makes this especially clean with Dependency Injection.

Example:

```
public interface PaymentStrategy {
void process(BigDecimal amount);
}
@Service("CARD")
public class CardPaymentStrategy implements PaymentStrategy {
@Override
public void process(BigDecimal amount) {
System.out.println("Processing card payment");
}
}
@Service
public class PaymentService {
private final Map<String, PaymentStrategy> strategies;
public PaymentService(Map<String, PaymentStrategy> strategies) {
```

this.strategies = strategies;
}
public void process(String type, BigDecimal amount) {
PaymentStrategy strategy = strategies.get(type);
if(strategy == null) {
throw new IllegalArgumentException("Invalid payment type");
}
strategy.process(amount);
}
}
Why teams adopt this approach:
* Easier extension of new behaviors
* Cleaner service layer
* Better unit testing
* Improved maintainability
* Better separation of concerns
* Follows Open/Closed Principle
Common enterprise use cases:
* Payment processing
* Notification providers
* Authentication handlers
* Workflow engines
* Export generators
* Pricing rules
* File processing pipelines
But architecture is always about balance.
Not every switch statement is bad.
If the logic is:
* small
* stable
* unlikely to grow
...then keeping a simple switch may actually be the cleaner solution.
Good software design is not about applying every design pattern.
It’s about reducing complexity while keeping systems easy to evolve.
A practical engineering rule:
If your switch statement grows every sprint, it probably wants a strategy pattern.
#java #springboot #designpatterns #softwarearchitecture #cleanarchitecture #microservices #backend #developers #refactoring #coding

#627 **Saumya Makker** **Java / Spring**

Software Engineer @ RoboMQ | Java 21 • Spring Boot • Microservices | Identity & Integration (AD, Entra ID, SCIM 2.0) | AI-Assisted Engineering (Claude Code, MCP) | 4.8 YOE | Ex-Nucleus Software

5mo · **View on LinkedIn**

A junior came to me with a problem:
“Payments are getting duplicated... no errors.”
We checked the code — everything looked fine.

He even had synchronized.
I asked:
“Where is this object created?”
“Per request.”
That’s when I said:
“Then you’re not locking anything.”
→ synchronized(this) works only if the object is shared
→ Here, every thread had its own instance
→ Result: race condition → duplicate payments
These bugs look small...
but in real-world systems, they can cause serious business impact.
Let’s break this down and see how to actually fix such issues
#Java #Multithreading #Concurrency

#628 Kanika Gosain System Design

Senior Java Backend Engineer | Spring Boot | Microservices | AWS | Scaled APIs to Millions | 45% Performance Optimization | Fintech

5mo · **View on LinkedIn**

Microservices Design Patterns Cheat Sheet (Save this)
Most developers know microservices...
But struggle when systems start failing in production.
Here are the patterns that actually keep distributed systems stable ↓

1. Saga Pattern
Manages distributed transactions without locking
→ Breaks into smaller steps + compensating actions

2. Circuit Breaker
Stops cascading failures
→ If a service fails repeatedly, calls are blocked temporarily

3. Retry with Backoff
Handles transient failures
→ Retry after delay (not instantly) to reduce pressure

4. API Gateway
Single entry point for clients
→ Handles auth, routing, rate limiting

5. Idempotency
Ensures same request = same result
→ Prevents duplicate payments / operations

6. Event-Driven Architecture
Loose coupling using events
→ Services communicate via queues (like AWS SQS)

7. Bulkhead Pattern
Isolates failures
→ One failing service won’t bring entire system down

8. Service Discovery
Dynamic service lookup
→ No hardcoded service URLs

Reality:
Most production issues are NOT due to code...
They happen because these patterns are missing.

Save this before your next system design interview
Repost to help other developers

#629 Srikant Mahanty Java / Spring

Software Engineer | Building Scalable Backend Systems | Java • Spring Boot • Microservices • Python • FastAPI • PostgreSQL | AI Enthusiast | 25K+ Community

5mo · **View on LinkedIn**

You’re Using This Pattern Daily in Spring... But Do You Really Understand It?
Proxy Design Pattern
Most developers think they are calling their service methods directly.
But in Spring Boot... that’s NOT true.
Whenever you use:
✓ @Transactional
✓ @Cacheable
You are actually calling a **Proxy**, not your real method.
→ Flow looks like this:
Client → Proxy → Actual Method
Now here’s where it gets interesting ↓
The Proxy:
✓ Starts transaction
✓ Decides cache hit/miss
✓ Calls actual method
✓ Commits / Rollbacks

Example:
First call → goes to DB
Second call → returned from cache
But both calls go through Proxy.

(!) Where most developers fail:
Self Invocation ✗
Calling a method inside the same class:
this.getUserDetails()
→ Proxy is bypassed
→ @Transactional / @Cacheable WON’T work

Real Insight:
Proxy is not just a pattern.
It’s the backbone of Spring AOP.

It enables:
✓ Transactions
✓ Caching
✓ Logging
✓ Security

Stop just using annotations.
Start asking:
* Who is actually executing my method?
* Where is the control happening?
* What is intercepted and why?
Because real engineering is not about fixing symptoms...
It’s about understanding the system.

Stay curious. Stay foolish.
— Srikant Mahanty

MEDIA advertisement

#630 Neo Kim

System Design

I Teach You AI Engineering & System Design • 0.5M+ Audience | View my newsletter
5mo · View on LinkedIn

6 algorithms that'll help you load balance traffic:
1 Round robin
→ Load balancer forwards the request to the servers in a sequential order
2 Weighted round robin
→ Load balancer forwards the request based on server capacity
3 Least connections
→ Load balancer forwards the request to the server with the fewest connections
4 IP hash
→ Load balancer forwards the request based on client's IP address
5 Least response time
→ Load balancer forwards the request to the fastest responding server
6 Adaptive
→ Load balancer forwards the request based on server health
What else should make this list?
===
PS - Want my System Design Playbook (for free)?
Join my newsletter with 200K+ software engineers now:
→ https://lnkd.in/ehTrcyak
===
Save now & repost to help others learn system design.
Follow Neo Kim + turn on notifications.

MEDIA 6 load balancing algorithms for software engineers

LINKS
- https://newsletter.systemdesign.one/join
- https://lnkd.in/ehTrcyak

#631 Vinay Kumar

System Design

Engineering Manager | Distributed Systems and Platform Engineering | Java I Spring boot | Microservices | Sql | NoSql I Multithreading | Netflix Conductor | Kafka | Redis | Dockers | Kubernetes | Agile

5mo · **View on LinkedIn**

Two developers. Same row. Same millisecond.

One update goes through. One gets rejected.

How your system handles that moment is the difference between data integrity and a silent bug in production.

Optimistic Locking — Built for Low Contention

→ No lock acquired on read

→ Every row carries a version number (v1, v2, v3...)

→ On write: your version must match the DB's current version

→ Someone else wrote first and bumped v1 → v2?

→ Your update is rejected. You retry with fresh data.

No blocking. No waiting. High throughput.

The tradeoff: under heavy write concurrency, retries compound fast.

Pessimistic Locking — Built for High Stakes

→ First transaction locks the row on read

→ Every other transaction queues and waits

→ No version checks. No retries. Serialized access.

→ Lock releases. Next writer proceeds.

Zero conflict risk. The tradeoff: throughput takes a hit.

The Decision Framework

Read-heavy, occasional writes → Optimistic

→ User settings, CMS edits, config updates

Write-heavy, expensive conflicts → Pessimistic

→ Seat reservation, inventory deduction, bank transfers

What Most Engineers Get Wrong

They choose optimistic locking because it "feels lighter."

Then they hit 10,000 concurrent writes on the same row.

Retry rate spikes.

DB CPU spikes.

Latency spikes.

Suddenly your "lightweight" strategy is your bottleneck.

The version check doesn't save you.

Knowing your writes-per-row-per-second does.

Profile your contention model before you pick your locking strategy.

Not after your first production incident.

Have you been burned by the wrong locking strategy in production? What was the workload?

CC: Alex Xu

#SystemDesign #BackendEngineering #DatabaseEngineering #DistributedSystems #SoftwareArchitecture

#632 Sameer Bhardwaj

System Design

AI Engineer | Founder @Layrs (Voice AI) | Ex-Google, Flipkart | LLMs · Real-time Voice · Distributed Systems

5mo · View on LinkedIn

If I travel back to my Interview Days, I will study these 40 Articles to learn system design Fundamentals. These articles and videos will save you 50+ hours of hopping to resources and wasting time.

Btw, if you're preparing for system design interviews, check out our AI Tutor: https://lnkd.in/gcWfR7jW

You can:

- voice chat about your questions in real-time
- get feedback in real time and improve with these sessions.
- learn concepts, practice HLD questions even if you're a complete beginner

All of these topics covered in the image are mentioned here in great detail:

[1] HLD Basics: https://lnkd.in/gmbzpdjV

[2] Core Concepts & Trade-offs: https://lnkd.in/gUFeCcpT

[3] Networking & DNS: https://lnkd.in/gENhQVgS

[4] Load Balancing & Scaling: https://lnkd.in/giQkSPc7

[5] Application Architecture: https://lnkd.in/gx8NQwKG

[6] Databases: https://lnkd.in/gX9VK-MM

[7] Caching: https://lnkd.in/gtcfXJAH

[8] Asynchronous Processing:https://lnkd.in/gYeJ2hca

[9] Communication Protocols: https://lnkd.in/giSSdMJR

[10] Performance & Monitoring: https://lnkd.in/giiKTJE4

[11] Cloud Design Patterns: https://lnkd.in/gNTmsgHA

[12] Reliability Patterns: https://lnkd.in/g6fvAknr

LINKS

- http://layrs.me
- https://lnkd.in/gcWfR7jW
- https://lnkd.in/gmbzpdjV
- https://lnkd.in/gUFeCcpT
- https://lnkd.in/gENhQVgS
- https://lnkd.in/giQkSPc7
- https://lnkd.in/gx8NQwKG
- https://lnkd.in/gX9VK-MM
- https://lnkd.in/gtcfXJAH
- https://lnkd.in/gYeJ2hca
- https://lnkd.in/giSSdMJR
- https://lnkd.in/giiKTJE4
- https://lnkd.in/gNTmsgHA
- https://lnkd.in/g6fvAknr

#633 Asmita Pednekar

Java / Spring

Software Developer 2 | ORACLE | Ex-IBM | Java Full stack

5mo · View on LinkedIn

Most Java developers say:

"I know Java 8... a bit of 17... heard about 21."

But here's the uncomfortable truth:

You can upgrade your Java version and still write outdated code.

Java 8 → Where most developers are stuck

class User {
private String name;
private int age;
// getters, setters, constructors
}
Works perfectly.
But:
• Too much boilerplate
• Harder to maintain at scale
• Concurrency = thread pools + complexity
This is still how a lot of production code looks today.
Java 17 → Where things got cleaner
record User(String name, int age) {}
Suddenly:
• Less code
• Better readability
• Fewer bugs
• More focus on logic
Also:
• Sealed classes → better control over inheritance
• Pattern matching → cleaner conditions
→ This is where Java started feeling modern again.
Java 21 → Where the real shift happens
Thread.startVirtualThread(() -> processRequest());
No more:
• Complex thread pool tuning
• Managing limited threads
• Overthinking concurrency for simple I/O tasks
Now you can handle thousands of requests
without writing complicated concurrency code.
What actually changed (this is the part people miss)
Java didn’t just add features.
It removed friction.
Java 8 → You write more code
Java 17 → You write better code
Java 21 → You scale code more easily
The real problem is that most developers are still writing:
Java 8-style code on Java 17 or 21
And then wondering why nothing feels different.
Real takeaway
Upgrading Java doesn’t make you better.
Upgrading how you think does.
The gap in 2026 is not:
Java 8 vs Java 21
It’s:
Developers who evolve

vs

Developers who don’t

If you’re still writing Java like it’s 2016...

that’s the real problem.

#Java #JavaDeveloper #SoftwareEngineering #BackendDevelopment #Programming #Developers #Java8 #Java17 #Java21 #CareerGrowth

#634 Sai Ram Somanaboina

System Design

Engineering Manager at NowFloats(Jio Group) | 15 years in Engineering

5mo · View on LinkedIn

Cloudflare handles over 45 million HTTP requests every single second. But turning on strict HTTPS won't stop their CDN from leaking your users' private data to a stranger.

HTTPS prevents man-in-the-middle attacks. But most engineers forget one terrifying fact: your CDN is literally a man-in-the-middle by design.

That is not a bug.

That is how CDNs work.

[1] HTTPS does not protect you from bad cache rules

HTTPS protects data while it is moving across the network. It does not protect you from your own CDN serving the wrong cached response.

For example, imagine User A opens the billing page and the frontend calls GET /api/billing/summary. Your backend returns private billing data for User A.

But because the CDN rule is too broad, that response gets cached only against the URL path.

Now User B opens their billing page and calls the same endpoint.

Same URL. Different user.

If the cache key does not include anything user-specific, the CDN may return User A’s billing data to User B. The CDN was not hacked. HTTPS was not broken.

Your backend did not even get called. The system simply followed the rule it was given: “same URL, same cached response.”

[2] The real issue is the cache key

A cache key decides whether two requests should be treated as the same. For static files, this is simple. /logo.png can be cached safely because every user should get the same file.

For authenticated APIs, this becomes dangerous.

/api/billing/summary may look like the same URL for every user, but the response is completely different for each logged-in account.

That means the cache key cannot be just the path. It either needs to include user-specific signals like authorization or cookies, or the response should not be stored in a shared cache at all.

[3] The safer way to handle authenticated routes

For user-specific responses, use Cache-Control: private so shared caches know the response is not meant for everyone.

For highly sensitive routes like billing, payments, invoices, medical data, account settings, permissions, and admin views, use Cache-Control: no-store.

If the response changes based on the logged-in user, use Vary: Authorization or Vary: Cookie so the CDN treats different sessions as different cache entries.

And most importantly, audit CDN rules like production code. Your CDN is not just infrastructure anymore. It can decrypt requests, skip your backend, and serve responses directly to users. That makes it part of your security boundary.

The lesson is simple:

HTTPS protects data in transit.

> Cache configuration protects data from being served to the wrong person.
> If you get the second part wrong, your CDN can become the fastest way to leak private user data.

#635 Prashant Varshney AI / LLM

Senior AI Engineer & Architect | Building Tech that Power Millions | System Design & AI Enthusiast

5mo · **View on LinkedIn**

The AI Learning Gap is Real. You Need a Map, Not Noise.
I just filtered the entire YouTube library for you.
Here are 20 YouTube channels that make learning AI simple & structured:
1. 3Blue1Brown: Understand complex math behind AI visually and intuitively.
Link: https://lnkd.in/edegfEEv
2. Andrej Karpathy: Deep, practical AI lectures explained clearly.
Link: https://lnkd.in/eay7TU2a
3. Lex Fridman: Conversations with leading AI researchers and innovators.
Link: https://lnkd.in/ebbtpsww
4. StatQuest (Joshua Starmer PhD): Makes ML concepts fun with humor and clarity.
Link: https://lnkd.in/eqTeYjMT
5. Jeremy Howard: Practical deep learning with hands-on coding examples.
Link: https://lnkd.in/e_vHAu84
6. Two Minute Papers: Summaries of the latest AI papers in minutes.
Link: https://lnkd.in/eyBhZC9p
7. DeepLearning.AI: Structured AI learning from Andrew Ng.
Link: https://lnkd.in/e62uRF2g
8. Machine Learning Street Talk (MLST): Insightful debates and interviews.
Link: https://lnkd.in/eUwV47cn
9. freeCodeCamp: Free AI and ML tutorials with certification paths.
Link: https://lnkd.in/eUn2JUiM
10. Sentdex: Python-based machine learning and data projects.
Link: https://lnkd.in/e-dCBfas
11. Data School: Simple ML and data analysis concepts for beginners.
Link: https://lnkd.in/egtSHRy8
12. Codebasics: Real-world ML use cases and career-focused projects.
Link: https://lnkd.in/ez2NmfVd
13. Siraj Raval: Story-driven tutorials combining creativity and AI.
Link: https://lnkd.in/ehJf3jzR
14. Google Cloud Tech: Learn how to deploy and manage AI models.
Link: https://lnkd.in/euJTVeyM
15. Serrano Academy: Step-by-step tutorials on ML, DL, and AI tools.
Link: https://lnkd.in/eSzJJJWY
16. Tina Huang: Smart AI learning strategies and productivity tips.
Link: https://lnkd.in/exwv6q7i
17. Matt Wolfe: Quick updates on new AI tools and technologies.
Link: https://lnkd.in/eiVMeZj3
18. AI Explained: Deep dives into AI ethics, models, and progress.
Link: https://lnkd.in/etfCYhMq

19. The AI Advantage: Practical ways AI is transforming business productivity.

Link: https://lnkd.in/egyKfySP

20. Hamel Husain: Advanced insights into LLMs, RAG, and model fine-tuning.

Link: https://lnkd.in/eSgQMg_d

Whether you want to build models, deploy systems, or understand AI theory, this list covers every learning style.

Which of these channels do you watch regularly and which one do you think deserves more attention?

Credits: Sandipan Bhaumik

LINKS

- https://lnkd.in/edegfEEv
- https://lnkd.in/eay7TU2a
- https://lnkd.in/ebbtpsww
- https://lnkd.in/eqTeYjMT
- https://lnkd.in/e_vHAu84
- https://lnkd.in/eyBhZC9p
- http://DeepLearning.AI
- https://lnkd.in/e62uRF2g
- https://lnkd.in/eUwV47cn
- https://lnkd.in/eUn2JUiM
- https://lnkd.in/e-dCBfas
- https://lnkd.in/egtSHRy8
- https://lnkd.in/ez2NmfVd
- https://lnkd.in/ehJf3jzR
- https://lnkd.in/euJTVeyM
- https://lnkd.in/eSzJJJWY
- https://lnkd.in/exwv6q7i
- https://lnkd.in/eiVMeZj3
- https://lnkd.in/etfCYhMq
- https://lnkd.in/egyKfySP
- https://lnkd.in/eSgQMg_d

#636 Java Developers Community (moderated)

Java / Spring

Sarvesh Kumar Singh • 2nd

5mo · **View on LinkedIn**

Garbage Collection in Java — The Silent Hero Behind Your Code

Most developers use Java every day...

But very few actually understand what’s happening in memory behind the scenes.

Let’s break it down ↓

What is Garbage Collection?

GC automatically reclaims memory occupied by objects that are no longer reachable — no manual free() needed like in C/C++.

How does it work?

→ JVM tracks all objects in Heap memory

→ Unreferenced objects are marked as garbage

→ GC reclaims that memory automatically

You can suggest GC to run via System.gc() — but you can't force it.

The 5-Step GC Process:

1. Object Created → stored in Heap

2. Object in Use → referenced by active threads

3. Object Becomes Unreachable → no references left

4. GC Identifies it → marks it as garbage

5. Memory Reclaimed → freed up ✓
Heap Memory Generations:
Young Generation (Eden + S0 + S1) → where most objects are born
Old Generation (Tenured) → long-surviving objects move here
Permanent Generation → class metadata (MetaSpace in Java 8+)
Types of Garbage Collectors:
• Serial GC → single-threaded, small apps
• Parallel GC → throughput-focused, multi-threaded
• CMS → minimizes pause time
• G1 GC → default from Java 9+, best for large heaps ★
(!) Key Things to Remember:
✓ GC is automatic — but it can cause stop-the-world pauses
✓ Proper GC tuning = better app performance
✓ Use VisualVM or JConsole to monitor GC activity
Where Developers Go Wrong
• Creating too many unnecessary objects
• Holding references → causing memory leaks
• Ignoring GC pauses in high-performance apps
Real-World Tip
If your app has:
• High traffic → Use G1 / ZGC
• Batch jobs → Use Parallel GC
Have you ever faced GC-related performance issues?
Repost to help a fellow Java developer!
#ProgrammingTips #LearnToCode #DeveloperLife #CodeNewbie #TechEducation #Engineers #SystemDesign

#637 Divya Pratap Singh

Java / Spring

Backend Engineer building high-throughput Java systems | Spring Boot · Kafka · Redis · Distributed Systems | Open-source @ github.com/divyapratapdev

5mo · **View on LinkedIn**

Concurrency looks simple until your backend starts behaving unpredictably
requests pile up
responses slow down
and bugs become hard to reproduce
Swipe through this
This is how I started understanding concurrency in real backend systems
In the beginning I thought creating threads would solve performance
but more threads do not always mean better systems
the real challenges come from shared data
race conditions
thread safety
and managing resources properly
Things started making more sense when I learned
how thread pools actually work
why synchronization is needed

where atomic operations help
and how small mistakes can lead to deadlocks
The more I learn
the more I realise concurrency is not about speed
it is about control and correctness
Still learning and improving
What part of concurrency do you find most confusing
#Java #Concurrency #BackendDevelopment #SoftwareEngineering #Multithreading #SystemDesign

#638 Ganesh Bankar

System Design

Java Microservice Expert | 1M+ post impressions | Writes to 4K+ | AI | 11+ Years | Spring Boot | AWS | Azure | Development Manager@Mastercard | Guest Speaker @CDAC Pune

5mo · View on LinkedIn

Modern Java Microservices Architecture The Backbone of Scalable Enterprise Systems
After spending years working on enterprise applications, one thing becomes very clear:
Building APIs is easy.
Building systems that are scalable, resilient, secure, and production-ready is the real engineering challenge.
A strong microservices ecosystem is not just about Spring Boot services.
It’s about how every layer works together seamlessly:
✓ API Gateway for routing, authentication & rate limiting
✓ Service Discovery & Centralized Configuration
✓ Event-Driven Communication using Kafka
✓ Distributed Caching with Redis
✓ Observability with ELK, Grafana & Prometheus
✓ Containerization using Docker & Kubernetes
✓ CI/CD pipelines for faster and safer deployments
✓ Cloud-native deployment on AWS/Azure/GCP
✓ Resilience, fault tolerance & high availability by design
The biggest mindset shift in modern backend engineering:
→ We are no longer building “applications”
→ We are building distributed ecosystems
And in distributed systems, success depends on: • Reliability
• Monitoring
• Scalability
• Automation
• Security
• Performance under load
The more I work on large-scale systems, the more I realize:
Clean architecture is not optional anymore , it’s a competitive advantage.
#Java #SpringBoot #Microservices #SoftwareArchitecture #Kafka #InterviewPreparation #JavaCareers #JavaInterview #Kubernetes #AWS #SystemDesign #BackendDevelopment #DevOps #CloudComputing #TechLeadership #DistributedSystems #Programming #Engineering #LearnWithGaneshBankar

#639 Sohel khan

Java / Spring

Technical lead at HCL Technologies Ltd

5mo · View on LinkedIn

Spring AI Cheat Sheet for Java Developers!

If you're working with Spring Boot and curious about AI integration, this cheat sheet will save you hours

What’s inside:

✓ ChatClient setup

✓ Prompt templates

✓ Memory handling

✓ RAG (Retrieval Augmented Generation)

✓ Streaming responses

Spring AI makes it insanely easy to integrate LLMs into your applications

Save this for later & share with your dev circle!

#SpringAI #JavaDeveloper #SpringBoot #ArtificialIntelligence

#640 Suraj Mishra

Java / Spring

Technical Lead & Staff Software Engineer @ PayPal

5mo · View on LinkedIn

Spring Boot Interview Question: Selective Retry on Network Errors

Scenario

You are building a Spring Boot Order Service that calls an external Payment Gateway to authorize payments.

Requirements:

- Retry transient network errors (like timeouts) and HTTP 5xx responses

- Fail fast on HTTP 4xx errors or validation failures

- Apply explicit timeouts to avoid hanging threads

- Do not retry non-transient network failures like SSLHandshakeException

During a traffic spike, some payment requests failed due to network timeouts, but no retries occurred because the retry configuration was misconfigured.

Why didn’t network timeouts trigger retries?

Checkout detailed discussion with solution:

https://lnkd.in/eC3gJAjB

Subscribe and join 7000+ Spring and Java devs and receive handbook consisting 100 resource’s :

https://lnkd.in/eC3gJAjB

LINKS

- https://lnkd.in/eC3gJAjB

#641 Dipu Kumar

Java / Spring

Senior Software Engineer @ Cisco | ex-Hikvision | ex-Varite | Java | Spring Boot | Microservices | System Design | DSA | Python | RAG | MCP | GenAI | Agentic AI

5mo · View on LinkedIn

*Design Patterns in Java: The Complete Cheat Sheet *

If you’re prepping for interviews or building scalable systems, design patterns are non-negotiable.

I broke down all 23 GoF patterns + 2 bonus ones into 3 categories:

1. Creational Patterns – _How objects are created_

For controlling object creation and ensuring flexibility.

- *Singleton*: Only 1 instance → DB Connection, Logger, Config
- *Factory Method*: Create objects without exposing logic → UI frameworks
- *Abstract Factory*: Factory of factories → Cross-platform UI, themes
- *Builder*: Build complex objects step-by-step → `StringBuilder`, `Lombok`
- *Prototype*: Clone instead of new → Caching, Games
- *Object Pool*: Reuse expensive objects → DB connections, Threads
- *Factory Method with Parameter*: Create based on input → APIs, Config-driven

2. Structural Patterns – _How classes/objects are composed_

For assembling objects and classes into larger structures.

- *Adapter*: Convert interfaces → Legacy code, 3rd party integration
- *Bridge*: Decouple abstraction from implementation → JDBC drivers
- *Composite*: Tree structures → File systems, UI components
- *Decorator*: Add behavior dynamically → `BufferedReader`, Logging
- *Facade*: Simple interface to complex system → `SpringApplication.run()`
- *Flyweight*: Share to reduce memory → String pool, Games
- *Proxy*: Control access → Lazy loading, Spring AOP, Caching

3. Behavioral Patterns – _How objects interact_

For communication between objects and assigning responsibilities.

- *Observer*: Notify on change → Event listeners, Kafka
- *Strategy*: Swap algorithms → `Comparator`, Payment methods
- *Command*: Request as object → Undo/Redo, Queues
- *Iterator*: Traverse collections → `for-each` loop
- *Mediator*: Centralize communication → Chat apps, Air traffic control
- *Memento*: Save/restore state → Undo, Snapshots
- *State*: Change behavior with state → Workflows, TCP states
- *Template Method*: Algorithm skeleton → Frameworks, Libraries
- *Visitor*: Add operations without changing classes → Compilers
- *Chain of Responsibility*: Pass request along chain → Servlet filters, Logging

Why learn them?

✓ Reusability ✓ Maintainability ✓ Flexibility ✓ Scalability ✓ Clean Code

Real-world Java use cases:

- *Singleton*: Spring Beans, Logger
- *Factory*: `Calendar.getInstance()`, Spring Bean Creation
- *Observer*: `EventListener`, RxJava
- *Strategy*: `Collections.sort()` with `Comparator`
- *Proxy*: Spring AOP, Hibernate lazy loading

LINKS

- http://SpringApplication.run

#642 Java Developers Community (moderated)

Java / Spring

Sarvesh Kumar Singh • 2nd

5mo · **View on LinkedIn**

Lazy vs. Eager Loading in Hibernate with Spring Boot: Choosing the Right Strategy

Is your Spring Boot application's performance slowing down as your database grows?

The real problem may not be SQL indexes or server memory...

It could be how Hibernate fetches your entities.

If you're working with JPA/Hibernate, understanding Lazy vs. Eager Loading can save you from slow APIs, memory bloat, and production surprises. Let's break it down. ↓

What is Fetching in Hibernate?

Fetching defines when related data is loaded from the database.

For example:

If you load an Author, should Hibernate also load all Books immediately? Or only when needed?

That's where Eager and Lazy loading come in.

Eager Loading = Fetch All at Once

✓ Benefits:

Easy to use

No extra queries later

Good when related data is always required

(!) Risks:

Fetches unnecessary data

Higher memory usage

Slower initial query performance

Lazy Loading = Fetch On Demand

With Lazy , related data is loaded only when accessed in code.

✓ Benefits:

Better performance for large relationships

Loads only required data

Reduces unnecessary joins

(!) Risks:

Can cause extra queries

May throw LazyInitializationException if session is closed

Needs proper query design

Example in Spring Boot

```
@Entity
public class Author {
@Id
@GeneratedValue
private Long id;
private String name;
@OneToMany(mappedBy = "author", fetch = FetchType.LAZY)
private List<Book> books;
}
@Entity
public class Book {
@Id
@GeneratedValue
private Long id;
private String title;
@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "author_id")
```

private Author author;
}
→ What happens here?
Author.books will load only when accessed.
Book.author loads immediately with Book.
Pro Tip: The N+1 Select Problem
One of the biggest Hibernate performance killers is the N+1 Select Problem.
How Lazy Helps:
Prevents loading unnecessary child data by default.
How Lazy Hurts:
If you loop through entities carelessly, it triggers multiple queries
Have you faced the dreaded LazyInitializationException in production?
How did you solve it?
↓ Share your experience in the comments.
Subscribe to Spring Boot Engineering Digest for practical backend engineering insights, performance tips, and real-world Spring Boot strategies.
From Spring Boot Engineering Digest — by Sarvesh Kumar Singh

LINKS

- http://Book.author

#643 Blesson V

Java / Spring

Energetic Java Developer | Hands on Experience in Java , Spring Boot, Microservices & Scalable Solutions | Passionate About Delivering High-Performance Code & Contributing to Innovative Projects

5mo · **View on LinkedIn**

Java Spring Boot Microservices Learning Roadmap (2026)
Best Learning Approach
Theory → Code → Project → Interview Questions → Real Scenarios If you want to become a strong Java Spring Boot Microservices Developer, do not learn only theory.
#Java #SpringBoot #Microservices #BackendDevelopment #SoftwareEngineering #JavaDeveloper #SystemDesign #SpringSecurity #Kafka #Redis #RESTAPI #CodingInterview #TechCareer #SoftwareArchitecture #DeveloperRoadmap

#644 Sunil Reddy

System Design

Lead Product Engineer @ Harman | Java · Spring Boot · Microservices · AI/LLM Integration | 7+ Years

5mo · **View on LinkedIn**

Most people add cache like this:
“Put Redis in front of DB.”
That’s not a cache strategy.
That’s hope.
Caching works only when you choose the right pattern.
Common strategies:
1. Cache Aside
App checks cache first.
If missing, fetch DB and update cache.
Best for: read-heavy systems.
2. Write Through

Write goes to cache and DB together.
Best for: strong consistency needs.
3. Write Behind
Write to cache first, DB later.
Best for: high write throughput.
Risk: data loss if not handled well.
4. Read Through
App reads from cache.
Cache itself loads from DB if missing.
Best for: simpler app logic.
5. TTL-based Cache
Data expires after fixed time.
Best for: data that can be slightly stale.
Golden rule:
Don’t ask “Should we use cache?”
Ask:
“What level of stale data can this feature tolerate?”
Because caching is not just performance.
It’s a consistency decision.
Be honest ↓
Which cache strategy do you mostly use in production?
#caching #redis #systemdesign #microservices #backend #java #softwarearchitecture

#645 HarshaVardhan Sakhamuri System Design

Sr. Full Stack Developer | Java 8–21 • Python •Kotlin| Spring Boot • Microservices • Apigee • Temporal | React • Angular | AWS •Azure •GCP |DevOps • Kubernetes • Terraform | GenAI •RAG •Vertex AI |FHIR • Epic Integration

5mo · **View on LinkedIn**

Your microservices aren’t slow because of traffic...
they’re slow because of THIS design flaw.
Most teams scale infrastructure first.
More pods. Bigger clusters. Higher costs.
But the real bottleneck?
Architecture.
We had a typical flow:
Client → API Gateway → Service A → Service B → Database
Average response time: ~2 seconds
Way too slow for real-time systems.
So instead of scaling infra, we fixed the design.
Here’s what actually made the difference:
Caching with Redis
Cached hot data and reduced repeated DB calls
→ Faster reads, lower load
Reduced service hops
Removed unnecessary chaining
Merged tightly coupled logic

→ Less network latency
Query optimization
Fixed N+1 issues
Added proper indexing
→ Faster database response
Async processing
Moved non-critical work to background jobs
→ Faster user-facing responses
Final result:
~2s → ~600ms
Here’s the real takeaway:
Performance issues are rarely about code.
They’re about design decisions.
Scale architecture first.
Infrastructure comes later.
Thanks & Regards,
Harshavardhan Sakhamuri
314-690-7292
harshasakhamuri.work@gmail.com
#Microservices #SystemDesign #BackendDevelopment #Java #SpringBoot #DistributedSystems #PerformanceOptimization #SoftwareEngineering #TechArchitecture #Scalability #Caching #Redis #DevOps #CloudComputing #EngineeringLife

#646 Kanika Gosain Java / Spring

Senior Java Backend Engineer | Spring Boot | Microservices | AWS | Scaled APIs to Millions | 45% Performance Optimization | Fintech

5mo · **View on LinkedIn**

Most developers use Spring annotations daily...
But very few actually understand what they mean in interviews.
Here’s a quick cheat sheet I wish I had earlier ↓
Core Stereotypes
@Component → Generic bean
@Service → Business logic
@Repository → DAO + exception translation
@RestController → APIs (returns JSON)
Dependency Injection
@Autowired → Inject by type
@Qualifier → Choose specific bean
@Primary → Default bean
@Value → Inject from properties
Configuration
@Configuration → Bean config class
@Bean → Manual bean creation
@Lazy → Initialize only when needed
Spring Boot Magic
@SpringBootApplication → Entry point

@EnableAutoConfiguration → Auto config based on classpath
@ConfigurationProperties → Bind properties to POJO
Web Layer
@GetMapping / @PostMapping → API endpoints
@RequestBody → JSON → Object
@PathVariable → Value from URL
@RequestParam → Query params
Transactions & AOP
@Transactional → DB transaction management
@Aspect → Cross-cutting logic
@Around → Wrap method execution
Caching & Async
@Cacheable → Cache result
@Scheduled → Run jobs
@Async → Run in background
Interview tip:
Don’t just name the annotation.
Always add why it exists.
That’s what separates average from senior.
If you’re preparing for backend interviews, save this — you’ll revise it multiple times.
#Java #SpringBoot #BackendDevelopment #InterviewPrep #SoftwareEngineering

#647 Java Developers Community (moderated)

Other

Amol Limaye • 3rd+ | Here is an overview of Java threads and concurrency. #java #interview | 316 | 1 | 37 | | Sahil Singh, Open to work Verified Profile 3rd+ | Sahil Singh | Junior Software Developer at Walkover
5mo · **View on LinkedIn**

Very informative.

MEDIA Java threads overview; advertisement

#648 Ambadas Garje

Java / Spring

Senior Software Developer | 8 Years Experience in Spring Boot Ecosystem | Java, Spring MVC, Microservices, REST APIs | Nelito Systems Pvt. Ltd.
5mo · **View on LinkedIn**

Spring Framework | Day 15
Spring MVC vs Spring Boot – Internal Working (Step-by-Step)
Many developers use both, but the real difference becomes clear when you understand how they work internally.
Let’s break it down in a simple, real-project perspective ↓
Using Spring MVC (Traditional Way)
→ Flow:
Client → DispatcherServlet → HandlerMapping → Controller → Service → DAO → ViewResolver → Response
Internal Working:
• Requires manual configuration (XML / Java Config)
• DispatcherServlet must be explicitly configured

• ViewResolver setup needed for UI apps
• External server (like Tomcat) deployment required
Real Challenge:
More setup, more control — but time-consuming for large projects
Using Spring Boot (Modern Approach)
→ Flow:
Client → Embedded Server → Auto-config → Controller → Service → Repository → JSON Response
Internal Working:
• Auto-configuration handles most setup
• Embedded server (Tomcat/Jetty) included
• No need for manual DispatcherServlet config
• Runs as standalone application (main method)
Real Advantage:
Less configuration, faster development, production-ready APIs
Key Difference (In One Line)
→ Spring MVC = Manual configuration + more control
→ Spring Boot = Auto-configuration + rapid development
Real Project Insight:
In modern backend systems, we prefer Spring Boot because:
✓ Faster setup
✓ Easy microservices development
✓ Clean REST APIs with minimal boilerplate
But understanding Spring MVC internally helps in:
✓ Debugging complex issues
✓ Custom configurations
✓ Legacy system maintenance
Pro Tip:
Don’t skip Spring MVC concepts — Spring Boot is built on top of it!
#SpringBoot #SpringMVC #Java #BackendDevelopment #Microservices #SoftwareEngineering #InterviewPreparation

#649 Anis Rahman

Java / Spring

Java Full Stack Developer | @AnisCodex - Peer Guide for Aspiring Developers | Spring Boot · Hibernate · React.js · REST APIs | DSA in Java · 200+ Pattern-Based Problems Solved | Open to opportunities

5mo · **View on LinkedIn**

8 Java Maps. 8 Different Problems. Which One Do You Actually Need?

Most developers default to HashMap for everything. That's like using a hammer for every job in the toolbox.

Here's the breakdown you didn't get in college:

1. HashMap → O(1) get/put, no ordering, not thread-safe. Use when: you just need fast key-value lookup.

2. LinkedHashMap → Maintains insertion order via doubly linked list. Use when: building LRU caches or ordered iteration matters.

3. TreeMap → Sorted keys, O(log n), backed by Red-Black tree. Use when: you need range queries like subMap(), floorKey().

4. Hashtable → Synchronized, no nulls, legacy class. Don't use it. Reach for ConcurrentHashMap instead.

5. ConcurrentHashMap → Thread-safe without locking the whole map. Use when: multiple threads read/write simultaneously.

6. WeakHashMap → Keys are weakly referenced — GC can collect them. Use when: you need memory-sensitive caching (won't cause leaks).

7. EnumMap → Only enum keys, array-backed, blazing fast. Use when: your keys are a fixed enum — state machines, configs.

8. IdentityHashMap → Uses == instead of .equals() for key comparison. Use when: object identity matters — serialization, graph traversal.

The rule is simple: Match the map to the problem, not the other way around. Choosing the wrong map is free today. The production bug it causes? Not so free.

Save this before your next code review. Repost if your team still uses Hashtable in 2026. Which map do you reach for most — and have you ever regretted it?

#Java #SoftwareEngineering #Programming #BackendDevelopment #100DaysOfCode #CleanCode #JavaDeveloper #TechTwitter #CodingTips #DataStructures

MEDIA advertisement

#650 Sarvesh Kumar Singh

Java / Spring

Java Backend Developer | Spring Boot | Microservices | REST APIs | SQL | System Design | Performance Optimization

5mo · **View on LinkedIn**

JAR vs WAR vs EAR — Java Packaging Demystified!

If you've ever stared at a Java project wondering "what even is this file?" — this one's for you. ↓

As Java developers, we deal with these three archive types constantly, but many beginners (and even some seniors!) mix them up. Let's break it down simply:

JAR — Java ARchive The most basic building block. Think of it as a zip file for your Java classes and resources.

✓ Used for: Libraries, utilities, standalone apps Server: Any JVM

Example: commons-lang.jar, jackson-databind.jar

WAR — Web ARchive Everything you need to deploy a web application. It's a JAR — but supercharged with web goodies: Servlets, JSPs, HTML, CSS, JS & WEB-INF config.

✓ Used for: Web applications Server: Tomcat, Jetty, Undertow

EAR — Enterprise ARchive The big boss of Java packaging. An EAR can contain multiple WARs + JARs (EJB modules) under one roof — perfect for large enterprise systems.

✓ Used for: Full-scale enterprise applications Server: JBoss, WildFly, WebLogic

Golden Rule to Remember:

JAR ⊂ WAR ⊂ EAR

Each level wraps the previous one. Start simple, scale when needed.

Drop a comment — are you still deploying WARs or have you moved to containerized JARs with Spring Boot? Let's talk!

#Java #JavaDeveloper #SpringBoot #WebDevelopment #SoftwareEngineering #BackendDevelopment #Programming #100DaysOfCode #TechTips #JavaEE #JVM #LearnToCode #DevCommunity #CodeNewbie #CleanCode

#651 Java Developers Community (moderated)

Other

Amaan Nirban • 3rd+

5mo · **View on LinkedIn**

The one thing that cost me FAANG interview (and how I fixed it)

45 minutes. 2 problems. Both solved. Still underperformed.

The culprit? 8 minutes wasted trying to recognize the pattern.

That's when I learned: Pattern recognition IS the interview.

After Part I's response, here's Part II — the advanced patterns from actual final rounds:

BINARY TREE PATTERNS — PART II

6. Tree Construction

Keywords: "construct", "build from traversals", "serialize"

The trick: Preorder[0] = root → process LEFT subtree first | Postorder[last] = root → process RIGHT subtree first

7. BST-Specific Properties

Keywords: "validate BST", "kth smallest", "successor/predecessor"

Golden Rule: Inorder traversal of BST = Sorted Array

8. Tree Comparison & Symmetry

Keywords: "same tree", "symmetric", "mirror", "invert"

The pattern: Compare OUTER pairs (L.left ↔ R.right) and INNER pairs (L.right ↔ R.left)

9. Tree Modification

Keywords: "flatten", "merge two trees", "prune", "trim"

Strategy: POSTORDER when modifying structure (bottom-up ensures children are processed first)

10. Tree Dynamic Programming

Keywords: "optimal", "maximum", "rob houses", "minimum cameras"

The secret: Return multiple states {include_node, exclude_node}

Which pattern have you seen in your interviews? Drop the number below!

Part I link in the comments!!

Follow Amaan Nirban for more such content!

#DSA #LeetCode #CodingInterview #FAANG #InterviewPrep #SoftwareEngineering #BackendDeveloper #Programming

10 pages

MEDIA AMAAN NIRBAN By Amaan Nirban | Binary Tree Mastery | Part II 1/10 Advanced Techniques for Final Round Interviews Construction Build • Serialize BST Patterns Validate • Kth • Iterator Comparison Same • Symmetric • Invert Modification Flatten • Merge • Prune Tree DP Rob • Cameras • Optimal My Google Interview Experience In my Google onsite, I was asked two tree problems that tested these patterns: Problem 1: BST Pattern — Required deep understanding of BST properties & inorder traversal → See Slide 4 Problem 2: Tree Construction — O(1) space tree building using Morris technique → See Slide 3 Master these patterns to ace tree problems in final rounds! PART II; 2/10 Pattern Recognition Flowchart - Part II See the keyword → Know the pattern instantly ADVANCED TREE PROBLEM? ↓ "construct" "build" "serialize" ↓ CONSTRUCTION Pre+In / Post+In "BST" "kth" "validate" ↓ BST PATTERN Inorder = Sorted "same" "symmetric" "invert" ↓ COMPARISON Parallel Traverse "flatten" "merge" "prune" ↓ MODIFICATION Transform Structure "optimal" "rob" "cameras" ↓ TREE DP Return Multiple Part II Rule: "Construction = Traversals, BST = Inorder, Comparison = Parallel, Modification = Postorder, DP = Multi-Return"; 3/10 Pattern 7: Tree Construction "construct" "build tree" "from traversal" "serialize" "deserialize" "sorted array to BST" "flatten" "Morris" "O(1) space" Build from Preorder + Inorder // Time: O(n) | Space: O(n) // Pre[0] = root, partition by inorder TreeNode* build(pre, inLeft, inRight) { if (inLeft > inRight) return null root = new TreeNode(pre[preIdx++]) mid = inorderMap[root->val] root->left = build(pre, inLeft, mid-1) // LEFT first! root->right = build(pre, mid+1, inRight) return root } Sorted Array to BST // Time: O(n) | Space: O(log n) // Middle = root (guarantees balance) TreeNode* build(nums, left, right) { if (left > right) return null mid = left + (right - left) / 2 root = new TreeNode(nums[mid]) root->left = build(nums, left, mid-1) root->right = build(nums, mid+1, right) return root } Serialize & Deserialize // Time: O(n) | Space: O(n) // Preorder + null markers (#) string serialize(node) { if (!node) return "#" return val + "," + serialize(left) + "," + serialize(right) } TreeNode* deserialize(tokens) { val = tokens.pop() if (val == "#") return null root = new TreeNode(val) root->left = deserialize(tokens) root->right = deserialize(tokens) return root } Pre[0]=root, LEFT first Post[last]=root, RIGHT first 7A: Pre + Inorder LC 105 7B: Post + Inorder LC 106 7C:

> Sorted → BST LC 108 7D: Serialize LC 297 7E: Flatten LC 114 7F: Morris Traversal Similar: O(1) space traversal, threaded trees Interview Tip: Morris-based problems test O(1) space tree operations. Practice: inorder traversal without stack, recover BST, flatten to linked list.; document/carousel (possible)

#652 Mandar Pandit

Java / Spring

Technical Architect – {Java, Spring Boot} | Legacy Systems Modernization Strategy | Real-World Architecture Trade-offs | Domain-Driven Design Practitioner | Supporting Engineering Teams

5mo · View on LinkedIn

"Architecting Knowledge" - Java Wisdom Series

Post #24: JPA Entity Lifecycle - States You Should Know ↓

Entities aren't just objects. They're state machines you need to understand.

Why This Matters:

Understanding entity states prevents data loss and unexpected behavior. TRANSIENT entities have no ID and aren't tracked. MANAGED entities auto-save changes - no explicit save needed. DETACHED entities lose change tracking when transaction ends. Use merge() or save() to reattach detached entities. LazyInitializationException happens when accessing lazy collections outside transaction. Keep operations in single @Transactional scope to avoid detachment. EntityManager.contains() checks if entity is managed.

Key Takeaway:

Entity lifecycle isn't optional knowledge - it's fundamental. Know when entities are managed vs detached. Keep transaction boundaries tight. Use @Transactional correctly. Dirty checking only works on managed entities. When in doubt, check the state.

#Java #JavaWisdom #JPA #Hibernate #Persistence #EntityLifecycle

All code examples on GitHub - bookmark for quick reference:

https://lnkd.in/dKgfps92

> **MEDIA** advertisement

LINKS

- https://lnkd.in/dKgfps92

#653 Panagiotis Kriaris

AI / LLM

FinTech | Payments | Banking | Advisor, Founder, Editor | View my newsletter

5mo · View on LinkedIn

Card payments are a complex business. A layered stack of providers, orchestration, and schemes sits behind what externally looks like a simple click (or tap). Here's how it works.

The layers:

1) Initiation

A customer taps or clicks pay - that’s all they see. From there, a multi-layered system determines approval, cost, and processing.

2) Acceptance

The payment is captured through POS systems, gateways, or PSPs. This layer determines which payment methods are available, how the checkout is structured, and how the transaction is passed into the network. It’s highly visible, but increasingly standardized.

3) Orchestration

Before the transaction reaches the network, it is directed across different acquirers and payment paths, checked against fraud and risk rules, and retried if needed. These choices are defined by the merchant’s setup and the providers they use.

4) Execution

Only at this stage does the transaction reach the core infrastructure: acquirers, card schemes, and issuing banks. This is where authorization, clearing, and settlement take place under strict scheme rules. It enables global acceptance and consistent processing across markets, but it is highly standardized, leaving limited room for differentiation.

The trends:

• Transaction outcomes are increasingly determined before they reach the schemes - through acquirer selection, routing rules, retry logic, and risk configuration applied at the merchant or PSP level.

• Payment acceptance is broadly standardized - most providers support the same methods and integrations, so it no longer creates a meaningful advantage on its own.

• The way merchants structure their payment setup now directly impacts results - combining providers, routing transactions, and setting retry strategies affects approval rates, cost, and failures.

• Card schemes continue to define the rules, handle authorization, and enable settlement - but an increasing share of how transactions are handled (e.g. routing, retries, risk checks) is determined earlier, by merchants and their providers.

• Approval rates, cost, and failure handling are no longer fixed characteristics of a single provider - they vary based on how transactions are distributed, managed, and continuously adjusted across the stack.

The role of AI:

• AI is increasing how often and how precisely transactions are adjusted after they are captured - routing, retries, and risk settings are updated continuously based on actual performance.

• This means the same providers no longer deliver the same results by default. Approval rates and costs now depend much more on how transactions are managed in real time than on the underlying infrastructure itself.

Opinions: my own, Graphic source: twenty second

Subscribe to my newsletter: https://lnkd.in/dkqhnxdg

LINKS

- https://lnkd.in/dkqhnxdg

#654 Priya Jain System Design

Senior Software Engineer @ Maersk | 12 Years in High-Performance Systems | BFSI → Logistics | Java | Kafka | Agentic AI | Spring AI

6mo · **View on LinkedIn**

I walked into an HSBC Karat interview completely unprepared for System Design — here's what I wish I knew at that time.

About 2 months ago, I gave a Karat round for HSBC. It was a completely new experience for me — something I had never encountered before. That's exactly why I'm sharing this today.

What is a Karat Interview?

Karat interviews are structured, company-specific, and strictly time-bound. The interviewer walks you through the format in the first few minutes — which is helpful — but if you haven't prepared for the type of round, that clarity doesn't save you.

For HSBC, it was a 1-hour System Design focused round, split like this:

First 40 mins → Scenario-based System Design questions

Next 20 mins → Identify issues in existing code + fix them + coding questions

And here's the part that caught me off guard — you earn credit points based on how quickly and how many questions you answer correctly. Speed matters just as much as accuracy.

I wasn't ready. And it showed.

But here's the silver lining.

That experience pushed me to go deep on System Design — really deep. And the preparation I built from that stumble? It helped me crack interviews at multiple top companies after that. Sometimes the rounds you fumble are the ones that teach you the most.

What I'd tell my past self:

✓ Practise system design out loud — simulate real scenarios, not just read theory

✓ Understand scalability, trade-offs, and common design patterns cold

✓ Don't underestimate the code review + fixing section — it's deceptively tricky under time pressure

✓ Preparation is everything. There's no bluffing your way through Karat.

Resources that I'd strongly recommend:

Piyush Garg's System Design playlist on YouTube is genuinely one of the best I've come across. The way he breaks down real-world scenarios helps you think, not just memorise.

System Design Playlist → https://lnkd.in/dbZzX8dM

Additional Prep →

https://lnkd.in/dTQQ3fXn

If you have a Karat round coming up — especially for HSBC — start preparing now, not the night before.

Sharing this because I believe in learning in public. Every stumble is a lesson worth passing on.

Have you given a Karat interview before? What was your experience? Drop it in the comments ↓

#SystemDesign #KaratInterview #HSBC #SoftwareEngineering #InterviewPrep #LearningInPublic #TechCareers #Java #BackendEngineering

System Design

System Design

youtube.com

LINKS

- https://lnkd.in/dbZzX8dM
- https://lnkd.in/dTQQ3fXn
- https://youtube.com/playlist?list=PLinedj3B30sBlBWRox2V2tg9QJ2zr4M3o

#655 Pramod Dhembare

Java / Spring

Senior Java Developer | 7 + yrs | Microservices | REST API | Spring Boot | JWT | Kafka | Open to Opportunities

6mo · **View on LinkedIn**

Mastering @Transactional in Spring — One of the MOST asked interview topics!

After struggling to truly understand how transactions work internally, I created this complete cheat sheet to simplify everything in one place ↓

What this covers:

✓ What @Transactional actually does

✓ How Spring uses AOP & Proxy internally

✓ Propagation types (REQUIRED, REQUIRES_NEW, etc.)

✓ Isolation levels (READ_COMMITTED → SERIALIZABLE)

✓ Rollback rules (very tricky in interviews!)

✓ Common mistakes developers make

(!) Biggest lesson:

→ Internal method calls DO NOT trigger transactions (proxy limitation)

→ By default, rollback happens only for RuntimeException

This topic is simple at surface level, but deep understanding = strong backend fundamentals

If you’re preparing for:

✓ Java Backend Interviews (5–10 years)

✓ Spring Boot / Microservices roles
→ This will definitely help you revise quickly
Save this post for later
Share with someone preparing for interviews
Would love to hear:
→ What's the trickiest @Transactional scenario you've faced?
#Java #SpringBoot #Transactional #BackendDevelopment #Microservices #InterviewPreparation #SoftwareEngineering #Developers

#656 Java Developers Community (moderated)

Java / Spring

Mohamed EL Laithy • 3rd+
6mo · View on LinkedIn

Most developers think Spring Boot = APIs...
That's why apps fail in production. ✗
Because real systems include:
Security
Messaging
Scaling
Observability
Not just CRUD.
I mapped the full Spring Boot ecosystem (100+ tools) ↓
But the real value isn't the tools...
It's how they work together
I explained it simply here:
→ https://lnkd.in/dmtnNQKX
Subscribe to get weekly updates and tips on backend & system design
#springboot #backend #microservices

MEDIA advertisement

LINKS
- https://lnkd.in/dmtnNQKX

#657 Eid Badr

Java / Spring

Senior Software Engineer @ Adesso Insurance Solutions
6mo · View on LinkedIn

Java interview classic: static vs non-static
Key idea:
- Static methods can access only static members directly
- Instance members need an object
Rule to remember:
- Static = class level
- Non-static = object level
#Java #Interview #Static #Compiler #Software

MEDIA advertisement

#658 AlgoMaster.io

Other

Visit website

6mo · View on LinkedIn

10 must-know Array patterns for coding interviews:

1. Two Pointers
2. Sliding Window
3. Prefix Sum
4. Kadane's Algorithm
5. Binary Search
6. Cyclic Sort
7. Merge Intervals
8. Monotonic Stack
9. Hash Map Lookup
10. Sorting + Greedy

→ If you are preparing for coding interviews and want to master all important DSA patterns checkout algomaster.io

Repost to help others in your network

LINKS

- https://algomaster.io/
- http://algomaster.io

#659 Parikh Jain

Other

Founder @ ProPeers | Ex-Amazon | Ex-Founding Member at Coding Ninjas | Youtuber(80k+) | DTU

6mo · View on LinkedIn

Solve these 11 Groups of 50+ DP problems to crack & solve any DP problem designed for SDE Interviews.

Spot these triggers: “count number of ways,” “min/max value,” “partition,” “longest subsequence/ path,” “edit distance,” “minimum cost.”

Fibonacci / Simple Recurrence

1. Climbing Stairs-https://lnkd.in/dpAHFxr8
2. Min Cost Climbing Stairs-https://lnkd.in/g9DXiD-H
3. Dice Combinations-https://lnkd.in/gmwhP22y
4. Range Sum Query - Immutable-https://lnkd.in/eAg9eKS4

I've already covered all of this in my DSA + Dev + GenAI Placement System: https://lnkd.in/gut39Xsr. (Over 1500+ students are using it)

It helps you because it is built around patterns, mock rounds, and the exact format companies actually test in.

0/1 Knapsack

1. Target Sum-https://lnkd.in/dsXUM9wF
2. Partition Equal Subset Sum-https://lnkd.in/emFwenrk
3. Knapsack-https://lnkd.in/gjha5a2G
4. Combination Sum IV-https://lnkd.in/e_dVS35w

Unbounded Knapsack

1. Coin Change-https://lnkd.in/ercepMHZ
2. Coin Combinations I-https://lnkd.in/gE5AWDxW
3. Cutting Rod-https://lnkd.in/gjGsA8ys

Longest Common Subsequence (LCS
1. Uncrossed Lines-https://lnkd.in/gEHhw6u9
2. Edit Distance-https://lnkd.in/gyYVE88f
3. LCS-https://lnkd.in/gjU5FU5r
4. Word Break-https://lnkd.in/e94ANxqa
Longest Increasing Subsequence (LIS
1. Longest Increasing Subsequence-https://lnkd.in/eyR8yCfP
2. Number of Longest Increasing Subsequence-https://lnkd.in/eBfvUH_f
3. Vacation-https://lnkd.in/g4thCcxi
Grid-Based DP
1. Unique Paths-https://lnkd.in/ec2XTnRH
2. Grid Paths-https://lnkd.in/gzxH5Hs4
3. Grid 1-https://lnkd.in/g6EZArGQ
Interval DP
1. Burst Balloons-https://lnkd.in/gwwRtcwK
2. Palindrome Partitioning II-https://lnkd.in/gkiwGJZw
3. Longest Palindromic Substring-https://lnkd.in/eS2J9eFn
Tree DP
1. House Robber III-https://lnkd.in/gWJjFDTV
2. Tree Matching-https://lnkd.in/gY2bfrJU
3. House Robber-https://lnkd.in/dFFqrMc4
4. House Robber II-https://lnkd.in/en5QCPiN
Bitmasking/State Compression
1. Campus Bikes II-https://lnkd.in/gNdqQkv7
2. Partition to K Equal Sum Subsets-https://lnkd.in/eJAWjSxJ
3. Traveling Salesman-https://lnkd.in/gfabcxXt
Digit DP
1. Numbers With Repeated Digits-https://lnkd.in/gRwS7GhC
2. Removing Digits-https://lnkd.in/g3XMWdMR
3. Sushi-https://lnkd.in/gKSzxYKY
Probability/Expectation DP
1. Dice Roll Simulation-https://lnkd.in/gstHYfP9
2. Candy Lottery-https://lnkd.in/gFU88CE3
3. Coins-https://lnkd.in/gaq5UNHq

LINKS

- https://www.propeers.in/digitalProducts
- https://lnkd.in/dpAHFxr8
- https://lnkd.in/g9DXiD-H
- https://lnkd.in/gmwhP22y
- https://lnkd.in/eAg9eKS4
- https://lnkd.in/gut39Xsr
- https://lnkd.in/dsXUM9wF
- https://lnkd.in/emFwenrk
- https://lnkd.in/gjha5a2G
- https://lnkd.in/e_dVS35w
- https://lnkd.in/ercepMHZ
- https://lnkd.in/gE5AWDxW
- https://lnkd.in/gjGsA8ys
- https://lnkd.in/gEHhw6u9
- https://lnkd.in/gyYVE88f

- https://lnkd.in/gjU5FU5r
- https://lnkd.in/e94ANxqa
- https://lnkd.in/eyR8yCfP
- https://lnkd.in/eBfvUH_f
- https://lnkd.in/g4thCcxi
- https://lnkd.in/ec2XTnRH
- https://lnkd.in/gzxH5Hs4
- https://lnkd.in/g6EZArGQ
- https://lnkd.in/gwwRtcwK
- https://lnkd.in/gkiwGJZw
- https://lnkd.in/eS2J9eFn
- https://lnkd.in/gWJjFDTV
- https://lnkd.in/gY2bfrJU
- https://lnkd.in/dFFqrMc4
- https://lnkd.in/en5QCPiN
- https://lnkd.in/gNdqQkv7
- https://lnkd.in/eJAWjSxJ
- https://lnkd.in/gfabcxXt
- https://lnkd.in/gRwS7GhC
- https://lnkd.in/g3XMWdMR
- https://lnkd.in/gKSzxYKY
- https://lnkd.in/gstHYfP9
- https://lnkd.in/gFU88CE3
- https://lnkd.in/gaq5UNHq

#660 SERKUT YILDIRIM

Java / Spring

Microsoft MVP | .NET, Java, Software Development

6mo · **View on LinkedIn**

Java/Spring Boot Tip - Missing Values

Exception vs Optional

Exceptions are designed for truly exceptional, unexpected errors that occur outside normal program flow. When thrown, they propagate up the call stack until caught by an appropriate handler.

✓ Exceptions should never be used for routine control flow due to significant performance costs from stack unwinding and object creation.

Optional provides an explicit alternative for representing absence of a value. It wraps a value that may or may not be present, making null-safe code more expressive and functional.

✓ Optional is ideal for modeling "value might be missing" scenarios and works seamlessly with streams and lambda expressions.

✓ When to Use Each

■ Exceptions: Rare failures like I/O errors, database connection issues, or invalid business transactions.

■ Optional: Expected absence like cache misses, lookup results not found, or optional configuration values.

■ Hybrid Approach: Use both strategically for optimal code clarity and performance.

Which approach do you prefer for handling missing values?

#java #springboot #programming #softwareengineering #softwaredevelopment

#661 Manish Kala Jain

System Design

Engineering & Technology Leader | CTO | Head of Engineering | VP Engineering | AI, Platform Engineering, Digital Commerce, Payments & Cloud | Led 100+ Engineers | 100M+ Scale | Dubai Golden Visa | Immediately Available

6mo · **View on LinkedIn**

**Payment System Architecture

This is not a payment system.

This is a revenue survival machine.
Most people look at a payment flow and see:
→ “User clicks → Payment succeeds”
But zoom into the architecture...
...and you’ll realize it’s a battlefield of distributed systems.

What you’re really seeing in this diagram:
It’s not just layers — it’s defense mechanisms against failure.
Client & Edge Layer
Where latency, rate limits, and security attacks are handled before they hit your core.
Orchestration Layer
Where checkout, cart, pricing — all align to create a seamless experience.
Core Payments Domain
The heart:
• Payment routing
• Tokenization
• 3DS / SCA
• Multi-method adapters (cards, wallets, BNPL, bank transfers)
→ This is where every millisecond = conversion rate

(!) But here’s where most systems fail:
They underestimate the Risk & Compliance Layer
Fraud detection, velocity checks, ML scoring —
This layer silently decides who gets approved vs declined

Messaging & Async Layer = The Unsung Hero
Queues, retries, DLQs, event buses...
Because in payments:
“Eventually consistent” still needs to feel instant to the user.

Ledger Layer = The Source of Truth
If this breaks:
• Money mismatches
• Reconciliation nightmares
• Legal impact
→ This is not just engineering. This is financial correctness at scale.

Data + Observability Layer
If you can’t trace a failed payment across 10 services in seconds...
You don’t have a payment system.
You have a black box.

Ops, Security & Reliability
Circuit breakers.
Feature flags.
Disaster recovery.
Because outages don’t ask for permission.

The real takeaway:
Payments are not a feature.
They are a distributed system of trust.

And the companies that win are not the ones with:

✗ The fastest UI

✗ The cheapest gateway

They win with:

✓ The most resilient architecture

✓ The smartest fallback strategies

✓ The highest success rate under failure

#SystemDesign
#DistributedSystems
#SoftwareArchitecture
#ScalableSystems
#HighAvailability
#BackendEngineering
#CloudComputing
#Microservices
#EventDrivenArchitecture
#APIDesign
#FinTech
#PaymentSystems
#DigitalPayments
#RealTimePayments
#BankingTechnology
#UPI
#CardPayments
#RiskManagement
#FraudDetection
#LedgerSystems
#EngineeringLeadership
#TechLeadership
#PlatformEngineering
#ReliabilityEngineering
#SRE
#DevOps
#ResilientSystems
#ProductionSystems
#SystemThinking
#DesignPatterns
#SoftwareEngineer
#TechCommunity
#BuildInPublic
#EngineeringExcellence
#TechInsights
#StartupEngineering
#BigTech
#Innovation
#LearnInPublic
#CareerGrowth

#662 Manish Kala Jain

System Design

Engineering & Technology Leader | CTO | Head of Engineering | VP Engineering | AI, Platform Engineering, Digital Commerce, Payments & Cloud | Led 100+ Engineers | 100M+ Scale | Dubai Golden Visa | Immediately Available
6mo · View on LinkedIn

Payment Wallet Architecture — What Really Happens Behind “Pay” in 2 Seconds?

Everyone talks about “real-time payments”...

Very few understand the orchestration behind that single tap.

I recently designed a full-scale Payment Wallet System Architecture — not just boxes & arrows, but the actual thinking layer behind scalability, reliability, and compliance.

Here’s what most people miss ↓

1. It’s NOT just an API call

A simple “Pay” triggers:

• Auth + risk checks
• Ledger consistency (double-entry)
• Idempotent transaction orchestration
• Bank / network integrations
• Event streaming + async reconciliation

2. The real backbone = Ledger + Events

Forget UI, forget APIs.

If your ledger is weak → your system is broken.

• Real-time balance ≠ actual settlement
• Event-driven architecture ensures auditability
• Every state change is immutable

3. Fraud & Risk are FIRST-class citizens

Not an afterthought.

• Device fingerprinting
• Velocity checks
• Behavioral scoring
• Dynamic rule engines

→ Payments don’t fail because of tech.

They fail because of trust gaps.

4. External systems are the biggest bottleneck

Banks. Networks. Gateways.

They:

• Timeout
• Retry inconsistently
• Break SLAs

So your system must:

• Be resilient by design
• Support async fallbacks
• Handle partial failures gracefully

5. Reconciliation is where truth lives

End-of-day ≠ end of system.

• Wallet ≠ Bank balance
• Automated + manual reconciliation
• Exception queues are critical

→ If you can’t reconcile, you don’t own your money flow.

6. Failure is the default state
Design for:
• Retries with backoff
• Circuit breakers
• Saga-based compensations
• Idempotency everywhere

7. Observability is non-negotiable
If you can’t trace a transaction across:
API → Service → Queue → Bank → Ledger
You’re blind.

Closing Analysis
Building a payment wallet is not a feature.
It’s a distributed, regulated, real-time financial system
where every millisecond and every cent matters.

#SystemDesign
#DistributedSystems
#SoftwareArchitecture
#ScalableSystems
#HighAvailability
#BackendEngineering
#CloudComputing
#Microservices
#EventDrivenArchitecture
#APIDesign
#FinTech
#PaymentSystems
#DigitalPayments
#RealTimePayments
#BankingTechnology
#UPI
#CardPayments
#RiskManagement
#FraudDetection
#LedgerSystems
#EngineeringLeadership
#TechLeadership
#PlatformEngineering
#ReliabilityEngineering
#SRE
#DevOps
#ResilientSystems
#ProductionSystems
#SystemThinking
#DesignPatterns
#SoftwareEngineer
#TechCommunity
#BuildInPublic
#EngineeringExcellence

#TechInsights
#StartupEngineering
#BigTech
#Innovation
#LearnInPublic
#CareerGrowth
#wallet
#founders
#startups
#architects

#663 Dinesh Anbumani

System Design

Solutions Architect | Engineering Manager | AWS Cloud | Microservices | APIs | React, NextJs | Node.js, Python | ELK | Docker & Kubernetes | SQL & NoSQL

6mo · **View on LinkedIn**

→ The Netflix Architecture Mystery

Something extraordinary happens every time you press Play.

It feels instant.

It feels effortless.

But behind that single click lies one of the most sophisticated engineering ecosystems ever built.

Netflix is not just a streaming platform.

It is a microservices powered machine operating at global scale.

→ The Journey of a Single Click

• You press Play. A simple action triggers a distributed system of services.

• Elastic Load Balancer routes traffic intelligently. No server gets overwhelmed.

• Zuul API Gateway acts as the control layer. It secures, filters, and manages every request.

• Application APIs handle specific tasks such as authentication, content discovery, and playback.

• Thousands of independent microservices process requests. Each service focuses on one responsibility. They are stateless and scalable.

• Data systems store, retrieve, and stream information in real time. This powers personalization and recommendations.

• Large scale storage solutions such as S3, Cassandra, and Hadoop support analytics and continuous improvement.

→ Why Microservices Changed Everything

• Independent scaling of services.

• Faster deployments without disrupting the full system.

• Higher resilience when individual components fail.

Netflix architecture proves one powerful lesson.

Speed and reliability at global scale require distributed thinking.

The next time you stream a show, remember the invisible system making it possible.

Follow Dinesh Anbumani for more insights

#664 Mandar Pandit

Java / Spring

Technical Architect – {Java, Spring Boot} | Legacy Systems Modernization Strategy | Real-World Architecture Trade-offs | Domain-Driven Design Practitioner | Supporting Engineering Teams

6mo · **View on LinkedIn**

"Architecting Knowledge" - Java Wisdom Series

Post #22: Spring Security Filters - The Chain Explained ↓

Authentication isn't magic. It's 15 filters in perfect order.

Why This Matters:

FilterChainProxy delegates to SecurityFilterChain beans. Filters execute in strict order - position matters! First matching SecurityFilterChain wins (use @Order). Each filter has a specific job - context loading, authentication, authorization, exception handling. Custom filters must be positioned correctly using addFilterBefore/After/At. OncePerRequestFilter ensures execution happens once per request. @EnableWebSecurity(debug=true) shows filter execution flow. Multiple SecurityFilterChain beans allow different security per URL pattern.

Key Takeaway:

Don't fight the filter chain - understand it. Position custom filters correctly. Use multiple SecurityFilterChain beans for different security requirements. Always call chain.doFilter() to continue. Debug mode is your friend during development.

#Java #JavaWisdom #SpringSecurity #Security #WebSecurity #BestPractices

Still confused about Spring Security? Master the filter chain!

All code examples on GitHub - bookmark for quick reference: https://lnkd.in/dJUx3Rd3

MEDIA advertisement

LINKS

- https://lnkd.in/dJUx3Rd3

#665 Alex Xu

System Design

Author of 4 Bestselling Books | Co-Founder of ByteByteGo

6mo · **View on LinkedIn**

Optimistic locking vs pessimistic locking

Imagine two developers updating the same database row at the same time. One of them will have their update rejected. How should the system handle this?

There are two common approaches.

Optimistic locking assumes conflicts are rare. Both users read the data without acquiring any lock. Each record carries a version number. When a user attempts to write, the database checks: does the version in your update match the current version in the database? If another transaction already incremented the version from 1 to 2, your update still references version 1. The write is rejected.

Pessimistic locking takes the opposite approach. It assumes conflicts are likely, so it blocks them before they happen. The first transaction locks the row, and every other transaction waits until that lock is released. No version checks needed.

If your system is read-heavy with occasional writes, optimistic locking is the best option. When concurrent writes occur frequently and the cost of a conflict is high, pessimistic locking is the safer choice.

Over to you: Have you ever run into a deadlock in production because of a locking strategy? How did you fix it?

--

Subscribe to our weekly newsletter to get a Free System Design PDF (368 pages): https://lnkd.in/gauQcE45

#systemdesign #coding #interviewtips

LINKS

- https://bit.ly/bbg-a-li-profile
- https://lnkd.in/gauQcE45

#666 Kubernetes

Cloud / K8s / DevOps

Ravi Ranjan • 3rd+
6mo · View on LinkedIn

If you’re still using single-stage Dockerfiles, you’re probably shipping unnecessary baggage into production.

→ Switch to a multi-stage Dockerfile:

- Builder stage – compile, install dependencies, run tests
- Runtime stage – copy only what’s needed to run

Result?

✓ Smaller images
✓ Faster deployments
✓ Better security

Example idea: build with a full Node/Python/Java environment, then run with a lightweight base image.

Less clutter. More performance.

If this helped you, repost it so more developers can level up their Docker game!

#667 Java Developers Community (moderated)

Java / Spring

Arman Shaikh • 3rd+
6mo · View on LinkedIn

HashMap internal working

HashMap stores data in key-value pairs using hashing.

A key’s hashCode() decides the bucket where the value is stored.

If collision happens, Java handles it using linked list or tree (Java 8+).

Average time complexity: O(1) for insert, search, delete.

In short: HashMap = Fast data access using hashing and buckets.

#668 Pedro Ostanik

Java / Spring

Software Engineer | AWS Certified
6mo · View on LinkedIn

Thread Pools to Virtual Threads.

OutOfMemoryError it is a nightmare that has always haunted Java developers. We're always been stuck with Platform Threads (heavyweight wrappers around OS threads). Since each one cost about 1MB of memory, handling 10,000 concurrent requests meant you either needed a massive, expensive server or had to write complex reactive code that nobody actually wants to debug.

Enter Project Loom (Java 21+).

I’ve been diving into Virtual Threads, and the "blocking" game has completely changed. Here’s why this matters for the modern backend:

Cheap as Chips: Virtual threads are managed by the JVM, not the OS. They only cost a few hundred bytes. You can literally spawn one million threads on a standard laptop without breaking a sweat.

The "Thread-per-Request" Revival: We can go back to writing simple, readable, synchronous code. No more "Callback Hell" or complex Mono/Flux chains just to keep the CPU busy while waiting for a database response.

Massive Throughput: In I/O-heavy applications (which most Spring Boot apps are), Virtual Threads allow the CPU to switch to other tasks instantly while one thread waits for a slow API or SQL query.

How to use it in Spring Boot 3.2+?

It’s literally one line in your application.properties:

spring.threads.virtual.enabled=true

By flipping this switch, Tomcat/Undertow starts using Virtual Threads to handle web requests. It’s a complete paradigm shift that lets us build more scalable systems with less infrastructure cost.

The takeaway for teams: We no longer have to choose between "easy-to-read code" and "high-performance code." With Java 21, we get both.

#Java #SpringBoot #BackendDevelopment #ProjectLoom #SoftwareEngineering #Scalability #JVM

LINKS

- http://application.properties

#669 Rocky Bhatia

System Design

400K+ Engineers | Architect @ Adobe | GenAI & Systems at Scale | View my portfolio

6mo · **View on LinkedIn**

Top 6 API architecture Styles

APIs serve as the backbone of modern software development, enabling seamless integration and communication between various components. Understanding the different API architecture styles is crucial for choosing the most suitable approach for your project. Below are the top six API architecture styles along with their recommended use cases:

1. SOAP (Simple Object Access Protocol): SOAP is ideal for enterprise-level applications that require a standardized protocol for exchanging structured information. Its robust features include strong typing and advanced security mechanisms, making it suitable for complex and regulated environments.

2. RESTful (Representational State Transfer): RESTful APIs prioritize simplicity and scalability, making them well-suited for web services, particularly those catering to public-facing applications. With a stateless, resource-oriented design, RESTful APIs facilitate efficient communication between clients and servers.

3. GraphQL: GraphQL shines in scenarios where flexibility and client-driven data retrieval are paramount. By allowing clients to specify the exact data they need, GraphQL minimizes over-fetching and under-fetching, resulting in optimized performance and reduced network traffic.

4. gRPC: For high-performance, low-latency communication, gRPC emerges as the preferred choice. Widely adopted in microservices architectures, gRPC offers efficient data serialization and bi-directional streaming capabilities, making it suitable for real-time applications and distributed systems.

5. WebSockets: WebSockets excel in applications requiring real-time, bidirectional communication, such as chat platforms and online gaming. By establishing a persistent connection between clients and servers, WebSockets enable instant data updates and seamless interaction experiences.

6. Webhooks: In event-driven systems, webhooks play a vital role by allowing applications to react to specific events in real-time. Whether it's notifying about data updates or triggering actions based on user activities, webhooks facilitate seamless integration and automation.

Selecting the appropriate API style is crucial for optimising your application's performance and enhancing user experience. By understanding the strengths and use cases of each architecture style, you can make informed decisions that align with your project's specific requirements.

LINKS

- https://www.youtube.com/@learnwithrockybhatia

#670 Aman Mishra

Java / Spring

Senior Software Engineer | Backend Developer | Java | Spring Boot | LLM | Microservice | Restful APIs | AWS | Kafka | Data Structure | SQL | No SQL | HLD | LLD | Scalable System

6mo · **View on LinkedIn**

Java Collections interview questions that keep showing up — with real answers

If you have a Java interview coming up, this is the one topic you cannot afford to skip.

Collections — especially HashMap — appears in every Java interview at every level.
Here are the questions that actually get asked, with answers that impress ↓
Q: How does HashMap work internally?
It useses an array of buckets.
Flow: hashCode() → perturbation → bucket index → data stored.
Since Java 8 — if a bucket exceeds 8 nodes, it converts to a Red-Black Tree.
O(1) average. O(log n) worst case. Know this cold.
Q: Why is the default load factor 0.75?
Sweet spot between memory and performance.
When entries exceed capacity × 0.75, HashMap doubles and rehashes.
Pro tip: pre-size your HashMap if you know the expected size. Avoid rehashing.
Q: Why must HashMap keys be immutable?
Mutate a key after putting it → hashCode changes → different bucket → get() returns null.
The entry becomes permanently unreachable — this is a memory leak.
This is why String and Integer are the best map keys.
Q: HashMap vs ConcurrentHashMap vs Hashtable?
HashMap — not thread-safe, fastest single-threaded.
Hashtable — locks entire map, legacy, never use it.
ConcurrentHashMap — locks only the bucket. Lock-free reads. 10-100× faster than Hashtable.
Q: Fail-fast vs fail-safe iterators?
Fail-fast (HashMap, ArrayList) — throws ConcurrentModificationException if modified during iteration.
Fail-safe (ConcurrentHashMap, CopyOnWriteArrayList) — works on a snapshot, never throws.
Q: How to implement an LRU Cache in Java?
LRU Cache = keeps the most recently used items, throws away the oldest ones when full.
In Java - Extend LinkedHashMap with accessOrder=true.
Override removeEldestEntry() to evict when size exceeds capacity.
10 lines of code. Interviewers love this answer.
The follow-up they always ask:
"What happens when two keys have the same hashCode?"
Hash collision → same bucket → linked list → equals() finds the exact match.
This is why hashCode() AND equals() must both be correctly implemented.
→ Follow Aman Mishra for more backend insights,content, and interview-focused tech breakdowns!
I've covered this in depth, GC tuning, JVM flags, and real interview Q&As — in my Java Core & Advanced Interview Mastery Guide.Offering 50% off for a limited time! ↓
Get the guide here: https://lnkd.in/gn3AG7Cm
Use code JAVA50

LINKS

- https://lnkd.in/gn3AG7Cm

#671 Kanak Agrawal Java / Spring

MTS (SDE-2) at Salesforce | Ex-Oracle, Fastenal | Java, Devops, Distributed Systems, Testing

6mo · **View on LinkedIn**

Came across one of the most structured and practical preparation guides for backend / Java / system design interviews — and I had to share this.
It covers everything in one place:
• Core Java + JVM + Multithreading

• Spring Boot, Kafka, Docker, Kubernetes
• Real-world project ideas (microservices, saga, outbox, etc.)
• Structured DSA roadmap (Striver, Neetcode, patterns)
• System Design (LLD + HLD resources)
• Resume, LinkedIn optimization & job strategy
• Cold outreach & referral templates

Basically, a complete roadmap from preparation to interviews to offers.

Sharing the doc here for anyone preparing/ bookmark it :)

(Highly recommend saving it)

https://lnkd.in/g5uSvHCD

Not my content (credits to the original creator), but genuinely one of the best compilations I've seen so far.

Guide

Guide

docs.google.com

MEDIA document/carousel (possible)

LINKS
- https://lnkd.in/g5uSvHCD
- https://docs.google.com/document/d/1hrBiHbEuFd6slKd0Y8aDVkLgn8vdur9o8CGGPxEb_ic/edit?usp=sharing

#672 Suraj Mishra

Java / Spring

Technical Lead & Staff Software Engineer @ PayPal

6mo · **View on LinkedIn**

Spring Boot Interview Question: Connection Pool Exhaustion

Scenario

You are working on a Spring Boot–based User Activity Service for a high-traffic SaaS application.

Every time a user logs in, the system updates the last_login timestamp in a PostgreSQL database.

During peak hours, production starts showing:

- Requests stuck waiting for database connections
- HikariCP pool exhaustion errors
- Sudden spike in API latency
- No obvious slow SQL queries

After investigation, the attached code is found.

Checkout the detailed questions with solutions and explanations:

https://lnkd.in/geSzv3FB

Subscribe and join 6500+ java and spring boot devs:

https://lnkd.in/gwiRqWBV

LINKS
- https://lnkd.in/geSzv3FB
- https://lnkd.in/gwiRqWBV

#673 Big Data Data Science | Machine Learning | Deep Learning | Artificial Intelligence

AI / LLM

Özer Cem K. • 3rd+ | View my services

6mo · View on LinkedIn

LLM API Request in 400ms

Understanding the mechanics behind a single API call is essential for optimizing high-performance AI applications.

When a POST request hits an endpoint, it initiates a complex sequence of operations that span across distributed systems and specialized hardware clusters.

The entire process is a masterclass in low-latency engineering where every millisecond is accounted for across multiple layers of infrastructure.

The journey begins at the API Gateway and Load Balancer where authentication, rate limiting, and geographic routing occur.

This initial phase ensures that the request is valid and directed to the most available GPU cluster. Once cleared, the raw text is converted into token IDs. This tokenization step is critical because it defines both the computational load and the financial cost of the transaction based on the input volume.

A hidden but vital component is the Model Router. This layer analyzes the request type and directs it to the appropriate hardware, whether it is a heavy inference cluster for large models or a dedicated embedding cluster.

This capacity-aware routing is what allows providers to manage peak loads without total system failure.

The core of the process happens within the Inference Engine during the Prefill and Decode phases.

The Prefill phase processes all input tokens in parallel to generate the KV Cache, which is stored in GPU memory to avoid recomputing past tokens.

The Decode phase then takes over in an autoregressive loop, generating one token at a time. This specific loop is the reason streaming exists, as it allows each token to be sent to the user as soon as it is generated rather than waiting for the entire sequence to complete.

Post-processing and billing conclude the journey. The generated tokens undergo detokenization, safety classification, and formatting into a JSON response.

Final costs are calculated based on the sum of input and output tokens, with output usually being significantly more expensive due to the sequential nature of generation. For architects building at scale, mastering these internal steps is the key to reducing latency and managing cloud expenditures effectively.

What part of the inference pipeline do you find most challenging to optimize in your own projects?

I am interested in hearing how you manage KV cache efficiency or handle the latency spikes during the prefill phase. Let us discuss the technical details in the comments.

MEDIA advertisement